cadence®

Tempus Stylus User Guide

Product Version 25.13

April 2026

© 2011-2026 Cadence Design Systems, Inc. All rights reserved.

Printed in the United States of America.

Cadence Design Systems, Inc. (Cadence), 2655 Seely Ave., San Jose, CA 95134, USA.

Trademarks: Trademarks and service marks of Cadence Design Systems, Inc. contained in this document are attributed to Cadence with the appropriate symbol. For queries regarding Cadence's trademarks, contact the corporate legal department at the address shown above or call 1-800-862-4522.

All other trademarks are the property of their respective holders.

Restricted Permission: This publication is protected by copyright law and international treaties and contains trade secrets and proprietary information owned by Cadence. Unauthorized reproduction or distribution of this publication, or any portion of it, may result in civil and criminal penalties. Except as expressly permitted below, this publication may not be copied, reproduced, modified, published, uploaded, posted, transmitted, or distributed in any way, including automated processes, training or services involving a large language model, foundation model, deep machine learning, generative artificial intelligence, or any other similar process or technology, without prior written permission from Cadence. Unless otherwise agreed to by Cadence in writing, customers are granted permission to print one (1) hard copy of this publication subject to the following conditions:

1. The publication may be used only in accordance with a written agreement between Cadence and its customer.
2. The publication may not be modified in any way.
3. Any authorized copy of the publication or portion thereof must include all original copyright, trademark, and other proprietary notices and this permission statement.
4. The information contained in this document cannot be used in the development of like products or software, whether for internal or external use, and shall not be used for the benefit of any other party, whether or not for consideration.

Disclaimer: Information in this publication is subject to change without notice and does not represent a commitment on the part of Cadence. Except as may be explicitly set forth in such agreement, Cadence does not make, and expressly disclaims, any representations or warranties as to the completeness, accuracy or usefulness of the information contained in this document. Cadence does not warrant that use of such information will not infringe any third party rights, nor does Cadence assume any liability for damages or costs of any kind that may result from use of such information. Cadence is committed to using respectful language in our code and communications. We are also active in the removal and/or replacement of inappropriate language from existing content. This product documentation may however contain material that is no longer considered appropriate but still reflects long-standing industry terminology. Such content will be addressed at a time when the related software can be updated without end-user impact.

Restricted Rights: Use, duplication, or disclosure by the Government is subject to restrictions as set forth in FAR52.227-14 and DFAR252.227-7013 et seq. or its successor.

Contents

About This Manual 8
Audience 8
How This Manual Is Organized 8
Conventions Used in This Manual 9
Related Documents 10

1
Product and Licensing Information 12

2
Design Import 28
2.1.1 Input Requirements 29
2.2.1 MMMC Timing Initialization 31
2.2.2 Simple Timing Initialization 33

3
Installation and Startup 37
3.2.1 Temporary File Locations 39
3.3.1 Completing Command Names 40
3.3.2 Command-Line Editing 40
3.3.3 Setting Preferences 42
3.4.1 Using the Log File Viewer 43

4
Analysis and Reporting 48
4.1.1 Overview 50
4.1.2 Base Delay Analysis Flow 50
4.1.3 Limitations of Traditional Delay Calculators 53

4.1.4 Performing Base Delay Analysis . . . 55
4.1.5 Base Delay Reporting . . . 60
4.2.1 Signal Integrity Delay and Glitch Analysis - An Overview . . . 64
4.2.2 Understanding Attacker and Victim Nets . . . 64
4.2.3 SI Delay Analysis - An Overview . . . 64
4.2.4 Signal Integrity Delay Analysis Flow . . . 66
4.2.5 Performing Signal Integrity Delay Analysis . . . 67
4.2.6 SI Delay Reporting . . . 94
4.2.7 SI Glitch Analysis - An Overview . . . 102
4.2.8 SI Glitch Analysis Flow . . . 103
4.2.9 Understanding SI Glitch Analysis . . . 105
4.2.10 Running Detailed SI Glitch Analysis . . . 110
4.2.11 SI Glitch Reporting . . . 112
4.2.12 Overshoot-Undershoot Glitch Analysis - An Overview . . . 119
4.2.13 Running Overshoot-Undershoot Glitch Analysis - Sample Script . . . 123
4.3.1 Overview . . . 126
4.3.2 Specifying Timing Analysis Mode Types . . . 127
4.3.3 Advanced On-Chip Variation (AOCV) Analysis . . . 145
4.3.4 Statistical On-Chip Variation (SOCV) Analysis . . . 170
4.4.1 Parametric Timing Robustness (PTR) Analysis Overview . . . 188
4.4.2 Library Requirements . . . 190
4.4.3 Early/Late Sigma LVF . . . 190
4.4.4 LVF Moment Models . . . 191
4.4.5 Enabling PTR Analysis . . . 192
4.4.6 PTR Analysis Reports . . . 192
4.5.1 PBA Overview . . . 195
4.5.2 GBA vs PBA Comparison . . . 195
4.5.3 Reporting in Path-Based Analysis . . . 195
4.5.4 Path-Based Analysis Reporting Models . . . 197
4.5.5 Path-Based Analysis Essence . . . 199
4.6.1 SSI Overview . . . 201
4.6.2 Implementation . . . 202
4.6.3 Reporting in SSI Mode . . . 204
4.6.4 Considerations for Path-Based Analysis (PBA) . . . 205
4.7.1 Overview . . . 207
4.7.2 Rail Swing Checks - An Overview . . . 207

4.7.3 Rail Swing Reporting . . . 208
4.7.4 Waveform Aware Pulse Width Checks - An Overview . . . 208
4.7.5 Waveform Aware Pulse Width Checks Reporting . . . 209
4.8.1 Iinstance Library Flow Overview . . . 212
4.8.2 MMMC set_instance_library Flow . . . 212
4.8.3 MMMC Flow Script . . . 213
4.8.4 Save and Restore MMMC set_instance_library Flow . . . 215
4.8.5 Limitations of set_instance_library . . . 216
4.9.1 IPD Analysis Overview . . . 218
4.9.2 IPD Analysis Flow . . . 219
4.9.3 Scope-Based IPD Analysis . . . 220
4.9.4 Scope-Based IPD Analysis Flow . . . 221
4.9.5 Enabling IPD Analysis . . . 222
4.9.6 IPD Analysis Reporting . . . 225
4.9.7 Inter-Power Domain (IPD) Logic Checks . . . 226
4.9.8 IPD Paths Classification . . . 228
4.9.9 Supported IPD Logic Checks . . . 231
4.9.10 Performing IPD Logic Checks . . . 234
4.10.1 Aging-Aware STA Analysis - Overview . . . 238
4.10.2 Tempus Aging-Aware STA . . . 240
4.10.3 Aging-Aware Tempus vs SPICE Correlation . . . 248
4.10.4 Spice Deck for Aging . . . 251
4.11.1 Advanced Multiple-Input Switching - Overview . . . 254
4.11.2 User Interface and Timing Analysis with Advanced MIS . . . 255
4.11.3 Enabling Advanced MIS Feature . . . 255
4.11.4 Enabling/Disabling MIS for Library Cells . . . 255
4.11.5 MIS and SSI Analysis . . . 259
4.11.6 MIS Delta Delay Reporting . . . 261
4.11.7 Spice Validation Support and Results . . . 263
4.12.1 AMIS-Derate - Overview . . . 266
4.12.2 AMIS-Derate Flow . . . 266
4.12.3 AMIS-Derate Use-Model . . . 268
4.12.4 Overlap Checking . . . 268
4.12.5 Handling Signals from Different Clocks . . . 271
4.12.6 AMIS-Derate Tuning Mechanism . . . 272
4.12.7 MIS-Derate Reporting . . . 273

4.13.1 VT Skew Analysis - Overview . . . 278
4.13.2 Voltage Threshold Skew Analysis Flow . . . 279
4.13.3 Enabling VT Skew Analysis . . . 280
4.13.4 Partitioning Libraries by Voltage Threshold . . . 280
4.13.5 Liberty Threshold Voltage Groups . . . 281
4.13.6 VT Skew Reporting . . . 281
4.14.1 Design Rule Violation (DRV) - Overview . . . 285
4.14.2 DRV Violation Reports . . . 285
4.14.3 Enabling DRV Reporting . . . 286
4.14.4 Limiting DRV Violations . . . 288
4.14.5 Flow Support for Configurable DRV . . . 288
4.15.1 Cell EM Analysis and ECO - Overview . . . 291
4.15.2 Cell EM STA Flow . . . 293
4.15.3 Required Inputs for Cell EM Analysis . . . 294
4.15.4 Reports . . . 302
4.15.5 ECO Flow Use Model . . . 310
4.15.6 Save/Restore of Design Database . . . 311
4.15.7 Hierarchical STA - Cell EM . . . 311
4.15.8 Current Limitations and Recommendations . . . 314
4.16.1 Hierarchical Timing Context Analysis - Overview . . . 316
4.16.2 Hierarchical Timing Context Analysis Flow . . . 317
4.16.3 Constraints Handling with Timing Context Analysis . . . 318
4.16.4 Clock Mapping in Context Flow . . . 319
4.16.5 Clock Mapping Approaches . . . 324
4.16.6 Manual Clock Mapping Specification . . . 343
4.16.7 Clock Mapping Reporting . . . 345
4.16.8 MIM Context Analysis . . . 348
4.16.9 Context Generation and Analysis in MMMC . . . 352
4.17.1 Wire Variation Analysis - Overview . . . 355
4.17.2 Performing Wire Variation Analysis . . . 358
4.17.3 Statistical Wire Variation Analysis . . . 360
4.17.4 Binomial Wire Variation Analysis . . . 361
4.17.5 Wire Variation Reporting . . . 362

5
Concurrent Multi-Mode Multi-Corner Timing Analysis 366

6
Distributed Multi-Mode Multi-Corner Timing Analysis 387

7
Distributed Static Timing Analysis 413
7.1.1 Static Timing Analysis (STA) 414
7.1.2 Distributed Static Timing Analysis (DSTA) 414

8
Signoff ECO 432

9
Timing Model Generation 494
9.1.1 Overview 496
9.1.2 ETM Generation 497
9.1.3 ETM Generation for MMMC Designs 499
9.1.4 Slew Propagation Modes in Model Extraction 499
9.1.5 Basic Elements of Timing Model Extraction 500
9.1.6 Secondary Load Dependent Networks 513
9.1.7 Characterization Point Selection 514
9.1.8 Constraint Generation during Model Extraction 516
9.1.9 Parallel Arcs in ETM 520
9.1.10 Latency Arcs Modeling 521
9.1.11 Latch-Based Model Extraction 521
9.1.12 Model Extraction in AOCV Mode 522
9.1.13 Stage Weight Modeling in ETM 522
9.1.14 AOCV Derating Mode 523
9.1.15 PG Pin Modeling During Extraction 524
9.1.16 Extracted Timing Models with Noise (SI) Effect 525

9.1.17 Extracted Timing Models with SOCV . . . 526
9.1.18 Merging Timing Models . . . 526
9.1.19 Limitations of ETM . . . 528
9.1.20 Validation of Generated ETM . . . 532
9.1.21 Auto-Validation of ETM . . . 535
9.1.22 ETM Extremity Validation . . . 536
9.1.23 Limitations/Implications of EV-ETM . . . 538
9.1.24 Ability to Check Timing Models . . . 538
9.2.1 Overview . . . 540
9.2.2 Boundary Model Flow . . . 540
9.2.3 Hierarchical Flow with Boundary Model . . . 541
9.2.4 Clock Mapping in Top Level Run with Boundary Models . . . 545
9.2.5 Recommendations for Generating Accurate Boundary Models . . . 546
9.2.6 Glitch-Specific Boundary Models . . . 547
9.2.7 Boundary Models in MMMC Mode . . . 550
9.2.8 Save-Restore of Hierarchical Analysis . . . 551
9.3.1 Overview . . . 553
9.3.2 Inputs and Outputs . . . 553
9.3.3 Using the PTM Flow to Generate Dotlib Model . . . 554
9.4.1 ILM Overview . . . 563
9.4.2 Creating ILMs . . . 564
9.4.3 Specifying ILM Directories at the Top Level . . . 567
9.4.4 ILMs Supported in MMMC Analysis . . . 568
9.4.5 ILM Validation Flow . . . 568
9.4.6 Use of SDF, SDC, and XTWF in ILM Flow . . . 570
9.4.7 Creating XILM . . . 574
9.4.8 Using ILM/XILM for Hierarchical Analysis . . . 574

10
Low Power Flows . . . 576

11
Appendix . . . 589

Appendix - Timing Debug GUI . . . 590

Appendix - Multi-CPU Usage 616

Appendix - Base Delay, SI Delay, and SI Glitch Correlation with SPICE 622

Appendix - Supported CPF Commands 632

12
Glossary 689

About This Manual

The Tempus Timing Solution software, also known as Tempus, provides a sign-off timing and signal integrity solution for a design flow. This manual describes how to install, configure, and use Tempus to implement digital integrated circuits.

Audience

This manual is written for experienced designers of digital integrated circuits. Such designers must be familiar with design planning, placement and routing, block implementation, chip assembly, and design verification. Designers must also have a solid understanding of UNIX and Tcl/Tk programming.

How This Manual Is Organized

The chapters in this manual are organized to follow the flow of tasks through the design process. Because of variations in design implementations and methodologies, the order of the chapters will not correspond to any specific design flow.

Each chapter focuses on the concepts and tasks related to the particular design phase or topic being discussed.

In addition, the following sections provide prerequisite information for using the Tempus software:

- Installation and Startup

  Describes how to install, set up, and run the Tempus software, and use the online Help system.

- Design Import

  Describes how to prepare data for analysis in the Tempus software.Tempus Timing Solution

Conventions Used in This Manual

This section describes the typographic and syntax conventions used in this manual.

`text`	Indicates text that you must type exactly as shown. For example: `report_annotated_check -missing_setup`
`text`	Indicates information for which you must substitute a name or value. In the following example, you must substitute the name of a specific file for *`worst_entries`*: `report_clock_timing -type skew` `  -nworst` *`worst_entries`*
text	Indicates the following: ■ Text found in the graphical user interface (GUI), including form names, button labels, and field names ■ Terms that are new to the manual, are the subject of discussion, or need special emphasis ■ Titles of manuals
`[ ]`	Indicates optional arguments. In the following example, you can specify none, one, or both of the bracketed arguments: `command [-arg1] [-arg2` *`value`*`]`
`[ | ]`	Indicates an optional choice from a mutually exclusive list. In the following example, you can specify any of the arguments or none of the arguments, but you cannot specify more than one: `command [arg1 | arg2 | arg3 | arg4]`
`{ | }`	Indicates a required choice from a mutually exclusive list. In the following example, you must specify one, and only one, of the arguments: `command {arg1 | arg2 | arg3}`

`{[ ] [ ]}`	Indicates a required choice of one or more items in a list. In the following example, you must choose one argument from the list, but you can choose more than one: `command {[arg1] [arg2] [arg3]}`
`{ }`	Indicates curly braces that must be entered with the command syntax. In the following example, you must type the curly braces: `command arg1 {x y}`
`. . .`	Indicates that you can repeat the previous argument.
. . .	Indicates an omission in an example of computer output or input.
Command – Subcommand	Indicates a command sequence, which shows the order in which you choose commands and subcommands from the GUI menu. In the following example, you choose *Analysis* from the menu, then *Specify Analysis Mode* from the submenu, and then *Advanced* from the tabs: *Analysis – Specify Analysis Mode – Advanced* This sequence opens the Specify Analysis Mode Advanced form.

Related Documents

For more information about Tempus, see the following documents. You can access these and other Cadence documents with Doc Assistant (cloud-based) documentation system.

- Tempus Known Problems and Solutions

  Describes important Cadence Change Requests (CCRs) for Tempus, including solutions for working around known problems.

- Tempus Text Command Reference

  Describes the Tempus text commands, including syntax and examples.

- Tempus Menu Reference

  Describes Tempus graphical user interface.

- New Features in Tempus

  Provides information about new features in this release of Tempus.

- Tempus Foundation Flows User Guide

  Describes the Cadence-recommended procedures for performing basic STA and SI signoff, MMMC signoff, and CPF-based MMMC signoff using Tempus Timing Solution.

- README file

  Contains installation, compatibility, and other prerequisite information, including a list of Cadence change requests (CCRs) that were resolved in this release. You can read this file online at downloads.cadence.com.

1

Product and Licensing Information

1.1 Tempus Product and Licensing Overview on page 13

1.2 About Licensing on page 13

1.3 Tempus Timing Solution Products on page 14

1.4 Tempus Timing Solution Product Options on page 17

1.5 Tempus License Options for Single View Distributed STA (DSTA) on page 19

1.6 Tempus License Options for Distributed MMMC (DMMMC) on page 20

1.7 Tempus License Options for Concurrent MMMC (CMMMC) on page 21

1.8 Tempus License Options for CMMMC in DSTA Mode on page 22

1.9 Tempus Paradime Flow Licensing for Interactive ECOs in CMMMC Mode on page 24

1.10 About Tempus Timing Solution Licenses on page 25

1.11 Checking Out Tempus Licenses for Product Options on page 26

1.1 Tempus Product and Licensing Overview

Each Cadence Tempus Timing Solution product is sold as part of a product package. Product packages might also include product options. The options provide advanced features and capabilities, such as support for enabling distributed client servers and/or accelerating performance through increased CPU multi-threading.

Each product and product option has a corresponding license. The software uses licenses to determine the features that are available when the software runs.

Related Topics

- About Licensing
- Tempus Timing Solution Products
- Tempus Timing Solution Product Options
- About Tempus Timing Solution Licenses
- Checking Out Tempus Licenses for Product Options

1.2 About Licensing

The following terminology is useful in understanding licenses.

- **Base license**

  The license that is checked out when the software starts. Only a full-fledged product license can be used as a base license. You cannot use a product option license as a base license to start the software.

- **Dynamic license**

  A license for a product option that is not checked out until a feature provided by the product option is needed. You can check out more than one dynamic license per base license. For more information on dynamic licenses, see Checking Out Tempus Licenses for Product Options.

- **Multi-CPU license**

  A license that enables additional CPUs for multi-threading, Superthreading, or distributed processing. Multi-CPU licenses must be product licenses, and can be checked out after the base license is checked out. You can check out more than one multi-CPU license per base license.

For more information on multi-CPU licenses, For more information on multi-CPU licenses, see Innovus System Licensing and Packaging on SourceLink ®.

- **Licensing Server license**

  Tempus running on Linux x86 platforms is integrated with FLEXnet version 11.16.4.0. The Tempus software using this version also requires the upgraded license server version, which is integrated with FLEXnet 11.16.4.0 or higher versions. If the license server is not updated, the Tempus software will not be able to run.

  You can check the version of the licensing server by running the following commands in the terminal where the license server is installed:

  ```
  lmgrd -v
  cdslmd -v
  ```

Related Topics

- Tempus Product and Licensing Overview
- Tempus Timing Solution Products
- Tempus Timing Solution Product Options
- About Tempus Timing Solution Licenses
- Checking Out Tempus Licenses for Product Options

1.3 Tempus Timing Solution Products

The Tempus Timing Solution package includes the products listed in the table below. To start any of these products, type the following UNIX/Linux command:

`tempus -stylus`

Table 1-1 Tempus Timing Solution Products

Name and License String	Abbreviation	Product Number	Description
Tempus Timing Solution L	Tempus L	TPS100	Tempus L is one of the two base licenses in the Tempus line. Tempus L provides better timing analysis performance for both Graph-Based Analysis (GBA) and Path-Based Analysis (PBA) versus Tempus XL and also allows for multi-threaded PBA, which is unavailable in Tempus. Tempus L is functionally equivalent to Tempus XL with the exception of concurrent MMMC operation. Concurrent MMMC is available only in Tempus XL.
`Tempus_Timing_Signoff_L`			
			Tempus L can be paired with Tempus ECO for Signoff ECO and it can be paired with Tempus MP to enable more CPUs for multi-threading. Tempus L can be used for acceleration and enables 8 CPUs. Tempus L can checkout Tempus ECO, or Voltus L/Voltus XL to load the concurrent MMMC view definitions by default. If any of these licenses are not available, the software will load the design with a SMSC (single-mode single-corner) definition.

Name and License String	Abbreviation	Product Number	Description
Tempus Timing Solution XL	Tempus XL	TPS200	Tempus XL provides all the functionality of Tempus L and includes the primary innovation of distributed design. Distributed design leverages massive parallel computation by leveraging multiple compute servers to perform static timing analysis for a single view for designs up to 100s of millions of placeable cell instances. In addition, Tempus XL provides eight additional CPUs over Tempus L - no compromise on hierarchical/ incremental analysis, or concurrent MMMC analysis in a single session.Tempus XL can be used for acceleration and enables 16 CPUs.
`Tempus_Timing_Signoff_XL`			
Virtuoso Digital Signoff Timing Solution		VDS100	
Tempus Library Validation Solution		TPS500	

Related Topics

- Tempus Product and Licensing Overview
- Tempus Timing Solution Products
- Tempus Timing Solution Product Options
- About Tempus Timing Solution Licenses
- Checking Out Tempus Licenses for Product Options

1.4 Tempus Timing Solution Product Options

The Tempus Timing Solution product options provide the extendibility and cost-effective access to additional advanced technologies, such as support for enabling distributed client servers and/or accelerating performance through increased CPU multi-threading. These options are listed in the table below.

Table 1-2 Tempus Timing Solution Product Options

Name and License String	Abbreviation	Product Number	Description
Tempus Timing Solution ECO	Tempus ECO	TPS300	The Tempus ECO option works with either Tempus L or Tempus XL to enable MMMC signoff ECO functionality. This enables physically-aware MMMC timing optimization for setup/hold and DRV violations. It also supports leakage power optimization. Tempus ECO is only an additional license, which is required to run physically-aware timing signoff optimization.Tempus ECO can also be used to enable concurrent MMMC operation when used with the base Tempus L license.
`Tempus_Timing_Signoff_TSO`			
Tempus Massively Parallel	Tempus MP	TPS400	Tempus MP is a dual use accelerator for Tempus. In non-distributed analysis, Tempus MP adds sixteen CPUs to the base license of either Tempus L or Tempus XL for enhanced multi-threaded performance.

Name and License String	Abbreviation	Product Number	Description
			While performing design distribution, Tempus MP is required for each distributed process. Distributed processes can be on separate hardware clients or can be packed on a single large compute server. In either case, it is one Tempus MP per process and each process enables up to sixteen CPUs. Tempus MP licenses can be stacked to provide more CPUs in blocks of sixteen.Tempus MP can be used for acceleration and enables 16 CPUs.
Tempus Advanced Analysis Option		TPS210	This is a single option required for all the designs using any of the functionality in this analysis package. This license is not tied to any specific technology node or foundry.
Tempus Timing Solution 3nm Option		TPS230	This is a single option required for all the 3nm designs independent of the foundry. This includes all the modeling features that are required by foundries to enable and support 3nm designs.
Tempus Timing Solution 2nm Option		TPS232	This is a single option required for all the 2nm designs independent of the foundry. This includes all the modeling features that are required by foundries to enable and support 2nm designs.
Certus Solution 2nm Option	Certus	CRTS232	This is a single option required for the Certus Client Manager and Clients for all the 2nm designs independent of the foundry.
Certus Multi-CPU Option	Certus	CRTS150	This option is required for ECO/ Optimization only for Certus clients.

Note: The default multi-CPU order is Tempus MP, Tempus L, and Tempus XL.

Related Topics

- Tempus Product and Licensing Overview
- About Licensing
- Tempus Timing Solution Products
- About Tempus Timing Solution Licenses
- Checking Out Tempus Licenses for Product Options

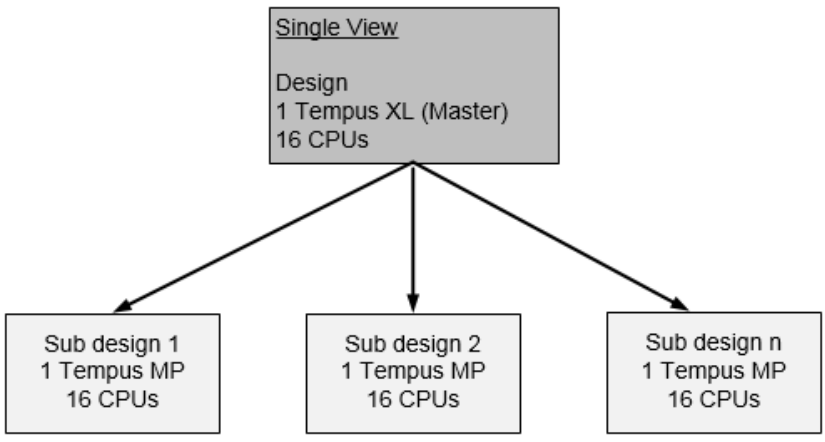
1.5 Tempus License Options for Single View Distributed STA (DSTA)

The following are the Tempus license requirements for single view DSTA mode:

- Master requires 1 Tempus XL (up to 16 CPUs).
- Clients require 1 Tempus MP each (up to 16 CPUs per client).
- Additional Tempus MP keys unlock additional CPUs for the master and client (up to 16 more per license).
- Each slave checks out a Tempus-MP license and also allows Tempus-XL functionality.

The order of licenses is shown in Figure 1-1 on page 19:

Figure 1-1 DSTA license requirements



Related Topics

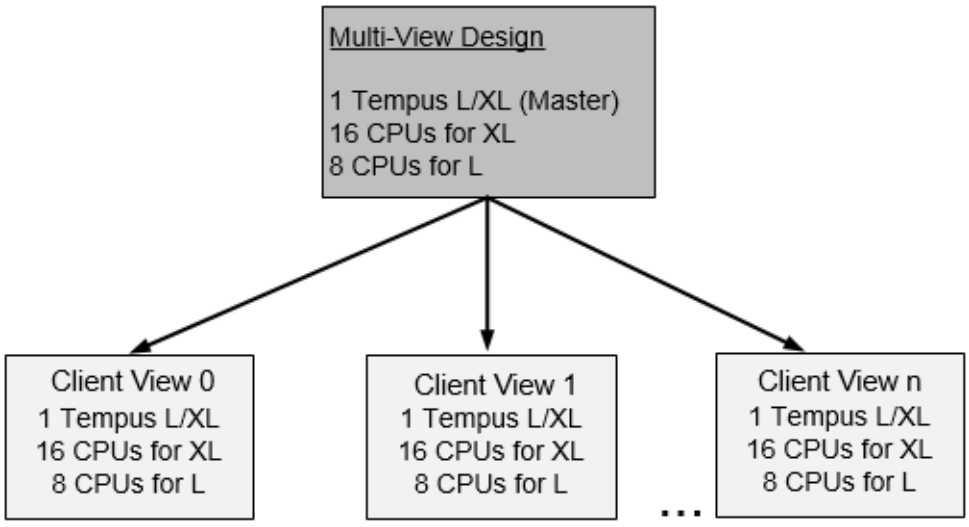
- Tempus License Options for Distributed MMMC (DMMMC)
- Tempus License Options for Concurrent MMMC (CMMMC)
- Tempus License Options for CMMMC in DSTA Mode

1.6 Tempus License Options for Distributed MMMC (DMMMC)

The following are the Tempus license requirements in DMMMC mode, see Figure 1-2 on page 20:

- 1 Tempus L or XL to start the master session
- 1 Tempus L or XL per view or slave (1:1 view/slave ratio)
- Tempus XL enables 16 CPUs, Tempus L enables 8 CPUs

Figure 1-2 DMMMC license requirements



Related Topics

- Tempus License Options for Single View Distributed STA (DSTA)
- Tempus License Options for Concurrent MMMC (CMMMC)
- Tempus License Options for CMMMC in DSTA Mode

1.7 Tempus License Options for Concurrent MMMC (CMMMC)

The following are the Tempus license requirements in CMMMC mode:

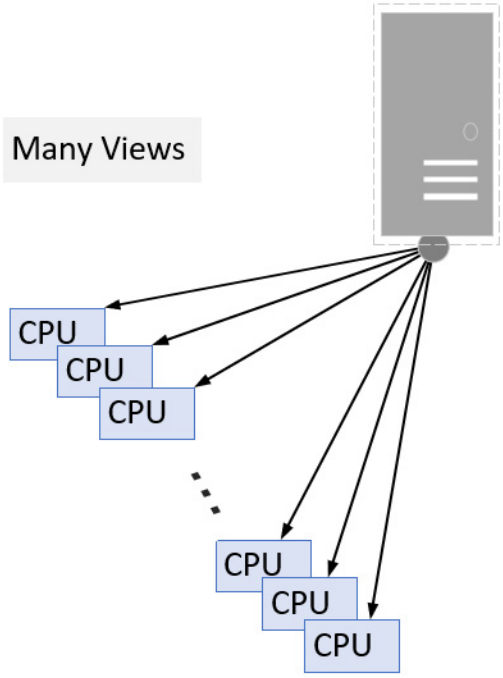
- When 'N' views are combined into a single STA job on a single machine, the following licenses are required:
  - 1 to 8 views: 1 XL (16 CPUs) or [1 L + 1 ECO] (8 CPUs)
  - 9 to 16 views: 2 XL (16 CPUs) (the L + ECO combo not accepted)
  - Unlimited views: 3 XL (16 CPUs) (the L + ECO combo not accepted)
- More CPUs enabled by checking out more L (8), XL (16), or MP (16) licenses
- Minimum configuration: 1/2/3 XL or 1(L + ECO)

Examples:

- CMMMC with 6 views on 12 CPUs = 1 XL
- CMMMC with 15 views on 18 CPUs = 2 XL + 1 XL or = 2 XL + 1 MP
- CMMMC with 20 views on 32 CPUs = 3 XL + 1 XL or = 3 XL + 1 MP

Note: The CMMMC STA licensing is different from Tempus ECO, which is also multi-mode/ multi-corner.

Figure 1-3 CMMMC license requirements



Related Topics

- Tempus License Options for Single View Distributed STA (DSTA)
- Tempus License Options for Distributed MMMC (DMMMC)
- Tempus License Options for CMMMC in DSTA Mode

1.8 Tempus License Options for CMMMC in DSTA Mode

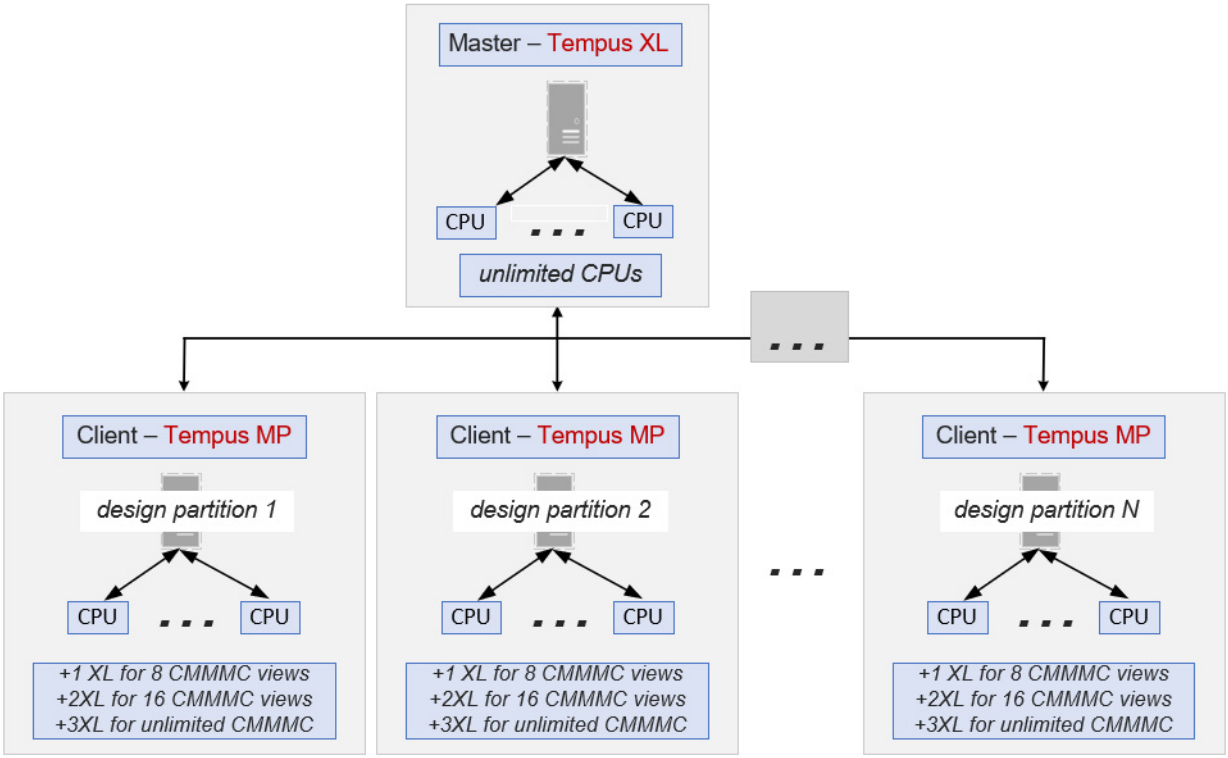
Tempus DSTA can distribute a CMMMC job across multiple computers. Each computer performs local multi-threading. The licensing requirements are given below:

- The master process requires an XL license:
  - unlimited views (master not affected by any number of views)
  - unlimited CPUs
- Each client requires a Tempus MP license - and licenses CMMMC just like non-DSTA jobs:
  - 1XL (16 CPU) or 1{L+ECO} (8 CPU) for <= 8 views

- ❑ 2 XL for 9 to 16 views
- ❑ 3 XL for unlimited views
- ❑ Add extra CPUs with additional MP licenses

- The required minimum configuration for this setup is 3 XL + 2 MP.

Figure 1-4 CMMMC license requirements in DSTA mode



Examples:

- Run CMMMC with 6 views across 5 computers = 1XL + 5MP +5 XL = 80 CPUs (5 clients@16)
- Run CMMMC with 10 views across 3 computers = 1XL + 3MP + 6XL= 48 CPUs (3 clients@16)

Related Topics

- Tempus License Options for Single View Distributed STA (DSTA)
- Tempus License Options for Distributed MMMC (DMMMC)

- Tempus License Options for Concurrent MMMC (CMMMC)

1.9 Tempus Paradime Flow Licensing for Interactive ECOs in CMMMC Mode

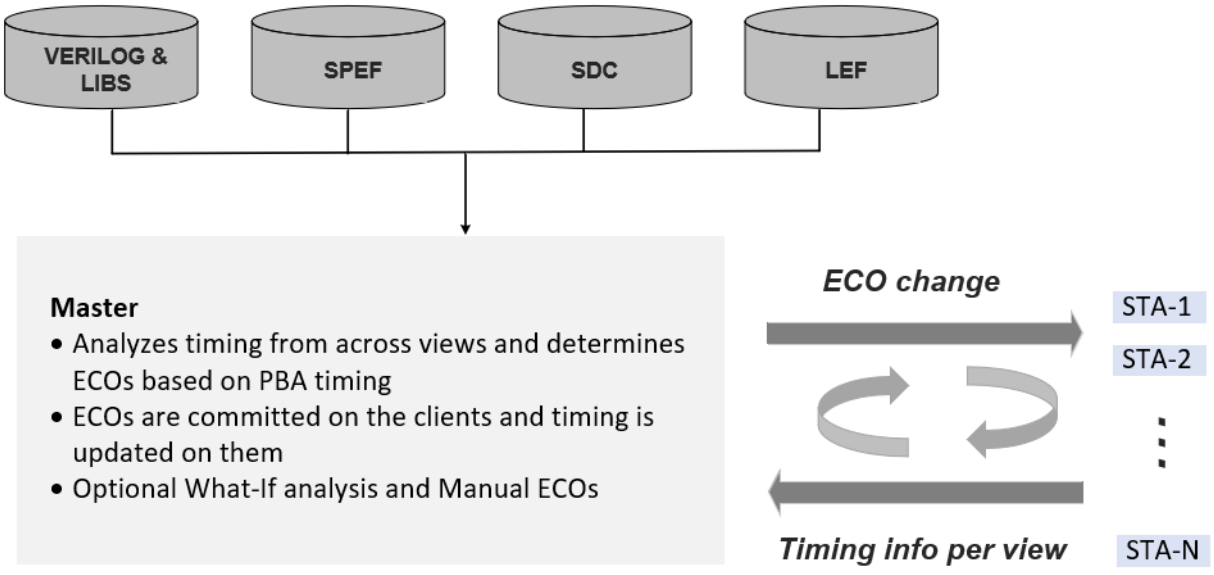
Tempus Paradime keeps all the STA sessions alive so that ECO changes can be committed and timing can be immediately updated incrementally across all the views. The Paradime licensing model is based on a master-client architecture (DMMMC), where:

- The Master session requires one Tempus XL and one Tempus ECO.
- Each client requires a Tempus XL license.

You can stack extra Tempus MP, Tempus L, and Tempus XL licenses to enable more CPUs for each client session. All the licenses are checked out upon start up and are valid throughout the session.

The license flow is illustrated in the diagram below.

Figure 1-5 Tempus Paradime Flow Licensing in CMMMC Mode



Related Topics

- Tempus License Options for Single View Distributed STA (DSTA)

- Tempus License Options for Distributed MMMC (DMMMC)
- Tempus License Options for Concurrent MMMC (CMMMC)
- Tempus License Options for CMMMC in DSTA Mode

1.10 About Tempus Timing Solution Licenses

When you run a command to invoke a product or product option, a license is checked out. Each product and product option has a unique license string (also called a license key). The following table lists the product and product option names, and provides the corresponding license strings.

Table 1-3 Product and product options and corresponding license strings

Product or Option	License String
Tempus Timing Solution L	`Tempus_Timing_Signoff_L`
Tempus Timing Solution XL	`Tempus_Timing_Signoff_XL`
Tempus Timing Solution ECO	`Tempus_Timing_Signoff_TSO`
Tempus Timing Solution MP	`Tempus_Timing_Signoff_MP`

The version of vendor daemon (cdslmd) and license daemon (lmgrd) should be 11.11.1.1 or higher. If the daemon versions have not been updated, Tempus may fail to checkout a license:

```
$ tempus
Checking out Tempus Timing Solution license ...
Fail to find any Tempus Timing Solution license...
Check your Tempus Timing Solution license status.
```

To fix the above error, restart the license server using the latest cdslmd and lmgrd versions included in the release tree.

Related Topics

- Tempus Product and Licensing Overview
- About Licensing
- Tempus Timing Solution Products
- Tempus Timing Solution Product Options

1.11 Checking Out Tempus Licenses for Product Options

The `tempus` command specifies the product options to be checked out when you invoke the software. The product options specified with the `-lic_startup_options` parameter are checked out immediately and the product options specified with the `-lic_options` parameter are checked out dynamically. The use model is as follows:

```
tempus -lic_startup_options "lic1 lic2 ..."
```

Note: If you do not want any product options to be checked out dynamically, use empty quotes with the `-lic_options` parameter, as follows:

```
tempus -lic_options "lic1 lic2 ..."
```

The following command can be used to check out product options after you have invoked the software:

`set license check` `-checkout` *license* `-optionList "license1 license2 …"`

The product option specified with the `-checkoutList` parameter is checked out immediately and the product options specified with the `-optionList` parameter are checked out dynamically. With the `-checkOut` parameter, you can specify only one product option.

Note: If you do not want any product options to be checked out dynamically, use empty quotes with the -optionList parameter, as follows:

```
set_license_check -checkout option -optionList " "
```

Important

You cannot check out a license for a product option if you have not checked out a base license.

Related Topics

- Tempus Product and Licensing Overview
- About Licensing
- Tempus Timing Solution Products
- Tempus Timing Solution Product Options
- About Tempus Timing Solution Licenses

2

Design Import

2.1 Design Import Overview on page 29

2.2 Input Requirements on page 29

2.3 Design Import Flow on page 31

2.4 Performing Design Sanity Checks in Tempus on page 35

2.5 Tempus File Management on page 35

2.1 Design Import Overview

This topic describes the tasks that you perform to import the data and prepare for analysis in Tempus. The Tempus software supports both MMMC (multi-mode multi-corner) and non-MMMC methods of importing design data. However, Tempus database has MMMC configuration settings by default. Tempus works in logical and physical modes. Physical information is available in the form of LEF/DEF format, Innovus, or Open Access (OA) database.

2.1.1 Input Requirements

Required Design Data

- Timing Libraries (Liberty dotlib files)
- Verilog Netlist
- SDC Constraints
- Parasitic Data (SPEF/RCDB)

Optional Design Data

- Delay Data (SDF file)
- Physical Data

Timing Libraries

Timing library files contain timing information in ASCII format for all standard cells, blocks, and I/O pad cells. Tempus reads timing library format files (.lib).You can read in the library files using the `read libs` command.

Verilog Netlist

Tempus reads synthesized netlist in Verilog. You can read in the netlist files using the `read_netlist` command.

SDC Constraints

- To ensure that your design meets the timing requirements, you must first specify the requirements by setting the constraints. You can use the timing constraints to set:

- Timing context for constraint assertions
- Boundary conditions, such as input and output delays
- Slew rates
- Path exceptions, such as false paths, path delays, and cycle additions
- Disable certain paths in the design

You can read the timing constraints using the `read_sdc` command.

Parasitic Data (SPEF/RCDB)

You can specify the parasitic data for more accurate timing analysis using a SPEF or RCDB file.

You can use the `read_spef` command to specify the SPEF file. The `read_parasitics` command is used for reading SPEF and RCDB-based RC parasitics information, and already created RCDBs in Tempus.

Delay Data (SDF file)

This is an optional input requirement.

An SDF file contains details about the absolute delays for all cells and interconnects, including IR drop and crosstalk impact. It annotates user-specified delay information from the SDF file into the timing system. You can use the `read_sdf` command to supply pre-calculated delays and timing check values from an external delay calculator, or from a previous Tempus session.

In Tempus, you can use the `read_sdf` command to read in a SDF file and then the timing analysis engine will use the delays annotated from this SDF file.

Physical Data

This is an optional input requirement. In Tempus, you can read physical information, such as placement, floorplan, and routing that you created using LEF and DEF in Innovus.

To read the physical data, use one of the following commands:

- The `-physical_data` parameter of the `read_db` command with either configuration file or Innovus file that you generate using Innovus.

- The `read def` command after reading the LEF files, netlist, and `.lib` files.

Note: To read LEF files, use the `-lefs` parameter of the `read_physical` command.

2.2 Design Import Flow

The design import flow involves the following steps:

- Initializing the design
- Writing the design
- Reading the design

Initializing the design involves reading the timing data, physical data, HDL/netlist, and power intent data. After initialization, the design is in consistent state and is ready for timing analysis.

There are two main types of initialization:

- MMMC Timing Initialization
- Simple Timing Initialization

2.2.1 MMMC Timing Initialization

The MMMC (multi-mode multi-corner) timing initialization uses the `read_mmmc` command to load a MMMC configuration file with the timing setup data. The physical data can optionally be loaded into Tempus to support physical flows, such as Tempus ECO. The power intent data is required for all multi-supply, multi-voltage (MSV) designs. This flow is common to all the tools and supports all the optimization and analysis flows.

The following is an example script to import MMMC data:

```
read_mmmc top.mmmc                              #required
read_physical -lef {tech.lef stdcell.lef}       #optional in Genus, Tempus,
                                                #required for Innovus
read_netlist top.v                              #required, Genus also allows
read_hdl/elaborate                              #Genus only
read_power_intent -1801 top.1801                #required for power domain designs
init_design                                     #required
read_def top.def                                #optional in all tools
```

The above flow is described below:

- `read_mmmc` will create the library_set, rc_corner, timing_condition, constraint_mode, and analysis_view objects as specified in the MMMC file. It will also load the timing libraries required by the active views from the `set_analysis_view` command in the MMMC file. The reading of other files like SDC constraints is deferred until `init_design`. During `init_design`, only the files required by the active views from the `set_analysis_view` command are loaded. Timing libraries or constraint files for non-active views are not loaded unless those views are made active with set_analysis_view later in the flow.
- `read_physical` is used to load LEF or OA technology data and physical library data. This command is required by Innovus and can be optionally loaded in Tempus and Genus to support physical design flows.
- `read_netlist` is used to load a Verilog gate-level netlist, or an OA netlist. Genus also allows the usage of `read_hdl`/elaborate instead of `read_netlist` to synthesize an RTL netlist (see the Genus documentation for details).
- `read_power_intent` can optionally be used to load a power intent file (IEEE 1801, or CPF). The power data is incomplete until the binding done with the timing data during `init_design`. A power intent description in either CPF or IEEE 1801 format is generally required for MSV designs.
- `init_design` loads any deferred timing data from `read_mmmc` above, and binds the timing data to the netlist, and binds the analysis views to any power domains, if needed. Various consistency checks are done, and after this step the design is ready for use.
- `read_def` can optionally be used to load an initial DEF file after `init_design`. Innovus also allows `read_floorplan` to read in a previously saved floorplan file instead of read_def. After read_def is done, it is important to run `commit_power_intent -keep_rows -power_domain` to make sure any DEF GROUPS statements (or floorplan file Groups section) has not changed the power_domain group definitions, but keep the rows from the DEF or .fp file.

The design import flow in MMMC mode is summarized below:

Table 2-1 MMMC Design Import Flow

Initialize Design	

Logical Data	`read_mmmc <>`
	`read_netlist <> [-top <>]`
	`read_power_intent <>` (optional)
	`init_design`

Physical Data	`read_mmmc <>`
	`read_physical -lefs <>`
	`read_netlist <> [-top <>]`
	`read_power_intent <>` (optional)
	`init_design`

OA Physical Data	`read_mmmc <>`
	`read_physical -oa_ref_libs <>`
	`read_netlist <> [-top <>]`
	`read_power_intent <>` (optional)
	`init_design`

Write Design	
Logical Data	`write_db <out_dir>`
Physical Data	`write_db <out_dir>`
OA Physical Data	`write_db -oa_lib_cell_view <lib cell view>`
Read Design	
Logical Data	`read_db <in_dir>`
Physical Data	`read_db <in_dir> -physical_data`
OA Physical Data	`read_db -oa_lib_cell_view <lib cell view> \` `-physical_data`

To load a design in DMMMC mode, refer to the "Distributed Multi-Mode Multi-Corner Timing Analysis" chapter of the *Tempus User Guide*.

2.2.2 Simple Timing Initialization

A single timing mode and delay corner (SMSC) analysis can use `read_libs` and `read_sdc` commands instead of loading a MMMC file using `read_mmmc`. The physical data is optional, and power intent is not supported. This is intended for simple synthesis flows or timing signoff analysis that only deals with one timing mode and delay corner at a time.

A typical initialization script is shown below:

```
read_libs all.lib          #required
read_netlist top.v         #required
read_sdc top.sdc           #required
init_design                #required
```

The above flow is described below:

- `read_libs` will save the file names, but defer reading them until the netlist is read. To configure the physical aware ECO flow of Tempus, use `read_physical` to add physical library data after `read_libs`.
- `read_netlist` will load the timing libraries and a Verilog netlist, or an OA netlist. `read_sdc` will read in the SDC timing constraints into the analysis.
- `init_design` does various consistency checks, finishes setting some default initialization values, and after this step the design is ready for use. To configure the physical aware ECO flow of Tempus, use `read_def` (after this step), to load a DEF file.

The design import flow in non-MMMC mode is summarized below:

Table 2-2 Non-MMMC Design Import Flow

Initialize Design	

Logical Data	`read_libs <>`
	`read_netlist <> [-top <>]`
	`read_sdc <>`
	`init_design`

Physical Data	`read_libs <>`
	`read_physical -lefs <>`
	`read_netlist <> [-top <>]`
	`read_sdc <>`
	`init_design`

OA Physical Data	`read_libs <>`
	`read_physical -oa_ref_libs <>`
	`read_netlist <> [-top <>]`
	`read_sdc <>`
	`init_design`

Write Design	
Logical Data	`write_db <out_dir>`
Physical Data	`write_db <out_dir>`
OA Physical Data	`write_db -oa_lib_cell_view <lib cell view>`
Read Design	
Logical Data	`read_db <in_dir>`
Physical Data	`read_db <in_dir> -physical_data`

OA Physical Data	`read_db -oa_lib_cell_view <lib cell view> \` `-physical_data`

When the data is imported using non-MMMC commands, Tempus internally stores it in MMMC configuration as the following MMMC objects:

```
default_emulate_view
default_emulate_delay_corner
default_emulate_rc_corner
default_emulate_libset_max
default_emulate_libset_min
default_emulate_constraint_mode
```

2.3 Performing Design Sanity Checks in Tempus

At various stages of the design process, you can check for missing or inconsistent library and design data.

To perform these checks, use the following commands:

- check_design
- check_timing_library
- check_timing

2.4 Tempus File Management

Tempus allows you to manage temporary and auto-generated files. These are described in the sections below:

Temporary Files Management

You can specify the directory for writing out temporary files during a Tempus session.

The Tempus software creates a unique temporary sub-directory for a session, specified by `setenv TMPDIR <dir-name>` at the command prompt, where:

- Default value of TMPDIR is “/tmp”.
- All temporary files will be created under directory:

  `“$TMPDIR/tempus_<user>_<pid>_<xxxxx/”`

- If write-permission is not provided in $TMPDIR, then TMPDIR will internally reset to "/tmp".
- If the specified TMPDIR does not have enough space, the software will not launch.
- All temporary files/directories will be deleted, once a session ends.
- No temporary file is expected to be created in the run directory.

This support is also available for DMMMC and DSTA flows.

Auto-Generated File Management

You can control the location and name of auto-generated files in a session. These files are created by Tempus automatically and are retained after the run completes. This includes report files, or other output files with default names that are auto-generated in the current run directory. The software allows you to control the directory that the files should be redirected to, and also add unique prefixes to file names. You can use the following attributes to control this behavior:

- `auto file dir`

  Represents the top-level directory to store files/sub-directories that are auto-generated by the software.

- `auto file prefix`

  Represents the prefix to be applied to all files/sub-directories that are auto-generated by the software.

3

Installation and Startup

3.1 Tempus Product and Installation Information on page 38
3.2 Setting Up the Tempus Run Time Environment on page 38
3.3 Launching the Tempus Console on page 39
3.4 Completing Command Names on page 40
3.5 Command-Line Editing on page 40
3.6 Setting Preferences on page 42
3.7 Starting the Tempus Software on page 42
3.8 Accessing Tempus Documentation and Help on page 44
3.9 Accessing Documentation and Help from Tempus GUI on page 44
3.10 Using the Tempus man and help Commands on the Command-Line on page 45
3.11 Other Sources of Cadence Product Information on page 47

3.1 Tempus Product and Installation Information

For product, release, and installation information, see the `README` file at any of the following locations:

- `http://downloads.cadence.com`
- In the software installation, where it is also available when you are using or running the software

For information about the Tempus licenses, see "Product and Licensing Information" chapter.

3.2 Setting Up the Tempus Run Time Environment

If `install_dir` is the location of the Tempus installation, then you should setup the run time environment as follows:

- Add the `install_dir/bin` directory to the path. The `bin` directory has links to all the public executable files in the install hierarchy.
- If you want the Tempus man pages to be available with the Unix `man` command, then you can add the `install_dir/share/tempus/man` to the MANPATH environment variable.
- If you want the Stylus Common UI Tempus man pages to be available with the Unix `man` command, then you can add `install_dir/share/tempus/stylus_man` to the MANPATH environment variable.
- If you want the Tcl man pages to be available with the Unix `man` command, then you can add the `install_dir/share/tcltools/man` to the MANPATH environment variable.

For example, you can add the following to the startup shell script:

```
set install_dir = /tools/tempus16.2/lnx86
set path = ($install_dir/bin $path)
setenv MANPATH $install_dir/share/tempus/man:$install_dir/share/tcltools/
man:$MANPATH
```

Note: When Tempus launches, it automatically adds the legacy or the Stylus CUI man pages (if the `-stylus` parameter is specified), and the Tcl man pages at the beginning of the current MANPATH inside Tempus. Therefore, from within Tempus the `man` command will show both the sets of man pages before any other man pages.

3.2.1 Temporary File Locations

Each Tempus session creates its own temporary directory to store temporary files at the beginning of the run.

By default, the `tmp_dir` is created in the `/tmp` directory. If the Unix variable TMPDIR is set, then the `tmp_dir` is created inside `$TMPDIR`.

The name of the `tmp_dir` will appear as given below:

```
ssv_temp_[pid]_xxxxxx
```

Where `_xxxxxx` is a string that is added to make the directory unique. For example:

```
ssv_temp_10233_nfp9ez
```

The temporary directory is automatically removed on exit or if the run is terminated with a catchable signal (for example, SIGSEGV).

Supported and Compatible Platforms

The `README` file lists the supported and compatible platforms for this release.

3.3 Launching the Tempus Console

When you start Tempus in the non-GUI environment using the `-no_gui` parameter, the UNIX window (shell tool, xterm, and so on) where you start the Tempus session is called the Tempus console. This is where you enter all Tempus text commands and where the software displays messages. When a session is active, this console displays the following prompt:

```
tempus>
```

When you start the Tempus in the GUI environment, the Tempus console is displayed within the main Tempus window.

If you use this console for other actions, for example, to use the vi editor, the session suspends until you finish the action.

If you suspend the session by typing `Control-z`, the `tempus>` prompt is no longer displayed. To return to the Tempus session, type `fg`, which brings the session to the foreground.

3.3.1 Completing Command Names

Use the `Tab` key within the software console to complete text command names.

After you type a partial text command name and press the `Tab` key, the software displays the exact command name that completes or matches the text you typed (if the string is unique to one text command) or all the commands that match the text you typed.

For example, if you type the following text and press the `Tab` key

```
report_ti
```

The software displays the following command:

```
report_timing
```

If you type the following text and press the `Tab` key

```
report_t
```

The software displays the following commands:

```
report_table
report_tclist
report_thermal
report_thresh
report_timing
report_timing_derate
report_timing_lib
```

3.3.2 Command-Line Editing

The Tempus software provides a GNU Emacs-like editing interface. You can edit a line before it is sent to the calling program by typing control characters. A control character, shown below as a caret followed by a letter, is typed by holding down the `Control` key when typing the character.

Most editing commands can be given a repeat count, n, where n is a number. To enter a repeat count, press the `Esc` key, the number, and then the command to execute. For example, `Esc 4 ^f` moves forward four characters. If a command can be given a repeat count, the text `[`n`]` is shown at the end of its description.

You can type an editing command anywhere on the line, not just at the beginning. You can press `Return` anywhere on the line, not just at the end.

Note: Editing commands are case sensitive: `Esc F` is not the same as `Esc f`.

Table 3-1 Control (^) Characters

^A	Move to the beginning of the line
^B	Move left (backwards) [*n*]
^C	Exits from editing mode, returning the console to normal Tempus System mode
^D	Delete character [*n*]
^E	Move to end of line
^F	Move right (forwards) [*n*]
^G	Ring the bell
^H	Delete character before cursor (backspace key) [*n*]
^I	Complete filename (Tab key); see below
^J	Done with line (Return key)
^K	Kill to end of line (or column [*n*])
^L	Redisplay line
^M	Done with line (alternate Return key)
^N	Get next line from history [*n*]
^P	Get previous line from history [*n*]
^R	Search backward (forward if [*n*]) through history for text; must start line if text begins with an up arrow
^T	Transpose characters
^V	Insert next character, even if it is an edit command
^W	Wipe to the mark
^X^X	Exchange current location and mark
^Y	Yank back last killed text
^[	Start an escape sequence (Esc key)
^]c	Move forward to next character *c*
^?	Delete character before cursor (Delete key) [*n*]

3.3.3 Setting Preferences

You set preferences at the beginning of a new design import. You can assign special characters for the design import parser for Verilog®, DEF, and PDEF files, and control the display of the Physical view window. You can also change the hierarchical delimiter character in the netlist before importing the design, and change the DEF hierarchical default character and the PDEF bus default delimiter before loading the file.

Note: If you change the default values for the DEF delimiter or PDEF bus delimiter, these changes become the default delimiters for the DEF and PDEF writers.

For information on setting design preferences, see *Tools - Preference*s in the *Tempus Menu Reference.*

Initialization Files

By default, various initialization files are loaded up at startup, if they exist. They can be used to configure the GUI, load utility Tcl files, or configure Tempus settings.

For a list of the files and the order in which they are loaded, refer to `tempus` in the *Tempus Text Command Reference*.

3.4 Starting the Tempus Software

To start a Tempus session, type the following command with the appropriate parameters on the UNIX/Linux command line.

```
tempus
```

For information on using this command, see `tempus` in the *Tempus Text Command Reference*.

This command starts one of the following products:

- Tempus L
- Tempus XL

For an overview of the products and product licensing, see “Product and Licensing Information”.

3.4.1 Using the Log File Viewer

The following methods are available to view the log file:

- Integrated Log File Viewer on page 43
- Standalone Log File Viewer on page 43

Integrated Log File Viewer

You can use the integrated log file viewer when the software is running. It has the following features:

- Ability to expand and collapse command information.
- Ability to view multiple log files in separate console windows simultaneously.
- Color coding of error, warning, and information messages.
- Access to the documentation in the *Tempus Text Command Reference*. When a log file is displayed, click on any of the underlined commands to open an HTML window that displays the documentation for that command.

Use one of the following methods to use the viewer:

→Select *Tools - Log Viewer* on the main menu.

The *Log File* window is displayed. Select the log file to view. The software opens a separate console window and displays the log file.

For more information, see Tools - Log Viewer in the "*Tools Menu*" chapter of the *Tempus Menu Reference*.

Standalone Log File Viewer

You can use the standalone viewer even if the software is not running. It provides most of the same functionality as the viewer that is run within the software but does not provide access to the documentation.

To use the standalone viewer, type the following UNIX/Linux command in the console window:

```
viewlog [-file logFileName]
```

The viewer opens the most recently created log file unless you specify a different file with the `-file` parameter.

3.5 Accessing Tempus Documentation and Help

You can access the Tempus documentation and help system by using the following methods:

- Launching Doc Assistant on page 44
- Accessing Documentation and Help from Tempus GUI on page 44
- Using the Tempus man and help Commands on the Command-Line on page 45
- Other Sources of Cadence Product Information on page 47

Launching Doc Assistant

To launch the Doc Assistant, you can use one of the following ways:

- Linux systems: Navigate to the `<install_directory>/lnx86/bin` directory, and run the `cda` command.
- Tool's Help menu.

For more information, see the Doc Assistant User Guide.

3.6 Accessing Documentation and Help from Tempus GUI

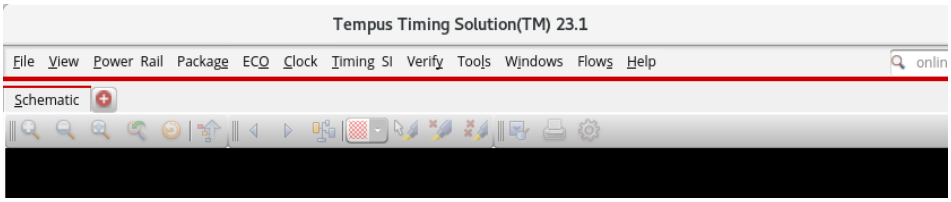
The Tempus software provides the following two methods to access documentation and help from the GUI:

- Select Help on the Main Tempus Menu on page 44
- Select Help from a Tempus Form on page 45

Select Help on the Main Tempus Menu

Follow the steps given below:

Figure 3-1 Help Menu

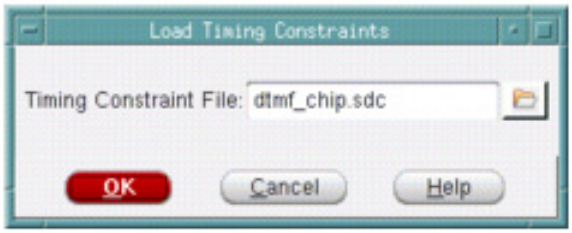


→ Select *Help*, and click any of the following options:

- *Text Command Reference*
  Opens the Table of Contents page of the text command reference.
- *User Guide*
  Opens the Table of Contents page of the user guide.
- *Known Problems and Solutions*
  Opens the Table of Contents page of the known problems and solutions document.

Select Help from a Tempus Form

→Click the *Help* button in the bottom right corner of a form.



Clicking the *Help* button opens the entry for the form in Doc Assistant.

3.7 Using the Tempus man and help Commands on the Command-Line

Using the help Command to View the Command Syntax

- To see syntax information for a command, type the following command in the software console:

  ```
  help command_name
  ```

For example, to see syntax information for the `read_lib` command, type the following command:

```
help read_lib
```

The software displays the following text:

```
Usage: read_lib <dotlib_file> [-min <min_lib_list>]
[-max <max_lib_list>] [-cdb <cdb_list>]
[-lef <file_names>] [-pgv <power_grid_list>]
```

- To see the entire list of Tempus commands and their syntax, type the following command in the software console:

  ```
  help
  ```

Using the man Command to View the Command Description

To see the complete set of information for a Tempus command, type the following command in the software console:

```
man command_name
```

For example, to see the complete set of information for the `read_lib` command, type the following command:

```
man read_lib
```

Using the help Command to View Message Summary

To see the message summary of a particular message ID, type the following command in the software console:

```
help msg_id
```

For example, to see the message summary for the TECHLIB-1002 message ID, type the following command:

```
help TECHLIB-1002
```

Using the man Command to View Message Detail

To see the message detail of a particular message ID, type the following command in the software console:

```
man msg_id
```

For example, to see the message summary for the SI-2253 message ID, type the following command:

```
man SI-2253
```

The detailed description is not available for all active message IDs.

3.8 Other Sources of Cadence Product Information

You can also get help on Cadence products by selecting *Customer Support* on the *Help* menu. The *Customer Support* sub menu provides access to the following Cadence resources:

- Cadence Online Support
  Opens Cadence Online Support in your browser.
- Web Collaboration
  Opens your default browser and explains the current methodology followed by Cadence to support web based collaboration.

4

Analysis and Reporting

4.1 Base Delay Analysis on page 49

4.2 Signal Integrity Delay and Glitch Analysis on page 63

4.3 Timing Analysis Modes on page 125

4.3.1 Advanced On-Chip Variation (AOCV) Analysis on page 145

4.3.2 Statistical On-Chip Variation (SOCV) Analysis on page 170

4.4 Parametric Timing Robustness Analysis on page 187

4.5 Path-Based Analysis on page 194

4.6 Analysis Of Simultaneous Switching Inputs - SSI on page 200

4.7 Waveform Aware and Rail Swing Checks on page 206

4.8 Set Instance Library Flow on page 211

4.9 Inter-Power Domain (IPD) Analysis on page 217

4.10 Aging-Aware STA Analysis on page 237

4.11 Advanced Multiple-Input Switching Analysis on page 253

4.12 Derate-Based Advanced Multi-Input Switching (AMIS-Derate) Analysis on page 265

4.13 Voltage Threshold Skew Modeling and Analysis on page 277

4.14 Design Rule Violation (DRV) Reporting on page 284

4.15 Cell EM Analysis and ECO on page 290

4.16 Hierarchical Timing Context Analysis on page 315

4.17 Wire Variation Analysis on page 354

4.1 Base Delay Analysis

4.1.1 Overview on page 50

4.1.2 Base Delay Analysis Flow on page 50

4.1.3 Limitations of Traditional Delay Calculators on page 53

4.1.4 Performing Base Delay Analysis on page 55

4.1.5 Base Delay Reporting on page 60

4.1.1 Overview

Tempus enables you to perform fast and precise signal integrity-aware delay calculations for cell-based designs. The software combines signal integrity (SI) analysis with timing analysis to check for functional failures due to SI glitch and performs accurate timing calculations (that account for both the SI and IR-drop effects). Tempus utilizes the multi-threaded circuit simulation methods to deliver accuracy, capacity, and performance needed for nanometer designs.

This chapter describes the delay analysis flow and reporting.

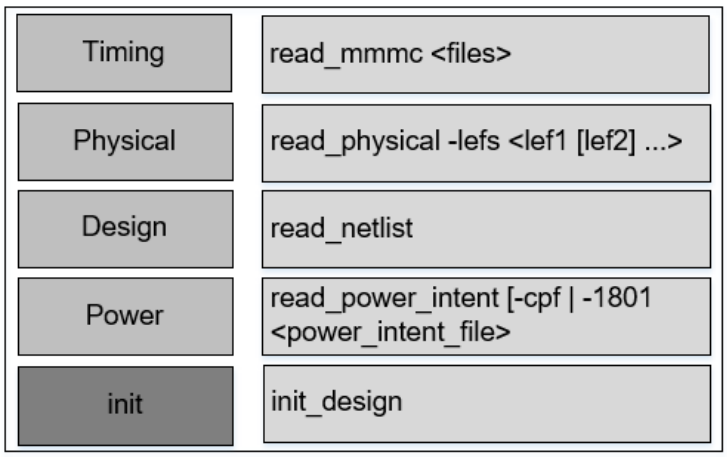
4.1.2 Base Delay Analysis Flow

The base delay analysis flow is described below:

Initialization Flow - MMMC Mode

The following figure shows the initialization flow in MMMC mode.

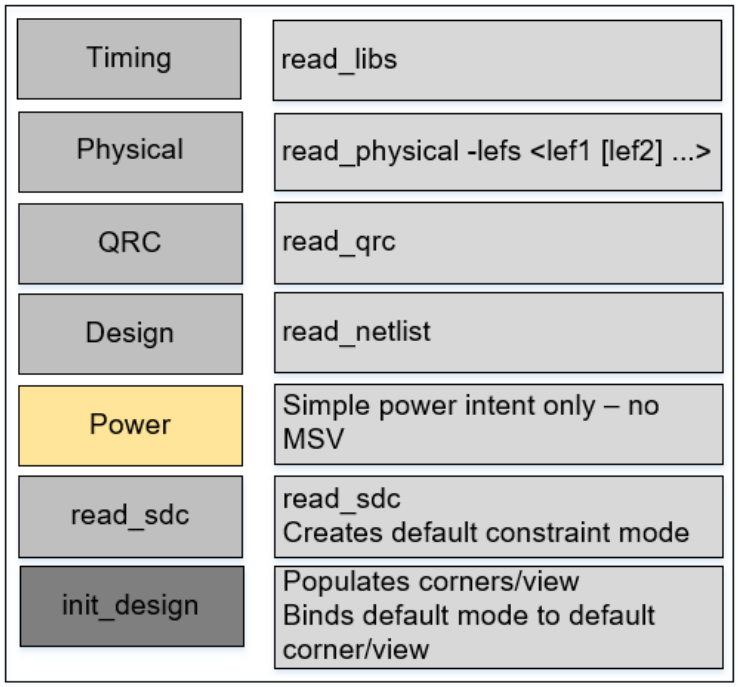
Figure 4-1 Base Delay Analysis Initialization Flow - MMMC Mode



Non-MMMC Initialization Flow

The following figure shows the initialization flow in non-MMMC mode.

Figure 4-2 Base Delay Analysis Initialization Flow - Non-MMMC Mode



Sample Base Delay Calculation Script

You can use the following script to calculate base delay:

1. Specify the design size:

   ```
   set_db -process 28
   ```

2. Load the design data. Follow the steps given below.

   - Load the timing libraries (.lib):

     ```
     read_libs
     ```

   - Schedule reading of structural gate-level verilog netlist:

     ```
     read_netlist dma_mac.v
     ```

   - Set the top module name, and check for consistency between the Verilog netlist and timing libraries:

     ```
     init_design
     ```

   - Load the timing constraints:

     ```
     read_sdc constraints.sdc
     ```

   - Load the cell parasitics:

```
read_spef cell.spef.gz
```

3. Generate the timing report:

   ```
   report_timing
   ```

4. Report base delay calculation details for the arcs:

   ```
   report_delay_calculation -from inst1/A -to inst1/Y
   ```

Base Delay Analysis - Inputs and Outputs

Inputs

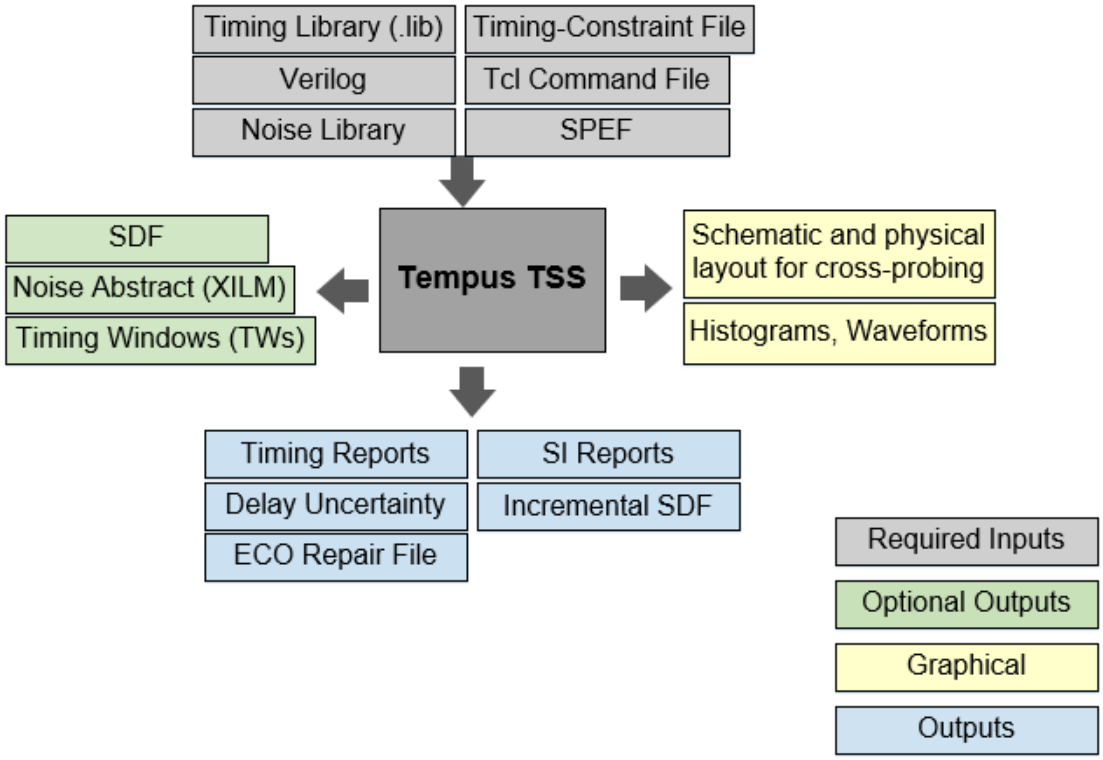
- Timing Library (CCS or ECSM preferred): Contains the timing data.
- Netlist: Specifies a circuit netlist in Verilog.
- SPEF: Specifies the RC parasitics information in SPEF format.
- Command File (Tcl): Contains a series of Tcl commands.
- Noise Library (CCS-N, ECSM-SI, or cdB) (Optional input): Contains characterized noise data.
- Timing Constraints: Specifies the timing constraints, including clock propagation, in a SDC file.

Outputs

- Timing Reports: Contains details about the critical paths in the design.
- Delay Report: Contains details about delay changes - with or without crosstalk.
- SDF File: Contains the negative and positive delay changes caused by crosstalk. The full SDF file contains details about the absolute delays for all cells and interconnects, including crosstalk impact.
- Noise Abstracts: Specifies an XILM abstract noise model. An XILM model is a detailed noise abstracts that can be used to perform hierarchical timing and noise analysis. This model contains cells that belong to block interface logic, and internal cells/nets that are coupled to the interface logic.
- Noise Report: Contains details about the glitch noise on nodes.

The inputs and outputs are shown in the diagram below.

Figure 4-3 Inputs and Outputs required for Base and SI Analysis

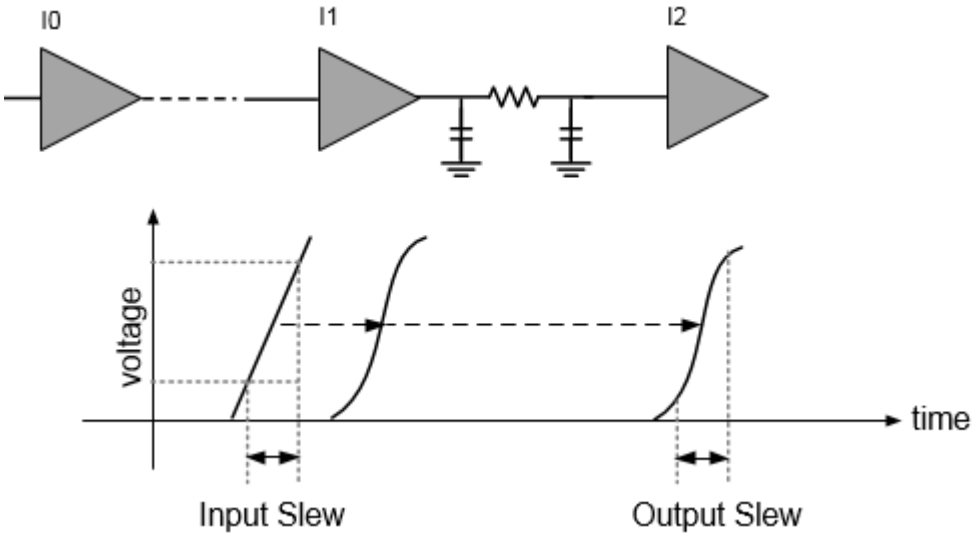


4.1.3 Limitations of Traditional Delay Calculators

Traditional delay calculators use delay as a function of the input slew and output load. With traditional delay calculators, a single linear slew value is used as the input to analyze a stage. This methodology cannot produce the desired accuracy that new technologies demand.

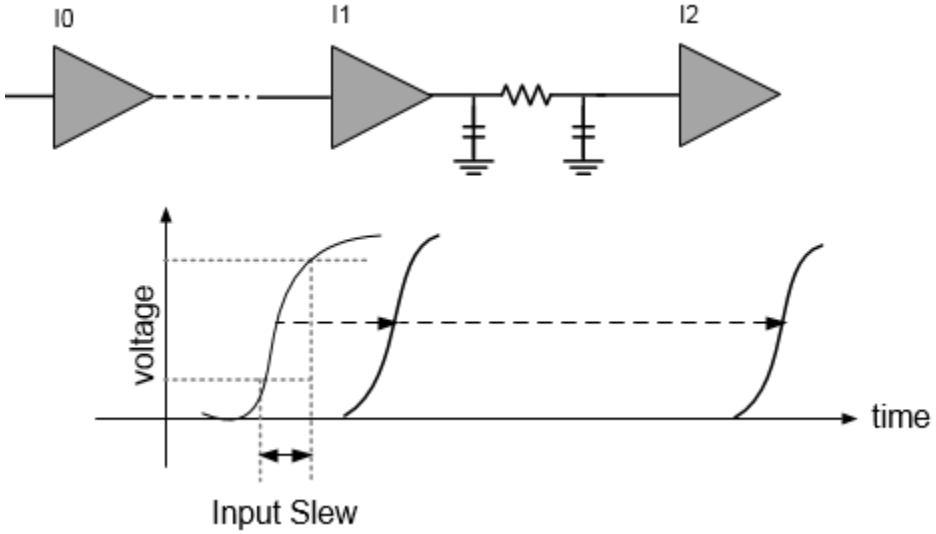
This is illustrated in the figure below.

Figure 4-4 Traditional Delay Calculation using Delay as function of slew and load



The advanced technologies (28nm and below) require waveform-based delay calculators to accurately calculate the delays based on waveforms. The waveform-based delay calculators use real waveforms as input to analyze a stage, as shown in the figure below.

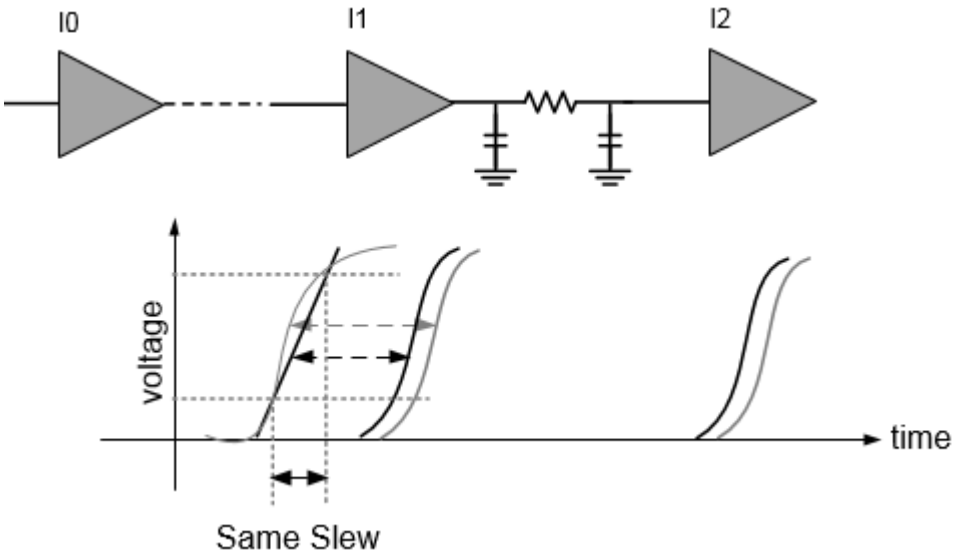
Figure 4-5 Ideal Waveform based Delay Calculation



There are several shortcomings when you use traditional delay calculators. Some of these are:

- Traditional delay calculators use single slew to calculate stage delays. This may not produce accurate results.
- Different waveforms with the same slew value produce the same delays.

Figure 4-6 Shortcomings of Traditional Delay Calculation



In the above figure, the grey waveforms indicate the input waveforms in an ideal case. The black lines indicate the linear input slew values and slew waveforms, respectively.

Here, the linear input slew value and the input waveforms use the same slew, however, the delay numbers are different. The long curve of the input waveform (grey) can produce larger delays as compared to the one produced with the linear input slew (black). Also, the measurement point shift can contribute to delay inaccuracy.

4.1.4 Performing Base Delay Analysis

To overcome the shortcomings of Traditional Delay Calculators, Tempus provides two different approaches for performing delay calculations. These are described below:

- Equivalent Waveform Model (EWM) on page 56
- Waveform Propagation on page 57

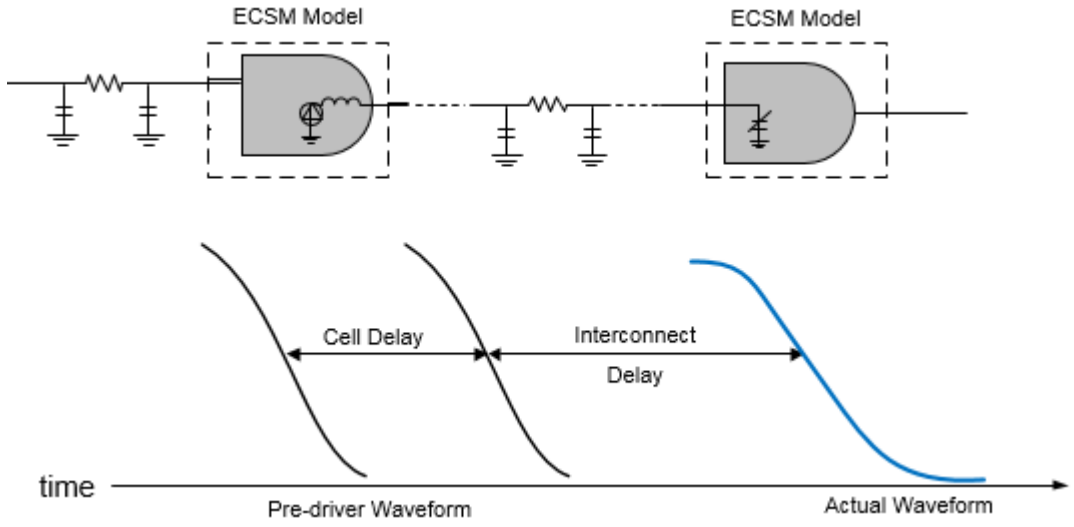
Equivalent Waveform Model (EWM)

To achieve accuracy, the waveform shapes are required to be included during delay calculations. The equivalent waveform model (EWM) computes equivalent receiver output based on the input waveform shapes and adjusts the interconnect delay accordingly. The adjustment in delay compensates for any inaccuracies that delay calculation might cause in the next stage due to lack of waveform shape information. The EWM approach provides a technique for producing higher accuracy results when compared to Spice.

When a stage is analyzed during delay calculation, a pre-defined waveform from the library (based on single input slew value) is used as a stimulus. When the EWM mode is not enabled, the input slew is measured from the actual waveform computed during the previous stage analysis. This may have entirely different characteristics compared to the pre-defined waveform used in the current stage. As a result, the delay impact due to waveform shape differences may be affected.

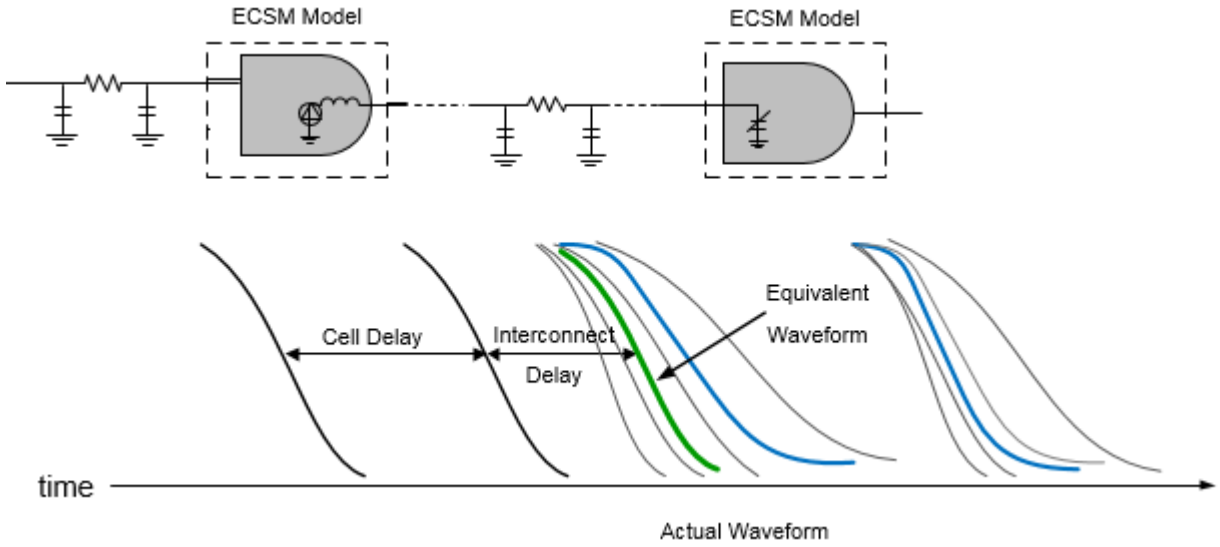
The following figure illustrates delay calculation without EWM.

Figure 4-7 Delay Calculation without EWM



The following figure illustrates delay calculation with EWM.

Figure 4-8 Delay Calculation with EWM



When EWM is enabled, the software computes the delay impact of waveform shapes on receiver cells, and computes the delay impact - thus providing an overall improvement in path delay accuracy. When EWM is enabled in SI analysis, the software provides delay adjustments based on the receiver noise response to a noisy transition. This helps to reduce the SI pessimism that will be reported if the total delay is measured on noisy waveforms at the receiver input.

Use Model

Equivalent waveform model can be enabled by using the following command:

```
set_db delaycal_equivalent_waveform_model no propagation
```

Waveform Propagation

The waveform shapes have a significant impact on delay calculations. Traditionally, delay calculation uses a single pre-driver waveform for specific slew value at the cell input to compute the response on the output of a cell.

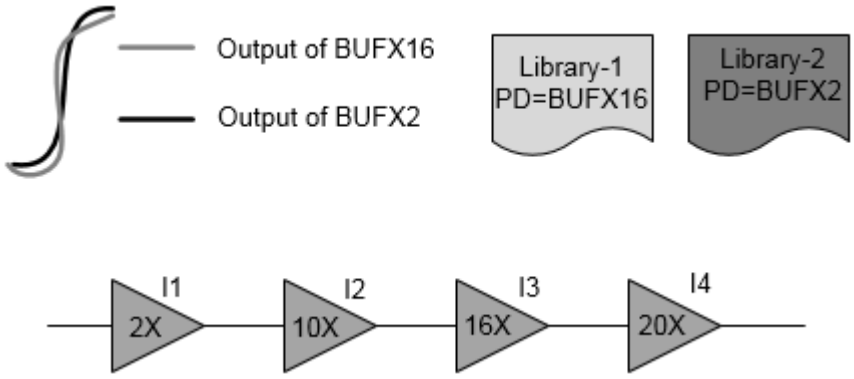
Consider two libraries characterized with pre-driver cells, BUFX16 and BUFX2. An accurate delay on all the instances driven by BUFX16 will be reported, when the library uses BUFX16 as a pre-driver cell. In this case, the instance I4 produces more accurate results as the input waveform matches with the pre-driver waveform. The accuracy of other cells is impacted due to a difference in results for the pre-driver cell versus the actual driving cell. To facilitate actual waveform propagation through a path, the waveform propagation feature stores the actual waveform instead of a single slew value at the input of each cell.

As shown in the figure below, both the BUFX16 slew (in grey) and BUFX2 slew (in black) have the same value but their waveform shapes are different. The same slew having different waveform shapes can produce different delays. The delay accuracy is a function of input waveform shapes, even if the slew values are same. If the input waveform shape is different from the input waveform used for cell characterization, the delay accuracy will be significantly affected.

The waveform propagation feature is supported in both graph- and path-based analysis modes.

The following figure illustrates waveform impact on delay.

Figure 4-9 Waveform impact on delay



Requirements for Waveform Propagation

The waveform propagation using ECSM models will be more accurate with additional receiver pincap points. Cadence recommends at least eight points. The last three points can be the lower slew threshold, delay measurement point, and upper slew threshold. The rest of the five points must be selected to represent the tail waveform accurately. It is also recommended to have actual pre-driver waveform in the ECSM libraries.

Use Model

Waveform propagation can be enabled by using the following command:

```
set_db delaycal_equivalent_waveform_model propagation
```

EWM-Only vs Waveform Propagation

EWM-Only

- Real waveform tail impact on the next stage is predicted and added to the current wire delay.
- The receiver cell is assumed to be the driver lumped load.

Waveform Propagation

- Real waveforms are stored and used as input for the next stage. The input waveform tail impact is used at the appropriate point.
- Unlike EWM-Only, the waveform propagation computes accurate impact of the tail as it uses distributed parasitics of wires.

EWM and Waveform Propagation - Impact on the Flow

The base delay flow is impacted in the following ways, when you use the equivalent waveform model or waveform propagation methodology:

- Enabling equivalent waveform modeling increases runtime by 10% - with no significant increase in the memory.
- Enabling waveform propagation increases runtime by 15%. There is an 8% increase in the memory in graph-based analysis (GBA) mode, and no significant memory changes take place in the path-based analysis (PBA) mode.

Normalized Driver Waveform in Library

The normalized driver waveform (pre-driver waveform) should be added to a library while characterizing so that delay can be computed accurately (as there can be different waveforms with the same slew). There are two types of pre-drivers – active and analytical. Cadence recommends that you use the active pre-driver. The active pre-driver waveform has a longer tail than the analytical pre-driver, and thus represents the real design scenario. In the absence of NDWs (normalized driver waveform) in a library, Tempus auto-generates analytic pre-driver waveforms.

An example of a normalized driver waveform in a library is given below:

```
normalized_driver_waveform (waveform_template_name) {
    driver_waveform_name : "PreDriver20.25:rise";
    index_1 ("0.00233, 0.01301, 0.05092, 0.1233, 0.2361, 0.3943, 0.6026");
    index_2 ("0, 0.083, 0.166, 0.25, 0.333, 0.417, 0.5, 0.583, 0.666, 0.75, 0.833,
0.917,1");
      values ( \
       "0, 0.00037, 0.0005041, 0.0006424, 0.0008009, 0.0009783, 0.001179, 0.00140,
0.00166, 0.001995 0.002375, 0.002833, 0.003389", \
        "0, 0.00220 0.0029643 0.003777, 0.004709, 0.005752, 0.006935, 0.00828374,
0.00988784, 0.0117336, 0.013969, 0.0166759, 0.0199283", \
        "0, 0.00862494, 0.0116002, 0.01473, 0.0184297, 0.0225117, 0.0271421,
```

```
0.0324164, 0.0386937, 0.0459165, 0.05466, 0.06571, 0.0779846", \
        "0, 0.0208867, 0.0280917, 0.0357977, 0.0446304, 0.0545157, 0.065729,
0.0785015, 0.0937029, 0.111194, 0.132378, 0.15803, 0.188852", \
        "0, 0.0399897, 0.0537844, 0.0685384, 0.0854496, 0.104376, 0.125845,
0.150299, 0.179404, 0.212893, 0.253452, 0.302566, 0.361577", \
```

Timing Library Requirement for Accurate Analysis for 16nm and Below

The ECSM/CCS library characterization for 16nm static timing analysis (STA) signoff meets the challenges of accurate timing analysis in 20nm. In order to meet accuracy demands for process nodes 16nm and below, the following are recommended:

- 8-piece pin capacitances in ECSM timing libraries
- 2-piece pin capacitance in CCS Timing libraries
- N -piece pin capacitance in CCS Timing libraries
- 2% - 98% ECSM waveform range

ECSM Libraries with 8-Piece Pin Capacitances

The 8-piece pin capacitance in the ECSM timing libraries are required to accurately model back miller current. Traditionally, the receiver pin capacitance in an ECSM library characterization is measured at the slew thresholds - that may be 30% to 70% of the VDD. As a result, the use of such thresholds in the ECSM libraries may result in some missing important data at the tail of the waveform. The 3-piece capacitance tables are extended to 8-piece tables for 20nm nodes to better capture the waveform distortions due to back miller current at the waveform tail. The selection of 8-piece pin capacitance is made such that the required 20nm waveform distortion information can be captured correctly. Since the waveform distortion happens mostly at the tail of waveforms, the pin-cap thresholds are selected so that there are more points on the tail.

4.1.5 Base Delay Reporting

The following commands can be used to generate reports on base delay:

- `report_timing`
- `report_delay_calculation`

Using the report_timing Command

You can use the `report_timing` command to report the base delays on a timing path. It is recommended to use the `report_timing -split_delay` parameter to produce a comprehensive report.

```
tempus > set_db timing_report_fields {hpin cell transition delay arrival}
tempus > report_timing -split_delay
Path 1: VIOLATED (-1.191 ns) Path Delay Check with Pin wdata_rw_o[150]
View: slow_corner_setup
              Group: default
Startpoint: (R) wdata_rw_i[150]
Clock: (R) CCLK
Endpoint: (R) wdata_rw_o[150]
Clock:
                  Capture      Launch
Path Delay:+      0.750        -
Src Latency:+     0.000        0.000
Net Latency:+     0.000 (I)    0.000 (I)
Arrival:=         0.750        0.000
Output Delay:-    0.000
Required Time:=   0.750
Launch Clock:=    0.000
Input Delay:+     1.200
Data Path:+       0.741
Slack:=           -1.191
#-------------------------------------------------------------------------------
# Pin                                  Cell            Trans     Delay   Arrival
#                                                       (ns)      (ns)    (ns)
#-------------------------------------------------------------------------------
wdata_rw_i[150]                     (arrival)         0.004     -      1.200
wdata_rw_i_150_ATibuf_i14079/A      BUFH_X13M_A12TS   0.004   0.002    1.202
wdata_rw_i_150_ATibuf_i14079/Y      BUFH_X13M_A12TS   0.024   0.028    1.231
wdata_rw_o_150_ATrbuf_i25015/A      BUFH_X11M_A12TS   0.046   0.025    1.255
wdata_rw_o_150_ATrbuf_i25015/Y      BUFH_X11M_A12TS   0.023   0.047    1.303
wdata_rw_o_150_ATrbuf_i1012019/A    BUFH_X9M_A12TS    0.029   0.009    1.312
wdata_rw_o_150_ATrbuf_i1012019/Y    BUFH_X9M_A12TS    0.030   0.046    1.358
wdata_rw_o_150_ATrbuf_i22662/A      BUFH_X13M_A12TS   0.094   0.053    1.411
wdata_rw_o_150_ATrbuf_i22662/Y      BUFH_X13M_A12TS   0.029   0.073    1.484
wdata_rw_o_150_ATmdrc_i17414/A      BUF_X9M_A12TS     0.063   0.037    1.521
wdata_rw_o_150_ATmdrc_i17414/Y      BUF_X9M_A12TS     0.032   0.071    1.592
wdata_rw_o_150_ATrbuf_i1013579/A    BUFH_X16M_A12TS   0.036   0.007    1.599
wdata_rw_o_150_ATrbuf_i1013579/Y    BUFH_X16M_A12TS   0.015   0.040    1.639
wdata_rw_o_150_ATrbuf_i25739/A      BUFH_X13M_A12TS   0.087   0.060    1.699
wdata_rw_o_150_ATrbuf_i25739/Y      BUFH_X13M_A12TS   0.020   0.064    1.763
wdata_rw_o_150_ATrbuf_i1013616/A    BUFH_X9M_A12TS    0.021   0.004    1.768
wdata_rw_o_150_ATrbuf_i1013616/Y    BUFH_X9M_A12TS    0.032   0.040    1.808
wdata_rw_o_150_ATdrc_i28560/A       BUFH_X13M_A12TS   0.060   0.026    1.834
wdata_rw_o_150_ATdrc_i28560/Y       BUFH_X13M_A12TS   0.023   0.054    1.888
wdata_rw_o_150_ATobuf_i14701/A      BUFH_X3P5M_A12TS  0.034   0.013    1.901
wdata_rw_o_150_ATobuf_i14701/Y      BUFH_X3P5M_A12TS  0.014   0.040    1.941
wdata_rw_o[150]                     ud_lpu_2x         0.014   0.000    1.941
#-------------------------------------------------------------------------------
```

Using the report_delay_calculation Command

You can use the `report_delay_calculation` command to report the delay calculation information for a cell or net timing arc.

```
tempus > report_delay_calculation -from wdata_rw_i_150_ATibuf_i14079/Y  -to
wdata_rw_o_150_ATrbuf_i25015/A

From pin     : wdata_rw_i_150_ATibuf_i14079/Y
Cell         : BUFH_X13M_A12TS_C35
Library      : ff3sigma_0p935v_125c
To pin       : wdata_rw_o_150_ATrbuf_i25015/A
Cell         : BUFH_X11M_A12TS_C35
Library      : ff3sigma_0p935v_125c
Base or SI   : SI delay
Delay type   : net
-------------------------------------------------------------
RC Summary for net wdata_rw_o_150_ATobuf_14701
-------------------------------------------------------------
Number of capacitance : 7
Net capacitance       : 0.029770 pF
Total rise capacitance: 0.037020 pF
Total fall capacitance: 0.037104 pF
Number of resistance  : 8
Total resistance      : 1614.910156 Ohm
-------------------------------------------------------------
                          Rise        Fall
-------------------------------------------------------------
Incr Net Delay          : 0.009600 ns 0.009900 ns
Net delay               : 0.033400 ns 0.033400 ns
From pin transition time: 0.005200 ns 0.004900 ns
To pin transition time  : 0.057900 ns 0.058300 ns
-------------------------------------------------------------
```

4.2 Signal Integrity Delay and Glitch Analysis

4.2.1 Signal Integrity Delay and Glitch Analysis - An Overview on page 64

4.2.2 Understanding Attacker and Victim Nets on page 64

4.2.3 SI Delay Analysis - An Overview on page 64

4.2.4 Signal Integrity Delay Analysis Flow on page 66

4.2.5 Performing Signal Integrity Delay Analysis on page 67

4.2.6 SI Delay Reporting on page 94

4.2.7 SI Glitch Analysis - An Overview on page 102

4.2.8 SI Glitch Analysis Flow on page 103

4.2.9 Understanding SI Glitch Analysis on page 105

4.2.10 Running Detailed SI Glitch Analysis on page 110

4.2.11 SI Glitch Reporting on page 112

4.2.12 Overshoot-Undershoot Glitch Analysis - An Overview on page 119

4.2.13 Running Overshoot-Undershoot Glitch Analysis - Sample Script on page 123

4.2.1 Signal Integrity Delay and Glitch Analysis - An Overview

Signal Integrity (SI) can impact the delay or introduce a glitch on a given net due to the switching of nets that may be lying in close proximity. Tempus performs SI analysis by analyzing the delay and slew changes of each switching signal in the presence of crosstalk noise. The delay and slew changes, due to crosstalk noise, are then used to determine the worst-case minimum or worst-case maximum path delays in the design. The software also checks for functional failures due to crosstalk glitch noise and reports the failing nets.

Tempus performs the following signal integrity analysis operations:

- SI Delay Analysis - An Overview
- SI Glitch Analysis Flow

To perform signal integrity analysis, it is important to understand the role of attackers and victim nets. These are discussed in the following section.

4.2.2 Understanding Attacker and Victim Nets

A victim net receives undesirable cross-coupling effects from any neighboring net. An attacker is a net that causes undesirable cross-coupling effects on a victim net. An **attacker** net can be a victim net and a victim net can also be an attacker net. The net which is analyzed for SI effects is called a **victim net.**

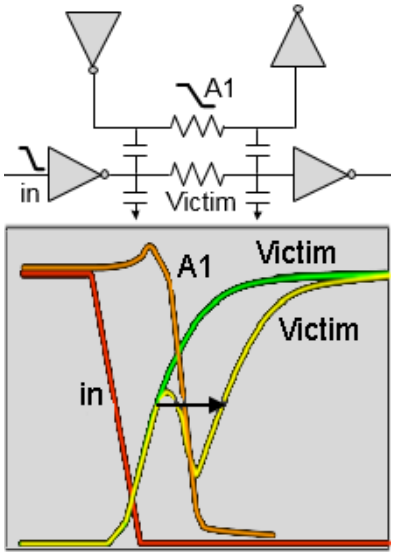
- The impact of an attacker net on a victim net depends on several factors:
- The amount of cross-coupled capacitance
- The relative time and the transition rate of the signal transitions
- The switching directions (rising, falling)
- The combination of effects from multiple attacker nets on a single victim net

4.2.3 SI Delay Analysis - An Overview

SI can increase or decrease signal delay, which can, in turn cause setup or hold failures.

Consider that attacker A1 switches in the opposite direction to the victim, as shown in the figure below, then there can be a potential increase in the victim delay. Also, notice the non-linear waveform shape that is caused by the glitch, as shown in the figure below.

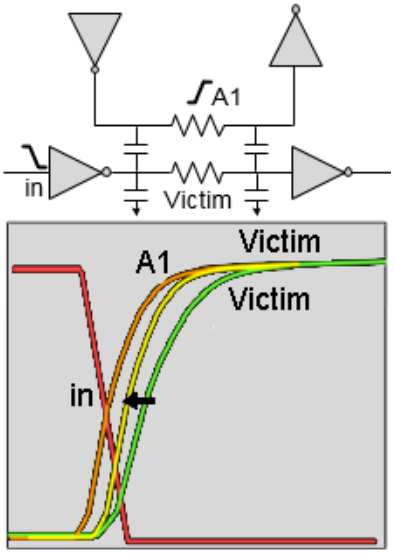
Figure 4-10 SI increases delay



SI can also decrease the delay and cause hold time failures. If both the attacker and victim are switching in the same direction, as shown in the figure below, then there is a decrease in the victim net delay. If this occurs on a critical minimum delay path, it can lead to hold violations. In other words, if a victim net transitions sooner than it was intended to, then it may cause hold violations

The following figure illustrates decreased delay.

Figure 4-11 SI decreases delay

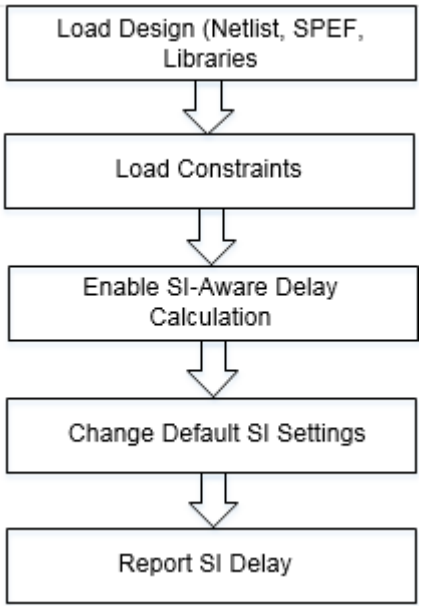


Tempus calculates the worst-case change in delay value due to cross-coupled attackers and reports the change in delay in the timing and SI reports.

4.2.4 Signal Integrity Delay Analysis Flow

The SI delay analysis flow is given below:

Figure 4-12 Tempus SI Delay Analysis Flow



Sample SI Delay Calculation Script

You can use the following script to calculate base and SI delay:

1. Specify the design size:

   ```
   set_db -process 28
   ```

2. Load the design data. Follow the steps given below:

   - Load the timing libraries (.lib) and characterized data (.cdB):

     ```
     read_libs  typ.lib -cdb typ.cdB
     ```

   - Schedule reading of a structural, gate-level verilog netlist:

     ```
     read_netlist  dma_mac.v
     ```

   - Set the top module name, and check for consistency between Verilog netlist and timing libraries:

```
init_design dma_mac
```

- Load the timing constraints:

  ```
  read_sdc constraints.sdc
  ```

- Load the cell parasitics:

  ```
  read_spef cell.spef.gz
  ```

3. Set the individual attacker threshold:

   ```
   set_db si_individual_aggressor_threshold 0.015
   ```

4. Perform 3 SI iterations:

   ```
   set_db si_num_iteration 3
   ```

5. Separate delta delay on data:

   ```
   set_db si_delay_separate_on_data true si_delay_enable_report true
   ```

6. Enable SI-aware delay calculation:

   ```
   set_db delaycal_enable_si true
   ```

7. Report the timing:

   ```
   report_timing
   ```

8. Report details of SI delay calculation for an arc:

   ```
   report_delay_calculation -si -from inst1/A -to inst1/Y
   ```

9. Report delay uncertainty due to SI:

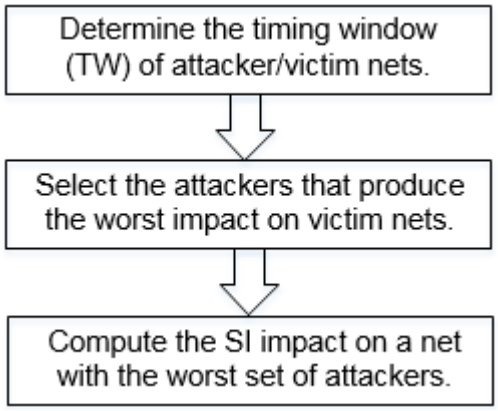
   ```
   report_noise -delay {min | max}
   ```

4.2.5 Performing Signal Integrity Delay Analysis

Tempus performs SI analysis in conjunction with base timing analysis. The SI delay analysis calculates the change in delay due to coupling noise acting both with (min) and against (max) a transitioning signal and outputs a delay uncertainty report in text format. SI delay analysis uses a fast transistor-level transient **Timing Windows (TW) Illustration** simulation engine to ensure the highest accuracy. This is done by accounting for non-linear waveforms created due to the coupling noise and attributing delay changes to the victim.

Tempus performs the following operations during SI delay analysis:

Figure 4-13 Tempus SI Delay Analysis Operations



These are explained in the sections below.

- Determining Timing Windows on page 68
- Selection of Attackers on page 73

Determining Timing Windows

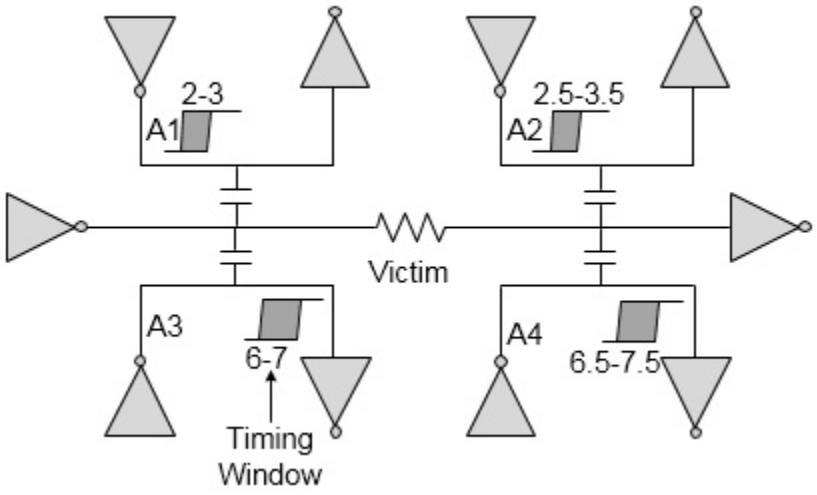
Tempus can generate or read in the timing window information (using the `read_twf` command) to determine which signals can (or cannot) switch simultaneously. The slews are used to switch attacking signals. Timing windows represent the range of time during which a net can transition. They are used to decide when it is possible for multiple attackers to combine.

Attacker timing windows are exhaustively checked for the worst-case allowable combination for the final combined simulation.

- The worst case is determined by each attacker's glitch noise contribution.
- The victim's timing windows are also checked for SI delay analysis.
- The attacker slew affects the magnitude of the noise peak/incremental delay.

A more accurate slew reduces the noise peak, if the transition time is greater than the default fast slew.

Figure 4-14 Timing Windows (TW) Illustration



In the above figure, the ranges represent the timing windows for the respective nets. For attacker A1, TW range is 2-3, that is, the time interval in which net A1 can switch. The attackers A1 and A2 can be combined to produce a bigger SI impact since their TWs are overlapping. Similarly, attackers A3 and A4 can be combined since TWs are overlapping.

Now, depending on which one of the two combinations (A1-A2, or A3-A4) produces worst SI impact, one of the combinations will be selected by the software for SI analysis.

Timing Windows Iterations

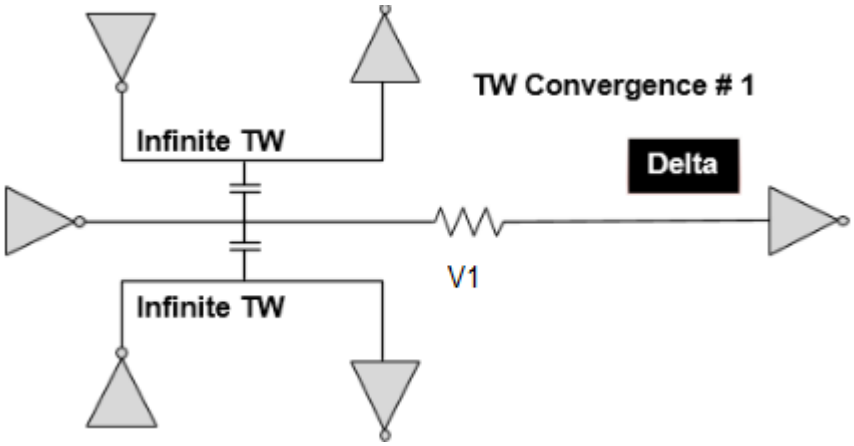
The SI effect on delay depends on timing windows and slew, whereas the timing window and slew depends on the SI effect of delay. Therefore, the timing windows impact SI delay and SI delay impacts timing windows.

Hence, an iterative approach is required to accurately compute the timing windows affected due to SI so that Tempus can accurately calculate the SI delay and glitch.

There are two primary approaches to timing window iteration. The default approach starts with an infinite timing window and iterates while shrinking the timing window width. The second approach starts with a nominal or noiseless timing window and iterates while growing the timing window width.

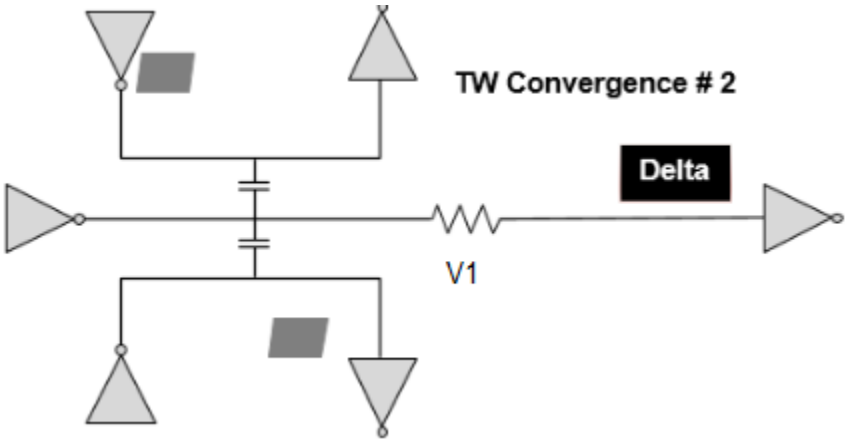
The following figure shows the default approach to start with infinite timing windows and calculates the initial delta delays.

Figure 4-15 Timing Windows (TW) 1st Iteration



In the second iteration, as shown in figure below, the delta delays are used to calculate new arrival times, and therefore new timing windows. These new timing windows are used to calculate new delta delays. The new delta delays are smaller because the timing windows are more precise (smaller) compared to infinite timing windows.

Figure 4-16 Timing Windows (TW) 2nd Iteration



By default, Tempus performs two iterations. You can change the number of Iterations by using the following command:

`set_db si_num_iteration` *`Number`*

All the nets are not reselected for analysis in each timing window iteration. Tempus supports the following methods of re-selection.

- Slack based
- Delta delay based

- Xcap ratio based

These reselection criteria can be specified by using the command:

`set_db si_reselection {{delta_delay | slack | xcap_ratio} (default is slack)`

The default approach of starting with infinite TW will incur certain pessimism as in the initial iteration, the nets will be having much bigger computed noise as compared to the actual scenario. This pessimism will be reduced in the subsequent iterations as the nets are reselected and their timing windows are shrunk. But reselecting all the nets will have a huge run time impact. Therefore, the above mentioned criteria determines which nets are to be reselected for further iterations. Understanding the trade-off between the runtime and pessimism reduction, you can appropriately decide how the above criteria needs to be deployed in the design.

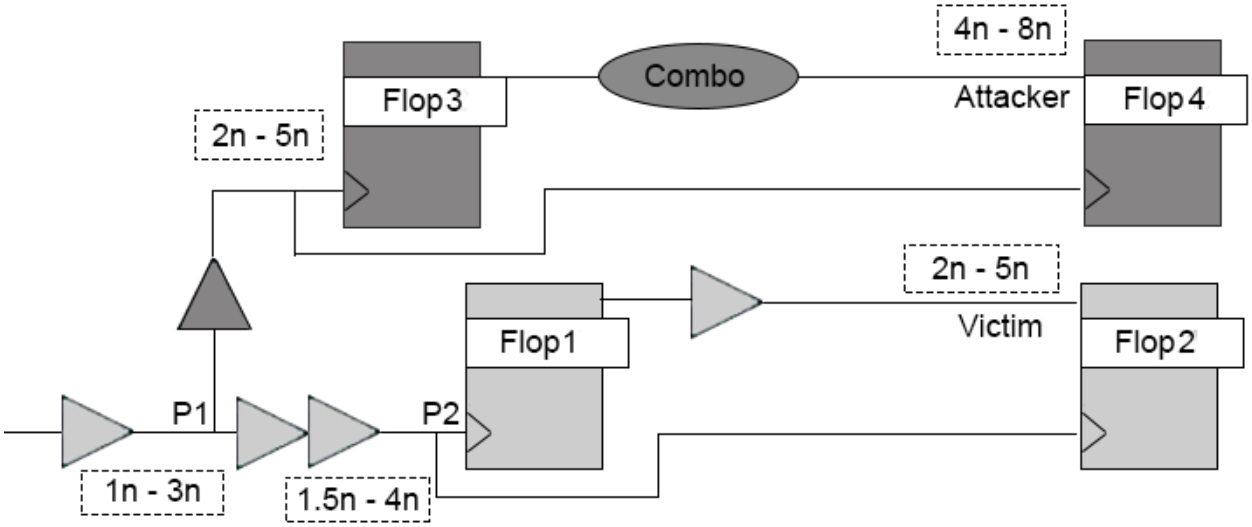
Also, it is important to understand that reselecting a particular net does not ensure that all pessimism is removed from SI analysis on that net. For instance, if the timing critical nets are being reselected, the attackers of the critical victim nets may not necessarily be critical, so the timing windows on the attackers may be pessimistic.

Timing Window CPPR

Timing windows of nets are computed irrespective of their relative positioning. There can be some pessimism in analysis when the victim/attacker share common clock path.

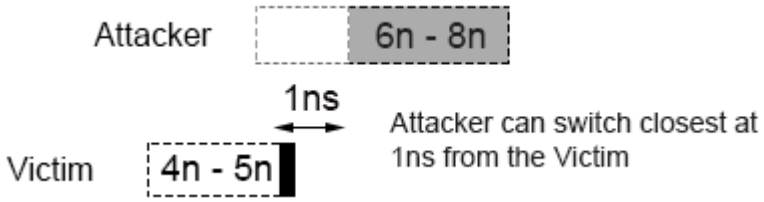
Attacker attacks a victim with its full strength if it switches at the same time as the victim, and the impact of the attacker on the victim declines as the distance between switching times increases. The computation of closest switching time based on these arrival times can be pessimistic if both attacker and victim share the same clock path.

Figure 4-17 Timing Windows on CPPR Mode



In the example above, for path from Flop1 to Flop2, Attacker's TW (4n – 8n) is overlapping with Victim's TW (2n – 5n). The clock arrival (at point P1), which is causing victim to reach at 5ns is 3ns. As both victim and attacker are sharing the same launch clock path till P1, they both will be launched with the same clock arrival time till the point. So corresponding to the arrival time of 3ns (at P1) the attacker can switch earliest by 6ns (and not 4ns). Therefore, the actual positioning of the attacker and victim will be, as shown in the figure below. The attacker can only attack from a shortest distance of 1ns from the victim switching time.

Figure 4-18 Timing window of Attacker and Victim not Overlapping



You can remove pessimism by enabling the following attribute:

```
set_db timing_enable_timing_window_pessimism_removal true
```

The adjustment in attacker TWs accounts for the difference in min-max delay on the top common clock path due to the following factors:

Difference in base arrival of the top clock path due to early/late derates.

Min/Max SI impact on common clock path (adjusted only during hold analysis).

Selection of Attackers

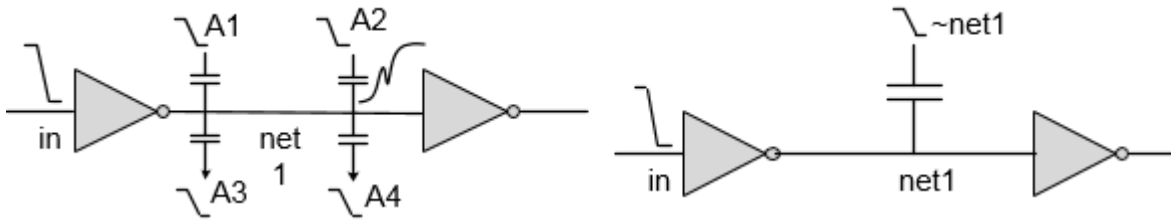
After determining the timing windows of attacker/victim nets, Tempus selects the attackers that produces the worst impact on the victim net. Tempus controls the attacker's selection based on the following factors:

- Strength of Attacker on page 73
- Attacker Selection Based on Alignment on page 75
- Attacker Selection Based on Logical Correlation on page 87

Strength of Attacker

The strength of an attacker is the amount of glitch it produces at victim's receiver input and it depends on the coupling capacitance between attackers and victims. With new processes (smaller geometries), the ratio of C_{cpl}/C_{tot} is increasing. Some victims can have 100s or 1000s of attackers and many of these attackers can be small ones. SI analysis run time is a strong function of the number of attackers. Higher numbers of attackers can result in high analysis run time. Tempus provides a better way of handling small attackers by combining these small attackers together. Tempus models many small attackers on a victim net and replaces them with an imaginary net, called a virtual attacker.

Figure 4-19 Virtual Attackers Modeling



In the figure above, the small attackers, namely A1, A2, a3, and a4, are combined together to form a virtual attacker ~net1.

Tempus identifies an attacker as an individual attacker if the glitch contribution from this attacker is more than the threshold value specified with the `set_db si_individual_aggressor_threshold` *`value`* command. If the peak of this glitch on the victim net exceeds the specified threshold value, then the attacker is significant and is individually reported in the constituents section of the SI report. You can use the `si_individual_aggressor_clock_threshold` attribute to vary small attacker threshold for victim nets lying on a clock path.

If the glitch contribution of an attacker is less than the threshold value defined with the `si_individual_aggressor_threshold` `set_db` attribute, then this attacker becomes eligible for the virtual attacker.

The two main characteristics of a virtual attacker are: coupling capacitance and the voltage source waveform used as a transition on the virtual attacker. You can use the `set_db si_accumulated_small_aggressor_mode` attribute to create and control the characteristics of the virtual attacker. Tempus provides the following two methodologies for creating virtual attackers:

- Current Matching-Based Methodology
- Coupling Capacitance Matching-Based Methodology

Note: Both the methodologies account for the low probability of all small attackers switching at once in order to reduce the pessimism on noise and SI delay analysis.

Current Matching-Based Methodology

To create virtual attackers using this methodology, you can set the following command:

```
set_db si_accumulated_small_aggressor_mode current
```

Tempus creates virtual attackers to match both the coupling capacitance and the noise current. The PWL waveforms are generated in such a way that current induced by the transition matches the total current induced by all virtual attacker components.

This methodology differs from the coupling capacitance methodology as follows:

- Keeps several of the top virtual attacker components and calculates a PWL waveform for each analysis type using TW filtering.
- Improves on the statistical method (3-sigma method) by accounting for the probability of switching.

Coupling Capacitance Matching-Based Methodology

To create virtual attackers using this methodology, you can set the following command:

```
set_db si_accumulated_small_aggressor_mode cap
```

Note: This is the default methodology for all process nodes above 45nm. The 45nm and below default mode is current.

Tempus creates virtual attackers that model the combined coupling capacitance effects of small attackers on a victim net. Every victim net can potentially have a virtual attacker. Tempus assigns a name to each virtual attacker based on the name of the victim net. For example, if the victim net name is ABC, then the virtual attacker net name will be ~ABC. The

pessimism of using the virtual attacker can be reduced by adjusting the value of the `si_accumulated_small_aggressor_factor` between 0 and 1.

`set_db si_accumulated_small_aggressor_factor` *`value`*

To recognize those attackers that may have a significant impact on a victim net, Tempus performs a quick and conservative estimation that takes into account the resistance and capacitance (RC) characteristics of the attacker and the victim net, as well as the worst-case drive strength and the load of the victim net. If the estimated coupling capacitance effect from an attacker is greater than five percent of VDD, the attacker is explicitly reported as an attacker constituent of that particular victim net. It is typical for a victim net to have as many as three or four attackers of this type. In general, attackers are usually very small and fall under this five percent threshold.

You can change the virtual attacker glitch threshold using the `set_db si_accumulated_small_aggressor_threshold` attribute. The default value of this parameter is 10.01 percent of VDD. For example, to disable virtual attacker you can use a threshold value greater than 1 (default), by using the following command:

```
set_db si_accumulated_small_aggressor_threshold 1.01
```

The virtual attacker provides time and memory savings, but sacrifices some accuracy, reporting detail, and control on slew and timing window information. Because the virtual attacker is used only for small attackers, these sacrifices should have a minimal impact on results.

Attacker Selection Based on Alignment

Tempus supports the following attacker alignment modes that allow you to select attackers based on their alignment with victim nets:

- Path Mode
- Path Overlap Mode
- Timing-Aware Edge Mode
- Input Arrival Mode

The attacker alignment mode can be set by using the following command:

`set_db si_aggressor_alignment {path | path_overlap | timing_aware_edge | input_arrival}`

The default mode is `input_arrival`.

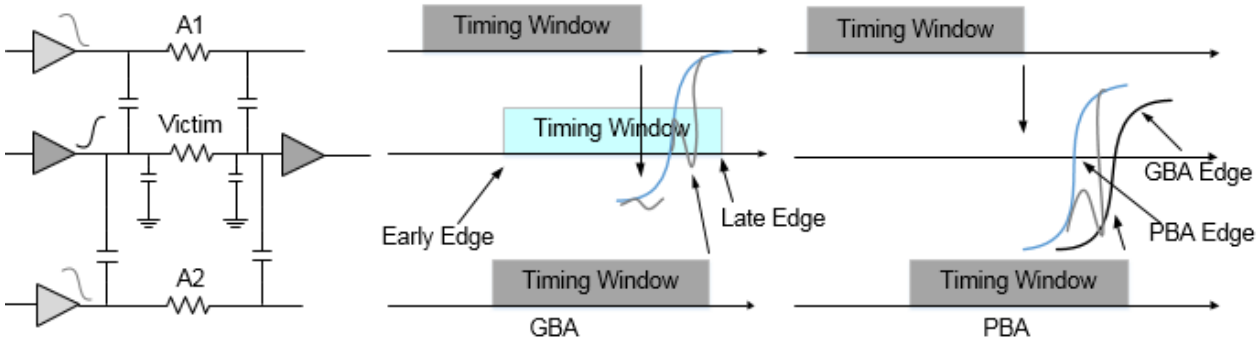
Path Mode

In path alignment mode:

- Tempus uses the victim's worst arrival edge (as shown in the figure) when determining which attackers can align with it.
- SI impact is computed according to the latest/earliest arrival edge of the victim for maximum/minimum analysis.
- This mode yields a more realistic analysis.

The path-based alignment mode annotates each attacker's impact on victim transition where attacker timing window edge aligns. The following example shows that in GBA mode Attacker 2 timing window edge is close to the delay measurement point, thus resulting in maximum impact.

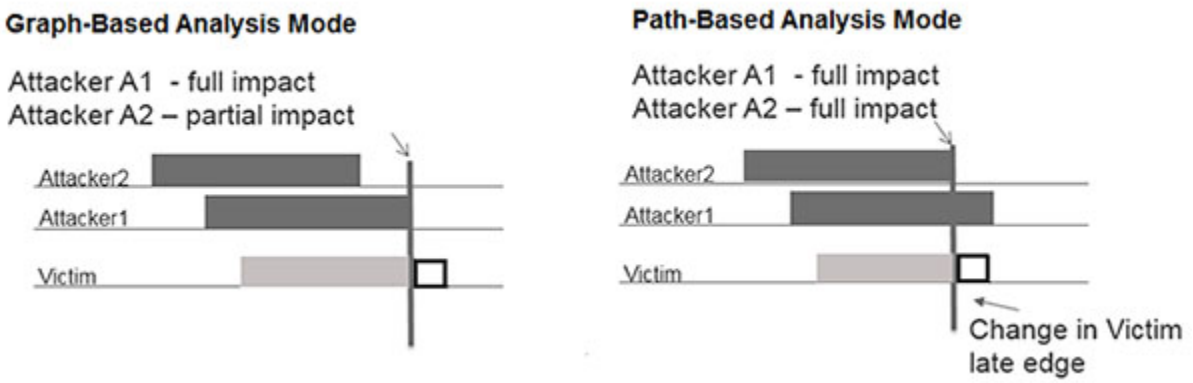
Figure 4-20 Path Alignment Mode



The path-based alignment is more realistic analysis and is less pessimistic than the other two modes. However, GBA is not guaranteed to bound PBA delays. In this example, the victim arrival time gets faster (move left) due to retiming of victim timing in PBA. With retimed arrival time of victim, both the attacker timing window edges now align at the delay measurement point, thus PBA sees a larger cross-talk impact than GBA.

The flow in path mode is illustrated in the figure below.

Figure 4-21 Attackers alignment in path mode



In the above figure, for paths that are based on the last arrival edge of victim net, only A1 will be selected for alignment in path mode.

Path Overlap Mode

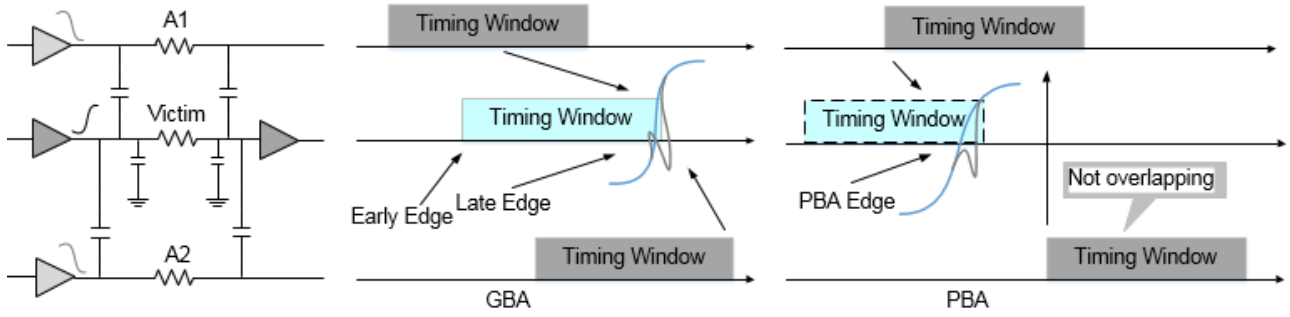
In path overlap mode, Tempus uses the full timing window of the victim, when determining which attackers can align with it.

The SI impact is calculated according to the victim edge that is available anywhere within the timing window where the maximum impact occurs.

The path overlap mode considers all the attackers which overlap anywhere on victim transition and annotate their combined impact at delay measurement point in both GBA and PBA. Due to this methodology, the computed SI impact based on path overlap is pessimistic. However, GBA is guaranteed to bound PBA analysis.

The following example illustrates path overlap alignment mode:

Figure 4-22 Path Overlap Alignment Mode

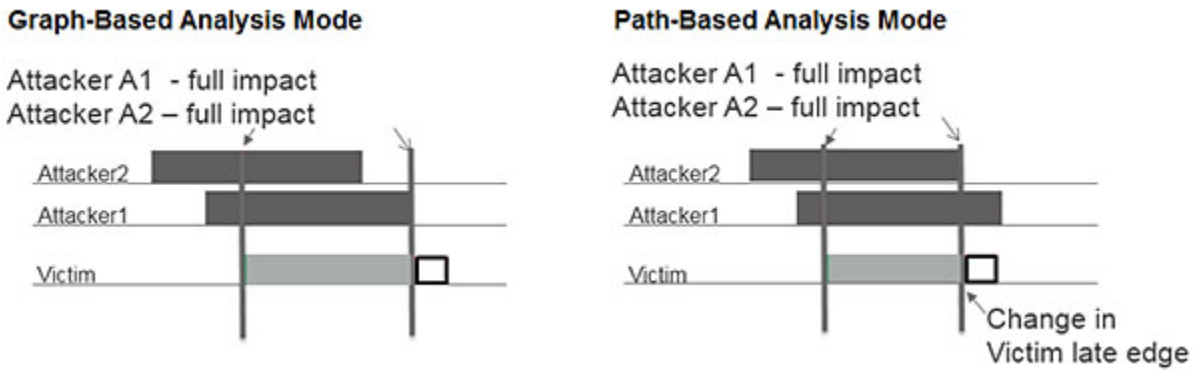


This mode is more conservative in cross-talk delay computation among the other available modes.

The PBA analysis does not change based on attacker alignment modes, that is, PBA analysis always annotates individual attacker impact on victim transition where ever attacker timing windows overlap in all the three modes.

The flow in path overlap mode is illustrated in the diagram below.

Figure 4-23 Attackers alignment in path overlap mode



Even if victim timing window changes, the attackers' selection will remain the same. This mode ensures that GBA bounds PBA delays.

In the figure above, consider a full timing window of a victim net - both attackers A1 and A2 will be considered for alignment with victim and hence both A1 and A2 are selected in the path overlap mode. This mode is more pessimistic.

In path-based analysis (PBA) mode, Tempus uses the specific edge for the victim that occurs for a specific path when determining the alignment of the attackers to the victim. It is possible

that the worst SI impact on a net occurs at a victim edge that is not the worst arrival edge for the victim, thus the PBA victim edge used inside the timing window for the victim can yield a worst-case SI result for a net. For more information, refer to section on Path-Based Analysis on page 194.

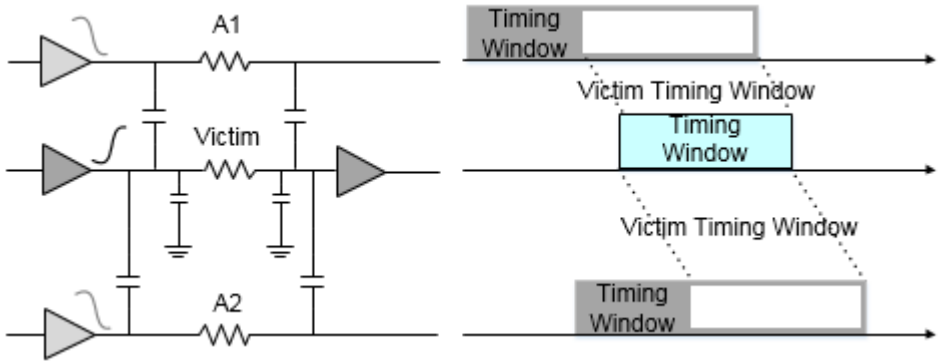
To ensure that GBA results bound PBA, you should use the path overlap alignment mode.

Timing-Aware Edge Mode

The timing-aware edge mode performs realistic analysis similar to path-based analysis. Each attacker's impact is annotated on victim transition where attacker timing window edge aligns with the victim. However, to ensure GBA bounds PBA analysis, the timing-aware edge mode handles some nets based on their timing properties in GBA analysis.

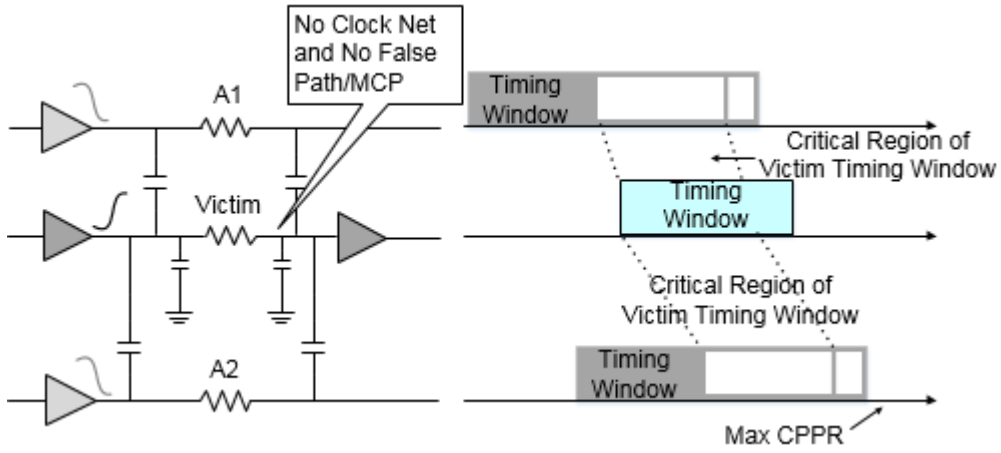
In the following example, if the victim net is a clock, or with exceptions like false paths or MCP, the attacker timing windows are extended by width of the victim timing window.

Figure 4-24 Timing-Aware Edge Alignment Mode - Illustration 1



If the victim net is not a clock, and has no exceptions like false path or MCP, then all the attacker timing windows are extended by a critical region of the victim timing window, in addition to the maximum CPPR, as shown below.

Figure 4-25 Timing-Aware Edge Alignment Mode - Illustration 2



During PBA analysis, the attacker timing windows are extended by the actual CPPR of the path, and no additional padding is required.

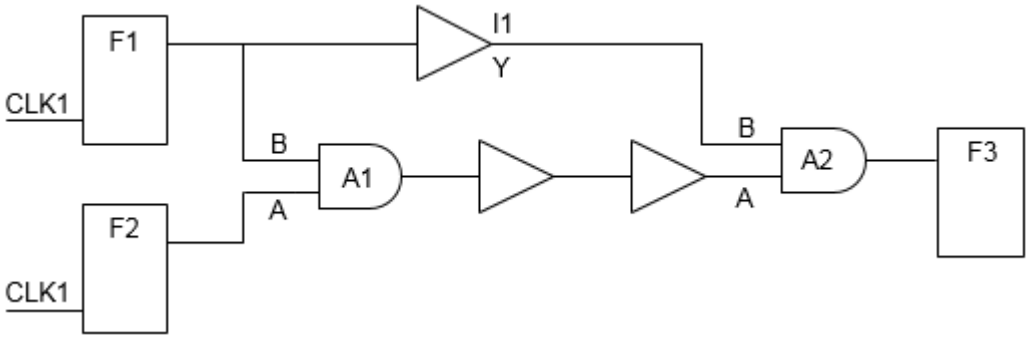
Clock and Exception Handling Timing-Aware Edge Mode

If the victim net is a clock net, and the attacker timing windows are extended by width of the victim timing window to ensure that an attacker overlaps with the victim arrival time. Any optimism on a clock net in GBA can result in large TNS optimism (in GBA) as compared to PBA, because many paths share the same clock path. To eliminate this situation, the clock nets are handled specially in timing-aware edge mode. The exceptions in certain scenarios can cause optimism in GBA in path-based alignment mode.

Consider the following example:

```
set_false_path -through A1/A -through A2/A
```

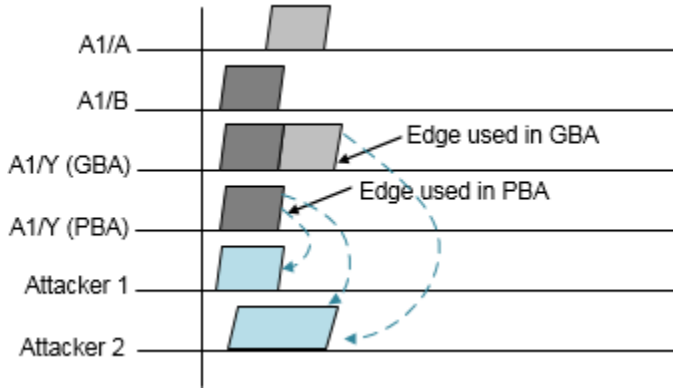
Figure 4-26 Timing-Aware Edge Mode - Example 1



In this example, a path going through A1/B->Y in GBA analysis uses late edge, which overlaps with attacker 2 only. However in PBA mode, a path going through A1/B->Y overlaps

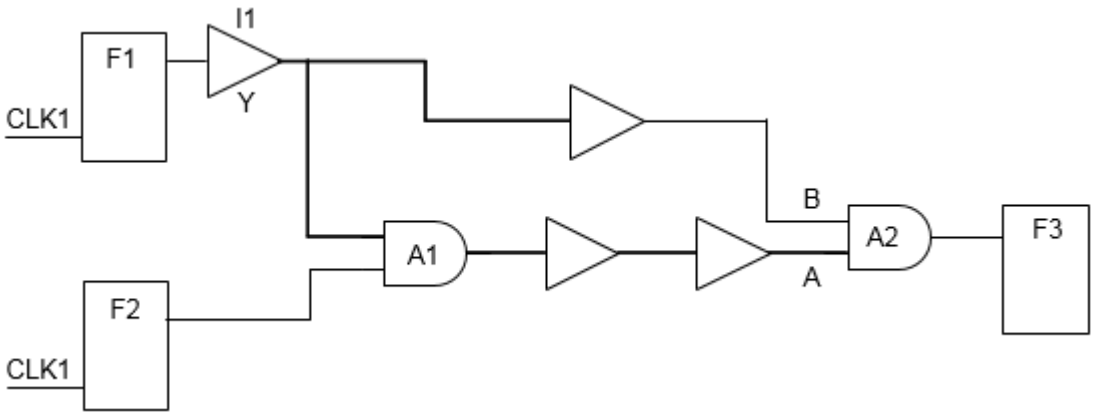
with attackers 1 and 2 as well, this results in a larger delay than GBA. If the path ending at F3 is a critical path, then the late arrival time used in GBA analysis at A1/Y is not an optimal edge as it produces optimism in GBA. In fact, late arrival used in GBA is coming from a pin that has a false path. To overcome this caveat, timing-aware edge mode extends the attacker timing windows by width of victim timing windows, such that GBA does not report optimistic delay on critical path.

Figure 4-27 Timing-Aware Edge Mode



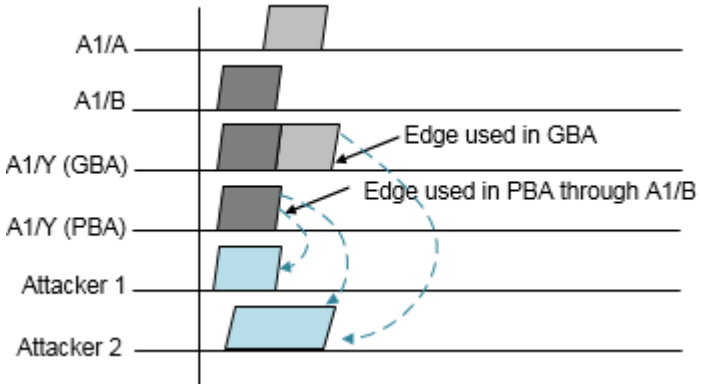
Similarly, consider the following multi-cycle path example:

Figure 4-28 Timing-Aware Edge Mode - Example 2



If the late arriving edge at A1/Y is based on arrival time from A1/A, then the SI delay computed based on that edge will see only one attacker, thus resulting in a smaller cross-talk delay compared to the cross-talk delay computed based on the arrival time from A1/B. If a path is

reported from F2 in GBA and PBA, then PBA reports a larger delay at stage A1 due to this optimism.



To address this optimism in GBA analysis in path-based alignment, the `timing_aware_edge` mode expands the attacker timing windows for stages with exceptions, such that GBA analysis will be pessimistic than PBA.

Input Arrival Mode

The input arrival attacker alignment mode can be set by using the following command:

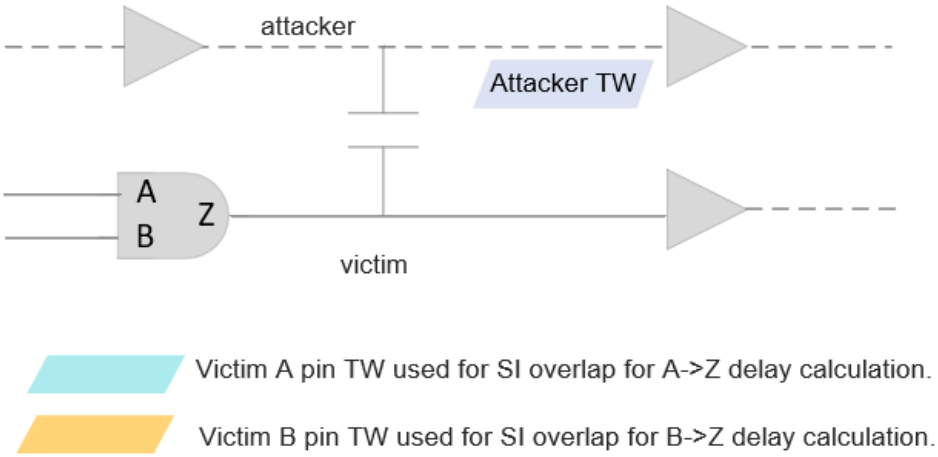
`set_db si_aggressor_alignment input_arrival`

In this mode, the attackers use timing windows formed by 2-sigma arrivals regardless of the analysis sigma multiplier.

Victim-Attackers Alignment

The victim driver input arrivals will be used for SI computation, as shown below:

Figure 4-29 Input Arrival Alignment Mode - Illustration 1



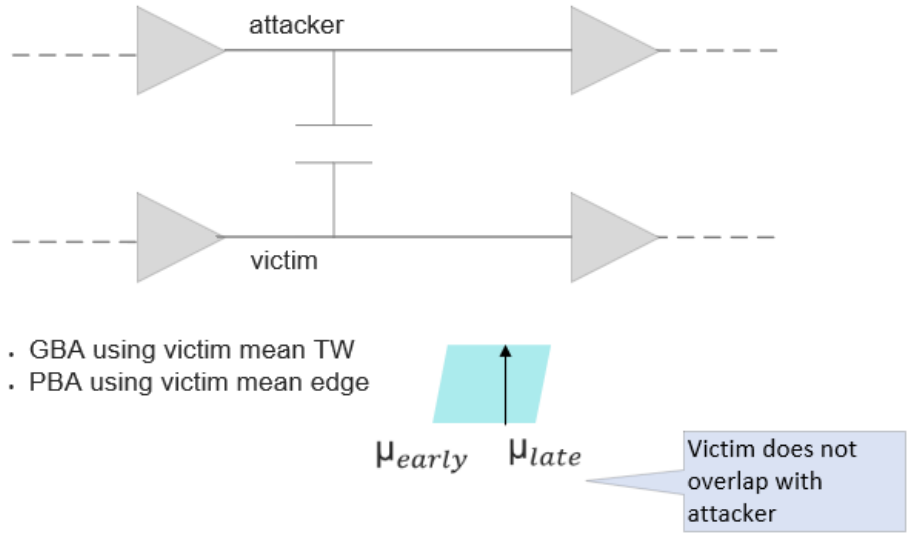
In this case, the software will internally account for victim driver cell delay while computing SI.

This behavior allows pessimism reduction in GBA SI analysis with narrower timing windows. Also, this provides a more accurate PBA SI analysis, with no dependency of driver cell GBA delays.

Victim Timing Window Handling with SOCV

The mean victim timing window and edge is used for SI computation in GBA and PBA modes, respectively. This behavior will allow GBA pessimism reduction, and a more accurate PBA analysis.

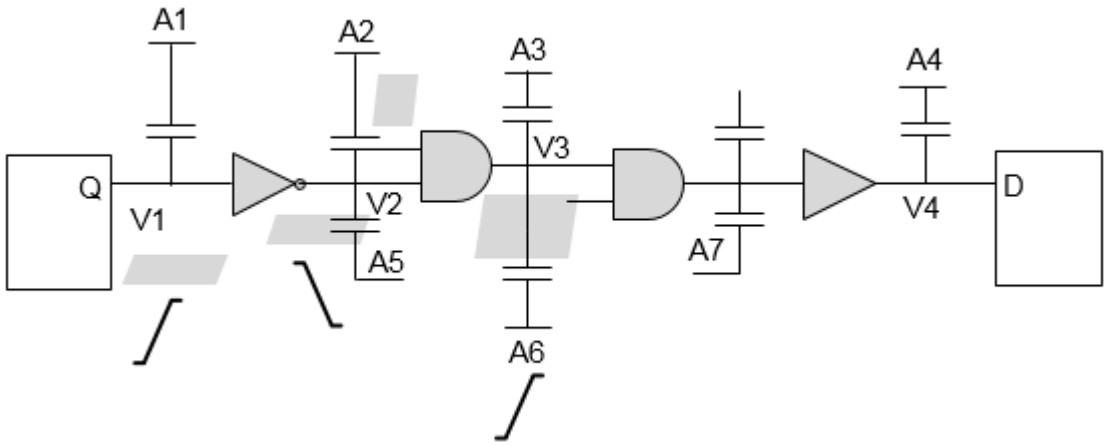
Figure 4-30 Input Arrival Alignment Mode - Illustration 2



Path-Based Analysis with Signal Integrity

Path-based SI analysis reduces the pessimism introduced during graph-based SI analysis by considering path-based slews and arrival for victims. This helps in reducing design closure ECO cycles, and saves area and power of design chips.

Figure 4-31 Analysis performed using actual path slew and arrival times of victim



Running PBA-SI on a complete set of paths in the design is computation intensive. It is recommended to run PBA-SI on critical paths identified from GBA-SI analysis. So, while using PBA-SI based signoff ensure that GBA-SI results are always pessimistic to PBA-SI.

This can be ensured by running GBA-SI in path overlap mode, by using the following command:

```
set_db si_aggressor_alignment path_overlap
```

Attacker Selection Based on Relationship Between Clocks

The selection of attackers also depends upon the relationships between the clocks. Presence of multiple clock groups - physically exclusive, asynchronous, and synchronous - in the design will influence attacker selection behavior of Tempus differently. Tempus by default considers all the clocks as synchronous clocks if no clock group is mentioned. You can change the behavior by using below command below:

```
set_db si_clock_synchronicity {asynchronous | synchronous} (default is
synchronous)
```

Synchronous clocks have defined phase relationships amongst each other. Timing windows are considered while evaluating the attacker/victim relationship.

The default behavior can be overridden by using the following command. This provides an option to specify the timing relationship between specific clock domains.

```
set_clock_groups
[-group <clock_list>]
[-name <name_list>]
[-logically_exclusive] | [-physically_exclusive] | [ [-asynchronous]]
```

The preferred order of clock relationships is physically exclusive (highest) and then asynchronous.

The table below explains the relationships and impact of different clock grouping on STA and SI analysis:

Table 4-1 Relationships and impact of different clock grouping

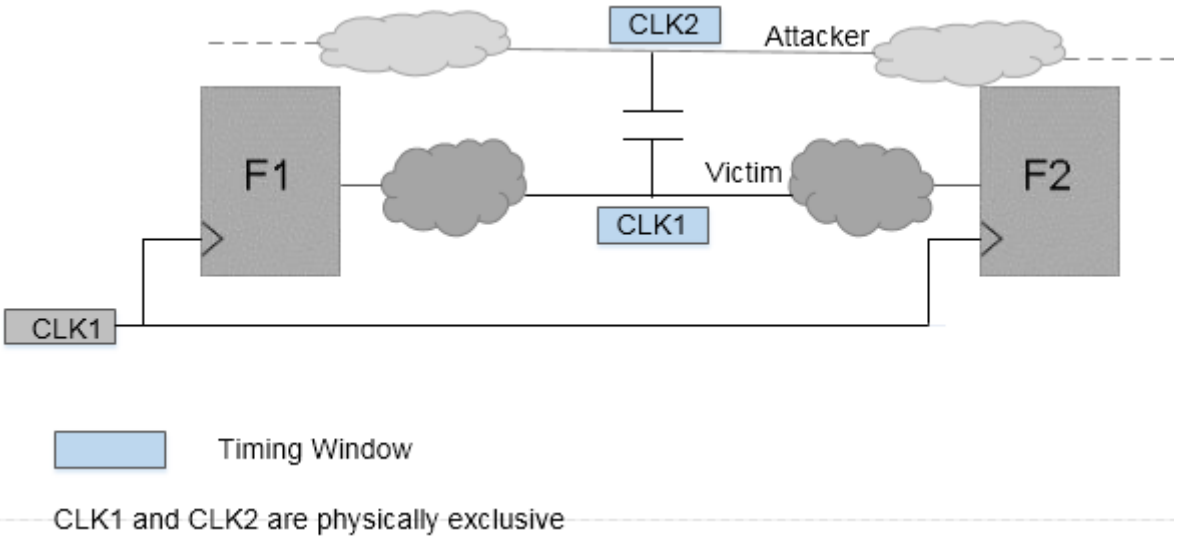
Relationship	Description	Impact on STA or Timing Paths	Impact on SI Analysis
Logically Exclusive	Specifies that there is no logical path between the clocks even though they exist.	Makes false path between such clocks.	No impact on SI Analysis. Crosstalk impact will remain the same.

Asynchronous	Indicates that there is no phase relationship between these clocks.	Makes false path between such clocks.	Attackers from asynchronous clock to victim clock are considered to have infinite timing windows.
Physically Exclusive	Specifies that these clocks will not exist physically at the same time in the chip. Such clocks mainly exist in constraints during mode merging.	Makes false path between such clocks.	Attackers from physically exclusive clock to victim clock are considered non-switching during SI analysis of the victim (that is, they do not attack the victim). **Note:** When multiple clocks on a victim net exist, then they are not marked physically exclusive.

Consider a few scenarios for clock grouping, as follows:

Scenario1: In the following figure, the attacker is receiving timing window (TW) with respect to the physically exclusive clock.

Figure 4-32 Attackers receiving TW with respect to Physically exclusive clock

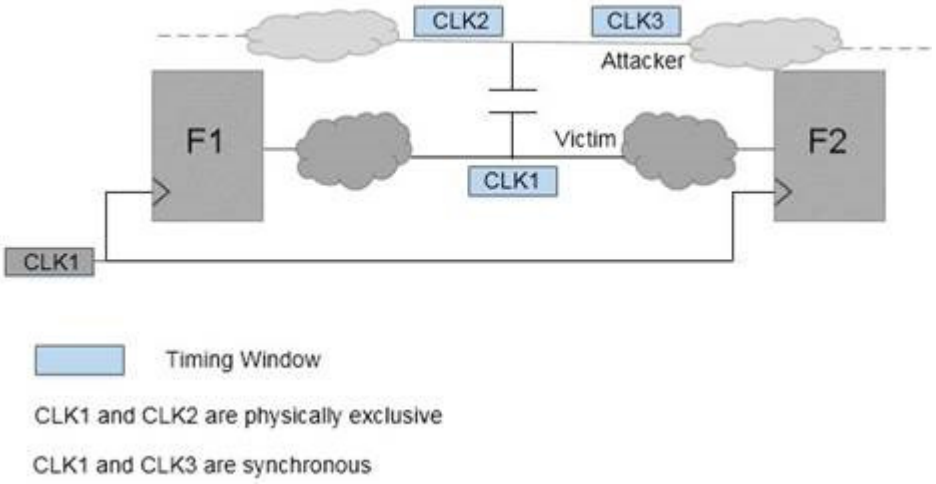


In this case, the attackers will be dropped even if the timing window overlaps with that of the victim.

No impact of attacker will be considered on the victim.

Scenario 2: In the following figure, the attacker has a timing window with respect to Synchronous as well as Physically Exclusive clocks to the victim.

Figure 4-33 Attackers having TW with respect to Synchronous as well as Physically Exclusive clock



In this case:

- Timing windows with respect to physically exclusive clocks to victim will be dropped (CLK2) and victim-attacker relationship is determined using the attacker timing window with respect to only synchronous clock (CLK3).
- Victim timing window (with respect to CLK1) overlapping is checked with attacker timing window (with respect to CLK3). As they do not overlap, attacker will not impact the victim.

Attacker Selection Based on Logical Correlation

During SI analysis Tempus introduces some pessimism when attackers that cannot logically attack together are allowed to attack. Tempus by default assumes that all attackers can switch together in a direction to cause a worst case delay (causing a slowdown) or speed up of transition on the victim. But in some cases, there could be logical relationship between the signals, that is, attackers might not actually switch together in the same direction.

Tempus performs logical correlation for inverters and buffers, except complex gates that are not included in logical correlation.

You can use the following command to enable logical correlation:

`set db si delay enable logical correlation true (Default: false)`

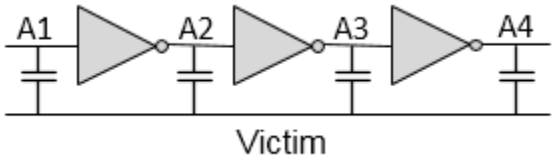
The following examples show some of the scenarios that logical correlation can handle.

Scenario 1: Relationship between Attackers

Exclusive sets are subsets of attackers that can switch simultaneously.

Exclusive sets can be {A1, A3} and {A2, A4}. Here, attackers A1 and A3 will be switching in one direction, whereas A2 and A4 will switch in the opposite direction, as shown in the figure below.

Figure 4-34 Logical Relationship Between Attackers

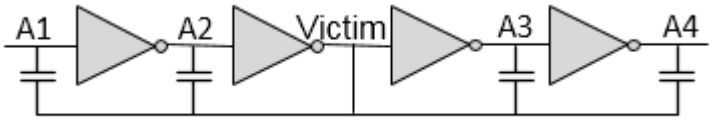


Scenario 2: Relationship between Victim and Attackers

In max analysis mode, attackers A1 and A4 can be excluded. Because, these nets cannot switch in the opposite direction to the victim to increase the delay.

In min analysis mode, attackers A2 and A3 can be excluded. Because, these nets cannot switch in the same direction as the victim to decrease the delay.

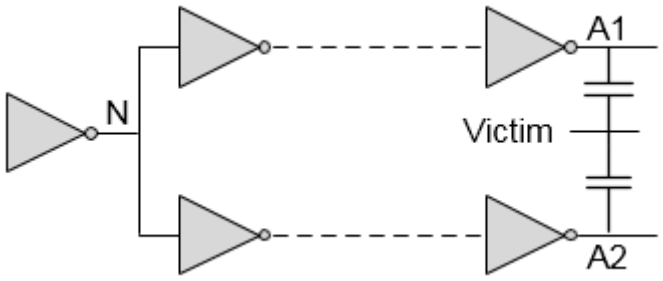
Figure 4-35 Logical Relationship Between Victim And Attackers



Scenario 3: Remote Attackers

If the number of inverters on each branch does not differ by a multiple of 2, then either A1 or A2 will be used as an attacker, as shown in the figure below.

Figure 4-36 Logical Relationship Between Remote Attackers



Waveform Computation

SI impact on delay (or glitch for glitch analysis) depends primarily on the following factors:

- Victim Slew
- Attackers' Slew
- Victim coupling cap to ground cap ratio
- Relative switching direction for victim/attacker
- Arrival windows of attackers/victim nets

SI impact on delay is performed in four steps. The first three steps will be applicable for SI Glitch analysis as well.

- Determine the slew of possible attackers and victim nets in their respective directions.
- Determine the alignments of overlapping attackers to compute the worst impact.
- Align all attackers at their respective positions.
- Compute the stage delay.

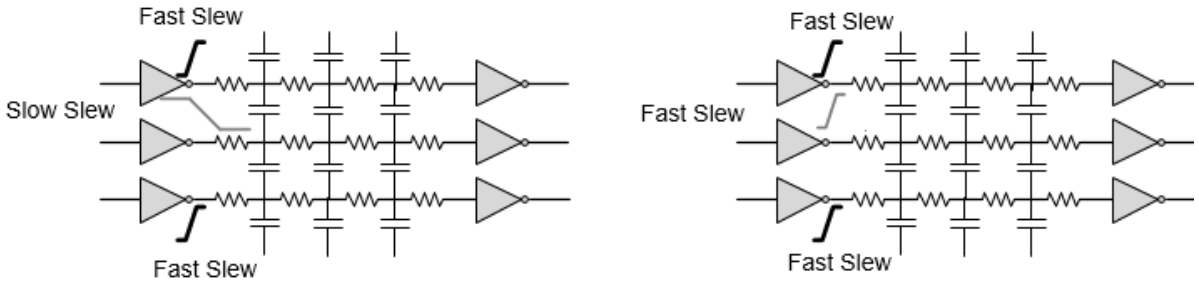
These are explained below:

Step 1: Determine Slew of possible Attackers and Victim Nets in respective directions.

Tempus implicitly considers attacker slew degradation by including the attacker's distributed parasitic network in the simulation.

Tempus determines the fastest and slowest slew possible in each direction (rise/fall) at every node in the design. For max delay analysis, the slowest slew for victim and the fastest slew (in opposite direction of victim slew) for attackers will be used, as shown in the figure below.

Figure 4-37 Determine slew for max delay and min delay



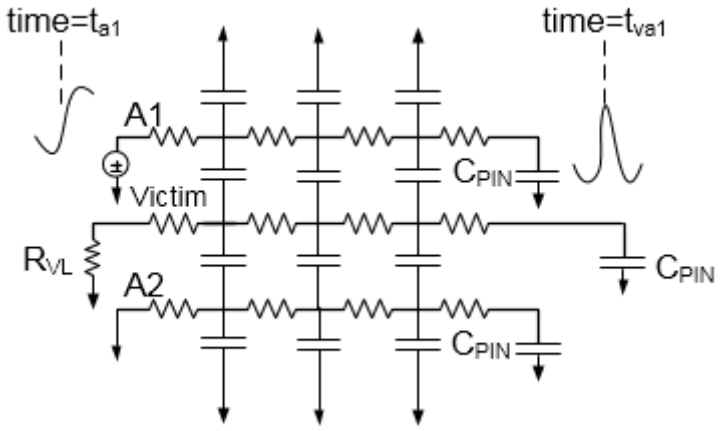
For min delay analysis, the fastest slew for victim and the fastest slew (in the same direction of victim slew) for attackers will be used, as shown in the figure above.

Step 2: Determine the alignments of overlapping attackers to compute the worst impact

Tempus aligns fully overlapping attacker's switching time in a way that creates worst impact of each attacker on the victim receiver at the same time. Tempus computes the time of flight for each attacker by individually switching each of them. The time of flight is the time between the attacker switching time at the driver output and the corresponding glitch peak time at the victim receiver input.

Tempus computes the delay from each attacker to the victim for peak alignment, as illustrated in the figure below.

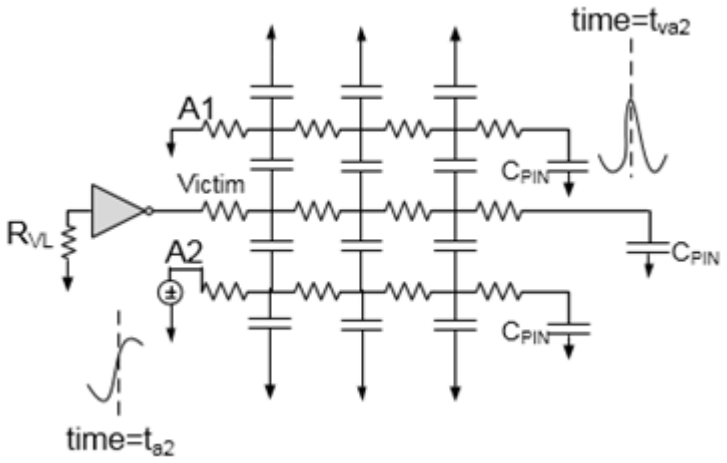
Figure 4-38 Compute Attacker A1 To Victim Delay (tvA1 – tA1)



To compute time of flight for attacker A1, a voltage source with a slew rate, that is either calculated internally or annotated from static timing analysis, is applied at attacker A1 driver output while keeping all other attackers and victims silent.

Tempus calculates the delay from the 50 percent point of the switching attacker A1 at the driving pin (t_{a1}), to the time the glitch effect on the victim receiver is maximum (t_{va1}). For attacker A1, the delay is t_{va1}-t_{a1}.

Figure 4-39 Compute Attacker A2 To Victim Delay (tvA2 – tA2)



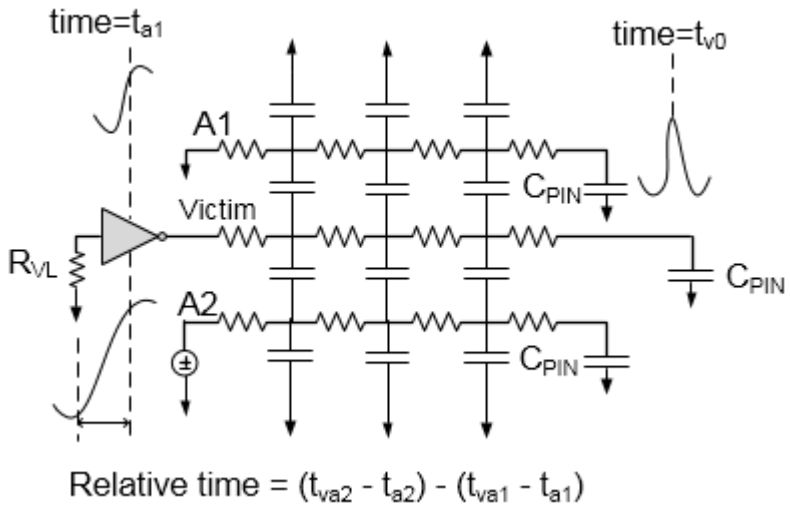
Similarly, Tempus calculates the delay from the 50% point of the switching attacker A2 at the driving pin (t_{a2}), to the time the glitch effect on the victim is maximum (t_{va2}). For this case, attacker A1 will be grounded. So, for attacker A2 the delay is t_{va2}-t_{a2}. These two delays can be different if one attacker is closely coupled to the near end of the victim wire, and the other attacker is closely coupled to the far end of the victim wire, and the victim wire is long.

Step 3: Aligning all attackers at their respective position

Tempus aligns the relative switch times of the fully overlapping attackers according to their time of flights, such that their individual peak at victim receiver input occurs at the same time.

The attacker relative alignment will be (t_{va2}-t_{a2}) - (tv_{a1}-t_{a1}) such that the peak due to attackers A1 and A2 at the victim receiver input occurs at the same time, as shown in the figure below.

Figure 4-40 Aligning Attackers to Have Worst Impact at Victim at Same Time

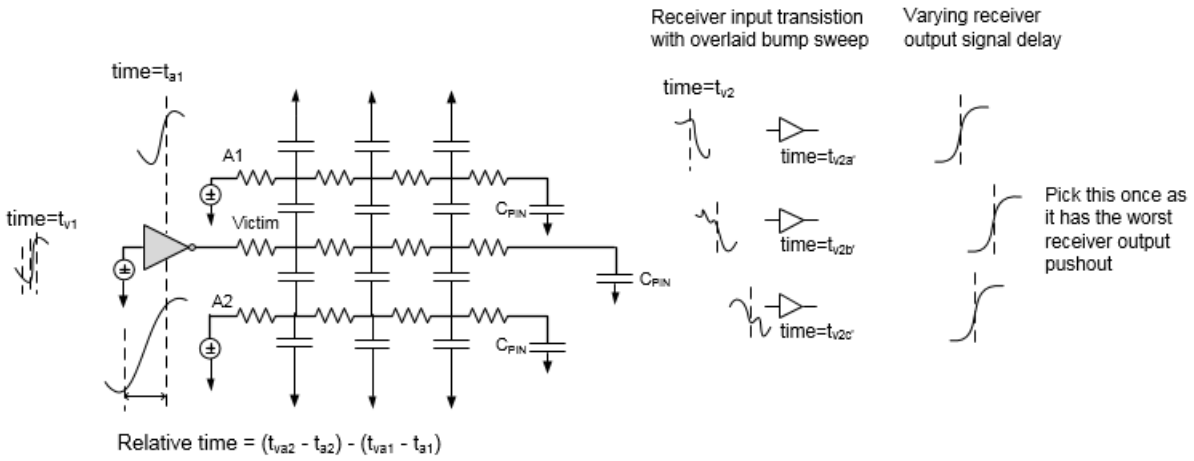


Step 4: Compute the stage delay

As a final step to compute the stage delay with SI, a transition is applied at the victim receiver, and the attacker's impact is evaluated across the victim slew to compute the worst-case stage delay.

In the figure below, the delay at the victim's receiver output is measured by considering the attackers across the victim slew. This will result in a change in the delay at the victim receiver's input. Tempus computes different delays as t_{v2a}', t_{v2b}', t_{vc2}', and so on for different victim alignment, with respect to attackers. The one resulting in the worst delay will be considered for the worst-case stage delay computation as t_{v2}'. Assuming that t_{v2} is the base delay for this stage, that is when all the overlapping attackers of this victim are silent. The delta delay will be (t_{v2}' - t_{v2}). This difference becomes the SI effect on the delay results (either slow down or speed up).

Figure 4-41 Compute stage delay with SI (t_{v2}' -t_{v1})



The SI impact on delay depends on the following factors:

Handling of Unconstrained Nets

If an attacker net is unconstrained, then Tempus does not consider timing windows of such nets for attacker selection. That means an unconstrained attacker is always selected in SI analysis, that is, timing window is infinite for these unconstrained nets. This makes SI analysis more pessimistic, but bounding.

To turn this off and use a constrained timing window, you can set `set db si_unconstrained_net_use_infinite_timing_window` to `false`. The default is `true`.

Constraining SI Analysis

SI analysis adheres to SDC constraints. However, you can additionally specify constraints on SI analysis to control whether a net can be an attacker or not. To do this, you can use the `set quiet aggressor` command. For example,

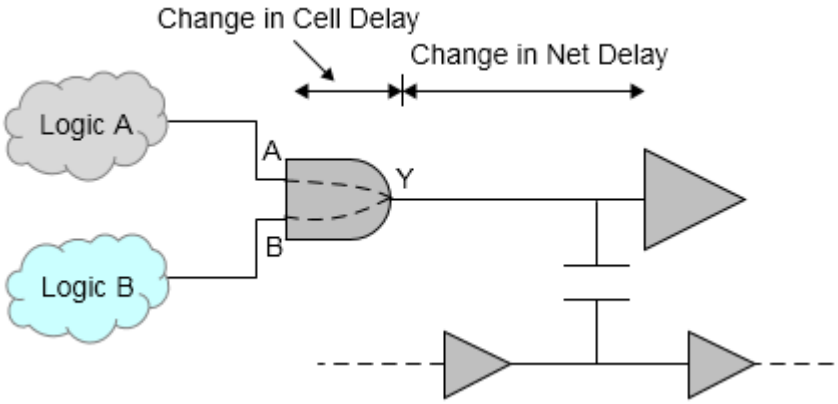
```
set_quiet_aggressor -victim vic -aggressor {att1 att2} -transition rise
```

This software ignores an attacker for certain edges when in reference to the victim.

Computing Timing Arc-Aware SI

The SI impact on delay is computed based on the timing arc through which the path is traversing.

Figure 4-42 SI Impact on Delay Computed on Timing Arc



By default, Tempus calculates the delay and SI for each arc. If you choose to view the SI delay separately, it will show annotated to each arc. This helps in reducing pessimism on paths that do not come through the worst arc.

This can be controlled by using the following command:

`set_db si_delay_delta_annotation_mode {lumpedOnNet | arc} (Default : arc)`

In lumped-on-net mode, Tempus annotates the worst arc delay on the net delay.

4.2.6 SI Delay Reporting

There are several commands for generating reports on SI effects on delay:

- `report_timing`
- `report_ delay calculation`
- `report_noise`
- `get_property`

These are explained below.

Using report_timing without Separating Delta Delay on Data

The `report_timing` command, by default, does not separate the delta delay on data (delta delay is always separated for clock). You can set the `incr_delay` option of the `timing report fields` attribute to display the delta delays of nets. It is recommended to use the `report_timing -split_delay` parameter for generating comprehensive reports. For example,

```
tempus > set_db timing_report_fields {hpin cell transition incr_delay delay
arrival}
tempus > report_timing -split_delay
#---------------------------------------------------------

Path 1: VIOLATED (-1.537 ns) Late Output Delay Assertion
View: slow_corner_setup
Group:          CCLK
Startpoint:    (F) rd_i[152]
Clock:         (R) CCLK
Endpoint:      (F) rd_o[152]
Clock:         (R) CCLK

Capture Launch
Clock Edge:    +1.666 0.000
Src Latency:   +0.000       0.000
Net Latency:   +0.000 (I)   0.000 (I)
Arrival:=       1.666       0.000
Output Delay:  -1.200
Uncertainty:   -0.100
Cppr Adjust:   +0.000
Required Time:  0.366
Launch Clock:   0.000
Input Delay:   +1.200
Data Path:     +0.703
Slack:         -1.537
#-------------------------------------------------------------------------
#  Pin                     Cell           Trans   Incr    Delay   Arrival
#                                          (ns)   Delay   (ns)    (ns)
#-------------------------------------------------------------------------
rd_i[152]                  (arrival)       0.004   -       -       1.200
rd_i_152_ATibuf_i164/A     BUFH_X16M_A12   0.004   0.000   0.001   1.201
rd_i_152_ATibuf_i164/Y     BUFH_X16M_A12   0.017   0.000   0.026   1.227
rd_i_152_ATrbuf_i225/A     BUFH_X13M_A12   0.031   0.000   0.014   1.241
rd_i_152_ATrbuf_i225/Y     BUFH_X13M_A12   0.023   0.000   0.040   1.281
rd_i_152_ATrbuf_i10126/A   BUFH_X16M_A12   0.038   0.000   0.016   1.296
rd_i_152_ATrbuf_i10126/Y   BUFH_X16M_A12   0.014   0.000   0.042   1.338
rd_i_152_ATrbuf_i263/A     BUFH_X16M_A12   0.054   0.000   0.027   1.365
rd_i_152_ATrbuf_i263/Y     BUFH_X16M_A12   0.017   0.000   0.051   1.416
rd_i_152_ATrbuf_i10136/A   BUFH_X16M_A12   0.042   0.000   0.020   1.436
rd_i_152_ATrbuf_i10136/Y   BUFH_X16M_A12   0.017   0.000   0.043   1.479
rd_i_152_ATrbuf_i185/A     BUFH_X16M_A12   0.028   0.000   0.012   1.491
rd_i_152_ATrbuf_i185/Y     BUFH_X16M_A12   0.016   0.000   0.037   1.528
rd_i_152_ATrbuf_i238/A     BUFH_X16M_A12   0.039   0.000   0.020   1.547
rd_i_152_ATrbuf_i238/Y     BUFH_X16M_A12   0.016   0.000   0.042   1.589
rd_i_152_ATrbuf_i265/A     BUFH_X11M_A12   0.037   0.000   0.017   1.606
rd_i_152_ATrbuf_i265/Y     BUFH_X11M_A12   0.018   0.000   0.043   1.648
rd_i_152_ATrbuf_i10136/A   BUFH_X16M_A12   0.031   0.000   0.013   1.661
rd_i_152_ATrbuf_i10136/Y   BUFH_X16M_A12   0.014   0.000   0.037   1.698
rd_i_152_ATrbuf_i202/A     BUFH_X13M_A12   0.042   0.000   0.020   1.718
rd_i_152_ATrbuf_i202/Y     BUFH_X13M_A12   0.017   0.000   0.043   1.761
rd_i_152_ATrbuf_i10136/A   BUFH_X16M_A12   0.028   0.000   0.012   1.773
rd_i_152_ATrbuf_i10136/Y   BUFH_X16M_A12   0.017   0.000   0.035   1.808
U2349/A                    OR2_X11M_A12    0.029   0.000   0.013   1.821
U2349/Y                    OR2_X11M_A12    0.019   0.000   0.050   1.871
rd_o_152_ATobuf_i155/A     BUFH_X6M_A12    0.020   0.000   0.003   1.874
rd_o_152_ATobuf_i155/Y     BUFH_X6M_A12    0.009   0.000   0.029   1.903
rd_o[152]                  ud_lpu_2x       0.009   0.000   0.000   1.903
#-------------------------------------------------------------------------
```

Using report_timing with Separating Delta Delay on Data

You can use the following commands to report delta delay on data separately under the `Incr Delay` column in the timing report:

```
tempus > set_db si_delay_separate_on_data true
tempus > report_timing -split_delay
```

```
------------------------------------------------------------

Path 1: VIOLATED (-1.537 ns) Late Output Delay Assertion
View: slow_corner_setup
Group: CCLK
Startpoint: (F) rd_i[152]
Clock: (R) CCLK
Endpoint: (F) rd_o[152]
Clock: (R) CCLK
                Capture     Launch
Clock Edge:+     1.666       0.000
Src Latency:    +0.000       0.000
Net Latency:    +0.000 (I)   0.000 (I)
Arrival:=        1.666       0.000
Output Delay:   -1.200
Uncertainty:    -0.100
Cppr Adjust:    +0.000
Required Time: =0.366
Launch Clock:  =0.000
Input Delay:    +1.200
Data Path:      +0.703
Slack:=         -1.537
#-------------------------------------------------------------------------
# Pin                     Cell            Trans  Incr    Delay   Arrival
#                                          (ns)  Delay    (ns)    (ns)
#-------------------------------------------------------------------------
rd_i[152]                  (arrival)      0.004     -       -     1.200
rd_i_152_ATibuf_i164/A    BUFH_X16M_A12  0.004   0.000   0.001    1.201
rd_i_152_ATibuf_i164/Y    BUFH_X16M_A12  0.017   0.002   0.026    1.227
rd_i_152_ATrbuf_i225/A    BUFH_X13M_A12  0.031   0.002   0.014    1.241
rd_i_152_ATrbuf_i225/Y    BUFH_X13M_A12  0.023   0.002   0.040    1.281
rd_i_152_ATrbuf_i10126/A  BUFH_X16M_A12  0.038   0.001   0.016    1.296
rd_i_152_ATrbuf_i10126/Y  BUFH_X16M_A12  0.014   0.002   0.042    1.338
rd_i_152_ATrbuf_i263/A    BUFH_X16M_A12  0.054   0.007   0.027    1.365
rd_i_152_ATrbuf_i263/Y    BUFH_X16M_A12  0.017   0.002   0.051    1.416
rd_i_152_ATrbuf_i10136/A  BUFH_X16M_A12  0.042   0.003   0.020    1.436
rd_i_152_ATrbuf_i10136/Y  BUFH_X16M_A12  0.017   0.000   0.043    1.479
rd_i_152_ATrbuf_i185/A    BUFH_X16M_A12  0.028   0.000   0.012    1.491
rd_i_152_ATrbuf_i185/Y    BUFH_X16M_A12  0.016   0.002   0.037    1.528
rd_i_152_ATrbuf_i238/A    BUFH_X16M_A12  0.039   0.003   0.020    1.547
rd_i_152_ATrbuf_i238/Y    BUFH_X16M_A12  0.016   0.001   0.042    1.589
rd_i_152_ATrbuf_i265/A    BUFH_X11M_A12  0.037   0.001   0.017    1.606
rd_i_152_ATrbuf_i265/Y    BUFH_X11M_A12  0.018   0.001   0.043    1.648
rd_i_152_ATrbuf_i10136/A  BUFH_X16M_A12  0.031   0.001   0.013    1.661
rd_i_152_ATrbuf_i10136/Y  BUFH_X16M_A12  0.014   0.001   0.037    1.698
rd_i_152_ATrbuf_i202/A    BUFH_X13M_A12  0.042   0.002   0.020    1.718
rd_i_152_ATrbuf_i202/Y    BUFH_X13M_A12  0.017   0.001   0.043    1.761
rd_i_152_ATrbuf_i10136/A  BUFH_X16M_A12  0.028   0.001   0.012    1.773
rd_i_152_ATrbuf_i10136/Y  BUFH_X16M_A12  0.017   0.000   0.035    1.808
U2349/A                    OR2_X11M_A12  0.029   0.000   0.013    1.821
U2349/Y                    OR2_X11M_A12  0.019   0.001   0.050    1.871
rd_o_152_ATobuf_i155/A    BUFH_X6M_A12   0.020   0.000   0.003    1.874
```

```
rd_o_152_ATobuf_i155/Y    BUFH_X6M_A12   0.009   0.000   0.029    1.903
rd_o[152]                  ud_lpu_2x     0.009   0.000   0.000    1.903
#-------------------------------------------------------------------
```

Using report_timing with lumpedOnNet Option

By default, Tempus calculates the delay, including SI for each arc, and if you choose to view the SI delay separately, it will show as annotated to each arc. Alternatively, you can choose to annotate the SI delay solely to the net. You can use the following command to lump the delta delay all onto the net:

```
tempus > set_db si_delay_delta_annotation_mode lumpedOnNet
tempus > report_timing
#-------------------------------------------------------------------
Path 1: VIOLATED (-1.537 ns) Late Output Delay Assertion
View: slow_corner_setup
Group: CCLK
Startpoint: (F) rd_i[152]
Clock: (R) CCLK
Endpoint: (F) rd_o[152]
Clock: (R) CCLK

                  Capture      Launch
Clock Edge:+      1.666        0.000
Src Latency:+     0.000        0.000
Net Latency:+     0.000 (I)    0.000 (I)
Arrival:=         1.666        0.000

Output Delay:-    1.200
Uncertainty:-     0.100
Cppr Adjust:+     0.000
Required Time:=   0.366
Launch Clock:=    0.000
Input Delay:+     1.200
Data Path:+       0.703
Slack:=          -1.537
#-------------------------------------------------------------------------------
# Pin                        Cell               Trans   Incr    Delay   Arrival
#                                               (ns)    Delay   (ns)    (ns)
#-------------------------------------------------------------------------------
rd_i[152]                  (arrival)            0.004   0.000   0.001    1.201
rd_i_152_ATibuf_i16431/Y   BUFH_X16M_A12TS_C35  0.004   0.004   0.039    1.240
rd_i_152_ATrbuf_i22570/Y   BUFH_X13M_A12TS_C35  0.031   0.002   0.056    1.296
rd_i_152_ATrbuf_i1012662/Y BUFH_X16M_A12TS_C35  0.038   0.008   0.069    1.365
rd_i_152_ATrbuf_i26345/Y   BUFH_X16M_A12TS_C35  0.054   0.004   0.071    1.436
rd_i_152_ATrbuf_i1013653/Y BUFH_X16M_A12TS_C35  0.042   0.000   0.055    1.491
rd_i_152_ATrbuf_i18556/Y   BUFH_X16M_A12TS_C35  0.028   0.004   0.056    1.547
rd_i_152_ATrbuf_i23875/Y   BUFH_X16M_A12TS_C35  0.039   0.002   0.059    1.605
rd_i_152_ATrbuf_i26553/Y   BUFH_X11M_A12TS_C35  0.037   0.002   0.055    1.661
rd_i_152_ATrbuf_i1013655/Y BUFH_X16M_A12TS_C35  0.031   0.002   0.057    1.717
rd_i_152_ATrbuf_i20278/Y   BUFH_X13M_A12TS_C35  0.042   0.002   0.055    1.772
rd_i_152_ATrbuf_i1013651/Y BUFH_X16M_A12TS_C35  0.028   0.000   0.048    1.820
U2349/Y                    OR2_X11M_A12TS_C35   0.029   0.001   0.053    1.873
rd_o_152_ATobuf_i15597/Y   BUFH_X6M_A12TS_C35   0.020   0.000   0.030    1.903
rd_o[152]                  ud_lpu_2x            0.009   0.000   0.000    1.903
#-------------------------------------------------------------------------------
```

Using report_delay_calculation -si Command

You can use the report__delay_calculation command to report the delay calculation information for a cell or net timing arc. When `-si` parameter is specified, the software will report the SI components, including incremental delay. For example,

```
tempus > report_delay_calculation -from wdata_rw_o_150__ATrbuf_i1013579/Y -to
wdata_rw_o_150__ATrbuf_i25739/A -si
-------------------------------------------------------

From pin   : rw_150_rbuf_i1013579/Y
Cell       : BUFH_X16M_A12TS_C35
Library    : ff3sigma_0p935v_125c
To pin     : rw_150_rbuf_i25739/A
Cell       : BUFH_X13M_A12TS_C35
Library    : ff3sigma_0p935v_125c
Base or SI : SI delay
Delay type : net


-------------------------------------------------------------
RC Summary for net rw_150_rbuf_1413352
-------------------------------------------------------------
Number of capacitance : 4
Net capacitance       : 0.032299 pF
Total rise capacitance: 0.040678 pF
Total fall capacitance: 0.040768 pF
Number of resistance  : 3
Total resistance      : 2394.699463 Ohm
-------------------------------------------------------------
SI information for rise
-------------------------------------------------------------------------------
Victim timing window  :  1.3741 ns 1.5132 ns  CCLK
-------------------------------------------------------------------------------
Attacker              Status Direction Slew      Coupling cap Fanout Voltage Bump
Timing window       Clock
-------------------------------------------------------------------------------

sbsel_r0_5_rbuf_14106  SY     Fall     0.010500ns  0.009328pF   1  0.935v
0.114257v  1.3739ns 1.4720ns  CCLK
addr_r0_11_rbuf_14131  SY     Fall     0.009375ns  0.008941pF   1  0.935v
0.136697v  1.3728ns 1.4745ns  CCLK
addr_r1_13_mdrc_193    SY     Fall     0.011750ns  0.000998pF   1  0.935v
0.022721v  1.3648ns 1.4503ns  CCLK
-------------------------------------------------------------------------------
SI information for fall
-------------------------------------------------------------------------------
Victim timing window  :  1.3777 ns 1.5183 ns  CCLK
-------------------------------------------------------------------------------
Attacker              Status Direction  Slew       Coupling cap  Fanout  Voltage
Bump     Timing window       Clock
-------------------------------------------------------------------------------
sbsel_r0_5_rbuf_14106 SY     Rise      0.010625 ns 0.009328 pF    1    0.935 v
0.114725 v  1.3698 ns 1.4689 ns  CCLK
addr_r0_11_rbuf_14131 SY     Rise      0.009375 ns 0.008941 pF    1    0.935 v
0.136790 v  1.3691 ns 1.4698 ns  CCLK
addr_r1_13_mdrc_193   SY     Rise      0.011625 ns 0.000998 pF    1    0.935 v
0.022907 v  1.3618 ns 1.4473 ns  CCLK
-------------------------------------------------------------------------------
Model types
```

```
-------------------------------------------------------------------------------
Net                       Term                     Cell                   Model
-------------------------------------------------------------------------------

rw_150_rbuf_14133         rw_150_rbuf_i1013579/Y   BUFH_X16M_A12TS_C35    NLDM
rw_150_rbuf_14133         rw_150_rbuf_i25739/A     BUFH_X13M_A12TS_C35    CCS
sbsel_r0_5_rbuf_14106     Y                        BUFH_X9M_A12TS_C35     CCS
addr_r0_11_rbuf_14131     Y                        BUFH_X11M_A12TS_C35    CCS
addr_r1_13_mdrc_193       Y                        BUFH_X11M_A12TS_C35    CCS
-------------------------------------------------------------------------------
                           Rise        Fall
-------------------------------------------------------------
Incr Net Delay          : 0.042800 ns 0.040500 ns
Net delay               : 0.088500 ns 0.085400 ns
From pin transition time: 0.006100 ns 0.006000 ns
To pin transition time  : 0.122700 ns 0.123700 ns
-------------------------------------------------------------
Abbreviative attacker status and filtered reason:
-------------------------------------------------------------
SY :  Synchronous
INF:  Attacker with infinite timing window
PE :  Filtered due to physical exclusion
UF :  Filtered by user
CN :  Filtered because it is a constant net
SB :  Filtered due to small bump
TA :  Filtered because of timing window does not overlap
LC :  Filtered due to logical correlation
ACT:  Attacker is Active.
-------------------------------------------------------------
```

Note: To access attacker information, you need to make the `set_db si_delay_enable_report` to `true` setting.

Using report_noise -delay {max | min} Command

You can use the `report_noise -delay max -nets` *`netName`* command to report incremental delay on nets. For example,

```
tempus > report_noise -delay max -nets rdata_r_2x_combined[211]
-------------------------------------------------------------------------------

Report Options:
-------------------------------------------------------------------------------
-type delay
-net r_2x_combined[211]
-threshold 10 (ps)
-delay max
[-style default]
[-sortBy IncrDelay]
-------------------------------------------------------------------------------
-------------------------------------------------------------------------------
Type  IncrDelay(ps) Delay(NomMax/Max)  Vdd(mV) VictimNet
R     0.200         2.600/2.800        740     U2307/B {r_2x_combined[211]}


Type  IncrDelay(ps)  Delay(NomMax/Max)  Vdd(mV)  VictimInst
R     21.600         184.900/206.500    740      r_2x_combined_reg_211_/Q
{SDFFRPQ_X1M_A12TS_C35:CK}
```

```
Constituents:
      Offset(ps)  Slew(ps)   Xcap(fF)  Edge  Status  Vdd(mV)   Net
Cpl:  -           26.125     0.609     F     INF     740  r_o_327__ATobuf_15422
Cpl:  -           26.250     0.452     F     INF     740  r_o_323__ATobuf_15426
Cpl:  -           26.750     0.360     F     INF     740  r_o_343__ATobuf_15406
Cpl:  -           75.125     0.491     F     INF     740  r_2x_combined[298]
Cpl:  -           77.875     0.473     F     INF     740  r_2x_combined[235]
-------------------------------------------------------------------------------
Type  IncrDelay(ps)  Delay(NomMax/Max)  Vdd(mV)  VictimNet
R        0.000          0.400/0.400      740     r_2x_combined_reg_206_/SI
{r_2x_combined[211]}


Type  IncrDelay(ps)  Delay(NomMax/Max)   Vdd(mV)  VictimInst
R       21.600        184.900/206.500     740   r_2x_combined_reg_211_/Q
{SDFFRPQ_X1M_A12TS_C35:CK}


Constituents:
      Offset(ps)   Slew(ps)    Xcap(fF)  Edge  Status Vdd(mV)   Net
Cpl:          -    26.125      0.609     F     INF    740  r_o_327__ATobuf_15422
Cpl:          -    26.250      0.452     F     INF    740  r_o_323__ATobuf_15426
Cpl:          -    26.750      0.360     F     INF    740  r_o_343__ATobuf_15406
Cpl:          -    75.125      0.491     F     INF    740  r_2x_combined[298]
Cpl:          -    77.875      0.473     F     INF    740  r_2x_combined[235]
-------------------------------------------------------------------------------
Type  IncrDelay(ps)  Delay(NomMax/Max)  Vdd(mV)  VictimNet
F        0.300          2.500/2.800      740     U2307/B {r_2x_combined[211]}

Type  IncrDelay(ps)  Delay(NomMax/Max)   Vdd(mV)  VictimInst
F       16.500        178.900/195.400     740   r_2x_combined_reg_211_/Q
{SDFFRPQ_X1M_A12TS_C35:CK}


Constituents:
      Offset(ps)   Slew(ps)    Xcap(fF) Edge Status   Vdd(mV)    Net
Cpl:          -    26.375      0.609    R    INF      740  r_o_327__ATobuf_15422
Cpl:          -    26.625      0.452    R    INF      740  r_o_323__ATobuf_15426
Cpl:          -    27.125      0.360    R    INF      740  r_o_343__ATobuf_15406
Cpl:          -    95.500      0.491    R    INF      740  r_2x_combined[298]
Cpl:          -    100.375     0.473    R    INF      740  r_2x_combined[235]
-------------------------------------------------------------------------------
Type   IncrDelay(ps)  Delay(NomMax/Max)  Vdd(mV)   VictimNet
F     0.000           0.400/0.400        740       r_2x_combined_reg_206_/SI
{r_2x_combined[211]}


Type   IncrDelay(ps)  Delay(NomMax/Max)  Vdd(mV)   VictimInst
F      16.500         178.900/195.400    740       r_2x_combined_reg_211_/Q
{SDFFRPQ_X1M_A12TS_C35:CK}


Constituents:
      Offset(ps)   Slew(ps     Xcap(fF) Edge Status   Vdd(mV)    Net
Cpl:          -    26.375      0.609    R    INF      740  r_o_327__ATobuf_15422
Cpl:          -    26.625      0.452    R    INF      740  r_o_323__ATobuf_15426
```

```
Cpl:          -    27.125      0.360    R    INF      740  r_o_343__ATobuf_15406
Cpl:          -    95.500      0.491    R    INF      740  r_2x_combined[298]
Cpl:          -    100.375     0.473    R    INF      740  r_2x_combined[235]
*****************************************************************************
```

Using Signal Integrity Attributes with get_property Command

Signal Integrity information can be reported using SI attributes for both graph-based as well as path-based analysis. These attributes can be queried using the `get_property` command.

You need to enable SI attributes by setting the `timing_report_enable_si_debug` attribute to `true`.

The use model for using SI attributes for reporting is given below:

For path-based analysis, the SI attributes can be reported as follows:

The `report_timing` `-retime` *`option`* `-collection` command creates a property called `timing_points` for each corresponding stage of the path. The `timing_points` property is a pointer that contains information for each stage. Each timing point has a property called `si_object`, which contains the victim information.

The following example shows a query on signal integrity attributes for timing points:

```
set path [report_timing -through $pin -retime path_slew_propagation -collection ]
set tps [ get_property $path timing_points ]
foreach_in_collection point $tps {
     set x [ get_property $point si_object ]
     set vic_name [ get_property $x hierarchical_name ]
     puts "victim: $vic_name"
     set num_agg [ get_property $x num_attackers ]
     puts "Number of attackers: $num_agg"
     set aggs [ get_property $x attackers ]
     puts "attackers info:"
     foreach_in_collection current_agg $aggs {
          set active [ get_property $current_agg is_active ]
          set state [ get_property $current_agg state ]
          set agg_name [get_property $current_agg hierarchical_name]
          puts "$agg_name $active $state"
     }
}
```

The above use model is valid for graph-based analysis also. Additionally, for GBA the SI attributes can be reported on the receiver input pin directly instead of timing points. Each pin has a property `si_victim`, which contains the victim SI information. The following example shows the usage of `si_victim`, where the number of active attackers are being printed for both rise and fall transitions in maximum delay analysis:

```
set vicArray [get_property [get_pins $pin] si_victim -view $view]
foreach_in_collection vic $vicArray {
     set view_type [get_property $vic view_type]
```

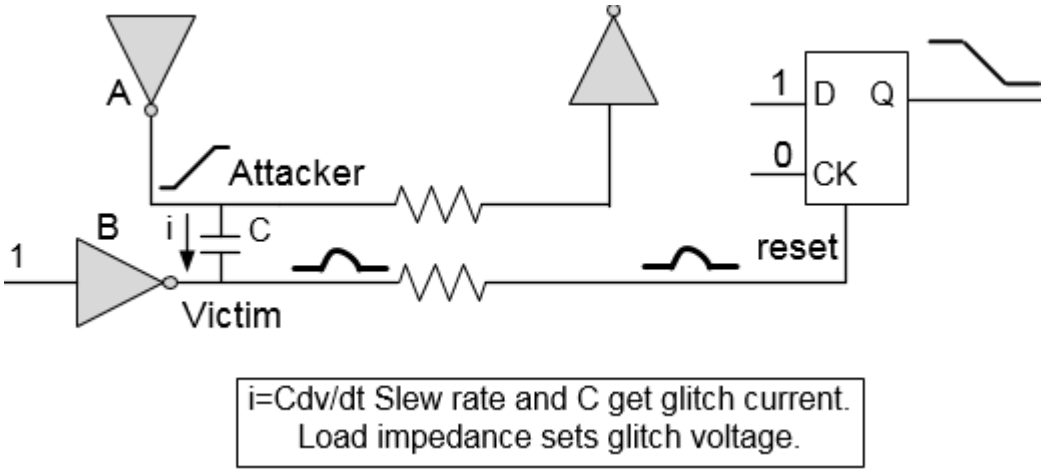
```
      set tran [get_property $vic transition]
      if {$view_type == "Late"} {
          if {$tran == "Rise"} {
             set num_att [get_property $vic num_active_attackers]
             puts "number of active attackers for rise transition: $num_att"
          }
          if {$tran == "Fall"} {
             set num_att [get_property $vic num_active_attackers]
             puts "number of active attackers for fall transition: $num_att"
          }
    }
}
```

For more information on signal integrity (SI) attributes/properties, refer to `get_property` command in the *Tempus Text Command Reference Guide*.

4.2.7 SI Glitch Analysis - An Overview

Glitch analysis is based on calculating the worst-case glitch waveforms on each net in the design. SI can result in functional failures due to glitch. This is illustrated in the diagram below:

Figure 4-43 Glitch due to SI resulting in functional failure



In this example, coupling from attacker A causes a significant glitch on the reset signal so that it resets the flip-flop and destroys the stored logic state. This problem does not show up in logic simulation, formal verification, or timing analysis. Isolating the source of a noise-induced functional problem can be very time consuming and may take a few months of debugging. Tempus calculates and reports the glitch induced in the design due to these scenarios.

Tempus performs glitch analysis by analyzing each potential victim net from primary inputs to primary outputs. The analysis of each net is based on calculating the worst-case glitch waveforms that can occur at the input pins of each of the victim net's receiving cells. The receiver output peak of each receiving cell instance is computed by analyzing the propagation

of the glitch waveform from the receiver's input through the first stage in the receiver. Tempus assumes all worst-case crosstalk scenarios are possible unless timing windows are used.

Tempus checks for glitch receiver peak and propagates the glitch at a single stage and checks to see if the noise glitch exceeds the failure criteria. This greatly reduces the number of potential false alarms as it utilizes the inherent glitch filtering properties of CMOS logic.

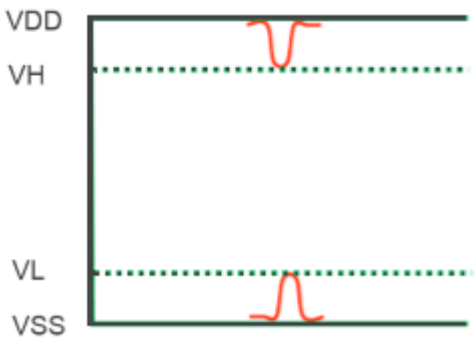
Glitch Type

There are two types of glitch depending on victim state and attacker switching direction.

Glitch Type	Victim State	Attacker Switching
Glitch over ground rail (VL)	Low	Rising
Glitch under power rail (VH)	High	Falling

These are illustrated in the figure below.

Figure 4-44 VL and VH Glitch type

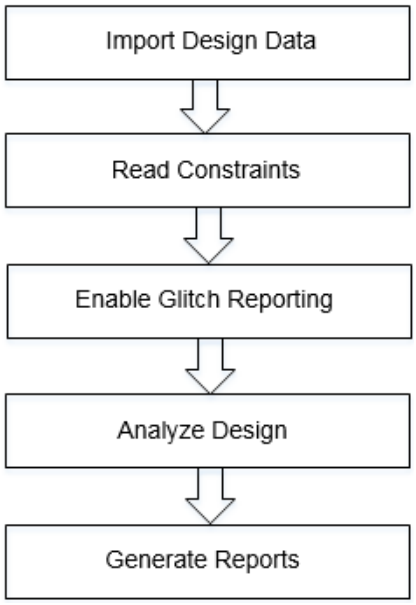


Glitch between ground and power rail voltages can cause logic failure, if they exceed logic thresholds of the technology.

4.2.8 SI Glitch Analysis Flow

The following figure shows the glitch noise analysis flow:

Figure 4-45 SI Glitch Analysis Flow



Sample Glitch Analysis Script

The following sample script shows the steps to perform glitch analysis:

- Load the timing libraries (.lib) and noise data (cdB, ECSM-Noise, or CCS-Noise):

  ```
  read_libs typ.lib -cdb typ.cdB
  ```

- Schedule reading of a structural, gate-level verilog netlist:

  ```
  read_netlist top.v
  ```

- Set the top module name, and check for consistency between Verilog netlist and timing libraries:

  ```
  init_design top
  ```

- Set the proper technology node:

  ```
  set_db design_process_node 16
  ```

- Load the timing constraints:

  ```
  read_sdc constraints.sdc
  ```

- Load the cell parasitics:

  ```
  read_spef top.spef.gz
  ```

- Enable signal integrity (SI) analysis.

  ```
  set_db delaycal_enable_si true
  ```

- Perform glitch analysis and enable the glitch reports to be generated

  ```
  set_db si_glitch_enable_report true
  ```

- Enable glitch propagation:

  ```
  set_db si_enable_glitch_propagation true
  ```

- Perform glitch analysis:

  ```
  update_glitch
  ```

- Generate SI glitch report:

  ```
  report_noise -txtfile glitch.txt
  ```

4.2.9 Understanding SI Glitch Analysis

In order to perform crosstalk functional failure analysis, Tempus first calculates the worst-case glitch that occurs on each victim net when it is not transitioning. This worst-case glitch is created from the worst-case combination of all allowable attackers where the attackers are nets that are coupled to the victim. The set of allowable attackers is determined from both timing and logic relationships between the victim and attacker nets. For example, attackers whose timing is such that they do not overlap are not combined. A unique worst-case glitch is calculated for each receiver (fanout) of the victim. Once the worst-case glitch is determined, Tempus performs a number of additional steps to determine if this glitch is harmful to the functionality of the design being analyzed.

There are three key components of SI Glitch analysis:

- Glitch Waveform Computation
- Glitch Failure Detection
- Glitch Propagation

Glitch Waveform Computation

For more details, refer to section on Waveform Computation.

Glitch Failure Detection

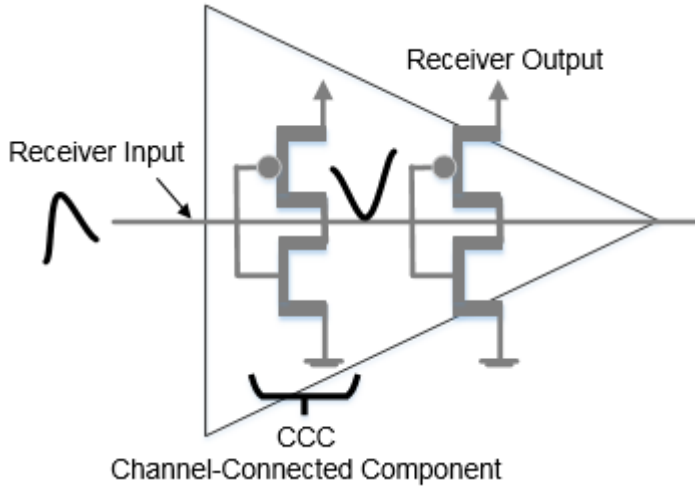
When worst glitch is computed, Tempus checks if a glitch can result in failure. Tempus uses two criteria for glitch failure by measuring the glitch at input and output of receiver cell, as shown in the figure below.

- Receiver Input Peak (RIP) Check

- Receiver Output Peak (ROP) Check

These are illustrated in the diagram below.

Figure 4-46 Glitch representation at input and output of receiver



Receiver Input Peak (RIP) Check

If a net has a receiver without noise data, then the receiver input peak (RIP) is compared against:

`set db si_glitch_input_threshold (or si_glitch_input_voltage_high_threshold, si_glitch_input_voltage_low_threshold)`

or

`set glitch threshold -failure_point input -threshold_type failure -value float`

In this case, failure can occur if RIP is greater than the tolerance threshold value.

Receiver Output Peak (ROP) Check

Tempus has the capability to compute glitch impact one step inside the cell, that is, at the output of the first Channel Connected Component (CCC) in the cell. This is known as ROP (Receiver Output Peak) analysis. Since CMOS logic acts as a low pass filter, it attenuates the glitch, so that the glitch checked at the output of the first CCC may be less than the glitch at the input of the cell. Using the ROP to determine failure may reduce the number of violation counts.

Note: In some cases, the ROP may be more than the RIP. The ROP can be as high as VDD in case of a full rail RIP, so any result close to the VDD ROP can be considered reasonable.

If a net has receiver with noise data (cdB or ECSM-Noise or CCS-Noise), then Tempus propagates the glitch one stage and calculates the Receiver Output Peak (ROP). If the arc is from input to output of the cell, then failure will occur if ROP is greater than the values specified using the following command:

set_db si_glitch_receiver_peak_limit (or si_glitch_receiver_clock_peak_limit, -si_glitch_receiver_latch_peak_limit)*double*

or

set_glitch_threshold -failure_point last -pin_type {clock | data | latch | all} -threshold_type failure -value *float*

If the arc is from an input to an internal node of a combinational cell, by default failure will occur if ROP is greater than 30% of the vdd value or if it is greater values specified using the following command:

```
set_glitch_threshold -failure_point input -pin_type {clock | data | latch | all} -
threshold_type failure -value float
```

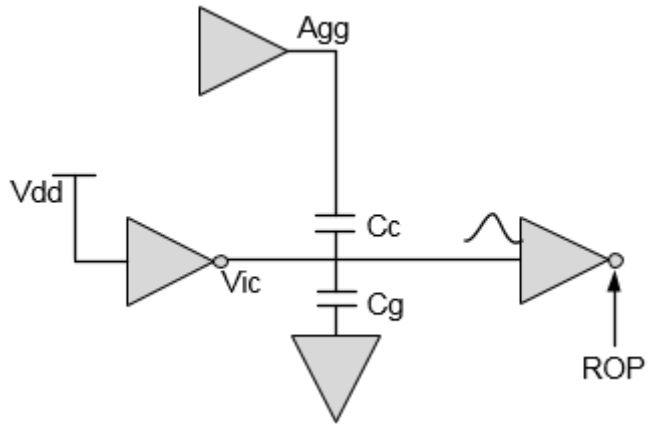
If the arc is from an input to an internal node of a sequential cell, then failure will occur if ROP is greater than the value specified using the following command:

```
set_db si_glitch_receiver_latch_peak_limit float
```

or

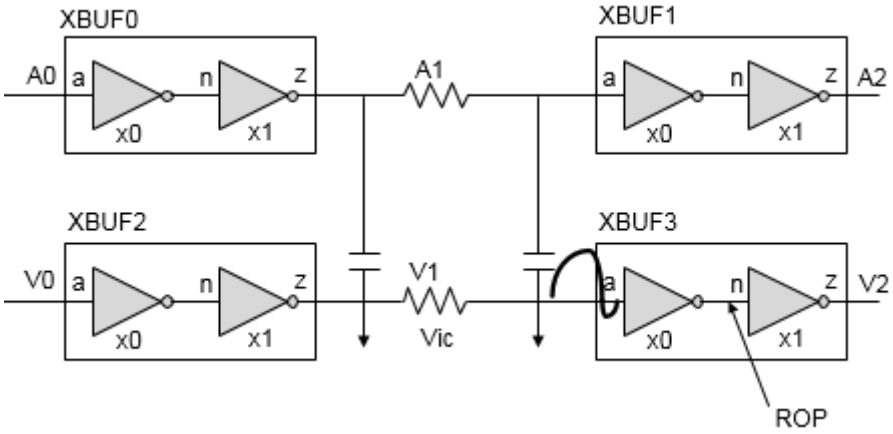
```
set_glitch_threshold -failure_point last -pin_type latch -threshold_type failure
-value float
```

Figure 4-47 ROP (Receiver Output Peak) for single stage cell



In case of single stage receiver cell, as shown in the figure above for Victim Vic, ROP will be performed at the output of receiver cell.

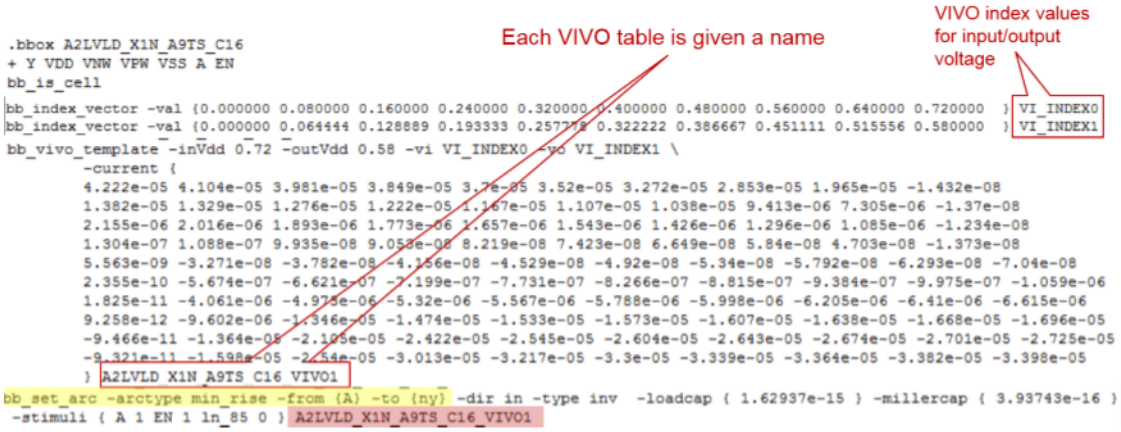
Figure 4-48 ROP (Receiver Output Peak) for multi-stage cell



While in case of multi-stage cell, as shown in the figure above for Victim V1, ROP will be performed at the output of the first stage of receiver cell XBUF3.

ROP is computed using the VIVO (Voltage In Voltage Out) current tables from the noise library (cdB), ECSM-N library, or CCS-N library. VIVO tables are characterized for single CCC. Tempus uses VIVO information, stored for the corresponding input pins, for ROP computation.

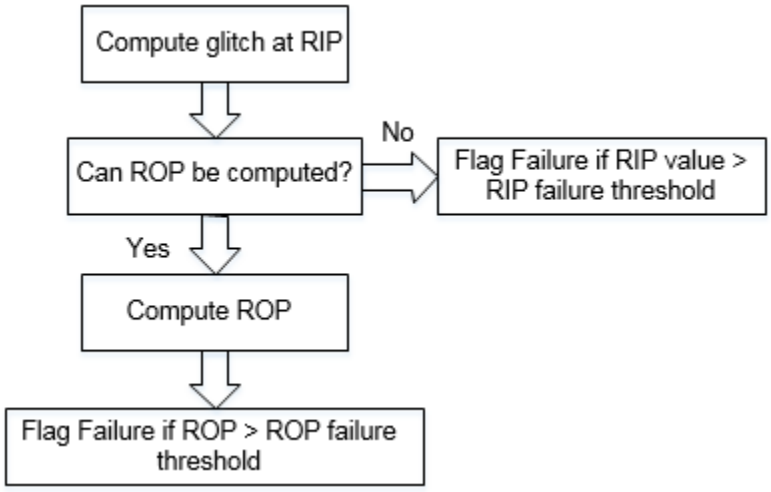
Figure 4-49 VIVO model



Glitch Failure Flow

The Glitch Failure Flow diagram is given below.

Figure 4-50 Glitch Failure Flow



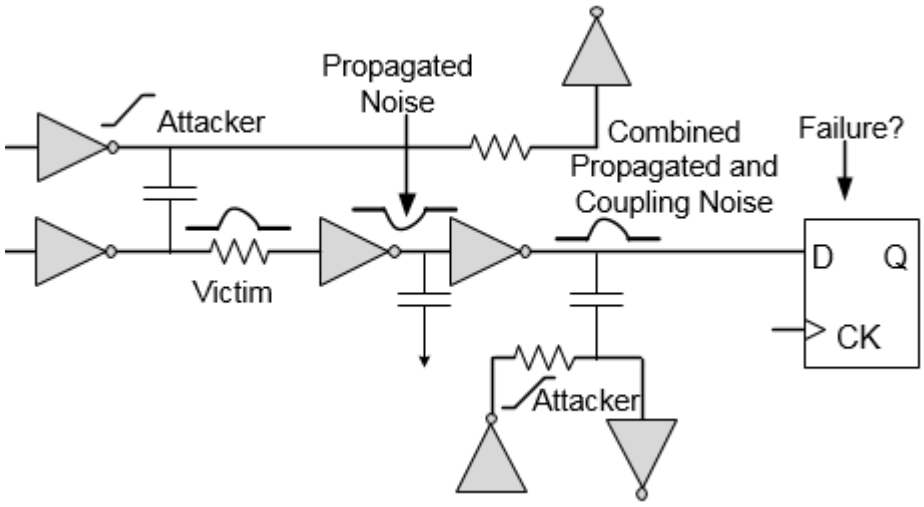
Glitch Propagation

Tempus performs glitch propagation that:

- Allows glitch to propagate and combine with coupling glitch noise along the path.
- Models driver weakening due to glitch coming at the input pin. Due to glitch at driver input, the current capability of the driver reduces leading to larger glitch at the output.

The following figure illustrates the glitch propagation feature:

Figure 4-51 Glitch Propagation



Driver weakening is needed for single-stage cells, not for multi-stage cells. In single-stage cells, when ROP is not failing, driver weakening may affect the next stage if the coupling on the next stage is big enough to be kept. In multi-stage cells, there is no coupling internal to the cell so glitch propagation does not need to be taken into account. This is because a non-failing glitch is not likely to propagate to the output of the cell.

Single-Stage Glitch Propagation and Driver Weakening

If glitch propagation is enabled, Tempus will propagate the glitch through single-stage cells as long as it is not failing. A glitch is deemed failing when the receiver output peak is greater than the threshold value specified using the following command:

`set_db si_glitch_receiver_peak_limit (or si_glitch_receiver_clock_peak_limit, si_glitch_receiver_latch_peak_limit)`

The failing receiver output peaks are not propagated. Even if the input glitch is not large enough to propagate to the output of the single-stage cell, it weakens the driver such that when the output of the cell is attacked, the coupling noise is much larger. Tempus models driver weakening when glitch propagation is enabled.

Single-stage glitch propagation and driver weakening is enabled automatically when using the `update_glitch` command or by specifying the `set_db si_enable_glitch_propagation true` command before using the `report_timing` or `update_timing` commands.

Failure Detection with Enabled Noise Propagation

Glitch impact computation starts with the first net in the path and traced downstream. A stage is analyzed taking into account the propagated glitch from last stage and the coupling of current stage. You can compare the total glitch seen at the receiver with the failure threshold. If the glitch exceeds failure threshold, then failure is flagged at the receiver and glitch impact from this stage is not propagated further to downstream logic. If the glitch is less than failure threshold, then the glitch impact at this stage is propagated to downstream logic.

4.2.10 Running Detailed SI Glitch Analysis

To run detailed glitch analysis, you can use the `update_glitch` command. This command performs the following steps in multiple iterations:

Iteration 0

This iteration is performed to estimate slews on all the nets.

Iteration 1

- Identify the attackers which are coupled to the victim net.
- Analyze each attacker individually for glitch peak on victim.
- Determine the attackers which can have glitch peak greater than a threshold of victim voltage.
- Estimate the combined impact of all such attackers, assuming infinite timing window, on victim net for glitch.
- Update the arrival times and compute slew values for all the nets in the design.

Iteration 2:

- Select the victim nets. A net will be reselected if any of the following criteria is met:
  - the net is violating
  - the net has a glitch greater than 30% VDD
  - the net has a glitch smaller than 30%VDD, but is capable of propagating downstream.

  Only those nets that are non-violating, have a glitch smaller than 30%VDD and are incapable of propagating, will not be reselected in the 2nd iteration.
- Analyze each attacker individually for glitch peak using slew/timing windows computed above. If the `set_db si_use_infinite_timing_window true` setting has been specified, then infinite timing windows is used in this iteration also.
- Determine the attackers which can produce glitch peak greater than the threshold of victim voltage.
- Identify the attackers which can produce worst impact on the victim net depending on timing window overlap, attacker slew, and other constraints.
- Perform combined simulation with identified attackers and peak aligned to compute the worst glitch at receiver input of victim net.

Failure Criteria Determination

- If a net has a receiver without noise data, then the receiver input peak (RIP) check is done.
- If a net has receiver with noise data, then the software propagates the noise one stage and receiver output peak (ROP) check is done.

4.2.11 SI Glitch Reporting

You can use the report_noise command to generate glitch reports. The -format parameter can be used to control the fields to be displayed in a glitch report.

Given below is a sample report generated using the report_noise command.

```
Glitch Violations Summary :
--------------------------
Number of DC tolerance violations (VH + VL) =  0
Number of Receiver Output Peak violations (VH + VL) =  0
Number of total problem noise nets =  0
-------------------------------------------------------------------------------
Peak(mV)  Level  TotalCap(fF)  TotalArea  Width(ps)  Vdd(mV)    Library  VictimNet
137.165    VL         10.10       5.94     86.640    935      ff3sigma_0p935v_125c
U2307/B {rd_r_2x_com[211]}


Receiver output peak:
Status    Slack      Value     ReceiverPin    ReceiverCell  cell_type
Arc                     ReceiverThresholdType
MET     280.500  0.000   (280.500)     U2307ny    (OR2_X8M_A12TS_C35)  [data]   (A
0 B 1)    Global


Constituents:
Source Peak(mV)  Offset(ps) Slew(ps) Xcap(fF) Vdd(mV) Edge Status     Net
Cpl:   42.027    0.000      10.375   0.446    935     R   ACT  rd_r_o_327_AT_1522
Cpl:   30.368    0.000      10.625   0.321    935     R   ACT  rd_r_o_323_AT_1526
Cpl:   23.085    0.000      11.250   0.253    935     R   ACT  rd_r_o_343_AT_1506
Cpl:   21.274    0.000      33.250   0.376    935     R   ACT  rd_r_2x_com[298]
Cpl:   20.369    0.000      30.000   0.333    935     R   ACT  rd_r_2x_com[235]
-------------------------------------------------------------------------------
```

The report sections and columns are explained below.

The top section Glitch Violations Summary shows a summary of each type of violation count.

- **Peak** value 1071.972 mV is the glitch at the receiver input pin (RIP).
- **Level** can be VH or VL, which shows the type of glitch at receiver input pin. VH is glitch under power rail and VL is glitch over ground rail.
- The pin name xv5/A below **VictimNet** is the receiver input pin having glitch.
- **Receiver output peak** (ROP) value 1226.940 is the glitch at the receiver output and 396.000 is the failure threshold for ROP. NOR4_F4 is the receiver library cell.
- **Constituents** provide the details of attackers.
  - **Source** column: Reports propagated noise as “Prp”.

- **Status** column: The Status column shows the abbreviated attacker status and filtered reason, as given below:
  SY: Synchronous
  INF: Attacker with infinite timing window
  PE: Filtered due to physical exclusion
  UF: Filtered by user
  SB: Filtered due to small bump
  TA: Filtered because of timing window does not overlap
  LC: Filtered due to logical correlation
  ACT: Attacker is Active.

■ The following sample report shows the DC tolerance violations:

```
-------------------------
Glitch Violations Summary :
-------------------------
Number of DC tolerance violations (VH + VL) =  0
Number of Receiver Output Peak violations (VH + VL) =  0
Number of total problem noise nets =  0
-------------------------------------------------------------------------

Peak(mV)  Level  TotalCap(fF)  TotalArea  Width(ps)  Vdd(mV)    Library  VictimNet
130.166    VH         11.68      13.49    207.278    740      ssg3sigma_0p74v_0c
U2307/B {rd_r_2x_combined[211]}
Receiver Input Threshold:
Status    Slack     Value (Threshold)     ReceiverNet    Receiver
failure_type    ReceiverThresholdType
MET    165.834  130.166 (296.000)  U2307/B   (OR2_X8M_A12TS_C35)  [bbox]   Global
Constituents:
Source Peak(mV) Offset(ps) Slew(ps) Xcap(fF) Vdd(mV) Edge Status  Net
Cpl:   40.820   0.000      26.125   0.609    740     F    ACT  rd_r_o_327_AT_1522
Cpl:   30.238   0.000      26.250   0.452    740     F    ACT  rd_r_o_323_AT_1526
Cpl:   23.911   0.000      26.750   0.360    740     F    ACT  rd_r_o_343_AT_1506
Cpl:   18.158   0.000      75.125   0.491    740     F    ACT  rd_r_2x_com[298]
Cpl:   17.013   0.000      77.875   0.473    740     F    ACT  rd_r_2x_com[235]
-------------------------------------------------------------------------------
```

In this report glitch at receiver input pin u9/D is more than the threshold.

■ The report_noise -style extended option can be used to report additional information, such as, timing windows of attackers. A sample report is shown below:

```
tempus > report_noise -style extended
-------------------------
Glitch Violations Summary:
-------------------------
Number of DC tolerance violations (VH + VL) =  0
Number of Receiver Output Peak violations (VH + VL) =  0
Number of total problem noise nets =  0
-------------------------------------------------------------------------------
Peak(mV)  Level  TotalCap(fF)  TotalArea  Width(ps)  Vdd(mV)    Library  VictimNet
```

```
329.868    VL         39.89      30.87    187.173    935      ff3sigma_0p935v_125c
u1_lpu/u0_logic/u0_ud_udm/wdata_rw_q1_77_ATd_i29398/A {u1_lpu/u0_logic/u0_ud_udm/
wdata_rw_q1_77_ATd_401797}


Receiver output peak:

Status   Slack   Value  ReceiverPin  ReceiverCell   cell_type
Arc      ReceiverThresholdType

MET       264.418   16.082  (280.500)     u1_lpu/u0_logic/u0_ud_udm/
wdata_rw_q1_77_ATd_i29398/ny     (DLY2_X4M_C35)    [data]    (A 1)   Global

Stage information:

            Cell         Pin

Driver   :  DLY2_X4M_C35 u1_lpu/u0_logic/u0_ud_udm/wdata_rw_q1_77_ATd_i20211/Y
Receiver :  DLY2_X4M_C35 u1_lpu/u0_logic/u0_ud_udm/wdata_rw_q1_77_ATd_i29398/A

Constituents:

Source Peak(mV) Offset(ps)  Slew(ps) Xcap(fF) Vdd(mV) Edge Status Net
Cpl:   130.793   31.000     13.375   5.128    935     R   ACT  sb_r2_o_1_AT_201
Cpl:   105.063   51.000     15.250  11.340    935     R   ACT  ad_r0_o_8_AT_193
Cpl:   76.101    38.000     11.750   4.427    935     R   ACT  wd_rw_o_163_AT_194
Cpl:   17.846    38.000     12.500   1.426    935     R   ACT  wd_rw_o_210_AT_141

Timing Windows: (NanoSecs)

MinTime  MaxTime Edge  Clock Net

0.113    0.265   F     CCLK  u1_lpu/u0_logic/u0_ud_udm/wdata_rw_q1_77_ATd_401797
0.970    1.320   F     NULL  u1_lpu/u0_logic/u0_ud_udm/wdata_rw_q1_77_ATd_401797
1.398    1.486   R     CCLK  sb_r2_o_1_AT_201
1.359    1.449   R     CCLK  ad_r0_o_8_AT_193
1.372    1.432   R     CCLK  wd_rw_o_163_AT_194
1.366    1.447   R     CCLK  wd_rw_o_210_AT_141
--------------------------------------------------------------------------------
```

- You can use the report_noise -histogram parameter to print a violation summary and histogram of RIP/ROP. A detailed glitch report is not printed.

```
tempus > report_noise -histogram -out_file rip_rop.hist

--------------------------

Glitch Violations Summary :
--------------------------
Number of DC tolerance violations (VH + VL) =  0
Number of Receiver Output Peak violations (VH + VL) =  0
Number of total problem noise nets =  0


Receiver Input Peak Histogram for View : slow_corner_hold
Percentage of Receiver Vdd      No. of Receivers
[ 0,        10          )       0
[ 10,       20          )       173213
[ 20,       30          )       41788
[ 30,       40          )       131
[ 40,       50          )       0
[ 50,       60          )       0
[ 60,       70          )       0
[ 70,       80          )       0
[ 80,       90          )       0
[ 90,       100         )       0
```

```
Receiver Output Peak Histogram for View : slow_corner_hold
Percentage of Receiver Vdd      No. of Receivers
[ 0,        10          )       0
```

Exceptions

Here are some exceptions:

The following command helps in specifying certain attackers as quiet for the victim net. This implies that the net mentioned in this command will not be considered as an attacker for a particular victim net, or all victim nets.

- set_quiet_aggressor

You can use the following parameter to specify a list of victim/attacker pairs to be made quiet:

```
set_quiet_aggressor -victim string
```

If the `-aggressor` parameter is not used, then all the attackers are made silent for the specified victim.

You can use the following parameter to specify the victim/attacker pair to be made quiet:

```
set_quiet_aggressor -aggressor string
```

If the `-victim` parameter is not used, then the given nets are made silent (as attackers) for all the victim nets in the design.

```
tempus > set_quiet_aggressor  -aggressor wdata_rw_o_163_AT_194 -victim u1_ud_lpu/
u0_ud_lpu_logic/u0_ud_udm/wdata_rw_q1_77_AT_401
tempus > update_glitch
tempus > report_noise -nets u1_ud_lpu/u0_ud_lpu_logic/u0_ud_udm/
wdata_rw_q1_77_AT_401
-------------------------------------------------------------------------
Glitch Violations Summary :
---------------------------
Number of DC tolerance violations (VH + VL) =  0
Number of Receiver Output Peak violations (VH + VL) =  0
Number of total problem noise nets =  0
-------------------------------------------------------------------------------
Peak(mV)  Level  TotalCap(fF)  TotalArea  Width(ps)  Vdd(mV)    Library  VictimNet
 247.868    VH        39.89      25.00    201.718    935      ff3sigma_0p935v_125c
u1_ud_lpu/u0_ud_lpu_logic/u0_ud_udm/umh_udm_wdata_rw_q1_77__ATdrc_i29398/A
{u1_ud_lpu/u0_ud_lpu_logic/u0_ud_udm/wdata_rw_q1_77_AT_401}
Receiver output peak:
Status    Slack   Value    ReceiverPin   ReceiverCell   cell_type   Arc   Receive
rThresholdType
MET       277.976   2.525  280.5   u1_ud_lpu/u0_ud_lpu_logic/u0_ud_udm/
wd_rw_q1_77_AT_i29/ny   (DLY2_X4M_A12TS_C35)     [data]   (A 1)   Global
Constituents:
Source Peak(mV)  Offset(ps) Slew(ps)  Xcap(fF) Vdd(mV) Edge Status Net
Cpl:   123.339   35.000     13.375    5.128    935     F    ACT  sb_r2_o_1_AT_201
```

```
Cpl:   106.286   53.000     15.125    11.340   935      F   ACT  ar_r0_o_8_AT_193
Cpl:   18.218    39.000      13.000   1.426   935      F   ACT  wd_rw_o_210_AT_14
-------------------------------------------------------------------------------
```

In the above noise report, status of attacker `attc4` is changed to UF (User Filtered).

- set_annotated_glitch
  - The following parameter allows you to specify the port/pin name at which glitch needs to be annotated:

    `set_annotated_glitch -ports`

    If a glitch is annotated at the driver output/input port, then it is combined with the coupling of the stage. If a glitch is annotated at the receiver input, then it overrides the glitch at receiver input with the annotated glitch.

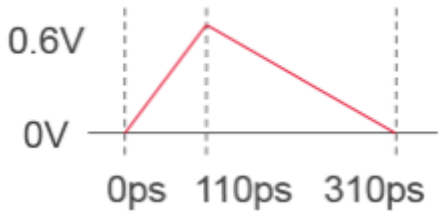
  - The following parameters allow you to specify the type of glitch waveform (vl or vh) to be annotated:

    `set_annotated_glitch -voltage_low/-voltage_high` {waveform with time stamp and voltage value}

    The `voltage_high` waveform should also be specified starting from “0”. The software automatically inverts the waveform depending on receiver voltage.

    The waveform is specified as PWL with multiple time and voltage points, such as, `{time-1 voltage-1 time-2 voltage-2…..}`

    The `set_annotated_glitch -voltage_low {0 0 110e-12 0.6 310e-12 0}` command defines glitch as shown below:



    For VH, the wave should be defined starting from 0V, the software automatically inverts it when applying according to the victim voltage.

For victim voltage 1V, the set_annotated_glitch -voltage_high {0 0 100e-12 0.6 320e-12 0} command defines glitch, as shown below:



Here's a sample report:

```
tempus > set_annotated_glitch -voltage_low {0 0 100e-12 0.1 200e-12 0.4 300e-12 0.3
600e-12 0} -port scan_in_0_i[5]
tempus > update_glitch
tempus > report_noise -nets scan_in_0_i[5]

--------------------------
Glitch Violations Summary :
--------------------------
Number of DC tolerance violations (VH + VL) =  1
Number of Receiver Output Peak violations (VH + VL) =  0
Number of total problem noise nets =  1
-------------------------------------------------------------------------------
Peak(mV)  Level  TotalCap(fF)  TotalArea  Width(ps)  Vdd(mV)  Library  VictimNet
625.818   VL     2.40          93.65      299.284    740      ssg3sigma_0p74v_0c
sc_in_0_i_5__ATibuf_i13381/A {sc_in_0_i[5]}


Receiver Input Threshold:
Status    Slack    Value (Threshold)    ReceiverNet    Receiver  failure_type
ReceiverThresholdType
VIOLATED   -329.818   625.818 (296.000)   sc_in_0_i_5__ATibuf_i13381/
A    (BUFH_X2M_A12TS_C35)    [bbox]    Global


Constituents:
Source  Peak(mV)  Offset(ps)  Slew(ps)  Xcap(fF)  Vdd(mV)  Edge Status Net
Cpl:    25.784    0.000       25.000    0.595     740      R    ACT    sc_in_0_i[6]
Prp:    599.992   --          --        --        --       --          sc_in_0_i[5]
*******************************************************************************
```

The software overrides the glitch value showing up at RIP with the annotated glitch, as glitch is annotated at the receiver input.

set_noise_lib_pin

This command helps in mapping noise properties of a library cell pin to other library cell pins. The software copies the noise properties of a buffer type cell for a macro cell pin, for which a noise property has not been defined.

- set_noise_lib_pin -from <>

Specify the `from_library/from_cell/from_pin`.

- `set_noise_lib_pin -to <>`

Specify the `to_library/to_cell/to_pin`.

For example,

```
set_noise_lib_pin -from cell_w_1.6.lib/CLKBUF/A -to memory.lib/MEM_256x32/
A
```

The software will map the noise properties from cdB linked to the specified library, even if there is a difference in voltage values. You must ensure that the voltage values are similar while mapping the properties.

Table 4-2 Impact of Timing Exceptions on Glitch Analysis.

Timing Exceptions	Impact on Glitch in Infinite Timing Window Flow
set_case_analysis	Attacker will be marked as silent, if constant value is input. The net will be analyzed as victim even with the constant value.
set_disable_timing	Impact due to change in attackers slew, if attacker is lying in the fanout path of disable timing.
set_false_path	No impact
set_multicycle_path	No impact
set_clock_uncertainty	No impact
set_clock_groups	Impact on noise results due to filtering of some attackers.

Impact of Timing Constraints on Glitch Analysis in Infinite TW Mode

- set_input_transition

  This input transition command defined at the input ports, serves as a starting point to compute at slew at all nets. If its not defined, the software uses a fast slew of 5ps at input ports.

- set_driving_cell

  This command is required to model holding resistance accurately, if there is coupling at the net connected to the input port.

- set_load

This command sets the load at the output load. This impacts glitch computation on the net connected at the output port.

Checking Noise Models

You can use the `check_noise` command before the `update_glitch` command to check consistency and completeness of noise models specified for a design. These checks help to ensure that the right set of models are available to perform SI analysis.

Library Requirement for Glitch Analysis

Tempus supports cdB, ECSM-N, and CCS-N library for glitch analysis. The key advantage of moving to ECSM-N/CCS-N is both use the same library for timing and glitch analysis. The cdB library is used only for glitch analysis and a separate timing library is required for timing analysis.

4.2.12 Overshoot-Undershoot Glitch Analysis - An Overview

The Tempus software allows you to perform overshoot and undershoot analysis. The life span of transistors can be described by the overshoot and undershoot values. If a transistor is consistently exposed to a certain percentage of overshoot/undershoot, then it may suffer a reduced life span. Tempus analyzes and generates reports for such cases.

Due to cross-coupled capacitance between interconnects, switching activity of one net (attacker) can induce undesirable noise (glitch) on a static "victim" net. This glitch can be classified into four categories based on the victim state and attackers' switching direction, as shown in the table below.

Table 4-3 Classification of glitch

Glitch Type	Victim State	Attacker Switching
Glitch over ground rail (VL)	Low	Rising
Glitch under power rail (VH)	High	Falling
Glitch under ground rail (VLU)	Low	Falling
Glitch over power rail (VHO)	High	Rising

The glitch types VL and VH, mentioned in Table 1, can result in functional failures, if the glitch exceeds the logic threshold.

The glitch types, VLU and VHO, are known as undershoot and overshoot, respectively. While VLU (undershoot) and VHO (overshoot) do not cause any immediate silicon failure, exposure to VLU/VHO glitch over a period of time causes degradation of the gate-oxide, which increases the threshold voltage (Vth), and in turn can cause silicon failure.

The following figures indicate the type of glitch waveform:

Figure 4-52 VHO Glitch

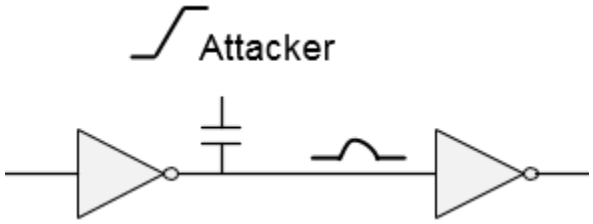


Figure 4-53 VLU Glitch

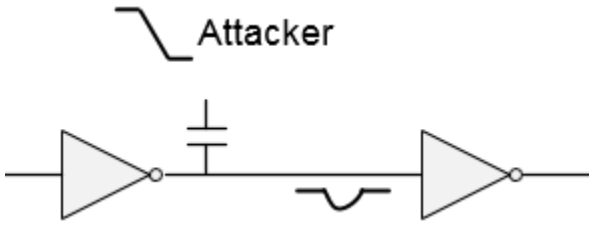
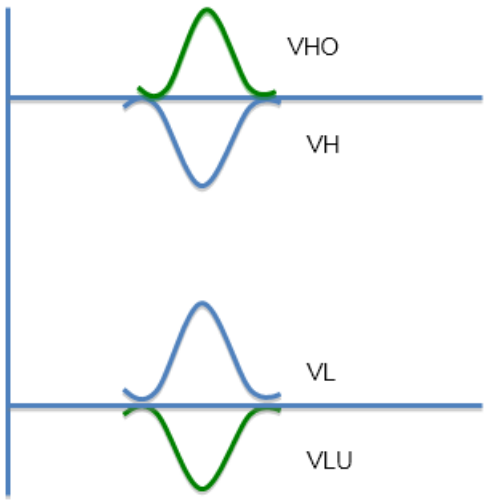


Figure 4-54 VL, VH, VLU, and VHO glitch waveforms



Tempus performs overshoot-undershoot glitch analysis by analyzing each potential victim net in the design. The analysis of each net is based on calculating the worst-case glitch waveform that can occur at the input pins of each of the victim nets' receivers.

The VLU/VHO type of noise cannot propagate through a transistor, and therefore, Tempus does not perform the glitch propagation and ROP (Receiver Output Peak) computation in overshoot-undershoot analysis.

Overshoot-Undershoot Analysis Flow

To run overshoot-undershoot analysis, perform the following steps:

- Setup the design for SI analysis.
- Enable overshoot-undershoot analysis:

  `set_db si_enable_glitch_overshoot_undershoot`
- Set failure thresholds:

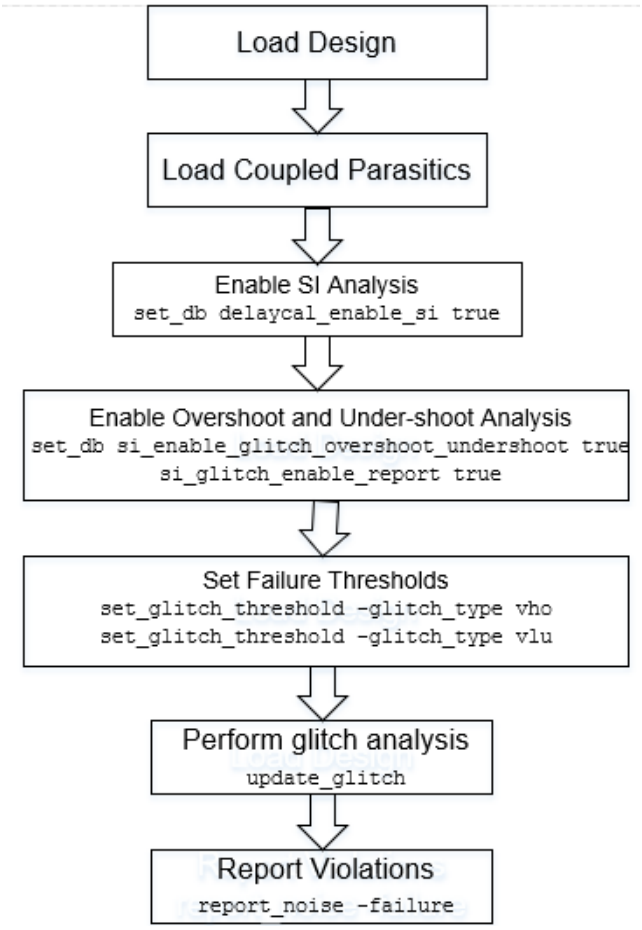
  `set_glitch_threshold -glitch_type vho/vlo`
- Analyze overshoot and undershoot glitches:

  `update_glitch`

You can also use the `update_timing` and `report_timing` commands for overshoot/undershoot glitch analysis and reporting.

The following figure shows the overshoot-undershoot analysis flow.

Figure 4-55 Overshoot-Undershoot Analysis Flow



Reporting

The reporting structure of the overshoot-undershoot analysis is similar to that of traditional glitch analysis.

Example VLU report

```
-------------------------------------------------------------------------------
Peak(mV)   Level   TotalCap(fF)    TotalArea   Width(ps)   Vdd(mV)   Library   VictimNet
310.282    VLU     50.91           29.49       190.083     740       ssg3sigma_0p74v_0c
u1_ud_lpu/u0_ud_lpu_logic/u0_ud_udm/umh_udm_wdata_rw_q1_77__ATdrc_i293987/A
{u1_ud_lpu/u0_ud_lpu_logic/u0_ud_udm/umh_udm_wdata_rw_q1_77__ATdrc_401797}


Receiver Input Threshold:
Status    Slack     Value
(Threshold)     ReceiverNet     Receiver    failure_type    ReceiverThresholdType
```

```
VIOLATED   -236.282   310.282   (74.000)  u1_ud_lpu/u0_ud_lpu_logic/u0_ud_udm/
umh_udm_wdata_rw_q1_77__ATdrc_i29398/A    (DLY2_X4M_A12TS_C35)   [bbox]    Global


Constituents:
Source  Peak(mV) Offset(ps) Slew(ps) Xcap(fF) Vdd(mV) Edge Status Net
Cpl:    117.124  32.000     26.500   7.246    740     F    ACT  sb_r2_o_ATr_201
Cpl:    111.941  52.000     30.875   16.054   740     F    ACT  ad_r0_o_8_ATm_19
Cpl:    64.862   36.000     29.500   5.680    740     F    ACT  rw_o_16_ATm_19
Cpl:    16.355   42.000     32.375   1.928    740     F    ACT  rw_o_21_ATr_14
--------------------------------------------------------------------------------
```

Example VHO report

```
--------------------------------------------------------------------------------
Peak(mV)  Level  TotalCap(fF)  TotalArea  Width(ps)  Vdd(mV)    Library  VictimNet
386.155   VHO         45.02      33.59    173.967    935      ff3sigma_0p935v_125c
rw_o_82__ATobuf_i14769/A {rw_o_82_ATr_214587}

Receiver Input Threshold:
Status   Slack   Value (Threshold)   ReceiverNet    Receiver
   failure_type   ReceiverThresholdType
VIOLATED  -12.155   386.155  (374.000)   rw_o_82__ATobuf_i14769/A
(BUFH_X7P5M_A12TS_C35)   [bbox]    Global

Constituents:
Source  Peak(mV) Offset(ps) Slew(ps) Xcap(fF)  Vdd(mV) Edge Status Net
Cpl:    214.724  26.000     8.625    14.204    935     R    ACT  rw_o_45_ATr_141
Cpl:    151.118  20.000    11.250     6.460    935     R    ACT  rw_o_136_ATr_141
Cpl:    20.314   15.000     9.750     2.056    935     R    ACT  rw_o_4_ATr_14186
Cpl:     0.000    0.000    14.000     2.909    935     R    SB   rw_o_45_ATr_1235
Cpl:     0.000    0.000    12.000     1.679    935     R    SB   rw_o_136_ATd_380
--------------------------------------------------------------------------------
```

The `report_noise` command parameters, such as, `-failure`, `-format`, `-histogram`, `-nets`, `-style`, `-threshold`, `-out_file`, and `-view` work with the overshoot-undershoot reporting as well.

For more information, refer to *Tempus Text Command Reference Guide*.

4.2.13 Running Overshoot-Undershoot Glitch Analysis - Sample Script

The following is a sample script to run overshoot-undershoot glitch analysis:

- Load the timing libraries (.lib) and characterized data (.cdB):

  ```
  read_libs -noise_libs typ.cdB
  ```

  Note: This sample script shows the usage of cdb noise data, however, this feature supports CCS-N and ECSM noise models as well.

- Schedule reading of a structural, gate-level verilog netlist:

  ```
  read_netlist top.v
  ```

- Set the top module name, and check for consistency between Verilog netlist and timing libraries:

  ```
  init_design top
  ```

- Load the timing constraints:

  ```
  read_sdc constraints.sdc
  ```

- Load the cell parasitics:

  ```
  read_spef top.spef.gz
  ```

- Enable signal integrity (SI) analysis:

  ```
  set_db delaycal_enable_si true
  ```

- Enable the Overshoot-Undershoot Analysis:

  ```
  set_db si_enable_glitch_overshoot_undershoot true
  si_glitch_enable_report true
  ```

- Set the failure threshold for the VLU and VHO type glitch:

  ```
  set_glitch_threshold -glitch_type vlu -value 0.3
  set_glitch_threshold -glitch_type vho -value 0.3
  ```

Or

- Set the failure threshold globally for both VLU and VHO by using the following command:

  ```
  set_db si_glitch_input_threshold 0.3
  ```

  In the above commands, the value specified is the ratio to VDD, that is, failure will occur if the overshoot/undershoot glitch height is greater than the mentioned threshold value times VDD. The default value is 0.4.

  If both `set_glitch_threshold` and `set_db si_glitch_input_threshold` commands have been specified, then the `set_glitch_threshold` command setting will take the precedence.

- Perform overshoot-undershoot glitch analysis:

  ```
  update_glitch
  ```

- Generate the overshoot/undershoot glitch report:

  ```
  report_noise -failure -out_file vlu_vho.rpt –overshoot_undershoot
  ```

Here, only the VLU/VHO type of glitches will be printed in the report. For VL/VH type of glitch numbers, you can re-run the `report_noise` command without the `-overshoot_undershoot` parameter.

4.3 Timing Analysis Modes

4.3.1 Overview on page 126

4.3.2 Specifying Timing Analysis Mode Types on page 127

4.3.3 Advanced On-Chip Variation (AOCV) Analysis on page 145

4.3.4 Statistical On-Chip Variation (SOCV) Analysis on page 170

4.3.1 Overview

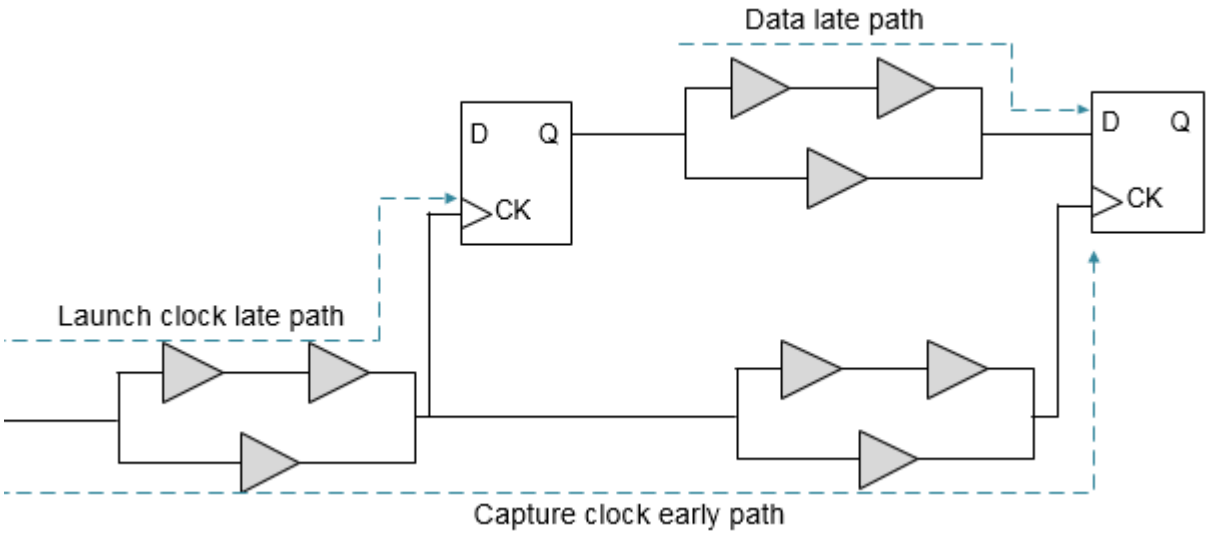
The Tempus software provides different timing analysis modes and accordingly performs calculations for setup and hold checks for each mode. For a better understanding of these modes, it is important to understand the impact of early and late paths in a design, as explained in the following section.

Definition of Early and Late Paths

Early and late paths are referred to as the shortest and longest paths, respectively, and are used for delay calculations. The early or minimum path delay is the minimum delay through cells and nets in the path. The late or maximum path delay is the maximum delay through cells and nets in the path.

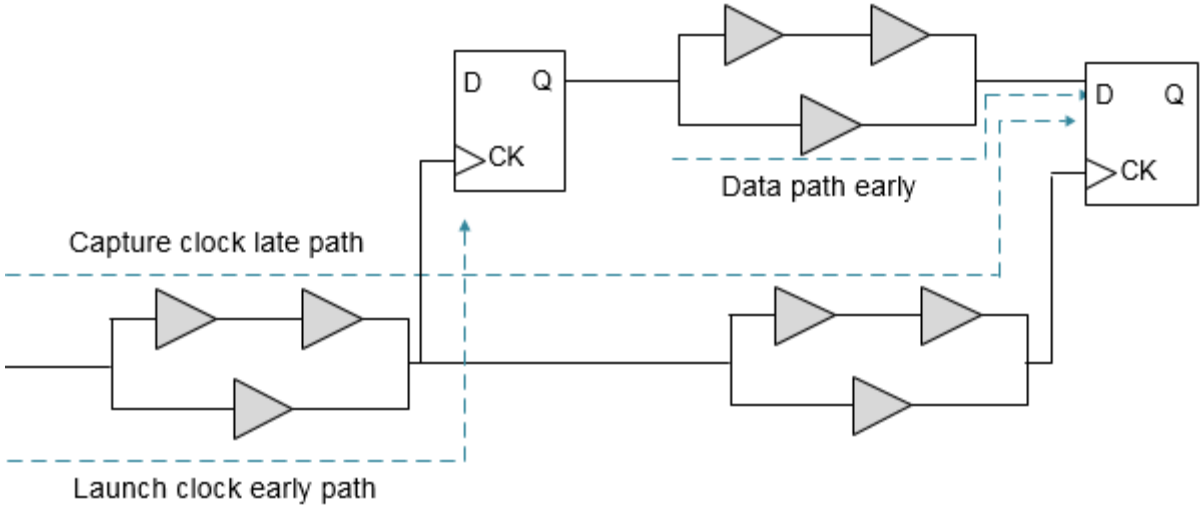
The following figure shows a setup check with late launch clock and early capture clock:

Figure 4-56 Illustration of Setup Check- Setup check with early and late paths



The following figure shows a hold check with early launch clock and late capture clock.

Figure 4-57 Illustration of Hold Check - Hold check with early and late paths



4.3.2 Specifying Timing Analysis Mode Types

The timing analysis mode types are:

- Single Timing Analysis Type Overview
- Best-Case Worst-Case (BC-WC) Timing Analysis Mode Overview
- On-Chip Variation (OCV) Timing Analysis Mode Type Overview

Single Timing Analysis Type Overview

In this mode, the Tempus software uses a single set of delays (using one library group) based on one set of process, temperature, and voltage conditions.

To set the timing analysis mode as single, you can use the following command:

```
set_db timing_analysis_type single
```

In single analysis mode, only maximum delay values are used for delay calculation. The `-max` options of commands in constraints are used for both minimum and maximum analysis in single analysis mode. For example, the `set_annotated_delay -max 10` command will be used for both minimum and maximum analysis.

Even with single library group, cell delay can have variation in maximum delay based on sdf_cond, timing derates, and other inputs. In single analysis mode, early and late path delays are the minimum and maximum, respectively, of this range of maximum delay.

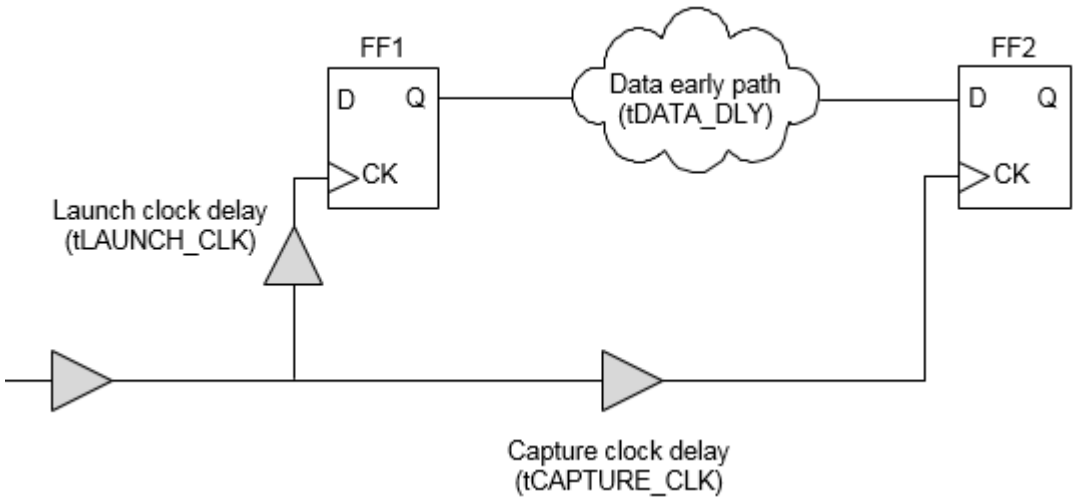
Single Timing Analysis - Setup and Hold Checks

These are explained in the sections below.

Setup Check in Single Timing Analysis Mode

For setup check, the software checks the late launch clock and late data paths against early capture clock path.

Figure 4-58 Setup Check in Single Timing Analysis Mode

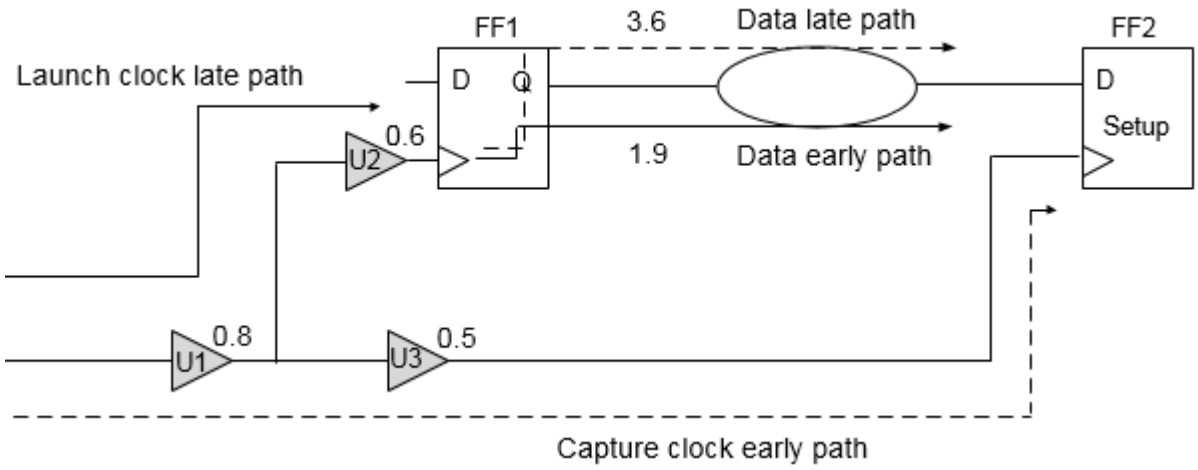


For zero slack value in a setup check, the following condition should be met:

launch clock late path (tLAUNCH_CLK) + *data clock late path* (tDATA_DLY) <= *capture clock early path* (tCAPTUE_CLK) + *clock period* - *setup*

Figure 4-59 on page 129 shows the setup check on a path from FF1 to FF2.

Figure 4-59 Setup Check in Single Timing Analysis Mode



The software uses a library to calculate the maximum delay. For setup check, the software considers two paths between the two registers, FF1 and FF2 - the late path delay is used to calculate slack during setup check.

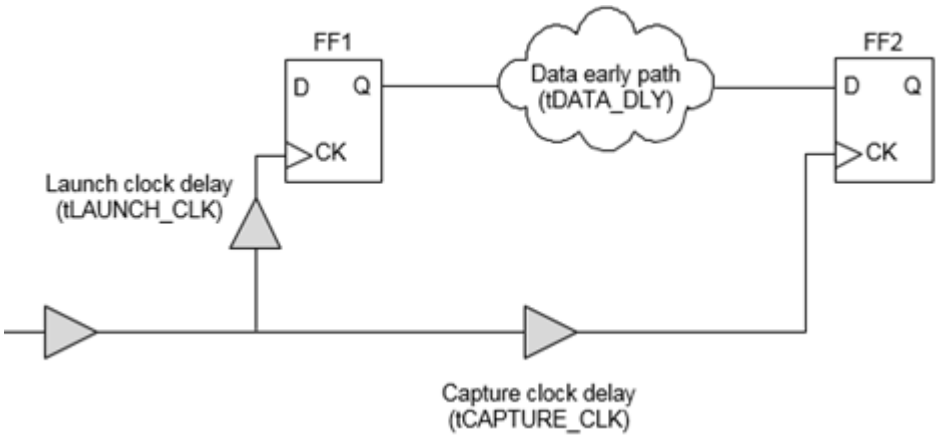
The following values are assumed in this example:
Clock period = 4; Clock source latency = none; Clock mode = Propagated

Data late path delay	3.6
Data early path delay	1.9
Launch clock late path delay	0.8 + 0.6 =1.4
Capture clock early path delay	0.8 + 0.5 = 1.3
Setup	0.2
Data arrival time	1.4 + 3.6 = 5
Data required time	4 + 1.3 - 0.2 = 5.1
Setup Slack	5.1 - 5 = 0.1

Hold Check in Single Timing Analysis Mode

For hold check the software compares the early arriving data against the late arriving clock at the endpoint.

Figure 4-60 Hold Check in Single Timing Analysis Mode

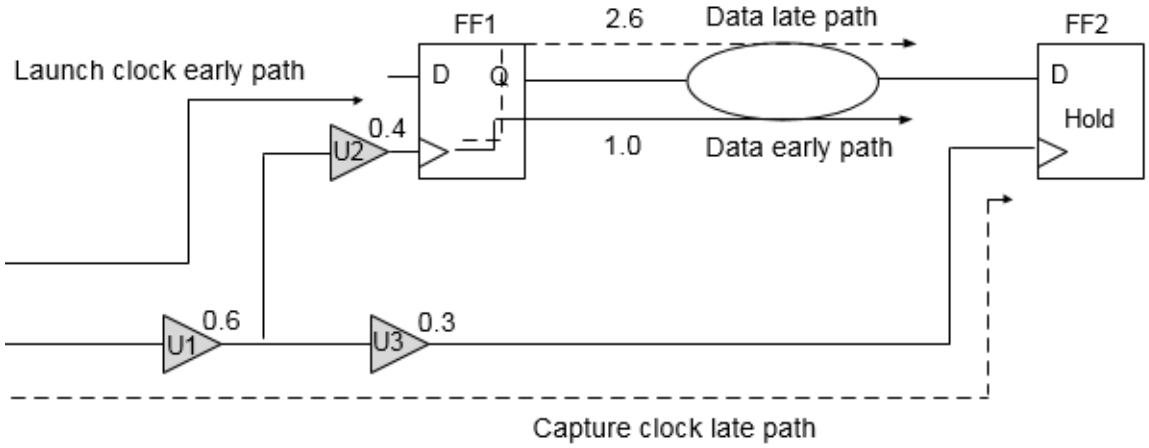


For zero slack value in a hold check, the following condition should be met:

launch clock early path (tLAUNCH_CLK) + *data early path* (tDATA_DLY) => *capture clock late path* (tCAPTUE_CLK) + *hold*

The following example shows the hold check on a path from FF1 to FF2.

Figure 4-61 Hold Check in Single Timing Analysis Mode



The following values are assumed in this example:

Clock period = 4; Clock source latency = none; Clock mode = Propagated

Data late path delay	2.6
Data early path delay	1.0

Launch clock early path delay	0.6 + 0.4 = 1.0
Capture clock late path delay	0.6 + 0.3 = 0.9
Hold	0.2
Data arrival time	1 + 1 = 2
Data Required Time	0.2 + 0.9 = 1.1
Hold Slack	2 – 1.1 = 0.9

Performing Timing Analysis in Single Analysis Mode

To perform timing analysis in single analysis mode, you need to perform the following steps:

- Set the analysis mode to single, setup and propagated clock mode:

  ```
  set_db timing_analysis_type single timing_analysis_check_type setup
  ```

- Generate the timing reports for setup:

  ```
  report_timing
  ```

- Set the analysis mode to hold:

  ```
  set_db timing_analysis_check_type hold
  ```

- Generate the timing reports for hold analysis:

  ```
  report_timing
  ```

Alternatively, you can use the `report_timing -early` command with the `set_db timing_analysis_check_type setup` attribute to report hold paths.

Best-Case Worst-Case (BC-WC) Timing Analysis Mode Overview

In best-case worst-case (BC-WC) analysis mode, the software uses delays from the maximum library group for all the paths during setup checks and minimum library group for all the paths during hold checks. In this mode, the Tempus software considers two operating conditions and checks both operating conditions in one timing analysis run. To set the timing analysis mode as BC-WC, you can use the following command:

```
set_db timing_analysis_type best_case_worst_case
```

You can use the set_clock_latency constraint to set the source latency for a clock in both ideal and propagated mode for setup and hold checks. You can also use this constraint to set the network latency for an ideal clock. The specified source or network latency affects the early and late clock paths for both capture and launch clocks for both minimum and maximum operating conditions.

Note: The software considers the network latency, specified using the `set_clock_latency -max` (or `-min`) command, for ideal clocks only. In propagated mode, actual clock propagated latency values will be used.

BC-WC Timing Analysis Mode - Setup and Hold Checks

These are explained in the sections below.

Setup Check in BC-WC Mode

For setup check, the software calculates delay values from the maximum library group for data arrival time, and network delay of both launch and capture clocks (in propagated mode).

The source latency in both ideal and propagated modes for setup checks is defined in the constraints used by various clock paths as follows:

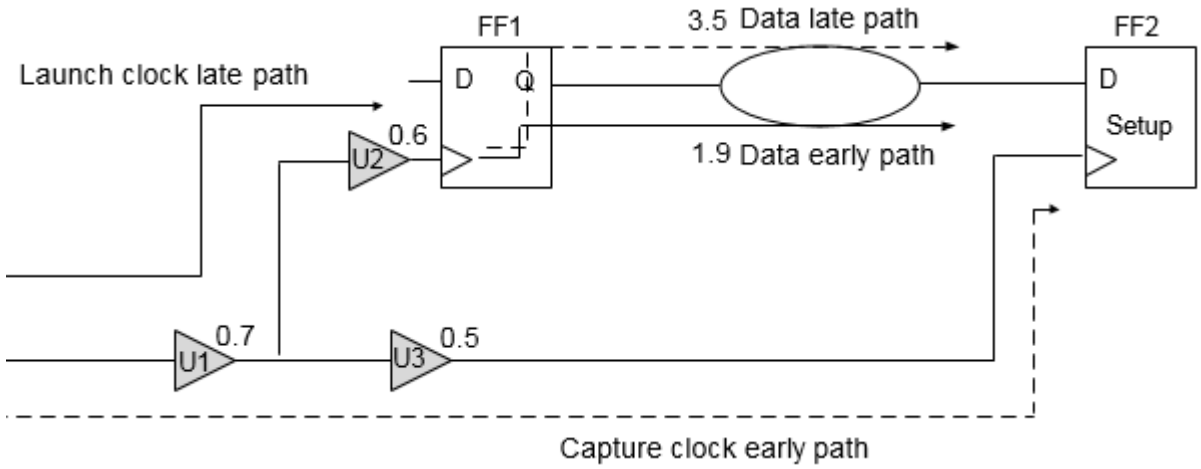
Clock Path (Operating Condition)	Constraint Used
Launch clock late path (max)	`set_clock_latency -source -late -max value`
Capture clock early path (max)	`set_clock_latency -source-early -max value`

The network latency in ideal mode for setup checks is defined in the constraints used by various clock paths as follows:

Clock Path (Operating Condition)	Constraint Used
Launch clock late path (max)	`set_clock_latency -max value`
Capture clock early path (max)	`set_clock_latency -max value`

The following example shows the setup check on a path from FF1 to FF2.

Figure 4-62 Setup Check in BC-WC Timing Analysis Mode



The software uses the max library to scale all delays at WC conditions. The following values are assumed in this example:

Clock period = 4; Clock source latency = none ; Clock mode = Propagated

Data late path delay	3.5
Data early path delay	1.9
Launch clock late path delay	0.7 + 0.6 = 1.3
Capture clock early path delay	0.7 + 0.5 = 1.2
Setup	0.2
Data arrival time	1.3 + 3.5 = 4.8
Data Required Time	4 + 1.2 - 0.2 = 5
Setup Slack	5 - 4.8 = 0.2

HOLD Check in BC-WC Mode

For hold check, the software uses the delay values from the min library for the data arrival time, and network delay of both launch and capture clocks (in propagated mode).

The source latency in both ideal and propagated modes for hold checks is defined in the constraints used by various clock paths as follows:

Clock Path (Operating Condition)	**Constraint Used**

Launch clock early path (min)	set_clock_latency -source -early -min value
Capture clock late path (min)	set_clock_latency -source -late -min value

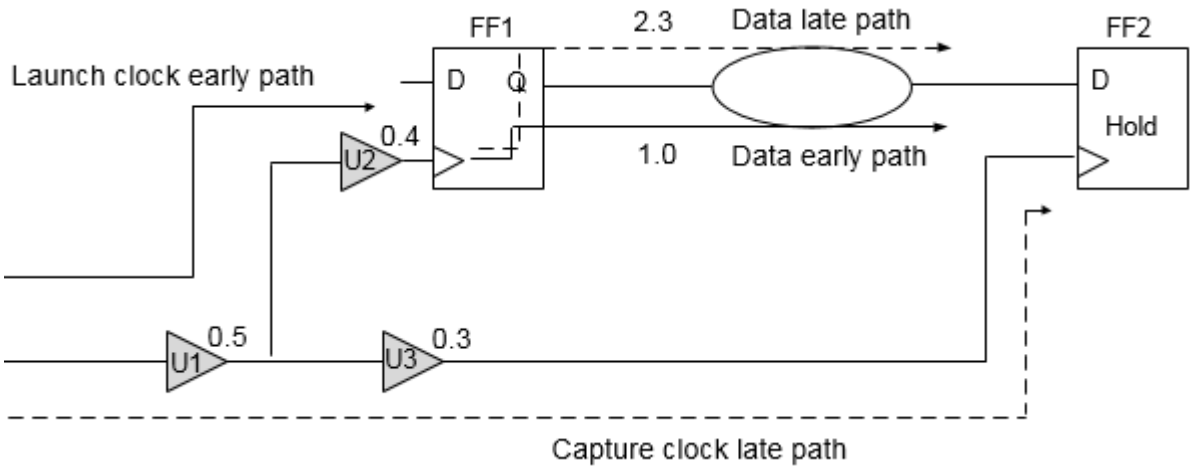
The network latency in ideal mode for hold checks is defined in the constraints used by various clock paths as follows:

Clock Path (Operating Condition)	**Constraint Used**
Launch clock early path (min)	set_clock_latency -min value
Capture clock late path (min)	set_clock_latency -min value

Note: You can also use one library containing two operating conditions in this mode.

The following example shows the hold check on a path from FF1 to FF2.

Figure 4-63 Hold Check in BC-WC Timing Analysis Mode



The software uses the min library to scale all delays at BC conditions.

The following values are assumed in this example:

Clock period = 4; Clock source latency = none; Clock mode = Propagated

Data late path delay	2.3
Data early path delay	1.0
Launch clock early path delay	0.5 + 0.4 = 0.9
Capture clock late path delay	0.3 + 0.5 = 0.8
Hold	0.1
Data arrival time	0.9 + 1 = 1.9
Data required time	0.1 + 0.8 = 0.9
Hold Slack	1.9 - 0.9 = 1

Performing Timing Analysis in BC-WC Analysis Mode

To perform timing analysis in BC-WC analysis mode, complete the following steps:

- Set the analysis mode to BC-WC:

  ```
  set_db timing_analysis_type best_case_worst_case
  timing_analysis_check_type setup
  ```

- Generate the timing reports for setup:

  ```
  report_timing
  ```

- Set the analysis mode to hold and propagated clock mode:

  ```
  set_db timing_analysis_check_type hold
  ```

- Generate the timing reports for hold:

  ```
  report_timing
  ```

Alternatively, you can use the `report_timing -early` command with `set_db timing_analysis_check_type setup/hold` option to report setup/hold paths.

On-Chip Variation (OCV) Timing Analysis Mode Type Overview

In on-chip variation (OCV) mode, the software calculates clock and data path delays based on minimum and maximum operating conditions for setup analysis and vice-versa for hold analysis. These delays are used together in the analysis of each check.

The OCV is the small difference in the operating parameter value across the chip. Each timing arc in the design can have an early and a late delay to account for the on-chip process, voltage, and temperature variation.

OCV Timing Analysis Mode - Setup and Hold Checks

These are explained in the sections below.

Setup Check in OCV Mode

In OCV mode setup check, the software uses the timing delay values from the late library set and operating conditions for the data and the launch clock network delay. The software uses the delay values from the early library set and operating conditions for the capturing clock network delay assuming that the clocks are set in propagated mode.

Note: You can use one library instead of both maximum and minimum libraries, and apply timing derates for performing min/max analysis, respectively.

The source latency in both ideal and propagated modes for setup checks is defined in the constraints used by various clock paths as follows:

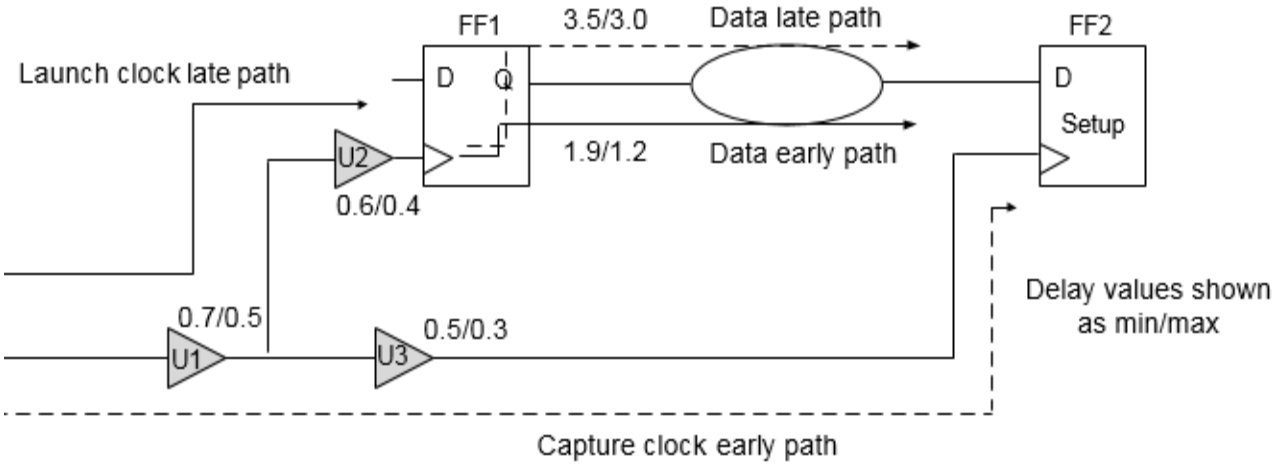
Clock Path (Operating Condition)	Constraint Used
Launch clock late path (max)	`set_clock_latency -source -late -max value` or `set_clock_latency -source -late value`
Capture clock early path (min)	`set_clock_latency -source -early -min value` or `set_clock_latency -source -early value`

The network latency in ideal mode for setup checks is defined in the constraints used by various clock paths as follows:

Clock Path (Operating Condition)	Constraint Used
Launch clock late path	`set_clock_latency -max value`
Capture clock early path	`set_clock_latency -min value`

The following example shows the setup check on a path from FF1 to FF2.

Figure 4-64 Setup Check in OCV Mode



The software uses the max library for all late path delays and min library for all early path delays.

The following values are assumed in this example:
Clock period = 4; Clock source latency = none; Clock mode = Propagated

Data late path delay (max)	3.5
Data late path delay (min)	3.0
Launch clock late path delay (max)	0.7 + 0.6 = 1.3
Capture clock early path delay (min)	0.5 + 0.3 = 0.8
Setup	0.2
Data arrival time	1.3 + 3.5 = 4.8
Data Required Time	4 + 0.8 - 0.2 = 4.6
Setup Slack	4.6- 4.8 = -0.2

Hold Check in OCV Mode

For OCV hold check, the software uses the timing delay values from the early library set and operating conditions for the data arrival time and launch clock network delay. The software uses delay values from the late library set and operating conditions for the capturing clock network delay assuming that the clocks are set in propagated mode.

The source latency in both ideal and propagated modes for hold checks is defined in the constraints used by various clock paths as follows:

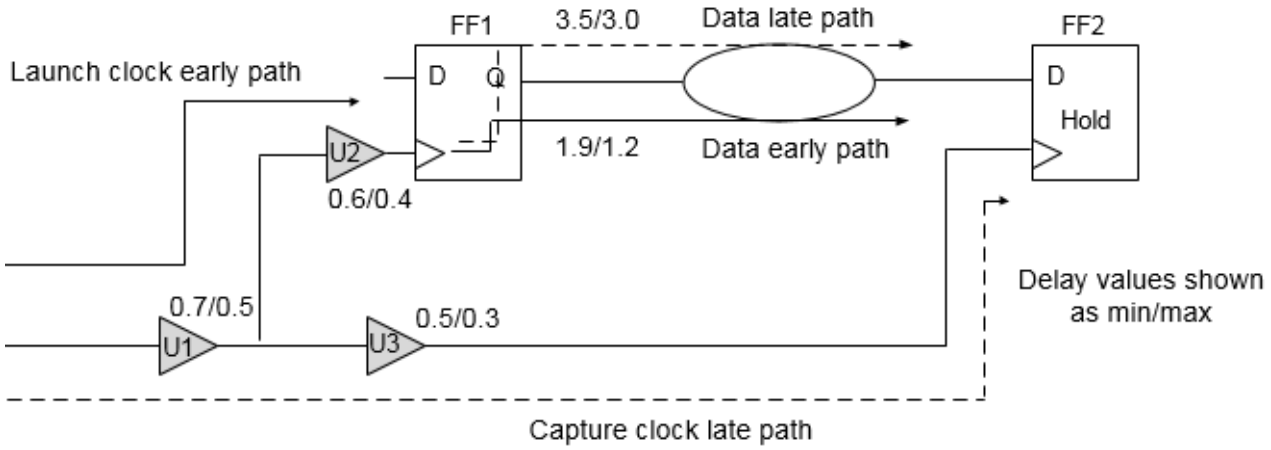
Clock Path (Operating Condition)	**Constraint Used**
Launch clock early path (min)	`set_clock_latency -source -early -min` *`value`* or `set_clock_latency -source -early` *`value`*
Capture clock late path (max)	`set_clock_latency -source -late -max` *`value`* or `set_clock_latency -source -late` *`value`*

The network latency in ideal mode for hold checks is defined in the constraints used by various clock paths as follows:

Clock Path (Operating Condition)	**Constraint Used**
Launch clock early path	`set_clock_latency -min` *`value`*
Capture clock late path	`set_clock_latency -max` *`value`*

The following example shows the hold check on the path from FF1 to FF2.

Figure 4-65 Hold Check in OCV Mode



The software uses the max library to scale all delays at WC conditions and minimum library to scale all delays at BC conditions.

The following values are assumed in this example:
Clock period = 4; Clock source latency = none; Clock mode = Propagated

Data early path delay (max)	
Data early path delay (min)	1.2
Launch clock early path delay (min)	0.5 + 0.4 = 0.9
Capture clock late path delay (max)	0.7 + 0.5 = 1.2
Hold	0.1
Data arrival time	0.9 + 1.2 = 2.1
Data required time	0.1 + 1.2= 1.3
Hold Slack	2.1 - 1.3 = 0.8

Performing Timing Analysis in OCV Mode

To perform timing analysis in OCV mode, complete the following steps:

- Set the analysis mode to OCV and propagated clock mode:
  `set db timing analysis type ocv`
- Generate the timing reports for setup:
  `report timing -late`
- Generate the timing reports for hold:
  `report_timing -early`

Using Timing Derates in OCV Analysis Mode

When the `set timing derate` command is used, the following paths in OCV mode are affected:

Violations	**Data**	**Launch Clock**	**Capture Clock**
Setup	`-late -data`	`-late -clock`	`-early -clock`
Hold	`-early -data`	`-early -clock`	`-late -clock`

Clock Path Pessimism Removal

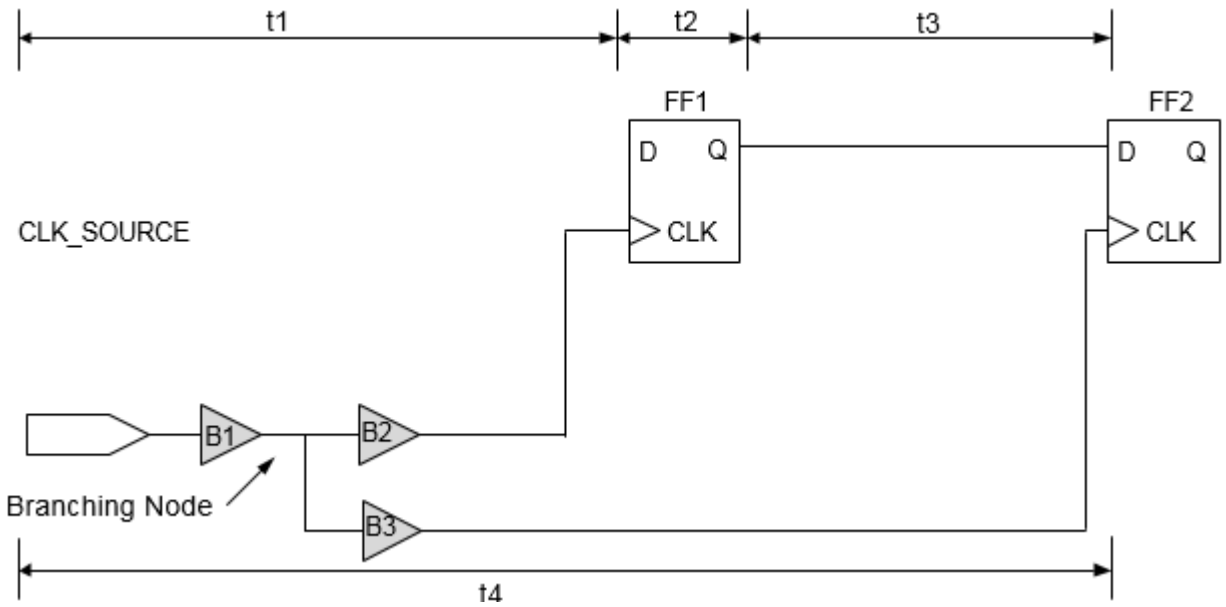
Clock Path Pessimism Removal (CPPR) is the process of identifying and removing pessimism introduced in slack reports for clock paths when launch and capture clock paths have a segment in common.

You can introduce early or late delay variations by using the `set_db timing_analysis_type ocv` attribute or by using the `set_timing_derate` command for early and late clock paths. In CPPR calculations, the difference between late and early delays (for the common clock segment between launch and capture clock path) is calculated first and then this number is adjusted in the slack calculations to remove the pessimism, which existed because of considering common clock path to be both late and early at the same time. To remove this pessimism in propagated clock mode, you can use the following command:

```
set_db timing_analysis_cppr true
```

Consider the following figure for setup check between flops FF1 and FF2.

Figure 4-66 Signal Path



The setup check equation for the example in the figure above with pessimism (without CPPR) is as follows:

$t_{1max} + t_{2max} + t_{3max} <= t_{4min} + t_{cp} - t_{su}$

where t_{cp} is the clock period and t_{su} is the setup requirement at D pin of flip-flop FF2.

The above setup check equation incorrectly implies that the common clock network, B1, can simultaneously use maximum delay for the launch clock late path (clock source to FF1/CLK) and minimum delay for the capture clock early path (clock source to FF2/CLK). Using the CPPR to remove this pessimism, the setup check equation is as follows:

$t_{1max} + t_{2max} + t_{3max} <= t_{4min} + t_{cp} - t_{su} + t_{cppr}$

where t_{cppr} is the difference in the maximum and minimum delay from the clock source to the branching node.

Similarly, hold check equation using CPPR is as follows:

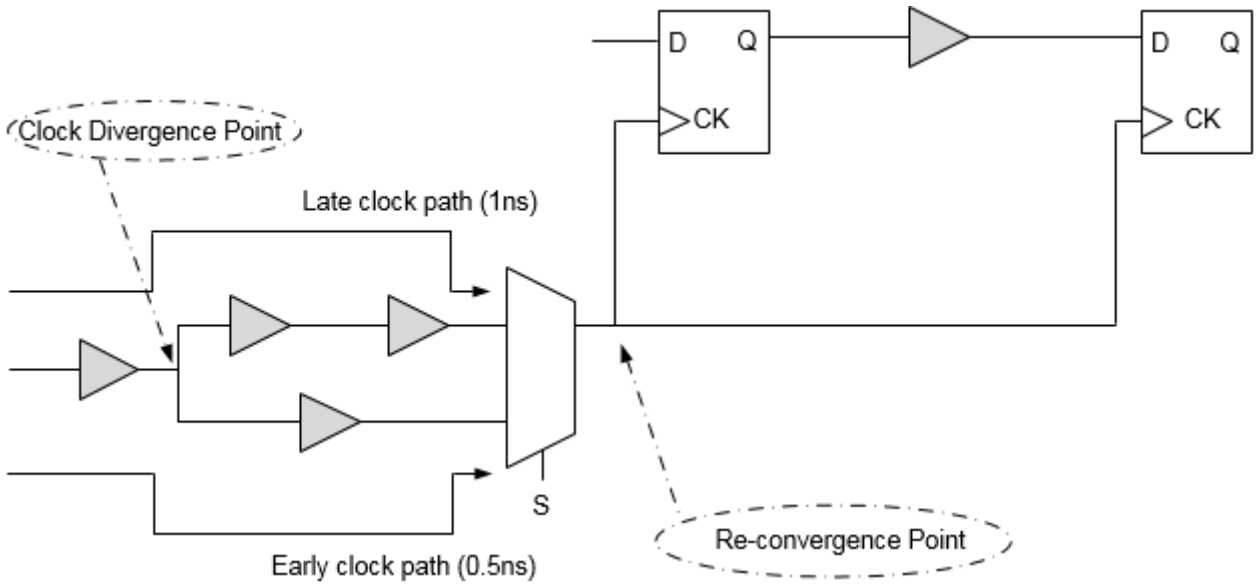
$t_{1min} + t_{2min} + t_{3min +} t_{cppr} <= t_{4max} + t_H$

where t_H is the hold requirement at D pin of flop FF2.

Clock Reconvergence and CPPR

If a design contains re-convergent logic on the clock path, then the timing analysis software might assume certain pessimism while calculating slack. The following figure shows a circuit in which timing analysis is performed in single analysis mode.

Figure 4-67 Illustration of Clock Reconvergence



In this case, if the `set_case_analysis` command has not been set at point S of the multiplexer, the timing analysis software will assume different delay values for early and late

paths. For example, if common early clock path from the clock source to the common clock point has a delay of 0.5ns, and the same common late clock path has delay of 1ns, then a pessimism equal to 0.5ns is introduced in the design. The above pessimism is not specific to single analysis mode only; it also applies to best-case/worst-case and on-chip variation methodologies. You can use CPPR to remove pessimism introduced due to reconvergence.

Supported CPPR Attributes

The Tempus software uses multiple attributes to control CPPR behavior. These can be changed based on the user requirements. When these attributes are changed, there can be changes in common clock path pessimism removal in terms of accuracy, common point selection, SI behavior, and so on.

Some of these attributes include:

- timing_cppr_remove_clock_to_data_pessimism
- timing_cppr_self_loop_mode
- timing_cppr_skip_clock_reconvergence
- timing_cppr_threshold_ps
- timing_cppr_transition_sense
- timing_enable_pessimistic_cppr_for_reconvergent_clock_paths
- timing_enable_si_cppr
- timing_cppr_opposite_edge_mean_scale_factor
- timing_cppr_opposite_edge_sigma_scale_factor

For more information, refer to timing Category Attributes.

Tempus uses a default threshold of 20ps during pessimism removal. This means that 20ps of uncertainty remains in the analysis. To set the threshold value you can use the `timing_cppr_threshold_ps` attribute. When you set this attribute to a specified value, all the paths may be reported without having pessimism removed by the given value of the attribute. During SI analysis, the CPPR behavior for setup and hold checks can be controlled by setting the `timing_enable_si_cppr` attribute.

Timing Analysis Results Before and After CPPR

The following example shows a full clock path timing report generated before CPPR analysis. The timing slack is 7.388 without any CPPR adjustments:

```
---------------------------------------------------------------------
Path 1: MET (0.285 ns) Setup Check with Pin CRE_IN/DATA_BUS_MACH_INST/
data_out_reg[9]/CP->D
Group: m_clk
Startpoint: (R) CRE_IN/EX_IN/arp_reg/CP
Clock: (R) m_clk
Endpoint: (F) CRE_IN/DATA_BUS_MACH_INST/data_out_reg[9]/D
Clock: (R) m_clk

               Capture   Launch
Clock Edge:+   7.000     0.000
Src Latency:+  0.000     0.000
Net Latency:+  0.514 (P) 0.623 (P)
Arrival:=      7.515     0.623

Setup:-          0.074
Required Time:=  7.441
Launch Clock:=   0.623
Data Path:+      6.533
Slack:=          0.285
Timing Path:
#-------------------------------------------------------------------------------
# Timing Point              Flags Arc   Edge Cell    Fanout Trans Delay Arrival
                                                          (ns)   (ns)  (ns)
#-------------------------------------------------------------------------------
CRE_IN/msvBuf21m_clk/Z         -  I->Z   R   LVLHLD8  1    0.040  0.119  0.196
CRE_IN/msvNet21m_clk_L1_I0/ZN  -  I->ZN  F   INVD16   2    0.088  0.053  0.249
CRE_IN/msvNet21m_clk_L2_I1/Z   -  I->Z   F   BUFFD12  1    0.044  0.082  0.331
CRE_IN/msvNet21m_clk_L3_I0/Z   -  I->Z   F   BUFFD12  1    0.029  0.088  0.419
#-------------------------------------------------------------------------------
Other End Path:
#-------------------------------------------------------------------------------
# Timing Point               Flags Arc   Edge Cell    Fanout Trans Delay Arrival
                                                          (ns)   (ns)  (ns)
#-------------------------------------------------------------------------------
m_clk__L1_I0/Z                 -     I->Z  R    BUFFD12 2      0.030  0.065  7.065
CRE_IN/msvBuf21m_clk/Z         -     I->Z  R    LVLHLD8 1      0.040  0.099  7.163
CRE_IN/msvNet21m_clk_L1_I0/ZN  -     I->ZN F    INVD16  2      0.088  0.044  7.208
CRE_IN/msvNet21m_clk_L2_I1/Z   -     I->Z  F    BUFFD12 1      0.044  0.068  7.276
#-------------------------------------------------------------------------------
```

When CPPR adjustment is made, the timing slack improvement is seen as pessimism and is displayed as “CPPR Adjustment” in the timing report.

In the example given below, “c2” is a common clock point after which the clock diverges:

```
#---------------------------------------------------------------------
Path 1: MET (0.328 ns) Setup Check with Pin CRE_IN/DATA_BUS_MACH_INST/
data_out_reg[9]/CP->D
Group: m_clk
Startpoint: (R) CRE_IN/EX_IN/ar0_reg[0]/CP
Clock: (R) m_clk
Endpoint: (F) CRE_IN/DATA_BUS_MACH_INST/data_out_reg[9]/D
Clock: (R) m_clk

               Capture     Launch
Clock Edge:+   7.000       0.000
Src Latency:+  0.000       0.000
Net Latency:+  0.514 (P)   0.674 (P)
```

```
Arrival:=      7.515       0.674

Setup:-          0.074
Cppr Adjust:+    0.042
Required Time:=  7.482
Launch Clock:=   0.674
Data Path:+      6.480
Slack:=          0.328
Timing Path:
#-------------------------------------------------------------------------------
# Timing Point              Flags Arc   Edge Cell    Fanout Trans Delay Arrival
                                                           (ns)   (ns)   (ns)
#-------------------------------------------------------------------------------
m_clk__L1_I0/Z                 -   I->Z   R   BUFFD12  2   0.030   0.078   0.078
CRE_IN/msvBuf21m_clk/Z         -   I->Z   R   LVLHLD8  1   0.040   0.119   0.196
CRE_IN/msvNet21m_clk_L1_I0/ZN  -   I->ZN  F   INVD16   2   0.088   0.058   0.254
CRE_IN/msvNet21m_clk_L2_I0/ZN  -   I->ZN  R   INVD16   9   0.045   0.069   0.323
CRE_IN/EX_IN/RC_CGIC_INST/Q    -   CP->Q  R   CKLNQD1  1   0.073   0.155   0.478
CRE_IN/EX_IN/rc_gclk_L1_I0/Z   -   I->Z   R   BUFFD6   1   0.079   0.113   0.591
#-------------------------------------------------------------------------------
Other End Path:

#-------------------------------------------------------------------------------
# Timing Point              Flags Arc   Edge Cell    Fanout Trans Delay Arrival
                                                           (ns)   (ns)   (ns)
#-------------------------------------------------------------------------------
m_clk__L1_I0/Z                 -   I->Z  R    BUFFD12 2     0.030  0.065  7.065
CRE_IN/msvBuf21m_clk/Z         -   I->Z  R    LVLHLD8 1     0.040  0.099  7.163
CRE_IN/msvNet21m_clk_L1_I0/ZN  -   I->ZN F    INVD16  2     0.088  0.044  7.208
CRE_IN/msvNet21m_clk_L2_I1/Z   -   I->Z  F    BUFFD12 1     0.044  0.068  7.276
CRE_IN/msvNet21m_clk_L3_I0/Z   -   I->Z  F    BUFFD12 1     0.029  0.072  7.348
CRE_IN/msvNet21m_clk_L4_I0/ZN  -   I->ZN R    INVD16  1     0.040  0.029  7.378
#-------------------------------------------------------------------------------
```

4.3.3 Advanced On-Chip Variation (AOCV) Analysis

4.3.3.1 Overview on page 146

4.3.3.2 Using AOCV Derating Library Formats on page 146

4.3.3.3 Enabling AOCV Analysis on page 151

4.3.3.4 Reading AOCV Libraries on page 151

4.3.3.5 Stage Counts and their Properties for Launch and Capture Paths on page 152

4.3.3.6 Handling OCV Derating with AOCV on page 157

4.3.3.7 Configuring AOCV Modes on page 159

4.3.3.8 Reporting in AOCV Modes on page 159

4.3.3.9 AOCV Reporting on page 167

4.3.3.1 Overview

In on-chip variation (OCV) mode, the software uses the timing delay values from the late or early library sets and operating conditions for the data, capture, and launch clocks. OCV analysis uses a single constant derating factor. However, in actual Silicon, there can be lot of sources of variation For example, variation of transistor length, distance, width, doping amount, and so on. To model these, advanced on-chip variation (AOCV) analysis can be used during timing analysis and optimization. AOCV analysis uses variable derating factors that are based on the distance of the cell and logic depth. Depending on the number of logic levels and placement locations, different derating factors can be used.

AOCV derating libraries are provided by library vendors, or can be characterized using Cadence's tool "Variety". An AOCV factor is chosen so that when multiplied by the lib nominal delay value, you get close to the mean ± 3 sigma value for that many stages in the path. As number of stages in the path increases, the sigma value (the spread) relative to the mean (nominal value) decreases on the order 1/sqrt (n). Due to which AOCV derate factor required decreases.

4.3.3.2 Using AOCV Derating Library Formats

There are two different AOCV derating library formats:

- External AOCV Format
- Cadence-Specific AOCV Derating Format

The characterization data within either format is essentially the same - the differences are mainly syntactic.

External AOCV Format

The external AOCV format is currently supported by third parties. For example,

```
version: 2.0
voltage: 1.5
object_type: lib_cell
object_spec: LIB/BUF1X
delay_type:   [net cell]+
path_type:    [data clock]+
derate_type: [early | late ]
rf_type:      [rise fall]+
depth:    1 2 3 4 5
distance: 500 1000 1500 2000
table:
1.123 1.090 1.075 1.067 1.062
1.124 ...
```

3rd Party AOCV Format

Cadence-Specific AOCV Derating Format

For example,

```
[Apply-[Net | Cell ]]
Vdd 1.5
[Early | Late][-Rise | Fall][Clock | Data]
[Cell | Net | Instance]  LIB/BUFX1
Stage 1 2 3 4 5
Distance 500 1000 1500 2000
1.123 1.090 1.075 1.067 1.062
1.124 ...
```

Cadence AOCV Format

Cadence-Specific AOCV Derating Format: Library Syntax

The Cadence specific AOCV derating format is made of several sections allowing you to customize how each table is applied:

Sections	Tags	Notes
[Apply- [Net | Cell]]	Table Group	Optional

[Voltage 1.5]	Table Voltage	Optional
[Early | Late] -[Rise | Fall]? -[Clock | Data]	Table ID	Early or Late is required
[Cell | Net | Instance] LIB/BUFS1	Object ID	Optional
Depth 1 2 3 4 5	Derating Table	It is possible to have a 1-D distance.
[Distance 500 1000 1500 2000]?		
1.123 1.090 1.075 1.067 1.062 1.124 1.091 1.076 1.068 1.063 1.125 1.092 1.077 1.070 1.065 1.126 1.094 1.079 1.072 1.067		

The table sections are described below.

Table Group

The Apply- tag can be used with either a net or cell descriptor to designate the table for use in derating net delays or cell delays, respectively. If the Apply- tag is not specified, then the table will be used for both net and cell delay derating. The Apply- tags are described below:
To designate a table for only net-based derating, you should specify:

```
Apply-Net
```

To designate a table for only cell-based derating, use the following:

```
Apply-Cell
```

If you use the Object ID field to specify unique tables for specified libraries and/or library cells, you should also specify Apply-Cell for these tables.

Table ID

The Table ID is a tag made up of up to three sub tags, which are used to further indicate to the derating table to control early versus late, rise versus fall, and clock path versus data path.

```
{Early | Late}
```

Each table specifies Early or Late tags. Each object related to AOCV derating has both Early and Late derating tables. AOCV derating of only early or late paths is not allowed. It is, however, legal to use derating values of 1.0 for the tables.

By default, an Early or Late derating table is used to supply derating for both rising and falling delays. If a unique derating for each transition is required, the -Rise or -Fall tag can be added to the Table ID for more specific results.

```
{Early-Rise}
```

To configure a table for only rising delays on early paths, you should use Early-Rise sub tag.

```
{Late-Fall}
```

To configure a table for falling delays on late timing paths, you can specify Late-Fall.

```
[Clock | Data]
```

Derating tables can also be configured so that they are specific to clock path delays or data path delays. This can be achieved by adding a final -Clock or -Data tag to the Table ID string.

The complete set of possible Table IDs is:

Early	Early-Rise-Clock	Early-Rise-Clock	Late-Rise-Clock
Early-Rise	Early-Rise-Data	Late-Rise	Late-Rise-Data
Early-Fall	Early-Fall-Clock	Late-Fall	Late-Fall-Clock
Early-Clock	Early-Fall-Data	Late-Clock	Late-Fall-Data
Early-Data		Late-Data	

In the case where there is an overlap in the table specification, the derating table with the tightest specification is used.

Object ID

You can further control the AOCV derating tables to be applied to specific timing libraries or library cell groups. The syntax also supports specifying design-level cell instances or nets, however, these types of design-level references are not currently supported.

Note: The AOCV libraries are configured and stored in the system the same way as Liberty timing libraries, however, only library or library-cell level derate factors are supported.

In the AOCV flow you can use the Cell Object ID to designate particular derating tables to be used for specific library-level groups. An object expression using simple wildcard rules can be used for a variety of selection criteria. For example, the following expression specifies a derating table to be used for a specific library:

```
Cell stdcell/*
```

The following expression specifies derating for a specific library cell:

```
Cell stdcell/BUF1S
```

The following expression specifies a common derate table for a group of cells with similar drive strength:

```
Cell stdcell/*1S
Cell stdcell/*4S
```

The following expression specifies a common derate factor for a group of libraries at the same corner:

```
Cell *-P1V15T120/*
```

Table Voltage

In the future, an optional voltage designation for the table can be specified by adding the Voltage specifier to the model description. You can specify several tables with different voltage specifications within the same `.aocv` library file. When performing derating, the system uses the table that matches the current operating voltage of the cell. If the operating voltage of the cell does not exactly match one of the specified voltage points, then the following are considered:

- If the operating voltage is between the Voltage points of two derating tables, the system will use linear interpolation to derive the proper derating factor.
- If the requested voltage is large than the maximum Voltage table, then that table data will be used without extrapolation.
- If the requested voltage is lower than the minimum Voltage table, then the minimum Voltage data is used without extrapolation.
- To specify a voltage of 1.5Volts for the derating table, use a specification of:
  ```
  Voltage 1.5
  ```

Derating Table

The AOCV format allows derating table specifications that are either one or two-dimensional – the possible axes being Distance, the bounding box diagonal of that path, and Stage - the stage count depth of the path. Although AOCV graph-based analysis supports stage-based derating only, you may specify either a 1-D stage-based table, or a 2-D Stage x Distance table.

When using a 2-D table in conjunction with AOCV graph-based stage-based only analysis, the system will attempt to pick the proper row of the table based on the estimated chip and/or core dimensions. If the software includes physical information, the chip and core size

are automatically determined from the design. The `timing_aocv_chip_size` and `timing_aocv_core_size` attributes can be used in the software to specify the physical dimensions when this information is not available. If this value falls off the table, then the final row of the table will be used without extrapolation.

The Stage and Distance keywords are added to the model to provide the axis points for the table. The table rows represent “distance” variance and the columns “stage count”, irrespective of the order in which stage and distance keywords are specified for the model. These are explained below:

Distance: List of distance (real) values, specified in microns, increasing in magnitude.

Stage: List of integer values representing stage depth, increasing in magnitude.

The following example shows a derating table with 5 Stage indices and 4 Distance indices:

```
Stage 1 2 3 4 5
Distance 500 1000 1500 2000
1.123 1.090 1.075 1.067 1.062
1.124 1.091 1.076 1.068 1.063
1.125 1.092 1.077 1.070 1.065
1.126 1.094 1.079 1.072 1.067
```

In case the stage or distance lookup value goes beyond the range of the table - either smaller or larger, the system will use the last bound entry point. Extrapolation will not be performed on either end of the axis.

4.3.3.3 Enabling AOCV Analysis

AOCV analysis can be enabled by using the following command variables.

```
set_db timing_analysis_aocv true
```

4.3.3.4 Reading AOCV Libraries

SMSC Environment

In SMSC environment, the AOCV libraries can be read using the command below:

```
read_libs -aocv_libs "myDerateLib.aocv.lib"
```

MMMC Environment

In MMMC environment, AOCV libraries can be read using the `create_library_set` command.

For example:

```
create_library_set -name slow -timing slow.lib -aocv test.aocv.lib
```

In case, the library set is already created, it can be updated to read the new aocv library or update the existing library using the `update_library_set` command.

For example:

```
update_library_set -name slow -aocv test.aocv.lib
```

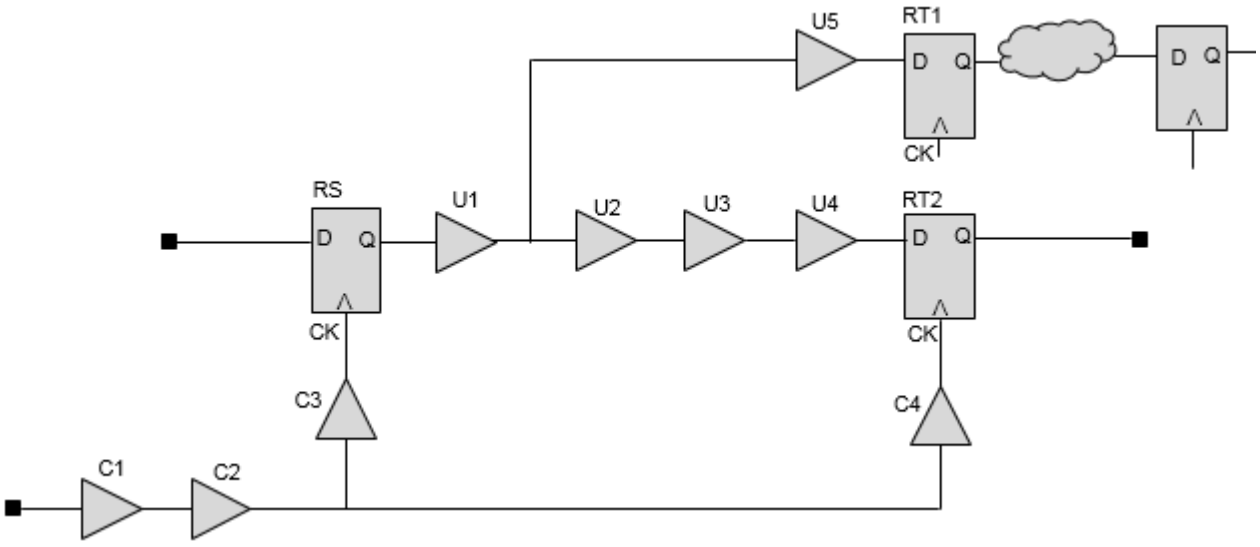
4.3.3.5 Stage Counts and their Properties for Launch and Capture Paths

Most cells in a clock path have both launch stage counts and capture stage counts. This means a clock buffer exists in a launch path and a capture path simultaneously so the buffer has its own launch stage count property and capture stage count property. This section covers a comprehensive explanation for the launch stage count and capture stage count.

The following four timing paths in the figure below explain launch and capture stage counts:

- The timing path from Input to RS/D (input to reg path)
- The timing path from RS/CK to RT1/D (reg to reg path)
- The timing path from RS/CK to RT2/D (reg to reg path)
- The timing path from RT2/CK to output1 (reg to output path)

Figure 4-68 Timing Paths



The following default settings are assumed:

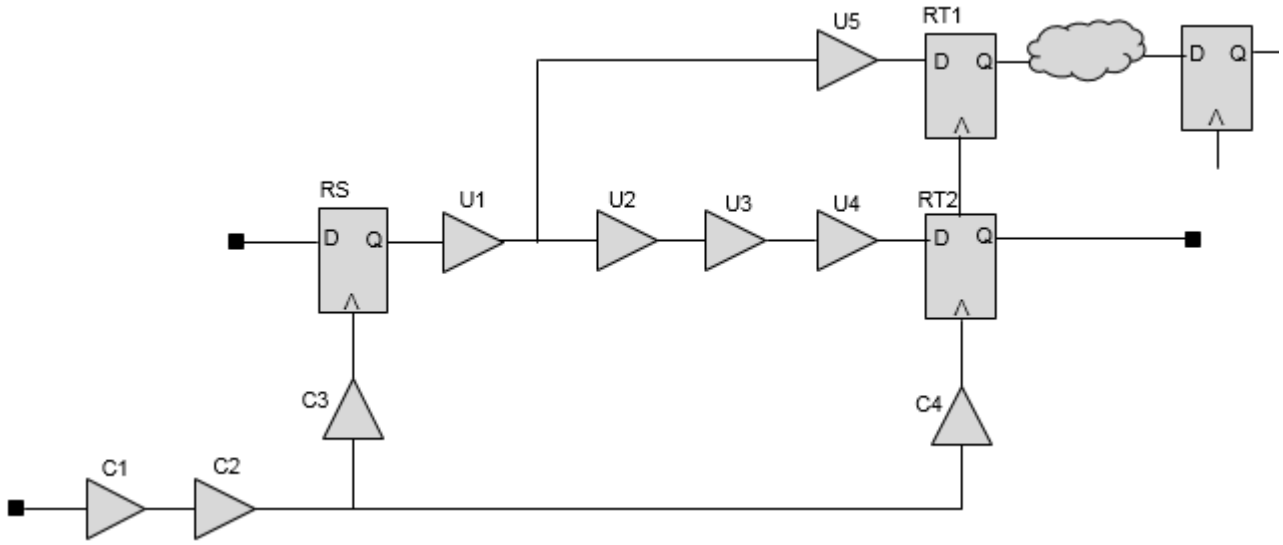
```
set_db timing_aocv_analysis_mode launch_capture;  #(default)
set_db timing_aocv_use_cell_depth_for_net false;  #(default: false)
```

Stage Counting Graph Based Analysis (GBA)

Graph-based AOCV analysis does choose a pessimistic situation so each cell has to its own pessimistic property (only one value, not two or more values like PBA) such as the late launch stage count for cell, early capture stage count for cell, and so on.

The following figure includes the launch cell and net stage counts:

Figure 4-69 Launch cell and net stage counts



The stage counts of C1 and C2 for launch paths in the above example are as follows:

The clock port is connected to input pin of C1.

1. From the clock port, there are 4 launch paths which include C1 and C2. that is, path from the clock port through C1/C2/C3/RS/U1/U5 to RT1.
2. The launch path from the clock port through C1/C2/C3/RS/U1/U2/U3/U4 to RT2.
3. The launch path from the clock port through C1/C2/C4/RT1 through some combination logic to the FF on the top-right side.
4. The launch path from the clock port through C1/C2/C4/RT2 to the output port.

The last path is the shortest launch path that includes C1 and C2 and each launch cell stage count is 4 (C1, C2, C4, and RT2).

Next, stage count is reset to 0 at the common point C2/Y.

In case of the launch path, which includes C3/RS/U1, each stage count of C3/RS/U1 is 4/4/4 for the end point RT1 and stage count of 6/6/6 for the end point RT2.

Now, the problem of choosing the stage count occurs on C3/RS/U1 because the cells exist in the same launch path in the case of the two timings paths (RS to RT1 and RS to RT2) and have different stage count based in path-based AOCV analysis.

Therefore, to maintain pessimism, the smaller stage count 4 is chosen. So, each launch cell stage count of C3/RS/U1 is 4 in case of graph-based AOCV analysis. U5 has a stage count of 4 and U2/U3/U4 has a stage count of 6, respectively. Smaller stage count means more variation, so it is mapped to the worse AOCV derate for late and early.

Note: In GBA, the algorithm requires resetting of stage count at clock branch points- a major source of pessimism.

In the given figure, another common point occurs at the pin C4/Y. which is connected to RT1/CK and RT2/CK. So, state count is reset to 0 at the C4/Y pin. The stage count of the path from the C4/Y through RT1 is longer than that of the path from the C4/Y through RT2 to the output port. RT2 has stage count of 1 and RT1 has a stage count of a value greater than 1 and C4 has a stage count of 2.

The following table summarizes the graph-based launch cell stage counts of the above example:

Segment	**Members**	**Stage Count**	**Stage Path**
Clock root -> CPPR Common Point (C2/Y)	C1-C2	4	C1-C2-C4-RT2
CPPR Common Point (C2/Y) -> Launch Clock Branch Point (RS) -> Data Path Branch	C3-RS-U1	4	C3-RS-U1-U5
Data Path Branch -> Data Path Endpoint	U5	4	C3-RS-U1-U5
Data Path Branch -> Data Path Endpoint	U2-U3-U4	6	C3-RS-U1-U2-U3-U4
CRPR Common Point (C2/Y) -> CRPR Common Point (C4/Y)	C4	2	C4-RT2

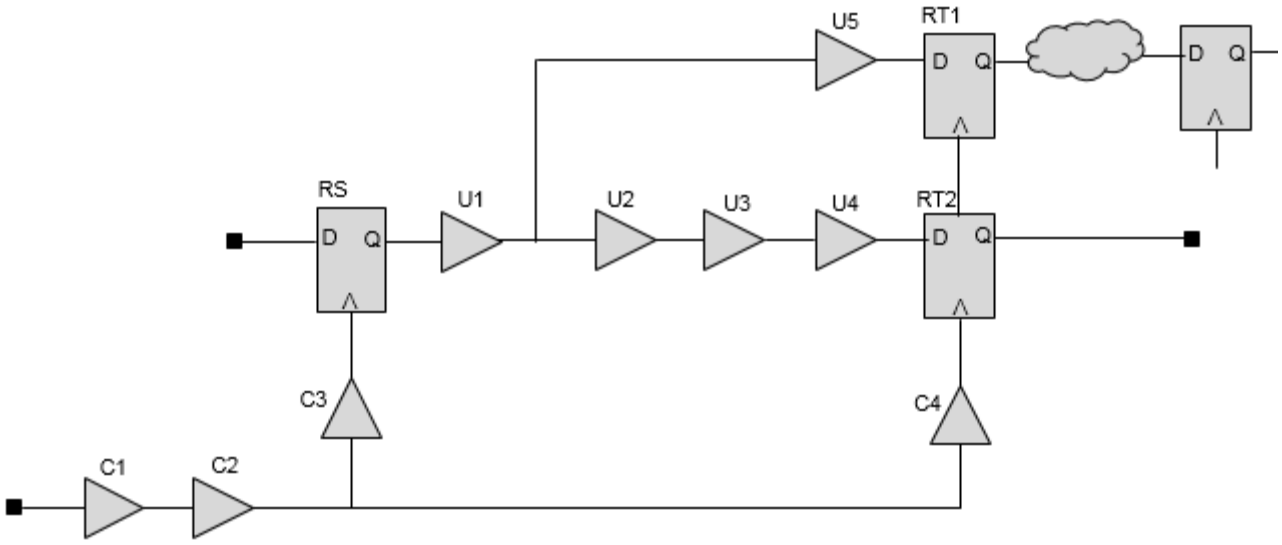
CRPR Common Point (C4/Y) -> Launch Clock Branch Point (RT2) -> data Path Endpoint	RT2	1	RT2

Stage Counting Path-Based Analysis (PBA)

The method to calculate the stage count under path-based AOCV mode is intuitive.

The following figure includes the launch cell and net stage counts:

Figure 4-70 Launch cell and net stage counts



Firstly, look for the CPPR common point on the clock launch path and the capture path of a timing path.

To derive the common point from RS to RT1:-

- The launch path from RS to RT1:
  clock->C1->C2->C3->RS->U1->U5->RT1/D
- The capture path from RS to RT1:
  clock->C1->C2->C4->RT1/CK.

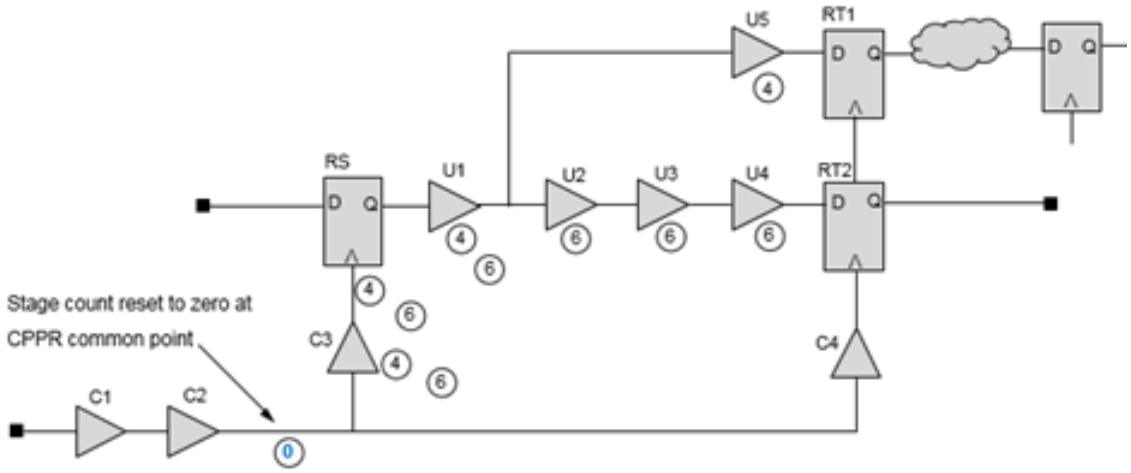
So, the common point (pin) of the above timing path is the C2/Y pin.

To derive the common point from RS to RT2:

- The launch path from RS to RT2:
  clock->C1->C2->C3->RS->U1->U2->U3->U4->RT2/D.
- The capture path from RS to RT2:
  clock->C1->C2->C4->RT2/CK.

So, the common point (pin) of the above timing path is the C2/Y pin.
Now, for calculation purpose, the stage count at C2/Y is reset to 0, as shown in below figure.

Figure 4-71 Stage count at C2/Y reset to 0



Therefore, the launch path from the pin C2/Y to RT1/D has a stage count of 4, that is, C3(1)->RS(2)->U1(3)->U5(4)->RT1/D

The launch path from the pin RS to RT2/D has a stage count of 6, that is, C3(1)->RS(2)->U1(3)->U2(4)->U3(5)->U4(6)->RT2/D

Segment	**Members**	**Stage Count**	**Stage Path**
Launch Clock Branch Point -> Data Path Endpoint (RS to RT1)	C3-RS-U1-U5	4	C3-RS-U1-U5
Launch Clock Branch Point -> Data Path Endpoint (RS to RT2)	C3-RS-U1-U2-U3-U4	6	C3-RS-U1-U2-U3-U4

The table above summarizes the stage count of the 2 launch paths in PBA.

Based on above derived stage count, the tool calculates the diagonal distance for the timing path and then gets the AOCV derate values from the AOCV derating library with the stage count and the distance.

Note: The `report_timing` command can be used to report each cell's AOCV derate and each net's AOCV derate.

So, think over which AOCV derates of the cells/nets on the common path are used for the common parts. In the above example, you have to check the stage counts of the cells C1 and C2 on the common path and AOCV derates of the cells. The stage count (SC) calculation on the common path cells is relating to graph-based AOCV analysis.

4.3.3.6 Handling OCV Derating with AOCV

The timing system offers a set of controls that enable you to flexibly configure the derating environment. You can perform the following functions:

- Control whether AOCV and OCV derating factors are multiplied or added.
- Specify whether the reference point for the AOCV and OCV factors are nominal around 0.0, or around 1.0 - and adjust the calculation appropriately.
- Allow the final OCV factor to be built up by successive multiplication or addition of sub-factors by using the `set_timing_derate` command.

These are described in the subsequent sections.

Using AOCV Addition or Multiplication with OCV

By default, when AOCV analysis is enabled, any OCV style derating specified with the `set_timing_derate` command is multiplied by the calculated AOCV factor to get the final derating value. You can use the following setting to add or multiply AOCV and OCV factors:

`set db timing aocv derate mode aocv_multiplicative (default) | aocv_additive`

Using Derating Reference Points

The AOCV and OCV derating factors may be added or multiplied, so it is important to understand whether each factor is nominal at zero or one. For example, if there is an AOCV +3% factor modeled as 1.03, and an OCV factor having a plus two percent factor modeled as 1.02, simple multiplication of factors yields a final derate of around 1.051. If the same factors are added up, a sum of 2.05 is obtained, which is not the expected value. The timing system can understand that both of these factors are nominally referenced to 1.0, and it can then automatically adjust the final result. In this case the timer will calculate:

(OCV) (AOCV) (Adjust)
1.02 + 1.03 - 1.0 = 1.05

The following attributes can be used to allow the specification of the reference point for both OCV and AOCV factors:

`set_db timing_derate_aocv_reference_point 0 | 1 (default)`
`set_db timing_derate_ocv_reference_point 0 | 1 (default)`

According to current AOCV library definitions, the AOCV reference point is always expected to be set at 1.0.

Using Addition and Multiplication of OCV Sub-Factors

The software provides the ability to perform incremental adjustments to the defined timing derate factors. The software also allows addition and multiplication (using `set_timing_derate -add` and `-multiply` parameters) to function on an instance-specific basis as well.

When `aocv_additive` mode is selected, the adjustments are performed as shown in the table below.

AOCV Factor	AOCV Reference Pt	OCV Factor	OCV Reference Pt	Add or Multiply	Final Derate Calculation
X	0/1	Y	0/1	aocv_multiplicative	X * Y
X	1.0	Y	1.0	aocv_additive	X + Y - 1
X	1.0	Y	0.0	aocv_additive	X + Y
X	0.0	Y	1.0	aocv_additive	X + Y
X	0.0	Y	0.0	aocv_additive	X + Y + 1

The table shows the hierarchy of derating factors in increasing order of precedence. No adjustments are performed for `aocv_multiplicative` mode.

Note: It is expected that AOCV derating libraries will be referenced to 1.0. So the combinations above are not expected to be actively used.

4.3.3.7 Configuring AOCV Modes

The Tempus software allows you can control the handling of AOCV analysis modes by using the `timing_aocv_analysis_mode` setting. You can specify one of the following modes for both PBA and GBA:

- combine_launch_capture
- launch_capture (default)
- clock_only
- separate_data_clock

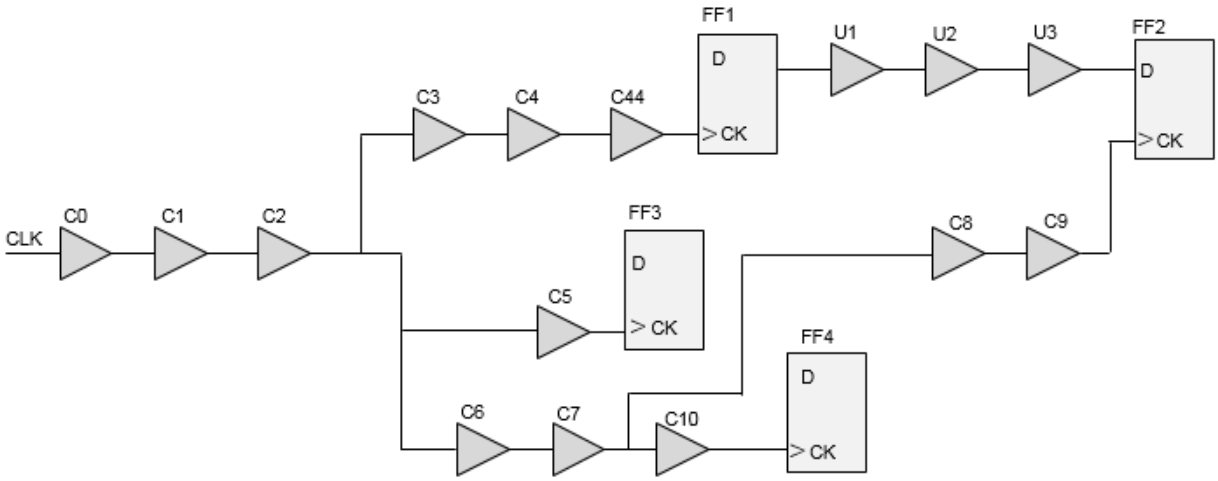
These are described below.

- **combine_launch_capture (default)**
  When set to combine_launch_capture, the software calculates the AOCV stage count using the combined depth for launch and capture paths. This is achieved by the summation of the path depth on the launch and capture paths.
- **launch_capture**
  When set to launch_capture, AOCV analysis will be performed on both the data and clock paths, and the AOCV derating factor will be calculated based for both clock and data path stages.
- **clock_only**
  When set to clock_only, AOCV analysis will be performed for clock paths only, and AOCV derating factor will be calculated based on only the clock path stage depth.
- **separate_data_clock**
  When set to separate_data_clock, the software calculates the AOCV stage count for clock paths and data paths separately. Both clock and data paths have related AOCV derating factors.

4.3.3.8 Reporting in AOCV Modes

Consider the following diagram:

Figure 4-72 AOCV stage count



The following examples show AOCV stage count in GBA and PBA mode:

timing_aocv_analysis_mode clock_only mode (GBA)

```
Path 1: VIOLATED (-48.474 ns) Hold Check with Pin cppr_DDF4_1/CK->D
View: default_analysis_view_setup
Group: clk_cppr
Startpoint: (F) data_cppr
Clock:
Endpoint: (F) cppr_DDF4_1/D
Clock: (R) clk_cppr

                 Capture      Launch
Clock Edge:+     0.000        0.000
Src Latency:+    0.000        0.000
Net Latency:+    48.540 (P)   0.000 (I)
Arrival:=        48.540       0.000

Hold:+   -0.026
AOCV Adjustment:-    0.040
Cppr Adjust:-    0.000
Required Time:=   48.474
Launch Clock:=    0.000
Data Path:+    0.000
Slack:=  -48.474
Timing Path:
#-------------------------------------------------------------------#
#Delay   Arrival  Pin             Arc         Aocv     Aocv   #
#(ns)     (ns)                               Derate   Stage  #
#                                                     Count  #
#-------------------------------------------------------------------
0.000    0.000    data_cppr       data_cppr   -        -      #
0.000    0.000    cppr_DDF4_1/D   -           1.000    -      #
#-------------------------------------------------------------------
Other End Path:
```

```
#-------------------------------------------------------------#
#Delay   Arrival  Pin               Arc        Aocv     Aocv  #
#(ns)     (ns)                                 Derate   Stage #
#                                                       Count #
#-------------------------------------------------------------#
0.000    0.000    clk_cppr          clk_cppr   -        5.000
12.110   12.110   cppr_CLKBUF_0/Y   A->Y       1.211    4.000
12.130   24.240   cppr_CLKBUFA_1/Y  A->Y       1.213    4.000
12.130   36.370   cppr_CLKBUFA_2/Y  A->Y       1.213    2.000
12.170   48.540   cppr_CLKBUF3_1/Y  A->Y       1.217    2.000
0.000    48.540   cppr_DDF4_1/CK    -          1.215    2.000
#-------------------------------------------------------------
```

timing_aocv_analysis_mode clock_only mode (PBA)

```
-------------------------------------------------------------------------
Path 1: VIOLATED (-48.474 ns) Hold Check with Pin cppr_DDF4_1/CK->D
Retime Analysis:  Data Path-Slew
View: default_analysis_view_setup
Group: clk_cppr
Startpoint: (F) data_cppr
Clock:
Endpoint: (F) cppr_DDF4_1/D
Clock: (R) clk_cppr

                 Capture      Launch
Clock Edge:+     0.000        0.000
Src Latency:+    0.000        0.000
Net Latency:+    48.540 (P)   0.000 (I)
Arrival:=        48.540       0.000

Hold:+              -0.026
AOCV Adjustment:-    0.040
Cppr Adjust:-        0.000
Required Time:=     48.474
Launch Clock:=       0.000
Data Path:+          0.000
Slack:=            -48.474
Slack(original):=  -48.474
Timing Path:
#-------------------------------------------------------------#
#Delay   Arrival  Pin               Arc        Aocv     Aocv  #
#(ns)    (ns)                                 Derate   Stage #
#                                                       Count #
#-------------------------------------------------------------#
0.000    0.000   data_cppr          data_cppr  -        -
0.000    0.000   cppr_DDF4_1/D      -          1.000    -
#-------------------------------------------------------------
Other End Path:
#-------------------------------------------------------------#
#Delay   Arrival  Pin               Arc        Aocv     Aocv  #
#(ns)    (ns)                                 Derate   Stage #
#                                                       Count #
#-------------------------------------------------------------#
0.000    0.000    clk_cppr          clk_cppr  -       5.000
12.110   12.110   cppr_CLKBUF_0/Y   A->Y      1.211   4.000
12.130   24.240   cppr_CLKBUFA_1/Y  A->Y      1.213   4.000
12.130   36.370   cppr_CLKBUFA_2/Y  A->Y      1.213   2.000
```

```
12.170   48.540   cppr_CLKBUF3_1/Y  A->Y      1.217   2.000
0.000    48.540   cppr_DDF4_1/CK    -         1.215   2.000
#-------------------------------------------------------------
```

timing_aocv_analysis_mode separate_data_clock mode (GBA)

```
Path 1: VIOLATED (-48.474 ns) Hold Check with Pin cppr_DDF4_1/CK->D
View: default_analysis_view_setup
Group: clk_cppr
Startpoint: (F) data_cppr
Clock:
Endpoint: (F) cppr_DDF4_1/D
Clock: (R) clk_cppr
                Capture     Launch
Clock Edge:+    0.000       0.000
Src Latency:+   0.000       0.000
Net Latency:+   48.540 (P)  0.000 (I)
Arrival:=       48.540      0.000

Hold:+     -0.026
AOCV Adjustment:-   0.040
Cppr Adjust:-       0.000
Required Time:=    48.474
Launch Clock:=      0.000
Data Path:+         0.000
Slack:=           -48.474
Timing Path:
#-------------------------------------------------------------#
#Delay   Arrival  Pin               Arc         Aocv     Aocv  #
#(ns)    (ns)                                   Derate   Stage #
#                                                        Count #
#-------------------------------------------------------------#
0.000    0.000    data_cppr         data_cppr   -        1.000
0.000    0.000    cppr_DDF4_1/D     -           0.814    1.000
#-------------------------------------------------------------
Other End Path:
#-------------------------------------------------------------#
#Delay   Arrival  Pin               Arc         Aocv     Aocv  #
#(ns)    (ns)                                   Derate   Stage #
#                                                        Count #
#-------------------------------------------------------------#
0.000    0.000    clk_cppr          clk_cppr    -        5.000
12.110   12.110   cppr_CLKBUF_0/Y   A->Y        1.211    4.000
12.130   24.240   cppr_CLKBUFA_1/Y  A->Y        1.213    4.000
12.130   36.370   cppr_CLKBUFA_2/Y  A->Y        1.213    2.000
12.170   48.540   cppr_CLKBUF3_1/Y  A->Y        1.217    2.000
0.000    48.540   cppr_DDF4_1/CK    -           1.215    2.000
#-------------------------------------------------------------
```

timing_aocv_analysis_mode separate_data_clock mode (PBA)

```
Path 1: VIOLATED (-48.474 ns) Hold Check with Pin cppr_DDF4_1/CK->D
Retime Analysis:  Data Path-Slew
View: default_analysis_view_setup
Group: clk_cppr
Startpoint: (F) data_cppr
```

```
Clock:
Endpoint: (F) cppr_DDF4_1/D
Clock: (R) clk_cppr
                Capture      Launch
Clock Edge:+    0.000        0.000
Src Latency:+   0.000        0.000
Net Latency:+   48.540 (P)   0.000 (I)
Arrival:=       48.540       0.000

Hold:+   -0.026
AOCV Adjustment:-    0.040
Cppr Adjust:-    0.000
Required Time:=   48.474
Launch Clock:=    0.000
Data Path:+    0.000
Slack:=  -48.474
Slack(original):=  -48.474
Timing Path:
#-------------------------------------------------------------------#
#Delay   Arrival  Pin               Arc         Aocv     Aocv  #
#(ns)    (ns)                                   Derate   Stage #
#                                                        Count #
#-------------------------------------------------------------------#
0.000    0.000    data_cppr         data_cppr   -        1.000
0.000    0.000    cppr_DDF4_1/D     -           0.814    1.000
#-------------------------------------------------------------------
Other End Path:
#-------------------------------------------------------------------#
#Delay   Arrival  Pin               Arc         Aocv     Aocv  #
#(ns)    (ns)                                   Derate   Stage #
#                                                        Count #
#-------------------------------------------------------------------#
0.000    0.000    clk_cppr          clk_cppr    -        5.000
12.110   12.110   cppr_CLKBUF_0/Y   A->Y        1.211    4.000
12.130   24.240   cppr_CLKBUFA_1/Y  A->Y        1.213    4.000
12.130   36.370   cppr_CLKBUFA_2/Y  A->Y        1.213    2.000
12.170   48.540   cppr_CLKBUF3_1/Y  A->Y        1.217    2.000
0.000    48.540   cppr_DDF4_1/CK    -           1.215    2.000
#-------------------------------------------------------------------
```

timing_aocv_analysis_mode launch_capture (GBA)

```
Path 1: VIOLATED (-48.474 ns) Hold Check with Pin cppr_DDF4_1/CK->D
View: default_analysis_view_setup
Group: clk_cppr
Startpoint: (F) data_cppr
Clock:
Endpoint: (F) cppr_DDF4_1/D
Clock: (R) clk_cppr

                 Capture     Launch
Clock Edge:+     0.000       0.000
Src Latency:+    0.000       0.000
Net Latency:+   48.540 (P)   0.000 (I)
Arrival:=       48.540       0.000

Hold:+   -0.026
AOCV Adjustment:-    0.040
```

```
Cppr Adjust:-    0.000
Required Time:=   48.474
Launch Clock:=    0.000
Data Path:+    0.000
Slack:=  -48.474
Timing Path:

#-------------------------------------------------------------------#
#Delay   Arrival  Pin               Arc         Aocv     Aocv  #
#(ns)    (ns)                                   Derate   Stage #
#                                               Count #
#-------------------------------------------------------------------#
0.000   0.000     data_cppr         data_cppr   -        1.000
0.000   0.000     cppr_DDF4_1/D     -           0.814    1.000
#-------------------------------------------------------------------
Other End Path:
#-------------------------------------------------------------------#
#Delay   Arrival  Pin               Arc         Aocv     Aocv  #
#(ns)    (ns)                                   Derate   Stage #
#                                               Count #
#-------------------------------------------------------------------#
0.000    0.000    clk_cppr          clk_cppr    -        5.000
12.110   12.110   cppr_CLKBUF_0/Y   A->Y        1.211    4.000
12.130   24.240   cppr_CLKBUFA_1/Y  A->Y        1.213    4.000
12.130   36.370   cppr_CLKBUFA_2/Y  A->Y        1.213    2.000
12.170   48.540   cppr_CLKBUF3_1/Y  A->Y        1.217    2.000
0.000    48.540   cppr_DDF4_1/CK    -           1.215    2.000
#-------------------------------------------------------------------
```

timing_aocv_analysis_mode launch_capture (PBA):

```
Path 1: VIOLATED (-48.474 ns) Hold Check with Pin cppr_DDF4_1/CK->D
Retime Analysis:  Data Path-Slew
View: default_analysis_view_setup
Group: clk_cppr
Startpoint: (F) data_cppr
Clock:
Endpoint: (F) cppr_DDF4_1/D
Clock: (R) clk_cppr

                  Capture    Launch
Clock Edge:+      0.000      0.000
Src Latency:+     0.000      0.000
Net Latency:+   48.540 (P)  0.000 (I)
Arrival:=       48.540      0.000

Hold:+             -0.026
AOCV Adjustment:-   0.040
Cppr Adjust:-       0.000
Required Time:=    48.474
Launch Clock:=      0.000
Data Path:+         0.000
Slack:=           -48.474
Slack(original):= -48.474
Timing Path:
```

```
#-------------------------------------------------------------#
#Delay   Arrival  Pin              Arc         Aocv     Aocv  #
#(ns)    (ns)                                 Derate   Stage #
#                                             Count #
#-------------------------------------------------------------#
0.000    0.000  data_cppr      data_cppr  -        1.000
0.000    0.000  cppr_DDF4_1/D  -           0.814   1.000
#-------------------------------------------------------------
Other End Path:
#-------------------------------------------------------------#
#Delay   Arrival  Pin              Arc         Aocv     Aocv  #
#(ns)    (ns)                                 Derate   Stage #
#                                             Count #
#-------------------------------------------------------------#
0.000    0.000    clk_cppr         clk_cppr   -         5.000
12.110   12.110   cppr_CLKBUF_0/Y  A->Y       1.211     4.000
12.130   24.240   cppr_CLKBUFA_1/Y A->Y       1.213     4.000
12.130   36.370   cppr_CLKBUFA_2/Y A->Y       1.213     2.000
12.170   48.540   cppr_CLKBUF3_1/Y A->Y       1.217     2.000
0.000    48.540   cppr_DDF4_1/CK   -          1.215     2.000
#-------------------------------------------------------------
```

timing_aocv_analysis_mode combine_launch_capture (GBA)

```
Path 1: VIOLATED (-48.474 ns) Hold Check with Pin cppr_DDF4_1/CK->D
View: default_analysis_view_setup
Group: clk_cppr
Startpoint: (F) data_cppr
Clock:
Endpoint: (F) cppr_DDF4_1/D
Clock: (R) clk_cppr
                 Capture     Launch
Clock Edge:+     0.000       0.000
Src Latency:+    0.000       0.000
Net Latency:+   48.540 (P)   0.000 (I)
Arrival:=       48.540       0.000

Hold:+             -0.026
AOCV Adjustment:-   0.040
Cppr Adjust:-       0.000
Required Time:=    48.474
Launch Clock:=     0.000
Data Path:+        0.000
Slack:=          -48.474
Timing Path:
#-------------------------------------------------------------#
#Delay   Arrival  Pin              Arc         Aocv     Aocv  #
#(ns)    (ns)                                 Derate   Stage #
#                                             Count #
#-------------------------------------------------------------#
0.000   0.000     data_cppr        data_cppr   -        1.000
0.000   0.000     cppr_DDF4_1/D    -           0.814    1.000
#-------------------------------------------------------------
Other End Path:

#-------------------------------------------------------------#
#Delay   Arrival  Pin              Arc         Aocv     Aocv  #
#(ns)    (ns)                                 Derate   Stage #
```

```
#                                                                Count #
#-----------------------------------------------------------------------#
0.000    0.000    clk_cppr          clk_cppr   -       5.000
12.110   12.110   cppr_CLKBUF_0/Y   A->Y       1.211   4.000
12.130   24.240   cppr_CLKBUFA_1/Y  A->Y       1.213   4.000
12.130   36.370   cppr_CLKBUFA_2/Y  A->Y       1.213   2.000
12.170   48.540   cppr_CLKBUF3_1/Y  A->Y       1.217   2.000
0.000    48.540   cppr_DDF4_1/CK    -          1.215   2.000
#-----------------------------------------------------------------------
```

timing_aocv_analysis_mode combine_launch_capture (PBA)

```
Path 1: VIOLATED (-48.474 ns) Hold Check with Pin cppr_DDF4_1/CK->D
Retime Analysis:  Data Path-Slew
View: default_analysis_view_setup
Group: clk_cppr
Startpoint: (F) data_cppr
Clock:
Endpoint: (F) cppr_DDF4_1/D
Clock: (R) clk_cppr
                  Capture     Launch
Clock Edge:+      0.000       0.000
Src Latency:+     0.000       0.000
Net Latency:+    48.540 (P)    0.000 (I)
Arrival:=        48.540        0.000

               Hold:+   -0.026
AOCV Adjustment:-     0.040
Cppr Adjust:-     0.000
Required Time:=    48.474
Launch Clock:=     0.000
Data Path:+    0.000
Slack:=  -48.474
Slack(original):=  -48.474
Timing Path:
#-----------------------------------------------------------------------#
#Delay   Arrival  Pin               Arc          Aocv     Aocv  #
#(ns)    (ns)                                   Derate   Stage #
#                                                        Count #
#-----------------------------------------------------------------------#
0.000    0.000  data_cppr      data_cppr  -         1.000
0.000    0.000  cppr_DDF4_1/D  -            0.814   1.000
#-----------------------------------------------------------------------
Other End Path:
#-----------------------------------------------------------------------#
#Delay   Arrival  Pin               Arc          Aocv     Aocv  #
#(ns)    (ns)                                   Derate   Stage #
#                                                        Count #
#-----------------------------------------------------------------------#
0.000    0.000    clk_cppr          clk_cppr    -        5.000
12.110   12.110   cppr_CLKBUF_0/Y   A->Y        1.211    4.000
12.130   24.240   cppr_CLKBUFA_1/Y  A->Y        1.213    4.000
12.130   36.370   cppr_CLKBUFA_2/Y  A->Y        1.213    2.000
12.170   48.540   cppr_CLKBUF3_1/Y  A->Y        1.217    2.000
0.000    48.540   cppr_DDF4_1/CK    -           1.215    2.000
#-----------------------------------------------------------------------
```

4.3.3.9 AOCV Reporting

The Tempus software allows you to generate AOCV reports in both GBA and PBA modes.

If the set_db timing_analysis_aocv true is set, the impact of each reporting command is as follows:

- report_timing -retime aocv

  PBA stage counting will be used for calculating AOCV derate factor for the given path. GBA slew will be used for slew propagation.

- report_timing -retime aocv_path_slew_propagation

  In this case, PBA stage counting will be used for calculating the AOCV derate factor for the given path. PBA slew will be used for slew propagation.

- report_timing

  In this case, GBA stage counting will be used for calculating the AOCV derate factor for the given path. GBA slew will be used for slew propagation.

4.3.3.10 Reporting with OCV/AOCV Derating

In a timing report there may be a combination of OCV and AOCV derating factors - aocv_derate and user_derate (OCV) fields of the report. An additional stage_count field is available to aid in better understanding the derivation of the aocv_derate factor.

In the previous example of an additive combination of OCV and AOCV, one of the reference points was 0 and the other was 1. In such cases, the software does not make any adjustments to normalize the final derate factor so the timing report columns are quite intuitive.

In the timing run below, the nominal delay for all arcs has been set to 1.0ns so that the value in the Delay field indicates the total derate factor applied.

```
set_db timing_report_fields {instance cell arc stage_count aocv_derate
user_derate delay arrival}
report_timing -net ...
```

```
+-------------------------------------------------------------------------+
| Instance |  Cell   |     Arc     |  Aocv |  Aocv | User | Delay | Arrival |
|          |         |             | Stage | Derate | Derate |      | Time  |
|          |         |             | Count |        |        |      |       |
|----------+---------+-------------+-------+--------+--------+------+---------|
| RS       | DFFX1   |             | 7.000 | 1.030 | 1.000 | 2.030 | 20.450 |
| RS       | DFFX1   | CK ^ -> Q v | 6.000 | 1.030 | 0.010 | 1.040 | 21.490 |
| U1       | BUFXX   |             | 7.000 | 1.030 | 1.000 | 2.030 | 23.520 |
| U1       | BUFXX   | A v -> Y v  | 6.000 | 1.030 | 0.030 | 1.060 | 24.580 |
```

If the derating factors are changed such that both AOCV and OCV are referenced to 1.0, then an implicit -1 will be used since the mode is still aocv_additive. In this case, the effect of the '-1' will be seen in the aocv_derate value.

For example, set both the reference points to 0, and modify the AOCV library so that it outputs a value of 0.030ns.

```
set_db timing_derate_aocv_reference_point 0
set_db timing_derate_ocv_reference_point 0
set_timing_derate -delay_corner dcMax -late -cell_delay 1.01
set_timing_derate -delay_corner dcMax -late -cell_delay 0.02 -add [get_cells U1]
```

Instance	Cell	Arc	Aocv Stage Count	Aocv Derate	User Derate	Delay	Arrival Time
RS	DFFX1		7.000	1.030	1.000	2.030	20.450
RS	DFQX1	CK ^ -> Q v	6.000	1.030	0.010	1.040	21.490
U1	BUFXX		7.000	1.030	1.000	2.030	23.520
U1	BUFXX	A v -> Y v	6.000	1.030	0.030	1.060	24.580

In this case, the +1.0 adjustment to normalize the final derating factor is again accounted for in the aocv_derate column.

AOCV Derate and Stage Count Properties of Timing Arcs

You can use stage count and AOCV derate properties to report and query on AOCV data. As an example consider the following path:

set_db timing_report_fields {arc hpin aocv_derate stage_count} -path_type full_clock

```
Endpoint:  lat3/D
Beginpoint: in2
-------------------------------------------------------------------------------
Arc              Pin                        Aocv  Derate        Aocv Stage Count
-------------------------------------------------------------------------------
in2 v            in2
A v -> Y v       buf133/Y                      1.217                    3.000
A v -> Y v       buf5/Y                        1.217                    3.000
A v -> Y v       buf0/Y                        1.217                    3.000
D v              lat3/D                        1.217                    3.000
-------------------------------------------------------------------------------
Other End Path:
-------------------------------------------------------------------------------
Arc              Pin                       Aocv  Derate        Aocv Stage Count
-------------------------------------------------------------------------------
clk^              clk
A ^ -> Y ^       buf11/Y                    0.888                    2.000
A ^ -> Y v       inv12/Y                    0.888                    2.000
GN v            lat3/GN                     0.888                    2.000
-------------------------------------------------------------------------------
```

For example, obtaining the various stage counts corresponding to the above path is shown below:

```
get_property [get_arcs -from buf133/A -to buf133/Y] aocv_stage_count_data_late
3.000
get_property [get_arcs -from buf11/A -to buf11/Y]
aocv_stage_count_capture_clock_early
```

To obtain derate corresponding to the above path:

```
get_property [get_arcs -from buf11/A -to buf11/Y]
aocv_derate_capture_clock_early_rise 0.888
```

For more information on AOCV derate properties, refer to report_property and get_property commands in the *Tempus Text Command Reference*.

4.3.4 Statistical On-Chip Variation (SOCV) Analysis

4.3.4.1 Overview on page 171

4.3.4.2 Introduction to SOCV on page 171

4.3.4.3 SOCV Data Flow on page 172

4.3.4.4 SOCV Analysis on page 174

4.3.4.5 Enabling SOCV Analysis on page 174

4.3.4.6 SOCV Analysis Reporting on page 175

4.3.4.7 Sample Variation on Delay Template from Liberty 2013.12 on page 182

4.3.4.8 Timing Behaviors in SOCV Analysis Mode on page 184

4.3.4.9 Specifying Statistical Via Variation File on page 184

4.3.4.10 Advanced Timing Tcl Scripting in SOCV Analysis Mode on page 185

4.3.4.1 Overview

Industrial demand for higher performing digital semiconductor products has highlighted the clear need to reduce design guard-band without sacrificing time-to-market. Improvements to on-chip variation modeling are a driving component of the guard band reduction strategy. In this section we review the current industry approaches to on-chip variation (OCV) modeling and introduce the Statistical OCV (SOCV) analysis, which utilizes a single statistical parameter representing sigma variation. The SOCV extends existing single variable statistical approaches by adding slew and load-dependent sigma per timing arc.

For many years, modeling of the on-chip process variation has been achieved through scaling of the nominal delays of all instances with a single derating factor. Such a uniform derating, often referred to as OCV derate, often causes excessive pessimism for some instances while optimism for others. In order to guarantee conservative analysis, OCV derates were chosen pessimistically, resulting in over-design and increase of turnaround times for timing closure.

To overcome these setbacks, an advanced OCV (AOCV) approach was introduced. With AOCV the derating factor is instance specific and depends on the number of logic levels in a path and/or location of the instance. The AOCV derating factors are pre-characterized for each cell typically using Monte-Carlo simulation. This methodology reduced pessimism and optimism of the traditional OCV methodology. However, AOCV presented challenges for computing of the stage count and distance of the path on a die for a specific path in graph-based analysis (GBA). In addition, AOCV assumes that the ratio of delay variation to nominal delay does not depend on the input slew and load.

On the opposite spectrum of the methods modeling variability in STA is statistical static timing analysis (SSTA). It models delay distributions due to local and global random parameter variations accurately and is, therefore, much more accurate than other mentioned methods. However, SSTA is much more expensive in terms of run time and memory, and requires special timing libraries, which is often too high cost to pay for design houses and foundries.

The SOCV analysis feature is available only for Tempus-XL (TPS200) license. For more details on licensing, refer to “Product and Licensing Information” chapter.

4.3.4.2 Introduction to SOCV

Statistical OCV (SOCV) is a new analysis technique that provides for a good compromise between the run time and accuracy for modeling variation.

SOCV computes the impact of local process variations on the delay and slew of each instance in the design at a given global variation corner. SOCV uses one parameter in SSTA mode. This single parameter models local cell variations. Similar to SSTA, SOCV propagates the sigma of arrival and required times through the timing graph, and then computes

statistical characteristics of slack at all timing pins. Modeling on-chip variations in terms of a single variable is fast, yet it yields better accuracy than modeling the variations as derating factors.

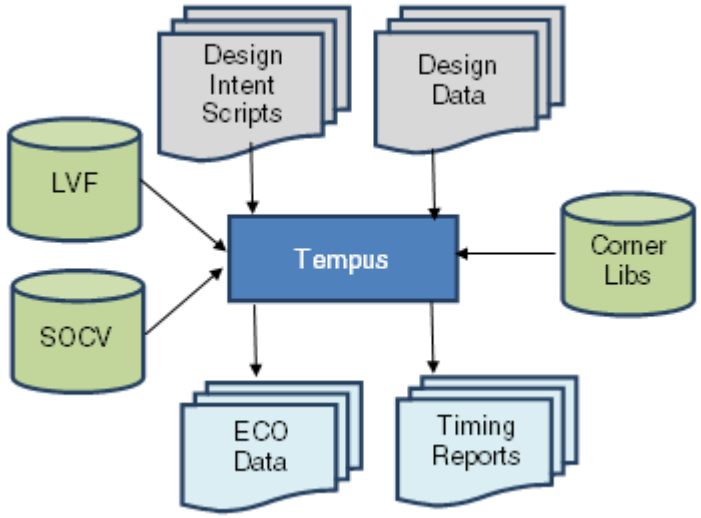
For accurate SOCV analysis, a special library data is required. It includes arc-level absolute variations of cell delays, output transitions, and timing checks as functions of input slew and load. Since SOCV is able to accurately compute the delay and slew variation on each instance, it solves the two main limitations of simpler methods: (i) cases where delay is close to zero while delay variation is not, and (ii) impact of input slew variations on delay.

SOCV analysis is currently available in the Cadence Tempus Timing Solution for sign-off timing analysis and timing closure.

4.3.4.3 SOCV Data Flow

SOCV analysis requires only the addition of variation libraries to the typical timing sign-off and closure configuration. Variation data can be provided either as a Cadence SOCV library or as Liberty Variation Format (LVF) library. Tempus fully supports both library formats including modeling for variation on delay, transition, and constraints.

Figure 4-73 SOCV Data Flow



Reading Variation Library Data

Tempus provides both a single-corner and MMMC use model, which can accommodate SOCV or LVF format variation libraries.

In single-corner mode, you can use the following parameter to read Cadence SOCV format libraries:

```
read_libs -socv_libs [list libA.socv libB.socv … ]
```

To read Liberty libraries with integrated LVF data, you should read the library in the same manner as a non-LVF Liberty timing library.

```
read_libs [list libA-LVF.lib libB-LVF.lib …]
```

When working in an MMMC environment, you should specify the variation libraries as part of the `create_library_set` command in your MMMC configuration file. To incorporate Cadence SOCV format libraries:

```
create_library_set -name libsetMax
-timing [list
${library_path_1}/lib1.lib
${library_path_1}/lib2.lib
]
-socv [list
${socv_lib_path}/lib1.socv
${socv_lib_path}/lib2.socv
]
```

To use Liberty libraries with merged LVF data, simply load the Liberty libraries with the `create_library_set -timing` parameter.

Using AOCV Derating Libraries with SOCV Analysis

Tempus allows the option of using AOCV libraries to provide variation modeling data for the SOCV flow. This method can be used for initial experimentation and testing of the SOCV flow, but lacks the accuracy required for signoff. Using AOCV as a source will provide only variation on delay; no variation on transition or constraints.

To utilize the AOCV bootstrap method, you must set the timing attributes to control transformation of AOCV data. These settings must be made before loading any timing library data.

To enable the transformation process, use the following command:

```
set_db timing_library_infer_socv_from_aocv true
```

The AOCV derating libraries are characterized to a particular sigma value - without additional guidance the transformation process will consider the AOCV library to be at 3-sigma. To define the characterization sigma of the AOCV libraries, you can make the following setting:

```
set_db timing_library_scale_aocv_to_socv_to_n_sigma 4.5
```

You should specify reading the AOCV libraries in the same way as for normal AOCV analysis.

For single-corner mode, use the following command:

```
read_libs -aocv_libs {myLib.aocv …}
```

For a MMMC environment, you can make the following settings:

```
create_library_set -name libsetMax
-timing [list
     ${library_path_1}/lib1.lib
     ${library_path_1}/lib2.lib
]
-aocv [list
     ${aocv_lib_path}/lib1.aocv
     ${aocv_lib_path}/lib2.aocv
]
```

4.3.4.4 SOCV Analysis

The SOCV analysis is essentially an optimized, single parameter statistical analysis, that is built on the underlying Tempus SSTA analysis engine. From a user-interface point of view, the software is able to present results in terms of discrete values for delays, arrivals, and slacks. The interface can also show statistical representation of data.

Enabling SOCV Analysis

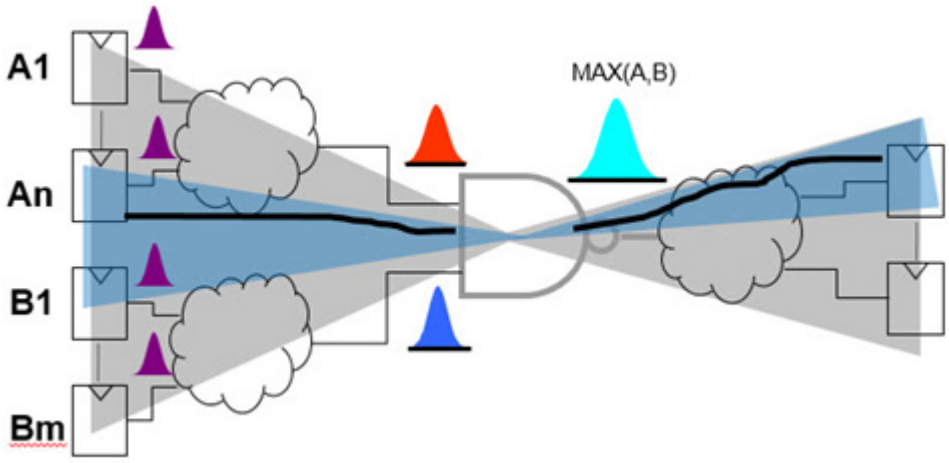
The SOCV analysis can be enabled using the `timing_analysis_socv` attribute, as shown below:

```
set_db timing_analysis_socv true  timing_analysis_type ocv
timing_analysis_cppr both
```

Configuring SOCV Max Operation

During arrival and required timing propagation, timing data values are represented concurrently by statistical mean and sigma values. At the output of multi-input gates, the statistical arrival times must be merged in a conservative operation. There are several options for controlling the statistical Max operation.

Figure 4-74 SOCV Max Operation



Selection of the Max operation can be controlled by making the following setting:

`set_db timing_socv_statistical_min_max_mode mean_and_three_sigma_bounded`

In this mode, the worst-case calculations of mean value and standard deviation are performed separately.

4.3.4.5 SOCV Analysis Reporting

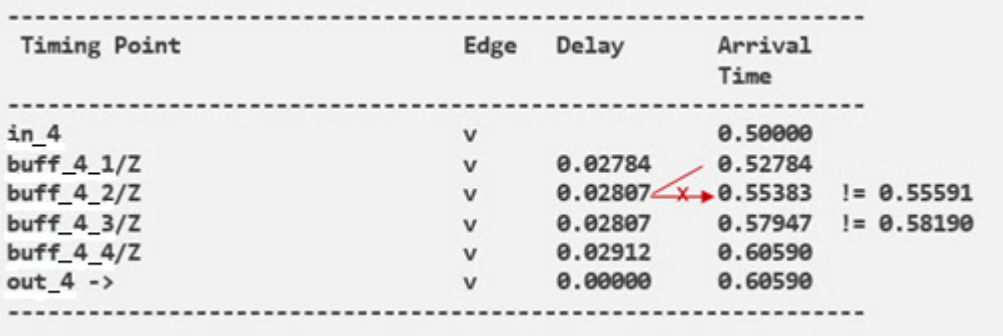
Reports in SOCV analysis mode will, by default, present timing data in discrete numbers rather than as statistical quantities. If desired, the timing report can be configured to report the statistical mean and sigma of the various timing data. The format of the timing report can be customized by a combination of global settings and `report_timing -fields` options.

All the timing calculations are performed in the statistical domain. For example, adding delays along a path to determine an arrival time calculation of slack-based on subtraction of arrival and required times. Discrete representations of these values are computed as mean + n*sigma – where n is typically 3.0.

Path Reporting With Discrete Values

The default timing report is configured to report timing quantities as discrete numbers rather than as statistical quantities. The `report_timing` fields: slew, delay, and arrival will still report discrete numeric values.

As a result of calculating arrival times in the statistical realm, adding the current delay to the previous arrival will generally not add up exactly to the next arrival time in the report:



Timing Point	Edge	Delay	Arrival Time	
in_4	v		0.50000	
buff_4_1/Z	v	0.02784	0.52784	
buff_4_2/Z	v	0.02807	0.55383	!= 0.55591
buff_4_3/Z	v	0.02807	0.57947	!= 0.58190
buff_4_4/Z	v	0.02912	0.60590	
out_4 ->	v	0.00000	0.60590	

The statistical mean and sigma representation of timing data is converted to discrete values by the formula: mean + n-sigma, where n is a user-specifiable value. To control the sigma multiplier, you can use the `set_socv_reporting_nsigma_multiplier` command. This command supports n-sigma specification per view per setup/hold combination.

To apply sigma multiplier to the specified view in setup mode, use the following command:

```
set_socv_reporting_nsigma_multiplier -setup 2.5 -view std_max_setup
```

Path Reporting with Statistical Data

The `report_timing` command now supports additional statistical report fields when operating in SOCV mode.

- The delay_mean and delay_sigma fields provide statistical data for each timing arc
- The slew_mean and slew_sigma fields provide data for transitions
- The arrival_mean and arrival_sigma fields provide the statistical information for arrival timing

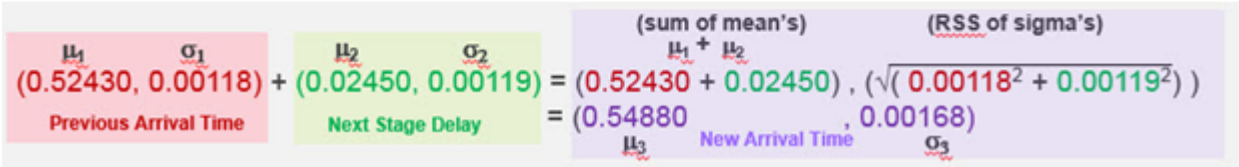
The conversion of statistical quantities in the report to discrete values is computed by:

- New mean is the sum of the means: $\mu 3 = \mu 1 + \mu 2$

- New sigma is the root-sum-squared of sigmas: σ3 = σ1^2 + σ2^2

report_timing Output With Statistical Fields and Discrete Conversion at 3-sigma

Timing Point	Edge	Delay Mean	Delay Sigma	Delay ($\mu + 3*\sigma$)	Arrival Mean	Arrival Sigma	Arrival ($\mu + 3*\sigma$)
in_4	v				0.50000	0.00000	0.50000
buff_4_1/Z	v	0.02430	0.00118	0.02784	0.52430	0.00118	0.52784
buff_4_2/Z	v	0.02450	0.00119	0.02807	0.54880	0.00168	0.55383
buff_4_3/Z	v	0.02450	0.00119	0.02807	0.57330	0.00206	0.57947
buff_4_4/Z	v	0.02540	0.00124	0.02912	0.59870	0.00240	0.60590
out_4 ->	v	0.00000	0.00000	0.00000	0.59870	0.00240	0.60590



Summary Section Reporting

The summary section of the `report_timing` report is where the final arrival and required times are calculated, and the path slack is determined. Other quantities such as CPPR, Uncertainty, and Setup/Hold checks values are also accounted in this section's calculations. Like the detailed path report, all calculations are initially performed using statistical arithmetic of the mean and sigma values. The discrete representations of these values are reported based on the via mean +n-sigma conversion.

By default, the `report_timing` command output will display discrete representations of the data in the summary section of the report. For example,

```
report_timing -path_type summary -from [all_reg] -to [all_reg] >
socv_nonverbose.rpt
############################################################
Path 1: MET (19.921 ns) Setup Check with Pin lsi/A1/CP->D
              Group: clk2
         Startpoint: (R) lsi/A15/CP
              Clock: (R) clk2
           Endpoint: (F) lsi/A1/D
              Clock: (R) clk2
            N-Sigma: 3.000


                      Capture      Launch
         Clock Edge:+   26.000       6.000
        Src Latency:+    0.000       0.000
        Net Latency:+    0.012 (P)   0.019 (P)
```

```
          Arrival:=   26.012       6.019

            Setup:-    0.021
      Cppr Adjust:+    0.006
    Required Time:=   25.995
     Launch Clock:=    6.019
        Data Path:+    0.065
            Slack:=   19.921
```

To enable the display of related statistical representation of data in the summary section, you can set the following attribute to `true`:

`timing_report_enable_verbose_ssta_mode`

With this setting, additional details will be added to the `report_timing` summary section. The current “n” value for n-sigma conversion is shown, along with the mean and sigma components for each data value. For example,

```
report_timing -path_type summary -from [all_reg] -to [all_reg] > socv_verbose.rpt
############################################################
Path 1: MET (19.921 ns) Setup Check with Pin lsi/A1/CP->D
             Group: clk2
        Startpoint: (R) lsi/A15/CP
             Clock: (R) clk2
          Endpoint: (F) lsi/A1/D
             Clock: (R) clk2
           N-Sigma: 3.000

                    Capture                        Launch
       Clock Edge:+   26.000                        6.000
      Src Latency:+    0.000    ( 0.000, 0.000)      0.000   (0.000, 0.000)
      Net Latency:+    0.012    ( 0.014, 0.001) (P)  0.019   (0.014, 0.002)(P)
          Arrival:=   26.012    (26.014, 0.001)      6.019   (6.014, 0.002)

            Setup:-    0.021   ( 0.011, 0.003)
      Cppr Adjust:+    0.006   ( 0.000, 0.002)
    Required Time:=   25.995   (26.003, 0.003)
     Launch Clock:=    6.019   ( 6.014, 0.002)
        Data Path:+    0.065   ( 0.052, 0.004)
            Slack:=   19.921   (19.936, 0.005)
```

Sample Initialization and Configuration Scripts

Single-Corner Use Model (non-MMMC) - SOCV Native Libraries

```
set_db timing_socv_statistical_min_max_mode three_sigma_bounded
set_socv_reporting_nsigma_multiplier -setup 2.5
set_db timing_report_enable_verbose_ssta_mode true
set timing_report_fields {pin edge cell transition load \
delay_mean delay_sigma delay \
arrival_mean arrival_sigma arrival}

read_libs ${LIB_DIR}/standard.lib
read_lib -socv_libs [list ${LIB_DIR}/socv/standard.socv]
```

```
read_netlist ./test.v
init_design
read_sdc ./test.sdc
set_db timing_analysis_socv true timing_analysis_cppr both
report_timing
```

Single-Corner Use Model (non-MMMC) - LVF Native Libraries

```
set_db timing_socv_statistical_min_max_mode three_sigma_bounded
set_socv_reporting_nsigma_multiplier -setup 2.5
set_db timing_report_enable_verbose_ssta_mode true
set_db timing_report_fields {pin edge cell transition load \
delay_mean delay_sigma delay \
arrival_mean arrival_sigma arrival}
read_libs ${LIB_DIR}/standard-lvf.lib
read_netlist ./test.v
init_design
read_sdc ./test.sdc
set_db timing_analysis_socv true timing_analysis_cppr both
report_timing
```

Single-Corner Use Model (non-MMMC) - AOCV Bootstrap Mode

When running in AOCV bootstrap mode, load the AOCV libraries. Before loading any library information, you must ensure that the required timing library attributes have been set.

```
set_db timing_library_infer_socv_from_aocv true
set_db timing_library_scale_aocv_to_socv_to_n_sigma 3
set_db timing_socv_statistical_min_max_mode three_sigma_bounded
set_socv_reporting_nsigma_multiplier -setup 2.5
set_db timing_report_enable_verbose_ssta_mode true
set_db timing_report_fileds {pin edge cell transition load delay_mean delay_sigma
delay arrival_mean arrival_sigma arrival}
read_libs ${LIB_DIR}/standard.lib
read_libs -aocv_libs [list ${LIB_DIR}/aocv/standard.aocv]
read_netlist ./test.v
init_design
read_sdc ./test.sdc
set_db timing_analysis_socv true timing_analysis_cppr both
report_timing
```

MMMC Use Model Using Native SOCV or LVF Libraries

In MMMC configurations, library data is provided through a MMMC library set object, using the `create_library_set` command.

In the following examples native SOCV or LVF format library is used to make changes in the MMMC configuration file.

```
set_db timing_socv_statistical_min_max_mode three_sigma_bounded
set_socv_reporting_nsigma_multiplier -setup 2.5
set_db timing_report_enable_verbose_ssta_mode true
set_db timing_report_format {pin edge cell transition load \
delay_mean delay_sigma delay \
arrival_mean arrival_sigma arrival}


read_mmmc ./file_name.tcl
read_netlist ./test.v
init_design

set_db timing_analysis_socv true timing_analysis_cppr both

report_timing
```

Common MMMC Initialization Script With Native SOCV or LVF

```
create_library_set -name libsetMax \
-timing [list ${library_path_1}/lib1.lib ${library_path_1}/lib2.lib ] \
-socv [list \
    ${socv_lib_path}/lib1.socv \
    ${socv_lib_path}/lib2.socv \
]
```

Portion of MMMC Configuration Script (viewDef.tcl) for Loading Native SOCV Libraries

```
create_library_set -name libsetMax \
-timing [list \
     ${library_path_1}/lib1-lvf.lib \
     ${library_path_1}/lib2-lvf.lib \
] \
```

Portion of MMMC Configuration Script (viewDef.tcl) For Loading AOCV Libraries

MMMC Use Model– AOCV Bootstrap Mode

When running in AOCV bootstrap mode, load the AOCV libraries. You must ensure that the required timing library attributes are set before loading any library information.

```
set_db timing_library_infer_socv_from_aocv true
set_db timing_library_scale_aocv_to_socv_to_n_sigma 3

set_db timing_socv_statistical_min_max_mode three_sigma_bounded
set_socv_reporting_nsigma_multiplier -setup 2.5
set_db timing_report_enable_verbose_ssta_mode true

set_db timing_report_format {pin edge cell transition load \
delay_mean delay_sigma delay \
arrival_mean arrival_sigma arrival}

read_mmmc./viewDef.tcl

read_netlist ./test.v
init_design

set_db timing_analysis_socv true timing_analysis_cppr both
report_timing
```

MMMC Initialization Script for AOCV Bootstrap Mode

```
create_library_set -name libsetMax \
-timing [list \
    ${library_path_1}/lib1.lib \
    ${library_path_1}/lib2.lib \
] \
-aocv [list \
     ${aocv_lib_path}/lib1.aocv \
     ${aocv_lib_path}/lib2.aocv \
]
```

Variation Library Formats

The Cadence SOCV library format and the standardized Liberty Variation Format (LVF) provide essentially the same information: variation sigma for delay, transition, and constraints per arc, indexed by load and slew.

Liberty Variation Format (LVF)

The detailed specifications of Liberty LVF for variation on delay and transition are available from OpenSource Liberty Version 2013.12.

```
cell (cell_name) {
ocv_derate_distance_group: ocv_derate_group_name;
...
pin | bus | bundle (name) {
direction: input | output;
timing() {
  ...
   ocv_sigma_cell_rise(delay_lu_template_name){
     sigma_type: early | late | early_and_late;
     index_1 ("float, ..., float");
     index_2 ("float, ..., float");
     values ( "float, ..., float", \
         ..., \
         "float, ..., float");
  }
   ocv_sigma_cell_fall(delay_lu_template_name){
      sigma_type: early | late | early_and_late;
      index_1 ("float, ..., float");
      index_2 ("float, ..., float");
         values ( "float, ..., float", \
         ..., \
        "float, ..., float");
     }
    ...
  } /* end of timing */
  ...
  } /* end of pin */
...
```

4.3.4.6 Sample Variation on Delay Template from Liberty 2013.12

Cadence SOCV Library Format (.socv)

The SOCV library format is as follows:

```
object_spec: libXYZ/NAND2
pin: Y
pin_direction: rise
related_pin: A
related_direction: rise
type: delay
when:
related_transition:  0.0002, 0.0050, 0.0125, 0.0500
output_loading:      0.0002, 0.0015, 0.0035, 0.0090
sigma:
  0.0029, 0.0034, 0.0042, 0.0066
  0.0035, 0.0036, 0.0044, 0.0065
  0.0039, 0.0043, 0.0050, 0.0072
  0.0061, 0.0066, 0.0071, 0.0088
```

Variation On Delay

```
object_spec: libXYZ/NAND2
pin: Y
pin_direction: rise
related_pin: A
related_direction: rise
type: slew
when:
related_transition:  0.0002, 0.0050, 0.0125, 0.0500
output_loading    :  0.0002, 0.0015, 0.0035, 0.0090
distance:
sigma:
  0.0006, 0.0024, 0.0037, 0.0072
  …
```

Variation on Slew

```
object_spec: libXYZ/DFF
pin: D
pin_direction: rise
related_pin: CK
related_direction: rise
type: hold
when:
related_transition:    0.0002, 0.0200, 0.0500, 0.1200
constraint_transition: 0.0002, 0.0200, 0.0400, 0.0900
distance:
sigma:
  0.0762, 0.0756, 0.0749, 0.0732
  …
```

Variation on Checks

The following table describes the SOCV Library Attributes:

SOCV Library Attribute	Attribute Type	Description
object_type	library | design	Use library to indicate that the SOCV data is with respect to library cells. Use design, for design-specific data such as spatial derate multipliers.
object_spec	libName/ libCellName	Specifies the internal library name and library cell name that is consistent with a Liberty timing library that has been loaded in the session. The SOCV library is considered an adjunct library file which inherits certain attributes – such as PVT from the master Liberty library. Simple wildcard expressions are allowed for both *libName* and *libCellName*.
pin	pinName	The output or constrained pin name of the arc being represented by this section.
pin_direction	rise | fall	The direction of the transition at the output pin.
related_pin	pinName	The related pin name of the arc being represented by this section.
related_direction	rise | fall	The direction of the transition at the related pin.
type	delay | slew | setup | hold | recovery| removal	Indicates whether the current arc section is modeling variation on delay, transition, or constraints.
when	Liberty compliant 'when' attribute string	The Liberty 'when' attribute string consistent with the same timing arc in the Liberty library.
related_transition	float+	Comma-separated list of transition indices on the arc's related_pin.
output_loading	float+	Comma-separated list of output load indices.
constraint_transition	float+	For constraint data, a comma-separated list of transitions on the arc's constrained pin.

sigma	2-D table of float	Table of variation data indexed by transition and load.
late_distance	float+	Comma-separated list of distance values.
late_distance_derate	float+	1-D table of spatial derating factors for application to late paths.
early_distance	float+	Comma-separated list of distance values.
early_distance_derate	float+	1-D table of spatial derating factors for application to late paths.

4.3.4.7 Timing Behaviors in SOCV Analysis Mode

- All timing constraint assertions (SDCs) are interpreted as a specification of the mean value with a sigma value of 0.0.
- By default, the `set_timing_derate` factors are applied to both the mean and sigma values. You can use the `-mean`/`-sigma` parameters to apply separate derating to the mean/sigma components of the delay.
- The `read_sdf` values are interpreted as annotations to the mean value – the sigma value is 0.0.
- The delay values written out by `write_sdf` are the mean + n-sigma representation. If the selected analysis type is SOCV, you can use the `-no_variation` parameter to export the SDF delay and check values without including the variation component.
- All timing reports are in terms of the mean + n-sigma discrete transformation of the value.
- The `report_delay_calculation` command by default reports the mean component of delay.

4.3.4.8 Specifying Statistical Via Variation File

You can specify a via variation file for modeling via variation. The file includes all the via layer names so that VIA resistance in SPEF can be mapped to their own variation multipliers. For a single corner design, use the `-via_variation_file` parameter of the `read spef` or `read_parasitics` command.

For MMMC designs, specify the file using the -via_variation_file parameter of the `create rc corner` or`update rc corner` command.

You can specify a different via variation file for each RC corner. The syntax is provided below:

```
read_spef/read_parasitics
   -rc_corner rcCornerName \
   -via_variation_file viaVariationFileName
create_rc_corner
   -rc_corner rcCornerName1 \
   -via_variation_file viaVariationFileName1
update_rc_corner
   -rc_corner rcCornerName2 \
   -via_variation_file viaVariationFileName2
```

4.3.4.9 Advanced Timing Tcl Scripting in SOCV Analysis Mode

The following properties are added to the `timing_arc` object and can be specified using the `get_property` command:

- `delay_mean_max_fall`
- `delay_mean_max_rise`
- `delay_mean_min_fall`
- `delay_mean_min_rise`
- `delay_sigma_max_fall`
- `delay_sigma_max_rise`
- `delay_sigma_min_fall`
- `delay_sigma_min_rise`

The following properties are added to the `pin` object and can be queried using the `get_property` command:

- `arrival_mean_max_fall`
- `arrival_mean_max_rise`
- `arrival_mean_min_fall`
- `arrival_mean_min_rise`
- `arrival_sigma_max_fall`

- arrival_sigma_max_rise
- arrival_sigma_min_fall
- arrival_sigma_min_rise
- slack_mean_max
- slack_mean_max_fall
- slack_mean_max_rise
- slack_mean_min
- slack_mean_min_fall
- slack_mean_min_rise
- slack_sigma_max
- slack_sigma_max_fall
- slack_sigma_max_rise
- slack_sigma_min
- slack_sigma_min_fall
- slack_sigma_min_rise
- slew_mean_max_fall
- slew_mean_max_rise
- slew_mean_min_fall
- slew_mean_min_rise
- slew_sigma_max_fall
- slew_sigma_max_rise
- slew_sigma_min_fall
- slew_sigma_min_rise

4.4 Parametric Timing Robustness Analysis

4.4.1 Parametric Timing Robustness (PTR) Analysis Overview on page 188

4.4.2 Library Requirements on page 190

4.4.3 Early/Late Sigma LVF on page 190

4.4.4 LVF Moment Models on page 191

4.4.5 Enabling PTR Analysis on page 192

4.4.6 PTR Analysis Reports on page 192

4.4.1 Parametric Timing Robustness (PTR) Analysis Overview

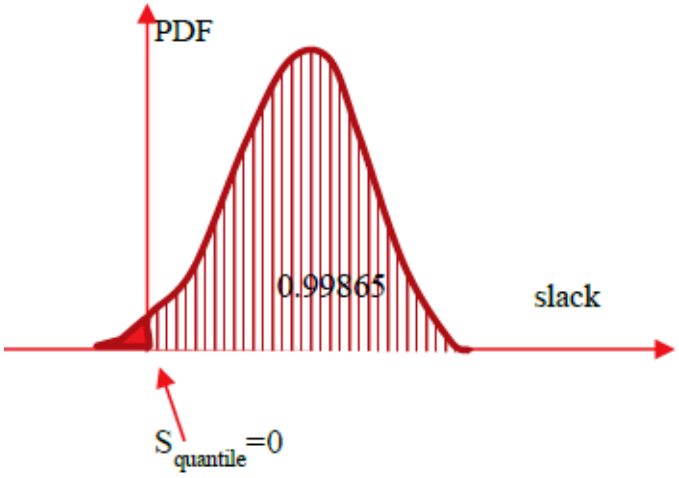
Tempus supports Parametric Timing Robustness (PTR) analysis capabilities that can be leveraged to calculate the probability of the entire chip meeting timing, instead of relying on the probabilities on individual paths meeting timing to compute the robustness of a chip.

In traditional SOCV timing analysis, the moments of delays and slews calculated at the arc and instance level are propagated through the paths in the design and Arrival Time, Slack distributions are computed at the path endpoints.

In the following example, the 99.865 quantile values of the endpoint slack distributions show the probability of the paths to the given endpoint meeting timing.

The figure below is the slack distribution of a given path with 99.865 percentile slack of zero. This means that the area under slack's PDF on the right of zero is exactly 0.99865, and on the left of zero it is 0.00135. The area on the left of zero corresponds to samples (chips) where slack of this path is negative (failing path). In other words, out of 1 million samples (chips) there will be about 1,350 of them where this path fails.

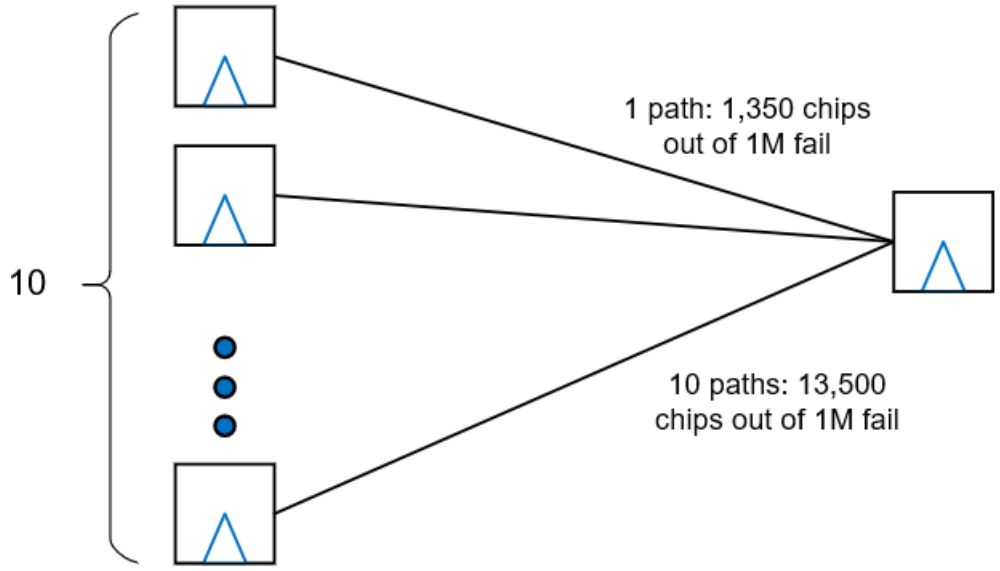
Figure 4-75



Similarly, in a case with 10 similar paths ending at the same end-point and all having 99.865 percentile slack of zero, assuming that the slack of each path is an independent random variable, the probability of each of them to fail is 0.00135, and then the probability of at least one of them to be negative is Pneg =1 – (1-0.00135)^10 =0.0134, which means that the probability of the end point to have negative slack (fail) is almost 10x of that of each individual path.

Considering 1M samples (chips), in this situation 13,400 of them will fail due to that end-point.

Figure 4-76



Similarly, when we consider 1M such endpoints, the probability of all of them to have non-negative slack is 0.99865^10000000, which is nearly zero. This means nearly all samples (chips) will fail.

However in reality, chips work with reasonably high robustness, even though they are optimized to make end-point's quantile slacks close to zero. This is because paths and end-points exhibit a non-zero correlation, and hence the calculation of probability used above is not valid.

In SOCV slacks are random variables, which are results of statistical addition of delays on the paths leading to end-points. If paths have common parts, the end-points' slacks are not independent variables, or in other words, have non-zero covariance (correlation). Computing correlation between end-point's slacks is very tedious task, leading to at least quadratic complexity over number of paths reported (and we will need to report slightly positive paths since the tails of their distributions will extend into negative slack region). This makes the correlation calculation through pair-wise common-portion calculation computationally unrealistic.

The Parametric Timing Robustness analysis models the correlation of paths in the computation of chip robustness, is akin to Monte-Carlo simulation. Here samples of delays and slews on each arc and pin are generated and propagated on both data and clocks paths such that correlation is modeled naturally in the computation of Arrival Time and Slacks. Any two paths crossing same arc will include same delay distribution as part of their ATs.

The PTR computed using this approach gives an accurate measure of the 'actual' chip robustness.

4.4.2 Library Requirements

The PTR analysis requires variation data to be characterized into the timing libraries. The Liberty Validation Format (LVF) has been adopted by the industry as a common vehicle for providing this information. The syntax for modeling the random variation of delays, transitions, and constraints with tables of sigma values is the most commonly available. Cadence Liberate® and Variety® characterization software and Tempus® sign-off timing analysis software are capable of creating and using more advanced models for improved accuracy at advanced nodes.

It is strongly recommended to use moment-based LVF models in PTR Analysis in Tempus for good accuracy.

4.4.3 Early/Late Sigma LVF

The variation data modeled in traditional LVF provides Liberty ocv_sigma_* tables for delays, transitions, and timing checks. These represent the variation offset from the nominal values contained in the related NLDM table data. This data is specified in terms of 1-sigma.

The LVF allows separate specification of a sigma value for early path analysis vs. late path analysis. The sigma_type syntax identifies which ocv_sigma_* tables are used for what purpose. In cases where the mean of the distribution is not aligned with the nominal value, having separate characterizations can help improve accuracy. The constraint variation tables do not support separate early and late sigma values.

LVF Early/Late Sigma Groups	**Purpose**
ocv_sigma_cell_rise	Modeling delay variation
ocv_sigma_cell_fall	
ocv_sigma_rise_transition	Modeling transition variation
ocv_sigma_fall_transition	
ocv_sigma_rise_constraint	Modeling timing check variation
ocv_sigma_fall_constraint	
ocv_sigma_retaining_rise	Modeling retain arc delay variation
ocv_sigma_retaining_fall	

ocv_sigma_retain_rise_slew	Modeling retain arc transition variation
ocv_sigma_retain_fall_slew	

4.4.4 LVF Moment Models

The advanced nodes (< 16nm) and ultra-low VDD's can produce distributions that are strongly non-Gaussian, thereby exhibiting both mean-shift and skewness effects. As a result, three additional models were included in the LVF delay, transition, and constraint variation set. These models represent the three top central moments of the distributions. It is strongly recommended to use these moments-based modes for PTR analysis in Tempus.

The new constructs do not replace the existing ocv_sigma_* constructs already in use. The LVF consuming applications will decide how best to utilize the older and newer constructs together for the best possible results.

With the exception of the sigma_type attribute, the new constructs use the same syntax as the existing LVF sigma constructs.

LVF Early/Late Sigma Groups	**Purpose**
ocv_std_dev_cell_rise	Moment modeling of delay variation
ocv_std_dev_cell_fall	
ocv_mean_shift_cell_rise	
ocv_mean_shift_cell_fall	
ocv_skewness_cell_rise	
ocv_skewness_cell_fall	
ocv_std_dev_rise_transition	Moment modeling of transition variation
ocv_std_dev_fall_transition	
ocv_std_dev_fall_transition	
ocv_std_dev_fall_transition	
ocv_std_dev_fall_transition	
ocv_mean_shift_fall_transition	
ocv_skewness_rise_transition	
ocv_skewness_fall_transition	

4.4.5 Enabling PTR Analysis

The PTR analysis flow can be enabled using the following setting:

`set_db timing_analysis_enable_robustness_mode true`

License Requirements for PTR Analysis

PTR analysis requires the `tpsng` license.

PTR Analysis on Clock Paths in PBA Mode

PTR analysis is performed on clock paths in PBA mode for better accuracy. The setting to enable this is

`set_db timing_disable_retime_clock_path_slew_propagation false`

Delay Calculation Controls - Use of Moments LVF

LVF libraries with moments data are strongly recommended for PTR analysis. The setting to read in moments LVF data is

`set db delaycal socv lvf mode moments`

Accuracy Modes

PTR analysis is performed in the highest accuracy mode of statistical analysis. The setting to enable this is:

```
set_db delaycal_socv_accuracy_mode ultra
```

4.4.6 PTR Analysis Reports

The PTR score of a design, which is a statistical measure of the timing slack for the whole design can be reported using the report_timing_robustness command. This command reports the percentage of parts (to be manufactured) which meet timing at all end points.

The `report timing robustness` analysis command reports the PTR score of a given design based on the following steps:

- Enumerates all the instances in the design.
- Identifies paths that are critical for PTR of the design.
- Computes the PTR based on Monte Carlo analysis.

The following example shows the final PTR value:

```
report_timing_robustness -check_type setup -path_group R2R -target_PTR 0.8 -
run_time_limit 15
collecting paths and reporting PTR ......
Final PTR 0.89735


set P1 [report_timing_robustness -check_type setup -path_group R2R -target_PTR 0.95
-run_time_limit 15]
collecting paths and reporting PTR ......
Final PTR 0.91295
report_property $P1
----------------------------
property | value
----------------------------
slack_mean | 0.961705
slack_ptr | 0.912950
slack_quantile | -2.085161
slack_skewness | -0.668439
slack_stddev | 0.682927
----------------------------
```

4.5 Path-Based Analysis

4.4.1 PBA Overview on page 195

4.4.2 GBA vs PBA Comparison on page 195

4.4.3 Reporting in Path-Based Analysis on page 195

4.4.4 Path-Based Analysis Reporting Models on page 197

4.4.5 Path-Based Analysis Essence on page 199

4.5.1 PBA Overview

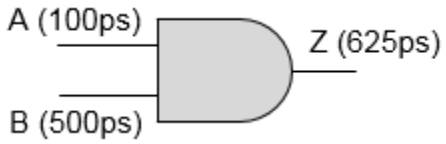
Static Timing Analysis (STA) software by default uses a pessimistic algorithm for timing, which is based on the worst slew propagation (slew merging), referred to as graph-based analysis (GBA). In GBA mode, the software considers both the worst arrival and the worst slew in a path during timing analysis, even if the worst slew is corresponding to a input pin different than the relevant pin for the current path. This gives a pessimistic result.

Path-Based Analysis (PBA) involves re-timing the components of a timing path based on the actual slew that is propagated in this path. This technique primarily aims to remove the pessimism that is introduced due to slew merging at various nodes in the design when graph-based analysis is run (represented in timing graph as nodes).

4.5.2 GBA vs PBA Comparison

When path-based analysis is run, the slew merging is not done and the actual slew on the timing arc of path is propagated. Thus, path-based analysis re-calculates timing in a timing path without considering side path slews that might otherwise affect the arrival times and slew values used in the calculation.

To understand slew merging in GBA, consider an example of a AND gate below.



Here, it is assumed that for any input slew, the output slew is 25% more than the input slew. So for a slew of 100ps at input A, corresponding output slew at Z is 125ps. Similarly, if slew at B is 500ps, then slew at Z is 625ps. In GBA for calculating delay and slew propagation through this AND gate, the worse input slew (through B) is always considered, irrespective of the fact that the path is through pin A or B. But in PBA, the actual slew through the concerned pin is accounted for in delay calculation and slew propagation.

4.5.3 Reporting in Path-Based Analysis

Commands for PBA Reporting

You can use the following commands to report violations in PBA mode:

`report_constraint -retime <option>`

`report_timing -retime <option>`

Note: To generate the analysis summary file, you can use the following options with the report_timing command - to give high value of nworst and max_path to get good coverage. The report summary will be stored in the pba.summ file.

```
report_timing -retime path_slew_propagation -max_paths <> -nworst <> -path_type
summary_slack_only -<late/early> -analysis_summary_file pba.summ
```

Sample Report Showing GBA vs PBA Timing Results

The following example demonstrates GBA vs PBA timing results. Given below is the GBA report for pin “A”:

```
Path 1: MET (1.399 ns) Setup Check with Pin f0/CK->D
View: setup_analysis_view
Group: master
Startpoint: (F) f0/CK
Clock: (F) master
Endpoint: (F) f0/D
Clock: (F) master
N-Sigma: 3.000

                  Capture                       Launch
Clock Edge:+      3.000                         1.000
Src Latency:+     0.000  (0.000, 0.000)         0.000  (0.000, 0.000)
Net Latency:+     0.067  (0.072, 0.002) (P)     0.098  (0.093, 0.002) (P)
Arrival:=         3.067  (3.072, 0.002)         1.098  (1.093, 0.002)


Setup:-            0.237  (0.237, 0.000)
Cppr Adjust:+      0.028  (0.021, 0.002)
Required Time:=    2.852  (2.856, 0.002)
Launch Clock:=     1.098  (1.093, 0.002)
Data Path:+        0.365  (0.345, 0.006)
Slack:=            1.399  (1.418, 0.006)

#-------------------------------------------------------------------------
#  Load   Trans   Pin   Delay   Arrival Cell                Arc     Required
#  (pf)   (ns)           (ns)    (ns)                                Time
#                                                                    (ns)
#-------------------------------------------------------------------------
  0.005   0.010  f0/CK  -       1.098   (arrival)           CK      2.511
  0.005   0.065  f0/Q   0.280   1.373   DFFX2               CK->Q   2.772
  0.003   0.009  p1/Y   0.088   1.458   BUFH_X10M_A9TL_C20  A->Y    2.856
  0.003   0.009  f0/D   0.000   1.458   DFFX2               -       2.856
#-------------------------------------------------------------------------
```

The slew through “A” pin is 0.093ns and B pin is 1.068ns (not shown in the above report). During GBA, the software performs slew merging and uses 1.068ns as the slew through “A” pin. Hence, a lot of pessimism is added in this analysis, which can be removed if PBA is used.

To enable PBA mode, you can use the following command:

```
report_timing -retime path_slew_propagation
```

Given below is the result of PBA analysis of the above path. The highlighted red numbers show how PBA numbers remove pessimism induced due to slew merging in GBA.

Note: To report re-timed numbers, you can add retime_slew, retime_delay, and retime_incr_delay options to the `timing_report_fields` attribute or use the `report_timing -fields` parameter.

```
Path 1: MET (1.400 ns) Setup Check with Pin f0/CK->D
Retime Analysis:  Data Path-Slew SOCV WP EWM
View: setup_analysis_view
Group: master
Startpoint: (F) f0/CK
Clock: (F) master
Endpoint: (F) f0/D
Clock: (F) master
N-Sigma: 3.000

                   Capture                      Launch
Clock Edge:+       3.000                        1.000
Src Latency:+      0.000  (0.000, 0.000)        0.000  (0.000, 0.000)
Net Latency:+      0.067  (0.072, 0.002) (P)    0.098  (0.093, 0.002) (P)
Arrival:=          3.067  (3.072, 0.002)        1.098  (1.093, 0.002)

Setup:-            0.237  (0.237, 0.000)
Cppr Adjust:+      0.028  (0.021, 0.002)
Required Time:=    2.852  (2.856, 0.002)
Launch Clock:=     1.098  (1.093, 0.002)
Data Path:+        0.363  (0.344, 0.006)
Slack:=            1.400  (1.420, 0.006)
Slack(original):=  1.399  (1.418, 0.006)

#-------------------------------------------------------------------------
#  Load   Trans   Pin   Delay   Arrival Cell                Arc     Required
#  (pf)   (ns)          (ns)    (ns)                                Time
#                                                                   (ns)
#-------------------------------------------------------------------------
  0.005   0.010   f0/CK  -      1.098  (arrival)            CK      2.513
  0.005   0.065   f0/Q   0.280  1.373  DFFX2                CK->Q   2.773
  0.003   0.009   p1/Y   0.088  1.457  BUFH_X10M_A9TL_C20   A->Y    2.858
  0.003   0.009   f0/D   0.000  1.457  DFFX2                -       2.858
#-------------------------------------------------------------------------
```

Note: The `timing_disable_retime_clock_path_slew_propagation` attribute controls whether retiming is done for both data as well as clock paths.

4.5.4 Path-Based Analysis Reporting Models

There are two types of path-based analysis reporting models:

- Regular
- Exhaustive

Regular

In the regular model of PBA reporting, you can use the report_timing command, as shown below:

```
report_timing -retime <retime type> -max_paths 'N' -nworst 'M' -max_slack 'S'
```

Tempus performs the following steps:

1. Collect top 'N' critical paths with GBA slack less than 'S' with maximum of 'M' paths per endpoint picked.
2. Perform PBA on the paths from the above step.
3. Report paths with PBA_Slack < 'S', sorted based on GBA slacks.

Exhaustive

Due to the impact of path-based analysis, slack ordering of paths can be changed due to path re-calculation. The most critical path before re-calculation may not be the most critical path after re-calculation. This is the reason why exhaustive path-based analysis is performed, which aims at reporting the true worst path after PBA is performed by examining all the violating paths in the design.

You can perform exhaustive reporting by using the report_timing `-retime_mode exhaustive` parameter.

To ensure optimal path search runtime, you can use the following setting to limit the worst paths:

```
set_db timing_pba_exhaustive_path_nworst_limit 'L'  (default: 10K)
```

Tempus performs the steps given below, when you use the following command:

```
report_timing -retime <retime type> -retime_mode exhaustive -max_paths 'N' -nworst
'M' -max_slack 0
```

1. Collect top 'N' GBA violating endpoints (max slack less than zero).
2. Perform PBA on top 'L' nworst paths for each endpoint in the above step. Whenever the Lth path on an endpoint is a PBA candidate (GBA violating), Tempus will issue a warning indicating the lack of full coverage on that end point.
3. Report top 'N' paths from step 2 above with PBA_Slack less than 0 - sorted by PBA slack with maximum of 'M' paths per endpoint are picked.

4.5.5 Path-Based Analysis Essence

The following sections describe the impact of PBA when used with SI and AOCV.

AOCV with PBA

The stage count in AOCV (advanced on-chip variation) is calculated differently in GBA and PBA, and hence AOCV derates differ in GBA and PBA mode. Further, the `timing_aocv_analysis_mode` attribute control in GBA and PBA differs based on the selected mode. The two methods of AOCV analysis with PBA are:

`report_timing -retime aocv`

and

```
report_timing -retime aocv_path_slew_propagation
```

For more information on AOCV, refer to the Timing Analysis Modes chapter.

Signal Integrity with PBA

PBA has significant impact when performing noise analysis. When PBA is enabled, the actual timing window of the victim is computed. The attacker timing window is always taken from GBA. In this way pessimism in SI can be reduced.

Below is sample report which shows change in incr_delay (shown under retime_incr_delay column) in PBA. For more information, refer to the Signal Integrity Delay and Glitch Analysis.

4.6 Analysis Of Simultaneous Switching Inputs - SSI

4.5.1 SSI Overview on page 201

4.5.2 Implementation on page 202

4.5.3 Reporting in SSI Mode on page 204

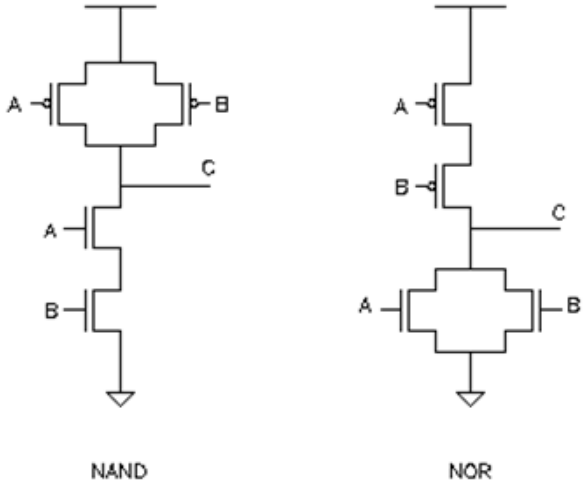
4.5.4 Considerations for Path-Based Analysis (PBA) on page 205

4.6.1 SSI Overview

Multiple input cells that have an underlying transistor topology with parallel P or N device pulling the output up or down respectively, can exhibit accelerated delays when multiple inputs are switching in the same direction. If a secondary input switches within a specified delay window with respect to the primary transition, the delay from the initial transition will be smaller.

The example below shows a NAND gate implementation with parallel PMOS pull-up transistors, and a NOR gate with parallel NMOS pull-down devices:

Figure 4-77 NAND gate implementation



Simultaneous falling transitions at the NAND gate's A and B inputs will accelerate the rising output transition. Conversely, rising transitions of the NOR gate's A and B inputs within a specified proximity window will accelerate the output falling delay.

Current characterization approaches typically consider a single-input switching. In real-world scenarios where multiple inputs of a cell can be switching at the same time, the delay of a particular arc can be substantially less.

Faster delays on early paths increase pessimism. Guard-banding approaches can be used to derate the early paths – but doing this globally can lead to overall increased pessimism over the whole design.

With SSI enabled, simultaneous switching effects can be detected by the software and specific SSI derating factors provided by the user can be applied in those cases. When these timing-guided derating assertions are applied, the overall guard-band derating can be reduced – resulting in an overall less pessimistic analysis.

4.6.2 Implementation

The implementation of the simultaneous switching input (SSI) derating functionality involves additional syntax provided to the set_timing_derate constraint, library properties for defining the switching sensitivity delay window, and a set of heuristics of determining which input phase arrival times should be considered as interacting, and therefore, subject to possible SSI derating.

SSI Derating Command Syntax

The SSI derating is controlled by the `set_timing_derate -input_switching` parameter. You can use this parameter to configure SSI derating on specific library cells when they are analyzed on early data paths, typically, the data paths for Hold analysis. For example,

```
set_timing_derate 0.5 -fall -data -early -input_switching [get_db lib_cells NOR2X]
```

By using additional `set_timing_derate` options, you can configure the derate behavior for different delay corners or operating voltage domains:

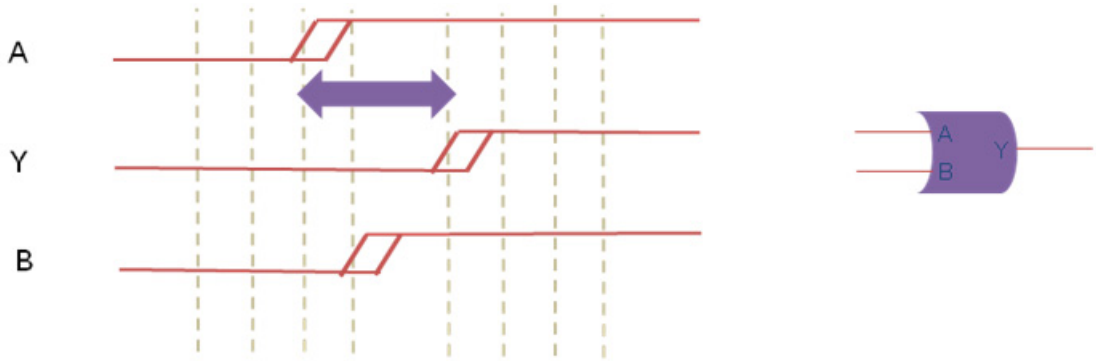
```
set_timing_derate 0.5 -fall -data -early
-delay_corner tt_800 -power_domain pdDefault
-input_switching [get_db lib_cells NOR2*]
```

Controlling the SSI Proximity Window

In order for a secondary signal transition to affect the delay from the initial input switching, it must occur within a certain proximity of the first input's transition. By default, the proximity window is equivalent to the early input-to-output delay from the first transitioning input pin.

In the diagram below, pin A is the first input to switch. For a transition at B to affect the delay from A to Y, it must transition after A has transitioned, but before Y has completed its transition to the new state.

Figure 4-78 SSI Proximity Window



The switching proximity window can be configured by utilizing library-level user-defined properties.

The `k_input_switching_window_rise` and `k_input_switching_window_fall` properties need to be first defined using the following syntax:

```
define_property -type float -object_type lib k_input_switching_window_rise
define_property -type float -object_type lib k_input_switching_window_fall
```

You can then set the proximity factor on a per library basis:

```
set_property [get_libs slow] k_input_switching_window_rise 1.1
set_property [get_libs slow] k_input_switching_window_fall 1.1
```

The property definitions and settings are preserved persistently in the SDC file if the design is saved.

Heuristics for Selecting Interacting Clock Phases

Based on the clocking relationships of the signals arriving on the input pins of a cell configured for SSI, the software will determine if the SSI derate should always be applied, never be applied, or applied based on the evaluation of the proximity window. Conservative analysis requires applying the SSI derate, unless it can be shown that the inputs are not interacting. The following table summarizes the situations where SSI should be applied or not applied:

Condition	SSI Behavior	Notes
Data are derived from synchronous clock phases of the same frequency and phase	Evaluate SSI switching window	

Data are derived from synchronous clock phases of different frequency or different phase	Apply SSI derate always	
Data are derived from physically-exclusive clocks	Do not evaluate SSI	
Data are derived from logically-exclusive clocks	Evaluate switching window if clocks are same period, phase, and are synchronous	
Data are derived from asynchronous clocks	Apply SSI derate always	Applied if an asynchronous clock is present on any input
Data arrival on same pin from multiple clocks that are synchronous, have the same period, and phase	Evaluate switching window based on worst-casing of arrival times	
Data arrival from two physically exclusive clocks	Checks for each clock are done separately between the inputs to see if a non-exclusive relationship exists between clocks of one input pin versus clocks of the other	

4.6.3 Reporting in SSI Mode

All the deratings applied via the `set_timing_derate` command are reflected in the “User Derate” column of the detailed `report_timing` report output.

In the example below, a nominal derate on early data paths is set as 0.90. An additional simultaneous switching derate of 0.50 is also specified. The User Derate column will show a cumulative derate of 0.45 if the simultaneous input switching condition has been met.

```
set_timing_derate 0.90 -early -data -fall
set_timing_derate 0.50 -early -data -fall  \
    -input_switching [get_db_lib_cells NOR2X1]
```

```
-------------------------------------------------------------------------
 Timing Point    Edge   Cell    Load    Slew      User     Delay     Arrival
                                                Derate              Time
-------------------------------------------------------------------------
in_test          ^                     0.004                       4.000
in_test                 (net)   0.023
U_W/A ->         ^      NOR2X1         0.004     1.000    0.000     4.000
U_W/Y            v      NOR2X1         0.066     0.450    0.450     4.450
out_win_2               (net)   0.006
U1/A             v      BUFX1          0.066     1.000    0.000     4.450
U1/Y             v      BUFX1          0.090     1.000    0.157     4.607
out_win_1               (net)   0.006
U2/A             v      BUFX1          0.090     1.000    0.000     4.607
U2/Y             v      BUFX1          0.076     1.000    0.154     4.761
out_win                 (net)   0.004
out_win          v                     0.076     1.000    0.000     4.761
-------------------------------------------------------------------------
```

In the timing report shown above, the delay from U_W/A to U_W/Y is subject to SSI derating; it received the full, cumulative derate of 0.45.

4.6.4 Considerations for Path-Based Analysis (PBA)

In path-based analysis (PBA), a timing path that is found by graph-based analysis (GBA), is re-timed for increased accuracy. Re-timing can remove pessimism due to slew merging effects, AOCV stage counts, and so on. During re-timing, only the specific pin-to-pin path of interest gets a new, focused analysis. If the timing path to one pin of an SSI sensitive gate is retimed, its arrival time (for an early data path) may come later than that in GBA analysis. The arrival times of other inputs, which are still based on GBA timing can result in SSI derates being applied in PBA mode where they are not in GBA, or vice-versa. To preserve GBA bounding of PBA results, and to ensure conservative analysis, SSI derating will be applied to a given instance based on the GBA handling. If SSI derates are applied in GBA mode, then they will also be applied in PBA.

4.7 Waveform Aware and Rail Swing Checks

4.6.1 Overview on page 207

4.6.2 Rail Swing Checks - An Overview on page 207

4.6.3 Rail Swing Reporting on page 208

4.6.4 Waveform Aware Pulse Width Checks - An Overview on page 208

4.6.5 Waveform Aware Pulse Width Checks Reporting on page 209

4.7.1 Overview

In high-frequency designs, clock pulses may get distorted or die down during propagation through the clock tree network. This may be due to slow slews or long waveform tails causing rise/fall waveforms to not reach the VDD/VSS levels before the opposing transition arrives.

The minimum pulse width checks may not capture such failures, as these checks work based on the arrivals at delay threshold, so the waveform shapes are not considered.

The Tempus software has a capability to perform the following checks to identify and report such potential clock distortions:

- Rail Swing Checks
- Waveform aware Pulse Width Checks

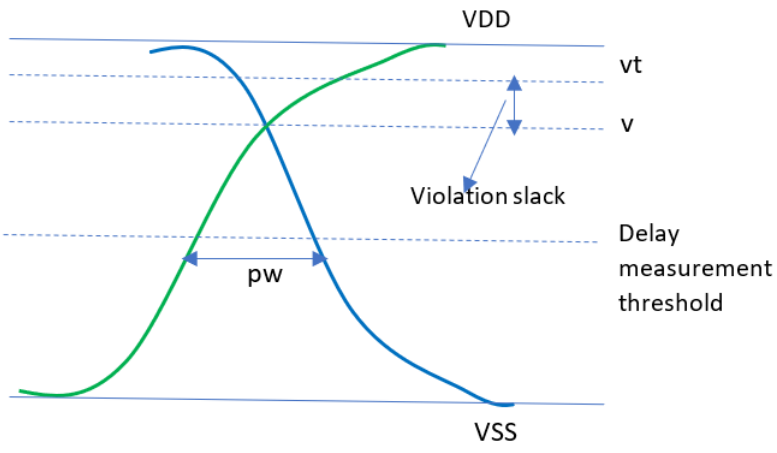
These are described below.

4.7.2 Rail Swing Checks - An Overview

The Tempus software provides detailed transition waveforms at each pin, which can be used for performing rail swing checks. This data allows the software to report violations on all the clock network pins, where the transition does not reach the ~VDD/VSS levels.

The pulse at any pin is formed by aligning the actual rise and fall transition waveforms over the pulse width (pw), that is computed using regular minimum pulse width checks. The intersection of these rise and fall transition waveforms defines the rail voltage achieved (v). The achieved rail voltage (v) is then checked against the user-defined threshold (vt) to check the violations.

This is illustrated in the diagram below.



Use Model

The following attributes can be used to specify the voltage thresholds as a ratio of the VDD:

- timing_rail_swing_checks_high_voltage_threshold
- timing_rail_swing_checks_low_voltage_threshold

For example,

```
set timing_rail_swing_checks_high_voltage_threshold 0.95
set timing_rail_swing_checks_low_voltage_threshold 0.05
```

4.7.3 Rail Swing Reporting

The rail swing violations can be reported using the following command:

```
report_constraint -check_type rail_swing -all_violators
```

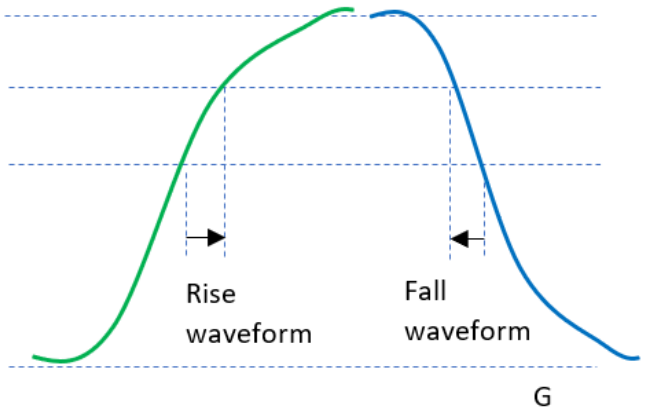
The following example shows a sample output report, where all the values are specified as ratio of [VDD-VSS] supply voltages:

```
-------------------------------------------------------------------
                        RAIL SWING VIOLATIONS
-------------------------------------------------------------------
Pin              Required       Actual         Slack   Clock    View
                 Signal Level   Signal Level                     Name
-------------------------------------------------------------------
inst8/A (low)    0.05           0.072          -0.022   CLK1   view1
f4/CP (low)      0.05           0.065          -0.015   CLK2   view1
inst1/A (low)    0.05           0.056          -0.006   CLK2   view2
ff2/CP (high)    0.95           0.938          -0.012   CLK1   view2
inst1/A (high)   0.95           0.946          -0.004   CLK1   view1
-------------------------------------------------------------------
```

4.7.4 Waveform Aware Pulse Width Checks - An Overview

Traditional pulse width checks only check the pulse widths at delay measurement thresholds and therefore do not provide any information on the waveform shape. To control waveform shape degradation on clock networks, the Tempus software provides waveform aware pulse

width checks to report minimum pulse width violations on user-specified voltage levels, and not just delay measurement thresholds. These checks can be applied on clock network pins.



Use Model

You can use the following attributes to specify voltage levels for waveform aware pulse width checks:

- timing_waveform_aware_pulse_width_checks_high_voltage_level
- timing_waveform_aware_pulse_width_checks_high_voltage_level

For example,

```
set timing_waveform_aware_pulse_width_checks_high_voltage_level 0.75
set timing_waveform_aware_pulse_width_checks_low_voltage_level 0.25
```

You can use the following command to specify the required pulse width at a user-defined voltage level:

set_min_pulse_width -waveform_aware_type {absolute | source_width_ratio}

For example,

```
set_min_pulse_width 0.1 [get_clocks clk1] -waveform_aware_type absolute
```

To reset these settings, you can use the reset_min_pulse_width command.

4.7.5 Waveform Aware Pulse Width Checks Reporting

To report waveform-aware pulse width checks, you can use the following command:

report_min_pulse_width -waveform_aware_type {regular | waveform_aware | both}

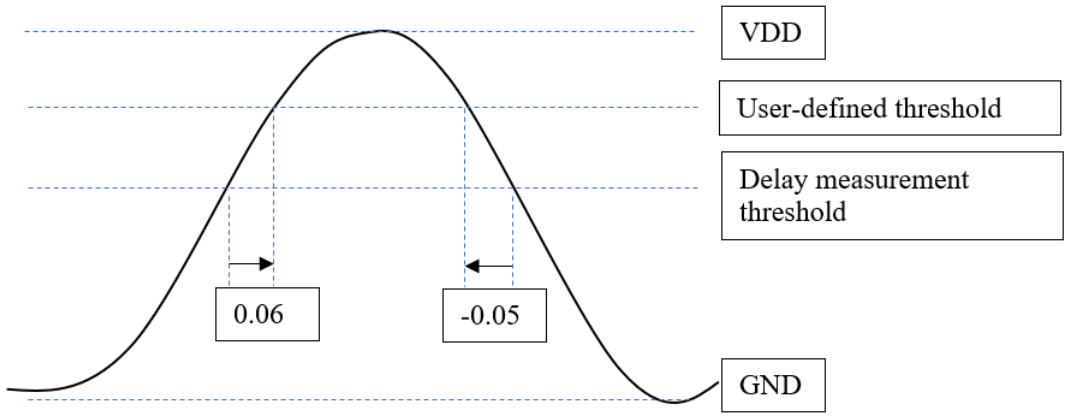
By default, the regular pulse width checks are reported.

The following command generates a report as shown below:

```
report_min_pulse_width -waveform_aware_type waveform_aware -pins FF1/CP -verbose

---------------------------------------------------------------------
Path 1: MET PulseWidth Check with Pin FF1/CP
Ending Clock Edge:    FF1/CP (v) checked with trailing edge of 'clk'
Beginning Clock Edge: FF1/CP (^) triggered by  leading edge of 'clk'
Other End Arrival Time        1.485000
+ Other End Arrival Time Adjustment -0.05
- Arrival Time Adjustment   0.06
- Pulse Width               0.047190
+ Phase Shift               0.000000
+ CPPR Adjustment           0.032300
= Required Time             1.420110
- Arrival Time              1.013900
= Slack Time                0.346210
```

The "Arrival Time Adjustment" and "Other End Arrival Time Adjustment" details in the above report denote the offsets measured over the actual launching transition waveforms and capturing transition waveforms respectively, as shown below:



These offsets are the time differences between the delay measurement threshold and user-specified voltage level measured over the actual transition waveforms. The traditional pulse widths are adjusted by the computed offsets to obtain a width at the user-specified voltage threshold.

4.8 Set Instance Library Flow

4.7.1 Iinstance Library Flow Overview on page 212

4.7.2 MMMC set_instance_library Flow on page 212

4.7.3 MMMC Flow Script on page 213

4.7.4 Save and Restore MMMC set_instance_library Flow on page 215

4.7.5 Limitations of set_instance_library on page 216

4.8.1 Iinstance Library Flow Overview

In MSV designs, if the power intent information is not available in the form of PGV, CPF, or UPF, the Tempus software does not have the mechanism to bind the correct PVT library. For this you can use the set_instance_library command to specify the library that you want to bind to an instance.

In case of macros where different PVT libraries are available and none of those libraries match with the PVT, the software will attempt to interpolate. You can use the set_instance_library command to bind it with the closest PVT library.

In MSV designs that have PGV netlist, CPF, or UPF, where power intent information is present, the set_instance_library command can be used on a handful of instances where power intent is not correct and can bind it correctly. This feature is useful during the initial design cycle.

4.8.2 MMMC set_instance_library Flow

Tempus requires timing libraries, netlist as input along with the other essential data required for static timing analysis. In addition to all these, the MMMC flow requires ViewDefinition template file. In this template file, you can define all the analysis views required, corresponding RC corner, constraint mode and delay corner. When you specify the set_instance_library command in the ViewDefinition file, it overrides the default binding mechanism for a cell from the library set.

ViewDefinition Template

To apply the set_instance_library command in the MMMC flow, you need to create a file with all the set_instance_library commands. For this you can use the create_delay_corner -instance_library_file parameter.

For example,

```
create_delay_corner -instance_library_file <file1>
```

```
create_library_set -name complete_library_set \
-timing [list \
/icd/etsproject3/mgarg/aayhis/myur/MSV_DBS/
NW_PGV_paradime_st_ao_buffer_cpf2_6_DSH_MP_CP/savedDB.dat/libs/mmmc/
C28SOI_SC_12_CORI_LL_ss28_0.90V_0.00V_m0.60V_0.60V_m40C.lib \
/icd/etsproject3/mgarg/aayhis/myur/MSV_DBS/
NW_PGV_paradime_st_ao_buffer_cpf2_6_DSH_MP_CP/savedDB.dat/libs/mmmc/
C28SOI_SC_12_CORR_LL_ss28_0.90V_0.00V_m0.60V_0.60V_m40C.lib \
/icd/etsproject3/mgarg/aayhis/myur/MSV_DBS/
NW_PGV_paradime_st_ao_buffer_cpf2_6_DSH_MP_CP/savedDB.dat/libs/mmmc/
C28SOI_SC_12_SHIFTPBP10_LR_ss28_0.90V_0.95V_m40C.lib \
/icd/etsproject3/mgarg/aayhis/myur/MSV_DBS/
```

```
NW_PGV_paradime_st_ao_buffer_cpf2_6_DSH_MP_CP/savedDB.dat/libs/mmmc/
C32_EPM_EPOD_D12SWSG_LR_ss28_0.90V_m40C.lib \
/icd/etsproject3/mgarg/aayhis/myur/MSV_DBS/
NW_PGV_paradime_st_ao_buffer_cpf2_6_DSH_MP_CP/savedDB.dat/libs/mmmc/
C32_SC_12_CORE_LL_ss28_0.90V_0.00V_m0.60V_0.60V_m40C.lib \
/icd/etsproject3/mgarg/aayhis/myur/MSV_DBS/
NW_PGV_paradime_st_ao_buffer_cpf2_6_DSH_MP_CP/savedDB.dat/libs/mmmc/new.lib \
my.lib \
newcell.lib \ ]

create_op_cond -name opcond1 -library_file
/icd/etsproject3/mgarg/aayhis/myur/MSV_DBS/
NW_PGV_paradime_st_ao_buffer_cpf1_4_DSH_MP_CP/savedDB.dat/libs/mmmc/
C28SOI_SC_12_CORI_LL_ss28_0.90V_0.00V_m0.60V_0.60V_m40C.lib -P 1.228 -V 0.9 -T -40

create_rc_corner -name min_corner

create_rc_corner -name max_corner

create_delay_corner -name slow_max\
-pg_net_voltages {VDD2@0.9 VDD1@0.9 vPD2@-0.6 vPD1@-0.6}\
-library_set complete_library_set \
-opcond_library C28SOI_SC_12_CORI_LL \
-opcond opcond1 \
-rc_corner max_corner \
-instance_library_file sil.tcl

create_delay_corner -name fast_min\
-pg_net_voltages {VDD2@0.9 VDD1@0.9 vPD2@-0.6 vPD1@-0.6}\
-library_set complete_library_set \
-opcond_library C28SOI_SC_12_CORI_LL \
-opcond opcond1 \ -rc_corner min_corner

create_constraint_mode -name func_mode\
-sdc_files\
[list /icd/etsproject3/mgarg/aayhis/myur/MSV_DBS/
NW_PGV_paradime_st_ao_buffer_cpf2_6_DSH_MP_CP/pgv_data/mmmc/modes/func_mode/
func_mode.sdc]

create_analysis_view -name setup_func_m40 -constraint_mode func_mode -delay_corner
slow_max -latency_file /icd/etsproject3/mgarg/aayhis/myur/MSV_DBS/
NW_PGV_paradime_st_ao_buffer_cpf2_6_DSH_MP_CP/pgv_data/mmmc/views/setup_func_m40/
latency.sdc

create_analysis_view -name hold_func_m40 -constraint_mode func_mode -delay_corner
fast_min -latency_file /icd/etsproject3/mgarg/aayhis/myur/MSV_DBS/
NW_PGV_paradime_st_ao_buffer_cpf2_6_DSH_MP_CP/pgv_data/mmmc/views/hold_func_m40/
latency.sdc

set_analysis_view -setup [list setup_func_m40 ] -hold [list hold_func_m40]
```

4.8.3 MMMC Flow Script

The MMMC flow sample script is shown below:

```
set_multi_cpu_usage -localCpu 16

set timing_library_read_without_power false

read_view_definition setup_func_m40_viewDefinition.tcl

read_verilog pg-explicit.v

set_top_module top
```

```
report_instance_library -inst { iP/dff1 iP/and1} > ril_test.rpt
```

The library that is used for binding instances with the `set_instance_library` command must be an active library loaded in the session via active library set defined with the `create_library_set` command.

The following command is used in the sil.tcl:

```
set_instance_library -library newcell.lib -instance {iP/and1 iP/dff1}
```

You can use the following command to verify whether the instance of the cell is bound with the library specified with the `set_instance_library` command:

```
report_instance_library -inst {iP/and1 iP/dff1} > ril_test.rpt
```

The output of the `report_instance_library` command is given below:

```
Instance           : iP/dff1
Cell               : C12T32_LL_SDFPQX8
Analysis View      : setup_func_m40
Power Domain       : Default
Library/Libset     : new3cell_C32_SC_12_CORE_LL/complete_library_set
Library File       : /it/sjclapa06p5_scratch01/aayhis/expire_2021_03_30/
scratch_dec/SIL_DOC/sil_view/doc_case/newcell.lib
User Setting       : True
Op Cond : P->1.228000, V->0.900000, T->-40.000000 (new3cell_C32_SC_12_CORE_LL/
%NOM_PVT)
Cell Type          : gateCell

Instance           : iP/and1
Cell               : C12T32_LL_AND2X8
Analysis View      : setup_func_m40
Power Domain       : Default
Library/Libset     : new3cell_C32_SC_12_CORE_LL/complete_library_set
Library File       : /it/sjclapa06p5_scratch01/aayhis/expire_2021_03_30/
scratch_dec/SIL_DOC/sil_view/doc_case/newcell.lib
User Setting       : True
Op Cond            : P->1.228000, V->0.900000, T->-40.000000
(new3cell_C32_SC_12_CORE_LL/%NOM_PVT)
Cell Type          : gateCell
```

The above output clearly shows the following:

1. The library binding was applied for the instance `iP/and1` and `iP/dff1` with the help of the `set_instance_library` command and the library file reported in the `report_instance_library` output is the same as that applied.
2. `User setting: True` indicates that the `set_instance_library` command has been applied.

4.8.4 Save and Restore MMMC set_instance_library Flow

In the save and restore flow, the `set_instance_library` command used with the `create_delay_corner` command in the viewDefinition file will be saved in the save session directory.

Save Session Script

The save session sample script is shown below:

```
set_multi_cpu_usage -localCpu 16
set timing_library_read_without_power false
read_view_definition setup_func_m40_viewDefinition.tcl
read_verilog pg-explicit.v
set_top_module top
update_timing -full
save_design new_test.db
```

The `set_instance_library` command used in the above script is saved in the following format:

```
set_instance_library \
    -library \
    " ${::IMEX::silLibVar}/mmmc/newcell.lib " \
    -instance \
    " iP/and1 \
    iP/dff1 "
```

Restore Session Script

The restore session sample script is shown below:

```
source new_test.db
report_instance_library -inst { iP/and1 iP/dff1} > ril_restore1_test.rpt
```

Once the design has been restored from the saved session, the library binding that was applied for the instance, using the `set_instance_library` command, in the saved session script will be the same as in the restored session.

The output of the `report_instance_library` command after the restore session is shown below:

```
Instance             : iP/and1
Cell                 : C12T32_LL_AND2X8
Analysis View        : setup_func_m40
Power Domain         : Default
Library/Libset       : new3cell_C32_SC_12_CORE_LL/complete_library_set
```

```
Library File          : /it/sjclapa06p5_scratch01/aayhis/expire_2021_03_30/
scratch_dec/SIL_DOC/sil_view/doc_case/newcell.lib
User Setting          : True
Op Cond               : P->1.228000, V->0.900000, T->-40.000000
(new3cell_C32_SC_12_CORE_LL/%NOM_PVT)
Cell Type             : gateCell

Instance              : iP/dff1
Cell                  : C12T32_LL_SDFPQX8
Analysis View         : setup_func_m40
Power Domain          : Default
Library/Libset        : new3cell_C32_SC_12_CORE_LL/complete_library_set
Library File          : /it/sjclapa06p5_scratch01/aayhis/expire_2021_03_30/
scratch_dec/SIL_DOC/sil_view/doc_case/newcell.lib
User Setting          : True
Op Cond               : P->1.228000, V->0.900000, T->-40.000000
(new3cell_C32_SC_12_CORE_LL/%NOM_PVT)
Cell Type             : gateCell
```

4.8.5 Limitations of set_instance_library

The voltage scaling is not supported in the `set_instance_library` command flow.

For example, if there are 4 libraries with different voltages in a design and you bind the instance that is present in all the 4 libraries, then the `set_instance_library` command binds with the first library only.

```
set_instance_library -library {L1 L2 L3 L4} -instance {i1}
```

In this example, the `set_instance_library` command will bind the instance `i1` with the first library that is `L1`.

4.9 Inter-Power Domain (IPD) Analysis

4.8.1 IPD Analysis Overview on page 218

4.8.2 IPD Analysis Flow on page 219

4.8.3 Scope-Based IPD Analysis on page 220

4.8.4 Scope-Based IPD Analysis Flow on page 221

4.8.5 Enabling IPD Analysis on page 222

4.8.6 IPD Analysis Reporting on page 225

4.8.7 Inter-Power Domain (IPD) Logic Checks on page 226

4.8.8 IPD Paths Classification on page 228

4.8.9 Supported IPD Logic Checks on page 231

4.8.10 Performing IPD Logic Checks on page 234

4.9.1 IPD Analysis Overview

The increase in the number of power domains in a design has become a challenging aspect for designers. This has also led to a significant increase in the number of timing signoff corners due to cross combinations of voltage corners that are needed to completely signoff Inter-Power Domain (IPD) paths. For example, a design having two power domains PD1 and PD2, each operating at 0.9v, 1.0v, 1.1v, may require 9 runs to signoff the IPD paths. This may led to long cycle times and large compute requirements for timing signoff. Also, a larger part of netlist may not belong to IPD paths being analyzed, which results in redundant analysis of large parts of netlist that go through analysis in these IPD analysis runs.

Another important aspect of IPD analysis is that some of the timing constraints, like clock frequency, can be different between the IPD analysis corners. Multiple frequencies per domain require additional constraint coding, and increases the number of STA runs and overall runtime/compute resource requirement.

To reduce the cycle time and computations required for IPD analysis, the Tempus software provides the capability to run IPD analysis, where only IPD logic in the design can be analyzed and timing reports only generate data relevant to that logic.

The Tempus IPD Analysis feature analyzes the IPD logic of the design efficiently. The reduced capacity requirement per IPD run, also enables designers to employ CMMMC capability to analyze various IPD voltage combinations with differing frequencies (or any timing constraint).

Terms

- IPD: Inter-Power Domain
- PD: Power Domain
- MSV: Multi-Supply Voltage
- IPD Logic: Design logic having PD transitions in fanin/fanout cone identified for IPD analysis
- Non-IPD Logic: Logic that is not considered IPD Logic

Salient Features

The following are the main features of IPD analysis:

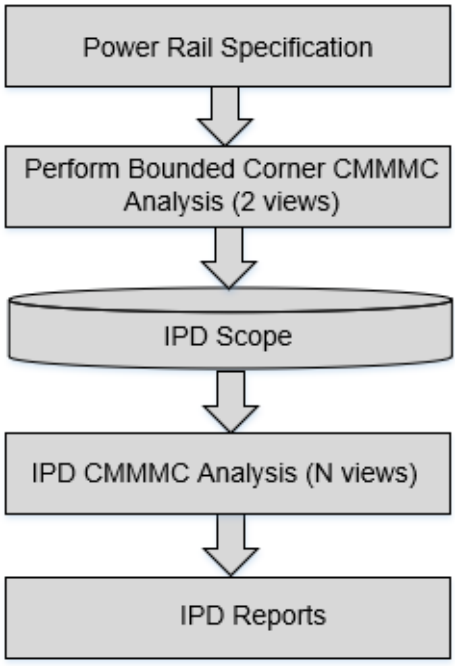
- Focused analysis of IPD logic, yields runtime/memory efficiency
- Enabled through IPD scope creation containing only IPD logic
- Simplified UI to capture design operating voltage specification

- Minimum/maximum bounded corner-based IPD scope creation, followed by CMMMC IPD scope-based analysis
- Automated exhaustive voltage combinations identification
- Flexible IPD/DVFS constraint generation/specification
- Automated CMMMC IPD setup creation and analysis
- Supports CPF/IEEE-1801/PG-Verilog interface for power intent specification
- Flexible IPD-only path reporting
- Tempus-ECO flow support

4.9.2 IPD Analysis Flow

The IPD analysis flow is given below:

Figure 4-79 IPD Analysis Flow



For details on IPD path classification, refer to section IPD Paths Classification on page 228.

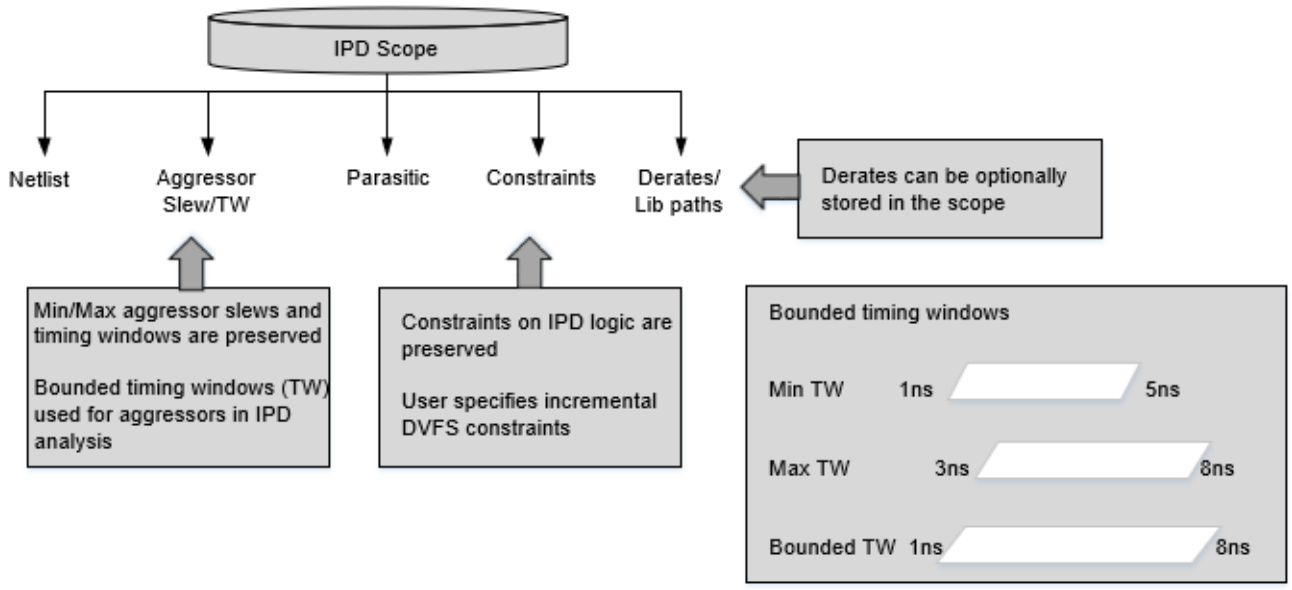
4.9.3 Scope-Based IPD Analysis

The IPD analysis checks only the IPD logic in the design, which may be less than 10% of the total design size. This reduced design is written out in the form of an IPD scope, during the scope creation step of the flow. The reduced scope can then be loaded and analyzed for exhaustive combination of voltage covering all the voltage signoff corners.

To address the pessimism with use of infinite timing windows (TW) and default slews for the aggressors, a bounded corner approach is followed in the scope creation run. The Tempus software automatically identifies the bounding minimum/maximum (the highest and lowest voltage corners), for slew and timing window information.

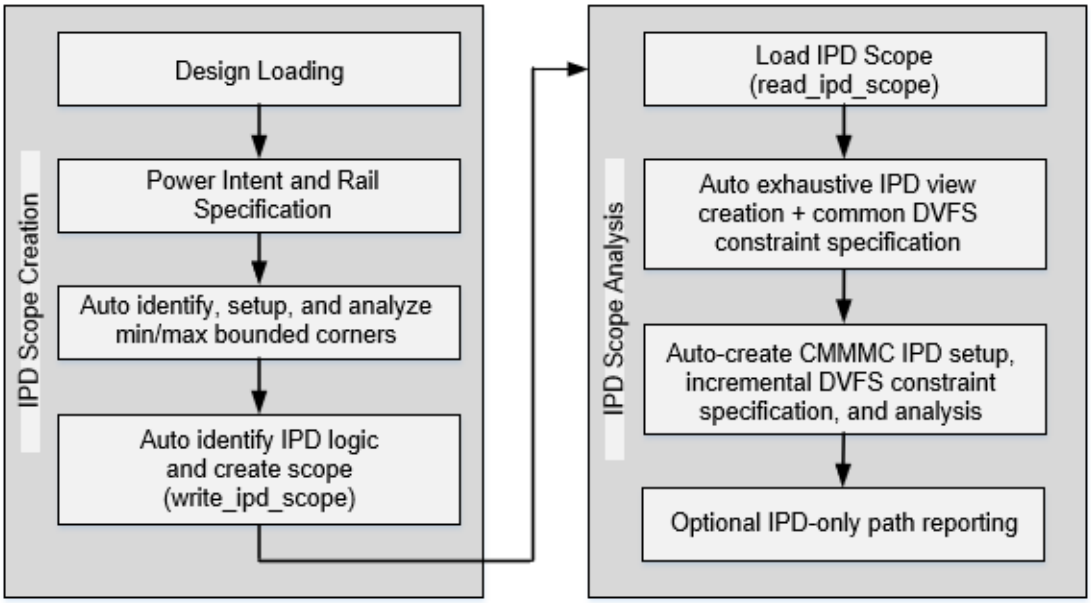
Apart from the reduced netlist, the constraints, parasitics, slew, and timing window information is stored in the scope. Optionally, the software can also store derate related information in the scope.

Figure 4-80 Scope Based IPD Analysis



4.9.4 Scope-Based IPD Analysis Flow

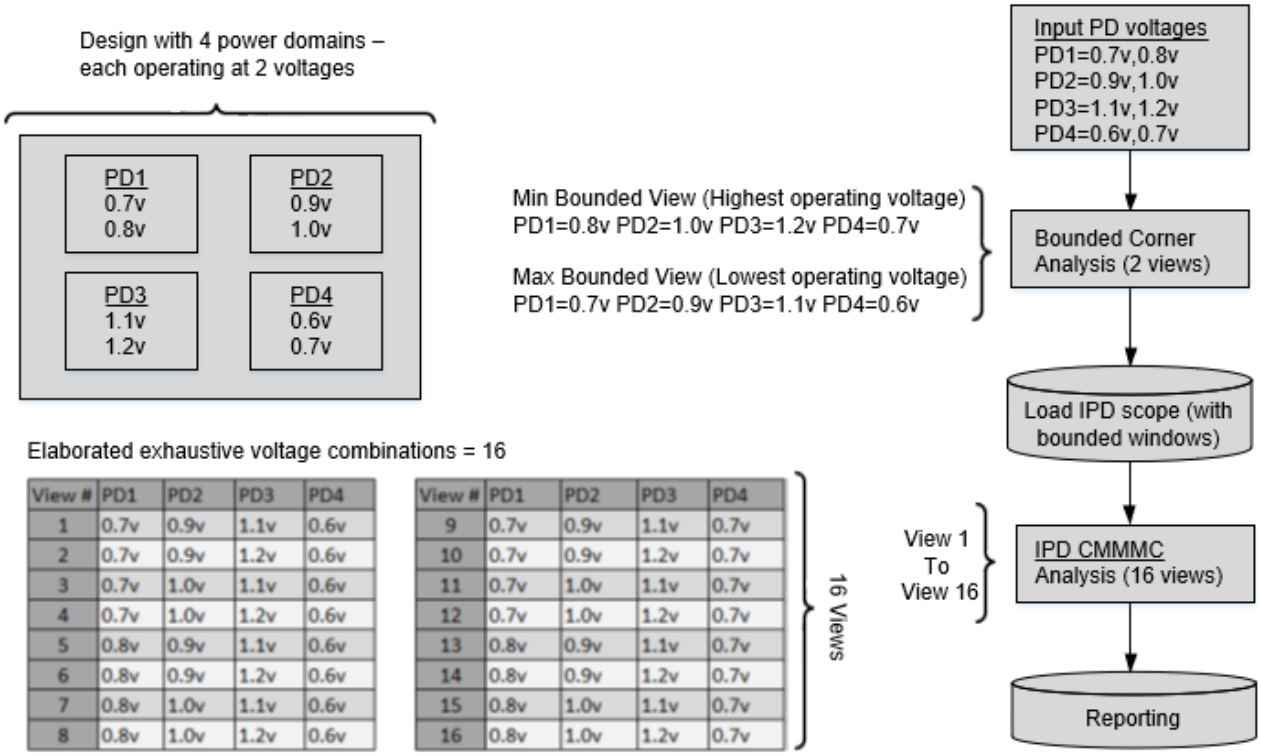
Figure 4-81 Scope Based IPD Analysis Flow



Consider the following illustration of scope-based IPD Analysis:

The design consists of 4 power domains each operating at 2 voltages. The Runs 1 and 16 form the traditional signoff corners, while Run 2-15 cover the IPD corners. To signoff, the exhaustive combination of Run1-16 needs to be analyzed. The scope creation run which involves full design analysis generates the minimum (Run 1) and maximum (Run16) bounded corner IPD scope, which contains IPD logic marking, partial Verilog, partial SPEF, partial constraints, slew, waveform, arrival windows, AOCV/TW-CPPR data, and constants on side pins. This data is read during the scope analysis run and is then analyzed for the exhaustive combination of Run 1-16 for a complete signoff.

Figure 4-82 Scope Based IPD Analysis - Illustration



View #	PD1	PD2	PD3	PD4
1	0.7v	0.9v	1.1v	0.6v
2	0.7v	0.9v	1.2v	0.6v
3	0.7v	1.0v	1.1v	0.6v
4	0.7v	1.0v	1.2v	0.6v
5	0.8v	0.9v	1.1v	0.6v
6	0.8v	0.9v	1.2v	0.6v
7	0.8v	1.0v	1.1v	0.6v
8	0.8v	1.0v	1.2v	0.6v

View #	PD1	PD2	PD3	PD4
9	0.7v	0.9v	1.1v	0.7v
10	0.7v	0.9v	1.2v	0.7v
11	0.7v	1.0v	1.1v	0.7v
12	0.7v	1.0v	1.2v	0.7v
13	0.8v	0.9v	1.1v	0.7v
14	0.8v	0.9v	1.2v	0.7v
15	0.8v	1.0v	1.1v	0.7v
16	0.8v	1.0v	1.2v	0.7v

4.9.5 Enabling IPD Analysis

To enable IPD Analysis, the following setting is required at the top of the Tempus script:

```
set_db timing_analysis_enable_inter_power_domain true
```

The following commands are required during several steps of the flow, as explained below:

- `set_ipd_rail_voltage`

  This command is used to specify the power rail specifications and excludes combinations that do not require analysis. When power intent is specified through CPF/IEEE-1801, the `set_ipd_rail_voltage` power rail specification refers to the primary power net of the power domain. The following command parameters can be used:

  - `-sweep`: Specifies the list of power rails and their corresponding voltage values.
  - `-exclude_voltage_signatures`: Specifies the list of voltage combinations to be excluded from IPD Analysis.

  In Figure 4-82 on page 222, if Run2 voltage combination is to be excluded from analysis, the following example shows the usage of `set_ipd_rail_voltage` command:

```
set_ipd_rail_voltage -sweep { PD1 {0.7 0.8} PD2 {0.9 1.0} PD3 {1.1 1,2}
PD4 {0.6 0.7} }
set_ipd_rail_voltage -exclude_voltage_signatures { {PD1 0.7 PD2 0.9 PD3
1.2 PD4 0.6} }
```

- create_pg_net_group

  This command is used to group a set of PG nets. This helps in simplification of the overall UI of the IPD flow by specifying sweeps for a group rather than a PG net. Also, the crossings between the nets of the same PG net group are automatically ignored.

  In the example shared below, the PG nets pgn1, pgn2, and pgn3 are part of PG net group PG1 and the sweep voltage specified for PG1 is applicable for the PG nets. The crossings between pgn1 and pgn2, pgn2 and pgn3, and pgn1 and pgn3 will automatically be ignored.

```
create_pg_net_group -name PG1 -pg_nets {pgn1 pgn2 pgn3}
        \set_ipd_rail_voltage -sweep { PG1 {0.7 0.8} }
```

- create_ipd_configuration

  This command is used for creating the IPD configuration or the analysis views in both the bounded scope generation and the scope analysis session. It auto identifies the corners based on the user-specified sweep voltages and loads up the view definition for CMMMC-based analysis.

  - `-bounded_corners` parameter is used in the scope creation run to create the minimum/maximum bounding voltage corners to be used for analysis.

```
set_ipd_rail_voltage -sweep {PD1 {0.7 0.8} PD2 {0.9 1.0} PD3 {1.1 1.2}
PD4 {0.6 0.7}}
create_ipd_configuration -bounded_corners
```

  Min bounding view: PD1=0.8v PD2=1.0v PD3=1.2v PD4=0.7v

  Max bounding view: PD1=0.7v PD2=0.9v PD3=1.1v PD4=0.6v

  - `-load_constraints` and `-load_derates` parameters are optionally used in the scope analysis session to load the constraints and derates stored in the IPD scope:

```
set_ipd_rail_voltage -sweep { PD1 {0.7 0.8} PD2 {0.9 1.0} PD3 {1.1 1.2}
PD4 {0.6 0.7} }
set_ipd_rail_voltage -exclude_voltage_signatures { {PD1 0.7 PD2 1.0 PD3
1.2 PD4 0.7} }
create_ipd_configuration -load_derates -load_constraints
```

  - Exhaustive number of voltage combinations = 2^4 = 16.
  - Analyze 15 IPD views after excluding the voltage combination specified by the user.

- report_ipd_configuration

This command reports the rail sweep information and software created voltage combinations, that is, the IPD views.

It reports and indicates the valid voltage combinations along the with excluded voltage combinations.

- IPD views are named automatically as `ipd_view<count>`.
- "Excluded" column reports "yes" against excluded voltage combinations.

A sample report output is given below:

```
POWER RAIL/GROUP MAPPING
---------------------------
Mapping     Power rail/group
---------------------------
PG1         VDD
PG2         VDDR
PG3         VDDF
VOLTAGE SWEEP
------------------------
Power       Sweep
Rail
------------------------
PG1       { 0.9 }
PG2       { 1.1 1.2 }
PG3       { 0.8 0.9 }
VOLTAGE SIGNATURES
-----------------------------------------
PG1   PG2   PG3       View      Excluded
-----------------------------------------
0.9   1.1    0.8    ipd_view1      no
0.9   1.2    0.8    ipd_view2      no
0.9   1.1    0.9    ipd_view3      no
0.9   1.2    0.9    ipd_view4      yes
```

- write ipd scope

  This command is used to write out the IPD scope in the scope creation run. The scope is generated in the directory location specified as command argument. The command parameters allow you to control whether the constraints and derates should be written out within the scope or not. For example,

  ```
  write_ipd_scope -include_derates -include_constraints -bounded_corners
  -overwrite Bounded_corner_scope
  ```

- read ipd scope

  This command is used to load scope data in the scope analysis run. For example,

  ```
  read_ipd_scope Bounded_corner_scope
  ```

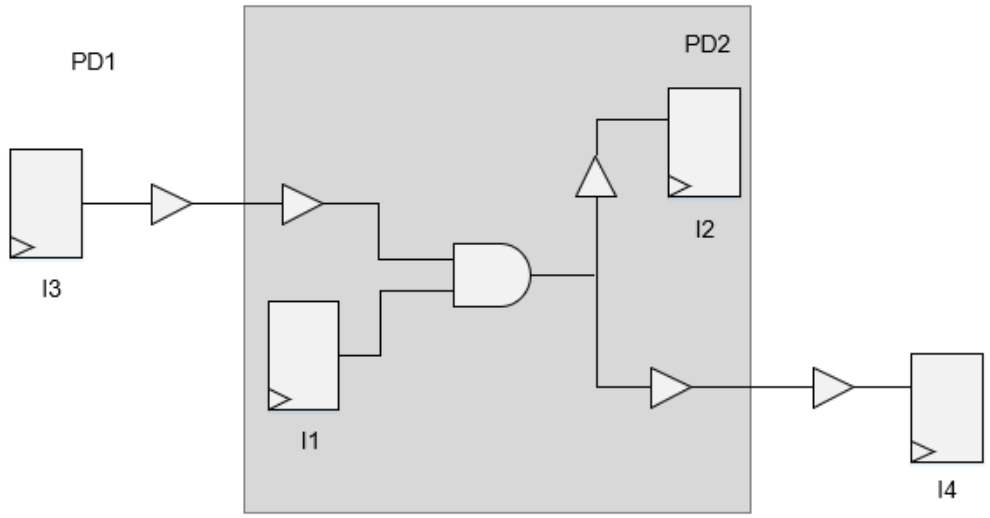
4.9.6 IPD Analysis Reporting

Due to the complexities involved in the identification of IPD logic, it may not always be possible to analyze only IPD paths during IPD GBA analysis - some non-IPD logic can also get analyzed, as part of GBA analysis.

For analysis of non-IPD paths, consider the following scenario:

- I3 to I2, I1 to I4 are the IPD paths (see figure below)
- I1 to I2 is a non-IPD path, though every cell in this path is part of another IPD path
- I1 to I2 path will also get analyzed and reported by default

Figure 4-83 Non-IPD Path Analysis - Illustration



Tempus supports additional settings to filter these non-IPD paths during reporting.

The `-ipd` parameter of the `report_timing` command reports only the IPD paths in the design:

```
report_timing -ipd
```

The IPD analysis header section in the timing report indicates the crossings that exist in the IPD paths.

```
IPD Analysis {pd_PD1 -> pd_PD2(L) pd_PD2 -> pd_PD3(D)}
```

In the above example, there is transition from power domain PD1 to PD2 in the launch path (L), post the CPPR point and from PD2 to PD3 in the data path (D).

The following abbreviations are used to identify the type of crossings that exist in a path:

- CC for the common clock path
- L for the launch clock path
- C for the capture clock path
- D for the data path

Sample Report

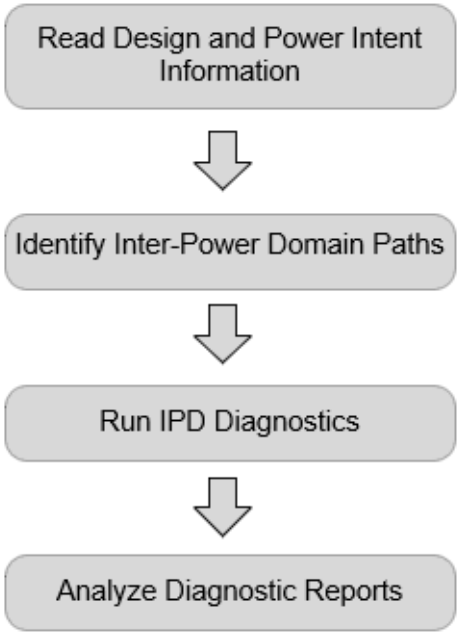
```
Path 1: MET Setup Check with Pin Inst1/regx1/CP
Endpoint: Inst1/regx1/D (^) checked with leading edge of 'clk'
Beginpoint: Inst2/regx8/Q (^) triggered by leading edge of 'clk'
Path Groups: {clk}
Analysis View: low_low_v
Retime Analysis { GBA }
IPD Analysis { pd_PD1 -> pd_PD2(L) pd_PD2 -> pd_PD3(D)}
Other End Arrival Time 0.955
- Setup 0.271
+ Phase Shift 3.000
= Required Time 3.684
- Arrival Time 3.552
= Slack Time 0.132
Clock Rise Edge 0.000
+ Clock Network Latency (Prop) 0.000
= Beginpoint Arrival Time 0.000
```

4.9.7 Inter-Power Domain (IPD) Logic Checks

Tempus provides an automated way of checking the sanity of IPD implementation to ensure completeness, thereby reducing the design cycle time and computations required for undesirable inter-power domain (IPD) crossings in a design.

The Tempus IPD analysis feature runs diagnostics on all the IPD paths and filters out those paths on which checks need to be performed. The concise diagnostic reports present all the details in a simplified format and provide a complete overview of the IPD implementation in a design. The flow is shown below:

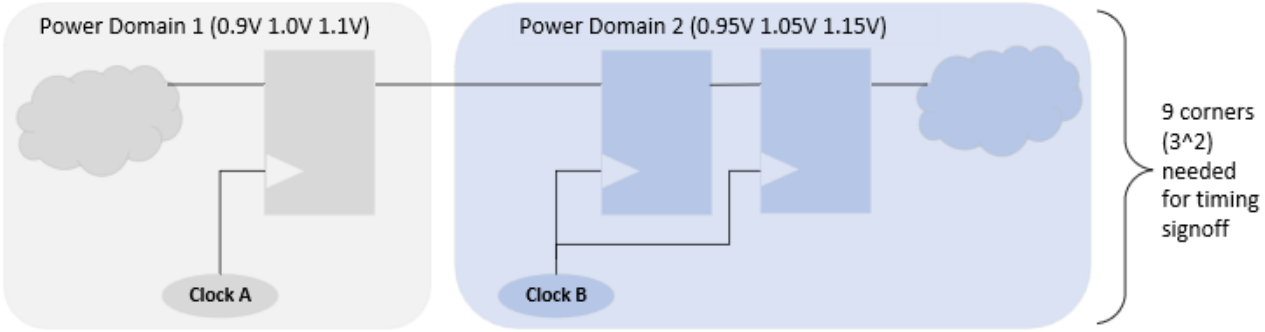
Figure 4-84 IPD Logic Checks Flow



The low power design techniques use designs that operate at multiple power domains for power optimization. The Dynamic Voltage and Frequency Scaling (DVFS) is a commonly used technique, which uses power domains for power or performance tradeoff.

The inter-power domain paths need to be carefully analyzed for timing signoff. For instance, a design with M voltages and N PDs may require M^N IPD signoff corners.

Figure 4-85 IPD Signoff Corners

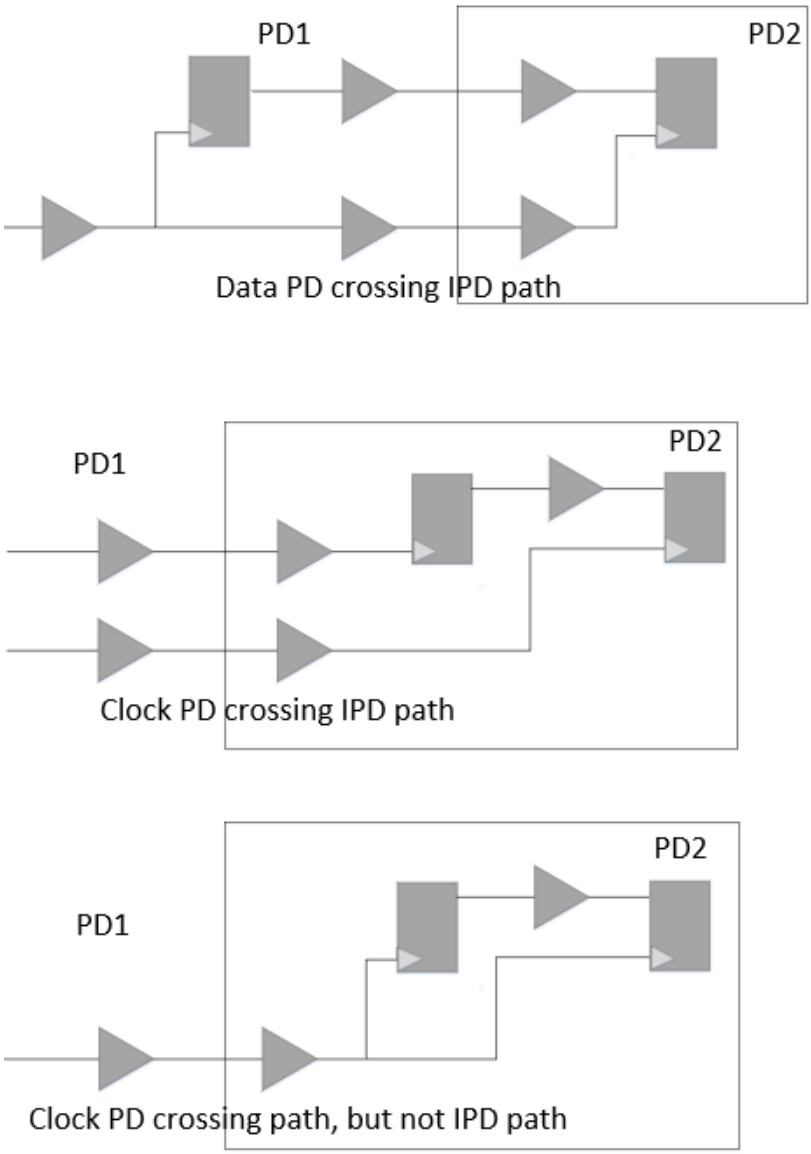


4.9.8 IPD Paths Classification

The design logic classification is based on the IPD analysis performed on the power domain crossing in the timing paths. The scenarios classified as IPD and non-IPD are described below:

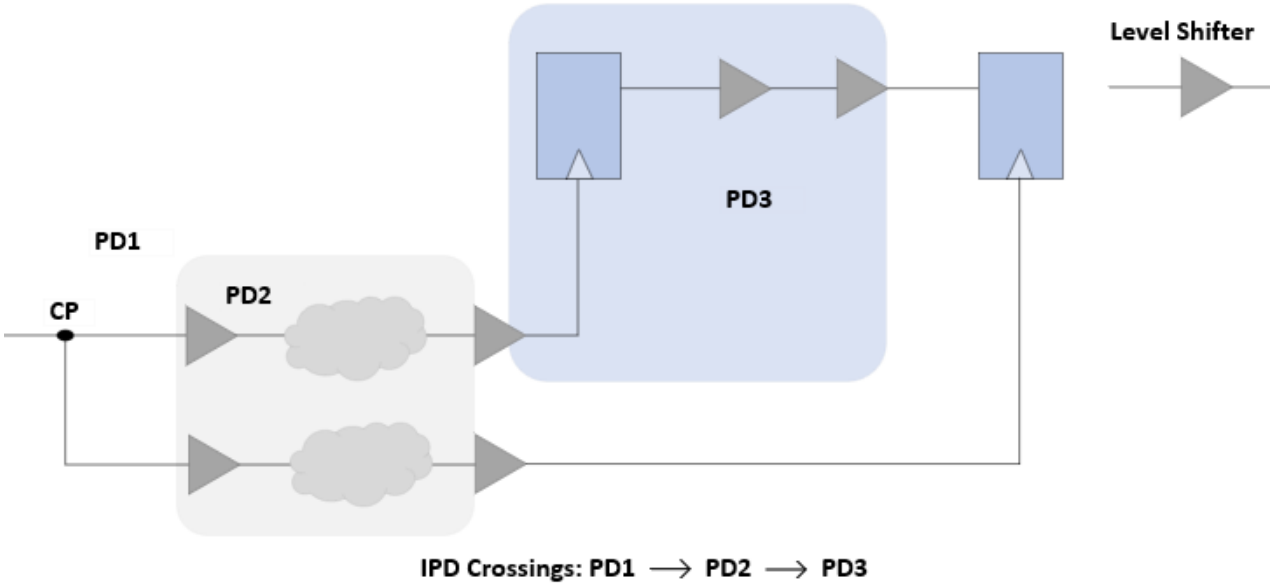
- Data path crossing more than one power domain will always be considered as an IPD path (irrespective of the power domains that the clock paths traverse, or the voltages on the power domains that the data path traverses).
- Launch and/or capture clock path (launch/capture clock path here refers to the path from the clock source to start point/endpoint clock pin) crossing more than one power domain will also be classified as IPD path when:
  - There is no common (CPPR) point on the clock path
  - CPPR point has different transition
  - Power domain transition occurs after the CPPR point of clock path

Figure 4-86 IPD Paths Classification



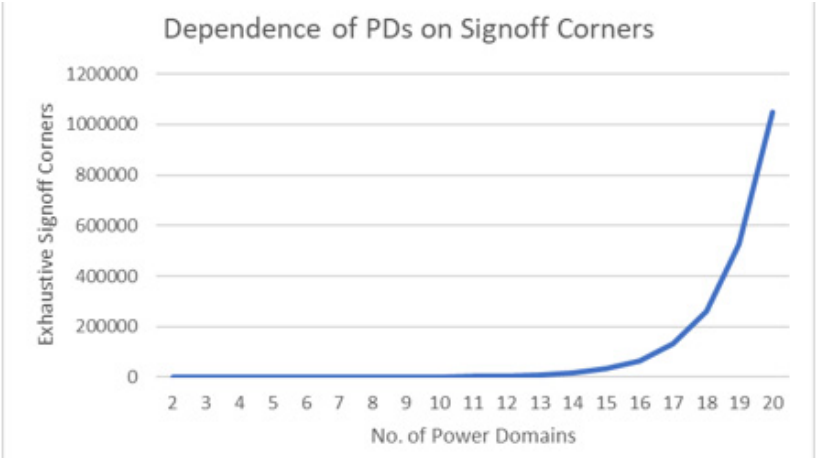
In some cases, the IPD paths may have undesired, yet avoidable IPD crossings. These undesired crossings through multiple power domains may add to extra voltage combinations for IPD paths signoff and cause issues or delays in low power signoff.

Figure 4-87 Undesirable IPD Crossings



The number of exhaustive IPD signoff corners are dependent on the number of voltage corners, where power domains operate, and their number. So an increase in the number of power domains may lead to an exponential increase in IPD signoff corners for a constant number of operating voltages.

Figure 4-88 Increase in Signoff Corners with increase in PDs



The minimum power domains required for a path to be an IPD path is two, post common point, and the minimum signoff corners can be four for two operating voltage conditions. Thus, even if only one path in the whole design travels more than one power domain, it can cause an exponential increase in signoff corners, thus affecting the overall turnaround time and computations.

4.9.9 Supported IPD Logic Checks

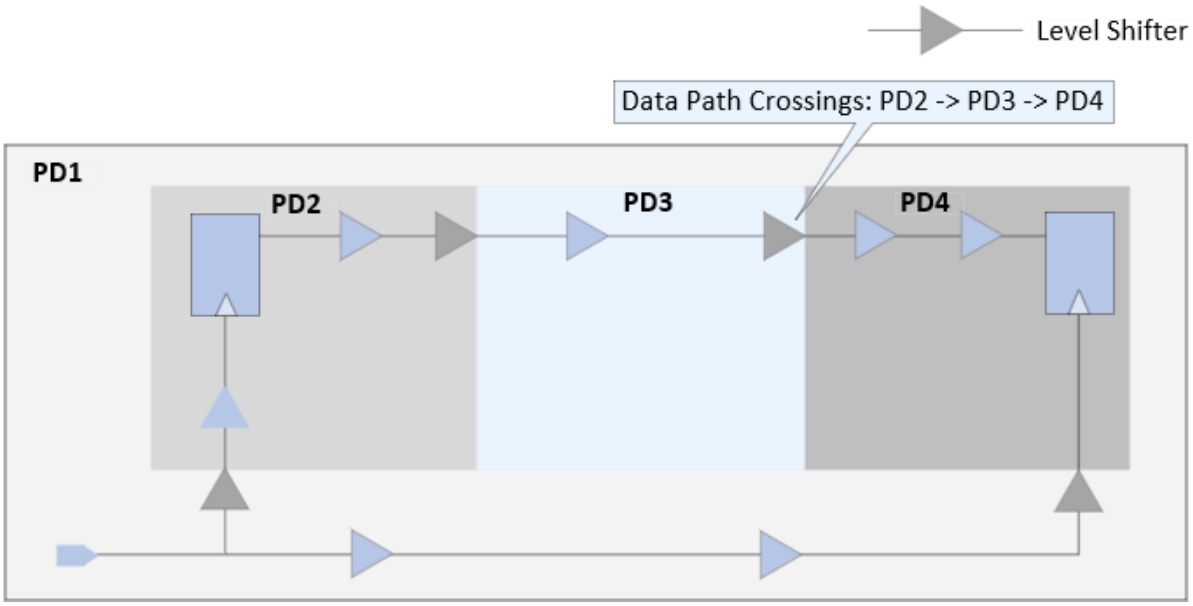
Tempus supports the following IPD logic checks:

- Multiple power domain crossings on data paths (Multiple Crossings on Data Paths)
- Multiple power domain crossings on clock paths (Multiple Crossings on Clock Paths)
- Similar crossings on the divergent clock paths (Similar Crossings on Divergent Clock Paths)
- Multiple power domains post common point (Multiple Power Domains Post Common Point)

Multiple Crossings on Data Paths

Consider the figure below, where the data path travels between three power domains, PD2 ' PD3 ' PD4, such a path will be highlighted by this check.

Figure 4-89 Multiple Crossings on the Data Path

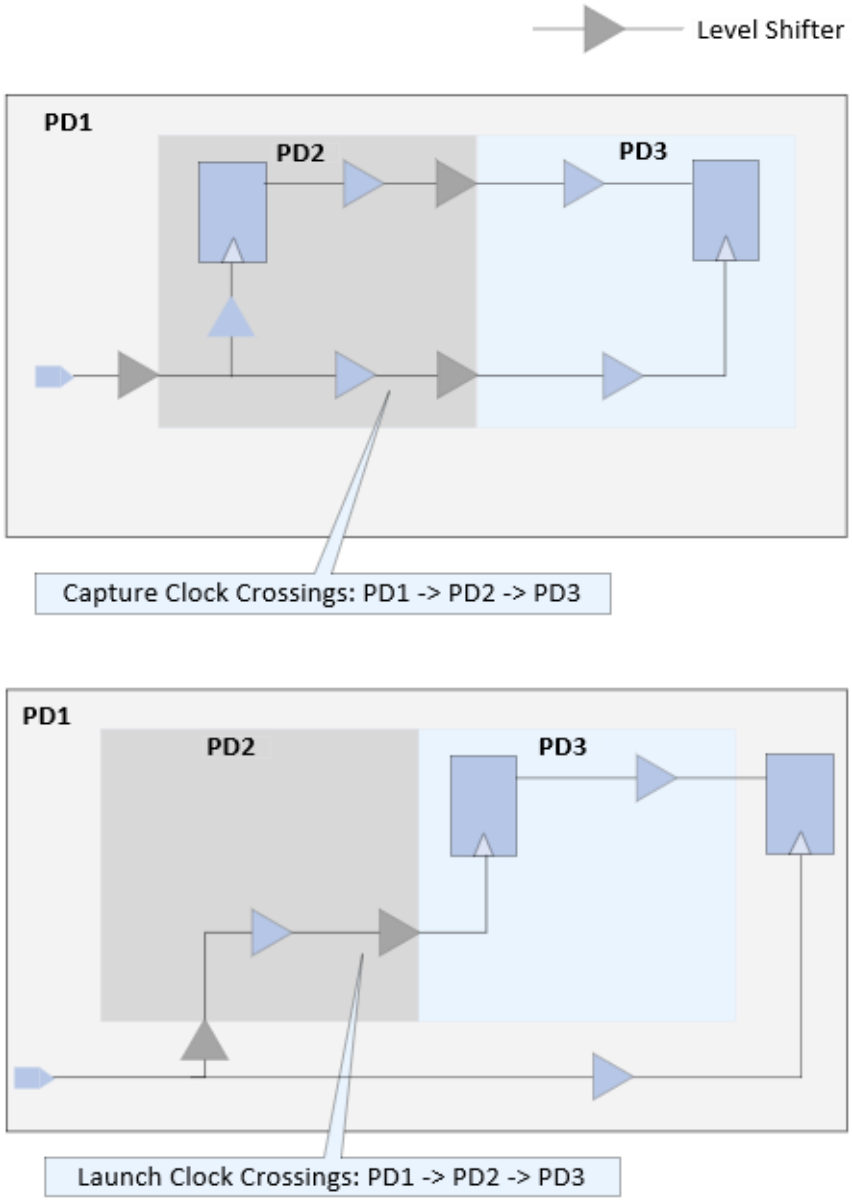


Multiple Crossings on Clock Paths

Consider the figure below, where in one case the capture paths crosses PD1' PD2 ' PD3, while in the other case, the launch path crosses the same PD transition, irrespective of the CPPR point. Both these cases will be flagged through this check. Only IPD paths are checked

for this check, and non-IPD paths having such crossings before the common points will not be reported.

Figure 4-90 Multiple Crossings on the Clock Path

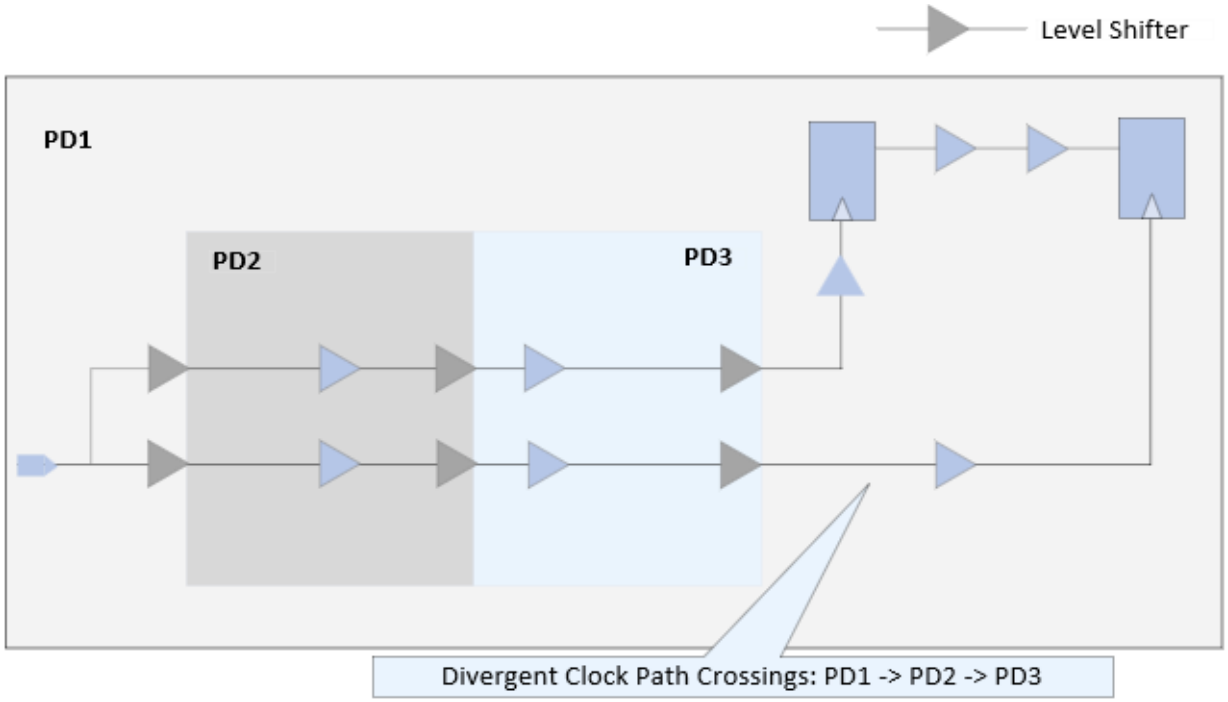


Similar Crossings on Divergent Clock Paths

In the figure below, both the launch and capture paths have the same PD transition, that is, PD1-> PD2 -> PD3. If the design had been implemented considering these transitions to be

on the common path, this path will not be considered an IPD path, thus leading to significant reduction in the added signoff corners. This check is useful in flagging such paths.

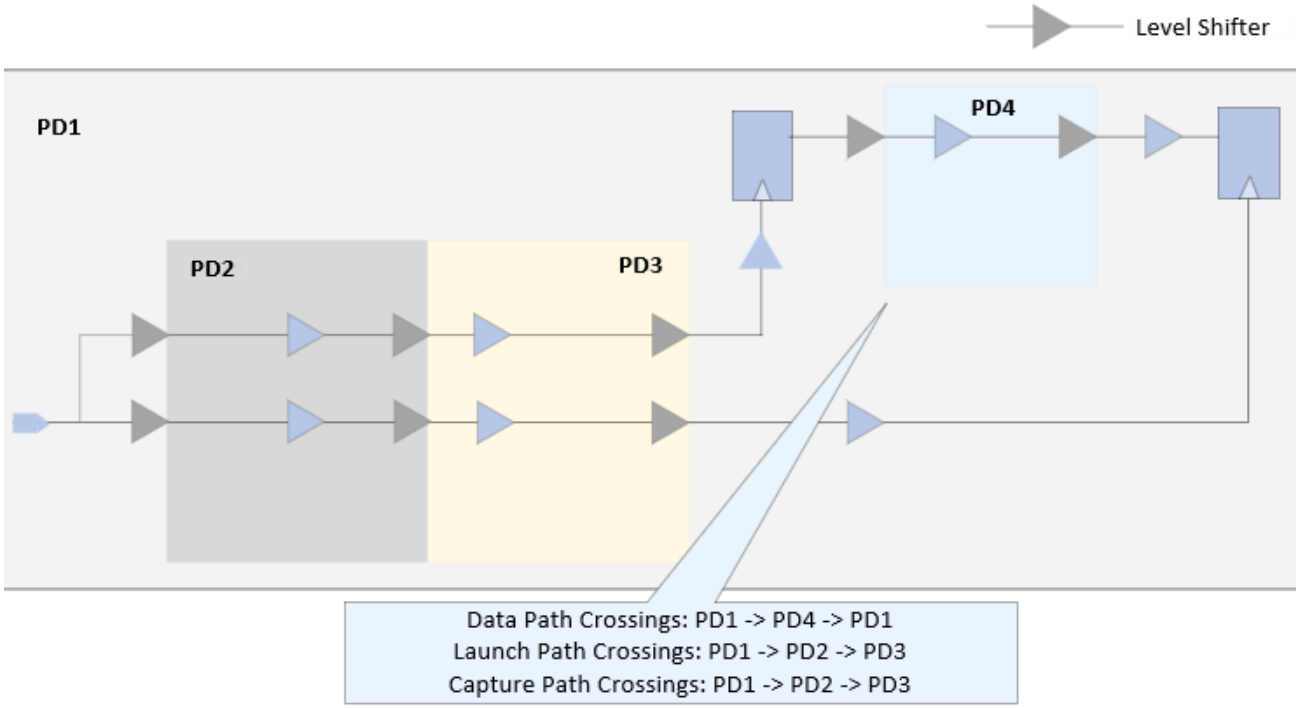
Figure 4-91 Crossings on Divergent Clock Paths



Multiple Power Domains Post Common Point

While the other IPD logic checks target IPD crossings, this check works on the number of power domains/PG net groups interacting on a timing path. As mentioned earlier, the minimum number of power domains needed for a path to be an IPD path is two. Any path having greater than two power domain interactions on either of the launch, capture, or data paths can be reported through this check. This check helps to understand the overall distribution of multiple PD crossings across the whole design.

Figure 4-92 Multiple PDs post common point



In the above figure, the total PD interactions on the entire path is 4 (PD1, PD2, PD3, and PD4). These type of paths can be reported through this check.

4.9.10 Performing IPD Logic Checks

The IPD logic checks can be performed in IPD analysis mode. To enable IPD analysis, the following setting is required at the top of Tempus script:

```
set_db timing_analysis_enable_inter_power_domain true
```

Note: The designs with power intent specification can be enabled for these checks post the `update_timing` command.

To report IPD logic checks in a design, use the following command:

- `check ipd logic`

Examples

The following command reports all the IPD logic checks in the design:

```
check_ipd_logic -multi_pd_type all -outfile ipd_logic_check_report -verbose
```

The output of ipd_logic_check_report.rpt report is as shown below:

```
################################################
# Multiple Power Domain Crossings On Data Paths  #
################################################
1.
Beginpoint: ff1/D (clk1)
Endpoint: ff2/Q (clk1)
PD1->PD2 lvl_cell1/Y
PD2->PD3 lvl_cell2/Y
2.
Beginpoint: ff3/D (clk1)
Endpoint: ff4/Q (clk2)
PD1->PD2 lvl_cell3/Y
PD2->PD3 lvl_cell2/Y

##########################
# Clock Divergence Data  #
##########################
1.
Beginpoint: ff4/D (clk2)
Endpoint: ff5/Q (clk2)
PD1->PD2 lvl_cell5/Y (L) level_cell6/Y (C)

########################################
# Multiple Clock Power Domain Crossings  #
########################################

#####################################
#Early Check Path Count Summary Output#
#####################################

Multiple IPD Data Crossing : 2
-----------------------------------
Count       Power Domain
-----------------------------------
2             PD1->PD2->PD3

IPD Clock Divergence : 1
-----------------------------------
Count          Power Domain
-----------------------------------
1              PD1->PD2
```

```
Multiple IPD Clock Crossing : 0
-----------------------------------
Count           Power Domain
-----------------------------------
ipd_logic_check.tcl

################################################
# Multiple Power Domain Crossings On Data Paths  #
################################################
report_timing -through lvl_cell1/Y -through lvl_cell2/Y
report_timing -through lvl_cell3/Y -through lvl_cell2/Y

##########################
# Clock Divergence Data  #
##########################
report_timing -from ff4/D -clock_from clk1 -to ff5/Q -clock_to clk2

########################################
# Multiple Clock Power Domain Crossings  #
########################################
```

4.10 Aging-Aware STA Analysis

4.9.1 Aging-Aware STA Analysis - Overview on page 238

- Bias Temperature Instability (BTI) Overview on page 238
- Hot Carrier Injection (HCI) Overview on page 239

4.9.2 Tempus Aging-Aware STA on page 240

- Input Requirements on page 240
- Modeling Recovery on page 246
- Aging Analysis Flow on page 247
- Enabling Aging Analysis on page 247

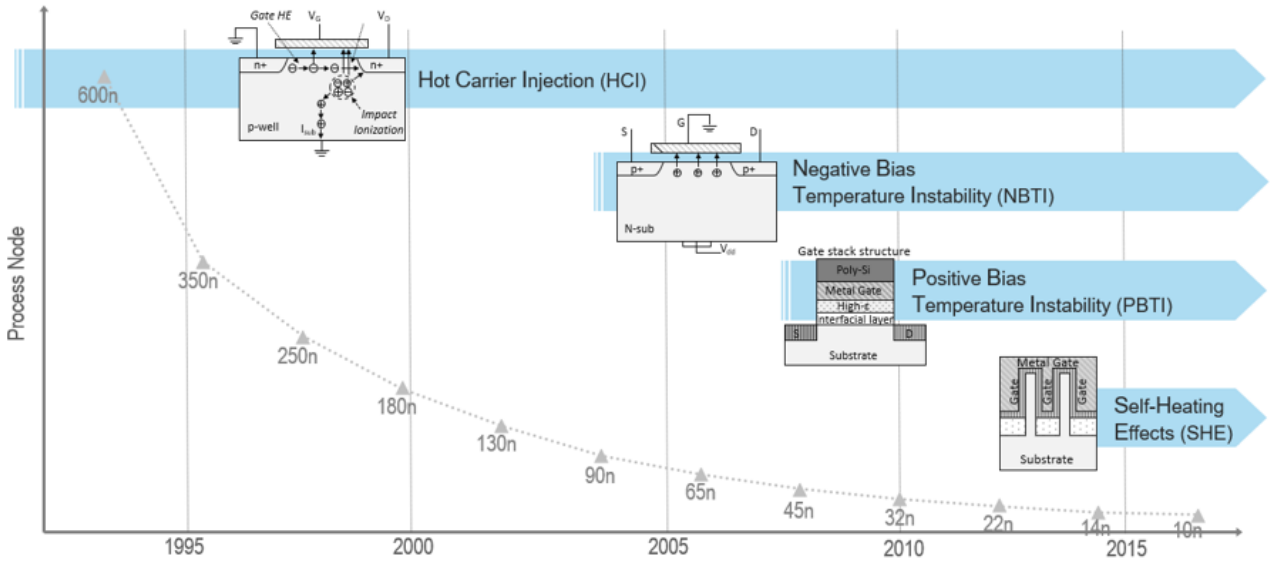
4.9.3 Aging-Aware Tempus vs SPICE Correlation on page 248

4.9.4 Spice Deck for Aging on page 251

4.10.1 Aging-Aware STA Analysis - Overview

The semiconductor device performance degrades over time due to physical factors, such as Bias Temperature Instability (BTI), Hot Carrier Injection (HCI), and so on. The following diagram shows process nodes and the sources of device failures.

Figure 4-93 Process nodes and sources of device failures - Illustration



Earlier, the following two methodologies were adopted to account for aging effects into STA:

- Flat timing derates to margin for aging impact
- Perform STA with timing libraries characterized for specific age

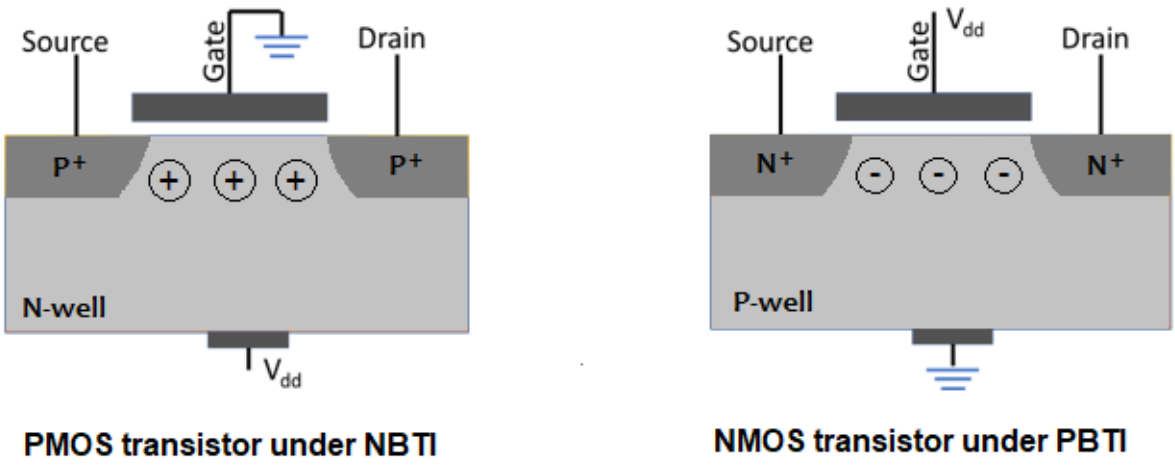
However, these methodologies had several limitations. The aging effects on device performance were never modeled comprehensively and accurately in STA. The Tempus advanced aging-aware STA addresses the limitations to improve the accuracy and PPA (power, performance, and area) of a design.

Bias Temperature Instability (BTI) Overview

Within the Si-SiO2 interface region, the silicon bonds are not satisfied and there exist unsatisfied or dangling bonds, that trap charges. The dangling silicon bonds are passivated by forming Si-H bonds (annealing the interface). When a transistor is biased in inversion, the dissociation of Si-H bonds along the silicon-oxide interface causes the generation of interface traps. The devices will be under higher BTI stress, when they are not switching. The rate of generation of these traps is accelerated at higher temperature and supply voltage. These

traps cause increase in threshold voltage (V_TH), decrease in drain current (I_d), and decrease in channel mobility (μ).

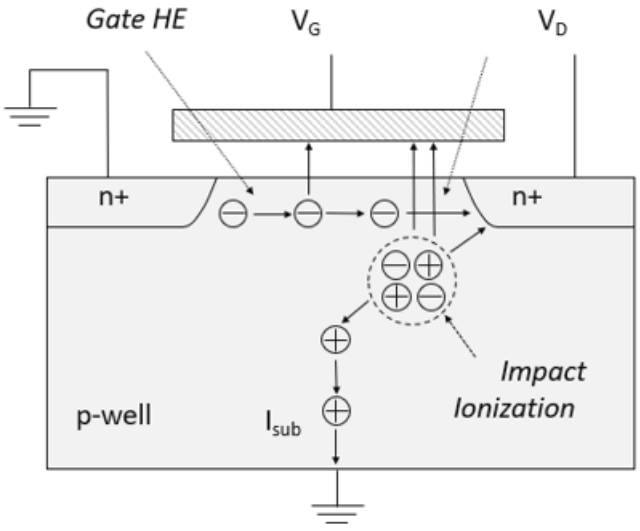
Figure 4-94 BTI - Illustration



Hot Carrier Injection (HCI) Overview

When transistor gate length decreases, the electrical field increases. This results in impact ionization. Some hot electrons (HE) generated by impact ionization are attracted to the gate. The hot electrons injected into the gate cause oxide damage and lead to device degradation. These devices are under higher HCI stress when they are switching.

Figure 4-95 HCI - Illustration



The main factors that influence device degradation are:

- Stress duration
- Temperature
- Supply voltage
- Logic conditions (duty cycle and switching activity)

Both BTI and HCI semiconductor devices degrade over a period of time. Typically, PMOS devices degrade more than NMOS devices.

4.10.2 Tempus Aging-Aware STA

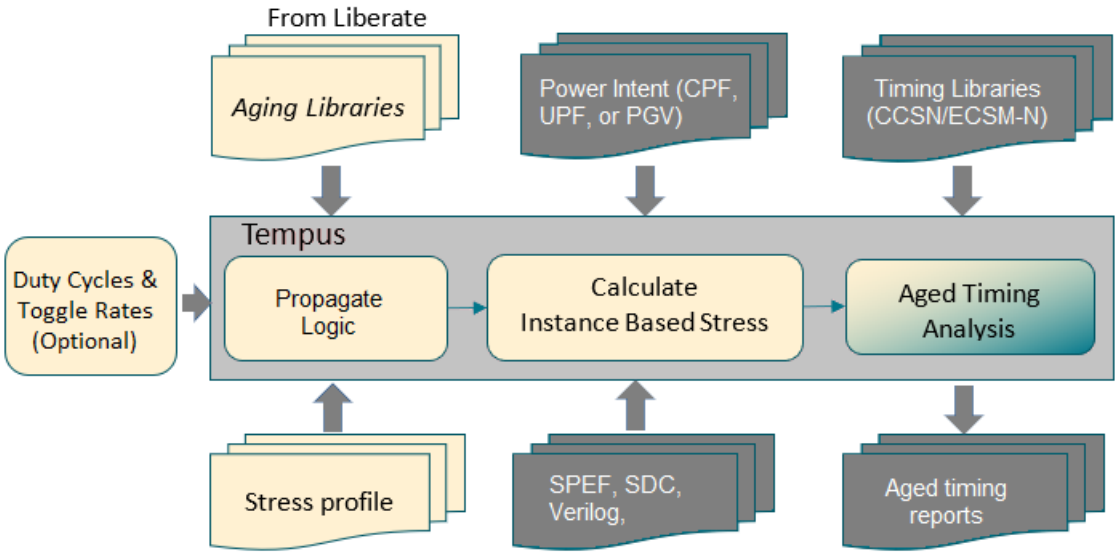
This section includes:

- Input Requirements on page 240
  - Fresh and Aged Libraries on page 241
  - Power Intent for Design on page 242
  - User-Defined Stress Profile on page 242
  - Duty Cycle and Toggle Rate Definition and Propagation on page 243
  - Duty Cycle Propagation for Data Path (Worst-Case) on page 245
- Modeling Recovery on page 246
- Aging Analysis Flow on page 247
- Enabling Aging Analysis on page 247

Input Requirements

The following diagram shows the input requirements for performing aging-aware STA.

Figure 4-96 Input Requirements



Fresh and Aged Libraries

The fresh libraries refer to the timing libraries in CCSN or ECSM-N format used in regular STA. The existing CCSN/ECSM-N timing libraries can be used for aging analysis. The Liberate generated and third-party generated timing libraries are accepted. Along with standard CCSN/ECSM-N libraries, aging libraries are also required for advanced aging analysis. These aging libraries contain cell level timing data for aging analysis and they must be generated by Liberate advanced aging characterization flow.

The aging library name should be the same as the corresponding timing library name for appropriate mapping of the information between aged library and timing libraries.

The aging libraries can be loaded into the Tempus software using the following commands:

MMMC Flow:

```
create_library_set -name lib_set1   \
        -timing fresh.lib \
        -liberty_incremental {{age {age.lib}}}
```

The MMMC flow with explicitly mapping age.lib to a specific fresh library is given below:

```
create_library_set -name lib_set1   \
        -timing fresh.lib \
        -liberty_incremental {{age{{age.lib ff105C}}}}
```

Where `fresh.lib` name is `ff105C`.

Non-MMMC Flow:

```
read_lib fresh.lib -liberty_incremental {{age {age.lib}}}
```

The MMMC flow with explicitly mapping age.lib to a specific fresh library is given below:

```
read_lib fresh.lib \
        -liberty_incremental {{age {{age.lib ff105C}}}}
```

Power Intent for Design

The power intent is required to calculate the stress for each instance in the design. The power intent of a design must be specified using PGV (Power- and Ground-aware Verilog) flow, UPF (Unified Power Format), or CPF (Common Power Format). The stress voltage specified in stress profiles for a power rail is allowed to be different than the nominal voltage of power rail at the given analysis corner.

User-Defined Stress Profile

The stress profile is a key input for aging-aware STA. The profile contains stress duration, stress temperature, and stress voltage per power rail. You can apply different stress profiles for different views, and different stress profiles for early/late timing. You can make stress profile settings using the `create_aging_stress_profile` command. The use model is given below.

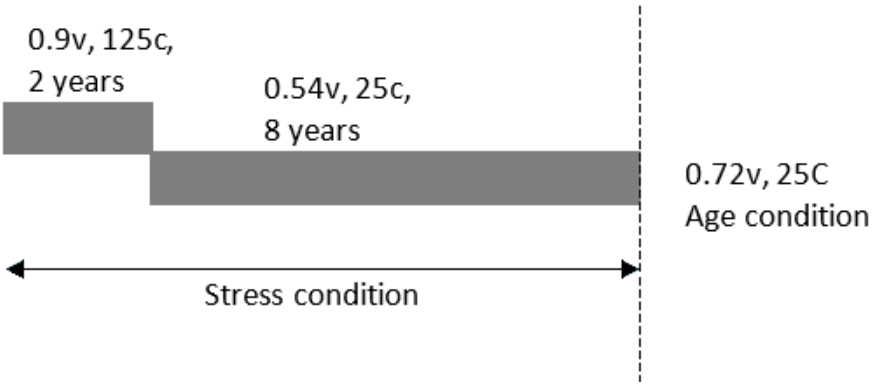
```
create_aging_stress_profile [-views {views}] [-early] [-late] \
[-bti_pmos_scale <scale_val>] \
[-bti_nmos_scale <scale_val>]
[-hci_pmos_scale <scale_val>] \
[-hci_pmos_scale <scale_val>] \
{segment1 segment2 …}
```

where `segment1`, `segement2`, etc, specifies the stress duration, voltages, and temperature information, such as:

```
{{duration <float>}[{voltage {<pg_net1> <float> <pg_net2> <float> …}}]
[{temperature <float>} ] }
```

The voltage and temperature details are optional in stress profiles. If not defined, the same voltage and temperature are used as operating voltage and temperature defined for fresh analysis. Optionally, BTI and HCI scales for PMOS and NMOS can be specified in the of range [0,1] (default: 1).

Figure 4-97 Stress Profile



The advanced aging analysis also supports multi-segment stress profile, as shown in the figure above. As shown in the illustration, you can specify initial 2 years of aging with one stress condition and the rest of the 8 years with different stress condition for a total 10 years of aging.

An example of multi-segment stress profile is given below:

```
create_aging_stress_profile -view view1  \
    {{{ duration 2} {voltage {VDD 0.9 VSS 0.0}}{temperature 125}} \
    {{duration 8 } {voltage {VDD 0.54 VSS 0.0}}{temperature 25}}}
```

Duty Cycle and Toggle Rate Definition and Propagation

By default, all the signals in STA are assumed to be 50% of duty cycle at primary pins. The duty cycle for clock waveforms can be specified in the SDC. The same duty cycle is assumed at every clock pin associated with that waveform. You can override the duty cycle defaults for primary ports, as well as force duty cycles on specific nets in the design.

Starting from the initial default values (and user-overrides), Tempus propagates the duty cycle across the logic. Tempus gives higher priority to user-defined duty cycle than duty cycle obtained from the SDC, using either constants or clock waveforms.

You can specify duty cycles on nets using the `set_aging_activity` command, as shown below:

```
set_aging_activity [-views {list of views}]    \
    [-min_duty_cycle <value>]  \
    [-max_duty_cycle <value>]  \
    -net <list or collection of nets>
```

For 50% duty cycle (default), the `-min_duty_cycle` and `-max_duty_cycle` values are 0.0 and 1.0, respectively. The duty cycle is propagated downstream from the net where it is

asserted. You can set the default duty cycle using command and options defined in Table 4-4 on page 246.

HCI analysis requires both duty cycle and toggle rate information. Toggle rate is the number of transitions that a signal makes within (change in logic state) one second. Traditional STA does not require toggle rate, thus does not assume any defaults.

You can specify toggle rates on nets using the following command:

```
set_aging_activity [-views {list of views}]    \
                   [-min_toggle_rate <value per second>]  \
                   [-max_toggle_rate <value per second>]  \
                   -net <list or collection of nets>
```

For aging STA, the defaults for toggle rate are 0.0 (min_toggle_rate) and 1e8 (max_toggle_rate) transitions per second. The clocks specified in SDC are initialized with toggle rate 2 per period. You can set the default toggle rates using command and options defined in Table 4-5 on page 246.

The duty cycle and toggle rate assertions can be queried or printed using the `get_aging_activity` and `report_aging_activity` commands, as shown below:

```
get_aging_activity [-views <list of views>] \
                   -net <list or collection of nets>
report_aging_activity [-duty_cycle] \
                   [-toggle_rate] \
                   [-views <list of views>] \
                   -net <list or collection of nets> \
                   [-out_file <filename>]
```

Tempus supports duty cycle and toggle rate propagation modes, as given below:

- Probabilistic (statistical average)
- Worst case

You can set the propagation mode using the following settings:

Probabilistic (statistical average) mode:

```
set_aging_analysis_mode -activity_propagation_mode probabilistic
```

Worst case mode:

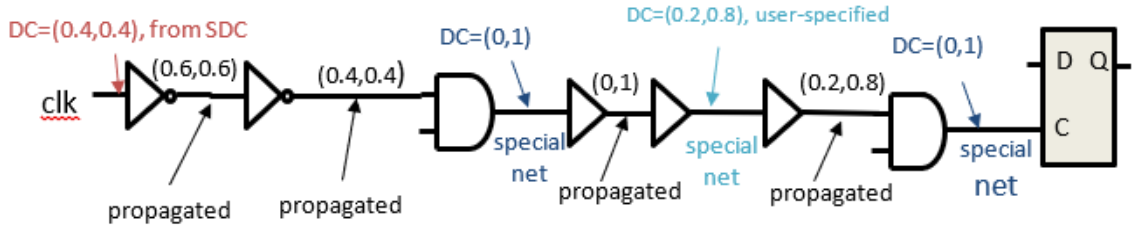
```
set_aging_analysis_mode -activity_propagation_mode worst_case
```

The activity propagation, by default, resets at sequential cell outputs. At the sequential outputs, the net duty cycles and toggle rates are initialized using sequential defaults (see

Table 4-4 on page 246 and Table 4-5 on page 246 to set defaults). If such a reset is not required, then you can control activity propagation across sequential cell delay arcs using the following command:

```
set_aging_analysis_mode -sequential_activity_propagation {true | false}
```

Figure 4-98 Duty cycle propagation through clock network (Worst-case)

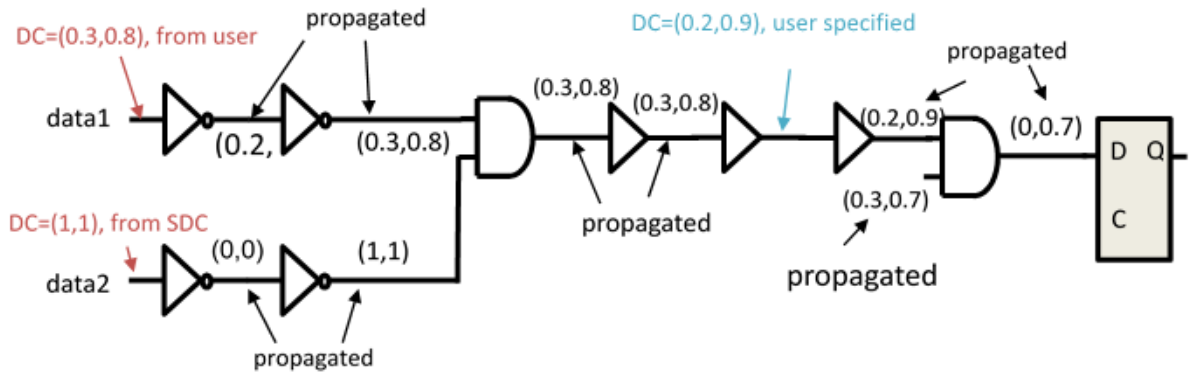


The duty cycle set on clock root propagates until it reaches a special net or clock endpoint. A clock endpoint is the clock pin of a flop from which a clock does not propagate. A special net in this case is defined as either a net driven by multi-input cell or a clock net, where the user-defined duty cycle is applied. The nets driven by multi-input clock cells assume default duty cycle, which is then propagated through downstream logic until it encounters another special net or clock endpoint.

Duty Cycle Propagation for Data Path (Worst-Case)

You can set duty cycles and toggle rates on primary data input ports. Tempus propagates these across the logic until it encounters special nets or data endpoints. Unlike clock network, a data path does not treat nets driven by multi-input cells as special nets.

Figure 4-99 Duty cycle propagation for data path (Worst-case)



You can force default duty cycles using the following settings:

Table 4-4 Setting default duty cycle values

Target Duty Cycle	0.0%	25%	100.0%	
-min_duty_cycle	0.0	0.25	1.0	Sets duty cycle on ports and special nets
-max_duty_cycle	0.0	0.25	1.0	
-sequential_min_duty_cycle	0.0	0.25	1.0	Sets duty cycle on sequential cell output pins
-sequential_max_duty_cycle	0.0	0.25	1.0	

You can define default toggle rates using the following settings:

Table 4-5 Setting default toggle-rate values

Target Toggle Rate	Value (transitions/second)	
-min_toggle_rate	0.0	Sets toggle rate on ports and special nets
-max_toggle_rate	1e8	
-sequential_min_toggle_rate	0.0	Sets toggle rate on sequential cell output pins
-sequential_max_toggle_rate	1e8	

Modeling Recovery

The OMI and URI Spice models are recovery-aware. The aging libraries characterized by Liberate include recovery as well. Tempus can model BTI recovery when aging libraries include recovery models.

You can use the following command for OMI and URI model recovery:

```
set_aging_analysis_mode -recovery true
```

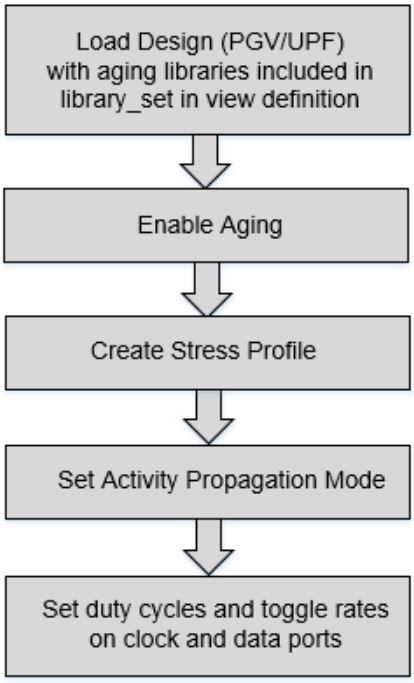
The TMI Spice models are not recovery aware. To model recovery for technology nodes, you must provide the recovery scaling factors using the following command:

```
create_aging_stress_profile …-bti_pmos_scale <value> \
                             -bti_nmos_scale <value> \
                             -hci_pmos_scale <value> \
                             -hci_nmos_scale <value>
```

Aging Analysis Flow

For enabling aging analysis, the flow is given below:

Figure 4-100 Aging Analysis Flow



Enabling Aging Analysis

The aging analysis can be enabled using the following command:

```
set_db timing_analysis_aging true
```

The aging feature must be enabled before a timing update and cannot be disabled later in the same session.

The impact from BTI and HCI can be controlled using following commands:

```
set_aging_analysis_mode -type {bti | hci |all}
```

The default value of `-type` parameter is `all` - both `BTI` and `HCI` are enabled.

Example Script:

```
read_view_definition  view_def_template.tcl  #include Aging libs
read_verilog test.v
set_top_module test
```

```
#PGV-Specific flow; below two steps required only for PGV flow
infer_pgv_power_intent
set_analysis_view -setup […] -hold […]
#CPF/UPF testcase; required only for CPF/UPF flow
read_power_domain -cpf <file>
set_analysis_mode -aging true
# Define stress profile
create_aging_stress_profile -view view1 \
       {{{duration 2} {voltage {VDD 0.9 VSS 0.0}}{temperature 125 }} \
       {{duration 8} {voltage {VDD 0.54 VSS 0.0}}{temperature 100}}}
# Set activity propagation mode
set_aging_analysis_mode -activity_propagation_mode probabilistic
# Run timing analysis
update_timing -full
```

Enabling Aging of Constraint Arcs

By default, the aging impact is not enabled on constraint arcs. You can enable it using the following command (aging library should have been characterized with constraint arcs):

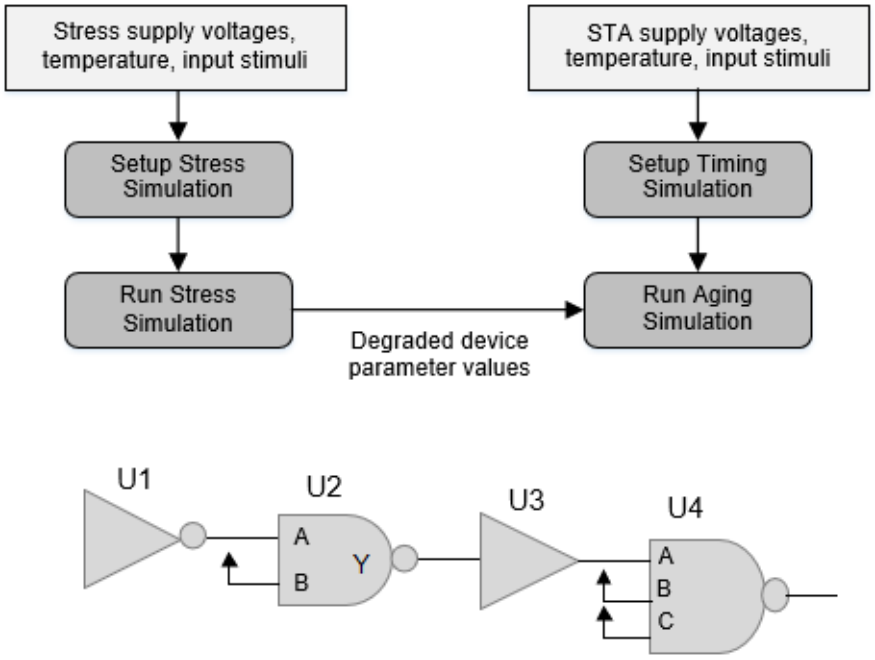
```
set_aging_analysis_mode -constraints {true | false}
```

4.10.3 Aging-Aware Tempus vs SPICE Correlation

The Tempus STA vs Spice correlation is performed at the path level. There are two main aspects of aging-aware Spice simulations.

- Stress simulation: Uses Spice simulator's reliability analysis flow to degrade devices. This requires reliability models from foundry (typically a set of text files and binaries).
- Aging simulation: Uses device degradation information produced by stress simulation to run degraded circuit analysis. The delays and slew measured from this simulation includes aging effects. The delays and slews produced by this may be larger than the delays and slews produced by fresh (i.e., not aged) circuit simulation.

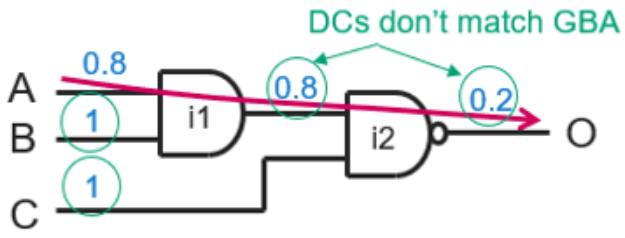
Figure 4-101 Spice correlation



Spice correlation to Tempus is path-based analysis. The side inputs of instances through which path does not traverse are tied to the constant values. Consider for example, U2/B, U4/B, and U4/C pins in the above circuit. Due to this, the real duty cycles cannot be propagated in Spice. During STA, however, Tempus analysis considers duty cycle propagation from all inputs of multi-input gates and computes duty cycles at the output. A lack of duty cycle propagation in Spice can cause mis correlation with Tempus. This is addressed in a two step Spice correlation approach.

- Stress and timing correlation
- Timing-only correlation

Figure 4-102 Stress and Timing correlation



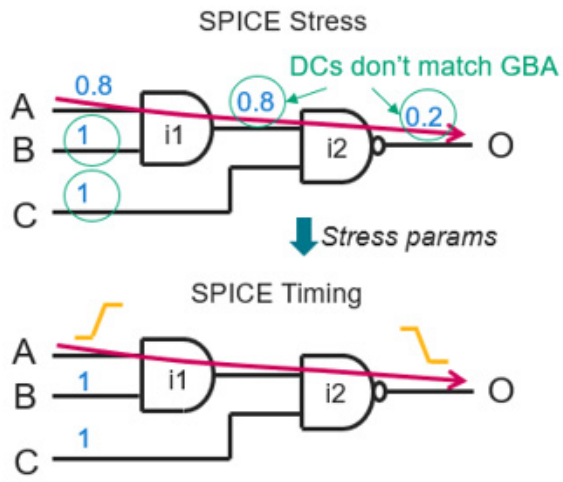
Method 1 (Stress and Timing Correlation)

The side pins in Spice are held at logic constants 0 or 1, so the duty cycle should be fixed for such pins. A fixed duty cycle is used for all the side pins of a cell in a path under test. Toggle rate for side pins is set to 0.

Spice simulation is performed in two iterations. In the first iteration, the stress parameters are computed using the fixed duty cycle at side pins. In the second iteration, those stress parameters are passed on to run aging simulation in order to get the degraded delays and slews.

Therefore, both the stress and aging validation are accomplished using this method. The only validation that is not covered in this method is the stress, which is computed based on the propagated duty cycles and toggle rates. This method uses the fixed duty cycle at every pin to compute the stress.

Figure 4-103 Method 1 (Stress and Timing correlation)

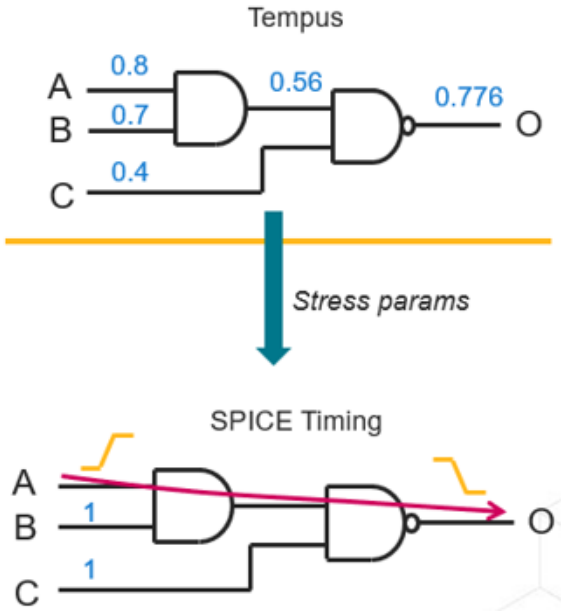


Method 2 (Timing-Only Correlation)

This method uses realistic duty cycles and toggle rates to be used for side pins. The device stress is computed based on the duty cycles and toggle rates used by the Tempus STA engine. Those device stress parameters are then directly fed to the SPICE deck for aging analysis.

With this approach, only timing validation is possible. Spice simulation is performed in a single iteration.

Figure 4-104 Method 2 (Timing-only correlation)



Spice correlation method can be selected using the following command:

```
write_spice_deck -aging_correlation_type {full | timingonly}
```

where `full` is method 1 and `timingonly` is method 2.

While using method 1, the following command is required for forcing fixed duty cycle:

```
set_aging_analysis_mode -use_fixed_duty_cycle true
```

4.10.4 Spice Deck for Aging

The `write_spice_deck` command can be used to generate Spice deck for aging. The use model is given below.

```
write_spice_deck
-reliability_model_type { URI | OMI | TMI }  \
     -aging_correlation_type { full | timingonly }  \
     [-aging_stress_use_timing_slew {false | true}] \
     [-aging_stress_num_cycles <integer greater than 0>] \
-uri_lib_file <file> \
-aging_level_only "[(<value>) (<value>) ..]" \
-omipath <path> \
[-reliability_commands <string>] \
[-supply_voltage_file_stress <file>]
     [-spice_include_file <string>]
```

For further details, refer to the *Tempus Text Command Reference* document.

4.11 Advanced Multiple-Input Switching Analysis

4.11.1 Advanced Multiple-Input Switching - Overview on page 254

4.11.2 User Interface and Timing Analysis with Advanced MIS on page 255

4.11.3 Enabling Advanced MIS Feature on page 255

4.11.4 Enabling/Disabling MIS for Library Cells on page 255

- Switching-Alignment Factor on page 256
- Library Cell Switching Alignment Factor on page 259

4.11.5 MIS and SSI Analysis on page 259

4.11.6 MIS Delta Delay Reporting on page 261

4.11.7 Spice Validation Support and Results on page 263

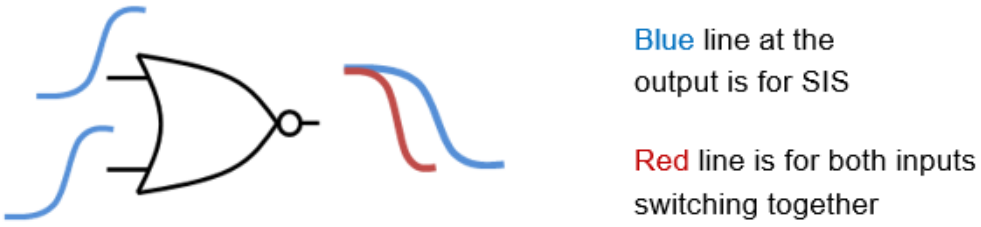
4.11.1 Advanced Multiple-Input Switching - Overview

A multi-input switching (MIS) event occurs when two or more inputs of a gate are switching together, close enough in time to cause either a larger or smaller delay than any of the inputs switching alone. Traditional STA assumes that only one input switches at a given time and all other side inputs are at static logical state. This is known as single-input switching (SIS).

Pessimism may be caused by multiple inputs switching on delay. Tempus supports derate-based methodology for modeling SSI (simultaneous switching inputs) for eliminating optimism. However, the derate-based approach is pessimistic and may impact PPA (performance, power, and area).

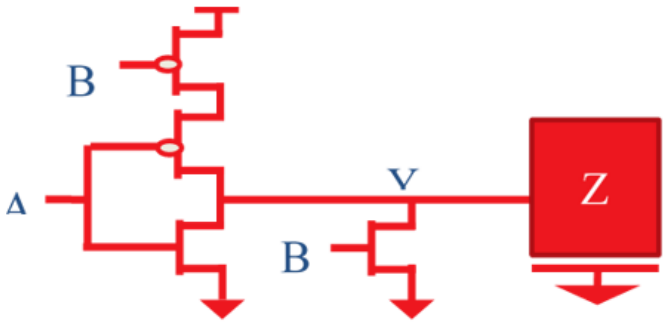
This phenomenon is most commonly observed in gates such as NOR and NAND (see figure below).

Figure 4-105 NOR-2 gate has parallel n-stacks both discharge output when turned on



The reason for this is that when both inputs rise together, they open two parallel n-stacks (see figure below). Both of these stacks will conduct current and hence cause the output load to discharge faster. Tempus STA with MIS models this combined effect of multiple currents from parallel stacks accurately.

Figure 4-106 NOR gate showing speed-up with two inputs rising.



4.11.2 User Interface and Timing Analysis with Advanced MIS

The Tempus delay calculation engine supports advanced MIS natively.

Figure 4-107 SSI in delay calculation



During delay calculation simulations, Tempus models SSI as concurrent current sources switching at certain times based on their arrival-timing windows. The MIS is also supported for tied inputs, which means the inputs tied together are recognized as switching at the same time. Advanced MIS is supported in Tempus for both GBA and PBA analyses. Tempus models only the speed-up caused by MIS for early analysis (early delays).

4.11.3 Enabling Advanced MIS Feature

The MIS feature can be enabled using the following setting:

```
set_db timing_analysis_multi_input_switching_mode { 0 | 1| 2 }
```

The options are described below:

- 0: (default) SSI is not considered.
- 1: MIS analysis considers instances for which multiple inputs of the cell are tied together.
- 2: MIS analysis considers both tied inputs and inputs of the instance which are switching independent of each other.

4.11.4 Enabling/Disabling MIS for Library Cells

When advanced MIS is enabled, Tempus recognizes the library cells that can be supported for MIS analysis. The `has_mis` property identifies the library cells that are considered for MIS analysis. You can query cells for the `has_mis` property using the `get_property` command or generate reports for MIS supported cells, as shown in the example given below:

```
foreach_in_collection each_lib_cells  [get_lib_cells *] {
   if {[get_property $each_lib_cells   has_mis] == "true"} {
```

```
    puts "[get_property $each_lib_cells   hierarchical_name]
  }
}
```

You can disable or enable MIS support for a given library cell by using the following command:

set_multi_input_switching_mode -disable_lib_cells <lib_cells collection>

Example:

```
set_multi_input_switching_mode -disable_lib_cells [get_libs_cell aoi*]
set_multi_input_switching_mode -disable_lib_cells [get_libs_cell lib_0.510v/aoi*]
```

Switching-Alignment Factor

Tempus models MIS when multiple inputs of a given instance are switching independent of each other, but in relative proximity to each other to make an impact on timing.

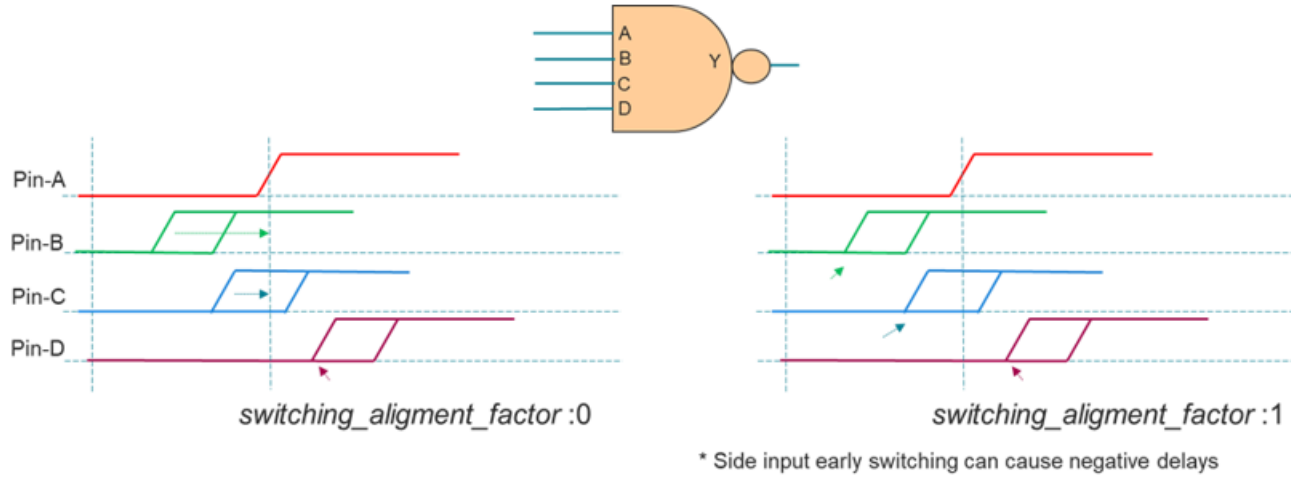
You can configure the relative alignment of side inputs switching with respect to the primary input to tune the impact of MIS using the following command:

set_multi_input_switching_mode -switching_alignment_factor [value between -1 to 1]

The values are described below:

- 1: Side inputs switch at the earliest arrival.
- 0: Side inputs switch at the same time as the actual input (default).
- -1: All the side inputs switch at the same time and align at the completion of the output pin transition.

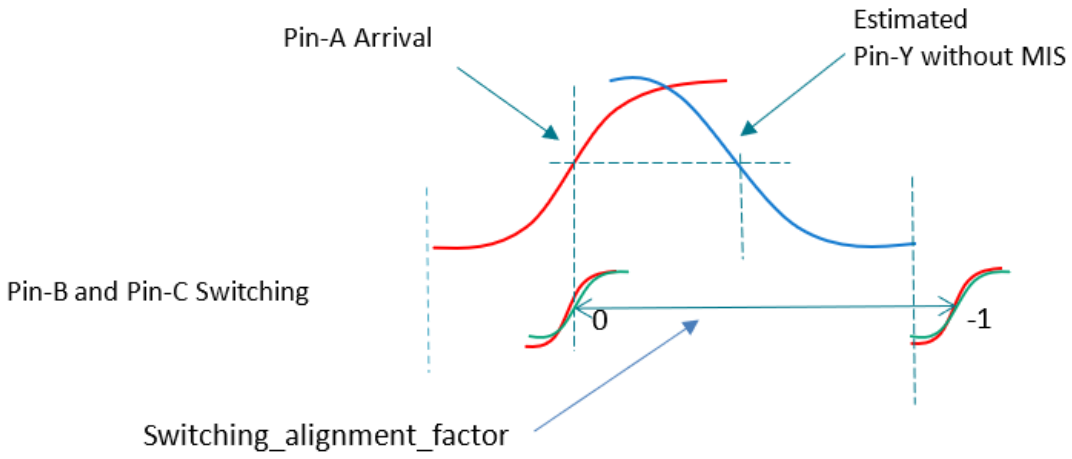
Figure 4-108 Switching-alignment factor values between 1 and 0



When the `-switching_alignment_factor` parameter value is set to 1, the side inputs switch at their earliest arrival timing. When the value is shifted towards 0, the side-input switching moves towards the primary-input arrival time. When the value is set to 0, the side inputs align with the primary input. When the factor is set between 1 and 0, the side inputs may switch much earlier than the primary input, causing the output to switch before the primary input, resulting in negative delays. The negative delays in data path are pessimistic for hold analysis. When a side input switches later than the primary input (as Pin-D in the figure above), it always uses the early-arrival time, irrespective of the `-switching_alignment_factor` value. This can influence output transition.

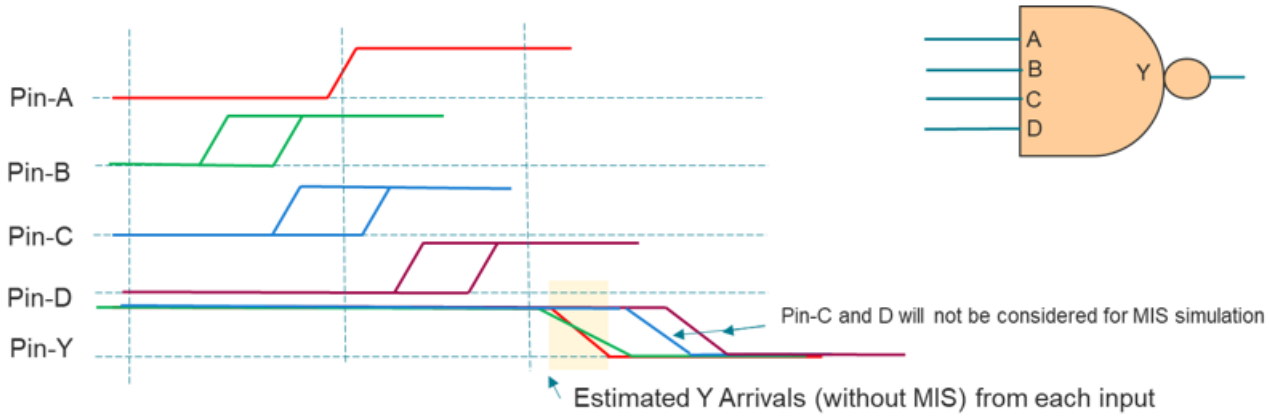
Setting the `-switching_alignment_factor` parameter to 1 can be very pessimistic if a side input switches much earlier than the main input. You can use the `set_multi_input_switching_mode -mis_alignment_mode` parameter to reduce pessimism in this mode.

Figure 4-109 Switching-alignment factor values between 0 and -1



When the `-switching_alignment_factor` parameter is set between 0 to -1, all the side inputs switch at the same time. When the value is -1, all the side inputs switch at the same time and are aligned at the completion of the output pin transition (see figure above). Since the output has already completed the transition, side-input switching has no impact on the timing of the A->Y arc. When the value moves towards 0, MIS impact will be observed. This behavior applies to both PBA and GBA.

Figure 4-110 Overlap alignment mode



In this mode, the side inputs that switch far from the main input are not considered for MIS. Early arrivals at the output pin caused by each input pin are evaluated to assess whether a side input should be considered for simultaneous switching or not. If estimated arrivals of a side input switch within certain vicinity of the main input, the side input is considered as SSI. Other inputs are considered as static values. The inputs considered as SSI are aligned based

on the switching-alignment factor value. This mode eliminates the inputs that are switching much earlier than the main input, which may cause large negative delays.

```
set_multi_input_switching_mode -mis_alignment_mode {0 |1}
```

Library Cell Switching Alignment Factor

The switching-alignment factor can be applied for all the instances for which MIS analysis is performed. However, the Tempus software allows you to tune the MIS impact for the instances of a specific library cell. You can set the following library cell properties to override the switching-alignment factor for the instances of a specific library cell. This property can be set for rise/fall transition, which gives a more granular control on tuning the MIS impact.

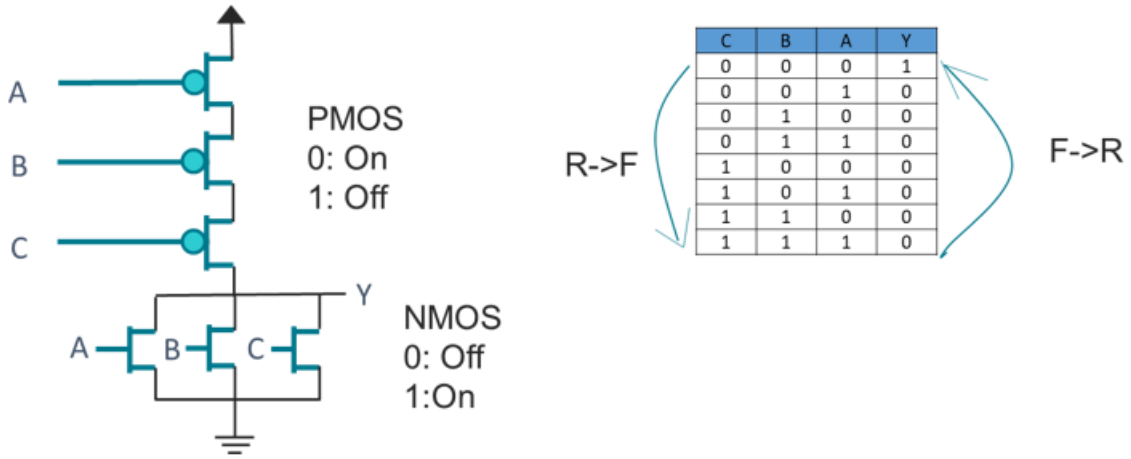
```
set_interactive_constraint_modes [all_constraint_modes]
define_property -object_type lib_cell -type float
mis_alignments_factor_min_rise \
          -lower_bound -1 -upper_bound 1
define_property -object_type lib_cell -type float mis_alignment_factor_min_fall \
          -lower_bound -1 -upper_bound 1
set_property [get_lib_cells ...] mis_alignments_factor_min_rise 0.5
set_property [get_lib_cells ...] mis_alignments_factor_min_fall 0.5
set_interactive_constraint_modes {}
```

4.11.5 MIS and SSI Analysis

It is a prerequisite to have conditional arcs for a library cell in order to support MIS analysis. Some complex standard cells or macro cells may not have conditional arcs. In such cases, the SSI derates specified on library cells can be used for modeling MIS impact.

When advanced MIS is enabled, and there might be cells that are supported for MIS analysis (`has_mis` property is `true`) and may have SSI derates defined, then MIS takes precedence. For cells that are not supported or for which MIS is disabled (as described in Enabling/ Disabling MIS for Library Cells on page 255), the SSI derates will be used.

Figure 4-111 3-input NOR Gate



C	B	A	Y
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	0

In the figure above, for the output to switch to fall, inputs have to switch from 0 to 1. This will make the NMOS transistors to turn on, making the current drain faster. So, the Rise->Fall arc will consider the MIS impact. However, for the output to switch to rise, the inputs will have to switch from 1 to 0. This will turn off NMOS, and until the PMOS transistors are turned on, Y will not complete the transition. MIS can only cause a slowdown for the Fall->Rise transition and, thus, it is not considered for MIS impact. However, the SIS, which uses a derate-based methodology, applies derates for the Fall->Rise transition as well.

Clock Group Handling

The MIS analysis honors clock group relations specified using the `set clock groups` command.

Physically Exclusive Clocks

If data signals of side inputs and main inputs are triggered by different clocks that are defined as physically exclusive to each other, such inputs are not treated as simultaneously switching.

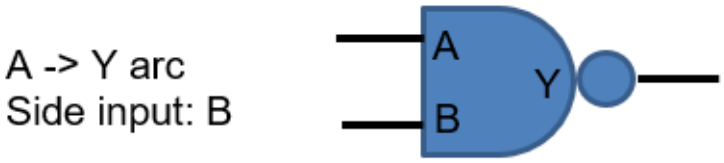
Asynchronous Clocks

If data signals of side inputs and main inputs are triggered by different clocks that are defined as asynchronous to each other, such inputs use infinite timing window. For MIS simulation, such inputs are treated as switching together.

In case Infinite timing windows are enabled for analysis (using the following command) all the inputs that are not triggered by physically exclusive clocks are treated as switching together.

```
set_db si_use_infinite_timing_window true
```

Few Scenarios of Multiple Clock Signals



1. Signal at input A triggered by CLK1 and input B has signals triggered by CLK1 and CLK2. Both CLK1 and CLK2 are synchronous to each other. In such scenarios, arrival at pin A is checked with timing windows of B with both clocks.

   Alignment that produces worst-case delay is used for analysis.

2. Signals at input A triggered with CLK1 and CLK2. Input B has signals triggered by CLK1. Both CLK1 and CLK2 are synchronous to each other. In such scenarios, arrival at pin A with each clock is checked with timing windows of B.

   Alignment that produces worst-case delay is used for analysis.

3. Both inputs have signals triggered by CLK1 and CLK2. Both CLK1 and CLK2 are synchronous to each other. Similar to above cases, arrival of A with CLK1 is checked with both timing windows of B (CLK1 and CLK2), and also arrival of A with CLK2 is checked with both timing windows of B. Alignment that produces worst-case is used.

4.11.6 MIS Delta Delay Reporting

Tempus performs simulation-based delay calculation for MIS. Multiple input switching is modeled by presenting switching inputs with equivalent current sources. Due to the nature of analysis, Tempus does not have delays without MIS impact, and thus, cannot determine MIS delta delay.

You can report MIS delta delay in timing reports (`report_timing` command) using one of the following commands:

```
set timing_report_fields {retime}
report_timing -format {retime_mis_delta}
```

Figure 4-112 MIS delta delay report

```
Path 1: VIOLATED Removal Check with Pin i20/CP
Endpoint:   i20/CDN (^) checked with  leading edge of 'clk'
Beginpoint: i1/Q                    (^) triggered by  leading edge of 'clk'
Path Groups: {async_default}
Retime Analysis { Data Path-Slew SI WP EWM AMIS(D) }
Other End Arrival Time         2.387900
+ Removal                      0.125900
+ Phase Shift                  0.000000
+ Uncertainty                  0.100000
= Required Time                2.613800
  Arrival Time                 2.002400
  Slack Time                   -0.611400
= Slack Time(original)         -0.660760
     Clock Rise Edge                    0.000000
     + Clock Network Latency (Prop)     0.514600
     = Beginpoint Arrival Time          0.514600
     ---------------------------------------------------------------------------
     Retime    Retime MIS  Arrival   Pin       Arc          Edge  Cell
     Delay     Delta       Time
     ---------------------------------------------------------------------------
     -         -           0.514600  i1/CP     CP ^         ^     -
     0.075700  0.000000    0.590300  i1/Q ->   CP ^ -> Q ^  ^     DFCNQOPTMAD8BWP12T35
     0.000200  0.000000    0.590500  i2/I      -            ^     BUFFD4BWP12T35
     0.045100  0.000000    0.635600  i2/Z      I ^ -> Z ^   ^     BUFFD4BWP12T35
     0.001400  0.000000    0.637000  i3/I      -            ^     CKBD1BWP12T35
     0.047400  0.000000    0.684400  i3/Z      I ^ -> Z ^   ^     CKBD1BWP12T35
     …
     …
     0.000000  0.000000    0.797000  i4/I      -            ^     BUFFXD2BWP12T35
     0.044400  0.000000    0.841400  i4/Z      I ^ -> Z ^   ^     BUFFXD2BWP12T35
     0.000500  0.000000    0.841900  i5/B2     -            ^     OAI222D2BWP12T35
     0.044800  -0.024800   0.886700  i5/ZN      B2 ^ -> ZN v v     OAI222D2BWP12T35
     0.000000  0.000000    0.886700  i6/C      -            v     AOI221D1BWP12T35
     ...
    ---------------------------------------------------------------------------
```

You can query on retime MIS delta delay using the `retime_mis_delta` property on timing point object of PBA path.

```
set path_col [report_timing -early -net -collection -retime path_slew_propagation
…]
foreach_in_collection each_tp [get_property $path_col timing_points] {
  puts "[get_property $each_tp hierarchical_name] \
        [get_property $each_tp retime_mis_delta]"
}
```

Output

```
  ...
  i5/B2 0.000000
  i5/ZN -0.024800
  i6/C 0.0000000
  i6/ZN 0.012600
  ...
```

Note: There is no change in reports when MIS is enabled.

4.11.7 Spice Validation Support and Results

The MIS analysis is supported for Spice-level validation. Spice analysis has to be done for 1-1/2 stages (like SI delay). The path-level Spice correlation may not be possible because side-input-switching times are fixed relative to the main input arrival in the Spice deck. Due to the differences in main input arrivals between Spice and STA, the side input alignment may drift away. The more the errors accumulate across the path, more misalignment happens in the Spice deck, causing the Spice analysis to become irrelevant for MIS. Due to this limitation in Spice analysis, MIS validation has to be done at the stage level.

You can use the following command to enable a MIS-based Spice deck:

```
write_spice_deck -enable_mis
```

Figure 4-113 Spice accuracy results

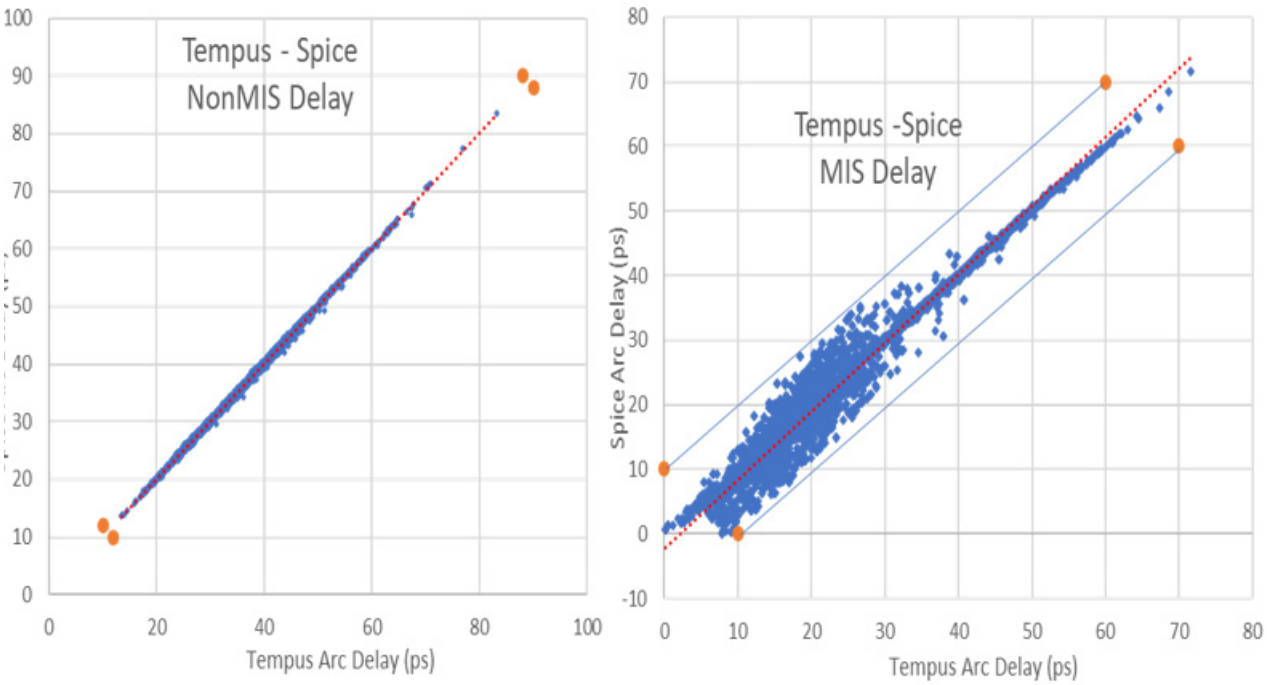
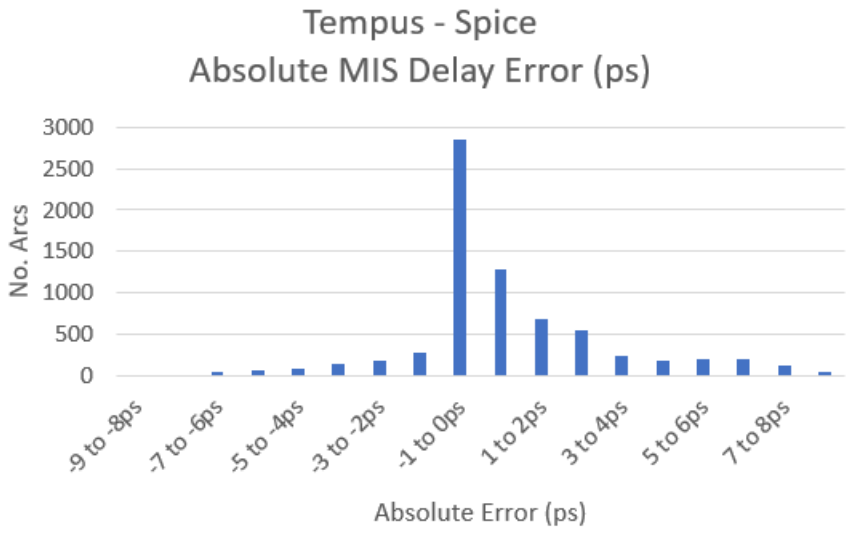


Figure 4-114 Negative error: Tempus more pessimistic than Spice



4.12 Derate-Based Advanced Multi-Input Switching (AMIS-Derate) Analysis

4.12.1 AMIS-Derate - Overview on page 266

4.12.2 AMIS-Derate Flow on page 266

4.12.3 AMIS-Derate Use-Model on page 268

4.12.4 Overlap Checking on page 268

4.12.5 Handling Signals from Different Clocks on page 271

4.12.6 AMIS-Derate Tuning Mechanism on page 272

4.12.7 MIS-Derate Reporting on page 273

4.12.1 AMIS-Derate - Overview

Tempus has the ability to model simultaneous switching inputs (SSI). However, this approach does not completely address inaccuracy in delay calculation results. Tempus supports the AMIS-Derate feature, which reduces complexity during timing analysis compared to AMIS-Simulation and provides accurate results. This feature also provides better debuggability and reporting. Unlike AMIS-Simulation, AMIS-Derate does not impact slew or SOCV.

The MIS derates are pre-characterized by Tempus for different slew/load conditions for each library cell that is prone to MIS impact.

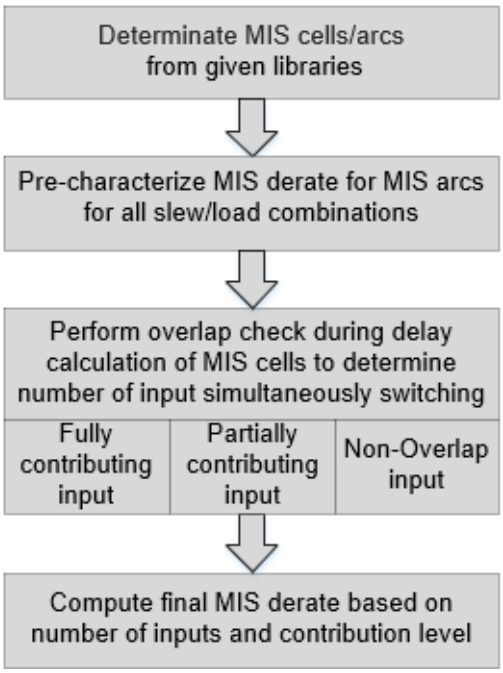
A few key points:

- MIS derate is computed for each library arc upfront for different slew/load combinations; same slew is used for all the inputs.
- MIS-derate is applied on the base delay (nominal/mean) values only.
- MIS-derate is not directly applied on variation components or cross-talk delays.
- Cross-talk delays might have a secondary impact due to MIS, in terms aggressor timing windows and victim arrival changes.

4.12.2 AMIS-Derate Flow

The AMIS-Derate flow diagram is as shown below:

Figure 4-115 AMIS Derate Flow



The AMIS Derate flow is explained below:

- Tempus uses the proprietary algorithm to determine the MIS library cells and arcs that are MIS prone.
- Tempus performs simulation (AMIS-Sim) to determine MIS-derate for each MIS library arc when all possible inputs are simultaneously switching at the same time and waveforms for different slew/load combinations (CCS table load/slew index) are formed.
- The MIS library cells/library arcs are tagged with the `has_mis_fall`/`has_mis_rise` property (using the `set_property` command).
- At the instance level, the software checks relative arrivals of all the inputs to determine simultaneous switching input - whether it is fully contributing, partial, or with no contribution.
- The software computes the final derate values based on the determined overlapping inputs.

4.12.3 AMIS-Derate Use-Model

Enabling Derate-Based MIS Analysis

To enable AMIS-derate, you can use the following command:

```
set_db timing_analysis_multi_input_switching_mode 3
```

When MIS-Derate is enabled, the software will automatically disable AMIS-Simulation.

The SSI feature can be enabled along with AMIS-Derate. However, AMIS-Derate will take precedence. For library cells that are not supported (or are user-disabled) by AMIS-Derate, the software will use SSI derates.

Enabling Library Cells

To enable (or disable) library cells, you can use the following command:

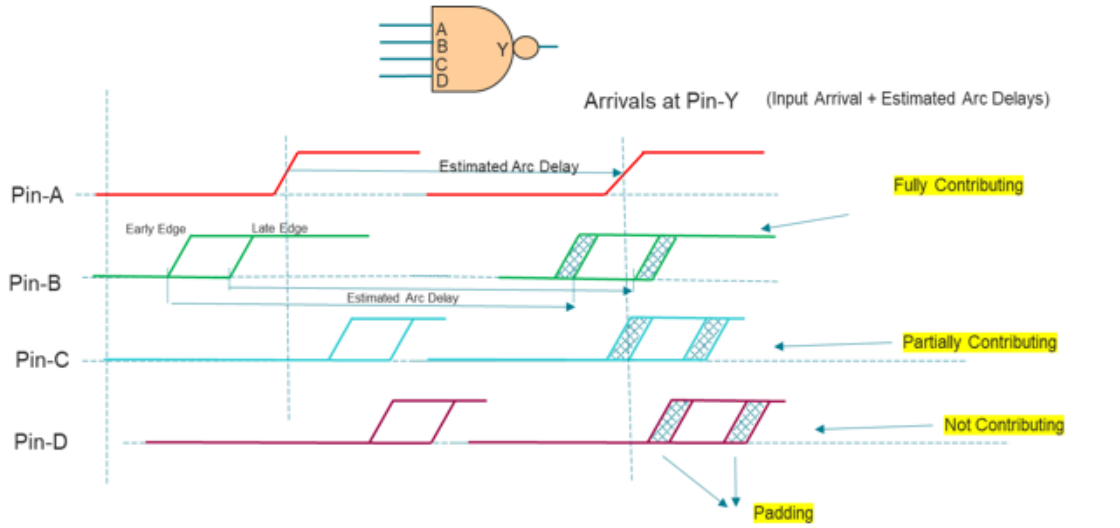
```
set multi input switching mode -disable_lib_cell string
```

Similar controls, which are used for MIS-simulation, can be used for MIS-derate as well.

4.12.4 Overlap Checking

AMIS derate overlap checking is described in the examples below.

Figure 4-116 Overlap check



Fully Contributed : If main input delay measurement point overlap with side input TW
Partially Contributed : If main input delay measurement point overlap with padded regions
Not Contributed : If main input delay measurement point does not overlap with TW or padded regions

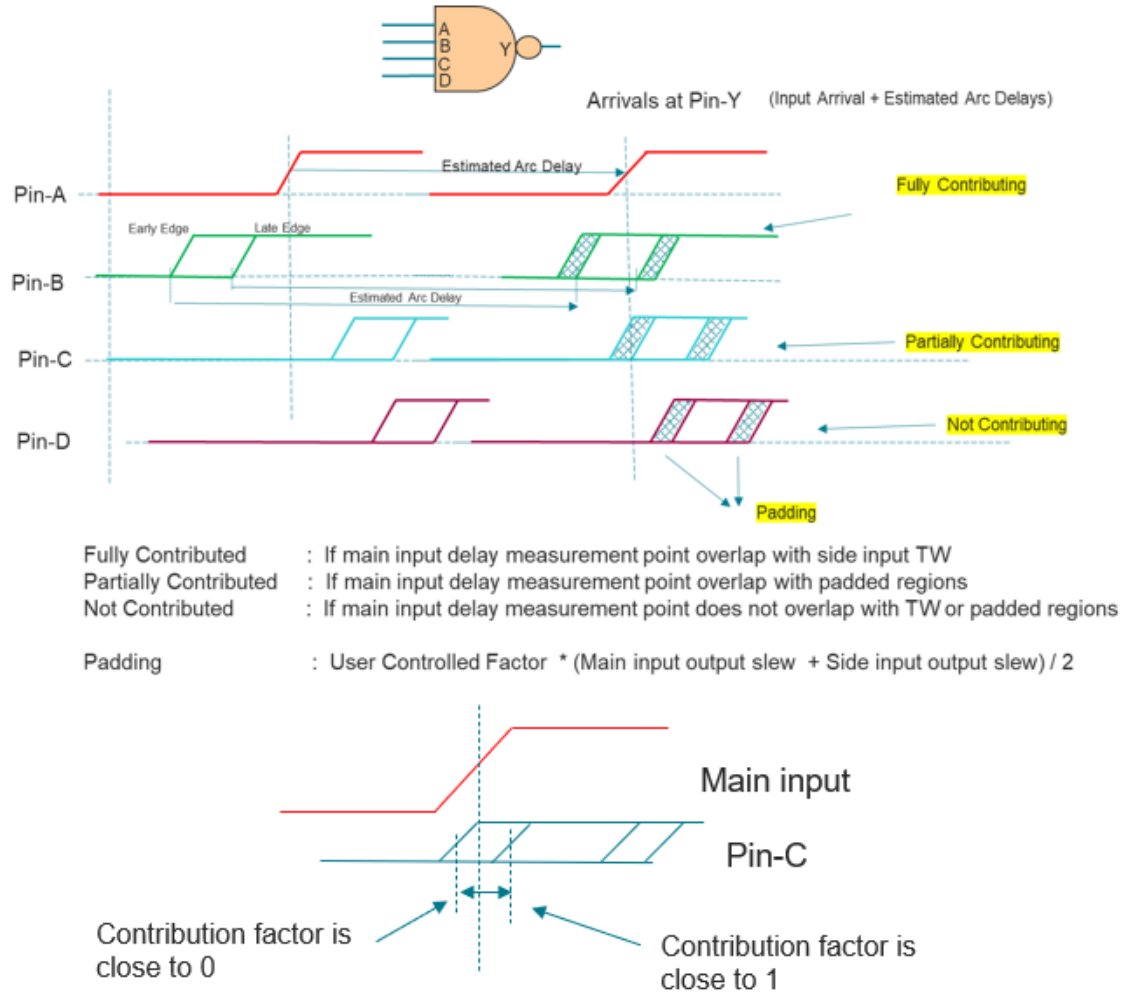
Padding : User Controlled Factor * (Main input output slew + Side input output slew) / 2

The main input and side inputs arrivals are propagated to the output pins by adding the corresponding NDLM delay of the arc. The overlap of main input with side input timing windows is checked at output pin. The side input timing windows are padded, which can be controlled using the formula shown in the diagram above.

If the main input is overlapped with the side input in the main region of timing window, then the input is considered fully contributing. If the main input is overlapped with side input in the padded area, then the input is considered partial contributing. If main input does not overlap in any region, then the input is not considered for MIS derate analysis.

Consider the following example.

Figure 4-117 Example



The pre-characterized derating factor computed for 4-input NAND for given slew/load: 0.25

The simultaneous switching factor is computed based on the overlap of side inputs:

- Main-input :1.0
- Pin-B (full contributing) : 1.0
- Pin-C (Partially contributing) : 0.5
- Pin-D (No contribution) :0.0

Simultaneous switching factor (SSF) = 2.5

For different values of SSF, the derate value will be calculated as shown below:

SSF = 4	Derate = 0.25
SSF = 1	Derate = 1
SSF = 2.5	The AMIS derate will be calculated based on the timing window overlap factor between the main and side inputs.

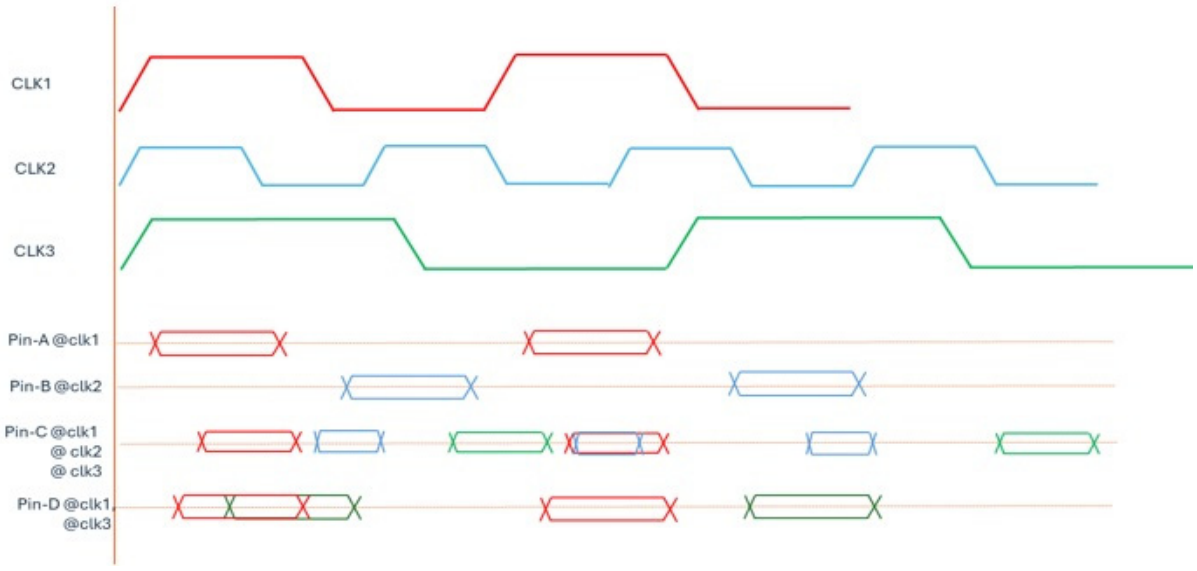
The partial contribution calculation will be as shown below:

- If main input overlap in the padded region, such as input contribution, is determined by linearly interpolating its contribution value.
- If overlap happens close to the timing window edge, contribution is close 1.
- If overlap happens close to the far end of padding region, contribution is close 0.
- Small differences in arrival may not result in big changes in final derate calculations.

4.12.5 Handling Signals from Different Clocks

The following example illustrates how AMIS-Derate handles signals from different clocks.

Figure 4-118 Clock-groups have no impact on AMIS-derate



In GBA Mode

- The overlap checks are performed on a per clock basis only. For example, if Pin-C is the main input, the timing windows will be based on clk1, clk2, and clk3.
- Pin-C, as main input, will consider all the inputs as attackers if it:
  - Overlaps with Pin-A with clk1 TW
  - Overlaps with Pin-B with clk2 TW
  - Overlaps with Pin-D with clk3 TW
- Pin-D as main input, will consider only Pin-A and C as attackers.
- Pin-B has timing window with clk2 and Pin-D does not have clk2 signal.

In PBA Mode

- The overlap checks are performed on a per clock basis only.
- Pin-A has data phase with clk1 only, thus consider checking pin C and D only for overlap check.
- Similarly, Pin-B has data phase with clk2, thus consider Pin-C only for overlap check.
- Pin-C has all the three clocks, thus check all the inputs as attackers.
  - Evaluate Pin-C and D as attackers based on clk1 TW
  - Evaluate Pin-B as attacker based on clk2 TW
  - Pin-C with clk3 TW does not overlap with Pin-C clk3 TW. Thus this combination is not considered
  - Worst-case factor among above is used for PBA

4.12.6 AMIS-Derate Tuning Mechanism

The AMIS-Derate computed final derate factor can be tuned for reducing pessimism. The tuning factor can be specified globally and applied for all the library cells. You can use the following command:

`set_multi_input_switching_mode -switching_alignment_factor`

The value of the `-switching_alignment_factor` can be specified between 0 to -1 described below:

- `0`: No tuning is performed. Full instance specific MIS derate is applied on the mean delay.
- Value moving from 0 towards -1: The MIS impact will be scaled down linearly.
- `-1`: The Instance specific MIS derate will be 1.

For example:

- If the final MIS derate computed for a cell arc is 0.75, The `-switching_alignment_factor` parameter is set to -0.5. Then the tuned derate factor is 0.75 - (1-0.75) * -0.5 = 0.875.
- If the `-switching_alignment_factor` parameter is set to -1, the tuned derate factor is 0.75 - (1-0.75) * -1 = 1.

AMIS-Derate Tuning Mechanism (Lib-Cell Specific)

The Lib-Cell specific tuning factor can be specified using the following properties:

```
set_interactive_constraint_modes [all_constraint_modes]
define_property -object_type lib_cell -type float\
           -mis_alignments_factor_min_rise \
           -lower_bound -1 -upper_bound 0
define_property -object_type lib_cell -type float\
           -mis_alignment_factor_min_fall \
           -lower_bound -1 -upper_bound 0
set_property [get_lib_cells ..] mis_alignments_factor_min_rise -0.5
set_property [get_lib_cells ..] mis_alignments_factor_min_fall -0.5
set_interactive_constraint_modes {}
```

4.12.7 MIS-Derate Reporting

The MIS-Derates can be reported using the following commands:

- `report_timing`
- `report_delay_calculation`

These are described below.

report_timing

The `report_timing` command supports the `retime_mis_derate` column (in PBA mode only) and shows the details in the timing report output.

For example, the following command generates MIS-derate output as shown below:

```
tempus > report_timing -early -from in2 -to_fall out  -net \
     -retime path_slew_propagation -format {instance arc cell retime_delay \
     retime_slew retime_mis_derate arrival}
------------------------------------------------------------------------------
Path 1: MET Early External Delay Assertion
Endpoint:   out (v) checked with  leading edge of 'CLK'
Beginpoint: in2 (v) triggered by  leading edge of 'CLK'
Path Groups: {CLK}
Retime Analysis { Data Path-Slew SI WP AMIS }
Other End Arrival Time          0.000
- External Delay                0.000
+ Phase Shift                   0.000
= Required Time                 0.000
  Arrival Time                  0.452
  Slack Time                    0.452
= Slack Time(original)          0.426
     Clock Rise Edge                      0.000
     + Input Delay                        0.010
     = Beginpoint Arrival Time            0.010
     ------------------------------------------------------------------------
     Instance  Arc          Cell              Retime  Retime  Retime   Arrival
                                              Delay   Slew    MIS      Time
                                                              Derate
     ------------------------------------------------------------------------
     -         in2 v        -                 -       0.001   -        0.010
     i0        -            U32B_NAND2X20     0.000   0.002   1.000    0.010
     i0        i1 v -> o ^  U32B_NAND2X20     0.004   0.006   0.740    0.015
     i1        -            E25B_CKINVX1      0.019   0.032   1.000    0.034
     i1        i ^ -> o v   E25B_CKINVX1      0.311   0.393   1.000    0.345
     i6        -            U32B_NAND2X20     0.036   0.394   1.000    0.381
     i6        i0 v -> o ^  U32B_NAND2X20     0.021   0.092   0.500    0.402
     i8        -            U32B_NAND4IX4     0.023   0.099   1.000    0.425
     i8        i2 ^ -> o v  U32B_NAND4IX4     0.027   0.042   0.750    0.452
     -         -            tst               0.000   0.042   1.000    0.452
      ------------------------------------------------------------------------
INFO: Path Based Analysis (PBA) performed on total '1' paths
```

report_delay_calculation

The command below shows the following report output:

```
report_delay_calculation -from i0/i1 -to i0/o -min -mis
-------------------------------------------------------------
From pin   : i0/i1
To Pin     : i0/o
Cell       : U32B_NAND2X20
Library    : lb2hpm_sc12v_ff025
Arc sense  : negative unate
Delay type : cell delay

Base or SI: SI delay
-------------------------------------------------------
RC Summary for net n3
-------------------------------------------------------
Number of capacitance : 3
Net capacitance       : 0.046090 pF
Total rise capacitance: 0.047103 pF
Total fall capacitance: 0.047090 pF
Number of resistance  : 2
Total resistance      : 2200.000000 Ohm
-------------------------------------------------------
                          Rise            Fall
-------------------------------------------------------
Input transition time  : 0.002300 ns     0.002300 ns
Cell delay             : 0.004000 ns     0.006300 ns
MIS Derate             : 0.741240        1.000000
Output transition time : 0.005700 ns     0.019100 ns
-------------------------------------------------------
Advanced MIS information
-------------------------------------------------------------------------
MIS enabled arcs
   Input_pin   Transition   Output_pin   Transition
-------------------------------------------------------------------------
    i1         fall         o            rise

Main input information
   Pin     Slew         Direction  Timing Window          Clock
```

```
-------------------------------------------------------------------------
   i1      0.005015 ns  fall       0.0103 ns : 0.0103 ns  CLK

Side inputs information
   Pin    Status  Slew         Direction  Timing Window          Clock
-------------------------------------------------------------------------
   i0     ACT     0.005015 ns  fall       0.0003 ns : 0.0003 ns  CLK

Overlap Alignment Mode (Estimated early arrival and slew at to_pin)

   From_pin   To_pin    Arrival      Slew
-------------------------------------------------------------------------
   i0         o         0.01131 ns   0.02394 ns
   i1         o         0.01226 ns   0.02323 ns
-------------------------------------------------------------------------
```

4.13 Voltage Threshold Skew Modeling and Analysis

4.13.1 VT Skew Analysis - Overview on page 278

4.13.2 Voltage Threshold Skew Analysis Flow on page 279

4.13.3 Enabling VT Skew Analysis on page 280

4.13.4 Partitioning Libraries by Voltage Threshold on page 280

4.13.5 Liberty Threshold Voltage Groups on page 281

4.13.6 VT Skew Reporting on page 281

4.13.1 VT Skew Analysis - Overview

Due to variations in the manufacturing process, the voltage threshold (Vth) of a given Vth class (e.g., SVT, LVT, ULVT) may also show variations, resulting in cells of that class operating either faster or slower than the nominal value.

- All cells of a given Vth class are expected to skew in the same direction (faster or slower than nominal) on a single die
- Since the fabrication of different Vth cell classes may be different, it is possible that different Vth classes will skew in different directions (fast vs. slow) on a single die
- Vth classes which share steps in the fabrication process may have correlated variation - they will tend to skew in the same direction on a given die

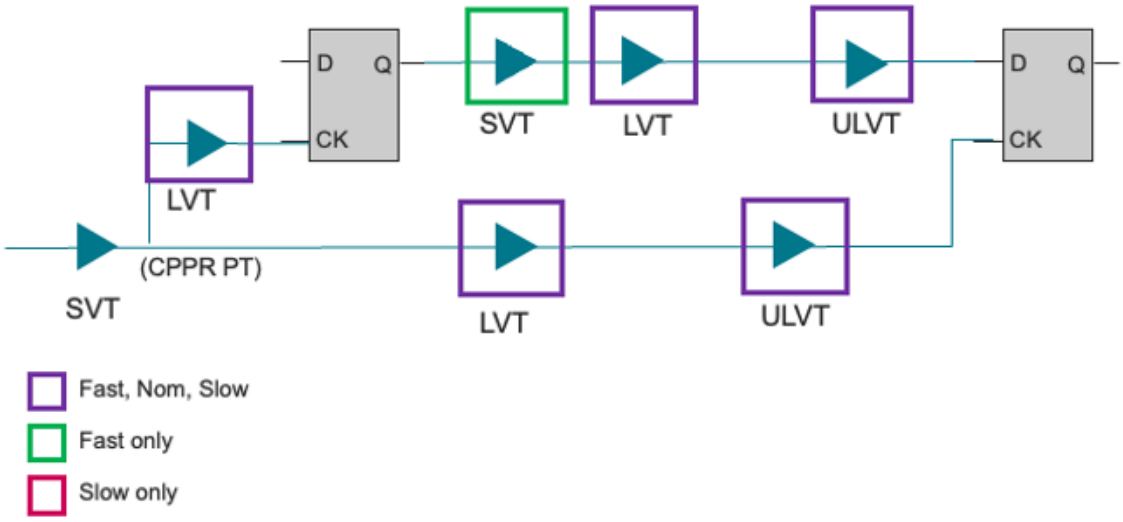
A robust timing analysis needs to consider the effects of Vth skew. In the past, accounting for Vth skew in the sign-off flow may have been handled by:

- Guard-banding the analysis by having all early timing paths assume the fastest timing of each Vth class, and all late paths use the slowest. This is a quick, but very pessimistic analysis. Any given Vth class cannot be simultaneously skewing both fast and slow.
- Script-based enumeration and analysis of all Vth permutations. This is accurate, but can be very time intensive.
- When using Tempus Vth skew analysis, you may need to adjust your existing settings to make sure to remove any existing guard-band or treatment for modeling Vth variation.

Tempus Vth skew analysis automates the process of selecting which Vth permutations need to be run to ensure a robust analysis - and optimizes these analyses to run quickly and efficiently.

In the following diagram, there are cells belonging to different Vth classes. Since cells from both LVT and ULVT are present in both the launch and data paths, and the capture clock path (downstream from the CPPR point) - nine different analysis are potentially required. For Hold analysis - SVT at fast, LVT at {slow nominal fast} and ULVT at {slow nominal fast}.

Figure 4-119 Vth skew analysis illustration



VT Skew Analysis Features

To support analysis of different VT skew combinations, the software provides the following features:

- Group timing libraries by user-specified Vth classes.
- Specify fast and slow derates for each Vth class.
- Customize the Vth skew analysis to focus only on the combinations that are relevant to the design.
- An efficient means under path-based analysis (PBA) of timing each potential Vth skew combination, and reporting the path and slack corresponding to the most pessimistic combination.
- Enable analysis with and without Vth skew consideration for in-session comparisons.

4.13.2 Voltage Threshold Skew Analysis Flow

To perform Vth skew analysis, you need to configure the following settings:

- Group timing libraries into different Vth classes (e.g., HVT, LVT, ULVT). This can be done using pre-existing attributes in the Liberty timing library, or by providing explicit grouping information using the `set_threshold_voltage_group` command.

- Use the set_vt_skew_derate command to specify fast and slow derates for each defined Vth class - for specific threshold voltage groups.
- Use the set_vt_skew_config command to customize the combinations of Vth classes to be analyzed. By default, the software will analyze all the possible combinations.
- Configure report_timing command settings to generate graph- and path-based VT skew reports.

The Vth skew analysis is performed during path-based analysis (PBA). Path-based analysis is a re-timing of the set of paths produced by graph-based analysis (GBA). The more paths that are provided from GBA, the more analysis is done in PBA mode. The Vt skew analysis in GBA mode can be run in the following ways:

- The Vt skew derates are applied in a worst-case manner in the GBA part of the flow so that early paths are sped up maximally and late paths are slowed down maximally. While this is bounding, it is also very pessimistic and may result in an unrealistic number of paths being selected for Vt skew analysis.
- The GBA step in path-based Vt skew analysis can be optimized for paths with unique Vt's, thus reducing the pessimism in GBA analysis. A path has a unique Vt if there is only one Vt group present on both the launch and capture paths. The paths with a unique Vt are quite common so this approach can be used to significantly speed up the analysis.
- PBA Vt skew analysis can use any of the available retime modes: path, exhaustive, or bounded exhaustive.

4.13.3 Enabling VT Skew Analysis

To enable or disable the VT skew analysis, you can use the setting:

```
set_db timing_enable_vtskew_derate_mode true
```

4.13.4 Partitioning Libraries by Voltage Threshold

The Liberty timing libraries may already include threshold voltage group definitions. These existing definitions may be used with the Vth skew analysis flow. If native group definitions are missing or if you want to have a different group name to use across different libraries, you can use the set_threshold_voltage_group command.

4.13.5 Liberty Threshold Voltage Groups

The Liberty standard allows a default voltage group to be defined at the library level and/or a library-cell specific group definition to be assigned. These are explained below.

Creating Threshold Voltage Groups

The `set_threshold_voltage_group` command can be used to define (or redefine) the threshold voltage group associated with the given timing library cells. You can group entire libraries into a single group, and since Liberty allows per library cell specifications - this command accommodates that as well.

Using Tcl-Based Query for Library Cell Voltage Thresholds

The threshold group property assigned to the library cells can be queried using the `threshold_voltage_group` property on library_cell object (see `get_property` command).

To find all the library cells of a given VT class, an example script is shown below:

```
# Add lib_cell's to a new LVT voltage group
> set_threshold_voltage_group
     -name LVT
     -lib_cells  [get_lib_cells *_lvt*/*  ]
# Stylus query
> get_db lib_cells -if {.threshold_voltage_group == LVT}
lib_cell:default_emulate_libset_min/stdcell_lvt_1p62V_cworst/BUFX1_L .
```

4.13.6 VT Skew Reporting

Unique VT Skew Mode Configuration

A path is defined as having a unique Vth if all of the clock cells present in the given path and downstream from the CPPR common point are of the same Vt class. By default, all the fast, slow, and nominal permutations of the Vth derate will be analyzed. You can specify only a single Vth derate to use for the unique Vth class on these paths. In several designs, unique Vth paths are common so reducing the amount of analysis can help performance considerably.

Using Unique Vth mode in GBA Mode

The GBA unique Vth mode improves performance considerably by reducing the initial GBA pessimism caused by using the Vth derating extremes.

You can configure the Vth derate for the unique Vt group (to use during GBA and PBA) using the following command:

```
set_vt_skew_config -unique_vt_mode slow
```

With the above settings GBA analysis will be run using the Vth slow derate value on both the launch and capture clock paths for all the Vt groups. PBA will also use the specified unique Vt derate mode only if there is a unique Vt group.

When the GBA unique Vt mode is configured, the `report_timing` command can be run with a `-max_slack` value greater than 0 to add additional pessimism.

report_timing Output

The `report_timing` command provides additional information for the Vth skew analysis output:

- A retime Vt-Skew analysis entry in the timing report header indicates the Vth combination that may have resulted in the PBA path with the least slack.
- The actual Vth skew derate applied to a given instance can be shown in the detailed path report by adding the `vt_skew_derate` field to the format. For example:

  ```
  timing_point cell vt_group vt_skew_derate total_derate delay
  arrivalset_db timing_report_fields {timing_point cell vt_group
  vt_skew_derate total_derate delay arrival}
  ```

- A summary of the slacks from all Vth combinations analyzed can be presented in a table at the end of the timing report for debugging purposes by using the `report_timing -debug vt_skew` command. For example:

  ```
  report_timing -from EX4_RS -to EX4_RT -retime path_slew_propagation
        \ -debug vt_skew
  ```

  For performance reasons, this parameter is not recommended when generating a large number of paths.

An example timing report is given below:

```
Path 1: MET Setup Check with Pin EX4_RT/CK
Endpoint:   EX4_RT/D (v) checked with  leading edge of 'PH1'
Beginpoint: EX4_RS/Q (v) triggered by  leading edge of 'PH1'
Retime Analysis { Data Path-Slew Vt-Skew  }
Retime Vt-Skew Analysis { ELVT@slow LVT@nom SVT@slow ULVT@slow }
Other End Arrival Time          0.241
```

```
- Setup                          0.296
+ Phase Shift                   10.000
= Required Time                  9.945
- Arrival Time                   1.031
= Slack Time                     8.913
= Slack Time(original)           8.868
-------------------------------------------------------------------
Timing Point   Cell       Vt-Skew    Total     Delay   Arrival
                          Derate     Derate             Time
-------------------------------------------------------------------
clk            -          -          -         -       0.000
EX4_C0/Y       BUFX1_L    1.000      1.000     0.128   0.115
...
EX4_RT/D ->    DFFHQX1_S  1.000      1.000     0.000   1.031
-------------------------------------------------------------------
-------------------------------------------------------------------
VT Skew Combinations Slack Summary
-------------------------------------------------------------------
ELVT@slow LVT@nom   SVT@slow  ULVT@slow  : 8.913
ELVT@slow LVT@slow  SVT@slow  ULVT@slow  : 8.913

-------------------------------------------------------------------
```

4.14 Design Rule Violation (DRV) Reporting

4.14.1 Design Rule Violation (DRV) - Overview on page 285

4.14.2 DRV Violation Reports on page 285

4.14.3 Enabling DRV Reporting on page 286

4.14.4 Limiting DRV Violations on page 288

4.14.5 Flow Support for Configurable DRV on page 288

4.14.1 Design Rule Violation (DRV) - Overview

Tempus performs design rule violation (DRV) checks to ensure that netlists meet the design rule constraints specified in the design library. You can also specify design rule limits using the Tempus commands.

The design rule violations (DRV) reporting includes the following checks:

- Minimum and maximum limit violation for transition time
- Minimum and maximum limit violation for capacitance
- Minimum and maximum limit violation for fanout

By default, the smallest limit value between a user-specified constraint and library design rule constraint is used for DRV checks. You can configure Tempus to always consider the user-specified constraint for DRV checks, by using the `-override` parameter with commands - `set_max_transition`, `set_min_transition`, `set_max_capacitance`, `set_min_capacitance`, `set_max_fanout` and `set_min_fanout`.

4.14.2 DRV Violation Reports

To report the worst DRV violation in a design for every DRV check, use the following command:

`report_constraint -drv_violation_type {`*`drv_check_list`*`}`

To report all the instances of DRV violations in a design, use the following command:

```
report_constraint -drv_violation_type {drv_check_list} -all_violators
```

The DRV check list includes the following class of DRV checks:

- `max_transition`
- `min_transition`
- `max_capacitance`
- `min_capacitance`
- `max_fanout`
- `min_fanout`
- `pulse_clock_max_transition`
- `pulse_clock_min_transition`

To perform more than one class of DRV checks, these can be used in a Tcl list as shown in the following example:

```
report_constraint -drv_violation_type {max_transition max_capacitance}
         \-all_violators
```

This command reports all the `max_transition` and `max_capacitance` DRV violations.

Default DRV Output Format

The default DRV report shows a fixed number of fields in a sequence, as shown in the following example:

```
tempus> report_constraint -drv_violation_type max_transition -all_violators
Check type : max_transition
---------------------------
-------------------------------------------------------------------------
Pin Name                            Required        Actual           Slack
-------------------------------------------------------------------------
TDSP_CORE_INST0/arp_reg/CK          0.838           13.514           -12.676
TDSP_CORE_INST0/read_prog_reg/CK    0.838           13.514           -12.675
DMA_INST/write_reg/CK 0.838         5.638           -4.800
                            ….
```

However, you can combine a set of reporting fields in any sequence and generate a custom DRV report in the backdrop of relevant context information (described in the next section). These reporting fields include information, such as driver-receiver of nets, timing slack at endpoints, net parasitics among others.

4.14.3 Enabling DRV Reporting

To enable configurable DRV reporting, you can use the `timing_report_constraint_format` attribute. For example,

```
set_db timing_report_constraint_format {drv_check_list}
report_constraint -drv_violation_type {drv_check_list} \
        -all_violators | pin_or_port_list
```

You can also use the following command to specify DRV fields:

```
report_constraint -drv_violation_type {drv_check_list} -drv_fields
{field_list}
```

For a complete list of DRV fields and their descriptions, refer to the `report_constraint` command in the *Tempus Text Command Reference*.

The field list specified with the `-drv_fields` parameter has higher precedence and will override the field list specified with the `timing_report_constraint_format` attribute. If DRV fields are not specified with this variable (or with the `report_constraint -drv_fields` command), then the default set of DRV fields will be displayed.

The resulting DRV report will be a single-line multi-column report, as shown in the following example:

```
report_constraint -all_violators -drv_violation_type max_transition -drv_fields
{transition_slack max_transition net_fanout  ref_cell pin }
# format : frame 0 : split 1
#Units: Time (1ns).
Check type : max_transition
--------------------------------------------------------------------------------
Transition Slack Required    Net Fanout   Violating      Pin
                 Transition               Libcell
--------------------------------------------------------------------------------
-1.030           2.300        1            NAND2XLMTR    TEST_CNTRL_INST/g118/Y
-0.962           1.770        1            NAND2XLMTR    TEST_CNTRL_INST/g112/B
-0.961           1.770        1            NAND3XLMTR    TEST_CNTRL_INST/g114/Y
-0.454           1.770        2            NAND3XLMTR    TEST_CNTRL_INST/g114/A
-0.454           1.770        2            NAND2XLMTR    TEST_CNTRL_INST/g55/A
-0.453           1.770        2            NOR2BXLMTR    DVFS_CNTRL/g638/Y
-0.253           1.770       18            AOI22XLMTR    DVFS_CNTRL/g645/A1
-0.113           0.838        7            TLATNTSCAX12MTR
```

By default, the `timing_report_constraint_format` attribute is an empty list. If the variable settings are overridden with a set of valid field names, the default DRV multi-line worst DRV report will also switch to the new single-line multi-column DRV reporting style. To revert to the default multi-line worst DRV report, you can reset the `timing_report_constraint_format` attribute to an empty list.

Tempus will ignore any field that is not relevant for the specified class of DRV check(s). For example, the `max_capacitance` and `capacitance_slack` fields are not relevant for transition DRV check and will be ignored even if you include these fields for reporting.

You can save the DRV violation report by exporting the `report_constraint` command output as space-separated text format (default) or as a comma separated value (`.csv`) format file by using the `-drv_output_format csv` option, as shown in the following example:

```
report_constraint -all_violators -drv_output_format csv \
         -drv_violation_type {max_transition} \
         -drv_fields {pin ref_cell transition_slack max_transition}\
         > drv_all_violations.csv
```

Note that `-drv_output_format csv` can be specified only with the `-drv_violation_type` parameter.

The hierarchical pin and net names might be long strings. To make a report visually compact, it is recommended to use `set_table_style -no_frame -nosplit` command and precede the DRV fields, such as, pin, driver_pin, receiver_pin, and net with other field values, which are either numerical values or shorter strings.

4.14.4 Limiting DRV Violations

The configurable DRV violation report provides additional options to zoom into specific design areas for DRV violations. To limit violations, you can use the following `report_constraint` parameters:

- When multiple leaf pins connected to a single net fails DRV check(s), Tempus reports all the failing pins as separate entries in the violation report.

  To report only the worst violating pin attached to a net, you can run a configurable DRV check along the `-worst_drv_per_net` parameter. This parameter will ensure that only the pin with the worst DRV slack is reported per net.

- If there are a large number of DRV violations in a design, the software will take longer to process the checks and might be hard to verify. To restrict the number of violations for each class of DRV checks you can run the configurable DRV check along the `-max_pins_per_drv` parameter.

- To restrict a DRV check to a clock or data network, you can use the `report_constraint -clock` or `-data` parameter, respectively.

Note: The configurable DRV does not support cell EM check in the current software version.

4.14.5 Flow Support for Configurable DRV

In the case of CMMMC, the worst setup and hold timing slacks correspond to the view in which the DRV violation occurs for a leaf pin.

To report the worst setup or hold timing slack across all active CMMMC views, set the `timing_report_drv_use_worst_timing_slack` attribute to `true`. If the worst setup or hold slack is found in a different view, the view name will also be annotated with setup and hold slack values.

In DSTA flow, all the fields and command parameters are supported, except for the following:

- `report_constraint -worst_drv_per_net` parameter

- `transition_si` field denoting the crosstalk delta transition
- `max_slack` field denoting the worst setup slack
- `min_slack` field denoting the worst hold slack

In DMMMC, the configurable DRV report supports all the options and reporting fields on the clients. However, it does not support merging of the configurable DRV report on the master through `merge_timing_reports` command.

4.15 Cell EM Analysis and ECO

4.15.1 Cell EM Analysis and ECO - Overview on page 291

4.15.2 Cell EM STA Flow on page 293

4.15.3 Required Inputs for Cell EM Analysis on page 294

4.15.4 Reports on page 302

4.15.5 ECO Flow Use Model on page 310

4.15.6 Save/Restore of Design Database on page 311

4.15.7 Hierarchical STA - Cell EM on page 311

- Cell EM Context-Based Flow on page 311
- Cell EM Boundary Model Flow on page 313

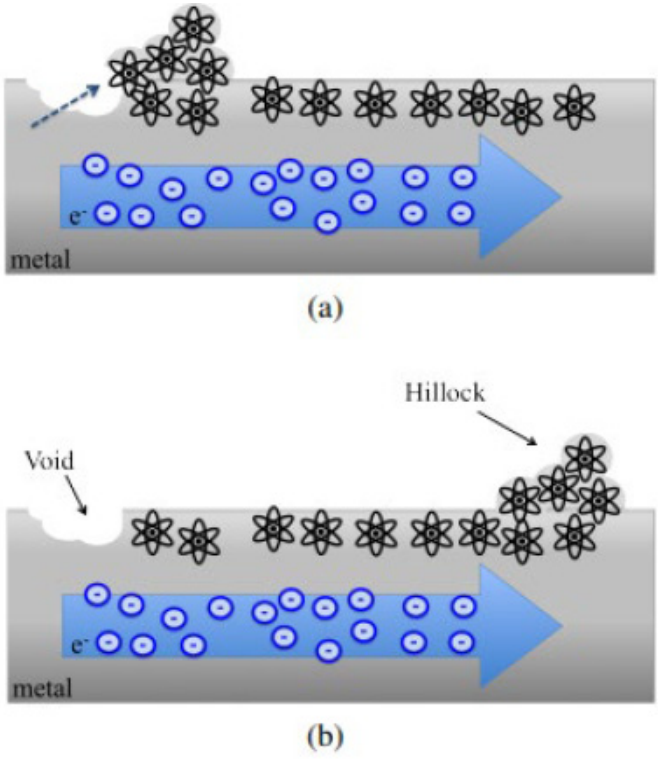
4.15.8 Current Limitations and Recommendations on page 314

4.15.1 Cell EM Analysis and ECO - Overview

The reliability of silicon is addressed by using a variety of design measures, which include the type of material used. As the structural dimensions of electronic devices and their interconnects are downscaled, the factors that reduce reliability need to be considered. In addition, the material migration process that occurs in electrical devices and interconnects has a significant impact during the IC design and development phase.

The material migration, also known as electron migration, is a term that describes various forced material transport processes in solid bodies. When high density current passes through a thin metal wire, the high energy electrons exert forces on atoms causing them to migrate. The process of electromigration can drastically reduce the lifetime of a chip by increasing the resistivity of a metal wire or creating a short circuit between adjacent lines.

The effect of electromigration is illustrated in the figure below.



This chapter explains the Cell electromigration (EM) constructs that are used in Tempus Static Timing Analysis (STA) and the ECO flow, which can help IC designers to analyze a design, prevent violations, and prepare a robust design having higher reliability.

The Tempus software mainly covers the cell-level EM analysis in STA and fixing as part of the timing ECO flow. Both Cell EM analysis and ECO provide a method to fix the standard cell

level electromigration violations. The standard cell level electromigration violations can cause degradation in cell reliability, if the frequency is higher than the pre-defined limit in the .lib file.

To determine the impact of Cell electromigration in STA, the Tempus software considers the following two parameters:

- Maximum capacitance limit of a pin
- Toggle rate at which a cell operates and input slew

This document assumes that the foundry or IP provider may supply the .lib files containing electromigration constructs for standard cells (refer Required Inputs for Cell EM Analysis on page 294). The scope of EM analysis in the context of STA is related to the standard cells for which appropriate constructs are defined or are available in the .lib file. Also, the designers can configure appropriate toggle rates and define activities in Tempus to simulate the possible behavior of a design in the real world. Using these parameters, Tempus can perform analysis, and Tempus ECO can fix cell level EM violations.

Terms Used in this Chapter

Switching Activity (SA)

The `set_switching_activity` command `-activity` parameter specifies the number of times the net, pin, or port switches in a clock cycle.

If the activity is set to 0.5, the switching activity happens once for every two clock cycles. If the activity is set to 2.0, the switching activity happens two times per clock cycle.

Transition Density/Toggle Count/Toggle Rate (TD)

Transition density specifies the number of transitions per second.

```
TD = (number of toggles)*(clock-frequency)
```

max_transition

`set_max_transition_per_freq` `/1D` library Freq Slew Table.

`max_transition` values in the timing libraries.

`set_max_transition` in SDC

max_capacitance

`max_cap` constraints from SDC

`max_cap` limit from .lib file

`set_max_cap_per_freq`/1D library Freq Cap Table

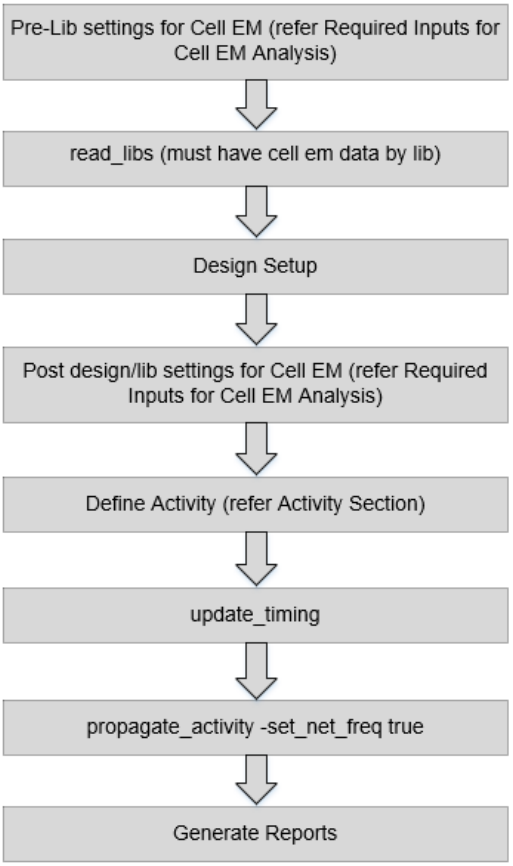
Transition Frequency (FR)

Freq = TD/2

4.15.2 Cell EM STA Flow

The following diagram shows the Cell EM STA flow:

Figure 4-120 Cell EM STA Flow



4.15.3 Required Inputs for Cell EM Analysis

This section describes the various inputs for the Cell EM analysis:

- Define Cell EM View
- Library Requirements
- User-Defined EM Constraints
- Define Activity
- Derating Frequency
- Activity Propagation
- STA Use Model

Define Cell EM View

The `set_analysis_view -dynamic <view_name>` parameter is used to control the EM-specific views for analysis and ECO. If this parameter is not defined, the first setup view defined with `set_analysis_view` command becomes the EM view for deriving the activity definition based on the below settings:

`set_db timing_analysis_check_type (Default: setup)`

If both the `-setup` and `-hold` parameters are defined with the `set_analysis_view` command, then Tempus picks the setup view for activity.

Library Requirements

The cell EM data (electromigration group or frequency-based max_cap table) should be present within the standard cell library.

- **Maximum Capacitance Group**

The maximum capacitance group contains capacitance values with respect to various Transition Frequency (FR) over the output pin of cell and input slews over related pin. The unit of Frequency is inverse of the library time unit.

In the max_cap table example below, the `index_1` and `index_2` values are written in the library header:

```
max_cap (maxcap_temp1ate_7x7) {
   re1ated_pin : "i";
```

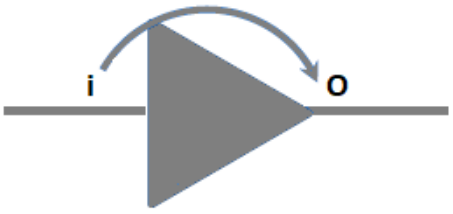
```
   index_1 ("0.0612105, 0.344434, 1.79586, 4.46565, 4.46575, 4.46706, 4.48205") ;
   index_2 ("0.002, 0.00537075, 0.0144225, 0.0387298, 0.104004, 0.27929, 0.75") ;
   values ( \
      "3.58855, 3.58855, 3.58855, 3.58855, 3.58612, 3.58022, 3.56612", \
      "0.670169, 0.664944, 0.652355, 0.624836, 0.622707, 0.60305, 0", \
      "0.141401, 0.138162, 0.129487, 0.108796, 0.0628273, 0, 0", \
      "0.0713813, 0.0704513, 0.0652987, 0.000574319, 0, 0, 0", \
      "0.0713797, 0.0704497, 0.0652969, 0.0001, 0, 0, 0 ", \
      "0.0713608, 0.0704304, 0.0652752, 0, 0, 0, 0", \
      "0.0711427, 0.0702078, 0.0650252, 0, 0, 0, 0" \
   ) ;
}
```

The following example shows a Lut table for max_cap group:

```
maxcap_1ut_temp1ate (maxcap_temp1ate_7x7) {
    variab1e_1 : frequency ;
    variab1e_2 : input_transition_time;
    index_1 ("1, 2, 3, 4, 5, 6, 7") ;
    index_2 ("0.002, 0.00537075, 0.0144225, 0.0387298, 0.104004, 0.27929, 0.75");
}
```

For example, for a buffer cell, it can be defined as follows:

Figure 4-121 Buffer Cell



In the above example, if frequency at pin “O” is 0.344434 GHz (FR=TD/2) and the slew value at the pin “i” is 0.00537075* ns, the maximum capacitance limit on pin “O” will be 0.664944* pF.

If the max_cap construct is not present, Tempus will use reverse lookup from the electromigration table.

■ Electromigration Group

The EM toggle rate consists of three types of current-based toggle rate tables:

- Average: Average current
- Peak: Peak current
- RMS: Root mean square current

For definitions of all types of currents and their equations, refer to the “Voltus EM Analysis” section in the *Voltus User Guide*.

Example 1

In the example below, the `index_1` and `index_2` values are written in the library header.

```
electromigration () {
   related_pin : "i 0";
   em_max_toggle_rate (em_temp1ate_7x7)
     current_type : "average" ;
     index_1 ("0.002, 0.00537075, 0.0144225, 0.0387298, 0.104004, 0.27929, 0.75") ;
     index_2 ("0.0001, 0.000362139, 0.00131145, 0.00474927, 0.017199, 0.0622842,
0. 225556 ") ;
     va1ues ( \
        "109.147, 106.267, 96.5867, 34.8601, 10.4366, 2.94733, 0.818873", \
        "103.488, 101.484, 96.711, 34.8472, 10.4201, 2.94551, 0.818521", \
        "80.2824, 80.3635, 79.7469, 34.6124, 10.3884, 2.94171, 0.818393", \
        "45.2761, 45.3841, 45.7716, 33.2348, 10.3482, 2.93614, 0.817915", \
        "19.6274, 19.6508, 19.7604, 20.2257, 10.2735, 2.92694, 0.816591", \
        "7.64072, 7.64348, 7.65755, 7.73124, 8.02434, 2.91884, 0.814551", \
        "2.87723, 2.87747, 2.87836, 2.88685, 2.92963, 2.61789, 0.812657" \
   ];
}
em_max_togg1e_rate (em_temp1ate_7x7) {
     current_type : "peak";
     index_l ("0.002, 0.00537075, 0.0144225, 0.0387298, 0.104004, 0.27929, 0.75");
     index_2 ("0.0001, 0.000362139, 0.00131145, 0.00474927, 0.017199, 0.0622842,
0.225556 ");
     va1ues ( \
        "4583.58, 4375.53, 3859.22, 1069.53, 289.815, 79.61, 12.7996", \
        "4334.35, 4363.88, 3787.44, 1070.82, 289.099, 79.5119, 12.7994", \
        "2749.18, 2760.16, 2785.69, 1086.47, 292.398, 79.825, 12 .8068", \
        "1308.28, 1315.43, 1324.62, 1080.56, 307.583, 80.9342, 12.8356", \
        "535.263, 542.185, 543.267, 547 .118, 319 .488, 85.2959, 12.9186", \
        "209.524, 210.654,211.149, 211 .451, 213.715, 91.1989, 11.8587" \
   ];
```

```
}
em_max_toggle_rate (em_template_7x7) {
     current_ type : "rms ";
     index_1 ("0.002, 0.00537075, 0 .0144225, 0.0387298, 0.104004, 0.27929, 0.75");
     index_2 ("0.0001, 0.000362139, 0.00131145, 0.00474927, 0.017199, 0.0622842,
0.225556 ") ;
     values ( \
) ;
        "1850.77, 1823.6, 1457.41, 364.993, 93.8955, 25.316, 6.94221", \
        "1892.52, 1876.97, 1464.18, 366.111 , 93.9605, 25.3202, 6.94236 ", \
        "1663.63, 1691.62,1494.75, 373.998, 94.6712, 25.3759, 6.94677 ", \
        "1081.37, 1090.89, 1120, 396.894, 97.1714, 25.574, 6.96208", \
        "527.49, 529. 404, 537.076, 429.715, 104.015, 26.1671, 7.00718", \
        "218.094, 218.32, 219.45, 224 .495, 116.875, 27.792, 7.13732 ", \
        "83.7114, 83.7487, 83.8333, 84.5024, 87.4348, 31.2242, 7.4894" \
   ];
  }
}
```

The following example shows a Lut table for the EM group:

```
em_lut_template (em_template_7x7) {
   variable_1 : input_transition_time;
   variable_2 : total_output_net_capacitance;
   index_1 ("0.002, 0.00537075, 0.0144225, 0.0387298, 0.104004, 0.27929, 0.75 ");
   index_2 ("0.0001, 0.00352928, 0.00882319, 0.022058, 0.055145, 0.137862,
0.344656 ");
```

The table index is used for input transition at the related pin, and output load exists at the current pin for which the electromigration table is created. The table is used to identify the maximum transition density limit for a design under various current types.

Refer to the Reports section on how to interpret these values.

If the `max_cap` table is not available and includes only electromigration, then the software performs reverse look up table. See figure Buffer Cell.

If the following conditions are met:

- Transition Frequency at pin “O” is 2* Ghz (F_R) and,
- Slew Value at the pin “i” is 0.00537075* ns,

The software will follow the steps given below:

1. For electromigration table, Tempus operates on Toggle Count/Transition Density (T_D). Here T_D = 4000000000 (2*F_R)
2. For slew **0.00537075**, Tempus selects the entire range of capacitance and corresponding Toggle Count. See Example 1 on page 296.
   - T_D range: 103.488, 101.484, 96.711, 34.8472, **10.4201**, **2.94551**, 0.818521
   - Cap range: 0.0001, 0.000362139, 0.00131145, 0.00474927, **0.017199**, **0.0622842**, 0.225556
3. T_D in library is written with unit of GHz (inverse of time unit), the `max_cap` value will be interpolated between **0.017199**, **0.0622842**.

Note: When both the `max_cap` and electromigration tables are used, the software gives preference to the `max_cap` table.

User-Defined EM Constraints

If there are no EM .libs available, you can define both `max_cap` and `max_trans` based on the frequency (toggle rate) constraint with the following commands:

- `set_max_cap_per_freq`
- `set_max_transition_per_freq`

Note: The `report_freq_violation` command does not work with the `set_max_cap_per_freq` and `set_max_transition_per_freq` commands use model because there is no lookup table data available.

Define Activity

The activity can be defined using the `set_default_switching_activity` or `set_switching_activity` command, or by using the dynamic switching activity files, such as, VCD, TCF, FSDB, PHY, or SAIF.

The defined activity follows the below priority order:

1. `set_switching_activity`
2. `read_activity_file`
3. `sdc_constant`
4. `set_default_switching_activity`
5. `propagate_activity`

The blackbox, sequential gates, and clock gates require special handling in Cell EM analysis for defining activity. The following default activities are a must for Cell EM analysis.

```
set_default_switching_activity -black_box_density 0
       -black_box_duty 0 -sequential_activity 0.2 -clock_gates_output 2.0
```

Note: The `set_switching_activity` command is set to density/duty on a particular pin or net, whereas the `set_default_switching_activity` command is set the default activity, `-input_activity`, or `-global_activity`, which is not for a particular net or pin.

Default Activity

If none of the activity is defined for pins or ports, Tempus uses the default values of activity. Refer to the `set_default_switching_activity` command for more details on activity defaults.

Default Period

If any pin or port is not having clocks or if there is any pin or port that has been kept open, Tempus uses the default period/frequency for defining transition density (T_D) over the pin/port. If you have not defined any default period, the software decides the default period based on the clock reaching the maximum number of cells in a design.

Note: You must use `set_default_switching_activity -period` parameter to define the default period.

After propagation activity, if any net with 0 toggling is present in the design either by annotation or by defined activity, except constant, the net can be directly assigned to the Toggle Rate (T_D) using the following command:

```
set_db timing_report_default_frequency_for_unconstrained_nets 2 (default :1e9)
```

Tempus calculates the Toggle Count/Transition density = 2 *F_R, in this example it will be 4; and default will be 2e9.

Derating Frequency

You can derate the required frequency for a pin using the `set_design_rule_derate` command. The value of derate gets multiplied to the required frequency (F_R) for a library pin. This command is used to derate the frequency:

The `set_design_rule_derate` command can also be run on an interactive shell after using the `update_timing` command.

For details on the effect of the `set_design_rule_derate` command and its usage, refer to the Reports section (Example 6).

Activity Propagation

The propagate activity is a required step to complete the cell EM Flow. Tempus will propagate activity based on the duty and activity defined for pins/port/nets using cell Boolean function.

The following command is used to run the propagate activity:

```
propagate_activity -set_net_frequency true
```

To start the run, the `propagate_activity` command requires the `update_timing` session. If the session is not timing updated, the command automatically runs the `update_timing` command and then executes `propagate_activity`.

Note:

- The `set_switching_activity` command should be placed before `propagate_activity`.
- The `propagate_activity` command must be invoked after `update_timing`.

Or,

- The `propagate_activity` command should be replaced with `update_timing`.
- Using the activity definition and propagation, the toggle rate at each pin will be calculated. This toggle rate is then used for maximum capacitance and frequency violation reporting.
- The propagate activity is an essential step, which is also supported in DSTA.

STA Use Model

To identify cell-level EM violations, you need to configure the required attributes before running any reporting commands. The following section covers Tempus-specific variables that are essential for reporting cell EM violations.

By default, Tempus supports reading electromigration constructs from the .lib file so you are not required to specify the attributes separately, except for reporting.

The following use model covers STA in Single-Model Single-Corner (SMSC), Concurrent Multi-Mode Multi-Corner (CMMMC), and Distributed Multi-Mode Multi-Corner (DMMMC) modes.

The reporting attributes for cell EM are given below:

- The following attribute flags a warning message when a cell is missing electromigration groups or `max_cap` group on any cell. This setting must be made before reading the .lib.

  ```
  set_db timing_library_flag_cell_missing_em_data 1
  ```

  Default: `0`

  For example:

  ```
  **ERROR: (TECHLIB-1260): Cell 'BUFX14' does not have electromigration
  max_cap group on any pin (File /path/dummy.lib, Line 97)
  ```

- The following attribute reports DRV violations caused by EM and must be used for EM analysis.

  ```
  set_db timing_report_drv_per_frequency_per_input_slew true
  ```

  Default: `false`

- The following attribute handles out-of-bound slew in the EM table when the enable warnings are printed. With this setting, Tempus does not perform extrapolation and will use the last value present in the library.

  For example, if a library is characterized for slew range: 4, 10, 20, 30, 40, and slew for input pin is 45, then Tempus performs extrapolation to find the correct value of `max_cap` for those 45 slews. However, when this attribute is enabled, Tempus uses the last value, which is equivalent to 40.

  ```
  set_db timing_library_em_cap_lookup_slew_out_of_bounds true
  ```

  Default: `false`

- The following attribute handles out-of-bound frequency violations and indicates when the frequency goes beyond the limit. Tempus performs clipping, as mentioned in the last example.

  ```
  set_db timing_library_handle_out_of_bound_frequency true
  ```

  Default: `false`

- The following attribute enables flags in the `report_freq_violation` command to show if the slew/load values are beyond the library index.

  ```
  set_db timing_report_enable_em_index_clipping_report true
  ```

  Default: `false`

  For more details, refer to the Reports section.

4.15.4 Reports

This section describes the reporting for Cell EM. To analyze cell EM, you can use the following commands:

- `report_constraint -drv_violation_type max_capacitance`

  With the below settings, the above command reports the maximum capacitance value, which is the worst of frequency-based maximum capacitance (cell em) and SDC, or dotlib-based maximum value (without cell em).

  `set_db timing_report_drv_per_frequency_per_input_slew true`

  Default: `false`

  In this case, the software performs worst of all three avg, rms, and peak as default.

- `report_constraint -check_type electromigration`

  With the below settings, the `report_constraint` saves the frequency-based maximum capacitance value.

  `set_db timing_report_drv_per_frequency_per_input_slew true`

  Default: `false`

  In this case, the software performs worst of all three avg, rms, and peak as default.

  Note: The `report_constraint -check_type electromigration` option is supported in DSTA.

- `report_freq_violation`

Reports Examples

Some examples of reports are given below:

- Example 1
- Example 2
- Example 3
- Example 4
- Example 5
- Example 6

Example 1

```
report_constraint -drv_violation_type max_capacitance -all_violators
-------------------------------------------------------------------------------
# format : frame 0: split 1
# Units scale:          capacitance: 1.000pf
#                       transition: 1.000ns
#                       toggle rate: toggles/second

Check type : max_capacitance
-----------------------------
-------------------------------------------------------------------------------
Pin Name        Related Pin  Required   Actual   Slack  Input Slew  Toggle count
Clock Name   Type     Supply      Failure Reason
-------------------------------------------------------------------------------
blk1/clk_launch    i         0.000      0.321    -0.321  0.070    20000000000.000
clk          Clock    1.100000    CellEH
blk1/clk_common    i         0.000      0.316    -0.316  0.100    20000000000.000
clk          Clock    1.100000    CellEH
blk1/clk_capture   i         0.000      0.270    -0.270  0.070    20000000000.000
  clk        Clock    1.100000    CellEH
blk1/flop_launch   setb      0.131      0.274    -0.143  0.100    1500000000.000
clk          Data     1.100000    CellEH
```

Example 2

```
report_constraint -check_type electromigration -all_violators -verbose
-------------------------------------------------------------------------------
# format : frame 0: split 0
# Units scale:          capacitance: 1.000pf
#                       transition: 1.000ns
#                       toggle rate: toggles/second

Check type : max_transition
-----------------------------
-------------------------------------------------------------------------------
Pin Name        Related Pin   Required   Actual   Slack  Input Slew  Toggle count
Clock Name   Type      Supply     Failure Reason    View
-------------------------------------------------------------------------------
u_vcpu/u_ana/U648/B    *      *          0.000    *      *          337837824.000
coreclk      Data      0.825000   *
u_vcpu/u_ana/U648/ZN   *      *          0.000    *      *          601435648.000
coreclk      Data      0.825000   *

CM Cell Statistics:
```

```
Total Cell Output pins :0
Cell Output pins Analyted :0 0.00 % of total pins
         Pass :0 0.00 % of pins analyzed
         Fail :0 0.00 % of pins analyzed
               lpeak : 0
               lrms : 0
               lavg : 0
               max_tran :0
               other_cellem :0
Cell output pins not analyzed:0 0.00 % of total pins
         Constant :0 0.00 % of total unanalyzed pins
         No EM Table :0 0.00 % of total unanalyzed pins
Cell pins used as clock : 0 0.00 % of total pins
Cell pins used as data : 0 0.00 % of total pins
Cell out pins stimulated by default activity factor : 0 0.00 % of total pins
-------------------------------------------------------------------------------
-------------------------------------------------------------------------------
# Units scale: capacitance: 1.000pf
# transition: 1.000ns
# toggle rate: toggles/second

Check type : max_capacitance
-----------------------------
-------------------------------------------------------------------------------
Pin Name      Related Pin   Required   Actual   Slack  Input Slew  Toggle count
Clock Name   Type      Supply     Failure Reason    View
-------------------------------------------------------------------------------
u_vcpu/u_ana/U229/ZN   A1   0.012      0.011    0.001   13.200     80531352.000
coreclk      Data      0.825000   Pass              view1


CM Cell Statistics:
Total Cell Output pins :2104567
Cell Output pins Analyted :2097179 99.64 % of total pins
         Pass :2097179 100.00 % of pins analyzed
         Fail :2 0.00 % of pins analyzed
               lpeak : 0
               lrms : 0
               lavg : 2
               max_cap :0
               other_cellem :0
Cell output pins not analyzed :7478 0.36 % of total pins
         Constant :0 0.00 % of total unanalyzed pins
```

```
         No EM Table :0 0.00 % of total unanalyzed pins
Cell pins used as clock : 9066 0.43 % of total pins
Cell pins used as data : 2095591 99.57 % of total pins
Cell output pins stimulated by default activity factor : 68295 3.24 % of total pins
--------------------------------------------------------------------------------
```

The fields related to cell CM in the above reports are described in the below table:

Field Name	Description
Required Cap	Calculates the `max_cap` based on `input_slew` and Toogle Count from .lib using the electromigration table.
Actual Cap	Specifies the `total_capacitance_max` on the net of output pin.
Input Slew	Specifies the `slew_max` on the input pin.
Toggle Count	Specifies the transition density over the output pin.

Note:

- The actual capacitance and input slew can be verified using the `get_property` command for pin objects.
- The `get_activity` `-pin`, `-port`, and `-net` parameters can be used to verify the Toggle Count on pins, ports, and nets.

Example 3

Use the `report_freq_violation` command to write out a file with default name, *freqViolation.rpt.*

```
-----------------------------------------------------
# Content: Frequency limit violation report
# Design: top netlist
# View Name : default emulate view
-----------------------------------------------------
#Summary:
#Info: Total 6 instance in design.
#Info: Total 6 instance has electromigration table.
#Info: Total 7 frequency recorder for 6 instances in this report file.
#Info: 7 violations in total 7 recorder in this report file.
#Info: 3 violations in total 3 recorder for clock net (remark C).
#Info: Units · >Frequency (MHz), Capacitance (pf ), Transition Time (ns).
```

```
-------------------------------------------------------------------------------
MaxFreq    Freq    Tran       Cap      Remark    Arc/Pin
-------------------------------------------------------------------------------
685.1      750     0.2188     0.2213   *         blk_1/data_inv/o ^ i
362        750     0.1        0.2738   *         blk_1/flop_launch/q ^ set b
360.2      750     0.1        0.2738   *         blk_1/flop_launch/q ^ reset b
436.6      750     0.0707     0.2738   *         blk_1/flop_launch/q ^ phi
1286       le+04   0.0699     0.2699   C         blk_1/clock_capture/o ^ i
1149       le+04   0.0699     0.3214   C         blk_1/clock_launch/o ^ i
1116       le+04   0.1        0.3163   C         blk_1//clock_common/o ^ i
```

The fields in the above report are described below:

- `MaxFreq = min {freq(minslew, minload) , freq(maxslew, maxload)}`

  MaxFreq will be derived from cell the EM .lib based on the slew and load.

- `Freq = toggle_count/2`
- `Tran = min (slew_mean_min_fall, slew_mean_min_rise)`
  `Tran = max (slew_mean_max_fall, slew_mean_max_rise) (with -slew max/ -transition max)`
- `Cap= max (total_capacitance_max_fall,total_capacitance_max_rise)`

The rest of the fields can be verified using the `get_property` command on a pin or net within a timed session.

Example 4

The following command adds flags for PASS/FAIL/VIOLATE to the report, as shown below:

```
report_freq_violation -report_format 1
-------------------------------------------------------
# Content : Frequency limit violation report
# Design : top netlist
# View Name: default emulate view
-------------------------------------------------------
#Summary:
#Info: Total 6 instance in design.
#Info: Total 6 instance has electromigration table.
#Info: Total 7 frequency recorder for 6 instances in this report file.
#Info: 7 violations in total 7 recorder in this report file.
#Info: 3 violations in total 3 recorder for clock net (remark C).
```

```
#Info: Units->Frequency (MHz), Capacitance (pf), Transition Time (ns).
#Info: Sta is Status, [F] is Fail, [V] is Violation, [P] is Pass, [?] is unknown.
-------------------------------------------------------------------------
Sta   Rem    Freq   minfreq   maxfreq   Arc/ Pin
-------------------------------------------------------------------------
[V]   *     750     685.1     927.2     blk_l/data_inv/o ^ i
[F]   *     750     362       362       blk_l/flop_launch/q ^ setb
[F]   *     750     360.2     360.2     blk_l/flop_launch/q ^ resetb
[F]   *     750     436.6     436.6     blk_l/flop_launch/q ^ phi
[F]   C     le+04   1286      1287      blk_l/clock_capture/o ^ i
[F]   C     le+04   1149      1150      blk 1/clock_launch/o ^ i
[F]   C     le+04   1116      1116      blk_l/clk_common/o ^ i
-------------------------------------------------------------------------
```

Example 5

The following command displays a report that includes more details related to slew/load values along with the flags:

```
report_freq_violation -report_format 2
-------------------------------------------------------------------------------
Content: Frequency limit violation ireport
# Design: top_netlist
# View Name: default_emulate_view
#
#Info: Total 6 instance in design.
#Info: Total 6 instance has electromigration table.
#Info: Total 7 frequency recorder for 6 instances in this report file.
#Info: 7 violations in total 7 recorder in this report file.
#Info: 3 violations in total 3 recorder for clock net (remark C).
#Info: Units->Frequency (MHz), Capacitance (pf), Transition Time (ns).
#Info: Sta is Status, [F] is Fail, [V] is Violation, [P] is Pass, [?] is unknown.
-------------------------------------------------------------------------------
Sta    Rem   Freq    minfreq    maxfreq    minSlew      maxslew     minLoad
maxLoad  Arc/Pin
-------------------------------------------------------------------------------
[V]    *     750      685.1      927.2     2.188e-01    3.036e-01   2.213e-01
2.215e-01    blk_l/data_inv/o ^ i
[F]    *     750      362        362       1.000e-01    1.000e-01   2.738e-01
2.738e-01    blk_l/flop_launch/q ^ setb
[F]    *     750      360.2      360.2     1.000e-01    1.000e-01   2.738e-01
    2.738e-01    blk_l/flop_launch/q ^ resetb
```

```
[F]    *     750      436.6      436.6     7.070e-02    7.090e-02   2.738e-01
    2.738e-01    blk_l/flop_launch/q ^ phi
[F]    C     le+04    1286       1287      6.999e-02    7.020e-02   2.699e-01
    2.699e-01    blk_l/clock_capture/o ^ i
[F]    C     le+04    1149       1150      6.999e-02    7.020e-02   3.214e-01
    3.215e-01     blk 1/clock_launch/o ^ i
[F]    C     le+04    1116       1116      1.000e-01    1.000e-01   3.163e-01
    3.163e-01    blk_l/clk_common/o ^ i
--------------------------------------------------------------------------------
```

In row1, Freq 750 is > `minFreq`; therefore, Tempus shows the [V] flag, which means it is a violation. Since Freq 750 is > `minFreq`/`maxFreq`, Tempus shows the [F] flag, which means it is a Fail.

The fields in the above report are described below:

- `Freq = toggle_count/2`
- `MinFreq = min {freq(minslew, minload), freq(maxslew,maxload)}`
- `MaxFreq = max {freq(minslew, minload), freq(maxslew,maxload)}`
- `MinSlew = min (slew_mean_min_fall, slew_mean_min_rise) (Default)`
  - `min (slew_mean_max_fall, slew_mean_max_rise) (with -slew max/ -transition max)`
- `MaxSlew = max (slew_mean_min_fall, slew_mean_min_rise) (Default)`
  - `max (slew_mean_max_fall, slew_mean_max_rise) (with -slew max/ -transition max)`
  - `MinLoad = min (total_capacitance_max_fall ,total_capacitance_max_rise)`
  - `MaxLoad = max (total_capacitance_max_fall ,total_capacitance_max_rise)`

If the slew/load values are beyond the .lib index range, Tempus reports the values in the `report_freq_violation` output (if the below attributes are set to `true`):

- `timing_library_handle_out_of_bound_frequency`
- `timing_library_em_cap_lookup_slew_out_of_bounds`
- `timing_report_enable_em_index_clipping_report`

```
--------------------------------------------------------------------------------
# Content: Frequency limit violation report
# Design : top netlist
# View Name: default emulate view
```

```
--------------------------------------------------------------------------------
#Summary :
#Info: Total 6 instance in design.
#Info: Total 6 instance has electromigration table.
#Info: Total 16 frequency recorder for 6 instances in this report file .
#Info: 16 violations in total 16 recorder in this report file.
#Info: 3 violations in total 3 recorder for clock net (remark C) .
#Info: 3 violations in total 16 recorder are for frequency extrapolation.
#Info: Units->Frequency (MHz), Capacitance (pf ), Transition Time (ns).
#Info: Sta is Status, [Fl is Fail, [Vl is Violation, [Pl is Pass, [?] is unknown.
#Info: [Vl is 'Violation' means actual freq is greater than minFreq limit from
library.
#Info: [F] is 'Failure' means actual freq is higher than maxFreq limit from library.
#Info: (Tr OB): Indicates slew coming to related pin is higher than table limit.
#Info: (Ld OB): Load at this stage might be lower than expected table range.
--------------------------------------------------------------------------------
Sta    Rem  Freq      minFreq        maxFreq        minSlew      maxSlew
minload      maxload          Arc/Pin
--------------------------------------------------------------------------------
[Fl    *    2.5e+04   2561 (Ld OB)   2561 (Ld OB)   1.000e-01   1.000e-01
0.000e+00      0.000e+00      blk_1/flop_capture/q ^ setb
[Fl    *    2.5e+04   6014 (Ld OB)   6014 (Ld OB)   1.000e-01   1.000e-01
0.000e+00      0.000e+00      blk_1/flop_capture/q ^ resetb
[Fl    *    2.5e+04   5212 (Ld OB)   5241 (Ld OB)   5.950e-02   6.000e-02
0.000e+00      0.000e+00      blk_1/flop_capture/q ^ phi
[Fl    *    5e+04     6480           6480           1.000e-01   1.000e-01
NA             NA               blk_1/flop_capture/ ti
[Fl    *    5e+04     7459           7459           1.000e-01   1.000e-01
NA             NA               blk_1/flop_capture/te
[Fl    *    2e+04     1.183e+04      1.222e+04      5.070e-02   5.400e-02
NA             NA               blk_1/flop_capture/d
[Fl    *    2e+04     685.1          927.2          2.188e-01   3.036e-01
2.213e-01   2.2              15e-01 blk_1/data_inv/o ^ i
[Fl    *    2.5e+04   362            362            1.000e-01   1.000e-01
2.738e-01   2.738e-01        blk_1/flop_launch/q ^ setb
[Fl    *    2.5e+04   360.2          360.2          1.000e-01   1.000e-01
2.738e-01   2.738e-01        blk_1/flop_launch/q ^ resetb
[Fl    *    2.5e+04   436.6          436.6          7.070e-02   7.090e-02
2.738e-01   2.738e-01        blk_1/flop_launch/q ^ phi
[Fl    *    5e+04     6480           6480           1.000e-01   1.000e-01
NA             NA               blk_1/flop_launch/ti
[Fl    *    5e+04     7459           7459           1.000e-01   1.000e-01
NA             NA               blk_1/flop_launch/te
[Fl    *    5e+04     6480           6480           1.000e-01   1.000e-01
NA             NA               blk_1/flop_launch/ d
```

```
[Fl    C    le+04     1286           1287          6.990e-02   7.020e-02
2.699e-01   2.699e-01         blk_1/clock_capture/o ^ i
[Fl    C    le+04     1149           1150          6.990e-02   7.020e-02
3.214e-01   3.215e-01         blk_1/clock_launch/o ^ i
[Fl    C    le+04     1116           1116          1.000e-01   1.000e-01
3.163e-01   3.163e-01         blk_1/clk_common/o ^ i
```

Example 6

The following is a usage example of the set_design_rule_derate command:

```
set_design_rule_derate [get_lib_pins *] 1.5
```

The given factor will be multiplied with the frequency, as shown below:

Before this setting:

```
[F]   C   le+04   1286  1287   6.990e-02   7.020e-02   2.699e-01   2.699e-01
blk_1/clock_capture/o ^ i
[F]   C   le+04   1149  1150   6.990e-02   7.020e-02   3.214e-01   3.215e-01
blk_1/clock_launch/o ^ i
[F]   C   le+04   1116  1116   1.000e-01   1.000e-01   3.163e-01   3.163e-01
blk_1/clk_common/o ^ i
```

After this setting:

```
--------------------------------------------------------------------------------
Sta    Rem  Freq      minFreq      maxFreq     minSlew      maxSlew
minload          maxload          Arc/Pin
--------------------------------------------------------------------------------
[F]   C     le+04     1929         1930        6.990e-02    7.020e-02
2.699e-01        2.699e-01        blk_1/clock_capture/o ^ i
[F]   C     le+04     1723         1724        6.990e-02    7.020e-02
3.214e-01        3.215e-01        blk_1/clock_launch/o ^ i
[F]   C     le+04     1116         1116        1.000e-01    1.000e-01
3.163e-01        3.163e-01        blk_1/clk_common/o ^ i
```

As you can observe, `minFreq` = **1.5 * 1286 = 1929**

4.15.5 ECO Flow Use Model

Tempus ECO supports fixing cell EM.

For more details, refer to "ECO Flow Use Model for Cell EM Analysis" section.

4.15.6 Save/Restore of Design Database

All the commands that are saved with the write_power_constraints command will be saved in the database, for example, read_activity_file (with file path) and set_switching_activity. The propagated activity data will not be saved for a design.

If you want to save the propagated activity data, follow the steps given below:

1. Save TCF using the write_tcf command.
2. Read back TCF in the STA session using the read_activity_file.
3. Save the database (DB).

Note:

- Restore DB runs the read_activity_file, set_switching_activity, or other commands.
- The propagate_activity command must be run after restoring the DB.

4.15.7 Hierarchical STA - Cell EM

The Tempus SmartScope feature constitutes the bottom-up (Boundary Models) and top-down (Context Models) hierarchical timing analysis and closure techniques to overcome the challenges faced by designers during timing and glitch analysis, and closure of large hierarchical designs. The context models for lower level modules are generated from top level analysis to perform accurate block level timing and glitch analysis. Tempus supports Context Analysis for a single as well as multiple instances of blocks.

The Cell EM feature is supported for both context and boundary model-based flows. The context feature preserves the required information for a block to replicate the same EM violation over a block as it does for the top-level. The boundary model directory will be used to preserve information required for EM.

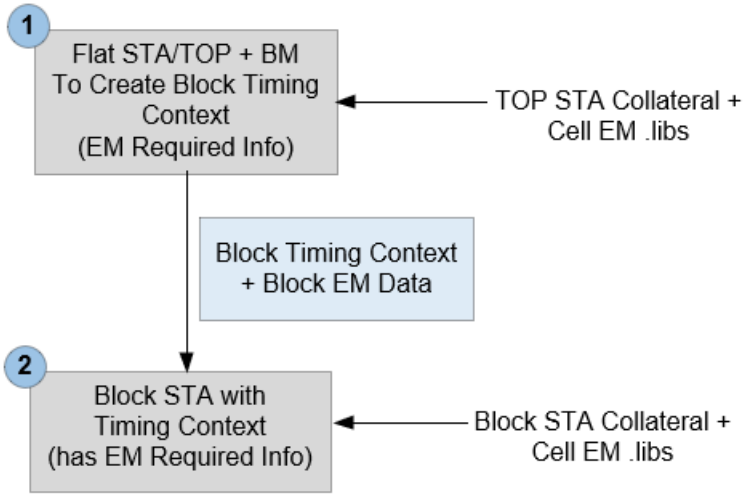
For MIM cases, Tempus will use the worse of all the instances. To enable this, you can use the setting given below:

```
set_db power_worst_case_vector_activity true
```

Cell EM Context-Based Flow

The timing analysis and closure with timing context is a two-step flow:

Figure 4-122 Cell EM Context-Based Flow



STEP1: Timing Context Generation from Top STA

The first step is to create the timing context of lower-level sub-modules from the top-level STA environment. This can either be done from the Top Flat or from Top BM analysis. Once the top-level design is fully loaded and ready to perform timing analysis, you can use the `write_timing_context -include_TCF` command to create timing context constraints of the required sub-module with EM required database.

Note: The `write_timing_context` command triggers an `update_timing -full` state. The design state after `update_timing -full` versus `write_timing_context`, however, remains the same. So, you can replace `update_timing -full` with the `write_timing_context` command in the top-level scripts and continue to generate reports after context creation.

```
read_libs (including Cell em .libs)
read_verilog
set_top_module top
read_spef
read_sdc
write_timing_context context.db -module block -include_TCF
propagate_activity -set_net_freq true
```

STEP2: Block STA with Timing Context

In this step, the software reads the block design with the context generated from STEP 1.

```
read_libs
read_verilog
set_top_module block
read_spef
read_sdc
* read_timing_context context.db -tcf_port_type {default:data; data, clock, both}
update_timing
propagate_activity -set_net_freq true
```

where:

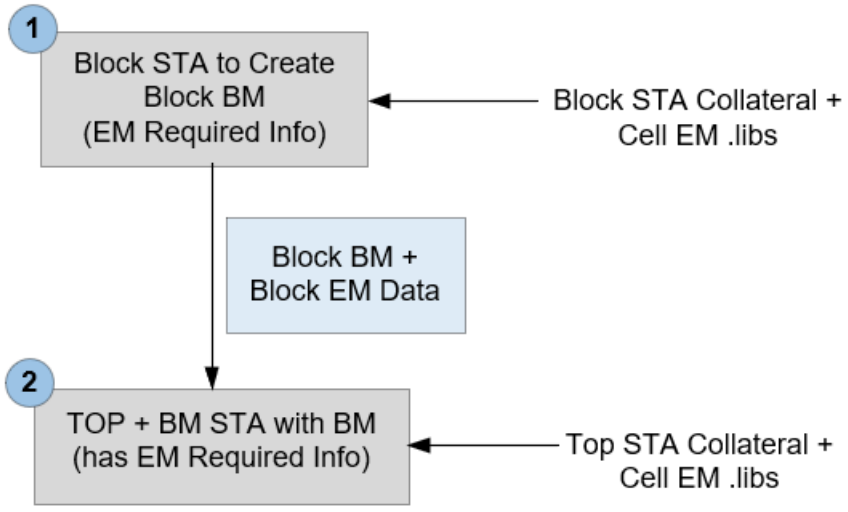
`-tcf_port_type` parameter utilizes activity information from the top over clock port, data port, or both. You can use the following options:

- `data`: Activity from top will be annotated only on data port, clock over block will be utilized internally for calculating activity over clock port.
- `clock`: Activity from top will be annotated only on clock port.
- `both`: Both clock and data port will have annotation.

Cell EM Boundary Model Flow

The timing analysis and closure with boundary model is a two-step flow:

Figure 4-123 Cell EM Boundary Model-Based Flow



STEP1: Boundary Model Generation from Block STA

The first step is to generate a boundary model BM from Block that is required to be plugged in to TOP.

You must run the `create_boundary_model -include_TCF` command to have a BM model with all the EM information required to be integrated in TOP, as shown below:

```
read_libs (including Cell em .libs)
read_verilog
set_top_module block
read_spef
read_sdc
create_boundary_model -dir DIR -include_TCF
propagate_activity -set_net_freq true
```

STEP2: TOP STA with Boundary Model

This step involves integrating the BM generated from Step 1 in TOP Level.

```
read_libs
read_verilog
set_top_module block
read_spef
read_sdc
* read_boundary_model DIR -module
update_timing
propagate_activity -set_net_freq true
```

Note: You must follow all the requirements and limitations of boundary models.

4.15.8 Current Limitations and Recommendations

The following are the current limitations of Cell EM analysis:

- The current activity is constraint mode-aware, which is picked up from the power view definition.
- For any defined power view, Tempus picks the clock definition and maps it to the activity definition obtained from the `set_switching_activity` or `read_activity_file` command to determine the switching and toggle rate.

It is recommended to consider the above limitations before evaluating Cell EM violations to ensure that the toggle rate reported is as per the design specifications. The above process aligns with the Tempus ECO flow where the dynamic view determines the activity definition.

4.16 Hierarchical Timing Context Analysis

4.16.1 Hierarchical Timing Context Analysis - Overview on page 316

4.16.2 Hierarchical Timing Context Analysis Flow on page 317

4.16.3 Constraints Handling with Timing Context Analysis on page 318

4.16.4 Clock Mapping in Context Flow on page 319

4.16.5 Clock Mapping Approaches on page 324

- 1-1 Clock Mapping Approach on page 324
- Port-Based Clock Mapping Approach on page 335

4.16.6 Manual Clock Mapping Specification on page 343

4.16.7 Clock Mapping Reporting on page 345

4.16.8 MIM Context Analysis on page 348

4.16.9 Context Generation and Analysis in MMMC on page 352

4.16.1 Hierarchical Timing Context Analysis - Overview

The Hierarchical Timing Context Flow offers top-down analysis techniques to model the accurate block boundary constraints as seen from its top hierarchy or top STA environment. Block Timing Context constraints accurately model the boundary constraints, such as, the arrival time of a top STA data signal entering the block at corresponding input port of the block, the required time of a data signal leaving the block at corresponding output port, or clock latencies of the clocks entering the block clock ports.

The block STA with Context constraints reproduces the reg-reg timing of the block as seen from the top STA. Hence, STA runs for block closure can be done with realistic constraints instead of pessimistic budgeted IO constraints. The Context is generated in proprietary format and can be read in Tempus only.

Tempus SmartScope constitutes the bottom-up (Boundary Models) and top-down (Context Models) hierarchical timing analysis/closure solution to address the challenges faced during timing and glitch analysis/closure of large hierarchical designs.

The Context Models for lower-level modules are generated from top level analysis to perform accurate block level timing and glitch analysis. Tempus supports the Context Analysis for single as well as multiple instances of the block.

This chapter provides the details of methodology for performing timing and glitch analysis of lower-level modules using Context Models. This document also covers the details of constraints handling, clock mapping, design requirements, sources of inaccuracy and limitations associated with context flow.

Terms Used in this Chapter

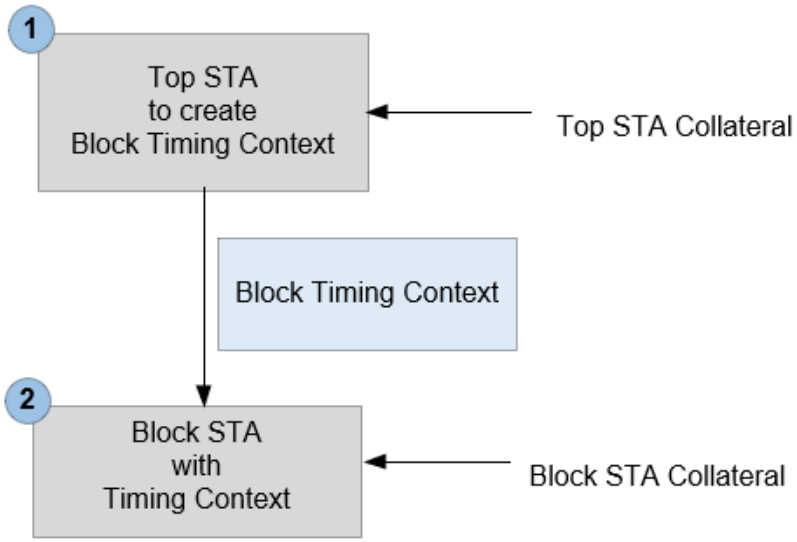
- STA: Static Timing Analysis
- Top/Top-level: Higher level module
- Block/Block-level: Lower-level module
- Block Analysis: STA at Block level
- Top Analysis: STA at Top level
- Top+BM Analysis: Top-level analysis with block boundary models
- Block Context Analysis: Block STA with Timing Context
- SIM: Single Instantiated Modules
- MIM: Multiply Instantiated Modules

- GBA: Graph-Based Analysis
- PBA: Path-Based Analysis
- Interface path: In-Reg, Reg-Out, In-Out timing paths
- Interface path pin: Pins which are part of any interface path
- Internal pin: Pins which are part of reg-reg paths only
- Boundary Net: Nets connected to ports

4.16.2 Hierarchical Timing Context Analysis Flow

Timing analysis/closure with Timing Context is a two-step flow -

Figure 4-124 Hierarchical Timing Analysis - Flow



STEP1: Timing Context Generation from Top STA

The first step is to create the Timing Context of lower-level submodules from Top level STA environment. This can be done either from Top Flat or from Top+BM analysis. Once the top level design is fully loaded and ready to perform timing analysis, use the `write_timing_context` command to create Timing Context constraints of the required sub-module.

Note: The `write_timing_context` command triggers the `update_timing -full` state and design state after `update_timing -full` vs `write_timing_context` remains the same.

Hence, you can replace `update_timing -full` with the `write_timing_context` command in the top level scripts and continue to generate reports after context creation.

```
read_libs
read_verilog
set_top_module top
read_spef
read_sdc
write_timing_context context.db -module block
```

STEP2: Block STA with Timing Context

The second step is to perform block STA with Timing Context generated in step1. The timing Context should be read after reading all the block design collaterals, including the block constraints. You can use the `update_timing` command after reading the context followed by the reporting commands.

```
read_libs
read_verilog
set_top_module block
read_spef
read_sdc
read_timing_context context.db
update_timing
```

4.16.3 Constraints Handling with Timing Context Analysis

Tempus models the following constraints in Timing Context:

- Latency and transition of top level clock signals entering the block as clock phase
- Arrival and transition of top level data signals entering the block as data phase
- Required time at block output port
- Path exceptions impacting the block interface paths
- Top level case analysis/pin-based uncertainty which enters the block

During block STA with Timing Context, all the block port constraints defined in block SDC, (except port clocks) are overwritten by the timing context constraints. However, the port clock definitions and constraints on the block internal pins/instances are derived from the block SDC. Hence, for accurate analysis with timing context, the block and top SDCs should have consistent clock definitions and consistent constraints applied on the block internal pins.

The following are some examples that illustrate interface constraints handling:

Example 1: The following constraints defined (using the set_false_path command) on the top SDC will be honored in Context Analysis:

```
set_false_path -through <block port>
set_false_path -from <top level pin> -to <block interface path pin>
set_false_path -through <top level pin> -through <block internal pin>
```

Example 2: The following constraints defined on the top SDC will be ignored during Context Analysis:

```
set_false_path -through <block interface path pin>
```

Example 3: The following constraints defined in the block SDC will be honored during Context Analysis:

```
set_false_path -through <block interface/internal pin>
```

Example 4: The following constraints defined in the block SDC will be ignored during Context Analysis:

```
set_false_path -through <block port>
set_false_path -through <block port> -to <block interface pin>
```

To avoid overwriting of context constraints, Tempus does not allow constraints application on block ports during block STA with Timing Context. If you try to apply constraints on the port after reading Timing Context, Tempus will issue a warning (TCLCMD-1541) and ignore such constraints.

Any constraint on instances within the block and their pins can be modified even after reading the Timing Context.

4.16.4 Clock Mapping in Context Flow

What is Clock Mapping

The Timing Context stores various types of data, the following four type of data are associated with top level clocks:

- Latency of top level clock signals entering the block as clock phase
- Arrival of top level data signals entering the block as data phase
- Required time at block output port
- Path Exceptions impacting the block interface paths

Tempus identifies the equivalent block and top level clocks to accurately assert the top level data with correct block level clocks during STA with Timing Context. This process of identifying the equivalent top and block clock pair is called clock mapping, and such clocks are called the mapped top and block clocks, respectively. Any top level clock whose corresponding block clock cannot be identified is called an unmapped top clock. Similarly, any block level clock whose corresponding top clock cannot be identified is called an unmapped block clock.

Tempus also writes out clock mapping reports, which can help to review the mapped or unmapped clock and data phases.

By default, Tempus does not allow the Timing Context analysis in the presence of unmapped top level clocks (either entering block as clock phase or data phase) and stops the context reading with error TA-1024. To continue the analysis with unmapped top level clock phases, the following attribute should be used before context reading:

```
set_db timing_hierarchical_context_continue_on_top_block_clock_mismatch true
```

Note: All the unmapped block/top clocks and their associated constraints are dropped from analysis in block STA run with Timing Context. Hence, you need to always review the unmapped clocks reported in clock mapping output file.

Types of Data Mapping

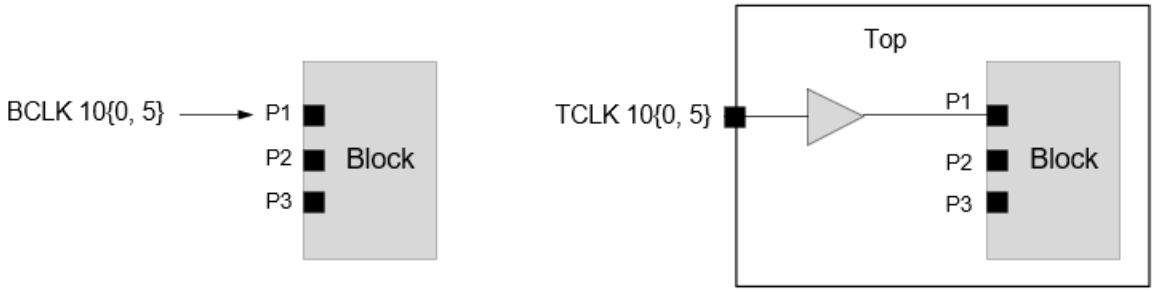
- Clock Phase Mapping
- Data Phase Mapping
- Path Exception-Based Mapping
- Block Internal Clock Mapping

Clock Phase Mapping

The following example represents a simple case of clock phase mapping, where:

- In block STA, the clock CLK is defined on port P1 of the block. This clock has period of 10ns with rising edge at 0ns and falling edge at 5ns.
- In Top STA, the clock phase of CLK enters the block port P1. This clock has period of 10 units with rising edge at 0ns and falling edge at 5ns.
- The top level CLK will be mapped to block level CLK.

The figure below illustrates clock phase mapping:



For the mapped block clock BCLK, the following properties will be derived from an equivalent mapped top clock:

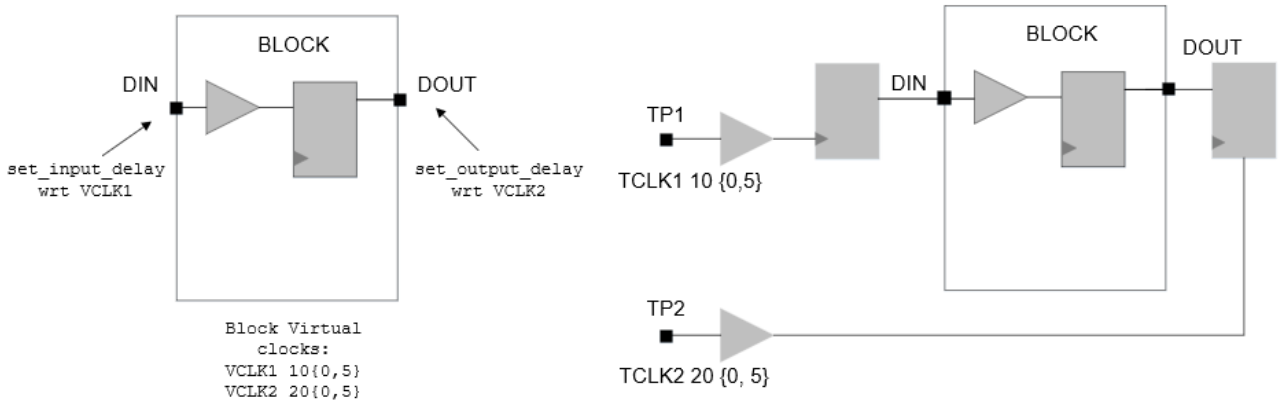
- Rise and fall latency at block port P1
- Rise and fall transition (slew) at block port P1
- Polarity of clock signal at block port P1
- Top level pin-based uncertainty (i.e., uncertainty at fanin of block/P1)

Data Phase Mapping

The figure below shows a simple case of data phase mapping, where:

- Block-level design has two virtual clocks, VCLK1 and VCLK2. The port DIN has data phase with respect to VCLK1 (`set_input_delay`) and port DOUT has data phase with respect to VCLK2 (`set_output_delay`).
- In the Top level design, TCLK1 enters the block as data phase (at DIN port). And TCLK2 captures the data from DOUT port of the block.
- In context analysis, TCLK1 data phase will be mapped with block virtual clock VCLK1. The TCLK2 data phase will be mapped with block virtual clock VCLK2.

Figure 4-125 Data Phase Mapping



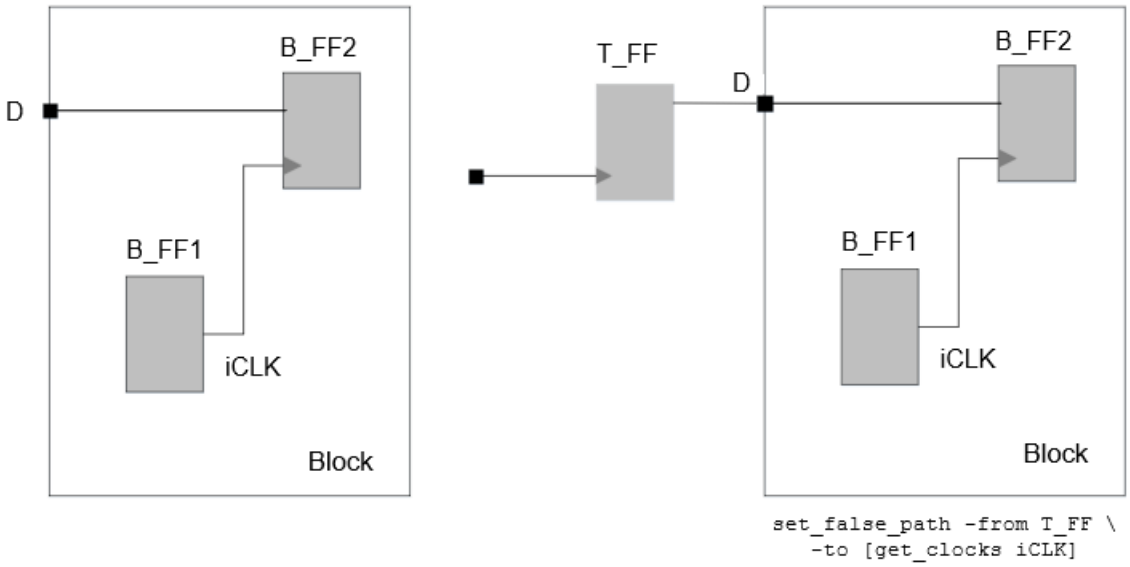
For mapped data phase, the following properties will be derived from the top level:

- Arrival time and transition at input ports
- Required time at output port

Path Exception-Based Mapping

The path exception-based mapping refers to mapping of block internal clocks that have top level path exceptions interacting with the block IO.

The figure below shows path exception-based mapping:



- The clock iCLK is defined inside the block and its clock/data phase does not interact with block IO. Hence, this clock is not mapped under clock or data phase mapping.
- However, there is a top level false path from top level flop T_FF to block internal clock iCLK. Since this exception is associated with block IO, it will be modeled in Context.

To apply this exception in block Context Analysis, the block internal clock iCLK needs to be mapped with top level clock iCLK.

Note: The block iCLK will be mapped to top iCLK, only if they pass the auto clock mapping rules.

Block Internal Clock Mapping

The clocks that are defined internal to the blocks are also mapped. This is controlled by the `timing_enable_hier_context_internal_clock_mapping` attribute. The source clocks (`create_clock` based) are automatically mapped when the clock is defined on the same pin in both the top and block constraints and have a matching clock property. The generated clocks are mapped based on the matching clock names.

Some of the unmapped internal clocks may not impact reading data from context, but it does indicate potential constraints mis-correlation between top level and block constraints. The correlation between the Top and Block constraints is critical for accurate context analysis.

Types of Clock Mapping

The following two types of clock mapping are supported in the context flow:

- **Auto Clock Mapping**: The software automatically maps the Top and Block STA clocks based on the auto clock mapping rules. The clocks which do not satisfy the rules remain unmapped (unless mapped by the user).
- **User Clock Mapping**: You can specify the Top and Block clock pair for mapping. The user-specified clocks are mapped by the software irrespective of whether they follow the auto clock mapping rules. The user-specified clock mapping has a higher priority and will override auto clock mapping. If you map the Top and Block level clocks of different waveforms, then the block level waveform will be used during analysis.

For a successful user clock mapping, the following are required:

- The block and top level clocks should be present at the same block port
- Internal clocks should interact with block IO (Path exception-based)

- If a top level clock enters the block only as data phase, then you should map it with block virtual clocks only

An example of user-defined clock mapping to map top level TCLK with block level BCLK clock is given below:

```
read_timing_context -clock_mapping_file clkMap.txt
```

Given below are contents of clkMap.txt:

```
set_timing_context_clock_mapping -top_clock TCLK -block_clock BCLK -view
<analysisViewName>
```

4.16.5 Clock Mapping Approaches

Tempus supports two clock mapping approaches that are controlled by the `timing_context_port_based_clock_mapping` attribute.

When set to `false` (default), Tempus supports 1-1 mapping of clocks between top and block clocks. See 1-1 Clock Mapping Approach.

When set to `true`, Tempus supports mapping multiple top clocks to a single block clock and vice versa. See Port-Based Clock Mapping Approach.

1-1 Clock Mapping Approach

When `timing_context_port_based_clock_mapping` is set to `false`:

The basic requirement for auto mapping is the presence of Block and Top STA clocks having consistent waveform at a block port. For example, as seen in the figure below, the top clock CLK will be automatically mapped with block clock CLK as both the clocks have the same properties and are present at the same port P1 in block and top STA.

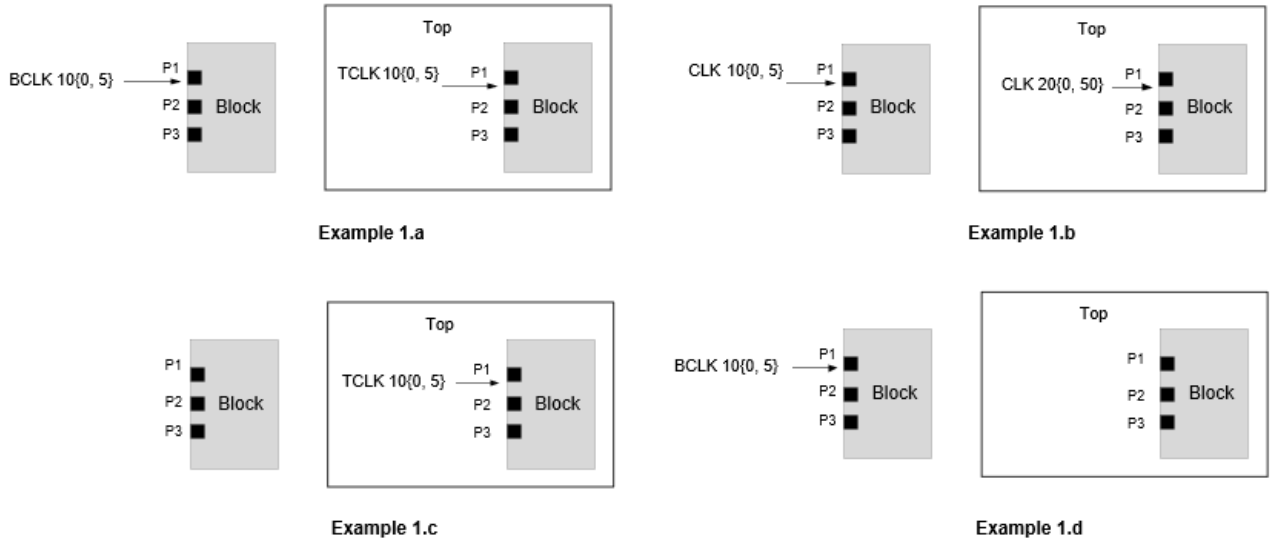
If multiple clocks are present at a port or if a clock is present at multiple ports, then Tempus follows the below mentioned rules (mentioned in order of precedence) to determine the mapping:

- Rule1: Clocks with matching property (period/waveform)
- Rule2: Clocks with matching name
- Rule3: Clocks present on maximum number of common ports
- Rule4: Any one of the matching clocks

The following figures illustrate the various Top-Block clock design scenarios and Tempus's auto clock mapping behavior:

- Example 1: Single clock at single block port
- Example 2: Multiple clocks at single block port
- Example 3: Single clock at multiple block ports
- Example 4: Clock mapping approach in case of multiple mapping candidates I (Rule 2 and Rule 3)
- Example 5: Clock mapping approach in case of multiple mapping candidates II (Rule 4)

Example 1: Single clock at single block port



Example 1.a:

- Block clock BCLK has same properties (Period is 10units, rise edge is 0 and fall edge is 5units) as the top clock phase TCLK. Hence, BCLK is mapped with TCLK even though their names are different.
- BCLK clock phase will propagate through port P1 in block context analysis.

Example 1.b:

- Top and block STA clocks CLK have the same name but different properties. The block clock period is 10units and top clock period is 20units. Hence, both the top and block CLK remains unmapped.

- Unmapped block clock CLK will be dropped from context analysis.

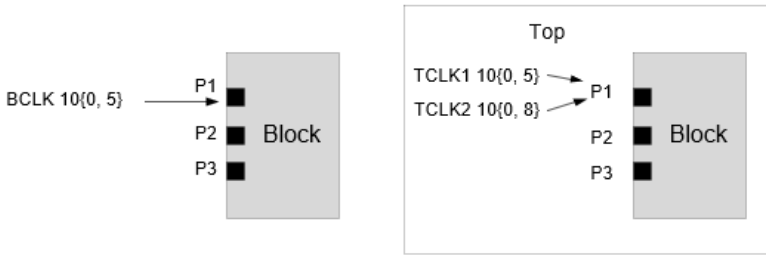
Example 1.c:

- TCLK clock phase enters block port P1 in top STA but no clock is defined on P1 in block STA. Hence, TCLK remains unmapped.
- Unmapped top clock TCLK will be dropped from analysis. So, no clock phase propagates through block port P1 in block context analysis.

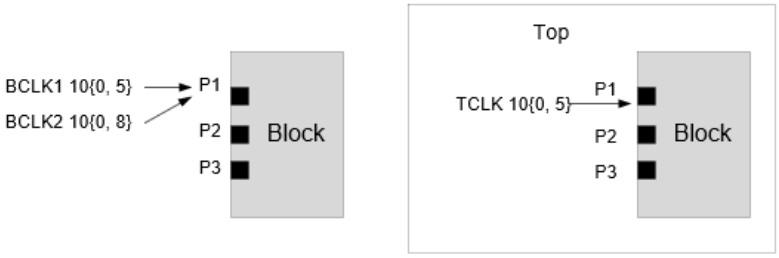
Example 1.d:

- BCLK is defined at port P1 in block STA but no clock phase enters this port in top STA. Hence, BCLK remains unmapped.
- Unmapped block clock BCLK will be dropped from analysis. So, no clock phase propagates through block port P1 in block context analysis.

Example 2: Multiple clocks at single block port



Example 2.a



Example 2.b

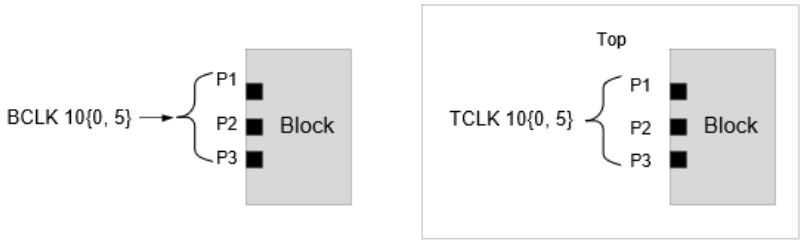
Example 2.a:

- Top level clock TCLK1 will be mapped with block level clock BCLK as they have the same waveform.
- Top level TCLK2 will be unmapped as there is no equivalent clock in block STA. The unmapped clock TCLK2 will be dropped from analysis.

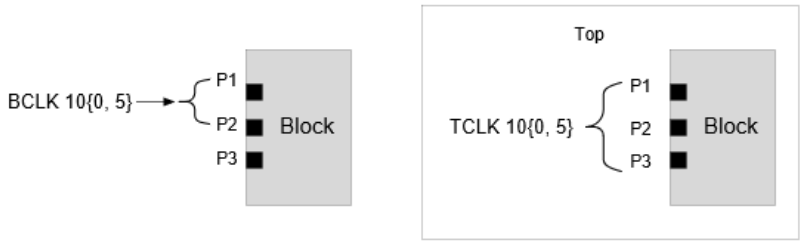
Example 2.b:

- Top level clock TCLK will be mapped with block level clock BCLK1 as they have the same waveform.
- Block level clock BCLK2 will be unmapped as there is no equivalent clock in Top STA. Only BCLK1 phase will propagate from port P1 in context analysis.

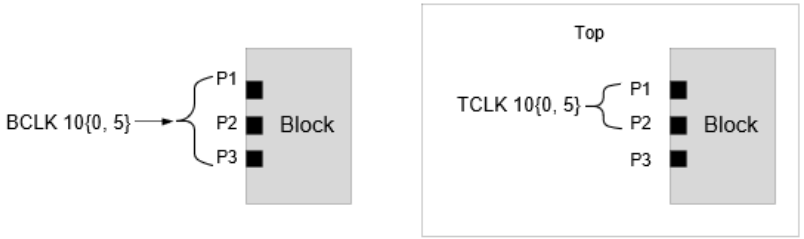
Example 3: Single clock at multiple block ports



Example 3.a



Example 3.b



Example 3.c

Example 3.a:

- Block clock BCLK and top clock TCLK have the same properties and they are present at the same set of ports - P1, P2, and P3. Hence, the BCLK is mapped to TCLK and BCLK phase propagates through P1, P2 and P3 ports.
- Top level latency of TCLK from its source to Block/P1, Block/P2 and Block/P3 will be asserted on BCLK clock for port P1, P2 and P3 respectively.

Example 3.b:

- BCLK is mapped to TCLK and BCLK clock phase propagates through P1, P2 and P3 ports (even though BCLK is defined on P1 and P2 ports only).
- Top level latency of TCLK from its source to Block/P1, Block/P2 and Block/P3 will be asserted on BCLK clock for port P1, P2, and P3, respectively.

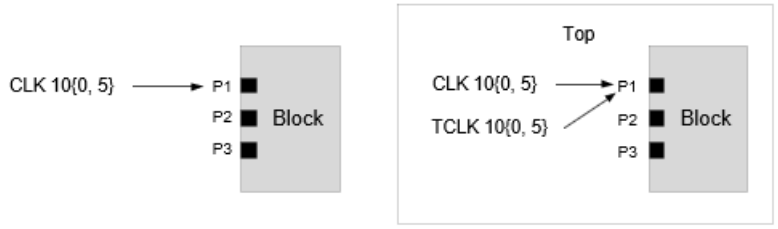
Example 3.c:

- BCLK is mapped to TCLK and BCLK phase propagates through P1 and P2 ports only.
- No clock phase will propagate through P3 port.
- Top level latency of TCLK from its source to Block/P1 and Block/P2 will be asserted on BCLK clock for port P1 and P2 respectively.

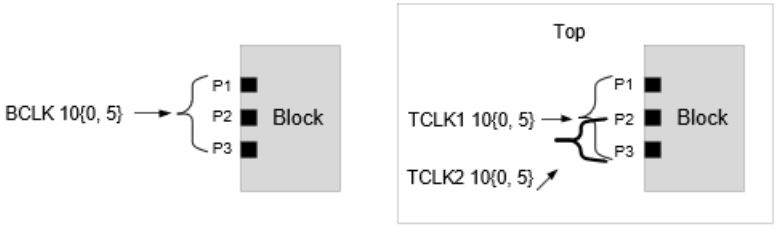
In the above cases, Tempus determines the clock mapping as per Rule1 of auto clock mapping.

Example 4: Clock mapping approach in case of multiple mapping candidates I (Rule 2 and Rule 3)

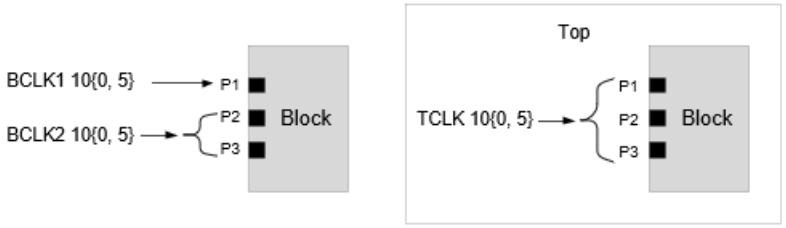
If multiple top (or block) clocks qualify Rule1 to map to block clock, then Tempus applies Rules 2, 3, and 4 (in precedence order) to determine a unique mapped clock pair. The figure below covers such cases.



Example 4.a



Example 4.b



Example 4.c

Example 4.a (Rule 2):

- Two top level clocks CLK and TCLK satisfy Rule1 (property-based mapping) of clock mapping and are candidates for mapping with Block clock BCLK.
- Since Rule1 cannot determine the mapping, Rule2 (name-based mapping) is checked. Top clock CLK satisfies the Rule2 and hence gets mapped with block CLK.
- Top clock TCLK remains unmapped and dropped from analysis.

Example 4.b (Rule 3):

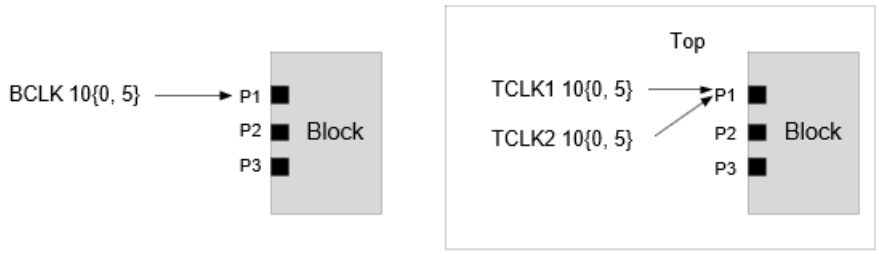
- Both the top level clocks TCLK1 and TCLK2 are candidates to get mapped with block clock BCLK. The rules 1 and 2 cannot identify the top clock that should be mapped with the block clock.

- Hence as per rule 3, TCLK1 gets mapped to BCLK - out of TCLK1 and TCLK2, TCLK1 has higher number of common ports with the block clock BCLK.
- TCLK2 remains unmapped and is dropped from analysis.

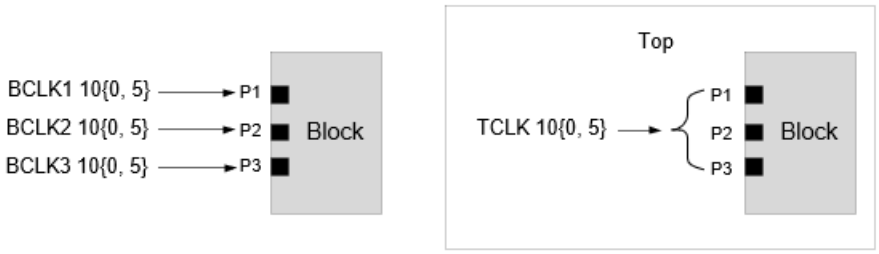
Example 4.c (Rule 3):

- Both block clocks BCLK1 and BCLK2 are candidates to get mapped with top clock TCLK. The rules 1 and 2 cannot identify the block clock that should be mapped with the top clock.
- Hence as per Rule3, BCLK2 gets mapped to TCLK - out of BCLK1 and BCLK2, BCLK2 has a higher number of common ports with the top clock TCLK.
- BCLK1 remains unmapped and is dropped from analysis.

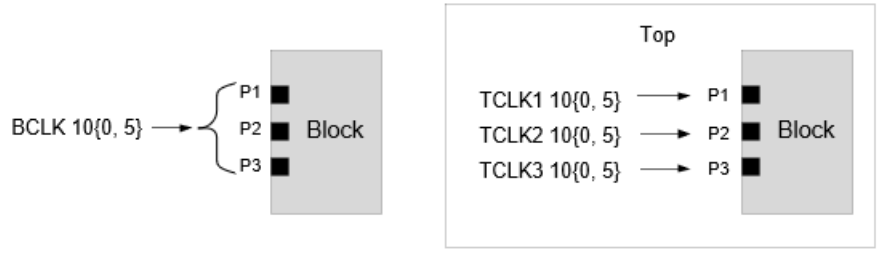
Example 5: Clock mapping approach in case of multiple mapping candidates II (Rule 4)



Example 5.a



Example 5.b



Example 5.c

Example 5.a:

- Both top level clocks TCKL1 and TCLK2 are candidates to get mapped with block clock BCLK. Rule1, 2, and 3 cannot identify the top clock that should be mapped with the block clock.
- Hence, any one of TCLK1 and TCLK2 will be selected for mapping with BCLK. This selection is deterministic.

Example 5.b:

- All the top and block level clocks have same properties, different names, and the same number of common ports between the block and top. Hence, rules 1, 2, and 3 cannot identify the top clock that should be mapped with the block clock.
- Hence, any one of the block clocks will be deterministically selected for mapping with TCLK. The remaining two block clocks will be unmapped.

Example 5.c:

All the block and top level clocks have same properties, different names, and the same number of common ports between the block and top. Hence, rules 1, 2, and 3 cannot identify the top clock that should be mapped with the block clock.

Hence, any one of the top clocks will be deterministically selected for mapping with BCLK. The remaining two top clocks will be unmapped.

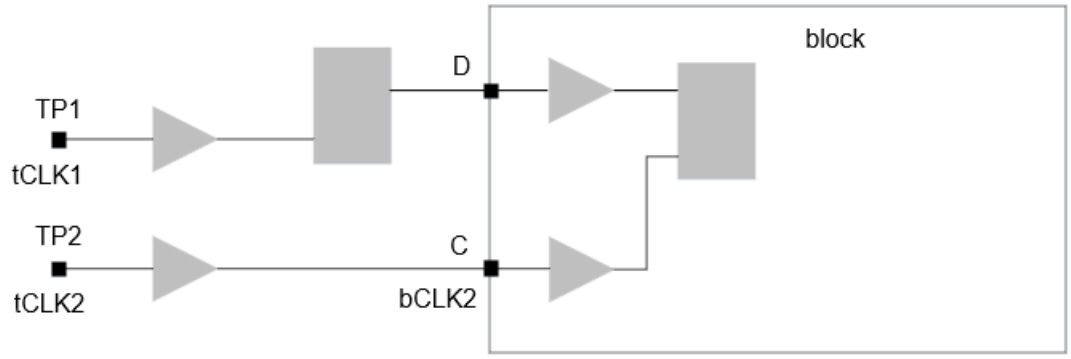
Data Phase Clock Mapping

Any top level clock (real or virtual) which enters the block only as data signal is attempted to map with block virtual clocks as per the below mentioned priority:

- User-specified clock mapping:
  Such top level clocks will always be mapped with block virtual clocks.
- Clocks with same name and period/duty cycle:
  Such top level clocks are mapped with block virtual clocks having same name and waveform.
- Mapping based on block `set_input_delay`/`set_output_delay` constraints.

Tempus first checks the presence of data signal with respect to the top level clocks on block ports and identifies the block virtual clocks having `set_input_delay`/`set_output_delay` on the corresponding ports. Then auto-clock mapping rules (See Types of Clock Mapping on page 323) are applied on these clocks to identify the mapped clocks.

Example 1:



Top constraints:

```
create_clock -period 10 -name tCLK1 -waveform {0 5} [get_ports TP1]
create_clock -period 10 -name tCLK2 -waveform {0 5} [get_ports TP2]
```

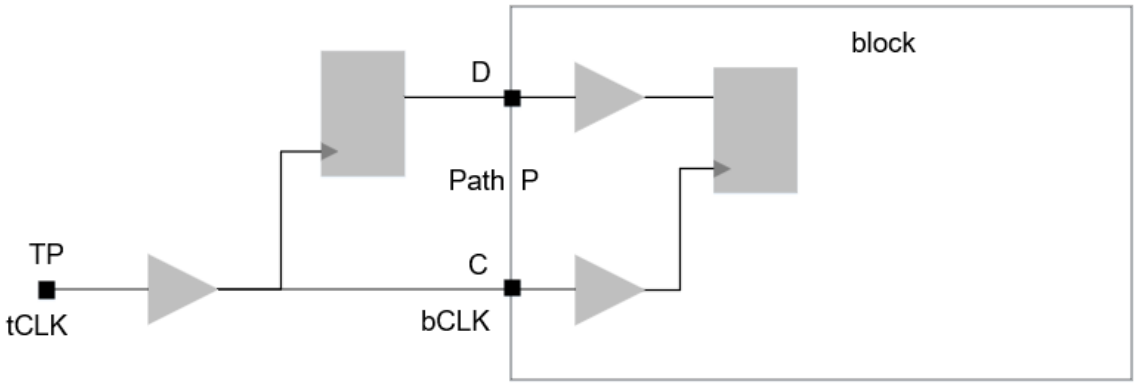
Block constraints:

```
create_clock -period 10 -name bCLK2 -waveform {0 5} [get_ports C]
create_clock -period 10 -name vCLK1 -waveform {0 5}
set_input_delay -clock vCLK1 2 [get_ports D]
```

Mapping result:

- Top clock tCLK2 enters the port C as clock signal. The block SDC has bCLK2 on this port and properties of bCLK2 match with tCLK2, hence these are mapped.
- Top clock tCLK1 enters the port D as data signal. The block SDC has SID with respect to the virtual clock vCLK1 and properties of these two clocks match, hence these are mapped.

Example 2 - Data Phase Clock Mapping



Top constraints:

```
create_clock -period 10 -name tCLK -waveform {0 5} [get_ports TP]
```

Block constraints:

```
create_clock -period 10 -name bCLK -waveform {0 5} [get_ports C]
create_clock -period 10 -name vCLK -waveform {0 5}
set_input_delay -clock vCLK 2 [get_ports D]
```

Mapping Result:

- Clock tCLK at port C mapped to bCLK.
- Data signal of tCLK at port D is also mapped to bCLK, since these are mapped clocks.
  - tCLK is entering as data as well as clock. Hence it is not a virtual clock for block.
- Path P will be analyzed with tCLK clock and data arrival.
- vCLK will remain unmapped.

Generated Clock Mapping

During context reading, generated clock waveforms are not available (since timing update is not done). Hence, block-level generated clocks are mapped with top level clocks based on their names only - without any waveform check. Hence, you must review the block vs top level waveforms of such clocks after timing update.

Clock Mapping Reports

Tempus automatically creates the following output files while reading the Timing Context:

- *timing_context_clock_mapping.rpt*
- *timing_context_clock_mapping_data.rpt*

These files honor `auto_file_dir` and `auto_file_prefix` attributes.

You must review the content of *timing_context_clock_mapping.rpt* file before proceeding with context analysis. This report file has the following four sections:

Section 1: Unmapped Top level clocks

This section reports the top-level clocks which could not be mapped with any block level clock. It also has the diagnostic message for possible reason of unsuccessful mapping

The table below indicates that the top-level clock CLK1 could not be mapped with any block level clock as there is no block level clock defined on block port P1 having waveform same as top level CLK1 clock.

```
Info:        Top clocks NOT mapped to any block clocks: <TA-1023.
-------------------------------------------------------------------------------

Top Clock   Period  Rise   Fall   Usage   View   Mapping Result
                 Block Ports
-------------------------------------------------------------------------------

CLK1       4.000    0.000  1.000  {C I 0} view1  No possible match by parameter/
name   P1
CLK2       100.000  0.000  50.000 {I}     view1  No possible match by parameter/
name   P2
-------------------------------------------------------------------------------
```

Usage Columns indicates the association of top-level clock with block. Possible Entries are:

- C -Top level clock phase
- I -Top level data phase on input ports
- O -Top level required time on output ports
- E -Top level exception on block input/output ports.

Section 2: Successfully mapped Top-Block Clocks

This section reports the top level clocks and the corresponding mapped block level clocks. It also prints the usage of top level clock at block level, indicating whether clock enters as clock/ Data phase/Exception etc.

For example, the below output indicates that the top level clock TCLK is mapped with block level clock BCLK. Usage {C} indicates that the TCLK enters the block as clock phase.

```
Info:        Top clocks successfully mapped to block clocks: <TA-1022>.
-------------------------------------------------------------------------------

Top Clock   Period    Rise   Fall    Usage   Block Clock   View   Type
-------------------------------------------------------------------------------

TCLK        40.000    0.000  20.000  {C}     BCLK          view1  auto
-------------------------------------------------------------------------------
```

Section 3: Block level unmapped port clocks

This section reports the block level port clocks which could not be mapped with any top-level clock.

```
Info:     Block input port clocks NOT mapped to any top clocks: <TA-1025>.
-------------------------------------------------------------------------------

Block Clock   Period       Rise       Fall        View
-------------------------------------------------------------------------------
```

```
BCLK1          10.000       0.000      5.000       view1
BCLK0          10.000       0.000      5.000       view1
-------------------------------------------------------------------------------
```

Section 4: Block level unmapped virtual clocks

This section reports the block level virtual clocks which could not be mapped with any top-level clock.

```
Info:     Virtual block clocks NOT mapped to any top clocks: <TA-1041>.
-------------------------------------------------------------------------------
Block Clock         Period      Rise       Fall        View
-------------------------------------------------------------------------------
VCLK2_VIRT          3.600       0.000      1.100       view1
-------------------------------------------------------------------------------
```

Port-Based Clock Mapping Approach

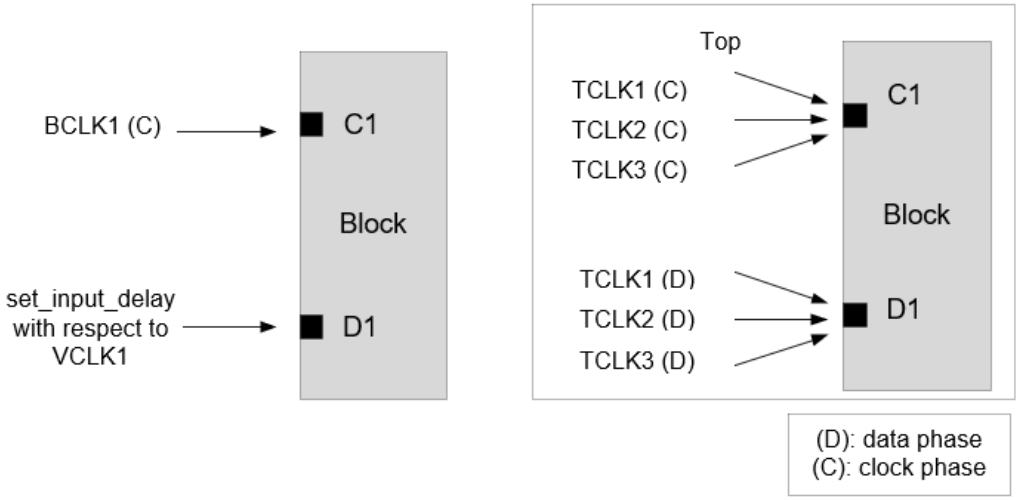
The 1-to-1 clock mapping helps to achieve tight correlation between context analysis and flat analysis. But there can be cases where multiple block-level clocks need to be mapped to single top level clock or vice-versa. Tempus supports port-based clock mapping approach to handle such cases.

The port-based clock mapping approach is enabled by setting the `timing_context_port_based_clock_mapping` attribute to `true`. In port-based clock mapping, clock mapping is done for each clock and data port independently based on clock property matching. This section covers details of port-based clock mapping approach.

Multiple Top Clock to Single Block Clock Mapping

In case of multiple top clocks reaching a block port with one block clock defined in block constraints, Tempus automatically maps one block clock to multiple top clocks on a block port if the clock property matches. In case of clock property mismatch, manual clock mapping needs to be specified by the user.

Port Based Clock Mapping Example



In above example, automatic clock mapping of TCLK1,TCLK2, or TCLK3 to BCLK1 on clock port C1 and to VCLK1 on data port D1 will be done if clock property matches.

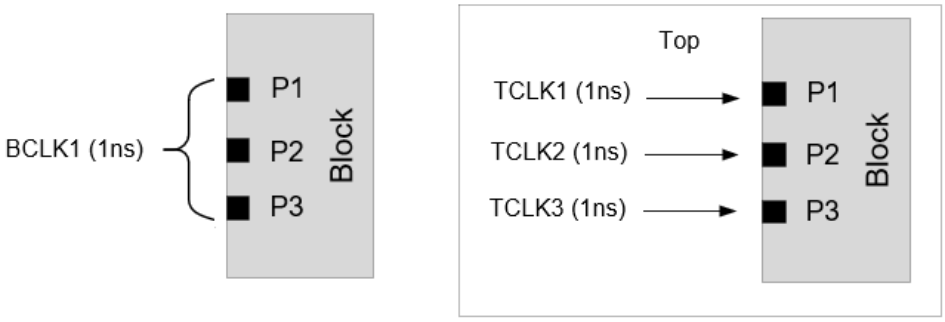
The table shown below gives the resultant mapping when clock property matches:

Port	Block Clock	Mapped Top Clock
C1	BCLK1(C)	TCLK1(C) / TCLK2(C) / TCLK3(C)
D1	VCLK1	TCLK1(D) / TCLK2(D) / TCLK3(D)

* (C) refers to clock phase and (D) refers to data phase

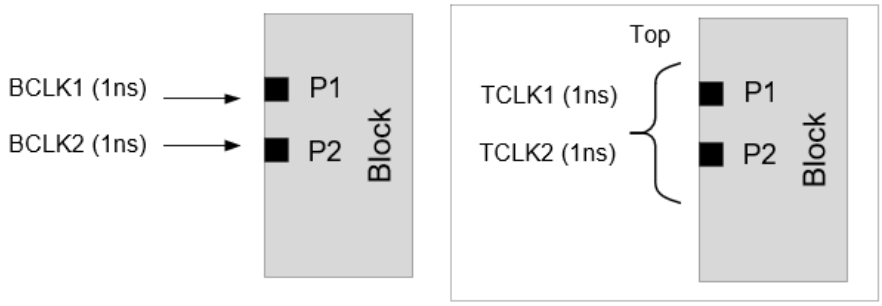
In this case, the worst-case latency of TCLK1/TCLK2/TCLK3 will be applied to BCLK1 at port C1 and worst-case arrivals of TCLK1/TCLK2/TCLK3 will be applied to VCLK1 at port D1 in context analysis. In top level analysis, separate paths will be reported with respect to TCLK1 / TCLK2 / TCLK3 clocks, while context analysis will report paths only with respect to BCLK1 / VCLK1. The worst casing of clock latency and arrivals can cause pessimism due to wider timing windows during context analysis. Also, any mismatch in constraints between TCLK1/ TCLK2/TCLK3 and BCLK1/VCLK1 can cause mis-correlation.

Given below are few other examples:



Block Port	Block Clock	Mapped Top Clock
P1	BCLK1	TCLK1
P2		TCLK2
P3		TCLK3

In above example, the port specific clock latency is applied to BCLK1 in context analysis at ports P1, P2, and P3, and there will be no worst casing of clock-latency.



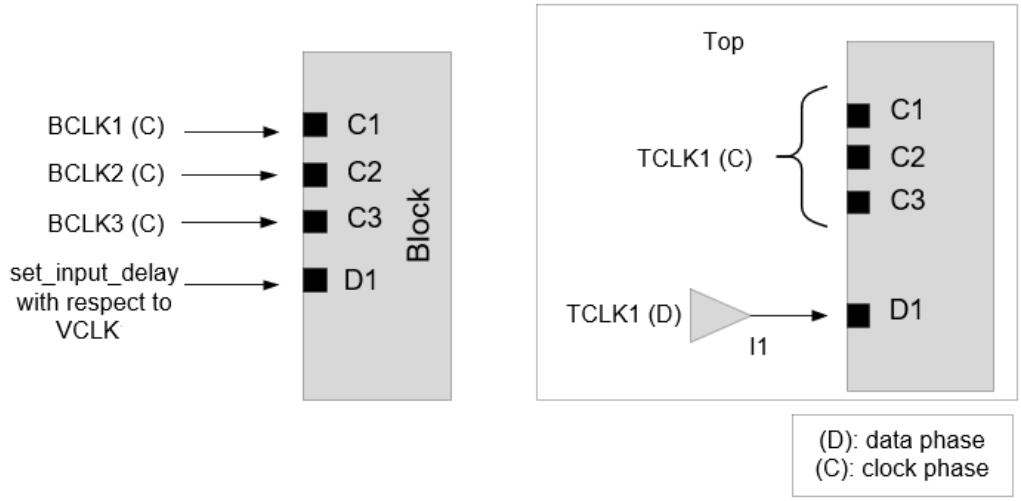
Block Port	Block Clock	Mapped Top Clock
P1	BCLK1	TCLK1 / TCLK2
P2	BCLK2	TCLK1 / TCLK2

In the above example, the port specific worst-cased top level clock latency is applied to BCLK1 and BCLK2 in context analysis at ports P1 and P2, respectively.

Multiple Block Clock to Single Top Clock Mapping

In case of one top clock reaching multiple block ports, with a different block clock defined on each of these block ports in block constraints, Tempus automatically maps different block clock to the same top clock on different block ports if the clock property matches. The top level data phases will be mapped (independent of clock phase mapping) to the block clock that constrains the data port. In case of clock property mismatch, manual clock mapping needs to be specified by the user.

Consider the below example:



In above example, automatic clock mapping will be done if period/waveform of TCLK1 and BCLK1/BCLK2/BCLK3/VCLK match. The table below gives the resultant mapping:

Block Port	Block Clock	Mapped Top Clock
CI	BCLK1(C)	TCLK1(C)
C2	BCLK2(C)	
C3	BCLK3(C)	
D1	VCLK	TCLK1(D)

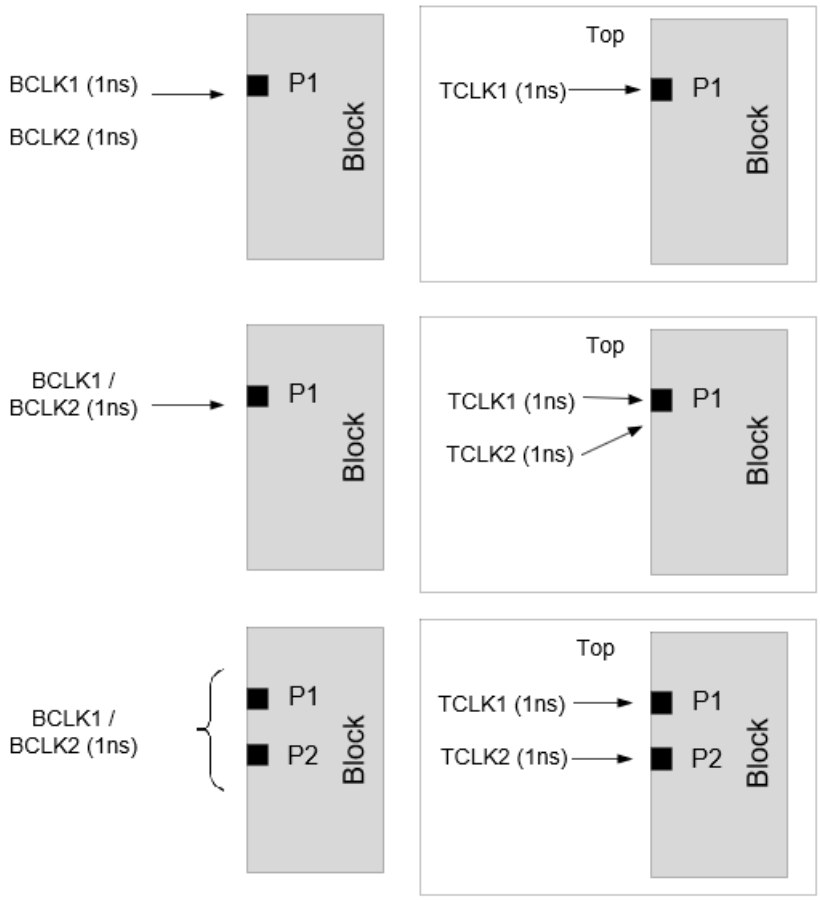
* (C) refers to clock phase and (D) refers to data phase

The port specific top level clock TCLK1 latency will be applied in context analysis for BCLK1/ BCLK2/BCLK3 clocks.

In the above example, any path exception "-to TCLK1" on input buffer I1 connected to D1 port in top level STA will be applied to BCLK1/BCLK2/BCLK3 clocks (for paths originating from port D1 only) in context analysis.

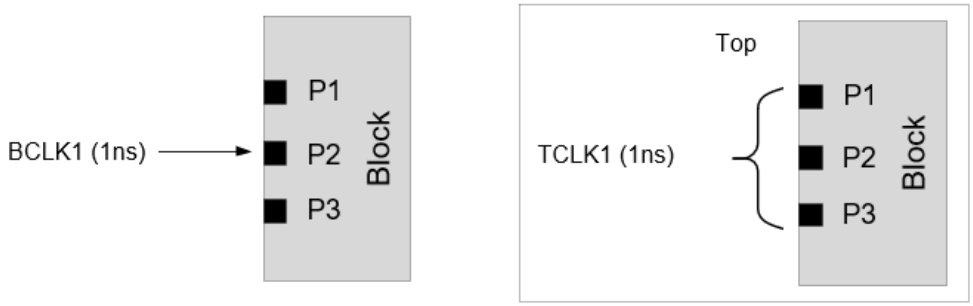
If there are multiple block clocks that can be mapped to the same top clock on the same block port, then the software will not perform auto mapping.

Consider the following examples:



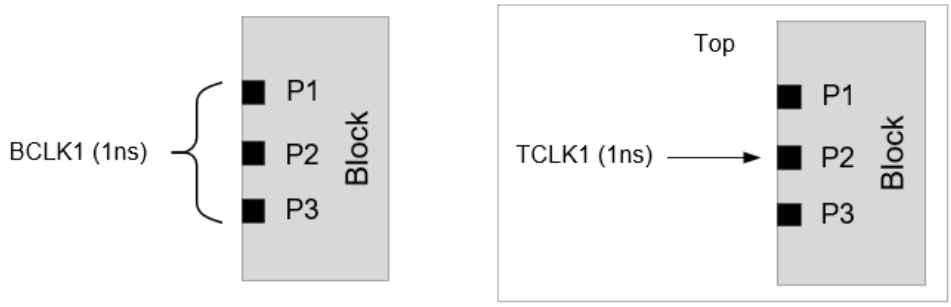
In the above examples, all the clocks will remain unmapped and will require manual clock mapping specification.

Example 1: Clock Phase Mapping



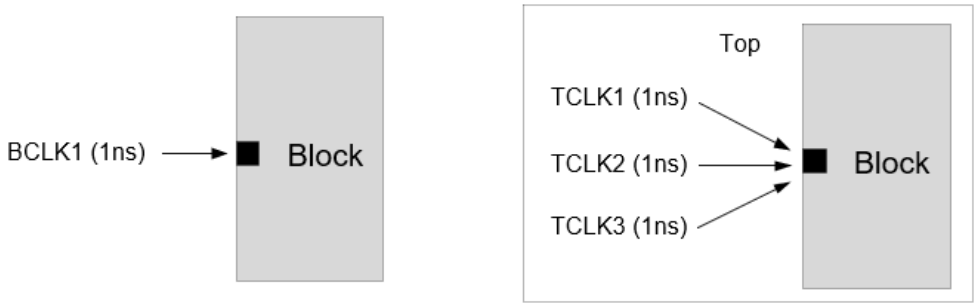
Expected clock mapping behaviour:

- TCLK1 will be mapped to BCLK1 at port P2
- TCLK1 will be unmapped at port P1 and P3.
- Clock would propagate only through P2. No clock propagation on P1 and P3



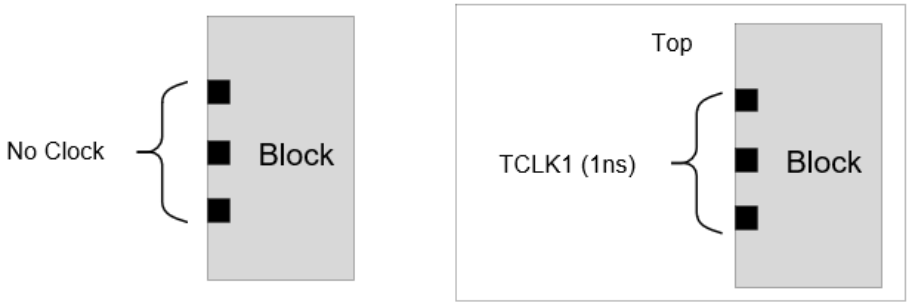
Expected clock mapping behaviour:

- TCLK1 will be mapped to BCLK1 at port P2
- BCLK1 will be unmapped at port P1 and P3.
- Clock would propagate only through P2. No clock propagation on P1 and P3



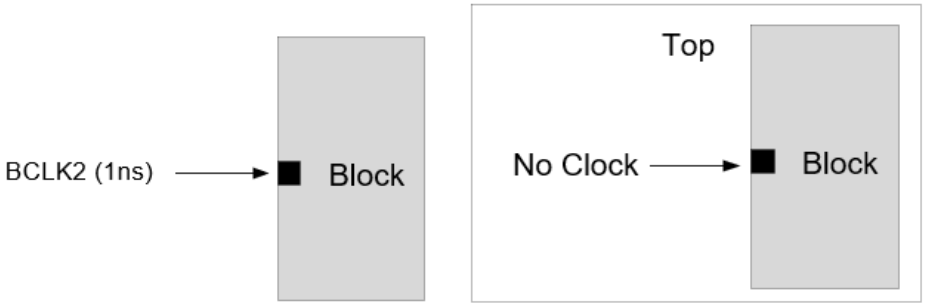
Expected clock mapping behaviour:

- TCLK1 will be mapped to BCLK1 only
- User needs to specify manual N-1 clock mapping when property does not match



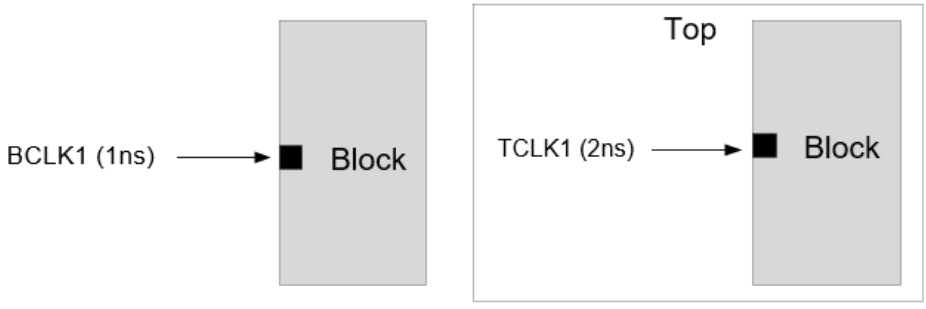
Expected clock mapping behaviour:

- TCLK1 dropped from context analysis.



Expected clock mapping behaviour:

- BCLK2 dropped from context analysis

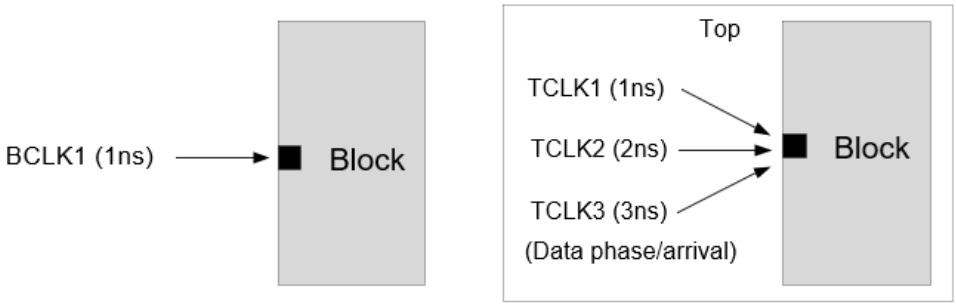


Expected clock mapping behaviour:

- BCLK1/TCLK1 will not be mapped

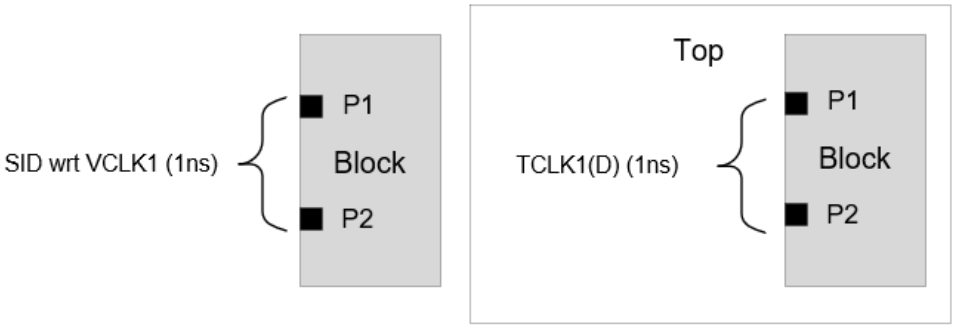
- User needs to specify manual 1-to-1 clock mapping

Example 2: Data Phase Mapping



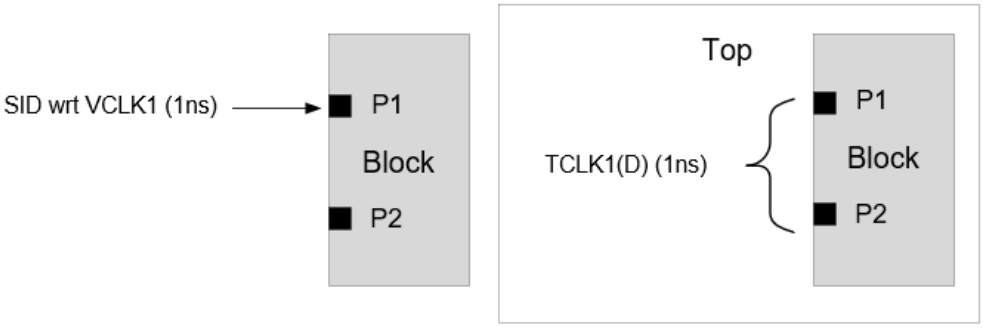
Expected clock mapping behaviour:

- BCLK1 mapped to TCLK1 only



Expected clock mapping behaviour:

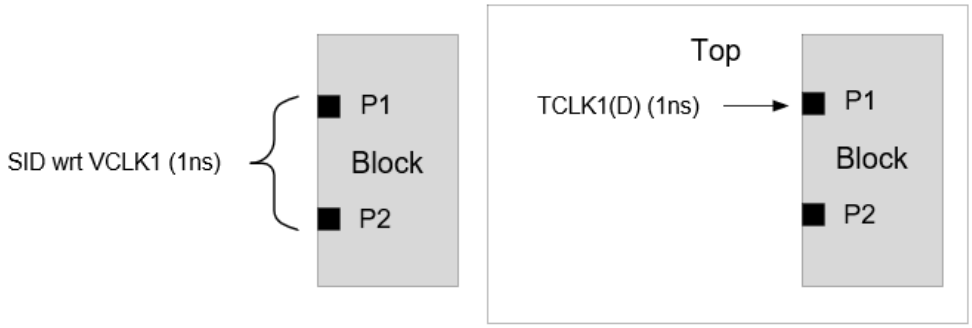
- TCLK1(D) mapped to VCLK1 at ports P1 and P2



Expected clock mapping behavior:

- TCLK1(D) mapped to VCLK1 only at port P1

- TCLK1(D) unmapped at ports P2 (and no arrival propagation from this port



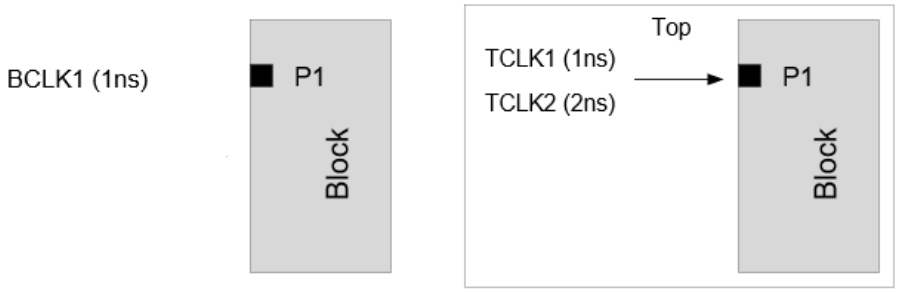
Expected clock mapping behaviour:

- TCLK1(D) mapped to VCLK1 only at port P1
- VCLK1 will be unmapped at port P2 (and no arrival propagation from this port)

4.16.6 Manual Clock Mapping Specification

To manually map multiple top clocks to a single block clock, Tempus supports the `set_timing_context_clock_mapping -add` command parameter. The multiple 1-to-1 clock mappings can be used to specify one block clock to multiple top clock mappings with the `-add` parameter. The manual clock mapping applies to both the clock and data phases. For example:

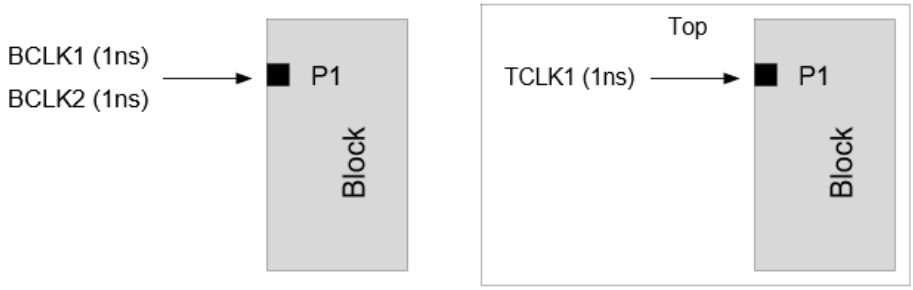
Example 1: To map block clock BCLK1 to both TCLK1 / TCLK2, the `-add` parameter is used:




```
set_timing_context_clock_mapping -block_clock BCLK1 -top_clock TCLK1 -view <view>
set_timing_context_clock_mapping -block_clock BCLK1 -top_clock TCLK2 -view <view>
\ -add
```

The block clock mapping specified without the `-add` parameter will override the previous mappings done for that block clock. In this example, without the `-add` parameter, BCLK1 will be mapped to TCLK2 only.

Example 2: No clock (BCLK1 or BCLK2) will be auto-mapped due to multiple matching block clocks. In this example, the block clock BCLK1 is mapped to top clock TCLK1, and block clock BCLK2 is mapped to top clock TCLK1.



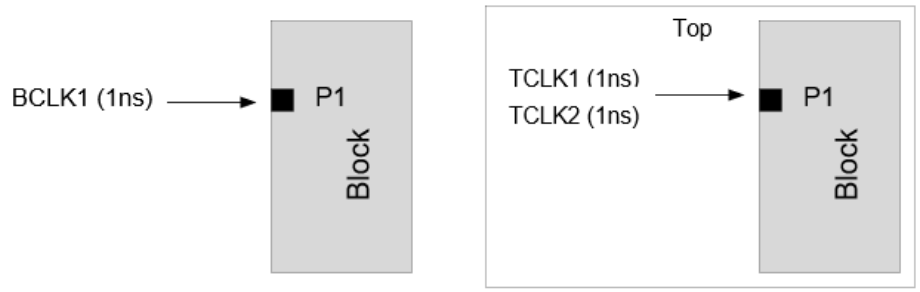
You will need to provide manual mapping commands as given below:

```
set_timing_context_clock_mapping -block_clock BCLK1 -top_clock TCLK1 -view <view>
set_timing_context_clock_mapping -block_clock BCLK2 -top_clock TCLK1 -view <view>
```

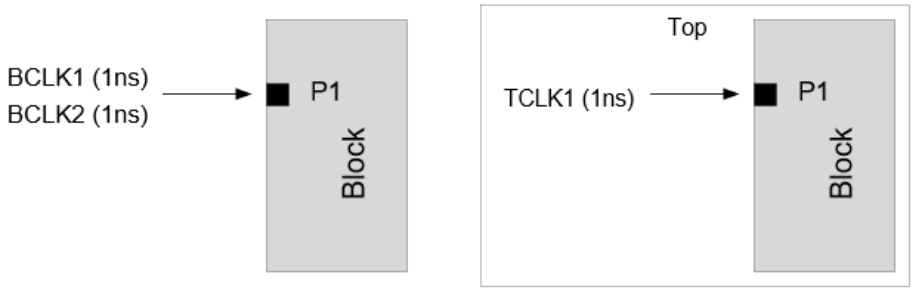
The `-add` parameter is not required since each block clock maps to only one top clock. The auto-clock mapping will not be attempted on any block clock for which manual mapping is user-specified. If a block clock is to be mapped manually, you should specify all the clock mappings applicable for that block clock.

The following examples illustrate some cases where manual clock mapping is required.

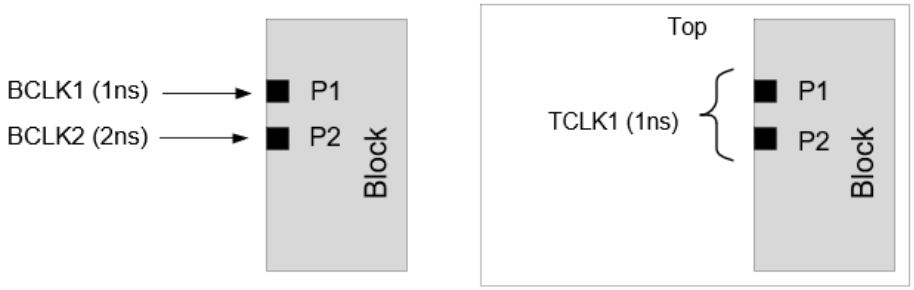
Example 3: By default, the software will map BCLK1 to TCLK1 (due to matching property) only. To map BCLK1 to TCLK2 also, you can manually map BCLK1 to both TCLK1 and TCLK2 with the `-add` parameter:



Example 4: By default, the software will map BCLK1 to TCLK1 only (because of matching property), and BCLK2 will remain unmapped. If BCLK2 is manually mapped to TCLK1, then BCLK1 will remain unmapped.



Example 5: By default, the software will map BCLK1 to TCLK1 and BCLK2 will remain unmapped. If BCLK2 is manually mapped to TCLK1 (on Port P2), then BCLK1 will be auto mapped to TCLK1 (on Port P1).



4.16.7 Clock Mapping Reporting

Tempus creates clock mapping report (*timing_context_clock_mapping.rpt*) while reading the timing context (`read timing context` command).

This file honors `auto_file_dir` and `auto_file_prefix` attributes.

It is important to review the *timing_context_clock_mapping.rpt* file for details on mapped and unmapped clocks. The clock mapping report can also be generated using the `report_timing_context_clock_mapping` command.

The clock mapping report has the following six sections:

- Section1: Mapped Block Clocks
- Section2: Unmapped Block Clocks
- Section3: Unmapped Top Clocks

- Section4: Block Level Unmapped Virtual Clocks
- Section5: Mapped Block Clocks on Data Ports
- Section6: Unmapped Top Clocks on Data Ports

Section1: Mapped Block Clocks

This section reports the block-level clocks and the corresponding mapped top level clocks. It also prints the usage, indicating whether clock enters as clock or data or used for exception modeling.

Below snippet indicates that block-level clock bclk is mapped to top level clock tclk. Usage {C} indicates that the bclk enters the block as clock phase at port b_clk.

```
Info:     Block clocks mapped to top clocks on clock pins/ports:
-------------------------------------------------------------------------------

Block Clock   Block Clock Waveform    Top Clock   Top Clock Waveform
       Top View   WF Match   Usage     Pins/Ports   Type    View
-------------------------------------------------------------------------------
bclk         200.000{0.000 1000.000}  tclk        200.000{0.000 1000.000}
       view1      yes        {C I O}   b_clk         auto    view1
-------------------------------------------------------------------------------
```

The following are the various values of usage:

Usage	Explanation
C	Top clock enters block clock ports as clock.
I	Top clock enters block data input ports as launching clock.
O	Top clock enters block data output ports as capturing clock.
R	Top clock enters block data output ports as reference clock to outside path checks.
E	Top clock enters block data ports as -from/to clock in path exceptions.

Section2: Unmapped Block Clocks

This section reports the block-level clocks that were not mapped with any top level clock. It also shows a diagnostic message for the possible reason of unsuccessful mapping.

The following example snippet indicates that the block level clock `cclk` cannot be mapped with any top level clocks, because any top level clock with a property matching with `cclk` was found at port `c_clk`:

```
Info:     Block clocks NOT mapped to top clocks on clock pins/ports:
--------------------------------------------------------------------------------
Block Clock  Block Clock Waveform     Mapping Result
         Pins/Ports      View
--------------------------------------------------------------------------------
cclk         500.000{0.000 2500.000}  No possible match by parameter/
name
         c_clk           view1
--------------------------------------------------------------------------------
```

Section3: Unmapped Top Clocks

This section reports the top level clocks that were not mapped with any block level clock. It also has the diagnostic message for possible reason of unsuccessful mapping.

The snippet shown below indicates that the top level clock `atclk` cannot be mapped with any block level clocks at the port `atport`.

```
Info:        Top clocks NOT mapped to block clocks on clock pins/ports:
--------------------------------------------------------------------------------
Top Clock    Top Clock Waveform        Usage     Mapping Result
      Pins/Ports      View
--------------------------------------------------------------------------------
atclk        4000.000{0.000 2000.000}  {C}       No possible match by parameter/
name   atport         view1
--------------------------------------------------------------------------------
```

Section4: Block Level Unmapped Virtual Clocks

This section reports the block level virtual clocks which could not be mapped with any top-level clock.

```
Info:        Virtual block clocks NOT mapped to top clocks:
--------------------------------------------------------------------------------
Block Clock      Block Clock Waveform           View
--------------------------------------------------------------------------------
v_bclk           10000.000{0.000 5000.000}      view1
--------------------------------------------------------------------------------
```

Section5: Mapped Block Clocks on Data Ports

This section reports the block-level clocks mapped to top level clocks on data ports.

The following example indicates that the block-level clock `coreclk` is mapped to top level clock `coreclk` on data port `coreclk`.

```
Info:     Block clocks mapped to top clocks on data ports:
--------------------------------------------------------------------------------
Block Clock   Block Clock Waveform    Top Clock   Top Clock Waveform
       Top View   WF Match   Usage     Ports   Type      View
--------------------------------------------------------------------------------
```

```
coreclk       740.000{0.000 370.000}  coreclk     740.000{0.000 370.000}
       view1      yes        {C I}    coreclk     auto   view1
-------------------------------------------------------------------------------
```

Section6: Unmapped Top Clocks on Data Ports

This section reports the top-level clocks that was not mapped to any block level clock. It also has shows a diagnostic message for the possible reason for unsuccessful mapping.

The example snippet below indicates that the top-level clock `t_clk` cannot be mapped with any block level clocks at port `t_clk`.

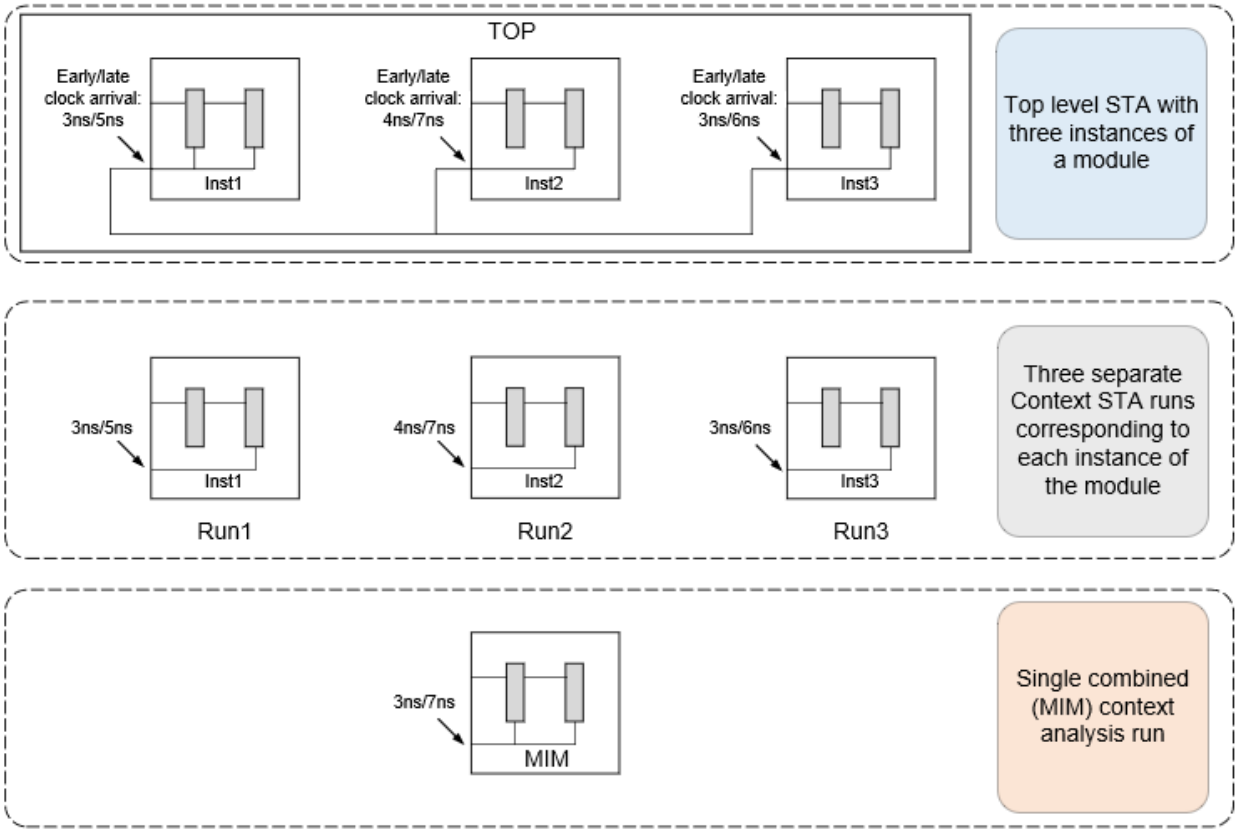
```
Info:        Top clocks NOT mapped to block clocks on data ports:
-------------------------------------------------------------------------------
Top Clock    Top Clock Waveform        Usage         Mapping Result
                 Ports      View
-------------------------------------------------------------------------------
t_clk        4000.000{0.000 2000.000}  {C I O}       No possible match by
parameter/name   t_clk      view1
-------------------------------------------------------------------------------
```

4.16.8 MIM Context Analysis

Tempus supports Context analysis of multiple instances of blocks in a single STA run. This is referred to as MIM (multi-instantiated module) context analysis. The MIM context analysis is more efficient than performing multiple STA runs corresponding to each instance of a block.

This is illustrated in the figure below:

Figure 4-126 MIM Context Analysis Illustration



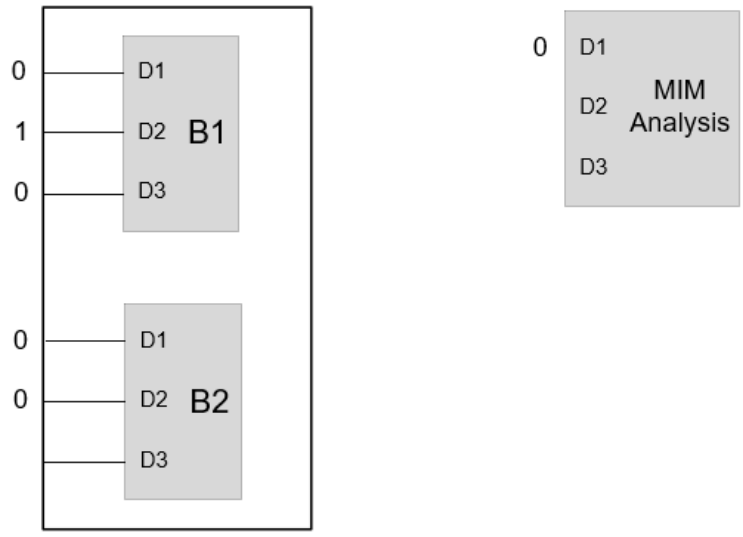
The MIM Context analysis will use worst-cased latency across all the instances of the block. For example, in the above figure, early latencies for Inst1, Inst2, and Inst3 of the block module are 3, 4, and 3ns respectively. Similarly, the late latencies for each of the instances are 5, 6, and 7ns respectively. MIM analysis will use the worst early (3ns) and worst late (7ns) latencies.

The following boundary constraints are worst-cased in MIM Context analysis:

- Latency
- Input transition
- Output loads

The top level constants propagating to data ports will only be modeled in MIM context analysis, if the constant on the port is consistent across all the instances. In case of difference

in top level constants across instances, the constant will not be modeled. This is illustrated in the figure below.



Since both B1 and B2 instances of module B have the same constant 0 on port D1, this will be modeled in MIM context analysis. However, on port D2, since B1 has constant value 1 and B2 has constant value 0, hence this is not modeled in MIM context analysis. Similarly, since only B1 has a constant on port D3 and B2 does not have a constant, no constant gets modeled on D3 port in MIM context analysis.

The MIM context analysis for interface paths is enabled by the `timing_context_enable_mim_interface_support` attribute. When set to `true` (default), the input and output interface paths are separately reported for each of the MIM instances.

MIM Clusters

There are constraints requirements for instances of a module to qualify for MIM Context analysis. Tempus creates clusters of instances which meet the clustering requirements in Top STA and MIM context analysis can be performed only for those instances which are part of the same cluster.

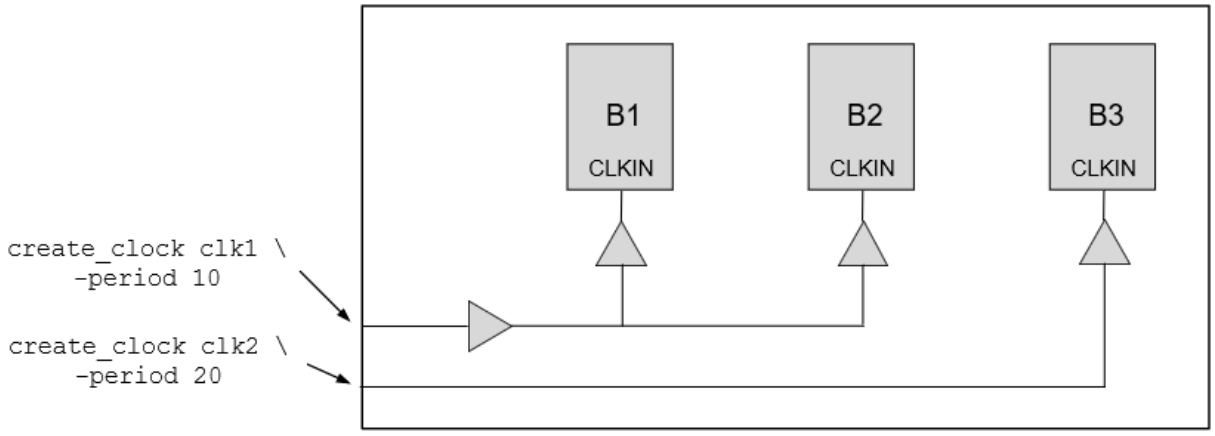
The instances of a module in top STA should meet the following requirements to be clustered in MIM context analysis:

1. All the instances of the block should have the same clock, clock waveform, and polarity at a given block port.
2. Same constants should reach the ports across instances if the port's fanout is interacting with the clock network.

3. All the instances should have the same top level clock constraints reaching the port (pin-based uncertainty, ideal vs propagated clock, etc.)

Tempus will write the *multi_instance_cluster_info.rpt* report file inside the context directory during context generation, which can be referred for details of clusters and instances associated with each cluster. This file has two sections - the first section gives information on instances belonging to different clusters, and the second section gives information on differences across clusters. The example below illustrate how to infer clustering information for a top level design TOP having three instances B1, B2, and B3 of module B.

Example 1: The following example illustrates the case where three instances of B do not meet requirement 1 (see MIM Clusters on page 350) of clustering and hence are made a part of separate clusters:



The MIM context analysis for interface paths is enabled by the `timing_context_enable_mim_interface_support` attribute. When set to `true` (default), the input and output interface paths are separately reported for each of the MIM instances.

The following are the contents of *multi_instance_cluster_info.rpt* file when the context is created for all the three instances of module B:

```
# Cluster definitions
cluster_1: Module: block Instances: {B1 B2}
cluster_2: Module: block Instances: {B3}
# Mismatch map
#Port   |   Cluster      |    MisMatch
CLKIN       cluster_1         clocks: {clk1(P) view1} tags: {Propagated}
CLKIN       cluster_2         clocks: {clk2(P) view1} tags: {Propagated}
```

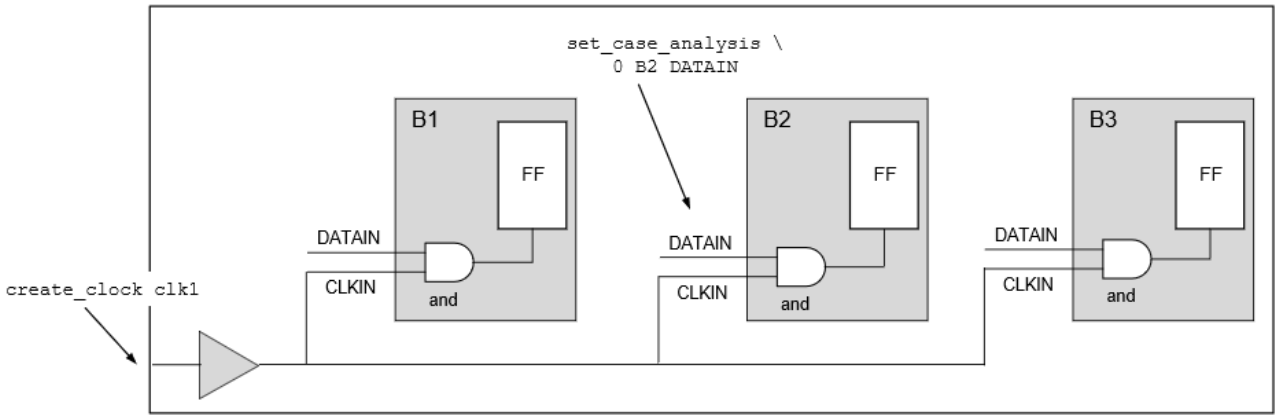
The first section indicates that the two clusters are created for the block. The instances `B1` and `B2` are part of `cluster_1` and instance `B3` is part of `cluster_2`.

The second section indicates that the top level clock `clk1` enters CLKIN port of instance(s) of `cluster_1`, whereas top level clock `clk2` enters the CLKIN port of instance(s) of `cluster_2`. In this case, only instances `B1` and `B2` can be analyzed together in MIM context analysis.

Example 2: The following example illustrates a case where three instances of B do not meet requirement 2 (see MIM Clusters on page 350) of clustering and hence these are made a part of separate clusters. The clustering report in this case is the same as Example 1.

```
# Cluster definitions
cluster_1: Module: cppr_block Instances: {B1 B3}
cluster_2: Module: cppr_block Instances: {B2}
# Mismatch map
#Port    |    Cluster      |       MisMatch
CLKIN         cluster_1            clocks:{ clk(P) view1 }  tags:{Propagated}
```

In this case, the propagation of top level clock `clk` is stopped within `B2` instance by applying constant 1 at the `DATAIN` port. Hence, the instance `B2` is not a part of the same cluster as `B1` or `B3` instances. The instances B1 and B3 only can be analyzed together in MIM context analysis.



4.16.9 Context Generation and Analysis in MMMC

In MMMC analysis, the `write_timing_context` command generates the context for all the active STA views in top level analysis.

In MMMC context analysis, it is required that context is specified for all the active STA views. The multiple contexts corresponding to the different views can be specified with the `read timing context -merge` parameter. The input context specified can be a mix of MMMC and SMSC contexts. The contexts specified as input for views that are not active in context analysis will be ignored. By default, the software applies context data to each view

based on the matching top and block view names. In case the top and block view names do not match, Tempus provides the capability to map the top level view names with the corresponding block level view names using the `-view_map_file` parameter.

4.17 Wire Variation Analysis

4.17.1 Wire Variation Analysis - Overview on page 355

4.17.2 Performing Wire Variation Analysis on page 358

4.17.3 Statistical Wire Variation Analysis on page 360

4.17.4 Binomial Wire Variation Analysis on page 361

4.17.5 Wire Variation Reporting on page 362

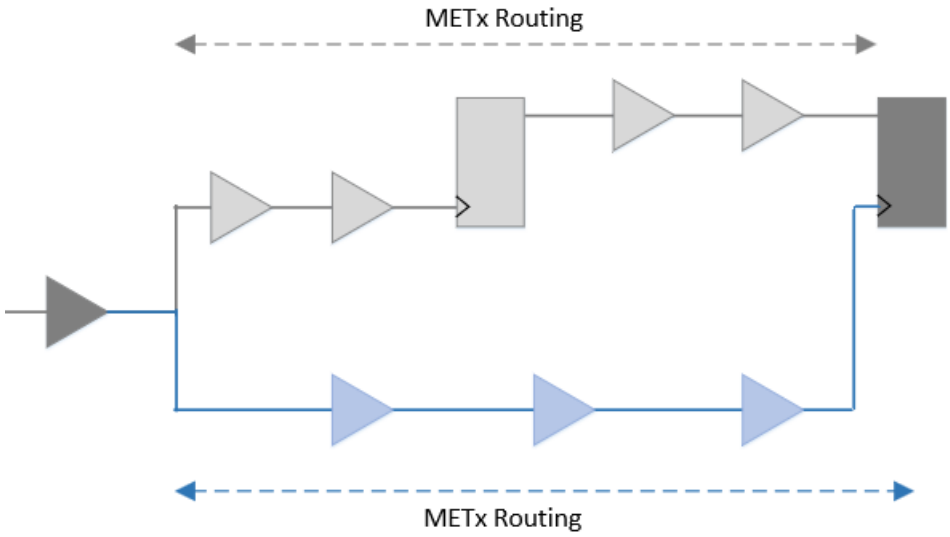
4.17.1 Wire Variation Analysis - Overview

The RC variation is modeled by performing STA signoff on multiple global parasitic extraction corners. These corners assume all the layers to be fully correlated. But the probability of these layers getting manufactured at the same corner is very less.

Once fabricated, the different metal layers may exhibit different variations. Such mis-tracking between layers might cause timing skew imbalance, thereby introducing new timing violations not seen earlier during global RC corner-based STA signoff. To report these missed violations, Tempus PBA solution supports wire variation analysis feature that comprehends the impact of layer mis-tracking.

Consider the following timing path example (see figure below), where the launch path is dominantly routed by METx, while the capture clock path is dominantly routed by METy. In such scenarios, the timing degradation induced by such interconnect skew may not get caught through global RC corner STA signoff.

Figure 4-127 Wire Variation Analysis



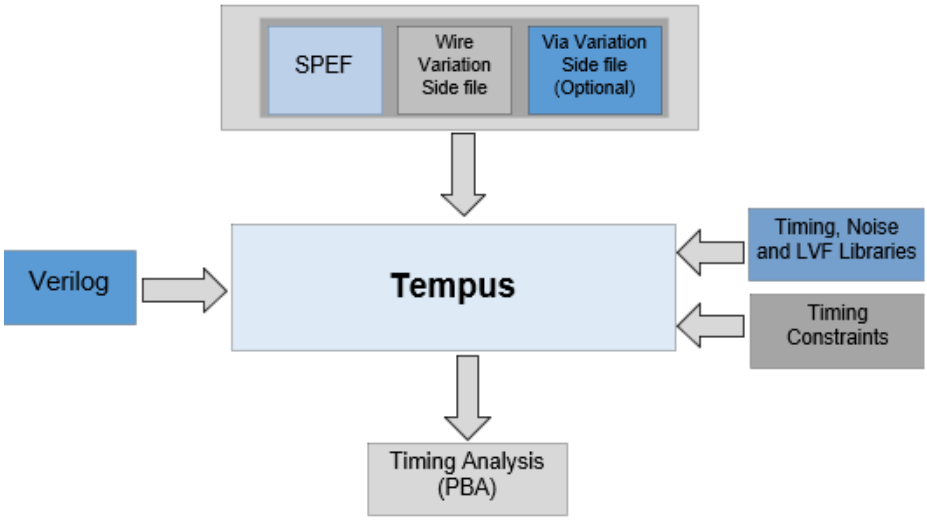
Note: Tempus wire variation solution is supported in PBA mode only - GBA, exhaustive-PBA and infinite-PBA are not supported. You can run regular PBA with high nworst and maximum slack to a value greater than 0 to cover all the violations.

Required Inputs

Along with the required STA inputs, the following two inputs are required for performing wire variation analysis:

- Extended SPEF containing layer information
- Wire variation side file

Figure 4-128 Inputs



Here is an example of an extended SPEF file, which contains the association of each resistor and capacitor to the corresponding metal layers:

```
(skipped)

// *LAYER_MAP
// *0 PWELL
// *1 M8
// *2 VIA7
// *3 M7
// *4 VIA6
// *5 M6
// *6 VIA5
// *7 M5
// *8 VIA4
// *9 M4
// *10 VIA3
// *11 M3
// *12 VIA2
// *13 M2
// *14 VIA1
// *15 M1
```

```
// *16 CO
// *17 PO
(skipped)

*D_NET *397718 1.09503
*CONN
*P hrdata_mp2[13] O *C 712.92 1077.97 // $llx=712.92 $lly=1077.97 $urx=712.92
$ury=1077.97 $LAYER=11
*I *19765:X O *C 712.875 1068.42 // $llx=712.875 $lly=1068.42 $urx=712.875
$ury=1068.42 $LAYER=15
*N *397718:1 *C 712.962 1068.42 // $llx=712.962 $lly=1068.42 $urx=712.962
$ury=1068.42 $LAYER=13
*N *397718:2 *C 712.985 1068.42 // $llx=712.985 $lly=1068.42 $urx=712.985
$ury=1068.42 $LAYER=11
*CAP
0 *397718:1 0.0573391
1 *397718:2 1.03769
*RES
0 hrdata_mp2[13] *397718:2 36.8198 // $L=8.595 $W=0.0476 $LAYER=11
1 *397718:1 *397718:2 5.2 // $A=0.00405 $LAYER=12
2 *19765:X *397718:1 10.4 // $A=0.002025 $LAYER=14
*END
```

An example of wire variation side file is as follows, which specifies the mean-shift and standard deviation of R and C of each metal layer. Each value represents a ratio of the nominal R/C obtained from the SPEF file.

```
version :1.0

layer_name                      :   M1
resistance_meanshift            :   -0.150
resistance_stddev               :   0.050
capacitance_meanshift           :   0.105
capacitance_stddev              :   0.035
coupling_capacitance_meanshift  :   0.105
coupling_capacitance_stddev     :   0.035

layer_name                      :   M2
resistance_meanshift            :   -0.120
resistance_stddev               :   0.030
capacitance_meanshift           :   0.095
capacitance_stddev              :   0.035
(skipped)
```

If coupling capacitance mean-shift and stddev factors are not explicitly specified for any metal layer, then capacitance mean-shift and stddev factors from that layer are used for xcap as well.

The extended SPEF and wire variation side file can be loaded using the `read_spef` command, as shown below:

```
read_spef -extended <extended SPEF path> \
    -wire_variation_file <wire variation side file path>
```

4.17.2 Performing Wire Variation Analysis

The following setting enables wire variation analysis in path-based analysis mode:

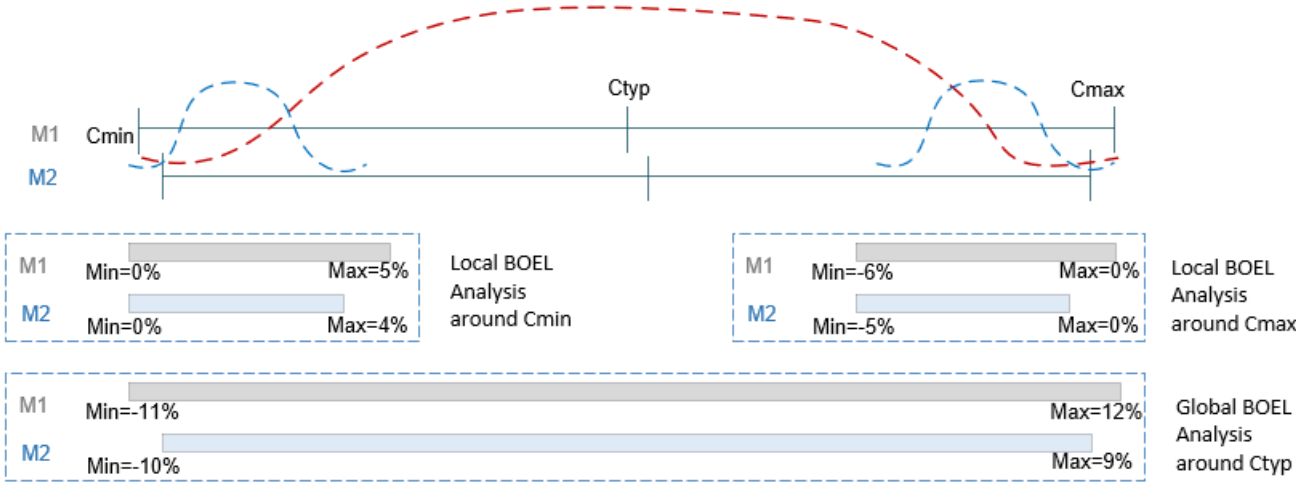
```
set timing_enable_wire_variation true
```

By default, the clock path PBA is disabled in Tempus. To model wire variation on clock paths, you can enable clock PBA using the following setting:

```
set timing_disable_retime_clock_path_slew_propagation false
```

The wire variation analysis can be run to model local as well as global RC variations. Consider the following illustration:

Figure 4-129 Wire variation analysis illustration



In the above example, there can be two use models:

1. **Local variation**: Consider the blue curves above, where wire variation analysis can be run at Cmin or Cmax corners, and metal layers' mean-shift and standard deviation can be accordingly defined by assuming the specified range to be +/-n*sigma.

For local variation analysis around Cmin corner:

M1 mean-shift = (0% + 5%)/2 = 2.5%

M1 stddev = (5% - 0%)/6 = 0.83%

M2 mean-shift = (0% + 4%)/2 = 2%

M2 stddev = (4% - 0%)/6 = 0.67%

2. **Global variation**: Consider the red curve above, where wire variation analysis can be run at Ctyp, and metal layers' mean-shift and standard deviation can be accordingly defined by assuming the specified range to be +/-n*sigma.

   For global variation analysis around Ctyp corner:

   M1 mean-shift = (-11% + 12%)/2 = 0.5%

   M1 stddev = (12% - (-11%))/6 = 3.83%

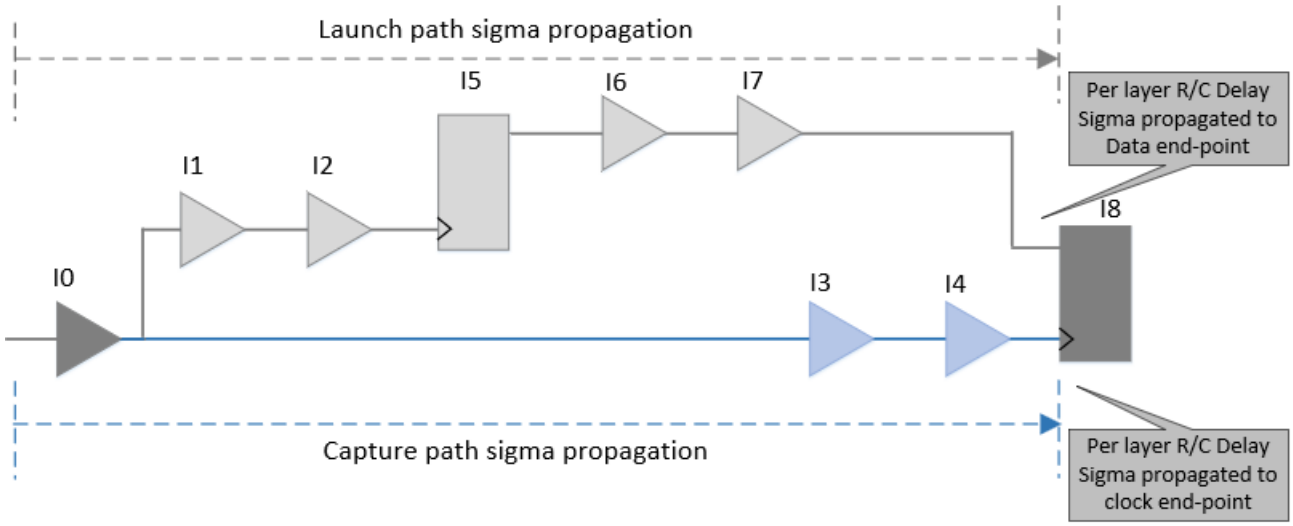
   M2 mean-shift = (-10% + 9%)/2 = -0.5%

   M2 stddev = (9% - (-10%))/6 = 3.17%

The following steps outline the internal working of Tempus to compute wire variation-aware timing slacks:

1. Compute delay sensitivities (sigma) of each stage with respect to resistance and capacitance of each metal layer.
2. Propagate R/C sensitivities per layer separately across launch and capture paths.
   - Resistance/capacitance delay variations within a layer across all stages are assumed to be fully correlated, that is, the delay sigma is linearly added for the same layer of R or C.

Figure 4-130



3. The timing slack is computed by combining the delay sensitivities across different components for launch and capture:

   - The R and C of the same layer are combined based on the correlation factor specified by the user, to get the combined delay sensitivities for each layer:

     `set_db delaycal_wire_rc_correlation_factors < -1 | 1 >`

     With '-1', R and C of same layer are considered as inversely correlated and with '1', then R and C of the same layer are considered as fully correlated.

   - Combined slack wire sensitivity across layers is then computed based on the correlation mode specified by the user:

     `set_db delaycal_wire_correlation_type <statistical | binomial>`

The subsequent sections explain the statistical and binomial modes.

4.17.3 Statistical Wire Variation Analysis

In statistical wire variation analysis, Tempus assumes different metal layers to be independent of each other. For wire variation analysis, the following are considered:

- Slack sigmas across layers are root-sum-squared to get combined slack wire sigma.
- Wire sigma is also statistically combined with random-cell (SOCV) sigma.

The command below is used to enable statistical mode:

```
set_db delaycal_wire_correlation_type statistical
```

4.17.4 Binomial Wire Variation Analysis

The binomial wire variation analysis performs bounding analysis, where reported timing slack is targeted to bound all different combinations of the possible RC corners.

Consider the table below, where 2^n combinations are possible for 'n' metal layers. Tempus reports the timing slack, which is worst across all 2^n combinations.

Skewed RC corner	**M1**	**M2**	**…**	**Mn**
1	min	min		min
2	min	min		max
3	min	max		min
4	min	max	…	max
5	max	min		min
6	max	min		max
7	max	max		min
…	…	…	…	…
2^n	max	Max		max

In contrast to statistical mode, which assumes the wire variation to be Gaussian distribution with given mean and sigma, the binomial mode assumes only two possibilities for each metal layer, it can be either at mean + n*sigma, or at mean - n*sigma.

You can enable binomial mode using the following setting:

```
set_db delaycal_wire_correlation_type binomial
```

By default, the wire variation slack sigma is statistically added (root-sum-squared) to the random-cell (SOCV) slack sigma. But to completely bound different skewed RC corners, the settings (as shown below) can be made to consider the wire variation and random-cell variation to be correlated, which will linearly add the wire and cell sigma instead of adding statistically:

```
set_db delaycal_wire_wire_cell_variation_correlation 1
```

Additionally, Tempus needs to perform bounding analysis to use n*wireSigma clock and data pins slews at endpoints to look-up the timing checks:

```
set_db delaycal_wire_use_nsigma_slew_for_timing_check true
```

Use Model

The following is a sample legacy UI script to run wire variation analysis in statistical mode:

1. Load the timing library containing noise data:

   ```
   read_libs timing.lib
   ```

2. Schedule reading of a structural, gate-level Verilog netlist:

   ```
   read_netlist top.v
   ```

3. Set the top module name, and check for consistency between Verilog netlist and timing libraries:

   ```
   init_design top
   ```

4. Set the proper technology node:

   ```
   set_db -process 7
   ```

5. Load the timing constraints:

   ```
   read_sdc constraints.sdc
   ```

6. Load extended SPEF and wire variation side file:

   ```
   read_spef -extended top.spef.gz -wire_variation_file wireVar.txt
   ```

7. Enable SOCV analysis:

   ```
   set_db timing_analysis_socv true
   ```

8. Perform timing analysis:

   ```
   update_timing -full
   ```

9. Enable wire variation analysis in PBA mode:

   ```
   set_db timing_enable_wire_variation true
   ```

10. Configure wire variation analysis:

    ```
    set_db delaycal_wire_correlation_type statistical
    set_db delaycal_wire_rc_correlation_factor < -1 | 1 >
    ```

11. Run path-based analysis to report wire variation-aware slacks:

    ```
    report_timing -retime path_slew_propagation
    ```

4.17.5 Wire Variation Reporting

The following additional timing report fields (in `report_timing` command output) have been introduced to print wire variation details in a timing path report:

- `-retime_slew_wire_stddev`

- `-retime_delay_wire_stddev`
- `-arrival_wire_stddev`

Additionally, the `report_timing -retime_delaycal_pins` parameter prints the slew and delay sigma per layer for user-specified pins.

Here is a sample PBA timing report:

```
tempus > report_timing -from FF1/QN0 -to FF2/D0 -retime_delaycal_pins NAND_inst1/B

Path 1: MET Setup Check with Pin FF2/CK
Endpoint: FF2/D0                    (^) checked with leading edge of 'myclk'
Beginpoint: FF1/QN0 (v) triggered by leading edge of 'myclk'
Path Groups: {reg2reg}
Analysis View: viewl
Retime Analysis {Data Clock Path-Slew Spatial(CS) SOCV(Ultra)-Corr RC SI WP NCPPR
BOEL}
Retime SOCV Analysis {Distance(um) : 323.616}
                            NSigma          | Mean     | Sigma    |  Stddev
Other End Arrival Time      1.0369 (o3.00S) | 1.0661   | 0.0098   |  0.0098
Setup                       0.0149 (+3.00S) | 0.0141   | 0.0003   |  0.0003
+ Phase Shift               1. 4706
+ CPPR Adjustment           0.0516 (+3.00S) | 0.0297   | 0.0073   |  0.0073
Uncertainty                 0.0441
Required Time               2.4886 (·3.00S) | 2.5082   |  0.0065  |  0.0065
Arrival Time                1.7370 (+3.00S) | 1.6516   |  0.0285  |  0.0269
Slack Time                  0.7744 (·3.00S) | 0.8566   |  0.0274  |  0.0257
Slack Time(original)        0.6467 (·3.00S) | 0.7461   |  0.0331  |  0.0331
     Clock Rise Edge                 0.0000
     + Clock Network Latency (Prop)  0.0057 (+3.00S) | 0.0057 | 0.0000 | 0.0000
     = Beginpoint Arrival Time       0.0057 (+3.00S) | 0.0057 | 0.0000 | 0.0000

Load    Retime  Retime  Retime  Retime  Arrival  Arrival Edge Cell Pin
        Slew    Slew    Delay   Delay   Wire     Time
        Wire            Wire            Std09dev

        Stddev          Stddev
0.0031  0.0000  0.0158  -       -       0.0000   0.0057  ^   -    myclk
0.0031  0.0000  0.0160  0.0000  0.0008  0.000    0.0065  ^   INV  INVinstl/A
0.0050  0.0000  0.0092  0.0000  0.0081  0.0000   0.0146  V   INV  INV=instl/Y
(skipped)
0.0223  0.0001  0.0460  0.0001  0.0044  0.0067   1.0915  ^   ICG  ICGinstl/CK
```

0.0190	**0.0000**	0.0484	**0.0000**	0.0562	**0.0067**	1.1424	^	ICG	ICG=ins/ECK
0.0190	**0.0001**	0.0506	**0.0001**	0.0032	**0.0068**	1.1454	^	FF	FFl/CK
0.0088	**0.0000**	0.0987	**0.0000**	0.0976	**0.0068**	1.2290	V	FF	FF1/QN0
0.0088	**0.0007**	0.1000	**0.0006**	0.0222	**0.0070**	1.2499	V	BUF	BUF instl/A
0.0061	**0.0000**	0.1608	**0.0000**	0.1320	**0.0070**	1.3603	V	BUF	BUF-instl/Y
0.0061	**0.0009**	0.1616	**0.0006**	0.0146	**0.0072**	1.3733	V	BUF	BUF-inst2/A
0.0168	**0.0000**	0.1937	**0.0000**	0.1636	**0.0072**	1.5170	V	BUF	BUF=inst2/Y
0.0168	**0.0016**	0.1952	**0.0010**	0.0329	**0.0078**	1.5473	V	NAND	NAND_instl/B
0.0007	**0.0000**	0.0462	**0.0000**	0.0595	**0.0078**	1.5857	^	NAND	NAND_instl/Y
0.0001	**0.0000**	0.0462	**0.0002**	0.0005	**0.0079**	1.5858	^	INV	INV_inst2/A
0.0129	**0.0000**	0.0640	**0.0000**	0.0374	**0.0079**	1.6152	V	INV	INV-inst2/Y
0.0129	**0.0005**	0.0773	**0.0007**	0.0185	**0.0081**	1.6317	V	A0I	ADI_instl/Dl
0.0011	**0.0000**	0.1029	**0.0000**	0.0830	**0.0081**	1.6983	^	A0I	ADI_instl/Y
0.0011	**0.0004**	0.1030	**0.0004**	0.0056	**0.0082**	1.7028	^	AND	AND_instl/A
0.0013	**0.0000**	0.0207	**0.0000**	0.0454	**0.0082**	1.7363	^	AND	AND_instl/Y
0.0013	**0.0002**	0.0209	**0.0002**	0.0013	**0.0082**	1.7370	^	FF	FF2/D0

(skipped)

Net Sigma Calculation	Rise	Delay Sigma	Slew Sigma
Via variation	(3-sigma)	0.00010 ns	
Layer Ml -C	(3-sigma)	0.00000 ns	0.00000 ns
Layer Ml -R	(3-sigma)	0.00000 ns	0.00000 ns
Layer M2 -C	(3-sigma)	0.00014 ns	0.00015 ns
Layer M2 R	(3-sigma)	0.00000 ns	0.00000 ns
Layer M3 C	(3-sigma)	0.00057 ns	0.00024 ns
Layer M3 -R	(3-sigma)	0.00000 ns	0.00000 ns
Layer M4 -C	(3-sigma)	0.00038 ns	0.00025 ns
Layer M4 -R	(3-sigma)	0.00000 ns	0.00000 ns
Layer MS - C	(3-sigma)	0.00063 ns	0.00067 ns
Layer MS -R	(3-sigma)	0.00014 ns	0.00002 ns
Layer M6 - C	(3-sigma)	0.00035 ns	0.00083 ns
Layer M6 -R	(3-sigma)	0.00014 ns	0.00002 ns
Layer M7 - C	(3-sigma)	0.00072 ns	0.00059 ns
Layer M7 -R	(3-sigma)	0.00004 ns	0.00000 ns
Layer MB - C	(3-sigma)	0.00012 ns	0.00030 ns
Layer MB -R	(3-sigma)	0.00003 ns	0.00000 ns
Layer M9 - C	(3-sigma)	0.00236 ns	0.00501 ns
Layer M9 R	(3-sigma)	0.00038 ns	0.00004 ns

```
Total Net Variation    (3-sigma)    0.00270 ns    0.00504 ns
```

5

Concurrent Multi-Mode Multi-Corner Timing Analysis

5.1 CMMMC Overview on page 367
5.2 Multi-Mode Multi-Corner Licensing on page 367
5.3 Configuring Setup for Multi-Mode Multi-Corner Analysis on page 367
5.4 Library Sets on page 368
5.5 Virtual Operating Conditions on page 371
5.6 RC Corner Objects on page 373
5.7 Delay Corner Objects on page 373
5.8 Power Domain Definitions on page 375
5.9 Constraint Mode Objects on page 376
5.10 Constraint Support in Multi-Mode and MMMC Analysis on page 379
5.11 Analysis Views on page 380
5.12 Saving Multi-Mode Multi-Corner Configuration on page 385
5.13 Configuring Setup for Multi-Mode Multi-Corner Analysis on page 367
5.14 Performing Timing Analysis on page 385
5.15 Generating Timing Reports on page 386

5.1 CMMMC Overview

As feature sizes decrease, it becomes difficult to determine the worst-case combinations of device corner (timing libraries and PVT operating conditions), RC corners, and constraint modes at which the designers need to validate timing requirements for a design. Multi-mode multi-corner analysis and optimization provides the ability to configure the software to support multiple combinations of modes and corners, and to evaluate them concurrently.

Multi-mode multi-corner analysis uses a tiered approach for defining the required data. It allows top-level definitions (analysis views) that share common information to be specified by referencing the same lower-level objects. The active analysis views defined in the software represent the different design variations that will be analyzed.

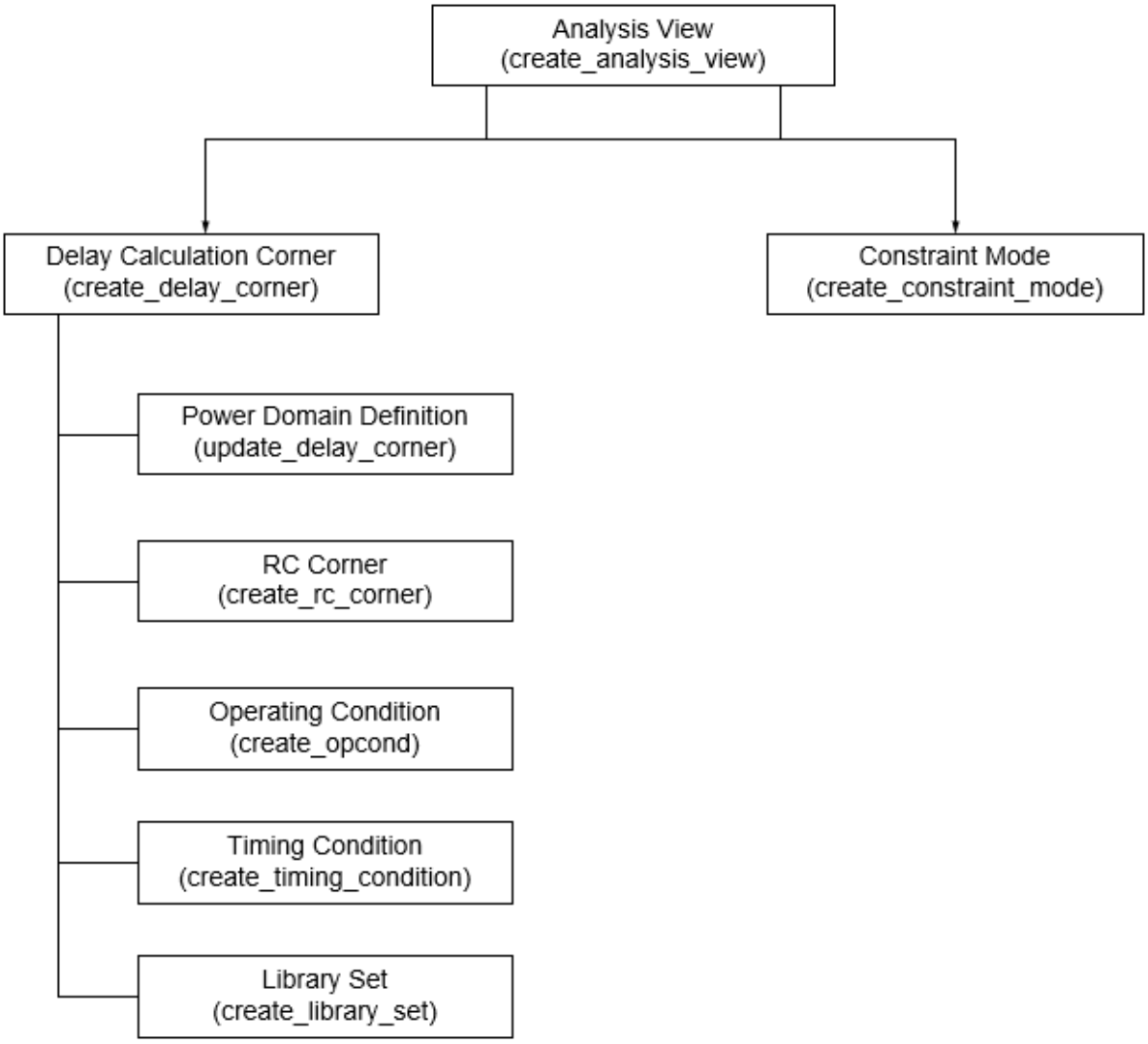
5.2 Multi-Mode Multi-Corner Licensing

Multi-corner analysis requires an XL license. Multi-mode analysis does not require a XL license, and is limited to only one delay corner for setup analysis, and one delay corner for hold analysis.

5.3 Configuring Setup for Multi-Mode Multi-Corner Analysis

Multi-mode multi-corner analysis uses a tiered approach to assemble the information necessary for timing analysis and optimization. Each top-level definition (called an analysis view) is composed of a delay corner and a constraint mode. The active analysis views defined in the software represent the different design variations that will be analyzed.

Figure 5-1 Hierarchical Analysis View Configuration



5.4 Library Sets

This section includes:

- Creating Library Sets
- Updating Library Sets

Creating Library Sets

Complex designs might require the specification of multiple library files to define all of the standard cells, memory devices, pads, and so forth, included in the design. Different library sets can be defined to provide uniquely characterized libraries for each delay corner or power domain.

Library sets allow a group of library files to be treated as a single entity so that higher-level descriptions (delay corners) can simply refer to the library configuration by name. A library set consists of timing libraries and can also include cdB/UDN models, AOCV, SOCV libraries which are supported through various options of `create_library_set` command.

For details on UDN syntax, refer to the *cdB Noise Library Format* section in the *Noise Library Characterization* chapter of the *make_cdB Noise Characterizer User Guide*.

For more information on UDN model, refer to the *User-Defined Noise Model - An Overview* section in the *Noise Library Characterization* chapter of the *make_cdB Noise Characterizer User Guide*.

The same library set can be referenced multiple times by different delay corners.

Note: You can create as many library sets as needed to support for delay corners that will be included in the multi-mode multi-corner analysis.

To create a library set, use the following command:

```
create_library_set
```

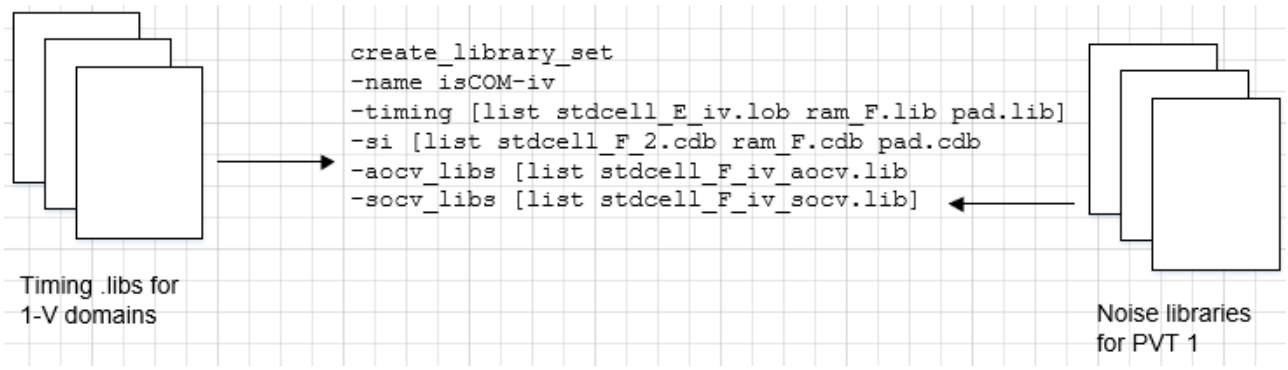
Flow Library Requirements

Timing libraries must be read into the system first, using `read_libs` command. If you want to create a library set that includes cdB libraries, they also must be read into the system first, using the `read_libs -noise_libs` command. This same flow is adopted in single-mode single-corner (SMSC) analysis.

The order in which you define timing libraries is important. The software considers the first library in the list as master library, with successive libraries having low priority.

The library sets can be created only after the top module is set. The following figure shows the creation of a library set that associates timing libraries and cdB libraries with a nominal voltage of 1 volt with the library name is COM-1V.

Figure 5-2 Creation of Library Sets



* Either AOCV or SOCV libraries can be specified at a time. The AOCV and SOCV files can also be added to the library set by either `-aocv [list aocv1.lib aocv2.lib]` or `-socv [list socv1.lib socv2.lib]` in the `create_library_set` settings.

Note: You can use the `create_library_set -timing` parameter and interpolate a set of timing libraries to perform IR-drop aware delay calculation.

Updating Library Sets

To change the timing and cdB library files for an existing library set, use the following command:

- `update_library_set`

You can update a library set in the GUI mode by using the following ways:

Click the *File* tab and select Configure MMMC. A new window opens, double-click on one of the library set objects, where the timing library files need to be updated. A new terminal will open that allows you to add or delete timing library files. After updating the files, click on **Apply** and **OK** to save the changes.

Or

Click the *Timing & SI* tab and select MMMC Browser. Select the library set and double-click on one of the library sets where the timing library files need to be updated. A new terminal will open where you can add or delete timing library files. After updating the files, click on **Apply** and **OK** to save the changes.

Note:

- It may not be necessary to update a library set, but Timing/SI libraries under a library set can be updated.

- You can use the `update_library_set` command or the *Edit Library Set* option (in GUI Mode), before (or any time later) the multi-mode multi-corner views are loaded into the design. Once the views are loaded, any change in the library set (using the `update_library_set` command (or *Edit Library Set* form) will reset the timing, RC data, and delay corner settings for all the analysis views.

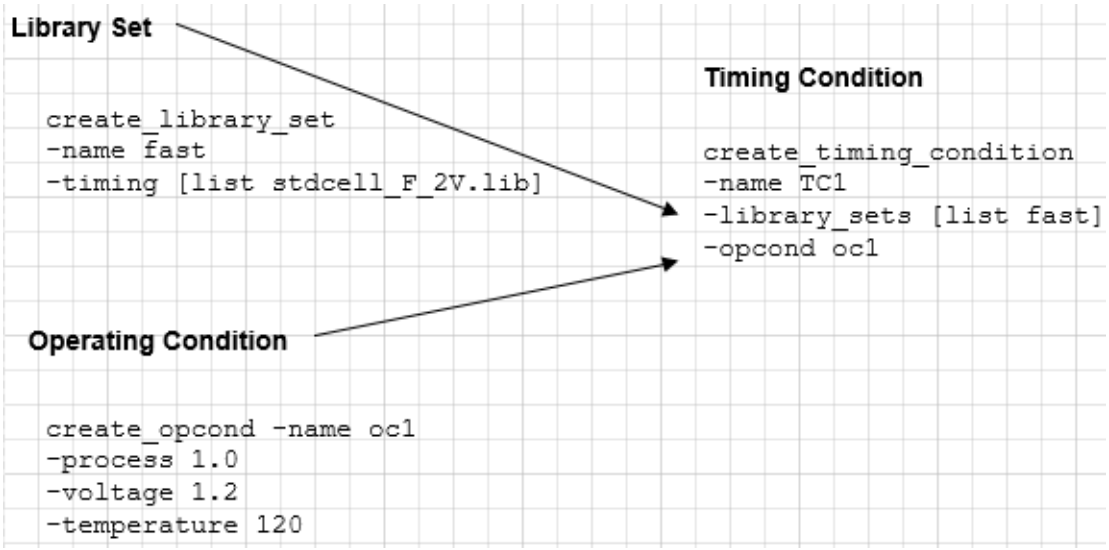
5.5 Defining Timing Conditions

You can define new operating/timing conditions using the `create_timing_condition` command. This command creates a set of virtual operating conditions in the specified library.

For example, the following command creates a timing condition called `TC1` for the library set `fast` and for the virtual operating condition `oc1`:

```
create_timing_condition -name TC1 -library_sets [list fast] -opcond oc1
```

Figure 5-3 Define timing conditions



5.6 Virtual Operating Conditions

This section includes:

- Creating Virtual Operating Conditions
- Editing Virtual Operating Conditions

Creating Virtual Operating Conditions

In general, the process, voltage, and temperature (PVT) point is specified by referring to a predefined operating condition defined in a timing library. The library operating condition provides the system with values for P, V, and T. However, there are situations when there are no predefined operating conditions in user timing libraries, or pre-existing operating conditions are not consistent with user's operating environment. Instead of modifying timing libraries to add or adjust operating condition definitions, you can create a set of virtual operating conditions for a library, to define a PVT operating point. These virtual operating conditions can then be referenced by a delay corner, as if they actually existed in the library.

The operating conditions scale the delay numbers if there is a mismatch between the PVT condition of libraries and the virtual operating conditions. To create a virtual operating condition for a library, use the following command:

- `create_opcond`

For example, the following command creates a virtual operating condition called oc1 :

```
create_opcond -name oc1
   -process 1.0
   -voltage 1.2
   -temperature 120
```

Editing Virtual Operating Conditions

In GUI mode, you can add or change attributes for a defined virtual operating condition using the “Add OP Cond” form.

Click the *File* tab and choose *Configure MMMC*. A new window opens in which you need to double-click on the *OP Conds* tab and add the operating condition. Click **Apply** and **OK** for the changes to take affect.

or

Click the *MMMC Browser* under the *Timing & SI* tab, and the double-click on the OP Conds tab. After adding the operating condition, click **Apply** and **OK** for the changes to take affect.

You can edit a virtual operating condition before multi-mode multi-corner view definitions are loaded into the design or any time later. However, once the software is in multi-mode multi-corner analysis mode, any changes to an existing object results in the timing, delay calculation, and RC data being reset for all analysis views.

5.7 RC Corner Objects

This section includes:

- Creating RC Corner Objects

Creating RC Corner Objects

An RC corner object provides the information necessary to properly annotate, and use the RCs for delay calculation. In Tempus, you read the RC information using the SPEF files. A SPEF file contains the parasitic information for a RC corner. In multi-mode multi-corner analysis, you can use a SPEF file for each view.

RC corner objects are referenced when creating delay calculation corner objects.

To create an RC corner, use the following command:

- `create_rc_corner`

For example, the following command creates an RC corner called `rc-typ`:

```
create_rc_corner -name rc-typ
```

5.8 Delay Corner Objects

This section includes:

- Creating Delay Corner Objects
- Editing Delay Corner Objects
- Power Domain Definitions

Creating Delay Corner Objects

A delay corner provides information necessary to control delay calculation for a specific analysis view. Each corner contains information on the libraries, the operating conditions with which the libraries should be accessed, and the RC corner for loading the parasitic data. Delay corner objects can be shared by multiple top-level analysis views.

To create a delay calculation corner, use the following command:

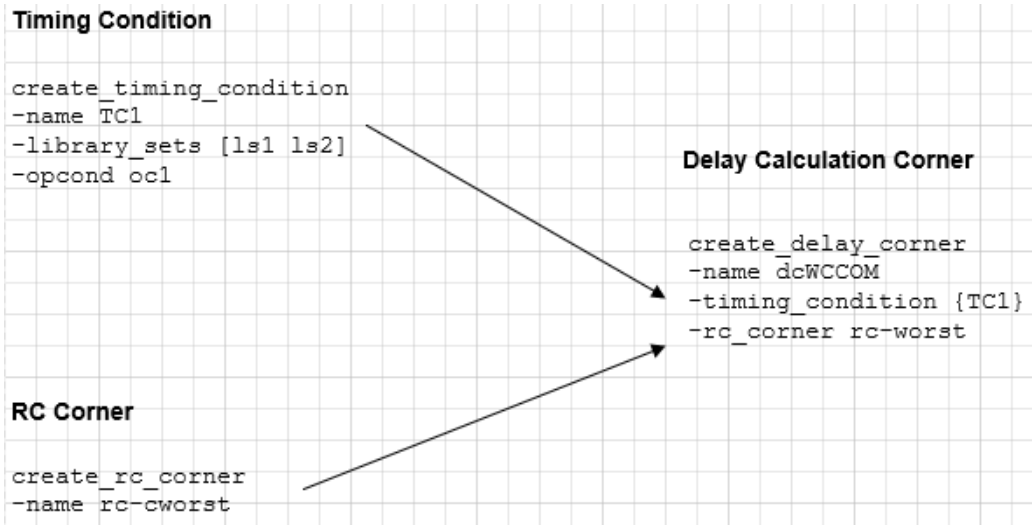
- `create_delay_corner`

Use separate delay calculation corners to define major PVT operating points (for example, Best-Case and Worst-Case).

Use the `-early_*` and `-late_*` parameters within a single delay calculation corner to control on-chip variation.

The following figure shows the creation of a delay calculation corner called `dcWCCOM`.This corner uses the libraries from `IsCOM-1V`, sets the operating condition to `WCCOM`, as defined in the `stdcell_1V` timing library, and uses the `rc-cworst` RC corner:

Figure 5-4 Creation of a delay calculation corner



Editing Delay Corner Objects

To add or change attribute values of an existing delay calculation corner object, use the following command:

- `update_delay_corner`

In GUI mode, you can make changes to a delay calculation corner object by using *Add Delay Corner* form.

Click the *File* tab and choose *Configure MMMC*. A new window opens, double-click on the *Delay Corners* tab. Select the type of analysis mode and specify the rc corner, library set, OpCond Lib, OpCond, IrDrop File. Once done, click **Apply** and **OK** to save the settings.

Or,

Click *Timing & SI* tab and choose *MMMC Browser*. A new window opens, double-click on the *Delay Corners* tab. Select the type of analysis mode and specify the rc corner, Library set, OpCond Lib, OpCond, IrDrop File. Once done, click **Apply** and **OK** to save the settings.

You can use the `update_delay_corner` command or the "Add Delay Corner" form before (or any time later) multi-mode multi-corner view definitions are loaded into the design. However, once the software is in multi-mode multi-corner analysis mode, any changes to an existing object results in the timing, delay calculation, and RC data being reset for all analysis views.

The following example shows how to create two corners and update the delay corners with the appropriate information.

Create worst-case RC corner based on the worst-case RC parameters:

```
create_rc_corner -name rc_cworst
```

Best-case RC corner based on the best-case RC parameters:

```
create_rc_corner -name rc_cbest
```

To associate the best-case and worst-case RC corners with the appropriate delay corner, use the `update_delay_corner` command:

```
update_delay_corner -name AV_HL_FUNC_MIN_RC1_dc -rc _corner rc_cworst
update_delay_corner -name AV_HL_FUNC_MIN_RC2_dc -rc_corner rc_cbest
update_delay_corner -name AV_HL_FUNC_MAX_RC1_dc -rc_corner rc_cworst
update_delay_corner -name AV_HL_FUNC_MAX_RC2_dc -rc_corner rc_cbest
update_delay_corner -name AV_HL_SCAN_MIN_RC1_dc -rc_corner rc_cworst
update_delay_corner -name AV_HL_SCAN_MAX_RC1_dc -rc_corner rc_cworst
update_delay_corner -name AV_HO_FUNC_MIN_RC1_dc -rc_corner rc_cworst
update_delay_corner -name AV_HO_FUNC_MAX_RC1_dc -rc_corner rc_cworst
update_delay_corner -name AV_LO_FUNC_MIN_RC1_dc -rc_corner rc_cworst
update_delay_corner -name AV_LO_FUNC_MAX_RC1_dc -rc_corner rc_cwors
```

5.9 Power Domain Definitions

Adding Power Domain Definition to Delay Calculation Corners

A single delay calculation corner object specifies the delay calculation rules for the entire design. If a design includes power domains, the delay calculation corner can contain domain-specific subsections that specify the required operating condition information, and any necessary timing library rebinding for the power domain.

You must use the same power domain names that you read in using the power domain file to define the physical aspects of the domain. For more information about power domain file, see `read_power_intent`.

To create a power domain definition for a delay calculation corner, use the following command:

- update_delay_corner

For example, the following command adds definitions for the power domain `domain-3V` to the dcWCCOM delay calculation corner:

```
update_delay_corner -name dcWCCOM
-power_domain domain-3V
-library_set libs-3volt
-opcond_library delayvolt_3V
-opcond slow_3V
```

Editing Power Domain Definitions

To add or change attribute values for an existing power domain definition, use the following command:

- update_delay_corner

In GUI mode, you can make changes to a power domain definition using the *Edit Power Domain* form:

Click the *File* tab and choose *Configure MMMC*. A new window opens, double-click on the *Delay Corners* tab. In the "*Power Domain List*" column, click **Add**. A new *Add Power Domain* window opens, you can enter the details for library set, OpCond Lib, OpCond, IrDrop File. Once done, click **Apply** and **OK** to save the settings.

or:

Click *Timing & SI* tab and choose *MMMC Browser*. A new window opens, double-click on the *Delay Corners* tab. In the "*Power Domain List*" column, click **Add**. A new "*Add Power Domain*" window opens, you can enter the details for library set, OpCond Lib, OpCond, IrDrop File. Once done, click **Apply** and **OK** to save the settings.

You can use the `update_delay_corner` command or the *Add Delay Corner* form before (or any time later) multi-mode multi-corner view definitions are loaded into the design. However, after the software is in multi-mode multi-corner analysis mode, any changes to an existing object results in the timing, delay calculation, and RC data being reset for all analysis views.

5.10 Constraint Mode Objects

This section includes:

- Creating Constraint Mode Objects

- Editing Constraint Mode Objects
- Entering Constraints Interactively
- Constraint Support in Multi-Mode and MMMC Analysis

Creating Constraint Mode Objects

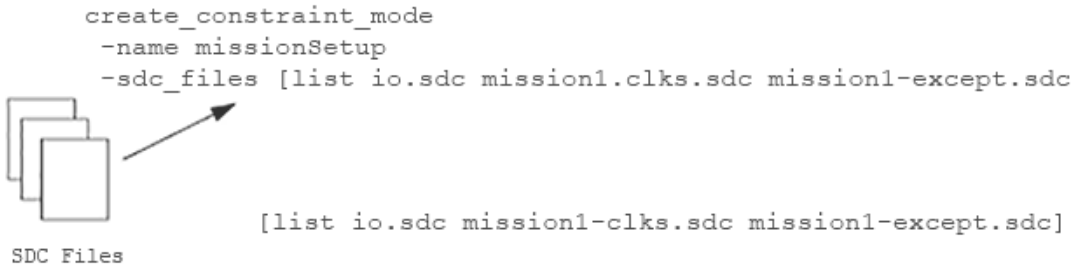
A constraint mode object defines one of possibly many different functional, test, or Dynamic Voltage and Frequency Scaling (DVFS) modes of a design. It consists of a list of SDC constraint files that contain timing analysis information, such as the clock specifications, case analysis constraints, I/O timings, and path exceptions that make each mode unique. SDC files can be shared by many different constraint modes, and the same constraint mode can be associated with multiple analysis views.

To create a constraint mode object, use the following command:

- `create_constraint_mode`

The following figure shows that the constraint mode “missionSetup” reads in 3 constraint mode files “`io.sdc`”, “`mission1-clks.sdc`” and “`mission1-except.sdc`”:

Figure 5-5 Constraint Mode Objects



Editing Constraint Mode Objects

To change the SDC constraint file information for an existing constraint mode object, use the following command:

- `update_constraint_mode`

In the GUI mode, you can make changes to a constraint mode object using the *Add Constraint Mode* form.

Click the *File* tab and choose *Configure MMMC*. A new window opens, double-click on the *Constraint Modes* tab and add the “SDC Constraint File(s)”. Once done, click **Apply** and **OK** to save the settings.

Or,

Click*Timing & SI* tab and choose *MMMC Browser*. A new window opens, double-click on the *Constraint Modes* tab and add the “*SDC Constraint File(s)*”. Once done, click **Apply** and **OK** to save the settings.

You can use the `update_constraint_mode` command or the *Add Constraint Mode* form before (or any time later) multi-mode multi-corner view definitions are loaded into the design. However, once the software is in multi-mode multi-corner analysis mode, any changes to an existing object results in the timing, delay calculation, and RC data are reset for all the analysis views.

Entering Constraints Interactively

The Tempus software allows you to update assertions for a multi-mode multi-corner constraint mode object, and have those changes take immediate effect, use the following command:

- `set_interactive_constraint_modes`

Specifying `set_interactive_constraint_modes` puts the software into interactive mode for the specified constraint mode objects. Any constraints that you specify after this command will take effect immediately on all active analysis views that are associated with these constraint modes. By default, no constraint modes are considered active.

For example, the following commands put the software into interactive constraint entry mode, and apply the `set_propagated_clock` assertion on all views in the current session that are associated with the constraint mode func1:

```
set_interactive_constraint_modes func1
set_propagated_clock [all_clocks]
```

The software stays in interactive mode until you exit it by specifying the `set_interactive_constraint_modes` command with an empty list:

```
set_interactive_constraint_modes { }
```

All the new constraints are saved in the SDC file for the specified constraint mode when you save the design.

You can use the `get_interactive_constraint_modes` command to return a list of the constraint mode objects in interactive constraint entry mode.

Note: Interactive constraint mode only works when the software is in regular multi-mode multi-corner timing analysis mode. You cannot use it in distributed multi-mode multi-corner analysis. In min/max analysis mode, constraints are always accepted interactively.

5.11 Constraint Support in Multi-Mode and MMMC Analysis

The Tempus software isolates the following SDC constraints from conflicting with each other, in both multi-mode and multi-mode multi-corner analysis modes:

- `create_clock`
- `create_generated_clock`
- `set_annotated_check`
- `set_annotated_delay`
- `set_annotated_transition`
- `set_case_analysis`
- `set_clock_gating_check`
- `set_clock_groups`
- `set_clock_latency`
- `set_clock_sense`
- `set_clock_transition`
- `set_clock_uncertainty`
- `set_disable_timing`
- `set_drive`
- `set_driving_cell`
- `set_false_path`
- `set_fanout_load`
- `set_input_delay`
- `set_input_transition`
- `set_load`
- `set_max_delay`
- `set_max_time_borrow`

- `set_max_transition`
- `set_min_delay`
- `set_min_pulse_width`
- `set_multicycle_path`
- `set_output_delay`
- `set_propagated_clock`
- `set_resistance`

Note: Path groups defined with `group_path` are considered to be global definitions across all analysis views.

5.12 Analysis Views

This section includes:

- Creating Analysis Views
- Editing Analysis View Objects
- Setting Active Analysis Views
- Guidelines for Setting Active Analysis Views

Creating Analysis Views

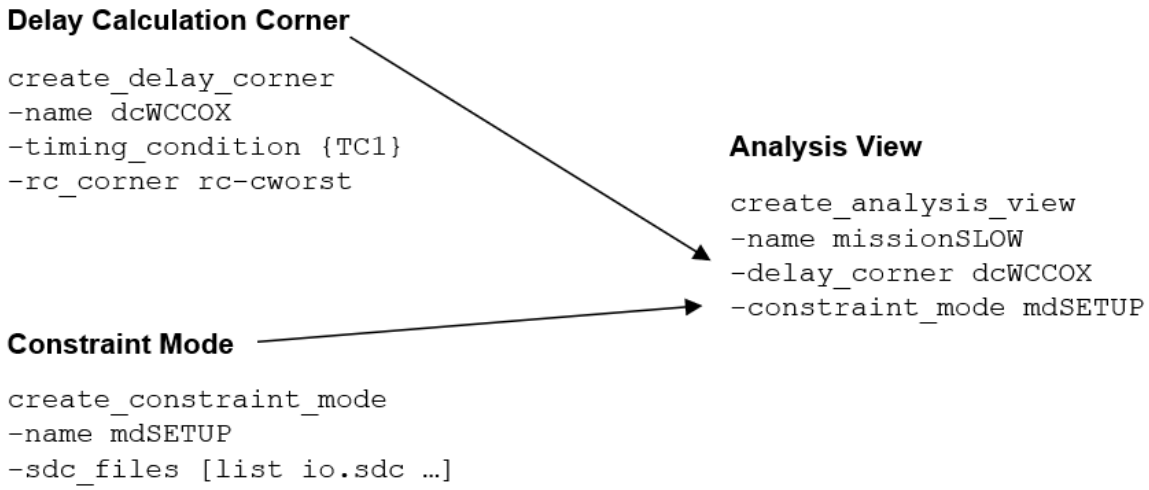
An analysis view object provides all of the information necessary to control a given multi-mode multi-corner analysis. It consists of a delay calculation corner and a constraint mode.

To create an analysis view, use the following command:

- `create_analysis_view`

The following figure shows the creation of the analysis view `missionSetup`:

Figure 5-6 Creation of New Analysis View



Editing Analysis View Objects

To change the attribute values for an existing analysis view, use the following command:

- `update_analysis_view`

In the GUI mode, you can make changes to an analysis view using the *Add Analysis View* form:

Click the *File* tab and choose *Configure MMMC*. A new window opens, double-click on the *Analysis Views* tab and specify the constraint mode, delay corner, and analysis view. Click **Apply** and **OK** to save the settings.

Or,

Click *Timing & SI* tab and choose *MMMC Browser*. A new window opens, double-click on the *Analysis Views* tab and specify the constraint mode, delay corner, and analysis view. Click **Apply** and **OK** to save the settings.

Note: You can use the `update_analysis_view` command or the *Add Analysis View* form before (or any time later) multi-mode multi-corner view definitions are loaded into the design. However, once the software is in multi-mode multi-corner analysis mode, any changes to an existing object in the timing, delay calculation, and RC data are reset for all the analysis views.

Setting Active Analysis Views

After creating analysis views, you must set which views the software should use for setup and hold analysis. These "active" views represent the different design variations that will be analyzed. Active views can be changed throughout the flow to utilize different subsets of views. Libraries and data are loaded into the system, as required to support the selected set of active views.

To set active analysis views, use the following command:

- set_analysis_view

Note: You must define at least one setup and one hold analysis view.

For example, the following command sets missionSlow and mission2Slow as the active views for setup analysis, and missionFast and testFast as the active views for hold analysis:

```
set_analysis_view
  -setup {missionSlow mission2Slow}
  -hold {missionFast testFast}
```

Guidelines for Setting Active Analysis Views

The order in which you specify views, using the set_analysis_view command, is important. By default, the first view defined in the -setup and -hold lists is the default view.

Changing the Default Active Analysis View

For example, if the analysis view missionSlow is currently the default active setup view in the design, the following command temporarily changes the default view to mission2Slow:

```
set_default_view -setup mission2Slow
```

Checking the Multi-Mode Multi-Corner Configuration

Use the following command to generate a hierarchical report of your current multi-mode multi-corner configuration:

- report_analysis_views

5.13 Design Loading and MMMC Configuration Script

The script to load a design must contain design loading as well as all MMMC configuration commands, such as, `create_library_set`, `create_delay_corner`, and `create_constraint_mode`.

A sample common_init.tcl script is shown below:

```
################ common_init.tcl #############
read_mmmc INPUT/viewDefinition.tcl
read_physical -lef "
    lib/lef/gsclib045.fixed.lef
    lib/lef/MEM1_256X32.lef
    lib/lef/MEM1_1024X32.lef
    lib/lef/MEM1_4096X32.lef
    lib/lef/MEM2_128X16.lef
    lib/lef/MEM2_128X32.lef
    lib/lef/MEM2_136X32.lef
    lib/lef/MEM2_512X32.lef
    lib/lef/MEM2_1024X32.lef
    lib/lef/MEM2_2048X32.lef
    lib/lef/MEM2_4096X32.lef
    lib/lef/pdkIO.lef
    "
read_netlist INPUT/asic_entity.v.gz
#read_def aa.def
#read_power_intent
init_design
if {[get_db program_short_name] == "tempus"}  {
read_db -physical_data tempus_test.db
}
```

A sample viewDefinition.tcl script is shown below:

```
################ viewDefinition.tcl ############
create_library_set -name fast\
    -timing\
    [list ${::IMEX::libVar}/mmmc/fast.lib\
    ${::IMEX::libVar}/mmmc/pdkIO.lib\
    ${::IMEX::libVar}/mmmc/MEM2_4096X32_slow.lib\
    ${::IMEX::libVar}/mmmc/MEM2_2048X32_slow.lib\
    ${::IMEX::libVar}/mmmc/MEM2_1024X32_slow.lib\
    ${::IMEX::libVar}/mmmc/MEM1_4096X32_slow.lib\
    ${::IMEX::libVar}/mmmc/MEM1_1024X32_slow.lib\
    ${::IMEX::libVar}/mmmc/MEM1_256X32_slow.lib\
    ${::IMEX::libVar}/mmmc/MEM2_512X32_slow.lib\
    ${::IMEX::libVar}/mmmc/MEM2_136X32_slow.lib\
    ${::IMEX::libVar}/mmmc/MEM2_128X32_slow.lib\
    ${::IMEX::libVar}/mmmc/MEM2_128X16_slow.lib\
    ${::IMEX::libVar}/mmmc/leon.lib]
create_library_set -name slow\
    -timing\
    [list ${::IMEX::libVar}/mmmc/slow.lib\
    ${::IMEX::libVar}/mmmc/pdkIO.lib\
    ${::IMEX::libVar}/mmmc/MEM2_4096X32_slow.lib\
    ${::IMEX::libVar}/mmmc/MEM2_2048X32_slow.lib\
    ${::IMEX::libVar}/mmmc/MEM2_1024X32_slow.lib\
```

```
    ${::IMEX::libVar}/mmmc/MEM1_4096X32_slow.lib\
    ${::IMEX::libVar}/mmmc/MEM1_1024X32_slow.lib\
    ${::IMEX::libVar}/mmmc/MEM1_256X32_slow.lib\
    ${::IMEX::libVar}/mmmc/MEM2_512X32_slow.lib\
    ${::IMEX::libVar}/mmmc/MEM2_136X32_slow.lib\
    ${::IMEX::libVar}/mmmc/MEM2_128X32_slow.lib\
    ${::IMEX::libVar}/mmmc/MEM2_128X16_slow.lib\
    ${::IMEX::libVar}/mmmc/leon.lib]
create_timing_condition -name default_mapping_tc_2\
    -library_sets [list fast]
create_timing_condition -name default_mapping_tc_1\
    -library_sets [list slow]
create_rc_corner -name rc_min\
    -cap_table ${::IMEX::libVar}/mmmc/capTable\
    -pre_route_res 1.34236\
    -post_route_res 0.994125\
    -pre_route_cap 1.10066\
    -post_route_cap 0.960235\
    -post_route_cross_cap 1.22327\
    -pre_route_clock_res 0\
    -pre_route_clock_cap 1.105\
    -post_route_clock_cap 0.969117\
    -temperature 0\
    -qrc_tech ${::IMEX::libVar}/mmmc/qrcTechFile\
create_rc_corner -name rc_max\
    -cap_table ${::IMEX::libVar}/mmmc/capTable\
    -pre_route_res 1.34236\
    -post_route_res 0.994125\
    -pre_route_cap 1.10066\
    -post_route_cap 0.960234\
    -post_route_cross_cap 1.22327\
    -pre_route_clock_res 0\
    -pre_route_clock_cap 0.967898\
    -post_route_clock_cap 0.969117\
    -temperature 125\
    -qrc_tech ${::IMEX::libVar}/mmmc/qrcTechFile\
create_opcond -name PVT1 \
    -process 1 \
    -voltage 2 \
    -temperature 120 \
create_delay_corner -name slow_max\
    -timing_condition {default_mapping_tc_1}\
    -rc_corner rc_max
create_delay_corner -name fast_min\
    -timing_condition {default_mapping_tc_2}\
    -rc_corner rc_min
create_constraint_mode -name functional\
    -sdc_files\
    [list ${::IMEX::libVar}/mmmc/eagle.sdc]
create_analysis_view -name func_slow_max -constraint_mode functional -delay_corner
slow_max
create_analysis_view -name func_fast_min -constraint_mode functional -delay_corner
fast_min
set_analysis_view -setup [list func_slow_max] -hold [list func_fast_min]
```

5.14 Saving Multi-Mode Multi-Corner Configuration

The software stores multi-mode multi-corner configuration as Tcl commands in the tempus.cmd file. This file contains information on all the library sets, RC corners, delay corners, constraint modes, and analysis view definitions that you created. You can reload the configuration information into a design session by sourcing the tempus.cmd file.

You can also save the design using the `write_db` command. If constraints are sourced interactively then a designer can save the design after interactive reading of constraints through this command.

5.15 Controlling Multi-Mode Multi-Corner Analysis through the Flow

The following table describes how Tempus applications behave in multi-mode multi-corner analysis mode throughout the flow:

Table 5-1 Application Behavior in Multi-Mode Multi-Corner Analysis Mode

Flow Step	Commands	Application Behavior
Delay calculation	`write_sdf -view`	Uses the early and late delays for the specified analysis view to populate the min and max SDF triplet slots.
Timing	`report_timing -view`	Analyzes all active analysis views in the design, or for a specified view (`-view`) concurrently.

5.16 Performing Timing Analysis

The software performs multi-mode multi-corner timing analysis concurrently on all active analysis views in the design. To run multi-mode multi-corner timing analysis, you use the following command:

- `report_timing`

By default, the `report_timing` command analyzes all active analysis views in the design concurrently. The timing report details indicate the views that are responsible for a path. By default, the software returns only one path per end point.

If you have a large number of views, you can use the `report_timing -view` command to process multiple views. The software stores results for each view, along with an aggregated report in machine-readable timing reports (.mtarpt). You can view the results graphically in the Tempus Timing Debug environment. You can use this procedure to determine the critical view subsets that should be used to drive the implementation flow.

Note: The SPEF data is only read after a view is created to avoid multi-corner errors.

5.17 Generating Timing Reports

The software can generate various reports that contain timing information for each active analysis view.

For more information on timing reports, see the `report_timing` *Sample Reports* section in the *Tempus Stylus Text Command Reference*.

For other reports, see the `report_*` commands in the “Timing Analysis Commands” chapter.

6

Distributed Multi-Mode Multi-Corner Timing Analysis

6.1 DMMMC Overview on page 388
6.2 Performing Distributed Multi-Mode Multi-Corner Analysis on page 389
6.3 Performing Concurrent MMMMC Analysis in DMMMC Mode on page 401
6.4 Setting Up MSV/CPF-Based Design in DMMMC Mode on page 402
6.5 Performing Setup/Hold View Analysis and Reporting on page 405
6.6 Running Single Interactive User Interface in DMMMC Mode on page 406
6.7 Running Interactive ECO on Multiple Views in DMMMC Mode on page 408
6.8 Important Guidelines and Troubleshooting DMMMC on page 411

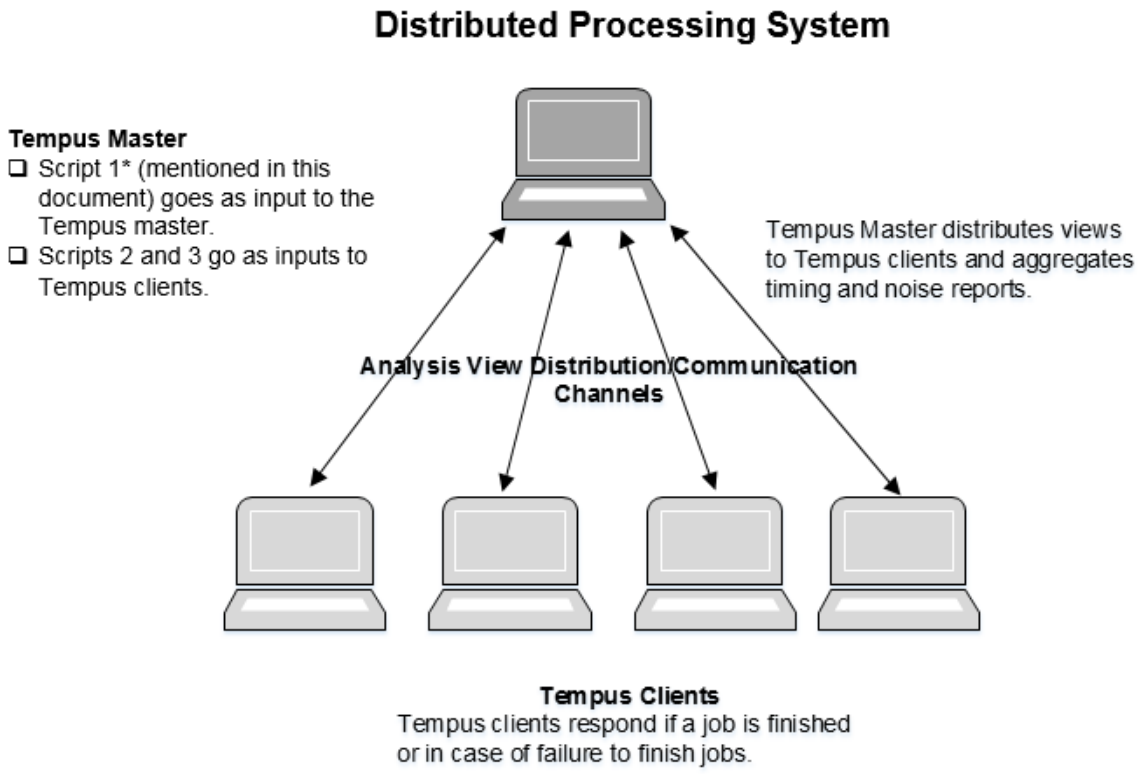
6.1 DMMMC Overview

The multi-mode multi-corner (MMMC) timing analysis can be run in distributed mode also. Distributed multi-mode multi-corner (DMMMC) enables controlled distribution of various analysis views to a selected or shared group of machines, and then merges the reports generated for timing and signal integrity across the views.

This chapter describes how an existing MMMC setup can be transitioned to distributed MMMC (that is, to multiple machines/CPUs). These multiple machines/CPUs will be referred as Tempus clients in this chapter.

Distributed MMMC is a method of processing timing analysis views where the runs are distributed on several Tempus clients. This allows you to run Timing/SI analysis for different analysis views on multiple Tempus clients and then merge/aggregate the generated reports from various views into a single report. The flow is shown in the figure below.

Figure 6-1 DMMMC Flow



The Tempus master invokes a set of Tempus clients on the specified machines/CPUs, using the `set distributed hosts` command. Besides tracking the activity on Tempus clients,

the Tempus master also governs the responsibility of adjusting queued analysis jobs to active Tempus clients, in case some Tempus clients fail due to an unexpected event.

Distributed MMMC in Tempus enables:

- Parallel analysis of multiple MMMC views running simultaneously on multiple Tempus clients controlled via a Tempus Master session.
- Merging of timing reports.
- Saving a significant amount of real time as compared to sequential runs when there are a large number of views to analyze.
- Accelerated utilization of available computing power (multiple CPU cores) on local or remote machines/hardware via super threading.

Licensing Requirements

Refer to "Tempus License Options for Distributed MMMC (DMMMC)" in the "Product and Licensing Information" chapter.

Restrictions

The following restriction applies to distributed MMMC analysis:

- Distributed MMMC analysis is supported in GUI mode. However, the GUI menu and forms are not available for DMMMC commands.

6.2 Performing Distributed Multi-Mode Multi-Corner Analysis

Distributed multi-mode multi-corner analysis is performed by reading a flow script into the master Tempus server, which then distributes the analysis runs to the user-specified machines. Analysis can further be multi-threaded on these machines.

The software saves all data from the distributed analysis runs in a user-specified output directory (specified with the `distribute_read_design` command).

- The software generates view-specific reports in directories with the corresponding view name within this directory.

  When concurrent multi-mode multi-corner timing analysis (CMMMC) is enabled, within a DMMMC framework, the software generates CMMMC session-specific reports in directories that have the corresponding CMMMC session names, under a user-defined

output directory (specified using the `distribute_read_design -out_dir` parameter). The CMMMC session names are cmmmc_1, cmmmc_2 and so on. The software creates a mapping file (views_dir_map.rpt) to show the mapping of CMMMC session names with a list of the combined views. The master log file contains messages that refer to the mapping file. If the mapping file already exists in the run directory, software will create the mapping file with name views_dir_map.rpt1, views_dir_map.rpt2 and so on.

An example of views_dir_map.rpt file is given below:

```
Dir: cmmmc_1 Views: {view1 view2}
Dir: cmmmc_2 Views: {view3 view4}
Dir: cmmmc_3 Views: {view5}
```

For more information on how to enable CMMMC, you can refer to the `distribute_views` command.

- In the DMMMC analysis flow, client monitoring is enabled by default. This feature enables robust master/client resource monitoring and re-launches the clients when they fail due to resource overloading. The master monitors the health of clients at regular intervals and writes the information about per client resources (cpu/memory/tmp) in the host-monitor.log file. If the host-monitor log file already exists in the run directory, software will create the new file with name host-monitor.log1, host-monitor.log2 and so on.

  An example report is given below:

```
######################
Status keys: E excellent, G good, B bad, T terrible.
Memory values:
total = Real memory of host
progs = Real memory used by processes, including swappable cache
used =  Real memory used by processes, excluding swappable cache
Started at: 19/09/03 23:45:37
sj-etam(0)    TMPDIR is : /tmp/ssv_tmpdir_3160_U03VpI

============ ======== ==================== ======================
Host         CPU      Memory (GB)              TMPDIR (GB)       (Mb/s)
name(id)     util %   total progs used   %used  total    avail    rate
============ ========  ===== ===== ===== ====== ====================
23:45:37
sj-etam(0)   10 E      3    3     3    100 T   44.29    27.45 E  N/A
23:46:37
sj-etam(0)   3 E       3    3     3    100 T   44.29    27.47 E  376 E
######################
```

The software checks for overloading of client resources and issues appropriate warning messages for over-utilization of client resources. If client resources are over-utilized and view analysis is not able to proceed, the software will automatically terminate the view job and restart it on another client.

The DMMMC flow enables re-launch of clients using the same LSF configuration in the LSF mode, if any overloading (cpu/mem) occurs in clients during view analysis. When `set_distributed_hosts -local` mode is set, the re-launch feature is disabled.

Handling Scripts

For distributed MMMC analysis the following Tcl scripts are required:

- Master Script on page 391
- Design Loading and MMMC Configuration Script on page 394
- Parasitic Information, Analysis, and Report Generation Script on page 397

Master Script

The master script is the overall flow script that runs the entire Tempus distributed MMMC session. This script includes all generic environment variables, distribution commands, and commands for specifying CPU usage, running DMMMC analysis, and merging reports.

The master script is read into the master Tempus server, which then distributes the analysis runs to the user-specified machines.

Creating the Master Script

The following steps list the order in which you should define the commands in the master script that are necessary for running a distributed MMMC analysis session.

1. Specify environment variables or shell variables to be used in the flow.
2. Specify distribution infrastructure like machine names, mode of execution (LSF or rsh), by using the `set_distributed_hosts` command.

   For example:

   ```
   set_distributed_hosts -rsh -add {hostName1 hostName2 hostName3
   hostName4}
   ```

3. Specify the Tempus client usage by using the `set_multi_cpu_usage -remote_host` command. The number specified here must be less than or equal to the number of machines specified in step 2, (if specific machines have been requested using the `-rsh` parameter of the `set_distributed_hosts` command).

   For example:

   ```
   set_multi_cpu_usage -remote_host 4
   ```

The following example shows that jobs on each remote host can further be multi-threaded:

```
set_multi_cpu_usage -remote_host 4 -cpu_per_remote_host 4
```

If LSF distribution is used, then you can specify the number of Tempus clients to be used. The software needs CPU usage information in order to figure out how many licenses the system can use for distribution.

4. Load the design in distributed mode using the distribute_read_design command. This requires providing an input script that contains all the design loading commands, except the read spef and read parasitics command. The distribute_read_design command opens up N number of Tempus clients (as specified with the set_multi_cpu_usage -remote_host) for the Tempus distributed processing session, and loads the design script into the Tempus clients.

   For example:

   ```
   distribute_read_design
   -design_script $rootDir/design_load.tcl
   -outdir $output_directory
   ```

5. Specify the active setup and hold analysis views.

   For example:

   ```
   set Views [list func_ss_rcmax test_ss_rcmax func_ss_rctyp]
   ```

6. Distribute and analyze views using the distribute_views command. This command distributes the active setup and hold analysis views to Tempus clients, and sources the specified command script to perform timing and signal integrity analysis.

   For example:

   ```
   distribute_views
   -views $Views
   -script $rootDir/analysis.tcl
   ```

7. Merge the timing and noise reports generated from various servers using the merge_timing_reports command.

8. Close all the Tempus clients using the reset_servers command to end the distributed processing session.

The following sample master script needs to be provided as input to the Tempus master.

Example: Sample Master Script

```
#################### Script 1: run.tcl ##############################

# Setup the environment variables or shell variables to be used in the flow.
```

```
set runLocal 0
set cwd [exec pwd]
set outDir "$cwd/dmmmc_logs"
```

Instructs Tempus to launch jobs in rsh. You can specify the list of machines that can be used to launch utility servers. You can also specify LSF queue (refer to the Tempus Text Command Reference).

set_distributed_hosts -rsh -add {*hostname1 hostname2*}

or

```
set_distributed_hosts -time_out 300 -lsf -queue lnx64 -args "-P
TEMPUS:15.1:PE:build -n ${threadCnt}" -resource "span\[hosts=1\]
select\[OSNAME==Linux \]"
```

Instructs 3 Tempus clients to be used for this session run. Each Tempus client will use 8 CPUs for multi-threading.

set_multi_cpu_usage -local_cpu 2 -remote_host 2 -cpu_per_remote_host 4

Invokes the Tempus clients based on the above information and loads Script-2 (design_load.tcl) on each client. Specified “outDir” is the area where the Tempus client log files will be generated. All the outputs relevant to this session will be created here. The master log will reside in the directory where the run is launched.

distribute_read_design -design_script sta_dmmmc.tcl -outdir $outDir

Distributes list of specified analysis views to Tempus clients and processes the commands specified in Script-3 (analysis.tcl) on each client.

```
set setupViews [list \
func_fast_RCMAX \
func_slow_RCMAX \
func_fast_RCMIN \
func_slow_RCMIN \
scan_fast_RCMAX \
scan_slow_RCMAX \
scan_fast_RCMIN \
scan_slow_RCMIN \
]
distribute_views -views $setupViews -script sta_dmmmc.tcl
```

Merges the timing report (machine readable format) files from specified views and generates a merged view file. The paths are sorted based on the slack value.

```
merge_timing_reports -view $setupViews -base_report setup_maxpath_machine.rpt
-base_dir $outDir > merged_setup_maxpath.rpt
merge_timing_reports -view $setupViews -base_report setup_nworst_machine.rpt  -
base_dir $outDir > merged_setup_worst.rpt
merge_timing_reports -view $setupViews -base_report hold_maxpath_machine.rpt  -
base_dir $outDir > merged_hold_maxpath.rpt
merge_timing_reports -view $setupViews -base_report hold_nworst_machine.rpt   -
base_dir $outDir > merged_hold_worst.rpt
```

#Close Tempus session

```
exit_servers
```

For more information on loading a MMMC configuration script, see Design Loading and MMMC Configuration Script on page 394.

For more information on parasitics, see Parasitic Information, Analysis, and Report Generation Script on page 397.

6.3 Design Loading and MMMC Configuration Script

This script contains the common information that needs to be shared by all Tempus clients. The script must include the design loading commands, as well as all MMMC configuration commands, for example, `create_library_set`, `create_delay_corner`, and `create_constraint_mode`.

The analysis views are specified with the `distribute_views` command, which distributes them internally to the Tempus clients for processing.This command is automatically issued by Tempus master via a distribution method (`distribute_views`).

Note: The script must not include parasitic file loading commands (`read_spef`) and analysis view setting commands (`set_analysis_view`). If specified, these commands will be ignored by the distribution mechanism.

This script is loaded using the `distribute_read_design` command.

Example: Sample MMMC Configuration Script

```
###############Script 2 : "viewDefinition.tcl" ##################
################################################
# Define the available libraries
################################################
create_library_set -name libset_slow\
    -timing\
[list ../../libs/liberty/FreePDK45_lib_v1.0_worst.lib\
../../libs/MACRO/LIBERTY/pllclk.lib\
../../libs/MACRO/LIBERTY/ram_256x16A.lib\
../../libs/MACRO/LIBERTY/rom_512x16A.lib]
create_library_set -name libset_fast\
    -timing\
[list ../../libs/liberty/FreePDK45_lib_v1.0_typical.lib\
../../libs/MACRO/LIBERTY/pllclk.lib\
../../libs/MACRO/LIBERTY/ram_256x16A.lib\
../../libs/MACRO/LIBERTY/rom_512x16A.lib]
################################################
# Define the available PVT corners
################################################
create_rc_corner -name corner_worst_RCMAX\
    -pre_route_res 1.2\
    -post_route_res {1.2 1.2 1.2}\
```

```
    -pre_route_cap 1.2\
    -post_route_cap {1.2 1.2 1.2}\
    -post_route_cross_cap {1.2 1.2 1.2}\
    -pre_route_clock_res 0\
    -pre_route_clock_cap 0
create_rc_corner -name corner_worst_RCMIN\
    -pre_route_res 0.8\
    -post_route_res {0.8 0.8 0.8}\
    -pre_route_cap 0.8\
    -post_route_cap {0.8 0.8 0.8}\
    -post_route_cross_cap {0.8 0.8 0.8}\
    -pre_route_clock_res 0\
    -pre_route_clock_cap 0
################################################
#Create the timing conditions
################################################
create_timing_condition -name default_mapping_tc_2\
    -library_sets [list libset_fast]
create_timing_condition -name default_mapping_tc_1\
    -library_sets [list libset_slow]
################################################
# Create the possible delay corners
# These are combinations of timing conditions and corners
################################################
create_delay_corner -name dly_crnr_slow_RCMAX\
    -timing_condition {default_mapping_tc_1}\
    -rc_corner corner_worst_RCMAX
create_delay_corner -name delay_corner_fast_RCMAX\
    -timing_condition {default_mapping_tc_2}\
    -rc_corner corner_worst_RCMAX
create_delay_corner -name delay_corner_slow_RCMIN\
    -timing_condition {default_mapping_tc_1}\
    -rc_corner corner_worst_RCMIN
create_delay_corner -name delay_corner_fast_RCMIN\
    -timing_condition {default_mapping_tc_2}\
    -rc_corner corner_worst_RCMIN
################################################
# Define the available constraint types
################################################
create_constraint_mode -name functional_mode -sdc_files [list ../../design/
dtmf_recvr_core.pr.sdc]
create_constraint_mode -name scan_mode -sdc_files [list ../../design/
dtmf_recvr_core.scan.sdc]
################################################
# Create the possbile views
# These are combinations of constraints, and delay corners
################################################
create_analysis_view -name func_slow_RCMAX -constraint_mode functional_mode -
delay_corner dly_crnr_slow_RCMAX
create_analysis_view -name func_fast_RCMAX -constraint_mode functional_mode -
delay_corner delay_corner_fast_RCMAX
create_analysis_view -name func_slow_RCMIN -constraint_mode functional_mode -
delay_corner delay_corner_slow_RCMIN
create_analysis_view -name func_fast_RCMIN -constraint_mode functional_mode -
delay_corner delay_corner_fast_RCMIN
create_analysis_view -name scan_slow_RCMAX -constraint_mode scan_mode -
delay_corner dly_crnr_slow_RCMAX
create_analysis_view -name scan_fast_RCMAX -constraint_mode scan_mode -
```

```
delay_corner delay_corner_fast_RCMAX
create_analysis_view -name scan_slow_RCMIN -constraint_mode scan_mode -
delay_corner delay_corner_slow_RCMIN
create_analysis_view -name scan_fast_RCMIN -constraint_mode scan_mode -
delay_corner delay_corner_fast_RCMIN
################################################
# Turn on the views that you want to use.
# Not all created views need to be used every time.
################################################
set_analysis_view -setup [list \
func_slow_RCMAX \
func_fast_RCMAX \
func_slow_RCMIN \
func_fast_RCMIN \
scan_slow_RCMAX \
scan_fast_RCMAX \
scan_slow_RCMIN \
scan_fast_RCMIN \
] -hold [list \
func_slow_RCMAX \
func_fast_RCMAX \
func_slow_RCMIN \
func_fast_RCMIN \
scan_slow_RCMAX \
scan_fast_RCMAX \
scan_slow_RCMIN \
scan_fast_RCMIN \
]
```

Handling TCL Vars in DMMMC Setup

Note the following points for handling Tcl variables:

- Tcl Variables defined in run.tcl are available in Tempus master.
- Tcl Variables defined in `loadDesign.tcl` or `sta_dmmmc.tcl` will be available in Tempus clients.
- Tempus software attributes are available in both master and clients. These global variables will have an impact on the analysis only when they are set inside the `loadDesign.tcl` or `sta_dmmmc.tcl` file.
- Variables set by either `distribute_command { …}` or in interactive mode (enabled with the `set_distributed_mmmc_mode -interactive true` setting) will be available or changed only on Tempus clients.

  Note: For more information on interactive mode, refer to sections “Running Interactive User Interface in DMMMC” and “Interactive ECO on Multiple Views in the *Tempus User Guide*.
- Use the `get_distribute_variables` command to fetch the value of a Tempus client Tcl variable from the Tempus master session.

Example: Setting Tcl vars on Tempus clients from master

```
# Create scripts to be run on client/slaves
distribute_command {
set clocks [get_clocks *]
set clock_names [get_object_name $clocks]
set totalClocks [sizeof_collection $clocks]
}
# Fetch client variable as array back to master
# Specify name of client variables w/o '$'
get_distribute_variables "totalClocks clock_names"
foreach viewName [array names totalClocks] {
Puts "View '$viewName' has total '$totalClocks($viewName)' clocks named
'$clock_names($viewName)'"
}
distribute_command {report_timing}
```

Parasitic Information, Analysis, and Report Generation Script

This script contains commands for SPEF reading, analysis, reporting, and ECO DB generation (for Tempus ECO flow, if needed).

The commands for reading parasitic information must be specified in the script before the reporting commands. You must specify parasitic files for all the RC corners. For example:

```
read_spef -rc_corner corner1 rc1.spef
read_spef -rc_corner corner2 rc2.spef
read_spef -rc_corner corner3 rc3.spef
read_spef -rc_corner corner4 rc4.spef
```

If the file includes report commands that redirect their output to a specified file (for example, `report timing` >rt.rpt), then the software generates the file in the active view directory (output directory specified with the `distribute_read_design` command). For report commands with no specified redirection, the software generates the report in the corresponding log file.

This script is loaded using the `distribute_views` command.

Example: Sample Parasitic Information and Report Generation Script

```
######################## Script 3 : "sta_dmmmc.tcl" ##########################
read_spef -rc_corner corner_worst_RCMAX ../../design/SPEF/
corner_worst_RCMAX.spef.gz
read_spef -rc_corner corner_worst_RCMIN ../../design/SPEF/
corner_worst_RCMIN.spef.gz
set_db delaycal_enable_si true
################################
# Setup threading
################################
set_multi_cpu_usage -local_cpu 2     ;# Not using multiple machines
################################
# Setup some global timer variables
```

```
################################
set_db delaycal_enable_si true            ;# Turn on SI analysis
set_db si_glitch_enable_report   true     ;# Turn on glitch analysis
# set_global timing_write_sdf_allow_negative_setuphold_sum true   ;# Enable SDF to
have negative delays
################################
# Select report formatting
################################
set_db timing_report_precision 3
set_table_style -name report_timing -max_widths {70}
set_db timing_report_fields {hpin edge cell transition  load delay incr_delay
arrival required}
################################
# set timing derates here if you have them
################################
set_timing_derate -delay_corner dly_crnr_slow_RCMAX -cell_delay -data -early 0.82
set_timing_derate -delay_corner dly_crnr_slow_RCMAX -cell_delay -data -late 1.01
set_timing_derate -delay_corner dly_crnr_slow_RCMAX -cell_delay -clock -early 0.9
set_timing_derate -delay_corner dly_crnr_slow_RCMAX -cell_delay -clock -late 1.1
set_timing_derate -delay_corner dly_crnr_slow_RCMAX -net_delay -data -early 0.82
set_timing_derate -delay_corner dly_crnr_slow_RCMAX -net_delay -data -late 1.01
set_timing_derate -delay_corner dly_crnr_slow_RCMAX -net_delay -clock -early 0.9
set_timing_derate -delay_corner dly_crnr_slow_RCMAX -net_delay -clock -late 1.1
update_timing -full
report_constraint -all_violators  > rep.vio.gz
report_analysis_coverage > rep.coverage
report_timing -path_type endpoint -max_slack 0 > setup.rpt
report_timing -path_type endpoint -max_slack 0 -early > hold.rpt
```

Saving and Restoring DMMMC Sessions

You can save and restore distributed MMMC sessions.

The save/restore flow is as follows:

1. The Tempus software reads the distributed MMMC data using the following command:

   distribute_read_design -design_script design_load.tcl

2. You can save the design using the following command:

   distribute_views -write_design *list_of_views* (or you can specify all)

   The distribute_views -write_design parameter creates N parallel saved design databases, where N is the number of views specified in the list of views. You can provide a list of views for which the saved session is required.

3. Start another Tempus master session to restore the above saved sessions. You can also restore individual saved sessions in a non-distributed Tempus run.

4. You can use the following command to create N Tempus clients, each of which will load one saved session:

```
distribute_read_design -read_db list_of_views -read_dir
pathname_of_directory_of_saved_session
```

5. The `-read_db` parameter specifies the design views to be restored.

The following script examples show distributed MMMC save and restore flows:

Distributed MMMC Save Flow

The following example saves sessions for N views:

```
set_distributed_hosts -local | -rsh ...
set_multi_cpu_usage -remote_hosts N
distribute_read_design -design_script design_load.tcl -out_dir Output
set Views [list func_rmax func_rcmin test_rcmax]
distribute_views -views $Views -script analysis.tcl –save_design all
```

Distributed MMMC Restore Flow

The following example restores saved sessions for N views:

```
set_distributed_hosts -local
set_multicpu_usage -remote_hosts N
set Views [list func_rmax func_rcmin test_rcmax]
```

output is the same directory used in the above example where views were saved.

```
distribute_read_design -restore_design $Views -restore_dir Output -outdir
new_output
distribute_command {report_timing}
```

To restore a single view from DMMMC saved session on a single machine environment use the following commands:

```
read_db Output/view1.enc.dat <topcell>
report_timing
```

The Output File Structure

The MMMC commands generate output data in directories as view names inside the specified output directory. You can specify the output directory in the `distribute_read_design` command with the `-out_dir` parameter. For example, consider the following scenario:

1. A design containing three views (view1, view2, and view3).
2. Run `distribute_read_design -design_script design_load.tcl –outdir Output` command.
3. Specify the following commands in sta_dmmmc.tcl (script-3):

   ```
   report_constraint -all_violators  > rep.vio.gz
   ```

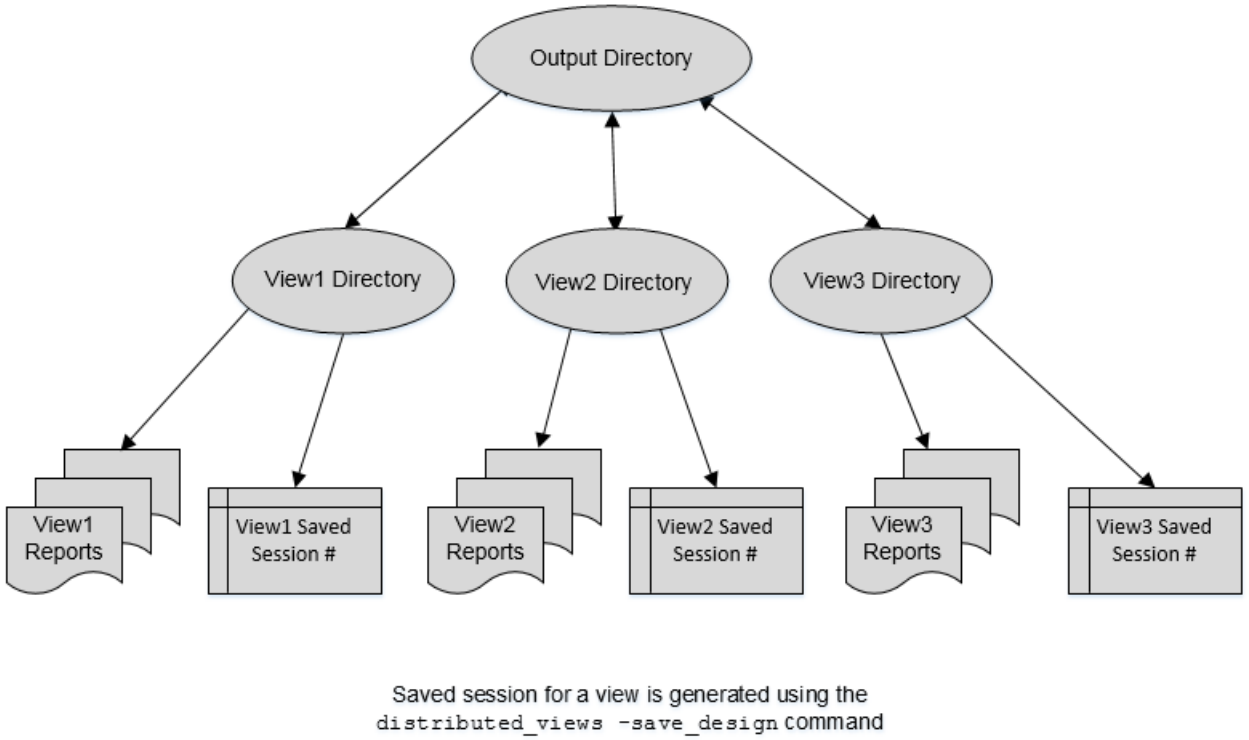
```
report_analysis_coverage > rep.coverage
report_timing -path_type endpoint -max_slack 0 > setup.rpt
report_timing -path_type endpoint -max_slack 0 -early > hold.rpt
```

The software will generate the following three reports inside each view's directory:

```
/dmmmc_logs/func_fast_RCMAX/setup.rpt
/dmmmc_logs/func_fast_RCMAX/setup.rpt
```

All the Tempus client-specific .log/.cmd files reports will be saved in the `./dmmmc_logs` directory. The directory structure is shown in the figure below.

Figure 6-2 Output directory structure in DMMMC mode



6.4 Aggregating Timing Reports

After processing the views in distributed mode, segregated reports will be available for each view in a specific directory that is to be analyzed. One of the prime requirements is to have a holistic look of all the analysis views for further action.

The `merge_timing_reports` command can be used to merge reports that are generated using the reporting commands, such as, `report timing` and `report constraint`.

You can merge view specific standard timing reports that can be generated using the `report_timing` and `report_constraints` commands.

For GUI-based timing analysis, you can merge view-specific timing reports generated in machine readable format (using `report_timing -output_format gtd` command in analysis.tcl), to create a single merged report, which can be loaded in Tempus Global Timing Debug (GTD).

Merging of timing reports is done based on slack criticality. As each path will contain the "view" field that represents the view to which a path belongs, so analysis of critical views becomes easier.

- Output generated from the `merge_timing_reports` command will be saved in a user-specified output file.

6.5 Performing Concurrent MMMMC Analysis in DMMMC Mode

By default, the `distribute_views` command distributes a single view for each Tempus client. The command can also perform concurrent multi-mode multi-corner (CMMMC) timing analysis within the DMMMC framework. You can combine the analysis views for CMMMC analysis on Tempus client(s). This can be done in the following two ways:

- **Delay Corner-Based Auto Grouping:**

  You can specify a simple list of view names using the `distribute_views -view` option and use the `-combine_num_views` option to specify the maximum number of views to be combined per Tempus client. For example:

  ```
  distribute_views -combine_num_views 4 -views {setup1 setup2 setup3
  setup4 setup5 setup6 setup7 setup8} -script ...
  ```

  Assuming `setup1` to `setup4` to have delay corner `dc1` and `setup5` to `setup8` to have delay corner `dc2`, then the above example will group `setup1` to `setup4` to Tempus `client1`, and `setup5` to `setup8` will be grouped to Tempus `client2`.

- **User-Controlled View Grouping:**

  You can specify a list of list formats for combining multiple concurrent views on Tempus client(s). For example,

  ```
  distribute_views -views {{setup1 setup2 setup3 setup4} {setup5 setup6
  setup7 setup8}} -script ...
  ```

  The above command will group `setup1` to `setup4` views to Tempus `client1` and will group `setup5` to `setup8` views to Tempus `client2`, irrespective of the delay corner.

To get optimum run time while using view grouping, it is recommended to use delay corner-based auto grouping, as mentioned in the example above.

6.6 Setting Up MSV/CPF-Based Design in DMMMC Mode

In this flow, the common power format (CPF) file should not contain any timing related information. All the library data is present in the viewdefinition.tcl file.

Script -1 (master.tcl):

Same as in non-MSV DMMMC flow.

Script -2 (loadDesign.tcl):

The following points should be noted while migrating to this flow:

1. All the library related data and operating conditions are defined in the viewdefinition.tcl file.
2. Power domains are created in the power.cpf file. This file should not contain any timing or operating conditions related information.
3. Timing data and PVT conditions are linked to power domains, by using the `update_delay_corner -power_domain` command in the `viewdefinition.tcl` file.
4. Before reading the CPF, the `update_delay_corner` command can be used to:

   a) Create place-holder domains.

   b) Populate domains with appropriate MSV data during CPF loading.

Sample design_load.tcl:

```
######################## Script 2 : "loadDesign.tcl" ##########################
##### View definition file does not have LEFs.
read_physical -lef {  ../../libs/lef/FreePDK45_lib_v1.0.lef   \
../../libs/MACRO/LEF/pllclk.lef  \
../../libs/MACRO/LEF/ram_256X16A.lef  \
../../libs/MACRO/LEF/rom_512x16A.lef}

read_mmmc viewDefinition.tcl
set_interactive_constraint_modes functional_mode

# MANUALLY TRANSLATE (WARN-12): Argument '-ignore_undefined_cell' for command
'set_top_module' is obsolete in common UI and is removed.
set_db init_read_netlist_allow_undefined_cells false
read_netlist "../../design/dtmf_recvr_core.v.gz" -top dtmf_recvr_core
init_design
read_def "../../design/full.def"
########################## viewdefinition.tcl ############################
```

```
################################################
# Define the available libraries
################################################
create_library_set -name libset_slow\
    -timing\
[list ../../libs/liberty/FreePDK45_lib_v1.0_worst.lib\
../../libs/MACRO/LIBERTY/pllclk.lib\
../../libs/MACRO/LIBERTY/ram_256x16A.lib\
../../libs/MACRO/LIBERTY/rom_512x16A.lib]
create_library_set -name libset_fast\
    -timing\
[list ../../libs/liberty/FreePDK45_lib_v1.0_typical.lib\
../../libs/MACRO/LIBERTY/pllclk.lib\
../../libs/MACRO/LIBERTY/ram_256x16A.lib\
../../libs/MACRO/LIBERTY/rom_512x16A.lib]

################################################
# Define the available PVT corners
################################################
create_rc_corner -name corner_worst_RCMAX\
    -pre_route_res 1.2\
    -post_route_res {1.2 1.2 1.2}\
    -pre_route_cap 1.2\
    -post_route_cap {1.2 1.2 1.2}\
    -post_route_cross_cap {1.2 1.2 1.2}\
    -pre_route_clock_res 0\
    -pre_route_clock_cap 0
create_rc_corner -name corner_worst_RCMIN\
    -pre_route_res 0.8\
    -post_route_res {0.8 0.8 0.8}\
    -pre_route_cap 0.8\
    -post_route_cap {0.8 0.8 0.8}\
    -post_route_cross_cap {0.8 0.8 0.8}\
    -pre_route_clock_res 0\
    -pre_route_clock_cap 0

################################################
#Create the timing conditions
################################################
create_timing_condition -name default_mapping_tc_2\
    -library_sets [list libset_fast]
create_timing_condition -name default_mapping_tc_1\
    -library_sets [list libset_slow]

################################################
# Create the possible delay corners
# These are combinations of timing conditions and corners
################################################
create_delay_corner -name dly_crnr_slow_RCMAX\
    -timing_condition {default_mapping_tc_1}\
    -rc_corner corner_worst_RCMAX
create_delay_corner -name delay_corner_fast_RCMAX\
    -timing_condition {default_mapping_tc_2}\
    -rc_corner corner_worst_RCMAX
create_delay_corner -name delay_corner_slow_RCMIN\
    -timing_condition {default_mapping_tc_1}\
    -rc_corner corner_worst_RCMIN
create_delay_corner -name delay_corner_fast_RCMIN\
    -timing_condition {default_mapping_tc_2}\
```

```
    -rc_corner corner_worst_RCMIN

################################################
# Define the available constraint types
################################################
create_constraint_mode -name functional_mode -sdc_files [list ../../design/
dtmf_recvr_core.pr.sdc]
create_constraint_mode -name scan_mode       -sdc_files [list ../../design/
dtmf_recvr_core.scan.sdc]

################################################
# Create the possbile views
# These are combinations of constraints, and delay corners
################################################
create_analysis_view -name func_slow_RCMAX -constraint_mode functional_mode -
delay_corner dly_crnr_slow_RCMAX
create_analysis_view -name func_fast_RCMAX -constraint_mode functional_mode -
delay_corner delay_corner_fast_RCMAX
create_analysis_view -name func_slow_RCMIN -constraint_mode functional_mode -
delay_corner delay_corner_slow_RCMIN
create_analysis_view -name func_fast_RCMIN -constraint_mode functional_mode -
delay_corner delay_corner_fast_RCMIN

create_analysis_view -name scan_slow_RCMAX -constraint_mode scan_mode -
delay_corner dly_crnr_slow_RCMAX
create_analysis_view -name scan_fast_RCMAX -constraint_mode scan_mode -
delay_corner delay_corner_fast_RCMAX
create_analysis_view -name scan_slow_RCMIN -constraint_mode scan_mode -
delay_corner delay_corner_slow_RCMIN
create_analysis_view -name scan_fast_RCMIN -constraint_mode scan_mode -
delay_corner delay_corner_fast_RCMIN

################################################
# Turn on the views that you want to use.
# Not all created views need to be used every time.
################################################
set_analysis_view -setup [list \
func_slow_RCMAX \
func_fast_RCMAX \
func_slow_RCMIN \
func_fast_RCMIN \
scan_slow_RCMAX \
scan_fast_RCMAX \
scan_slow_RCMIN \
scan_fast_RCMIN \
] -hold [list \
func_slow_RCMAX \
func_fast_RCMAX \
func_slow_RCMIN \
func_fast_RCMIN \
scan_slow_RCMAX \
scan_fast_RCMAX \
scan_slow_RCMIN \
scan_fast_RCMIN \
]
```

Script - 3 (sta_dmmmc.tcl):

This script contains all the SPEF reading, analysis, and reporting commands.

```
###################### Script 3: "sta_dmmmc.tcl" ##########################
read_spef -rc_corner corner_worst_RCMAX ../../design/SPEF/
corner_worst_RCMAX.spef.gz
read_spef -rc_corner corner_worst_RCMIN ../../design/SPEF/
corner_worst_RCMIN.spef.gz

set_db delaycal_enable_si true
################################
# Setup threading
################################
set_multi_cpu_usage -local_cpu 2     ;# Not using multiple machines

################################
# Setup some global timer variables
################################
set_db delaycal_enable_si true ;# Turn on SI analysis
set_db si_glitch_enable_report   true ;# Turn on glitch analysis
# set_global timing_write_sdf_allow_negative_setuphold_sum true ;# Enable SDF to
have negative delays

################################
# Select report formatting
################################
set_db timing_report_precision 3
set_table_style -name report_timing -max_widths {70}
set_db timing_report_fields {hpin edge cell transition  load delay incr_delay
arrival required}

################################
# set timing derates here if you have them
################################
set_timing_derate -delay_corner dly_crnr_slow_RCMAX -cell_delay -data -early 0.82
set_timing_derate -delay_corner dly_crnr_slow_RCMAX -cell_delay -data -late 1.01
set_timing_derate -delay_corner dly_crnr_slow_RCMAX -cell_delay -clock -early 0.9
set_timing_derate -delay_corner dly_crnr_slow_RCMAX -cell_delay -clock -late 1.1
set_timing_derate -delay_corner dly_crnr_slow_RCMAX -net_delay  -data  -early 0.82
set_timing_derate -delay_corner dly_crnr_slow_RCMAX -net_delay  -data  -late  1.01
set_timing_derate -delay_corner dly_crnr_slow_RCMAX -net_delay  -clock -early 0.9
set_timing_derate -delay_corner dly_crnr_slow_RCMAX -net_delay  -clock -late  1.1

update_timing -full
report_constraint -all_violators  > rep.vio.gz
report_analysis_coverage > rep.coverage
report_timing -path_type endpoint -max_slack 0 > setup.rpt
report_timing -path_type endpoint -max_slack 0 -early > hold.rpt

# Merge these reports later in the main script
report_timing -output_format gtd -max_path 10 -max_slack 10
>setup_maxpath_machine.rpt
report_timing -output_format gtd -nworst 1 > setup_nworst_machine.rpt
report_timing -output_format gtd -max_path 10 -max_slack 10 >
hold_maxpath_machine.rpt
report_timing -output_format gtd -nworst 1 > hold_nworst_machine.rpt
```

6.7 Performing Setup/Hold View Analysis and Reporting

You can perform view specific analysis and reporting in DMMMC mode.

The script example below demonstrates how to perform view-specific reporting. The following example shows 4 views. The section in the analysis.tcl script, which is provided as input to the distribute_views command, generates setup.rpt and hold.rpt reports for two views each:

Example: Performing view-specific reporting

```
set mySetupViews "view1 view2"
set myHoldViews "view3 view4"
```

#Reports For Setup Analysis Views

```
set currentView [all_setup_analysis_views]
set reportSetup 0
foreach vName $currentView { if { [lsearch $mySetupViews $vName ] != -1 } { set
reportSetup 1 ; break; } }
if { $reportSetup } {
```

All Setup View reporting Commands

```
report_timing -late > setup.rpt}
```

#Reports for hold analysis views

```
set currentView [all_hold_analysis_views] ;
set reportHold 0
foreach vName $currentView { if { [lsearch $myHoldViews $vName ] != -1 } { set
reportHold 1 ; break; } }
if { $reportHold } {
```

All Hold View reporting Commands.

6.8 Running Single Interactive User Interface in DMMMC Mode

The Tempus software supports distributed MMMC interactive interface mode. In this mode, some commands that are run in a master session are also implemented to all the Tempus clients.

The following are distributed MMMC interactive interface commands:

- set_distributed_mmmc_mode
- distribute_command
- get_distribute_variables

The DMMMC interactive interface is available only after the views have been dispatched to Tempus clients.

The DMMMC interactive mode is successfully enabled when the Tempus shell prompt changes from the standard `tempus>` to `tempus_dmmmc>`. Once in interactive mode (set using `set_distributed_mmmc_mode -interactive true` setting), the commands are no longer needed to be dispatched using the `distribute_command {}`, and can be directly run in the Tempus master session.

The DMMMC interactive mode supports reporting commands, timing constraints, file writing commands (such as `write_sdc`) and ECO commands.

The DMMMC interactive interface is available only after the `distribute_views` commands are run.

Example:Distributed MMMC Interactive Flow

The following example shows the distributed MMMC interactive flow:

```
tempus> set viewnames {view01 view02 ...}
tempus> distribute_read_design -restore_design (or you can use distribute_views -
views $viewnames)
```

To enable DMMMC interactive mode.

```
tempus> set_distributed_mmmc_mode -views $viewnames -interactive true
```

Displays timing for each selected view on master console.

```
tempus_dmmmc> report_timing
```

Run some ECO commands on interactive views.
Performs ECO in each view-server independently.

```
tempus_dmmmc> change_cell -inst U1 -upsize
```

Run some reporting commands on interactive views.
Displays updated timing for each view on the master console.

```
tempus_dmmmc> report_timing
tempus_dmmmc> report_timing -view view01
```

Will turn on all the loaded view-servers for interactive mode.

```
tempus> set_distributed_mmmc_mode -interactive true -views all
```

Apply constraint commands on interactive views.

```
tempus_dmmmc> set_interactive_constraint_modes [all_constraint_modes -active]
tempus_dmmmc> set_input_delay -clock CK 0.2 {EN}
tempus_dmmmc> set_case_analysis 0 SEL
```

The `distribute_command` can be run several times to create a synchronized flow with multiple commands.

6.9 Running Interactive ECO on Multiple Views in DMMMC Mode

You can perform the following steps interactively by using the interactive DMMMC feature, after basic distributed analysis has been performed:

- Perform What-If ECO and check results from multiple views using `-evaluate_all` or `evaluate_only` parameters of ECO commands.
- Perform ECO commit operation in Tempus on multiple views in parallel environment.
- Run `get_property` query to know the slack and other property values with ECO commands from multiple views.
- Save distributed session with ECO.

The above steps can be executed on the interactive prompt, or directly from the master in the distributed environment.

Example: Interactive ECO Usage in Distributed MMMC Environment on Interactive prompt

#MultiCore Machine: Local mode (Default)

```
tempus> set_distributed_host -local
```

#RSH mode

```
tempus> set_distributed_host -rsh -add {machine1 machine2}\
```

OR

#LSF mode useful for getting a machine from a remote farm environment

```
tempus> set_distributed_host -lsf -queue normal –lsf Resource "OSNAME==Linux && OSREL==EE50"
tempus> set_multi_cpu_usage -remote_host 2 -cpu_per_remote_host 8
```

Create servers with basic MMMC design

```
tempus> distribute_read_design -design_script design_load.tcl -out_dir output
tempus> set views [list view01 view02]
```

Run analysis script on select views at remote machines

```
tempus> distribute_views -views $views -script analysis.tcl
```

Distribute Interactive Mode commands, scripting, and so on

```
tempus> set_distributed_mmmc_mode -interactive true -views all
```

Get reports back on master console

```
tempus_dmmmc> report_timing -through [get_cells i2]
```

Scripts examples to be run on remote hosts

```
tempus_dmmmc> set clocks [get_clocks *]
tempus_dmmmc> set clock_names [get_object_name $clocks]
tempus_dmmmc> set totalClocks [sizeof_collection $clocks]
```

Perform What-If ECO and check results from multiple views using -evaluate_all / -evaluate_only options of ECO commands

```
tempus_dmmmc> change_cell -inst i2 -upsize -evaluate_all
tempus_dmmmc> change_cell -inst i2 -upsize -evaluate_only
```

Perform ECO commit operation in Tempus on multiple views in parallel environment

```
tempus_dmmmc> change_cell -inst i2 -upsize
```

Perform `get_property` query to know slack and other property values with ECO commands from multiple views

```
tempus_dmmmc> report_timing -through [get_cells i2]
tempus_dmmmc> set newSlack [get_property [get_pins i2/A] slack_max_rise]
tempus_dmmmc> set_distributed_mmmc_mode -interactive false
tempus> get_distribute_variables newSlack
```

Save distributed session with ECO

```
tempus> distribute_views -write_design all
tempus> puts "Waiting remote servers to exit"
```

Merge Analysis reports

```
tempus> merge_timing_reports -view_dirs {view01 view02} -base_report setup_sta.rpt
-base_dir output > merged_setup.rpt
tempus> merge_timing_report -view_dirs {view01 view2} -base_report hold_sta.rpt -
base_dir output > merged_hold.rpt
```

LOG Snapshot from Interactive ECO

```
tempus_dmmmc 13> change_cell -inst i2 -upsize -evaluate_all
Variable ::_dmi_result(view01) on master session updated with value '1'.
Variable ::_dmi_result(view02) on master session updated with value '1'.
### RESULTS FROM VIEW view02 ###
---- Cell: slow/INVX16 (view: view02) ----
Through-object Slack: 1.842
Max Tran Slack: 4.3224
---- Cell: slow/INVX12 (view: view02) ----
Through-object Slack: 1.831
Max Tran Slack: 4.3495
---- Cell: slow/INVX8 (view: view02) ----
Through-object Slack: 1.784
Max Tran Slack: 4.0948
### RESULTS FROM VIEW view01 ###
---- Cell: slow/INVX16 (view: view01) ----
Through-object Slack: 0.542
Max Tran Slack: 4.3224
---- Cell: slow/INVX12 (view: view01) ----
```

```
Through-object Slack: 0.530
Max Tran Slack: 4.3495
---- Cell: slow/INVX8 (view: view01) ----
Through-object Slack: 0.484
Max Tran Slack: 4.0948
tempus_dmmmc 14> change_cell -inst i2 -upsize -evaluate_only
Variable ::_dmi_result(view01) on master session updated with value '1'.
Variable ::_dmi_result(view02) on master session updated with value '1'.

### RESULTS FROM VIEW view02 ###
---- Cell: slow/CLKINVX1 (view: view02) ----
Through-object Slack: 1.671
Max Tran Slack: 4.3935

### RESULTS FROM VIEW view01 ###
---- Cell: slow/CLKINVX1 (view: view01) ----
Through-object Slack: 0.371
Max Tran Slack: 4.3935
0


tempus_dmmmc 15> change_cell -inst i2 -upsize
Variable ::_dmi_result(view01) on master session updated with value '1'.
Variable ::_dmi_result(view02) on master session updated with value '1'.

### RESULTS FROM VIEW view02 ###
Resize i2 (INVX1) to CLKINVX1.

### RESULTS FROM VIEW view01 ###
Resize i2 (INVX1) to CLKINVX1.
0
tempus_dmmmc 18> set newSlack [get_property [get_pins i2/A] slack_max_rise]
Variable ::_dmi_result(view01) on master session updated with value '1.3085'.
Variable ::_dmi_result(view02) on master session updated with value '-0.9915'.
0
tempus_dmmmc 21> set_distribute_mmmc_mode -interactive false
tempus 22> distribute_views -save_design all
CPUs with views 'view01 view02' have been enabled for interactive distributed
mmmc mode.
Variable ::_dmi_result(view01) on master session updated with value '1'.
Variable ::_dmi_result(view02) on master session updated with value '1'.

### RESULTS FROM VIEW view02 ###
Writing Netlist "output_di_mmmc/view02/mmmc.enc.dat/mmmc.v.gz" ...
Saving timing graph ...
Saving configuration ...
Saving preference file output/view02/mmmc.enc.dat/enc.pref.tcl ...
Saving parasitic data in file 'output/view02/mmmc.enc.dat/mmmc.rcdb.d' ...
Writing RC database file using in-memory parasitic updation done by ECO commands.
Creating parasitic data file 'output/view02/mmmc.enc.dat/mmmc.rcdb.d/header.da' in
memory efficient access mode for storing RC.

### RESULTS FROM VIEW view01 ###
Writing Netlist "output_di_mmmc/view01/mmmc.enc.dat/mmmc.v.gz" ...
Saving timing graph ...
Saving configuration ...
Saving preference file output/view01/mmmc.enc.dat/enc.pref.tcl ...
Saving parasitic data in file 'output/view01/mmmc.enc.dat/mmmc.rcdb.d' ...
Writing RC database file using in-memory parasitic updation done by ECO commands.
Creating parasitic data file 'output/view01/mmmc.enc.dat/mmmc.rcdb.d/header.da' in
memory efficient access mode for storing RC.
```

6.10 Important Guidelines and Troubleshooting DMMMC

This section provides information on various kinds of error scenarios.

Specify all the parasitic file loading commands (pertinent to all RC corners used in the design) in the analysis.tcl (Script 3) script that are accepted as an argument to the `distribute_views` command.

- If distributed MMMC session is not getting launched, you need to check the following:
  - There are no errors in your design loading file (loadDesign.tcl) that is used as an argument to the `distribute_read_design` command.
  - If the error message "`Unable to open socket, connection refused`" is displayed, then check for items listed in bullets 'c' to 'g'. In all these cases, the DMMMC system will issue the appropriate error/warning messages.
  - Log on to the machines specified as resources to the DMMMC session. You must have direct login access to these machines (that is, without requiring a password at the time of login). Update your host settings to avoid this issue.
  - Make sure that the specified Tempus clients in the DMMMC setup are not heavily loaded. Enough resources should be available on each Tempus client to process a view.
  - If using LSF mode, then the required LSF variable and environmental settings should be done prior to the launch. Similarly, for rsh the DMMMC system assumes you have configured the RSH access to the Tempus client properly.
  - The commands that are supported beyond timing and noise analysis flow are not used in the scripts. The MMMC parameters and options related to RC extraction must be removed from the loadDesign.tcl to run the flow smoothly.
- The `set_analysis_view` command is not allowed in any of the scripts that are accepted as input to the DMMMC system. Specify a list of views using the `distribute_views` command. This command will distribute these views internally to the Tempus clients for processing.
- All merging commands work on the Tempus master and the output of these commands will be placed in the specified output file. These merge commands will not follow the protocol of moving the output reports to the specific output area, as mentioned above. This applies to any command that is capable of generating an output specific to a view.
- Single machine MMMC and distributed MMMC cannot be used concurrently in the same Tempus master session.

- DMMMC is a batch mode execution feature - distributed save/restore and interactive command entry capabilities are supported in Tempus.
- Merged noise report will not specify the view name for each record of delay/glitch. You will see multiple entries of records where each record will correspond to a specific Tempus client. To match the record to a view and make noise reports handy, you need to generate text reports along with the merged view text report.

7

Distributed Static Timing Analysis

7.1 Tempus Timing Analysis Overview on page 414
7.2 Static Timing Analysis (STA) on page 414
7.3 Distributed Static Timing Analysis (DSTA) on page 414
7.4 Converting STA to DSTA on page 416
7.5 Hardware Recommendations for DSTA on page 417
7.6 Accessing Specific Machines using LSF on page 418
7.7 DSTA Log Files on page 419
7.8 DSTA Operational Tips on page 420
7.9 Distributed Static Timing Analysis Flow on page 423
7.10 Additional Commands Supported in DSTA on page 424
7.11 Commands Not Supported in DSTA Mode on page 427

7.1 Tempus Timing Analysis Overview

The Tempus Timing Analysis tool provides enhanced performance and capacity to real world designs by using an innovative massive parallel computer architecture. Timing Analysis has two modes of operation:

- Static Timing Analysis (STA)
- Distributed Static Timing Analysis (DSTA)

These are described below.

7.1.1 Static Timing Analysis (STA)

Static timing analysis (STA) refers to a traditional single-machine, multi-threaded use model. Tempus operates efficiently in the STA model with improved performance due to highly optimized multi-threaded routines.

To launch Tempus in the GUI environment, use the following command:

```
tempus -stylus
```

Use the following command to launch Tempus without the GUI:

```
tempus -stylus -no_gui
```

7.1.2 Distributed Static Timing Analysis (DSTA)

Distributed static timing analysis (DSTA) refers to the master/client architecture that fully leverages multiple threads on master/client processes allowing maximum scalability for growing and complex designs. Apart from the overall run time reduction, another significant advantage of the DSTA master/client architecture is the overall per-process memory usage reduction. The master holds the entire design while each of the clients hold a portion of the current timing graph.

You must execute the main script on the master machine - work is shared between multiple client machines.

To launch Tempus in DSTA mode, use the following command:

```
tempus -stylus -distributed
```

Distributed Tempus does not have a GUI mode.

To run the DSTA flow, you must specify the following additional commands:

- set_distributed_hosts
- distribute_start_clients
- distribute_partition

Note: All the STA commands are not supported in the DSTA mode. For more details, see section Commands Not Supported in DSTA Mode on page 427.

Using STA Versus DSTA

The following table describes some of the key features of STA and DSTA:

STA	DSTA
■ Utilizes a single machine and supports multi-threading. ■ Multi-threaded performance has improved over the earlier timing analysis solution - Encounter Timing System (ETS). ■ Maintains signoff accuracy. ■ Ideal for small designs (<15M instances).	■ Utilizes multiple parallel machines and supports multi-threading. ■ Leverages large number of less powerful machines. ■ Reduces overall runtime on large designs. ■ Reduces per-process memory footprint on large designs. ■ Maintains signoff accuracy. ■ Requires minor script changes. ■ Ideal for large designs (>15M instances).

The choice of the static timing analysis mode depends on the design size and runtime.

- A one million instance design may not realize the benefits of DSTA.
- A 200 million instance design is not possible using a single machine STA.
- A 15 million instance design may choose STA or DTSA based on runtime expectations.

If you need to reduce the run time, and have already maximized the threading of STA, then Tempus DSTA is the best choice. Alternatively, if the run time is less than an hour and your memory requirements are not a limiting factor, then STA should be selected. Running STA on a large design will cause longer run time and memory usage. Similarly, running DSTA on a small design will consume processors with no significant gain.

Performance Considerations for STA and DSTA

Use the following to improve performance of static timing analysis:

- LDB for libraries
- RCDB for parasitics
- Pre-defined Tcl constraints
- Local disks with fast read/write speed

7.2 Converting STA to DSTA

The procedure to convert a STA script to a DSTA script is given below:

- Run a single view in the STA mode (single machine, 8 threads).
- Investigate if there are any errors.
- Read the library file.
- Read the verilog file.
- Read the SPEF file.
- Read constraints.
- Update timing.
- Custom reporting.

The table below shows sample scripts for STA and DSTA flows:

STA Script	**DSTA Script**

# Non-distributed script 	
`puts "Starting script:`
`set_multi_cpu_usage -local_cpu 4`
`read_netlist`
`read_libs`
`set_top_module`
`read_spef`
`read_sdc`
`update_timing`
`report_timing`
	# Distributed script
	`puts "Starting script:`
	`set_multi_cpu_usage …`
	`set_distributed_hosts …`
	`distribute_start_clients`
	`read_verilog`
	`read_libs`
	`set_top_module`
	`read_spef`
	`distribute_partition`
	`read_sdc`
	`update_timing`
	`report_timing`

The basic scripting rules for DTSA are:

- You must have at least 2 clients.
- Use the set_multi_cpu_usage command before starting a client(s).
- Load parasitics before running the distribute_partition command.
- Load constraints after the distribute_partition command.

The set_multi_cpu_usage command can be specified as shown below:

```
set clientCnt     2
set threadCnt     4
set_multi_cpu_usage -cpuPerRemoteHost $threadCnt \
-localCpu $threadCnt -remoteHost $clientCnt
```

You can specify the LSF parameters using the set_distributed_hosts command:

```
set_distributed_hosts -timeout 300 -lsf -queue lnx64 \
  -args {-n 4 -R "(CPUS>=4) span[hosts=1] rusage[mem=15000]"}
```

7.3 Hardware Recommendations for DSTA

The following are the guidelines for running DSTA:

- Start with 4 clients, 8 threads per client. Typically, 4 client DSTA will have half the run time of a 0 client STA.
- Add clients for each additional ~15 million instances.
- All clients must have similar performance architecture. The heterogeneous machines will result in dissimilar client run times – slowest client dictates overall run time.
- Memory allocation - 1.5GB peak memory per million instances (master and client).

An example of a 100 million instance design is given below:

- 5 - 10 machines
- 8 - 16 CPUs per machine
- 128 GB RAM per client
- Run time 3-5 hours

The following table shows reasonable client counts and thread counts for various sizes of designs:

Design Instance Count	Client Count	Client Thread Count	Master Thread Count	Resulting Processor Count	Comments
A few million	2	2	2	6	For a small tutorial design
20 million	4	4	4	20	-
50 million	4	8	8	40	-
100 million	8	8	8	72	-
200 million	12	8	12	108	■ Increased client count saves memory
400 million	12	12	16	160	■ Increased client count saves memory

7.4 Accessing Specific Machines using LSF

You can specify the following LSF parameters to use specific machines:

- To avoid a bad machine, use:

```
bsub -q ssv -m ssvpe -n 4 \
   -R "(hname != sjfsb416) rusage[mem=3000] " "xterm"
```

- To target a specific machine, use:

```
bsub -q ssv -m ssvpe -n 4 \
     -R "hname = sjfsb416 rusage[mem=3000] " "xterm"
```

- To target a group of machines, use:

```
bsub -q ssv -m ssvpe -n 4 \
    -R "(hname == sjfsb415 || hname == sjfsb416 || \
    hname == sjfsb417 || hname == sjfsb418 || \
    hname == sjfsb419) rusage[mem=30000] "xterm"
```

7.5 DSTA Log Files

Master Log

Master logs are unique to DSTA. You can name a log file using the following command:

```
tempus -log
```

The default log file name is `./tempus.log`. The file naming increments to `tempus.log1` in the next run. If you do not want to increment, but want to overwrite the existing log file, use the `tempus -overwrite` command.

The master log shows that the master machine starts the clients, and loads the design. The client machines do most of the work with constraints, timing, and reports.

Sample Master Log File

```
--- Running on machine1(x86_64 w/Linux 2.6.18-308.el5) ---

This version was compiled on Sun Sep 1 14:53:30 PDT 2013.

INFO (DSTA-1025): Sourcing /icd/flow/ETS/ETS132/13.20-b059_1/lnx86/share/
tcltools/icd8.5.9/lib/tcl8.5/history.tcl
INFO (DSTA-1025): Sourcing ../blank/run_dsta.tcl

INFO (DSTA-1017): Starting 2 client processes.
INFO (DSTA-1013): Reading verilog file: ./zondammav10gs80.final.v
INFO (DSTA-1600): Loading library file: ./GS80_W_-40_0.92_0.92_CORE.lib.gz
INFO (DSTA-1025): Sourcing ./tim_ets_settings.tcl

INFO (DSTA-1193): Reading spef file: ./zondammav10gs80.spef.maxc_maxvia_-40.gz

INFO (DSTA-1421): After generating partitions: Cpu=00:01:43 Real=00:01:40
Peak_mem=4257meg Cur_mem=4257meg

INFO (DSTA-1020): Running command: update_timing -full
INFO (DSTA-1389): Cond arc const client 1 starts at 1378131471.
INFO (DSTA-1389): Cond arc const client 0 starts at 1378131522.
INFO (DSTA-1389): Cond arc const client 0 all done at 1378131523.
INFO (DSTA-1389): Cond arc const client 1 all done at 1378131523.
```

Client Log

The client log files are the same as the Tempus STA log files. Each client runs a copy of Tempus. The client machine has its own directory and log file, as shown below:

```
partOutput_0/tempus.log
partOutput_1/tempus.log
```

The log files will increment to `tempus.log1` in the next run.

Note: It is possible that you have client logs that are numbered in an unordered manner, as shown below:

```
partOutput_0/tempus.log12
partOutput_1/tempus.log6
```

This is caused due to multiple runs with different number of clients. It is recommended that you check the time stamp to ensure that the logs are for the specific master/client set.

Sample Client Log File

```
--- Starting "Tempus Timing Solution v13.20-b059_1" on Mon Sep  2 12:38:07 2013
(mem=70.9M) ---
 Server is up on machine1 ( PID=830 )
 Multi-CPU acceleration using 4 CPU(s).

 Scheduling LEF file(s) ./uc_CS402LND.lef to be loaded when the set_top_module
command is issued.
 Scheduling LEF file(s) ./uc_CS402LZD.lef to be loaded when the set_top_module
command is issued.
 Scheduling  timing library file(s) ./LIB/scIMux_f_worst.lib to be loaded when the
set_top_module command is issued.
 Scheduling  timing library file(s) ./LIB/scxOpt_f_worst.lib to be loaded when the
set_top_module command is issued.

 Set top cell to scTop.
 ** info: there are 406704 modules.
 ** info: there are 13891666 stdCell insts.
 ** info: there are 53 Pad insts.
 ** info: there are 293926 macros.

 <CMD> report_timing
 Analyzing view default_emulate_view with delay corner[0]
default_emulate_delay_corner, rc corner[0] ...
 All-RC-Corners-Per-Net-In-Memory is turned ON...
 Analyzing view default_emulate_view with delay corner[0]
default_emulate_delay_corner, rc corner[0] ...
 Analyzing view default_emulate_view with delay corner[0]
default_emulate_delay_corner, rc corner[0] …
```

7.6 DSTA Operational Tips

Although the operating environment of DSTA is similar to STA, you need to manage the new operational requirements in the distributed mode to maximize the DSTA performance. This section provides operating tips for the DSTA environment:

Name the Master and Client Logs

Master

- To name the master log and overwrite the old log file, use the following command:

  ```
  tempus -distributed -overwrite -log tempus.log
  ```

- Consider using 4c8t to indicate 4 clients 8 threads, use:

```
    tempus -distributed -overwrite -log tempus_4c8t.log
```

Client

To avoid randomly numbered log file names, delete, or rename the old directories before the next run, use the following command:

```
rm -rf partOutput*
```

Save the DSTA Logs

Saving the master and client log files allows comparison of various client/thread configurations and scripts. To save logs, use the following commands:

```
mkdir saveLog1
cp –rfp tempus*.log pathOutput* saveLog1
```

Master Log Completion

Add a print message at the end of the script to confirm completion of DSTA.

```
# Master run.tcl Script
     . . .
     . . .
     puts “All done”
```

Use a Single run.tcl for both STA and DSTA

You can use a single `run.tcl` file for both STA and DSTA. This can ensure that there are no errors when DSTA is selected over STA. This can be done by wrapping the DSTA commands with a '`if`' statement, and allowing STA to skip these commands and DSTA to use them.

For example,

```
if {[info command distribute_partition] != "" } {
   puts "You are using Tempus DSTA"
   set_multi_cpu_usage -localCpu 4   -cpuPerRemoteHost 4   -remoteHost 2
} else {
  puts "You are using Tempus STA"
  set_multi_cpu_usage -localCpu 4
}
```

When you start Tempus, it automatically triggers the '`if`' statement.

Configure the Script for both Batch and Interactive Modes

You can run a script in both the batch and interactive modes without changing the Tcl commands, by specifying the following in the `run.tcl` file:

```
# Master run.tcl Script
. . .
. . .
return     ; # Stops the script if it is in the interactive mode but does not exit
exit       ; # Exits Tempus if it is in the batch mode
```

Tracking Master Runtime

You can use the following command to track the start to finish runtime:

```
# Initalize at the top of the master run.tcl
set initTimeTick     [clock seconds]
. . .
. . .
. . .
# Calculate runtime at the bottom of the master run.tcl
set lastTimeTick   [clock seconds]
set totalTime        [expr $lastTimeTick - $initTimeTick]
set totalMin          [format "%4.1f" [expr $totalTime / 60.0]]
puts "The total walltime minutes was: $totalMin minutes"
```

Tracking Master Memory

You can track the master current and peak memory using the following command:

`report_resource`

This command will print the resource usage for the master. The following is an example output:

```
Cpu=03:02:02 Real=02:31:31 Peak_mem=70502meg Cur_mem=70494meg
```

Tracking Client Runtime and Memory

To print the resource usage for each client, you can use the following command:

`distribute print client usage`

This command does not interrupt or wait for whatever the client is currently doing.

A sample output of this command is given below:

```
INFO (DSTA-1039): Client resource usage:
 Client 0: Cpu=00:44:25 Real=01:25:57 Peak_mem=11095meg Cur_mem=9816meg
 Client 1: Cpu=00:44:29 Real=01:25:58 Peak_mem=11338meg Cur_mem=10072meg
```

```
Client 2: Cpu=00:44:25 Real=01:25:57 Peak_mem=11095meg Cur_mem=9816meg
Client 3: Cpu=00:44:29 Real=01:25:58 Peak_mem=11338meg Cur_mem=10072meg
```

Monitoring tmp Disk Space

You can measure the /tmp free disk space by inserting the following code at the beginning and at the end of the script:

```
set dir /tmp
set a [lindex [lindex [split [exec df -k $dir] \n] end] end-2]
set a [expr {$a / 2.0**20 }]
set localTmpGigs [format "%0.1f" $a ]
puts "The /tmp disk has $localTmpGigs gigs free"
```

7.7 Distributed Static Timing Analysis Flow

The DSTA flow consists of the following steps:

- Define and start DSTA clients:

  ```
  set_multi_cpu_usage
  set_distribute_host ...
  }
  distribute_start_clients
  ```

- Load the design. The constraints must be read after the parasitics.

  ```
  source load_libs.tcl
  source load_netlist.tcl
  source load_parasitics.tcl
  ```

- Initiate DSTA partitioning:

  ```
  distribute_partition
  ```

- Load constraints and time the design:

  ```
  source load_constraints.tcl
  update_timing -full
  distribute_print_client_usage
  ```

- Generate PBA reports:

  ```
  set reportName rt_max_200K.rpt.gz
  report_timing -late -max_paths 200000 -nworst 1 -path_type \
  full_clock -net > $reportName
  set reportName rt_max_pba_200K.rpt.gz
  set maxPathCnt 200000
  ```

```
set nWorstCnt 1
report_timing -retime path_slew_propagation -max_paths $maxPathCnt \
-nworst $nWorstCnt -path_type full_clock > $reportName
```

If a pre-defined sorting criteria is not defined, then the timing report output is displayed in any order.

Note: The path numbers that are assigned to each path in the timing report are not reported in DSTA output reports.

7.8 Additional Commands Supported in DSTA

The following commands are supported in both non-distributed Tempus and DSTA:

- write_sdf
- write_twf
- read_db
- write_db
- report_disk

The use models of `write_sdf` and `write_twf` commands are the same for both DSTA and non-distributed Tempus. The use models of these commands have been modified for DSTA, as given below:

- `write_db`: Allows you to save a snapshot of the design in the distributed timing analysis mode. All the design data, partitioning information, and client sessions are saved in the directory specified using the `write_db` command. The saved data size can be 2 times the size of the flat saved session due to partition overlap.
- The command usage of `write_db` is:

  ```
  Usage: write_db [-help] <out_dir> [-oa_lib_cell_view {libname
  cellname viewname} ] [-no_rc] [-overwrite]
  ```

  Where `-oa_lib_cell_view` and `-no_rc` parameters are not supported in DSTA.

  Example

  ## DSTA setup

  ```
  set_multi_cpu_usage -local_cpu 4 -remote_host 2 -cpu_per_remote_host 4
  set_distributed_hosts -local -time_out 300
  ```

  ## Start clients

```
distribute_start_clients
read_libs
read_netlist
set_top_module
read_spef/read_rcdb
distribute_partition
read_sdc
report_timing
```

Save current session

```
save_design <session> -overwrite
```

- `read_db`: Enables you to restore the analyzed design in the distributed mode for further reporting (or analysis).

  The command usage of `read_db` is:

  ```
  Usage: read_db [-help] [-oa_lib_cell_view {libname cellname viewname}][-physical_data]
  ```

  Where `-oa_lib_cell_view` and `-physical_data` parameters are not supported in DSTA.

  Example

  ## DSTA setup

  ```
  set_multi_cpu_usage -local_cpu 4 -remote_host 2 -cpu_per_remote_host 4
  set_distributed_hosts -local -time_out 300
  ```

  ## restore saved session

  ```
  read_design distSave -cell <cellname>
  ```

  ## Design is now in fully analyzed state

  The number of clients must be same for the saved and restored sessions. The number of master and client threads can be changed in the `read_db` from that used in the `write_db`.

- `report_disk`: Measures the disk metrics of the current working directory and the /tmp directory. The output of this command allows you to investigate Mb/sec throughput of the disk and disk I/O requirements. This command is supported in both distributed (DSTA) and non-distributed Tempus.

  The command usage of `report_disk` is:

  ```
  report_disk [-help] [-dir directory_name]
  ```

  Example

  ```
  dsta> report_disk
  DISK_INFO : 206.66 Mb/S   381G total   1.1G used   359G free   /tmp
  {206.66 381G 1.1G 359G}
  ```

```
dsta> distribute_start_clients -count 2
INFO (DSTA-1017): Starting 2 client processes.
Connected to sjfsb439 39262 1 ( PID=31660 )
Connected to sjfsb439 35186 0 ( PID=31658 )
INFO (DSTA-1019): Received application variables on master.
0
dsta> report_disk
DISK_INFO (Master)   : 182.53 Mb/S   381G total   1.1G used   359G
free   /tmp
DISK_INFO (Client 0) : 94.90 Mb/S    381G total   1.1G used   359G
free   /tmp
DISK_INFO (Client 1) : 104.28 Mb/S   381G total   1.1G used   359G
free   /tmp
{182.53 381G 1.1G 359G} {94.90 381G 1.1G 359G} {104.28 381G 1.1G 359G}
```

Sample Scripts

STA

The following is a sample STA Tcl script:

Non-distributed script

```
puts "Starting script:

set_multi_cpu_usage -local_cpu 4
read_netlist
read_libs
read_spef
read_sdc
update_timing
report_timing
```

DSTA

```
set clientCnt   4
set threadCnt   2
set lsfQueue   ssv
set hostGroup   ssvpe
set maxMem    91000
set_multi_cpu_usage -cpu_per_remote_host $threadCnt -local_cpu \
$threadCnt -remote_host $clientCnt
set_distributed_hosts -time_out 300 -lsf \
-queue ${lsfQueue} -args "-m ${hostGroup} -n ${threadCnt} \
-R \"(CPUS>=${threadCnt}) span\[hosts=1\] \
rusage\[mem=${maxMem}\]\""
 }
distribute_start_clients
```

```
source load_libs.tcl
source load_netlist.tcl
source load_parasitics.tcl

distribute_partition

source load_constraints.tcl
update_timing -full

distribute_print_client_usage

set reportName rt_max_200K.rpt.gz
report_timing -late  -max_paths 200000 -nworst 1 -path_type \
full_clock -net > $reportName

set reportName rt_max_pba_200K.rpt.gz
set maxPathCnt 200000
set nWorstCnt 1

report_timing -retime path_slew_propagation -max_paths $maxPathCnt \
-nworst $nWorstCnt -path_type full_clock > $reportName
```

7.9 Commands Not Supported in DSTA Mode

The following commands are not supported in DSTA mode:

System Commands - Basic Tool-Tcl Commands

- `gui_show`
- `gui_hide`

General Commands

- `update_inst_name`
- `get_metric`
- `write_testcase`
- `set_preference`
- `write_flow_template`

Design Import Commands

- `reset_design`
- `read_def`
- `set_license_check`
- `write_def`
- `write_rcdb`

Multiple-CPU Processing Commands

- check_multi_cpu_usage

Distributed MMMC Commands

- distribute_command
- distribute_design_views
- distribute_read_design
- distribute_views
- exit_serversreset_servers
- get_distribute_variables
- reset_distributed_views
- set_distribute_variables

Timing Constraint Commands

- write_load

Timing Analysis Commands

- create_boundary_model
- create_path_scope
- gui_show_timing_map
- get_constant
- get_propagated_clock
- merge_scope
- merge_timing_reports
- read_boundary_model
- read_scope
- read_scope_model
- read_scope_spef
- read_scope_verilog
- report_annotated_assertions
- report_cell_instance_timing
- report_clock_sense

- report_critical_insts
- report_freq_violation
- report_lib_cells
- report_slack_histogram
- report_statistical_timing_derate_factors
- report_wire_load
- write_annotated_transition
- write_loop_break_sdc

Tcl Interface Commands

- get_activity
- get_power

Signal Integrity Commands

- report_voltage_scaling
- report_voltage_swing
- set_outbound_report
- write_noise_eco_nets

Timing Model Commands

- check_timing_model
- write_timing_model
- merge_model_timing
- report_timing_model

Interface Logic Models - ILM

- create_ilm_from_files
- read_ilm
- set_ilm_mode
- set_ilm
- write_ilm

Timing Debug Commands

- analyze_paths_by_default_path_group
- analyze_paths_by_bottleneck
- analyze_paths_by_clock_domain
- analyze_paths_by_critical_false_path
- analyze_paths_by_drv
- analyze_paths_by_hier_port
- analyze_paths_by_hierarchy
- analyze_paths_by_view
- create_path_category
- delete_path_category
- write_histogram_view
- gui_gtd_highlight_timing_report
- read_path_categories
- read_timing_debug_report
- write_path_categories
- write_category_summary
- write_text_timing_report

Manual ECO Commands

- attach_module_portconnect_hpin
- delete_nets
- disconnect_hpin

Clock Tree Debugger Commands

- gui_ctd_close
- gui_ctd_highlight
- gui_ctd_open
- gui_ctd_get_window_id
- gui_ctd_get_title
- gui_ctd_set_title

RC Extraction Commands

- `extract_parasitics`
- `write_parasitics`
- `report_rcdb`
- `report_unit_parasitics`

Power Calculation Commands

- `report_unannotated_nets`
- `read_activity_map_file`
- `propagate_activity`
- `read_activity_file`
- `report_inst_power`
- `report_power`
- `report_vector_profile`
- `reset_power_activity`
- `read_power_db`
- `set_default_switching_activity`
- `set_dynamic_power_simulation`
- `set_instance_temperature_file`
- `set_power`
- `set_power_analysis_temperature`
- `set_power_include_file`
- `set_power_output_dir`
- `set_twf_attribute`
- `set_virtual_clock_network_parameters`
- `write_power_constraints`
- `write_tcf`

Unsupported Parameters

The `-tcl_list` parameter of all the **report_*** commands, is not supported in the DSTA mode.

8

Signoff ECO

8.1 Signoff ECO Overview on page 434

8.2 Key Features of Signoff ECO on page 435

8.3 Signoff ECO Flow on page 436

8.4 Setting Up Signoff ECO Environment on page 437

- Starting ECO on page 438
- Terminating ECO on page 439

8.5 Signoff Optimization Use Models on page 439

- Generating ECO Timing DB and Running ECO Timing Closure in Separate Sessions on page 439
- Running Signoff Optimization with ECO Timing Generated in Batch Mode Using DMMMC on page 441
- Running Signoff Optimization with ECO Timing Generated Using CMMMC on page 442
- Parallel Distributed Interactive MMMC Analysis and ECO (Paradime) Flow on page 442
- Running Signoff Timing/Power Optimization from Innovus on page 449

8.6 Basic Signoff ECO Optimization Techniques on page 452

8.7 Fixing SI Violations in Tempus Signoff ECO on page 455

- SI Glitch Violations on page 455
- SI Slew Violations on page 456
- SI Crosstalk Delta Delay Violations on page 457

8.8 Fixing IR Drop in Tempus Signoff ECO on page 458

8.9 Path-Based Analysis (PBA) Mode Optimization on page 459

8.10 Total Power Optimization on page 460

8.11 Setup Timing Recovery After Large Leakage or Total Power Optimization on page 461

8.12 Recommendations for Getting Best Total Power Optimization on page 462

8.13 Hierarchical ECO Optimization on page 463

- Sample Script for Hierarchical Flow on page 464
- Optimization of Master/Clone Designs on page 464

8.14 High Capacity ECO Flow for Timing and Power Closure on page 467

8.15 Generating Node Locations in Parasitic Data on page 474

- Using Innovus Engines on page 474
- Using Standalone Quantus on page 475
- Using a Third-Party Tool on page 475

8.16 Optimization Along Wire Topologies on page 475

8.17 Optimization using Endpoint Control on page 476

8.18 Metal ECO Flow on page 479

8.19 Handling Large Number of Active Timing Views Using SmartMMMC on page 481

8.20 Performing Clock Skewing for Setup Timing Closure on page 483

8.21 ECO Flow Use Model for Cell EM Analysis on page 485

8.22 Hierarchical Top+Boundary Model ECO Flow on page 487

8.1 Signoff ECO Overview

Tempus ECO is a licensed feature available in Tempus to address certain signoff concerns. Tempus ECO is easy to plug in any Signoff analysis flow and allows the user to clean up the remaining timing violations as well as reducing the power consumption of the netlist. Also, this feature allows fixing timing violations on large designs (multi-million instances) with many analysis views. You can run the ECO feature in Tempus by invoking Tempus using the `-eco` option.

In Signoff environment, designers always find some remaining timing violations that must be cleaned up. These violations can be due to many reasons:

- Block-level versus top-level timing analysis do not match.
- Imprecise Timing budgeting, which has generated the setup/hold/drv/cap/glitch violations.
- Timing was not optimized across all the analysis views during implementation flow to avoid over-fixing, and the designer is relying on the Signoff stage to catch final violations. The final violations are identified at the Signoff stage for which the absolute number can be slightly different in the implementation flow.
- Often additional corners and/or modes are added at the time of signoff.
- There may be timing model differences.
- Implementation tools cannot optimize timing with full Signoff STA precision.

Instead of back-annotating the timing into the implementation tool, it is much easier to stay in the Signoff environment and perform various transforms to fix the remaining violations. In the Signoff stage, the netlist changes committed are written in an ECO file, which will be used in the implementation tool to perform incremental place and route.

In addition to timing closure, Tempus ECO is also a solution to reduce overall power consumption of the netlist. At Signoff stage, some amount of timing pessimism is removed due to Path Based Analysis (PBA), also called as retiming, and that extra timing margin can be used to perform netlist change leading to overall power reduction. This is especially beneficial when AOCV or SOCV timing modes are enabled.

Related Topics

- Key Features of Signoff ECO
- Signoff ECO Flow
- Setting Up Signoff ECO Environment

- Signoff Optimization Use Models
- Basic Signoff ECO Optimization Techniques

8.2 Key Features of Signoff ECO

The key features of Tempus ECO are as follows:

- Focus on timing and power optimization while being MMMC-aware, STA Signoff aware, SI-aware, and Physical-aware (optional).
- Highly scalable concurrent and distributed timing analysis.
- Supports PBA timing and power optimization to benefit from reduced timing pessimism.
- Targets minimal netlist change for timing closure.
- Considers local density and routing congestion when adding buffers in Physical-aware mode.
- Supports MSV designs and hierarchical based designs (including different row height).
- Supports replicated hierarchies (also named as Master/Clone optimization).
- Honors all modules/instances/cells/nets/clocks specific attributes.
- Requires only a single ECO loop to close timing.
- Supports large capacity in netlist size (>200M gates) and the number of active views timed (>100).
- Supports multi-threaded ECO optimization, on-route buffering, clock skewing, and dummy load cell insertion.

Usage of Signoff ECO Features

These features can be used in the scenarios mentioned below to fix timing violations:

Full flat assembled design in hierarchical flow

The goal is to fix the inter-partition timing violations after assembling the design on full flat DB.

In this scenario, the timing violations to be fixed can be large but their number should not be big. This means that you can have some paths with negative slack because those are inter-partitions paths, which might not have optimized correctly earlier in the flow. But the number of such violations is usually not too much because there are not many such paths.

Block implementation flow

At the block level, there may be remaining violations at the Signoff stage. This is because one methodology is to remove any pessimism in the timing constraints used by the implementation tool, and then rely on the Signoff STA tool to fix the real remaining violations.

In this scenario, since optimization is already performed once in the flow, thus the timing violations to fix are probably small.

Here the challenge is to perform precise fixing across all analysis views without creating any new violations.

This is typically where Path Based Analysis mode would be applied in order to allow Power, Performance, Area push with Signoff timing analysis accuracy.

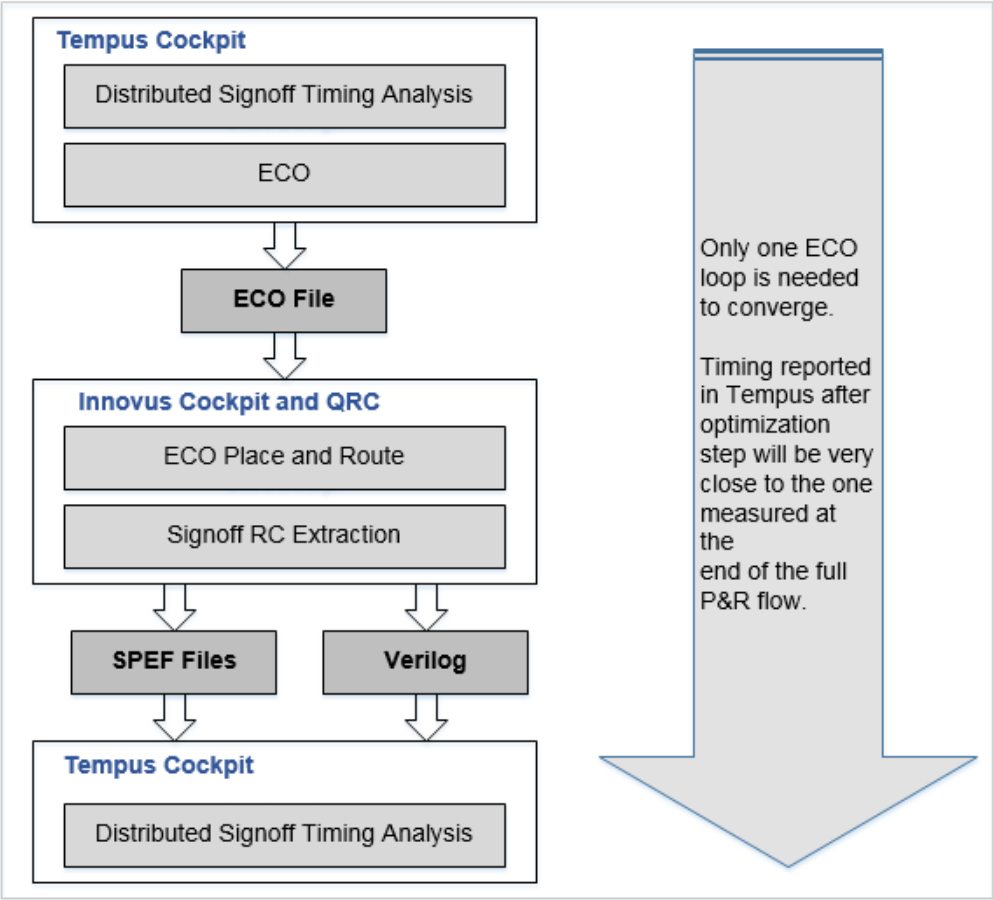
Related Topics

- Signoff ECO Overview
- Signoff ECO Flow
- Setting Up Signoff ECO Environment
- Signoff Optimization Use Models
- Basic Signoff ECO Optimization Techniques

8.3 Signoff ECO Flow

The diagram below shows how Tempus ECO fits in the flow. The design is first loaded into Tempus for signoff timing analysis. To address any remaining timing violations, Tempus ECO is executed, generating an ECO file. This file is then passed to the implementation tool to apply the Tempus-generated ECOs to the design. After the Tempus ECO is implemented, ECO place-and-route is performed, followed by design re-extraction. The updated design or Verilog netlist along with the re-extracted SPEF files is then brought back into Tempus for another round of signoff timing analysis.

Figure 8-1 ECO Flow



Related Topics

- Signoff ECO Overview
- Key Features of Signoff ECO
- Setting Up Signoff ECO Environment
- Signoff Optimization Use Models
- Basic Signoff ECO Optimization Techniques

8.4 Setting Up Signoff ECO Environment

This topic describes the tasks that you perform to set the ECO environment, and the commands to start and terminate ECO.

The input and output data, and various fixing methodologies and targets for setting up the ECO environment are as follows:

Input Data

- Technology data (timing libraries)
- Design data (Verilog, MMMC, SPEF)
- Optional design data (DEF, CPF/UPF, ILM, and LEF)

Timing and Power Optimization Methodology and Targets

- Uses cell swapping, cell resizing, and buffer/inverter insertion and deletion.
- Fixes timing violations, such as:
  - Hold timing
  - Setup timing
  - Design Rule (max_cap/max_tran)
  - SI violations (SI Slew, SI Xtalk and SI Glitch)
  - Reduces Power and Area:
    - Area reduction
    - Leakage power reduction
    - Dynamic power reduction
    - Leakage and dynamic power reduction concurrently

Output Data

- Detailed reporting on all the ECOs being performed.
- Detailed diagnostic reports on the remaining violations that are not fixed.
- Standard format ECO file.
- Final timing summary reports.

Starting ECO

To start a ECO session, type the following command with the appropriate parameters on the UNIX/Linux command line.

```
tempus > tempus -stylus -files <file_name> -tso
```

Terminating ECO

To end a ECO session, type "`exit`" in the Tempus shell.

Related Topics

- Signoff ECO Overview
- Key Features of Signoff ECO
- Signoff ECO Flow
- Signoff Optimization Use Models
- Basic Signoff ECO Optimization Techniques

8.5 Signoff Optimization Use Models

The following section describes the various signoff optimization use models:

- Generating ECO Timing DB and Running ECO Timing Closure in Separate Sessions
- Running Signoff Optimization with ECO timing generated in batch mode using DMMMC
- Running Signoff Optimization with ECO timing generated using CMMMC
- Parallel Distributed Interactive MMMC Analysis and ECO (Paradime) Flow
- Running Signoff Optimization from Innovus

Generating ECO Timing DB and Running ECO Timing Closure in Separate Sessions

ECO Timing DB is a lightweight timing graph built using the design input data. Since this timing graph is lightweight compared to the complete timing graph and stores only the data that is required for ECO fixing, it reduces the memory.

Within ECO fixing, Timing analysis is based on the hold and setup light-weight timing graph. Evaluation and commit time are also reduced when done on light-weight timing graphs. Therefore, it reduces both memory as well as turnaround time (TAT) significantly.

The `write eco opt db` command will save the ECO Timing DB after timing analysis, in non-distributed mode or in a DMMMC session. In case several views are active at the same

time, ECO Timing DB for each of them will be saved. The data will be saved in the directory pointed to by the set_db opt_signoff_write_eco_opt_db option.

Example

```
tempus > set_db opt_signoff_write_eco_opt_db mySignOffTGDir
tempus> write_eco_opt_db
```

This will save the ECO Timing DB information in the mySignOffTGDir directory.

The write_eco_opt_db command is also supported in the concurrent MMMC analysis mode. In addition, ECO Timing DB are saved in a directory that contains all active views' timing graph. So if the ECO Timing DBs are generated in a separate session, you can assemble them by simply copying into the same directory and use the following naming convention:

```
<topModuleName>_<viewName>.tgd and <topModuleName>_<viewName>.ckd
```

Example

The below figure shows that in Session 1, the clients performing timing analysis in parallel can be launched on the local machine, but can also be distributed on LSF farm (set_distributed_hosts -lsf).

Figure 8-2 ECO Timing Closure in Separate Sessions



Running Signoff Optimization with ECO Timing Generated in Batch Mode Using DMMMC

When `opt_signoff` is called without providing a pointer to ECO Timing DB database through `set_db opt_signoff_read_eco_opt_db` *`name`*, it will automatically generate the ECO Timing DB in batch mode.

Example of Hold fixing with embedded Distributed timing analysis:

```
read_libs $liberty
read_physical -lefs $lef
read_netlist $netlist
init_design
source viewDefinition.tcl
read_def $def_file
source spef.tcl
set_distributed_hosts -local
set_multi_cpu_usage -local_cpu 8 -remote_host 2 -cpu_per_remote_host 4
opt_signoff -hold
```

Note: `spef.tcl` must contain the `read_spef` or `read_parasitics` commands so that every active `rc_corner` is parasitic annotated.

Running Signoff Optimization with ECO Timing Generated Using CMMMC

For small to medium size designs where not many views are active (below 8), you can generate ECO Timing DB in CMMMC and perform ECO fixing in one single Tempus session,

Example of Hold fixing with ECO Timing DB generation and ECO fixing in one single Tempus CMMMC session:

```
read_libs $liberty
read_physical -lefs $lef
read_netlist $netlist
init_design
source viewDefinition.tcl
read_def $def_file
source spef.tcl
set_distributed_hosts -local
set_multi_cpu_usage -local_cpu 8
set_db opt_signoff_write_eco_opt_db myEcoTimingDB
write_eco_opt_db
set_db opt_signoff_read_eco_opt_db myEcoTimingDB
opt_signoff -hold
```

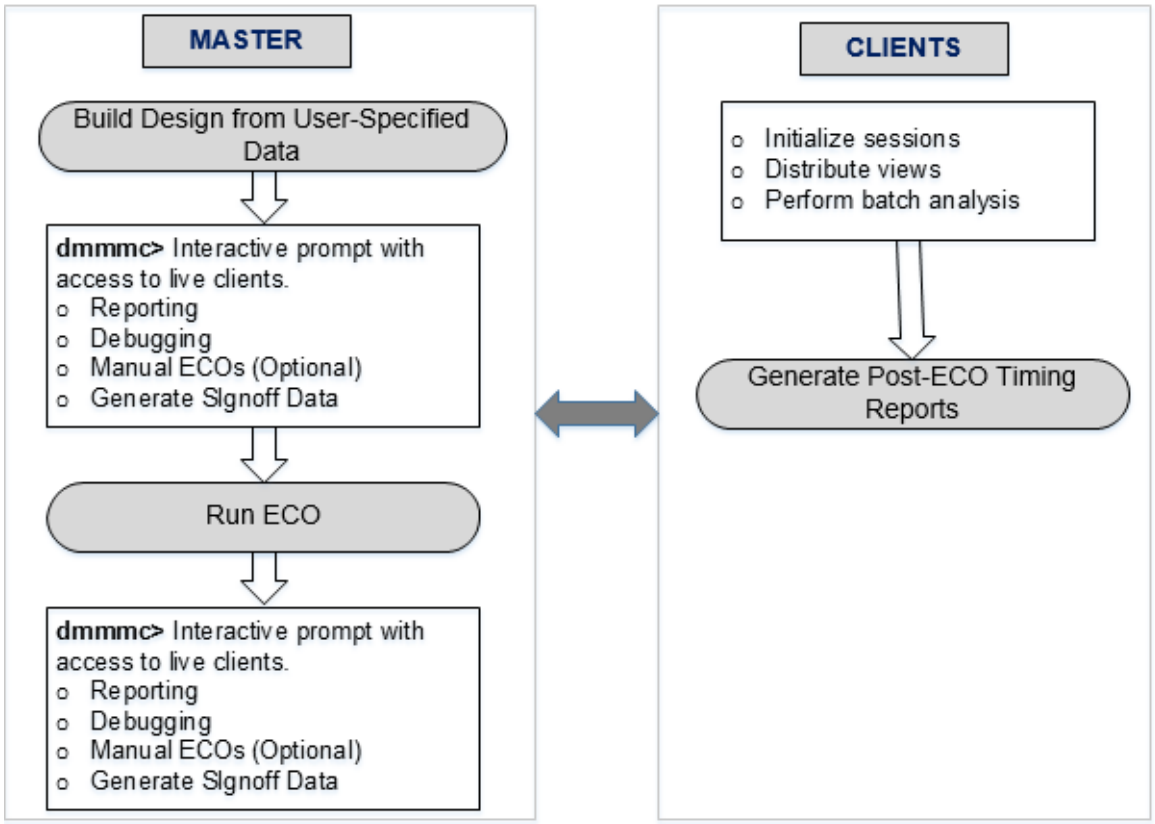
Note: `spef.tcl` must contain the `read_spef` or `read_parasitics` commands so that every `active rc_corner` is parasitic annotated.

Parallel Distributed Interactive MMMC Analysis and ECO (Paradime) Flow

The Paradime flow allow you to perform various tasks, such as distributed timing analysis, ECO, and any interactive debugging (such as, reporting, what-if analysis) in a single Tempus session leading to improved usability of the tool. The licensing requirements for this flow require Tempus to be invoked with the `-eco` license option for performing optimization in addition to the Tempus Distributed Multi-Mode Multi-Corner licensing requirements.

The following diagram shows the launch of the Paradime flow, where Tempus master is used for ECO runs and clients are used for timing analysis runs.

Figure 8-3 The Paradime Flow



Enabling the Paradime Flow

To enable this flow, you need to make the following setting in the script provided to the Tempus master.

```
set_distributed_mmmc_mode -eco true
```

In this flow, the number of remoteHost should be equal to number of views.

Running the Paradime Flow

The Tempus master and clients load the design data for running either of the following flows:

- Full analysis
- Restored analysis

Full Analysis Flow

To run the full analysis flow, you need to make the following settings:

```
distribute_read_design -design_script design_loading.tcl -out_dir output
distribute_views -script report.tcl -views {view1 view2}
```

Tempus master launches multiple DMMMC clients to load the design data as specified in `design_loading.tcl` and `run report.tcl` in order to perform full timing analysis runs and generate timing reports. Meanwhile, Tempus master also loads MMMC design for all active views of interest.

Once the design loading and timing analysis is done on the clients, the software is available with an interactive `dmmmc>` prompt with access to live clients. You can do the following:

- Perform merging of report after full timing analysis run or run manual commands to find out critical views of interest.
- Perform interactive querying and debugging on one, few, or all views of clients.

By default, all active views will be specified as setup and hold views for Tempus ECO run on master. If setup and hold views for Tempus ECO run are different, they can be specified using the variables below:

```
set_db eco_setup_view_list "<list of setup views>" # To specify setup views for
ECO run
set_db eco_hold_view_list "<list of hold views>" # To specify hold views for
ECO run
```

Sample Script

```
# To configure multiple CPUs
set_distributed_hosts -local set_multi_cpu_usage -local_cpu 8 -remote_host 2
-cpu_per_remote_host 8

# To enable the flow
set_distributed_mmmc_mode -eco trueset_distributed_mmmc_mode -eco_merge_mmmc

# To specify different Tempus version for DMMMC client runs
set_db eco_client_tempus_executable "/icd/flows/142_tempus/bin/tempus"

# Variables to specify list of setup and hold view for Tempus ECO (in case different
than the views restored on clients). By default, all clients' views will be in
setup/hold view list
set_db eco_setup_view_list "view1"
set_db eco_hold_view_list "view2"

#Variables to specify which view must be taken for power reporting
set_db eco_dynamic_view "view2"
set_db eco_leakage_view "view2"

# Variables to specify physical data
set_db eco_lef_file_list "tech.lef design.lef"
set_db eco_def_file_list "design.def"
set_db data_dir /a/b/c
distribute_read_design -design_script design_loading.tcl -out_dir
```

```
eco_restore_output_dmmmc
distribute_views -script report.tcl -views {view1 view2}

report_timing > before_ECO.rpt
opt_signoff  -hold
report_timing > after_ECO.rpt

The design_loading.tcl script is the following:

read_mmmc <view definition file>
read_netlist design.v
init_design

The report.tcl script is the following:

read_spef <all corner spef files>
update_timing –full
report* commands (optional)
```

Restored Analysis Flow

In this flow, saved sessions from individual STA runs are restored. These individual SMSC STA runs have unique view names and are saved in MMMC configuration using a view definition file with a unique library set, delay-corner, rc-corner, constraint-mode, analysis-view, and so on. This flow uses the saved SMSC sessions as clients. This is done by setting up the compatible DMMMC sessions to restore the flow structure, as shown below:

```
set_db Views [list func_rmax func_rcmin test_rcmax]
distribute_read_design -restore_design $Views -restore_dir
<pathname_of_directory_of_saved_session> -out_dir Output
```

Alternatively, a way to restore saved SMSC sessions is available if sessions are not saved in compatible DMMMC restore flow structure. You can provide a list of sessions' directories and corresponding view names as input in pairs as mentioned below:

```
set_db data_dir <unix path>
distribute_read_design -restore_design { {view1 $data_dir/view1/session.enc}
{view2 $data_dir/view2/session.enc} ...{viewN $data_dir/viewN/session.enc} } -
out_dir Output
```

Same number of clients are required as the number of sessions to be restored. Each saved session can have data for one MMMC view.

Data from same restored sessions is loaded on the master for all views together. List of setup and hold views for Tempus ECO run can be specified using the variables below:

```
set_db eco_setup_view_list "<list of setup views>" # To specify setup views for
Tempus ECO run
set_db eco_hold_view_list "<list of hold views>" # To specify hold views for Tempus
ECO run
```

All data for master is picked from the saved sessions, including constraints and parasitic information. Only the libraries are picked from the UNIX path that are given in the view definition file because the library data is not saved in saved sessions. In the scenario where there is no common view definition file for restore sessions, you can auto-merge (or

concatenate) all the view definition files, and create a single view definition file (which is the superset of all the individual sessions' view definition files) to load on the master.

If there is no common view definition file, you need to do the following settings:

```
set_distributed_mmmc_mode -eco_merge_mmmc
```

Sample Script

```
# To configure multiple CPUs
set_distributed_hosts  -local/-rsh/-lsf … set_multi_cpu_usage -local_cpu 8 -
remote_host 2 -cpu_per_remote_host 8
# To enable the flow
set_distributed_mmmc_mode -eco true
# To specify different Tempus version for DMMMC client runs
set_db eco_client_tempus_executable "/icd/flows/142_tempus/bin/tempus" -
# Variables to specify list of setup and hold view for Tempus ECO (in case different
than the views restored on clients). By default, all clients' views will be in
setup/hold view list.
set_db eco_setup_view_list "view1"


set_db eco_hold_view_list "view2"
# Variables to specify physical data


set_db eco_lef_file_list "tech.lef design.lef"


set_db eco_def_file_list "design.def


set_db data_dir /a/b/c
# Restoring SMSC saved sessions (not saved in DMMMC compatible directory structure)
     distribute_read_design -restore_design { {view1 $data_dir/setup_view1.enc}
{view2 $data_dir/hold_view2.enc} } -out_dir eco_restore_output_dmmmc
# Interactive DMMMC prompt pre-Tempus ECO
dmmmc> report_timing > before_ECO.rpt


opt_signoff -hold
# Interactive DMMMC prompt post-Tempus ECO
dmmmc> report_timing > after_ECO.rpt
```

Sample Script with Interactive Commands

```
# To configure multiple CPUs
set_distributed_hosts -local/-rsh/-lsf … set_multi_cpu_usage -local_Cpu 8 -
remote_host 2 -cpu_per_remote_host 8
# To enable the flow
set_distributed_mmmc_mode -eco true
```

```
# To specify different Tempus version for DMMMC client runs
set_db eco_client_tempus_executable "/icd/flows/142_tempus/bin/tempus" -

# Variables to specify list of setup and hold view for Tempus ECO (in case different
than the views restored on clients). By default, all clients' views will be in
setup/hold view list.
set_db eco_setup_view_list "view1"
set_db eco_hold_view_list "view2"

# Variables to specify physical data
set_db eco_lef_file_list "tech.lef design.lef"
set_db eco_def_file_list "design.def"
set_db data_dir /a/b/c

# Restoring SMSC saved sessions (not saved in DMMMC compatible directory structure)

distribute_read_design -restore_design { view1 view2 } -restore_dir $data_dir -
out_dir eco_restore_output_dmmmc

# Interactive DMMMC prompt pre-Tempus ECO

dmmmc> report_timing > before_ECO.rpt

# Perform some manual ECOs

dmmmc> eco_update_cell inst1 -up_size
dmmmc> eco_add_repeater -pins inst2/Y -cell BUFX6
dmmmc> set_db opt_signoff_add_inst false
opt_signoff -hold

# Interactive DMMMC prompt pre-Tempus ECO
dmmmc> report_timing > after_hold_ECO.rpt
opt_signoff -setup
```

Pre-ECO Interactive Shell

After the design loading and analysis (relevant only in full analysis flow) is done on clients, the interactive prompt is available to user. You can do report merge after full timing analysis run or run manual commands to find out critical views of interest. Any pre-requisite commands/setting for Tempus ECO run should be applied by this stage.

Interactive querying and debugging can be done on any number (one, few or all) of clients. To select few views, use:

```
distribute_command –views { <list of views to select>} { <list of commands>}
```

If any command (s) are to run on all views, use:

```
distribute_command { <list of commands>}
```

Manual ECOs (if any) must be done on Tempus master directly without `distribute_command`. Netlist update is done for all clients and master database. Tempus software detects if manual ECOs are performed using `distribute_command` and errors out.

Running ECO

When you run ECO file written by Tempus ECO, it is sent automatically to clients for post ECO signoff timing report generation. On completion of the Tempus ECO run, the master will return to interactive prompt with access to Tempus clients for signoff timing report generation and manual what-if analysis. You can also do successive Tempus ECO runs.

Note that running Tempus ECO as part of the Paradime flow is different from running normal ECO (when invoked from outside this flow) where all the `set_db` attributes are available in this flow except for the attributes below:

- `opt_signoff_read_eco_opt_db`: ECO cannot accept pre-generated ECO Timing DB from outside.
- `opt_signoff_write_eco_opt_db`: This parameter is ignored and ECO Timing DBs are stored in DMMMC output directory.

Post ECO, you can generate sign-off reports, debugging, perform manual what-if analysis and so on.

Post-ECO Interactive Shell

The interactive prompt after ECO is available to the user. It can be used for generating sign-off reports, debugging, doing manual what-if analysis, and so on. You can also run another Tempus ECO run at this stage.

Use Model

- Master does not have any timing information available. There is check in software to ensure that timer update is never triggered in master session.
- All timing related queries are to be done from clients using two ways:
  - Use `distribute_command` in non-interactive mode. You can also make selected views active in this mode.
    ```
    distribute_command –views {<list of views | all}
    ```
  - Use `set_distributed_mmmc_mode` in interactive mode. You can also make selected views active in this mode.
    ```
    set_distributed_mmmc_mode[-interactive {true | false} [-views
    {list_of_views | all}]]
    ```
- If any command needs to apply to both master and clients, it needs to be specified for both master and clients using separate commands. For example, if you need to specify timing derates, it needs to be done for master as well as clients:

- For master: Use `set_timing_derate….`
- For clients: Use `distribute_command {set_timing_derate …}`

Any settings/commands, which affect timing analysis should be applied to clients using `distribute_command {list of commands}`

Different Tempus version can be specified for client runs. This is useful in scenarios where timing analysis and ECO need to be run with different Tempus versions.
`set_db eco_client_tempus_executable "tempus executable path"`

Note: The `eco_client_tempus_executable` global variable has to be set before `distribute_read_db`.

Running Signoff Timing/Power Optimization from Innovus

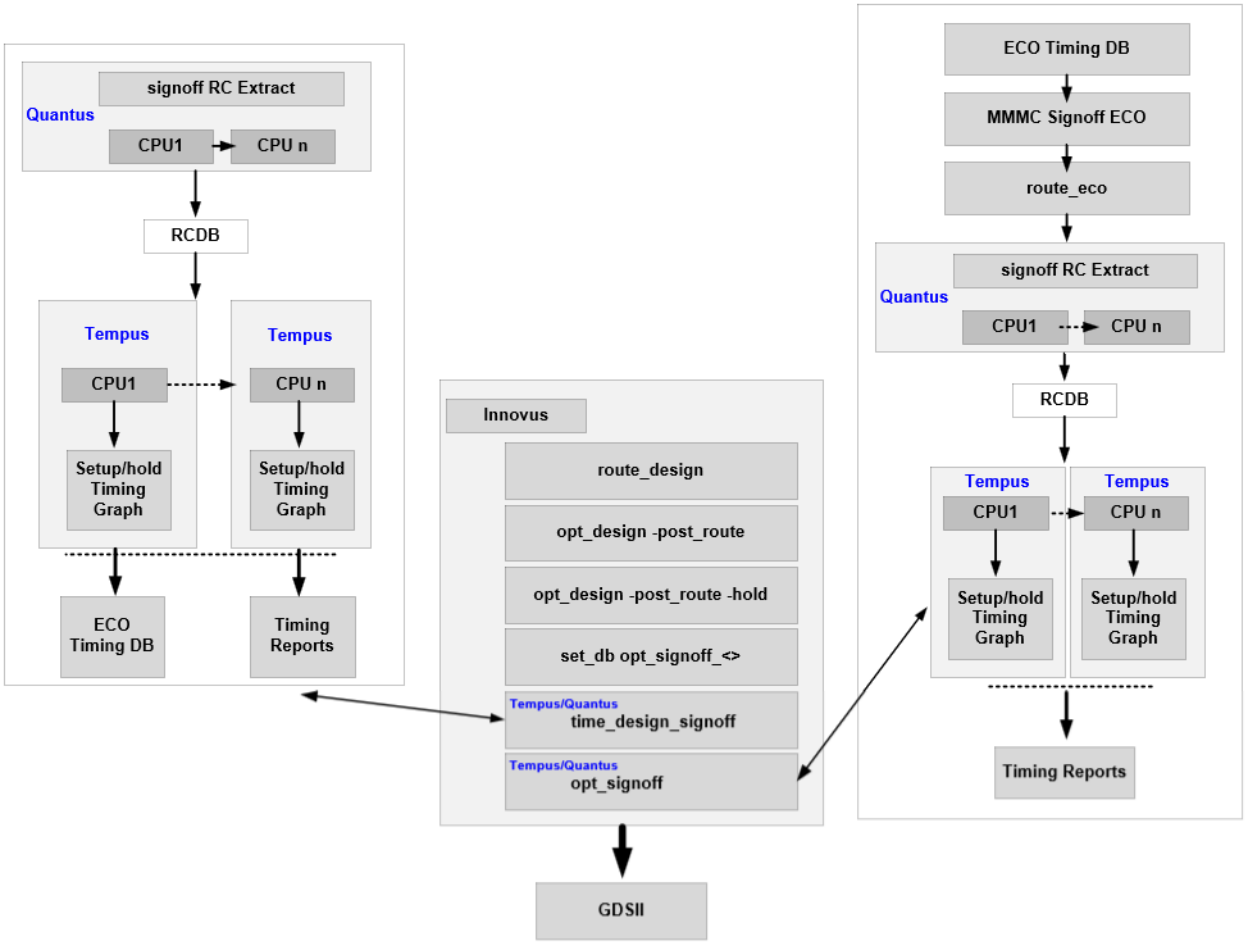
Signoff Timing Optimization feature allows you to run timing and power optimization within Innovus on Signoff parasitic from Quantus and Signoff timing from Tempus. This feature gives a complete automated solution for using the entire signoff tools through one high-level super command.

In a good timing closure methodology, at the implementation stage the timing targets that are set by the signoff Static Timing Analysis (STA) tool should be met. In order to ensure that the design state is close to sign-off quality, the timing reported by the implementation tool must correlate as much as possible to the signoff STA tool. From that stage, signoff timing optimization provides the following features:

- Allows you to run timing and power optimization on signoff timing within Innovus.
- Maximizes the usage of Cadence tools - Innovus will enable you to run extraction (Quantus) and Tempus without extra effort.
- To provide signoff timing report in Innovus using Tempus, you can use the `time_design_signoff` command of Innovus. This command uses Quantus and Tempus in standalone mode to perform signoff STA using the DMMMC infrastructure and saves an ECO Timing DB per view. This signoff timing can be optimized using `opt_signoff`. Similarly, the `opt_signoff` command can be used to perform power optimization on this signoff timing.

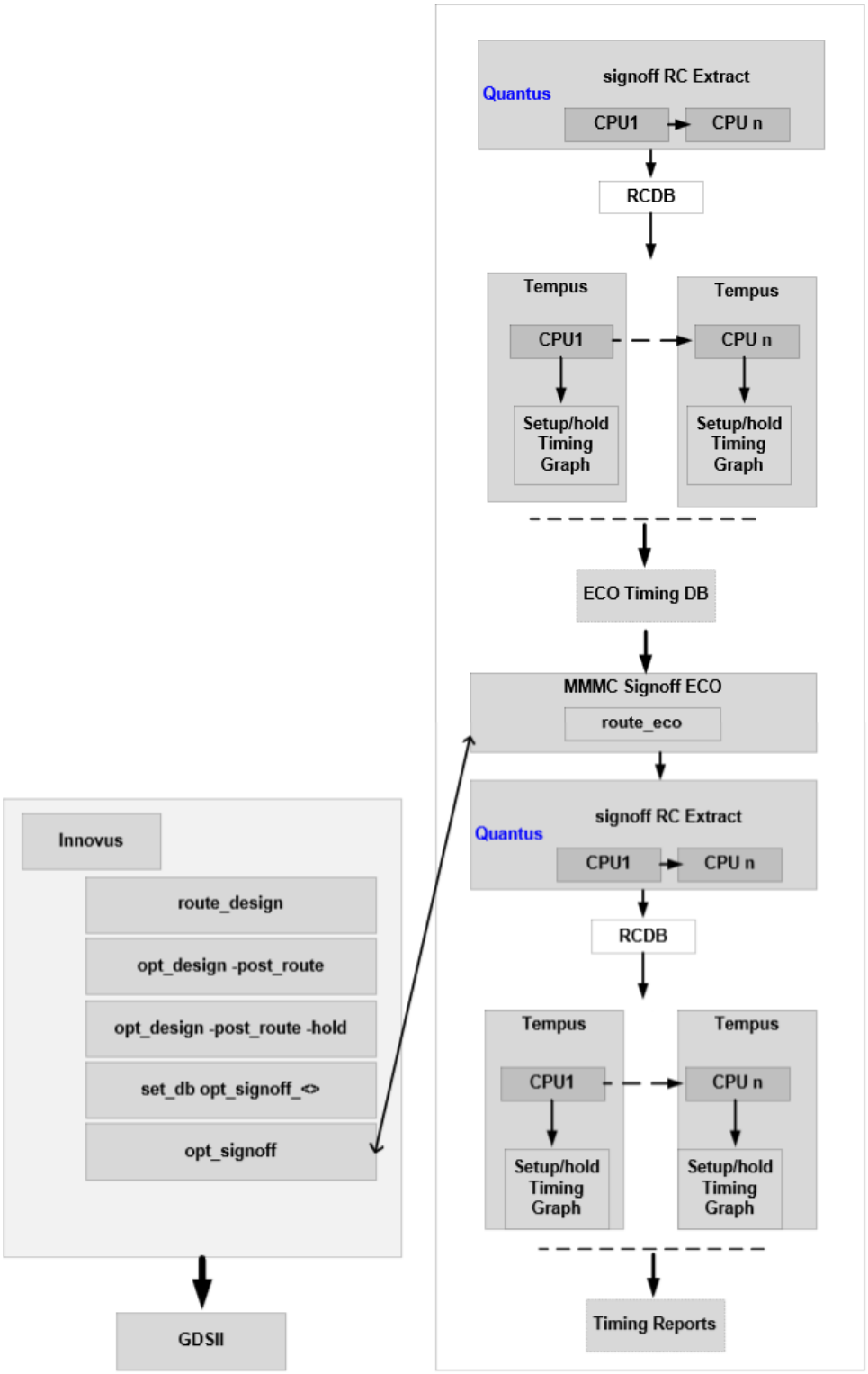
The following diagram illustrates the flow and the architecture of each signoff command:

Figure 8-4 Signoff Timing Optimization Flow



In case the ECO Timing DB is not available when the `opt_signoff` command is run, then the super command will automatically run `time_design_signoff`, as shown in the figure below:

Figure 8-5 Signoff Timing Optimization Flow


Quantus
signoff RC Extract
CPU1
CPU n
RCDB
Tempus
CPU1
Setup/hold Timing Graph
Tempus
CPU n
Setup/hold Timing Graph
ECO Timing DB
MMMC Signoff ECO
route_eco
Quantus
signoff RC Extract
CPU1
CPU n
RCDB
Tempus
CPU1
Setup/hold Timing Graph
Tempus
CPU n
Setup/hold Timing Graph
Timing Reports
Innovus
route_design
opt_design -post_route
opt_design -post_route -hold
set_db opt_signoff_<>
opt_signoff
GDSII

Signoff Timing Optimization provides a flexible flow to accommodate any specific methodology:

If parasitics should be extracted using TQuantus or IQuantus, this can be done manually by you. Then `time_design_signoff` can be used with the `-report_only` option to reuse those parasitics and skip the Quantus call.

In case specific steps/options should be performed before/during Tempus ECO routing, this step can be performed manually after running `opt_signoff` with the `-no_eco_route` option.

When specific signoff STA commands/options/attributes should be applied, you can set these in a file and pass it to Tempus through `set_db opt_signoff_pre_sta_tcl FILE` parameter.

The syntax below is the basic template flow when user runs parasitic extraction and ecoRoute manually:

```
set_multi_cpu_usage -local_cpu 16 -remote_host 4 -cpu_per_remote_host 4
set_db extract_rc_effort_level high
extract_parasitics
time_design_signoff -report_only -report_dir RPT_initial
time_design_signoff  -hold   -no_eco_route ecoRoute
route_eco
extract_parasitics
time_design_signoff -report_only -report_dir RPT_final
```

Related Topics

- Signoff ECO Overview
- Key Features of Signoff ECO
- Signoff ECO Flow
- Setting Up Signoff ECO Environment
- Basic Signoff ECO Optimization Techniques

8.6 Basic Signoff ECO Optimization Techniques

Tempus ECO is able to optimize timing/DRV/Leakage/Area/Hold/Power/Setup. The main command to perform those different optimizations is `opt_signoff` and you can select the targeted optimization using the following options:

```
opt_signoff [-area] [-drv] [-dynamic] [-hold] [-leakage] [-power] [-setup]
```

This command will generate the `eco_innovus.tcl` file, which should be used in Innovus to perform place and route. It also generates the `eco_tempus.tcl` file, which can be sourced in Tempus to perform timing analysis after ECOs. Each optimization engine always ensures no new violations. For example, when fixing Setup timing, the tool ensures no new DRV/Hold violations, or when fixing Hold timing, the tool ensures no new DRV/Setup violations. This is the key to ensure good timing convergence in a minimum of ECO loops.

Notes before running `opt_signoff`:

- It is mandatory to provide parasitic before running `opt_signoff`. The syntax below shows how to load the SPEF files for every active RC corner:

  ```
  read_spef -rc_corner rc_max max.spef.gz
  read_spef -rc_corner rc_min1 min1.spef.gz
  read_spef -rc_corner rc_min2 min2.spef.gz
  ```

  To perform SI analysis/fixing, SPEF must contain the coupling capacitance data.

- All timing derates, AOCV tables, delay calculation settings, CTE attributes must also be provided before running the `opt_signoff` since those will the timing updates done during ECO fixing.

- It is important to correctly set up the timing environment and multi-CPU settings before running `opt_signoff` to avail the benefit of DMMMC timing analysis and Multi-threading during timing/leakage/area optimization.

- In case there are filler cells in the design, `opt_signoff` will automatically delete them at the start. To implement this, it is mandatory to specify the filler cells that need to be removed by using the command, `set_db add_fillers_cells {{list-of-cells1} {list-of-cells2} …}`.

By default, `opt_signoff` will use buffering/resizing/swapping techniques to improve timing/DRC/Leakage/Area. For Setup timing closure, `opt_signoff` is also able to insert pairs of inverters. Additionally, sequential elements can also be sized by enabling `set_db opt_signoff_optimize_sequential_cells true` and buffer deletion can be enabled during area recovery by enabling `set_db opt_signoff_delete_inst true`.

Examples

- Example 1:

```
opt_signoff -hold
```

This will perform Hold fixing by using Vth swapping, buffering, and resizing techniques on the combinational cells.

- Example 2:

```
set_db opt_signoff_optimize_sequential_cells true
```

```
opt_signoff -leakage
```

This will perform Leakage recovery by using Vth swapping technique on both the combinational and sequential cells.

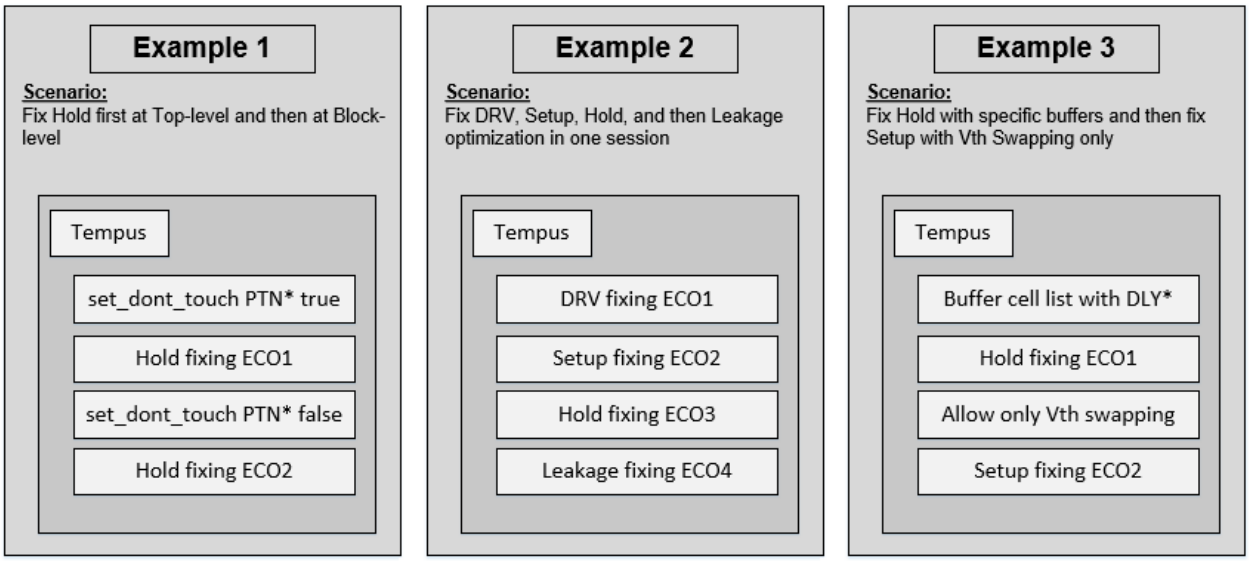
- Example 3:

```
set_db opt_signoff_delete_inst true
opt_signoff -area
```

This will perform Area recovery by using downsizing on both the combination and sequential cells in addition to buffer removal.

At the signoff stage, you can optimize all remaining violations, such as DRV, Setup, and Hold, but also to recover Power or Area when needed. This can be achieved in one single Tempus session using the incremental optimization feature enabled with `set_db opt_signoff_allow_multiple_incremental true`.

Figure 8-6 Examples



Notes about incremental optimization are as follows:

The recommended flow is the one described in Example 2 above. In case, area optimization is needed, it should be called as the first flow step.

Note that if more Tempus ECOs are done in a session then more routing changes will be needed when implementing those ECOs and therefore larger is the risk to get timing degradation.

Each optimization step will output an ECO file so in order to avoid overwriting the previous one, you should apply a unique prefix for each with the `set_db allow_multiple_incremental` *`prefixName`* option.

Example

```
set_db opt_signoff_read_eco_opt_db ECOdb
set_db opt_signoff_allow_multiple_incremental true
set_db opt_signoff_eco_file_prefix DRV_FIX
opt_signoff -drv
set_db opt_signoff_eco_file_prefix HOLD_FIX
opt_signoff -hold
```

This will perform DRV fixing followed by Hold fixing, while giving a different prefix for the ECO files.

Related Topics

- Signoff ECO Overview
- Key Features of Signoff ECO
- Signoff ECO Flow
- Setting Up Signoff ECO Environment
- Signoff Optimization Use Models

8.7 Fixing SI Violations in Tempus Signoff ECO

Tempus ECO is able to fix different violations related to SI glitch, SI slew, and SI crosstalk delta delay.

SI Glitch Violations

A signal integrity noise glitch, also called as voltage bump, generated by crosstalk coupling can propagate and amplify while traveling along a path. As a result, this glitch can cause incorrect signal state change. Tempus provides the ability to fix SI glitch violations in Tempus ECO. This is done using the `opt_signoff_fix_glitch` attribute of the `set_db` command.

Syntax:

```
set_db opt_signoff_fix_glitch true|false
```

When set to `true`, the software performs SI glitch fixing during DRV fixing using resizing and buffering techniques.

The default value is `false`.

Note:

- This parameter must also be set during ECO DB generation.
- The glitch fixing is done before max_tran/max_cap fixing.
- It is mandatory to set `set_db si_glitch_enable_report true` and use the `report_noise` command to analyse and get a signoff glitch report.
- In case remaining SI glitch could not be fixed using resizing and buffering techniques, it is recommended to select those nets and re-route them with extra SI property.

Example

The below example shows fixing Si glitch violations only:

```
set_db opt_signoff_fix_max_cap false
set_db opt_signoff_fix_max_transition false
set_db opt_signoff_fix_glitch true
```

The below example shows fixing of SI glitch violations using DRV fixing in addition to regular max_tran/max_cap violations:

```
set_db opt_signoff_fix_glitch true
opt_signoff -drv
```

SI Slew Violations

Crosstalk effects caused by parasitic capacitance between adjacent nets can lead to a larger signal transition (SI slews) on the victim nets. Tempus provides the ability to consider SI slews apart from base slews while performing transition violation fixing during DRV optimizations. All other optimizers that include setup, hold, leakage, area, power, and dynamic do not consider SI slews. You must run this as the first step of optimization before proceeding for any other incremental optimization.

This is done using the `opt_signoff_fix_si_slew` attribute of the `set_db` command.

Syntax

```
set_db opt_signoff_fix_si_slew true|false
```

When this parameter is set to `true`, the tool checks max_tran rule against SI slew and such violations will be resolved during DRV fixing using resizing and buffering techniques.

The default value is `false`.

Note: This parameter must also be set during ECO DB generation in addition to `set_db si_enable_drv_with_delta_slew true`.

To analyse and report SI slew violations:

```
set_db timing_use_incremental_si_transition true
set_db si_enable_drv_with_delta_slew true
report_constraint -drv_violation_type max_transition -all_violators
```

Examples:

The below example shows fixing SI slews violations only:

```
set_db opt_signoff_fix_max_cap false
set_db opt_signoff_fix_si_slew true
```

To analyze and report SI slew violations:

```
set_db timing_use_incremental_si_transition true
set_db si_enable_drv_with_delta_slew true
report_constraint -drv_violation_type max_transition -all_violators
```

SI Crosstalk Delta Delay Violations

In Tempus when attackers are transitioning opposite the victim, it causes the arrival time to increase (positive delta delay). And, when attackers are transitioning at the same time as the victim, it causes the arrival time to reduce (negative delta delay).

Tempus provides the ability to fix crosstalk delta delay in Tempus ECO. This helps in improving the design robustness, SI-delay fixing convergence, and minimizing the Setup versus Hold timing conflicts on critical paths.

This is done using the `opt_signoff_fix_xtalk` parameter of the `set_db` command.

Syntax:

`set_db opt_signoff_fix_xtalk true|false`

When set to `true`, the software reduces the crosstalk on the net that violates the thresholds set by you.

The default value is `false`.

Note:

- This parameter must also be set during ECO DB generation.
- xtalk delta delay fixing is done after max_tran/max_cap fixing as a separate phase.

- It is mandatory to set the below SI mode options:
  `set_db si_delay_separate_on_data true si_delay_delta_annotation_mode lumpedOnNet`

  In addition, the following parameters of the `set_db` command are used to select nets for xtalk fixing during optimization:

- To fix the nets with the crosstalk delta delay value equal to or greater than a specified threshold value:
  - For setup views: `set_db opt_signoff_setup_xtalk_delta_threshold <value in ns> (default 0.3 ns)`
  - For hold views: `set_db opt_signoff_hold_xtalk_delta_threshold <value in ns> (default 0.3 ns)`
- To fix the nets with the slack threshold value equal to or less than a specified threshold value:

For setup views:

```
set_db opt_signoff_setup_xtalk_slack_threshold <value in ns> (default 1000 ns)
```

For hold views:

```
set_db opt_signoff_hold_xtalk_slack_threshold <value in ns> (default 1000 ns)
```

Example

The following example uses DRV fixing to reduce the crosstalk delay on all nets that violated the 300ps threshold rule in setup views:

```
set_db opt_signoff_setup_xtalk_slack_threshold 0.3
set_db opt_signoff_fix_xtalk true

opt_signoff -drv
```

Related Topics

- Fixing IR Drop in Tempus Signoff ECO

8.8 Fixing IR Drop in Tempus Signoff ECO

Tempus ECO provides the ability to fix IR drop voltages for all instances in the design. The IR drop files that are generated through rail analysis in Voltus include victim and attacker data within hotspots and IR drop voltages. The IR drop hotspot information includes victim

instance names, voltage drop, and the current drawn by each of the instances. These IR drop files are then read into the Tempus-specified directory.

The software picks the worst victim and its attackers from ECO-DB. If the victim is violating then the attackers with positive slack are identified and downsized to avoid any new timing violations. Else, non-violating victims are ignored.

To fix IR drop in Tempus ECO, use the `opt_signoff_fix_ir_drop` and `opt_signoff_read_irdrop_db` parameters of the `set_db` command. When enabling the `opt_signoff_fix_ir_drop` parameter along with providing an IR Drop DB using the `opt_signoff_read_irdrop_db` parameter, `opt_signoff -drv` performs an IR drop fixing by downsizing instances without creating any Setup/Hold/DRV violations.

For more information, refer to the "Timing-aware IR Drop Fixing" section in the *Voltus User Guide*.

Related Topics

- Fixing SI Violations in Tempus Signoff ECO

8.9 Path-Based Analysis (PBA) Mode Optimization

At implementation stage, tools are using the Graph Based Analysis (GBA) mode because that allows fast turnaround time to analyze timing and to update timing incrementally. The drawback is that the GBA mode is generating pessimistic timing. At the signoff stage, where perfect accuracy is needed, you can enable the PBA mode and perform final pass of timing closure. In addition, it is recommended to run the area or power recovery engine to reach the best possible Power Performance Area (PPA) metrics. When enabling PBA, each timing path is timed in its own context so that the slews or AOCV factors are computed based only on the path being timed.

The following five options are specific to the PBA mode for Tempus ECO:

To enable PBA and select the retiming mode:

```
set_db opt_signoff_retime #default "path_slew_propagation"
```

To select whether PBA is applied to Setup or Hold or both:

```
opt_signoff_check_type #default "both"
```

To specify the maximum slack to be considered for retiming:

```
opt_signoff_max_slack #default 0
```

To specify the maximum number of paths to be retimed in total:

```
opt_signoff_max_paths #default -1
```

To specify the number of paths to be retimed for each endpoint:

```
opt_signoff_nworst #default -1
```

The following is an example of SOCV PBA based Leakage optimization:

```
read_db -physical_data postroute.enc.dat topChip
<set all signoff STA views>
<apply signoff STA settings/options/globals>
<load parasitic>
set_multi_cpu_usage -local_cpu 16
set_db opt_signoff_retime path_slew_propagation
set_db opt_signoff_check_type both
set_db opt_signoff_max_slack 10
set_db opt_signoff_max_paths 5000000
set_db  opt_signoff_nworst 50
set_db opt_signoff_pba_effort high
set_db opt_signoff_write_eco_opt_db ECO-DB-PBA
write_eco_opt_db
set_db opt_signoff_read_eco_opt_db ECO-DB-PBA
opt_signoff -leakage
```

Note: In above example, the `-max_slack` option is set to 10 (DB timing unit) so that positive GBA slack paths are also getting retimed in order to get even more positive setup slack on those paths. That extra positive slack can be then used to perform even more power recovery.

Related Topics

- Total Power Optimization
- Setup Timing Recovery After Large Leakage or Total Power Optimization
- Recommendations for Getting Best Total Power Optimization
- Hierarchical ECO Optimization

8.10 Total Power Optimization

ECO performs total power optimization to reclaim total power in the design. This is done using the `-power` parameter of the `opt_signoff` command. Using this parameter, the tool replaces the need to run the leakage and dynamic power optimizations in consecutive steps. During Total Power optimization, the engine will concurrently minimize leakage, switching, and internal power in order to achieve the lowest overall power consumption.

During total power optimization, `report_power` needs to be run before `opt_signoff`. The netlist activity can either be computed using default toggling rate on the inputs or input from an external VCD/SAIF file. In addition, the software performs swapping and sizing on both the combinational and sequential cells, and also does buffer removal when enabled.

Example:

```
report_power
set_db opt_signoff_delete_inst true
set_db opt_signoff_optimize_sequential_cells true
opt_signoff -power
```

In order to allow sequential elements to be resized during power optimization, ensure that there is no Fixed attribute applied on them. If you measure Setup timing degradation after doing a Power optimization, see section *Setup Timing Recovery After a Large Leakage or Power Optimization*.

Related Topics

- Path-Based Analysis (PBA) Mode Optimization
- Setup Timing Recovery After Large Leakage or Total Power Optimization
- Recommendations for Getting Best Total Power Optimization
- Hierarchical ECO Optimization

8.11 Setup Timing Recovery After Large Leakage or Total Power Optimization

It is quite common to get large number of ECOs generated when doing Power optimization. This means that 30 to 80 percent of the netlist has been changed. Although the tool is doing an accurate timing estimation for each of the ECOs generated; however, it will not be able to guarantee zero impact on Setup timing. In order to recover the Setup timing up to an initial value, a few ECOs might be needed having low or no power cost. This setup timing recovery can be achieved by running a new signoff timing analysis flow and then calling Setup optimization as mentioned below:

```
 set_db opt_signoff_setup_recovery true
opt_signoff -setup
```

The recovery will be done with Vth swapping only, which means that same size and same pin geometry cell sizing. Because of this there is no need to re-route the design and final timing can be generated.

Related Topics

- Path-Based Analysis (PBA) Mode Optimization
- Total Power Optimization

- Recommendations for Getting Best Total Power Optimization
- Hierarchical ECO Optimization

8.12 Recommendations for Getting Best Total Power Optimization

For Total Power optimization, here are the main recommendations to achieve the best QOR:

- Timing analysis must be done in PBA mode.
- Re-timing in PBA mode must be done on positive GBA slack paths.
- The `set_db opt_signoff_pba_effort` parameter must be set to `high`.
- Ensure that the list of usable cells is as large/rich as possible, and remove any Fixed placement attribute on the sequential elements to allow their resizing.
- Verify that all the sequential elements are not Fixed, otherwise they will not be the candidates for resizing.
- When doing aggressive Power optimization, a large amount of netlist changes is generated and the timing histogram attains a huge peak at around `0ns` slack.

  In this context, the software cannot completely avoid Setup timing degradation and a light Setup recovery based on the Vth cell swapping will help to get back to the initial timing. The solution is to perform a setup timing recovery optimization after generating a fresh signoff timing post-power optimization.

Template Script for Total Power Optimization in Tempus Cockpit

```
read_db physical_data postroute.enc.dat top
<set all signoff STA views>
<apply signoff STA settings/options/globals >
<load parasitics>
set_multi_cpu_usage -local_cpu 16
set_db opt_signoff_allow_multiple_incremental true
set_db opt_signoff_retime path_slew_propagation \
opt_signoff_check_type both \ opt_signoff_pba_effort high \
opt_signoff_max_slack 10 \ opt_signoff_max_paths 10000000 \
opt_signoff_nworst 50 \  opt_signoff_delete_inst true\
opt_signoff_write_eco_opt_db ECO-DB-PBA
write_eco_opt_db
report_power
set_db opt_signoff_read_eco_opt_db ECO-DB-PBA
opt_signoff -power
set_db opt_signoff_max_slack 0 opt_signoff_max_paths 10000000 opt_signoff_nworst
50 \ opt_signoff_write_eco_opt_db ECO-DB-PBA2
write_eco_opt_db
set_db opt_signoff_read_eco_opt_db ECO-DB-PBA2
```

```
opt_signoff_setup_recovery true
opt_signoff -setup
update_timing -full
```

Related Topics

- Path-Based Analysis (PBA) Mode Optimization
- Total Power Optimization
- Setup Timing Recovery After Large Leakage or Total Power Optimization
- Hierarchical ECO Optimization

8.13 Hierarchical ECO Optimization

Very large netlists are usually not implemented flat since the memory usage and runtime would be too large to be productive. In such a case, you apply a hierarchical flow methodology, either bottom-up or top-down, where the top-level netlist and the block-level netlist are implemented in parallel. Once those independent netlists are implemented, you have to assemble the design to perform signoff timing analysis in full flat mode, meaning that the entire design is timed. Most of the time this full flat signoff timing analysis will reveal timing violations on the timing path between top-level and partitions, or even only between partitions. This can be explained by the fact that the input/ouput timing constraints at the block level never models the required time from the top level perfectly, but also because at the top level, each partition is represented by a timing model, which is also never fully accurate. In order to fix those timing violations, the solution is to run the Tempus ECO feature to perform hierarchical chip finishing.

On a hierarchical design, you have the flexibility to optimize timing only at the top level, or on the top-level and block-level netlists. You can apply the `dont_touch` attribute on the modules that should not be touched by ECO fixing. The output of the `opt_signoff` command is an ECO file for the top level and for each identified partition, so that you can go back to implement those ECOs in each of the respective netlists.

On a hierarchical design, the buffering happens in logical and physical hierarchy-aware mode. This also implies that the partitions' interface remain unchanged.

In order to perform hierarchical chip finishing in Tempus, there are two main phases that need to be correctly set up:

- Assembling the design
  First, the top-level and all block-level verilog must be loaded in Tempus. Then, the `read_hierarchical_def` command will merge the top-level DEF and all the block-

level DEF files. Next, the `read_spef` command loads the top-level SPEF and all block-level SPEFs for every corner.

- Specifying which modules should be treated as partitions
  In order for the tool to know which modules should be treated as partitions, you have to provide the list of those module cells through an external file. This is done using `set_db opt_signoff_partition_list_file` *`file`*.

Sample Script for Hierarchical Flow

```
read_libs $liberty
read_mmmc filename.tcl
read_netlist "my_top.v pnt1.v pnt2.v"
init_design
source timing_settings_and_derating.tcl
read_hierarchical_def "my_top.def pnt1.def pnt2.def"
read_spef -rc_corner rc_max "my_top_max.spef ptn1_max.spef ptn2_max.spef"
read_spef -rc_corner rc_min1 "my_top_min1.spef ptn1_min1.spef ptn2_min1.spef"
read_spef -rc_corner rc_min2 "my_top_min2.spef ptn1_min2.spef ptn2_min2.spef"
set_db opt_signoff_buffer_cell_list "BUFX1 BUFX2 BUFX8 DLYX1 DLYX4"
set_db opt_signoff_read_eco_opt_db myECODB
set_db opt_signoff_partition_list_file partition.txt
opt_signoff -hold
```

Note: You must provide the pointer to the ECO Timing database using `set_db opt_signoff_read_eco_opt_db` since `opt_signoff` cannot do it in batch mode for hierarchical designs.

Optimization of Master/Clone Designs

The intent of the Master/Clone design methodology is to build common blocks once and instantiate them multiple times. It is also called a hierarchical implementation with Master/Clones. This methodology allows the duplicated functionality within a chip to be built and verified once, and then instantiated multiple times, thus reducing work load and data storage. Keeping a non-uniquified netlist makes the scope and breadth of late logic changes easier. The logic changes would then be limited to a subset of the chip, reducing the critical path turnaround time and amount of re-verification required to achieve final signoff.

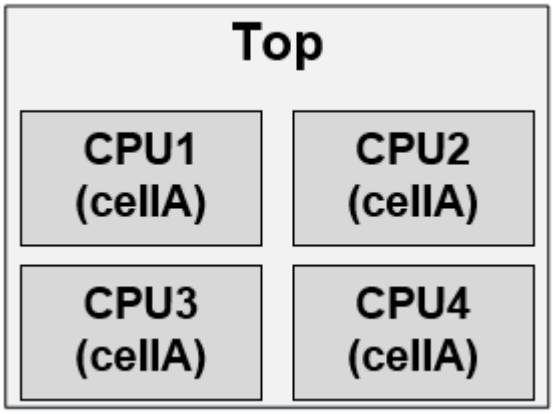
A cloned block is implemented so that it can operate in the worst-case corner scenario in any of its instantiation but once assembled in its top level, the full flat STA might reveal new timing violations. Master/Clone timing optimization is needed because each clone can have:

- Different physical environments (such as different IO loads)
- Different timing environments (such as change in clock slew/skew/OCV between blocks and flat)

- Different signal integrity environments

The Master/Clone timing optimization capability in Signoff ECO is able to fix timing violations and optimize timing in the replicated modules while earlier those would have been treated as `dont_touch`. Any netlist change will be checked against each clone's timing and physical context to ensure that no violations are created. Some of the benefits of Master/Clone timing optimization are:

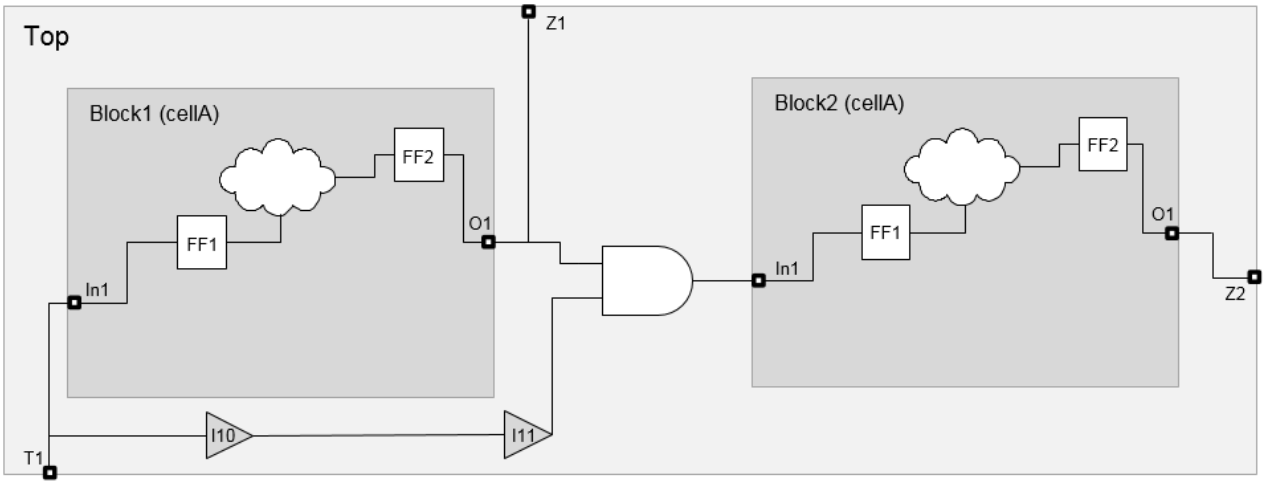
- Top and CPUs are optimized



- One ECO file for all CPUs
- Generated ECOs are valid for each CPU's timing context
- Optimal timing closure for full flat timing

The use model for running ECO timing closure on a hierarchical design with unique or replicated instances is the same.

Figure 8-7 : Illustration of Master Clone Implementation



- Physical/timing context is different for every clone boundary
- Timing optimization works on any kind of path:
  - from FF1 to FF2 on all clones or inter-clones paths too (if any)
  - from Top/T1 to Block1/FF1 and from Top/T1 to Block2/FF2
  - from Block/FF2 to Top/Z1 and from Block/FF2 to Top/Z2
  - Paths crossing multiple clones such as Block1/FF2 to Block2/FF1
- Buffering can happen on any of the nets, even the nets crossing the clone boundary

The Master/Clone use model for a design with 2 clones block 1 and block 2 is given below:

```
read_libs $liberty
read_physical -lefs "$techno_lef $all_lef"
read_netlist "my_top.v cellA.v"
init_design
read_hierarchical_def "my_top.def cellA.def"
```

Instead of the above commands, you can specify the command:

```
read_db physical_data top.enc.dat my_top
source viewDefinition_and_derating.tcl
read_spef -rc_corner rc_max “my_top_max.spef cellA_max.spef”
read_spef -rc_corner rc_min1 “my_top_min1.spef cellA_min1.spef”
read_spef -rc_corner rc_min2 “my_top_min2.spef cellA_min2.spef”
set_db  opt_signoff_read_eco_opt_db myECODB
set_db opt_signoff_partition_list_file partition.txt
Content of partition.txt is:
```

```
cellA
set_db opt_signoff_optimize_replicated_modules true
opt_signoff -hold
```

To run this flow, you must do the following:

- Specify the `opt_signoff_optimize_replicated_modules true` attribute of the `set_db` command to optimize timing in the replicated modules. This also enables leakage optimization in the Master/Clone mode. When this command is specified, a single ECO file is generated for each set of clone.
- Specify an ECO Timing database using `set_db opt_signoff_read_eco_opt_db` since `opt_signoff` cannot generate it automatically when the design is hierarchical.
- When replicated instances have different orientations, the design must be assembled in Tempus because loading one full flat DEF/Verilog will not work. Or you can also load an assembled DB from Innovus.

Related Topics

- Path-Based Analysis (PBA) Mode Optimization
- Total Power Optimization
- Setup Timing Recovery After Large Leakage or Total Power Optimization
- Recommendations for Getting Best Total Power Optimization

8.14 High Capacity ECO Flow for Timing and Power Closure

The *High Capacity ECO* flow is used to perform timing and power closure on large netlists. This flow allows huge reduction of the database being used during ECO phase, and therefore reduces the memory peak and runtime significantly.

Based on the user-defined settings, the database is reduced using any of the following methods:

- Reduction based on the timing violations seen during STA (Setup/Hold/DRV).
- Reduction based on list of partitions to be optimized for inter or intra timing paths.
- Reduction based on the user-defined path groups and list of DRV violated nets or pins.

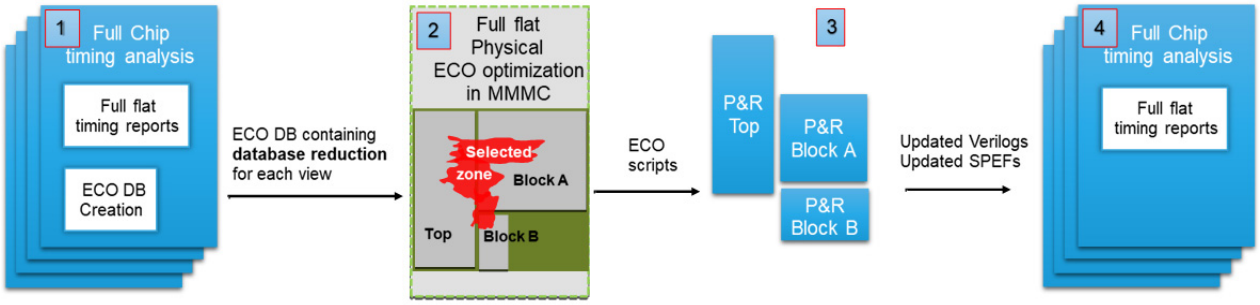
To use this flow, Tempus ECO follows two approaches:

- Run STA in independent Tempus sessions (SMSC or CMMMC or DMMMC) in order to collect the ECO timing DB. And, then run an ECO session (MMMC) with Physical data to perform timing/power closure. This methodology is known as the *Two Sessions* flow. In this flow, the settings must be provided during the STA sessions before performing ECO DB generation in order to decide on what criteria the database should be reduced.

  Note: This methodology also supports D-STA for ECO timing DB generation.

The following figure describes the different flow steps required for the user to complete initial STA for ECO DB generation, ECO physical-aware timing/power closure, ECO implementation at block and top only levels, and final STA with implemented ECO.

Figure 8-8 Steps for Completing STA



Step 1: Run full flat timing analysis and ECO DB generation for each view.

Step 2: Physical aware timing closure in MMMC mode on reduced violating zone database.

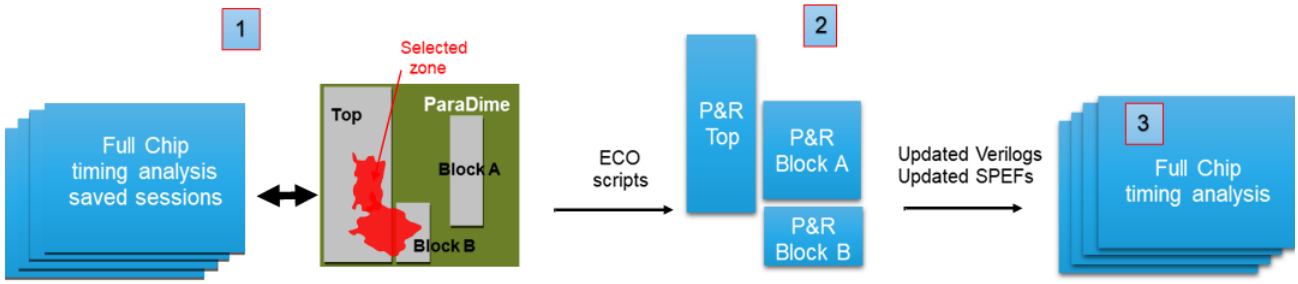
Step 3: Implement ECOs independently at top-level and block-level in Placement and Route (P&R) tool.

Step 4: Run full flat final timing analysis.

- Run Tempus Paradime to start Tempus clients (one per view) and each client collects data so that the master session can run ECO fixing in MMMC mode with Physical data in order to perform timing/power closure. This methodology is known as the *Paradime* flow. In this flow, the settings must be provided before the distributed design loading on clients in order to decide on what criteria the database should be reduced.

The following figure describes the different flow steps required for the user to run Tempus Paradime for ECO physical aware timing/power closure, ECO implementation at block and top only levels, and final STA with implemented ECO.

Figure 8-9 Steps for Running Tempus Paradime



Step 1: Start Paradime with Physical data on saved sessions, and select the desired sub-part of the design to focus on (for example, top-only, or some partitions, or some path groups, or some endpoints..)

Step 2: Implement ECOs independently at top-level or block-level in P&R tool.

Step 3: Run full flat final timing analysis.

In order to use the flow set up, below are the templates for the *Two Sessions* and *Paradime* flows, which uses three different database reduction mode:

Template 1: The *Two Sessions* flow performs full chip Hold timing closure on a reduced database based on the timing violations seen during STA. You can choose to enable either GBA or PBA mode.

Session1: The STA session can be SMSC, MMMC, or DMMMC.

```
<load_design_logical_mode_with_SPEF>
set_db opt_signoff_enable_high_capacity_eco true
set_db opt_signoff_retime path_slew_propagation
set_db opt_signoff_max_path 5000000
set_db opt_signoff_nworst 50
set_db opt_signoff_max_slack 0
set_db opt_signoff_pba_effort high
set_db opt_signoff_save_eco_opt_db HC-ECO-DB
set_db power_dynamic_power_view setup_ss_1p170v
set_db opt_signoff_write_eco_opt_db HC-ECO-DB
write_eco_opt_db
```

Session2: The ECO DB directory must be provided in the script before the design is loaded.

```
set_multi_cpu_usage -local_cpu 32
set_db opt_signoff_read_eco_opt_db HC-ECO-DB
```

```
read_physical -lefs {../libs/lef/FreePDK45_lib_v1.0.lef\ ../libs/MACRO/LEF/
pllclk.lef \ ../libs/MACRO/LEF/ram_256X16A.lef \ ../libs/MACRO/LEF/
rom_512x16A.lef }
read_mmmc viewDefinition.tcl
read_netlist "top.v blockA.v blockB.v"
init_design
set_db add_fillers_cells_<options>
read_hierarchical_def top.def blockA.def blockB.def
<load SPEFs>
set_db opt_signoff_pba_effort high
set_db opt_signoff_partition_list_file partition.txt
set_db opt_signoff_optimize_replicated_modules true
opt_signoff -hold
```

Template 2: The *Two Sessions* flow to perform inter-partition Hold timing closure on a reduced database based on a user-defined partition during STA.

Session 1: The STA session can be SMSC, MMMC or DMMMC.

```
<load_design_logical_mode_with_SPEF>
set_db opt_signoff_enable_high_capacity_eco true
set_db opt_signoff_inter_partition {blockA blockB}
set_db opt_signoff_intra_partition {}
set_db opt_signoff_retime path_slew_propagation
set_db opt_signoff_max_path 5000000
set_db opt_signoff_nworst 50
set_db opt_signoff_max_slack 0
set_db opt_signoff_pba_effort high
set_db opt_signoff_save_eco_opt_db HC-ECO-DB
set_db power_dynamic_power_view setup_ss_1p170v
set_db opt_signoff_write_eco_opt_db HC-ECO-DB
write_eco_opt_db
```

Session 2: The ECO DB directory must be provided in the script before the design is loaded.

```
set_multi_cpu_usage -local_cpu 32
set_db opt_signoff_read_eco_opt_db HC-ECO-DB
read_physical -lefs {../libs/lef/FreePDK45_lib_v1.0.lef\                ../libs/
MACRO/LEF/pllclk.lef \          ../libs/MACRO/LEF/ram_256X16A.lef \
../libs/MACRO/LEF/rom_512x16A.lef }
read_mmmc viewDefinition.tcl
read_netlist "top.v blockA.v blockB.v"
init_design
set_db add_fillers_cells_<options>
read_hierarchical_def top.def blockA.def blockB.def
```

```
<load SPEFs>
set_db opt_signoff_inter_partition {blockA blockB}
set_db opt_signoff_intra_partition {}
set_db opt_signoff_pba_effort high
set_db opt_signoff_partition_list_file partition.txt
set_db opt_signoff_optimize_replicated_modules true
opt_signoff -hold
```

The same template can be used to perform inter-partition Power recovery.

Template 3: The *Two Sessions* flow performs DRV, Setup, Hold timing closure on a reduced database based on the user-defined Setup/Hold path_groups and a list of DRV pins during STA.

Session 1: The STA session can be SMSC, MMMC or DMMMC.

```
<load_design_logical_mode_with_SPEF>
<TCL script which creates mySetupPG and myHoldPG path_groups>
set_db opt_signoff_enable_high_capacity_eco true
set_db opt_signoff_include_setup_path_groups {mySetupPG}
set_db opt_signoff_include_hold_path_groups {myHoldPG}
set_db opt_signoff_select_drv_pin_file {myDRVPinList.txt}
set_db opt_signoff_retime path_slew_propagation
set_db opt_signoff_max_path 5000000
set_db opt_signoff_nworst 50
set_db opt_signoff_max_slack 0
set_db opt_signoff_pba_effort high
set_db opt_signoff_save_eco_opt_db HC-ECO-DB
set_db power_dynamic_power_view setup_ss_1p170v
set_db opt_signoff_write_eco_opt_db HC-ECO-DB
write_eco_opt_db
```

Session 2: The ECO DB directory must be provided in the script before the design is loaded.

```
set_multi_cpu_usage -localCpu 32
set_db opt_signoff_read_eco_opt_db HC-ECO-DB
read_physical -lefs {../libs/lef/FreePDK45_lib_v1.0.lef\ ../libs/MACRO/LEF/
pllclk.lef \ ../libs/MACRO/LEF/ram_256X16A.lef \ ../libs/MACRO/LEF/
rom_512x16A.lef }
read_mmmc viewDefinition.tcl
read_netlist "top.v blockA.v blockB.v"
init_design
set_db add_fillers_cells_<options>
read_hierarchical_def top.def blockA.def blockB.def
<load SPEFs>
```

```
set_db opt_signoff_include_setup_path_groups {mySetupPG}
set_db opt_signoff_include_hold_path_groups {myHoldPG}
set_db opt_signoff_select_drv_pin_file {myDRVPinList.txt}
set_db opt_signoff_pba_effort high
set_db opt_signoff_partition_list_file partition.txt
set_db opt_signoff_optimize_replicated_modules true
opt_signoff –drv; opt_signoff –setup; opt_signoff -hold
```

Template 4: The *Paradime* flow performs a full chip Hold timing closure on a reduced database based on timing violations from the STA clients. You can choose to enable either GBA or PBA mode.

The STA saved session can be SMSC or MMMC.

```
set_distributed_hosts -local -time_out 10000
set_multi_cpu_usage -local_cpu 24 -remote_host 4 -cpu_per_remote_host 4
set_distributed_mmmc_mode -eco true
set_distributed_mmmc_mode -eco_merge_mmmc
set_distributed_mmmc_mode -enable_high_capacity_eco
set_db eco_setup_view_list {setup_1v setup_2v setup_3v setup_4v}
set_db eco_hold_view_list {hold_1v hold_2v hold_3v hold_4v}
set_db eco_dynamic_view {setup_3v} ; set_db eco_leakage_view {setup_3v}
set_db eco_lef_file_list “tech.lef std.lef ram_256X16A.lef"
set_db eco_def_file_list “top.def blockA.def blockB.def "
set_distributed_mmmc_mode -set_filler_mode {-corePrefix FILL}
distribute_read_db -restore_design all -restore_dir PARADIME_SESSIONS_211/ -
out_dir reports_Paradime
set_db opt_signoff_retime path_slew_propagation
set_db opt_signoff_max_path 5000000
set_db opt_signoff_nworst 50
set_db opt_signoff_max_slack 0
set_db opt_signoff_pba_effort high
set_db opt_signoff_partition_list_file partition.txt
set_db opt_signoff_optimize_replicated_modules true
opt_signoff –hold
```

Template 5: The *Paradime* flow performs inter-partition Hold timing closure on a reduced database based on a user-defined partition seen on the STA clients.

The STA saved session can be SMSC or MMMC. You can use the same template to perform inter-partition Power recovery.

```
set_distributed_hosts -local -time_out 10000
```

```
set_multi_cpu_usage -local_cpu 24 -remote_host 4 -cpu_per_remote_host 4
set_distributed_mmmc_mode -eco true
set_distributed_mmmc_mode -eco_merge_mmmc
set_distributed_mmmc_mode -enable_high_capacity_eco
set_db opt_signoff_inter_partition {blockA blockB}
set_db opt_signoff_intra_partition {}
set_db eco_setup_view_list {setup_1v setup_2v setup_3v setup_4v}
set_db eco_hold_view_list {hold_1v hold_2v hold_3v hold_4v}
set_db eco_dynamic_view {setup_3v} ; set_db eco_leakage_view {setup_3v}
set_db eco_lef_file_list "tech.lef std.lef ram_256X16A.lef"
set_db eco_def_file_list "top.def blockA.def blockB.def "
set_distributed_mmmc_mode -set_filler_mode {-corePrefix FILL}
distribute_read_db -restore_design all -restore_dir PARADIME_SESSIONS_211/ -
out_dir reports_Paradime
set_db opt_signoff_retime path_slew_propagation
set_db opt_signoff_max_path 5000000
set_db opt_signoff_nworst 50
set_db opt_signoff_max_slack 0
set_db opt_signoff_pba_effort high
set_db opt_signoff_partition_list_file partition.txt
set_db opt_signoff_optimize_replicated_modules true
opt_signoff -hold
```

Template 6 : The *Paradime* flow performs DRV, Setup, Hold timing closure on a reduced database based on the user-defined Setup/Hold path_groups and a list of DRV pins.

The STA saved session can be SMSC or MMMC.

```
set_distributed_hosts -local -time_out 10000
set_multi_cpu_usage -local_cpu 24 -remote_host 4 -cpu_per_remote_host 4
set_distributed_mmmc_mode -eco true
set_distributed_mmmc_mode -eco_merge_mmmc
set_distributed_mmmc_mode -enable_high_capacity_eco
set_db opt_signoff_path_group_file_name {TCL script which creates mySetupPG and
myHoldPG path_groups}
set_db opt_signoff_include_setup_path_groups {mySetupPG}
set_db opt_signoff_include_hold_path_groups {myHoldPG}
set_db opt_signoff_select_drv_pin_file {myDRVPinList.txt}
set_db eco_setup_view_list {setup_1v setup_2v setup_3v setup_4v}
set_db eco_hold_view_list {hold_1v hold_2v hold_3v hold_4v}
set_db eco_dynamic_view {setup_3v} ; set_db eco_leakage_view {setup_3v}
set_db eco_lef_file_list "tech.lef std.lef ram_256X16A.lef"
```

```
set_db eco_def_file_list "top.def blockA.def blockB.def "
set_distributed_mmmc_mode -set_filler_mode {-corePrefix FILL}
distribute_read_db -restore_design all -restore_dir PARADIME_SESSIONS_211/ -
out_dir reports_Paradime
set_db opt_signoff_retime path_slew_propagation
set_db opt_signoff_max_path 5000000
set_db opt_signoff_nworst 50
set_db opt_signoff_max_slack 0
set_db opt_signoff_pba_effort high
set_db opt_signoff_partition_list_file partition.txt
set_db opt_signoff_optimize_replicated_modules true
opt_signoff –drv; opt_signoff –setup; opt_signoff -hold
```

Related Topics

- Signoff ECO Flow
- High Capacity ECO Flow for Timing and Power Closure
- Metal ECO Flow

8.15 Generating Node Locations in Parasitic Data

The parasitic data must contain information about the node location in order to use the `set_db opt_signoff_along_route_buffering true` setting.

You can generate the location of nodes in the parasitic data using the following modes:

- Using Innovus Engines
- Using Standalone Quantus
- Using Third-Party Tool

Note: Once you have the SPEF file containing the node locations, you can load them using the `-starN` option of the `read_spef` command.

Using Innovus Engines

Generating node locations is done by default using the Innovus RC/TQuantus/ IQuantus engines. The information is saved automatically inside the RCDB file.

Using Standalone Quantus

You can generate SPEF along with node locations using the following option in the Quantus command file:

```
output_db -type spef -subtype standard -disable_subnodes false
```

Note: Use the following command to pass a command file to Quantus when running it from the Innovus cockpit.

```
set_db extract_rc_qrc_cmd_file <file>
```

Using a Third-Party Tool

You can use a third-party tool to generate SPEF with node locations.

Example:

The syntax below shows how node locations are saved in a SPEF file. A SPEF file with the node locations has extra lines with the *N prefix in the CONN section.

```
*CONN
     *I *15488:z O *C 71.65 237.505 *L 0 *D MUX2_X1
     *I *15465:D I *C 73.035 236.08 *L 0.00316 *D SDFFR_X1
     *N *907:2 *C 72.675 236.95 //CDNZ 1
     *N *907:3 *C 72.675 236.95 //CDNZ 2
     *N *907:4 *C 72.675 235.76 //CDNZ 2
```

Related Topics

- Optimization Along Wire Topologies

8.16 Optimization Along Wire Topologies

By default, buffers are inserted only at the start or/and end points of a net. This limits the scope of timing optimization, especially for DRV and Setup optimizations. To improve the delay of a net, Tempus splits the net/pin load efficiently when buffers are added along the route. In addition, this change allows buffers to be added at the branching point in case the net has multiple fanouts. Once the buffer is inserted somewhere along the route, the tool precisely estimates the new parasitic for the current and newly created net by using the node location information from the parasitic database.

To enable this feature, there are two prerequisites:

Use `set_db opt_signoff_along_route_buffering true`.

To provide parasitic with node location (See section, Generating Node Locations in Parasitic Data on page 474)

In addition, when enabling this feature, the setup timing optimization will be able to insert pairs of inverter along the route, if that is seen as a better choice for timing closure compared to single buffer insertion.

Example:

```
set_db opt_signoff_along_route_buffering true
opt_signoff -drv
```

This will perform DRV fixing in a mode where buffering will also be able to insert buffers along the route topology.

Related Topics

- Generating Node Locations in Parasitic Data
- Optimization using Endpoint Control

8.17 Optimization using Endpoint Control

Tempus ECO supports the following ways for path group-based fixing:

1. Endpoint-based inclusion/exclusion per view
2. Endpoint-specific slack adjustment per view (both positive and negative adjustments)
3. Option to fix only register-to-register paths

Path group support for Tempus ECO can be used for hold or setup fixing.

Note: This feature is currently not supported for DRV or leakage optimization.

1. Endpoint-based inclusion/exclusion per view

You can specify endpoints that need to be included or excluded during the optimization for a view with the following `set db` attributes:

- `opt signoff select hold endpoints`
- `opt signoff select setup endpoints`

File format:

```
<View> include/exclude <Endpoints>
```

<View> can be specified as either viewName or V* (if the endpoints are to be included/ excluded for all views)

<Endpoints> can be specified in three ways:

- As space-separated list of endpoint names: for example, E1 E2 E3.
- Slack range in nanosecond: <minSlack> <maxSlack>, where all endpoints with the slack greater than or equal to minSlack and slack less than or equal to maxSlack will be included/excluded, for example, -2.0 -1.0.
- E*, if all endpoints for the view need to be included/excluded.

Note:

- When few pins are excluded from the view, those pins cannot be included back even after overriding the excluded pins, as shown in the below example:

  ```
  > V* exclude E*

  > V* include inst/FF/D
  ```

- Tempus default behavior refers to the inclusion of all pins for the view, as shown below:

  ```
  > V* include E*
  ```

- Wildcards cannot be used for including/excluding pins, as shown in the below example:

  ```
  > V* include inst/FF/*

  > V* include inst*/FF/D
  ```

- You can set exclude endpoints one by one without using any wild card.

Sample configuration files

The lines starting with # are comments and will be skipped.

```
File : hold_exclude_example
     # Exclude endpoints EXECUTE_INST/pc_acc_reg/SE and EXECUTE_INST/
     read_prog_reg/SE from view core+typ-rcMin for hold fixing
     core+typ-rcMin exclude EXECUTE_INST/pc_acc_reg/SE EXECUTE_INST/
     read_prog_reg/SE

File : hold_include_example_with_range
     # Include only the endpoints that have slacks >= -0.5 ns and <= 0.04
     ns for view core+best-rcTyp for hold fixing
     core+best-rcTyp include -0.5 0.04
```

2. Endpoint-specific slack adjustment per view

You can specify slack adjustment (margin) for selected endpoints during optimization for a view with the following set_db attributes:

- opt_signoff_set_hold_endpoints_margin
- opt_signoff_set_setup_endpoints_margin

File format:

```
<View> <Margin> <Endpoints>
```

<View> can be specified as either viewName or V* (if the endpoints are to be included/excluded for all views)

<Margin> is specified as a float value in nanosecond. Margin value is subtracted from the endpoint slack, so a positive margin means that the endpoint slack would be degraded by the margin amount.

<Endpoints> can be specified in three ways:

- As space-separated list of endpoint names: for example, E1 E2 E3.
- Slack range in nanosecond: <minSlack> <maxSlack>, where all endpoints with the slack greater than or equal to minSlack and slack less than or equal to maxSlack would be included/excluded: for example, -2.0 -1.0.
- E* if all endpoints for the view need to be included/excluded.

Sample configuration files

The lines starting with # are comments and would be skipped.

```
File : hold_margin_allViews_slackRange_example
# Apply margin of 0.1 nanosecond to all endpoints that have slacks between -0.07
to -0.03 for all hold views
V* 0.1 -0.07 -0.03
File : hold_margin_allEndPoints_example
# Apply margin of 0.2 nanosecond to all endpoints for hold view core+typ-rcMin
core+typ-rcMin 0.2 E*
File : hold_margin_selectedEndPoints_example
# Apply margin of 0.6 ns to endpoints TDSP_CORE_MACH_INST/phi_6_reg/SE and
TDSP_CORE_MACH_INST/phi_1_reg/SE for hold view core+best-rcTyp
core+typ-rcMin 0.6 TDSP_CORE_MACH_INST/phi_6_reg/SE TDSP_CORE_MACH_INST/
phi_1_reg/SE
```

3. Option to fix only register-to-register paths

You can choose to optimize timing/leakage/area only on register-to-register paths during opt_signoff with the options specified below. Other path group configurations will be ignored in this mode. The timing for non-register-to-register paths for fixing mode (hold or

setup) will be adjusted to 0, but the timing for other mode (setup during hold fixing/hold during setup fixing) is not affected.

```
set_db opt_signoff_optimize_core_only <true/false>
read_spef <all corner spef files>
update_timing -full
report* commands (optional)
```

In addition to timing closure, Tempus ECO is also a solution to reduce overall power consumption of the netlist. At Signoff stage, some amount of timing pessimism is removed due to Path Based Analysis, also called retiming, and that extra timing margin can be used to perform netlist change leading to lesser overall power. This is especially beneficial when AOCV or SOCV timing mode are enabled.

Related Topics

- Optimization Along Wire Topologies

8.18 Metal ECO Flow

The *Metal ECO* flow, also known as *Post-Mask* ECO flow, is supported via Gate Array (GA) filler cells and Programmable Filler Cells (PFC) in Tempus ECO. In both these flows, the timing closure engine performs netlist changes without touching the base layer mask.

Prerequisites for Using Metal ECO Flow:

The prerequisitie information for using the Metal ECO flow is as follows:

- All instances of the design must be placed (no unplaced instance allowed).
- The initial design placement density must be 100% that is no empty spaces left.
- The list of Gate Array filler cells must contain enough granularity to ensure that any empty space created during metal ECO can be filled back.
- Do not specify cells through `setTieHiLoMode` if Tie cell insertion is not allowed

Running Metal ECO Flow:

The following command is used to run the Metal ECO flow:

```
set_db opt_signoff_post_mask true
```

By default, `set_db opt_signoff_post_mask` is set to `false`.

The software performs the following netlist changes in the post-mask ECO mode:

- Inserts the GA/PFC buffers or an inverter pair in place of existing GA/PFC filler cells.
- Resizes regular cells to GA/PFC cells. It also resizes GA cells to GA cells/PFC cells to PFC cells.
- Deletes a regular cell.
- Fills up empty spaces by GA/PFC fillers to ensure 100% placement density.

Note:

- The Metal ECO flow is supported by DRV, Setup, and Hold fixing.
- When `set_db add_tieoffs_cells {{<tieLo_name> <tieHi_name>}}` setting is used, this flow will insert the TieLo cells to connect dangling input pins, in case the existing TieLo cells in the floorplan are extremely far or too overloaded.

Example - Use Model of PFC for Metal ECO

```
set_db opt_signoff_post_mask true
```

List all the ecoFillercells that are allowed to be removed/re-inserted:

```
set_db opt_signoff_post_mask_filler_cells  {ln10_ecofiller40x …
ln10_ecofiller4x}
```

List all the ecoDecapcells that are allowed to be removed:

```
set_db opt_signoff_post_mask_decap_cells {ln10_ecofillsgcap40x …
ln10_ecofillsgcap4x}
```

List all the eco regular PFC cells that are allowed to be inserted in place of filler/decap

```
set_db opt_signoff_post_mask_logic_cells
{ln10_ecodlybt01 ln10_ecobuft04 … ln10_ecoand02 … ln10_ecoinv04 …}
```

List all the Tie cells that can be inserted in place of filler/decap

```
 set_db add_tieoffs_cells{{ln108_ecotielo01 ln10_ecotiehi01}}
opt_signoff -hold
```

This will force the hold optimization engine to insert PFC regular cells in place of PFC fillers, and fill up the empty gaps by a smaller PFC filler.

Example - Use Model of GA Fillers for Metal ECO

```
set_dbopt_signoff_post_mask true
```

List all the ecoFillercells that are allowed to be removed/re-inserted:

```
set_db opt_signoff_post_mask_gate_array_filler_cells {GFILL1BWP GFILL2BWP
GFILL4BWP GFILL10BWP}
```

List all the Tie cells that can be inserted in place of filler/decap:

```
 set_db add_tieoffs_cells {{GATIELO GATIEHI}}
```

```
opt_signoff -hold
```

This will force the Hold timing optimization engine to insert buffers in place of GA fillers. The software automatically identifies the Gate Array cells by checking which library instances have the same SITE name as the Gate Array filler cells specified by the user. Here, empty spaces will be filled up by the Gate Array filler cells to ensure 100% placement density.

Related Topics

- Signoff ECO Flow
- High Capacity ECO Flow for Timing and Power Closure
- Generating Node Locations in Parasitic Data

8.19 Handling Large Number of Active Timing Views Using SmartMMMC

The *SmartMMMC* feature handles a large number of active timing views. This feature is used to enable reduced memory and runtime solution when many timing views are enabled during timing or power ECO fixing stage.

To enable the *SmartMMMC* feature in the two sessions flow, use the below attributes of the `set_db` command:

- `opt_signoff_create_smart_mmmc_db true|false`: Enables saving the additional compact timing information in the ECO DB directory during the STA session. When set to `true`, the parameter must be applied before the `write_eco_opt_db` command and has no effect on `opt_signoff`.

  Example:

  The following command saves the regular timing and additional compact timing information in the ECO DB directory named `EcoTimingDB` in the STA SMSC session.

  ```
  tempus> set_db opt_signoff_write_eco_opt_db
  EcoTimingDB
  ```

```
tempus> set_db opt_signoff_create_smart_mmmc_db true
tempus> write_eco_opt_db
```

- `opt_signoff_read_smart_mmmc_db <dirName>`: Points Tempus ECO to the ECO DB directory containing the additional compact timing information during the ECO fixing session. This parameter must be set before the design is loaded and must point to an ECO DB directory, which was previously generated through `write_eco_opt_db` using the setting, `set_db opt_signoff_read_smart_mmmc_db true`.

  Note: Before `opt_signoff`, you still need to point to the ECO DB directory using the setting, `set_db opt_signoff_read_eco_opt_db <dirName>`.

  Example:

  The following command loads the design with a minimized set of data to reduce the memory or runtime during an ECO fixing stage when many timing views are enabled, and performs Setup timing closure in the ECO MMMC session.

```
tempus > set_db opt_signoff_read_smart_mmmc_db
EcoTimingDB
tempus> <load_design>
tempus> set_db opt_signoff_read_eco_opt_db EcoTimingDB
tempus> opt_signoff -setup
```

To enable the *SmartMMMC* feature in the *Paradime* flow, use the below parameters of the `set_distributed_mmmc_mode` command:

- `set_distributed_mmmc_mode`:
  - `-enable_smart_mmmc`: when this is enabled, it must be applied before the design is loaded in Paradime and `set_distributed_mmmc_mode -eco true` should be set.

    Example:

    The following commands enable design loading in Paradime with the minimized set of data to reduce the memory or runtime during ECO fixing stage when many timing views are enabled, and performs Setup timing closure.

```
tempus > set_multi_cpu_usage -local_cpu 16 -remote_host 4 -
cpu_per_remote_host 4
tempus > set_distributed_mmmc_mode -eco true
tempus > set_distributed_mmmc_mode -enable_smart_mmmc
tempus > set_db opt_signoff_retime path_slew_propagation
tempus > set_distributed_mmmc_mode -merge_view_definitions
tempus > set eco_setup_view_list "<list_of_Setup active_views>"
tempus > set eco_setup_view_list "<list_of_Hold active_views>"
tempus > set eco_lef_file_list "<list_of_LEF_files>"
tempus > set eco_def_file_list "<list_of_DEF_files>"
```

```
tempus > distribute_read_db -read_db
all -read_dir PARADIME_SESSIONS/ -out_dir reports_Paradime
tempus > opt_signoff -setup
```

Related Topics

- High Capacity ECO Flow for Timing and Power Closure

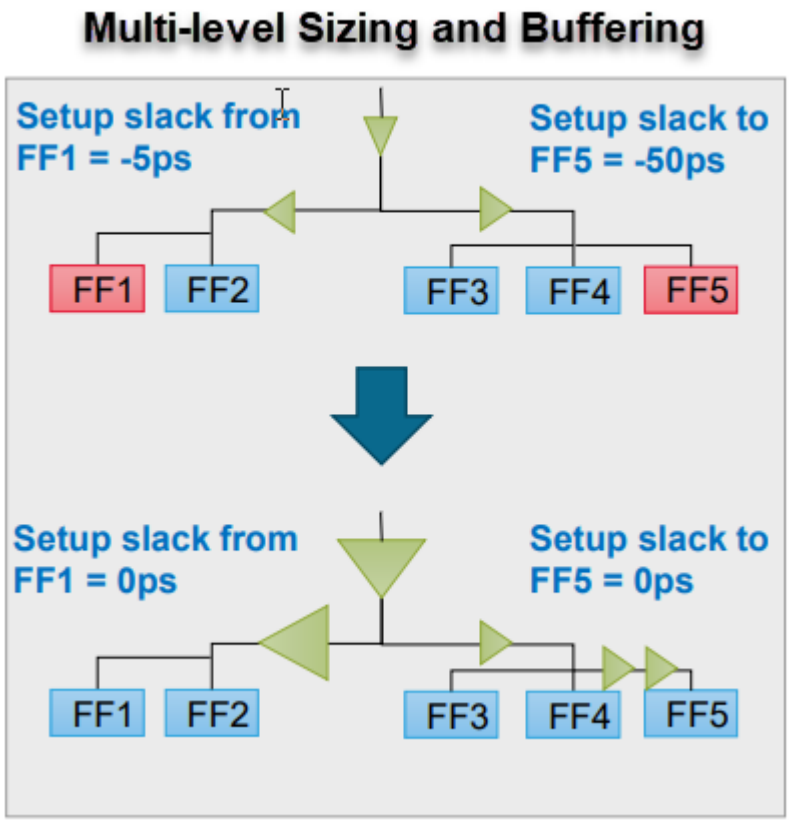
8.20 Performing Clock Skewing for Setup Timing Closure

Tempus ECO provides the ability to fix clock paths in order to fix the remaining setup timing violations. If the data path is fully optimized, the software uses useful skew, which includes two methods: early clock and late clock. This either reduces the launch clock delay or increases capture clock delay. The Tempus ECO engine can advance and delay clock arrival time to improve setup timing while not creating any new hold timing violations.

Below are the key features of a skew engine:

- Advanced clock tree manipulation to allow larger flexibility for clock skewing
- Sizing, deletion, and buffering (includes inverter pair and load cell) at any clock tree level
- Concurrent data path and clock path optimization for optimal timing closure
- Sophisticated push-pull weight and timing bottleneck-based engine

- Support for max level skewing to avoid excessive padding using `set_db opt_signoff_clock_max_level <value>`



Note: It is mandatory to provide a list of clock cells that are allowed to be used on the clock tree. Those cells can be inverter, buffer, or any other combinational cells used as clock gaters.

Example:

```
set_db opt_signoff_clock_cell_list {CKBUFX1 CKBUFX4}
set_db opt_signoff_allow_skewing true
opt_signoff -setup
```

The above commands perform clock skewing during setup fixing. Only `CKBUFX1` and `CKBUFX4` can be added to the clock tree.

Related Topics

- Signoff Optimization Use Models
- Hierarchical ECO Optimization

8.21 ECO Flow Use Model for Cell EM Analysis

The following Tempus ECO flows support Cell EM analysis:

- Standalone Tempus MMMC ECO Flow
- Innovus-integrated Tempus ECO Flow
- Paradime ECO Flow

Standalone Tempus MMMC ECO Flow

- To enable cell EM fixing in the standalone Tempus MMMC ECO flow, use the below command:

  ```
  set_db opt_signoff_fix_cell_em true
  ```

  Default: `false`

- To optionally fix the EM type, use the below command:

  ```
  set_db opt_signoff_em_type {avg | peak | rms | all}
  ```

  Default: `all`

- To enable cell EM fixing during DRV optimization, use the below command:

  ```
  opt_signoff -drv
  ```

  Note: The cell EM optimizer can optimize on clock nets. To fix DRVs on clock nets, use the `opt_signoff_fix_clock_drv true` command. If set during DRV fixing, the tool will fix only the cell EM violations. This implies that fixing of max_transition/ max_capacitance/SI glitch/SI Xtalk/IR drop will be disabled.

Innovus-integrated Tempus ECO Flow

- To enable cell EM fixing in the Innovus-integrated Tempus ECO flow, use the below command:

  ```
  set_db opt_signoff_fix_cell_em true
  ```

  Default: `false`

- To optionally fix the EM type, use the below command:

  ```
  set_db opt_signoff_em_type {all | avg | rms | peak}
  ```

  Default: `all`

- To enable cell EM fixing during DRV optimization, use the below command:

```
    opt_signoff -drv
Initial summary
+----------+----------------------------------+-----------------------------+
|          |             Signal nets          |      Clock nets             |
|  DRVs    +------------------+---------------+----------------+------------+
|          |  Nr nets(terms)  | Worst Vio     | Nr nets(terms) | Worst Vio  |
+------------------------------+---------------+----------------+------------+
| Cell EM  | 53 ( 53)         | -0.033        | 216 ( 216)     | -0.072     |
| max_tran | 0  ( 0)          |  0.000        |   0 (   0)     |  0.000     |
+----------+----------------------------------+----------------+------------+
Final Summary
+----------+----------------------------------+-----------------------------+
|          |             Signal nets          |      Clock nets             |
|  DRVs    +------------------+---------------+----------------+------------+
|          | Nr nets(terms)   | Worst Vio     | Nr nets(terms) | Worst Vio  |
+----------+------------------+---------------+----------------+------------+
|  Cell EM | 3 ( 3)           | -0.006        | 65 ( 65)       | -0.035     |
|  max_tran| 0 ( 0)           |  0.000        |  0 (  0)       |  0.000     |
+----------+------------------+---------------+----------------+------------|
```

The results of both Tempus and Innovus optimization are visible in a log file.

Paradime ECO Flow

The activity definition gets utilized from the dynamic power view using the following setting if set during the STA session:

```
set_analysis_view -setup [list <>] -hold [list <>] -leakage <> -dynamic <>
```

You can also define the following variables (before using the `distribute_read_db` command):

```
set_db eco_dynamic_view {<powerViewName>}
set_db eco_leakage_view {<powerViewName>}
```

Note:

- You should ensure that the `set_switching_activity` or `read_activity_file` command is used in the same way as for STA flow.
- Since the activity definition is not part of constraints, it needs to be defined before running the `opt_signoff` command to fix the cell EM violations.

Related Topics

- Cell EM Analysis and ECO

8.22 Hierarchical Top+Boundary Model ECO Flow

The *Hierarchical Top+Boundary Model ECO* flow can be used for performing timing and power closure over interface paths, while loading lower level models/blocks and closing a higher level module/block. This flow analyzes the design hierarchically using boundary constraints at the top level, thus allowing faster and efficient timing analysis. It also reduces memory usage and runtime significantly on larger designs.

When the *High Capacity ECO (HC-ECO)* flow with Top+BM is used, the `timing_enable_boundary_model_eco` command is applied before generating and loading the boundary model. This is done to ensure that boundary model can be used for ECO. For analysis part of boundary model, refer Boundary Models.

Based on the user-defined settings, the Top+BM database is optimized using any of the following methods:

- Reduction based on the timing violations seen during Top+BM STA session (Setup/Hold/DRV).
- Reduction based on the list of partitions to be optimized for inter_partition timing paths.
- Reduction based on the user-defined path groups and a list of DRV violated nets or pins.

To use this flow, Tempus ECO follows two approaches as mentioned below:

- Top+BM HC-ECO Using Two Sessions Flow
- Top+BM HC-ECO Using Paradime Environment

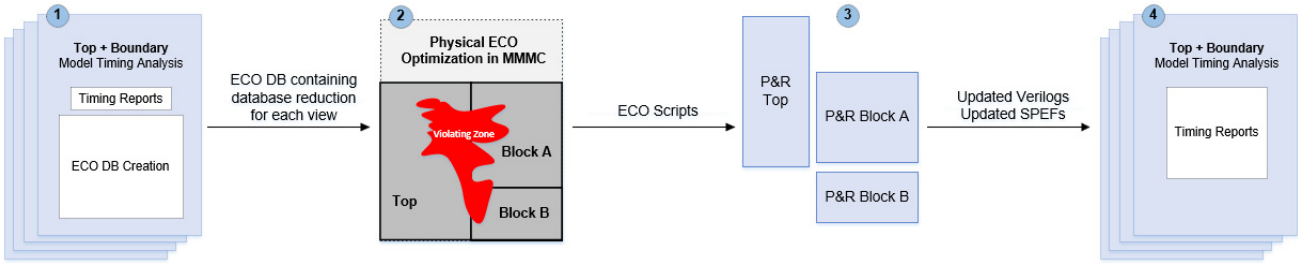
Top+BM HC-ECO Using Two Sessions Flow

Using this approach, the tool:

- Performs DRV+Setup+Hold timing closure on the violating zone.
- Performs Top+BM timing closure using High Capacity ECO:

The below figure describes the different flow steps required for a higher level module STA with lower levels models loaded for ECO DB generation, ECO physical-aware timing/power closure, ECO implementation at block and top only levels, and final STA with the implemented ECO.

Figure 8-10 Top+BM HC-ECO (Using Two Sessions Flow)



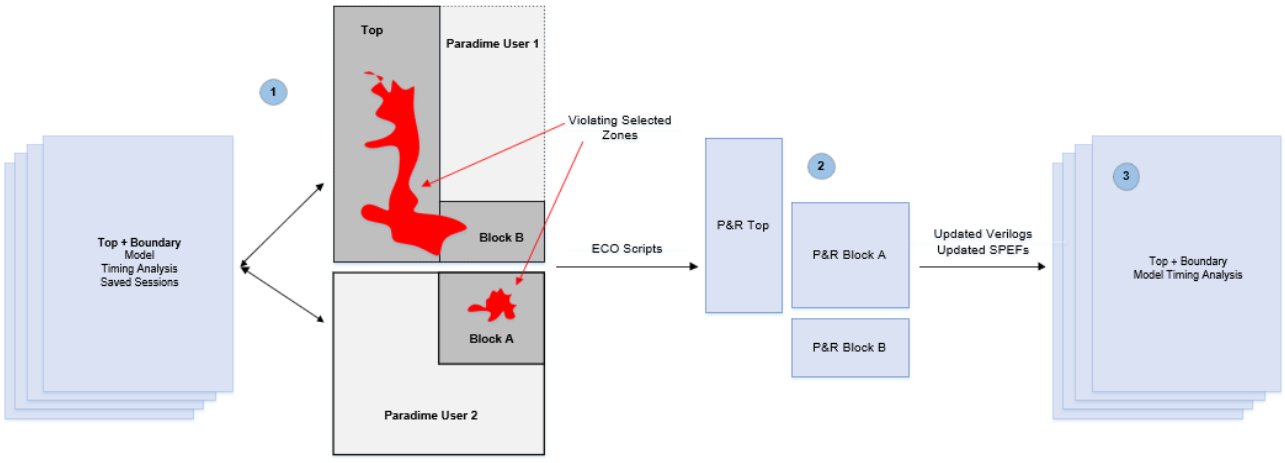
- **Step 1:** Run Top+BM timing analysis and ECO DB generation for each view.
- **Step 2:** Perform physical aware timing closure in MMMC mode on reduced violating zone database.
- **Step 3:** Implement ECOs independently in top level and block in P&R tool.
- **Step 4:** Run Top+BM final timing analysis.

Top+BM HC-ECO Using Paradime Environment

Using this approach, the tool:

- Performs DRV+Setup+Hold timing closure on violating zone.
- Performs Top+BM timing closure using High Capacity ECO inside Paradime.

Figure 8-11 Top+BM HC-ECO (Using Paradime)



- **Step 1:** Start Paradime with physical data on saved sessions, and choose the specific part of the design to focus on. This is either top only, or partitions, or path_groups, or endpoints, and so on.
- **Step 2:** Implement ECOs independently at top level and block level in P&R tool.
- **Step 3:** Run Top+BM final timing analysis.

In order to use the flow set up, below are the templates for the Two Sessions and Paradime flows:

Template 1:

The *Two Sessions* flow performs hold timing closure on a Top+BM database based on timing violations seen during STA. You can choose to enable either GBA or PBA mode.

BM Generation Session

The BM session can be SMSC, MMMC, or DMMMC:

```
set_db timing_full_context_flow 1
set_db timing_sum_cc_for_all_small_aggressors 1
set_db timing_write_aggressors_to_disk 1
set_db timing_enable_model_accurate_nef 1
set_db timing_enable_filter_fanin_cone 1
timing_enable_boundary_model_eco
...
create_boundary_model -overwrite -no_extended_fanin \ -latch_effort_low -
skip_constants -exclude_no_delay_ports \ -dir BM_${BlockName}_${view} -overwrite
```

Session 1:

The Top+BM session can be SMSC, MMMC, or DMMMC.

```
timing_enable_boundary_model_eco
<load_Top+BM_design>
set_db opt_signoff_enable_high_capacity_eco true
set_db opt_signoff_write_eco_opt_db HC-ECO-DB
set_db power_dynamic_power_view setup_ss_1p170v
set_db opt_signoff_write_eco_opt_db HC-ECO-DB
write_eco_opt_db
```

Session 2:

The ECO DB directory must be provided in the script before the design is loaded:

```
set_multi_cpu_usage -local_cpu 32
set_db opt_signoff_read_eco_opt_db HC-ECO-DB
read_physical -lef {../libs/lef/FreePDK45_lib_v1.0.lef\  ../libs/MACRO/LEF/
pllclk.lef \          ../libs/MACRO/LEF/ram_256X16A.lef \  ../libs/MACRO/LEF/
rom_512x16A.lef }
read_mmmc viewDefinition.tcl
read_netlist "top.v blockA.v blockB.v" -top top
init_design
```

```
set_db add_fillers_<options>
read_hierarchical_def top.def blockA.def blockB.def
<load SPEFs>
set_db opt_signoff_partition_list_file partition.txt
set_db opt_signoff_optimize_replicated_modules true
opt_signoff -hold
```

Template 2:

The *Two Sessions* flow performs the inter-partition Hold timing closure on a Top+BM database based on a user-defined partition during STA. The boundary model generation flow will remain the same as specified in *Template 1*.

Session 1:

The Top+BM session can be SMSC, MMMC, or DMMMC:

```
timing_enable_boundary_model_eco
<load_Top+BM_design>
set_db opt_signoff_enable_high_capacity_eco true
set_db opt_signoff_inter_partition {blockA blockB}
set_db opt_signoff_intra_partition {}
set_db opt_signoff_write_eco_opt_db HC-ECO-DB
set_db power_dynamic_power_view setup_ss_1p170v
set_db opt_signoff_write_eco_opt_db HC-ECO-DB
write_eco_opt_db
```

Session 2:

The ECO DB directory must be provided in the script before the design is loaded.

```
set_multi_cpu_usage -local_cpu 32
set_db opt_signoff_read_eco_opt_db HC-ECO-DB
read_physical -lef {../libs/lef/FreePDK45_lib_v1.0.lef\  ../libs/MACRO/LEF/
pllclk.lef \  ../libs/MACRO/LEF/ram_256X16A.lef \     ../libs/MACRO/LEF/
rom_512x16A.lef }
read_mmmc viewDefinition.tcl
set_db add_fillers_<options>
read_hierarchical_def top.def blockA.def blockB.def
<load SPEFs>
set_db opt_signoff_inter_partition {blockA blockB}
set_db opt_signoff_intra_partition {}
set_db opt_signoff_partition_list_file partition.txt
set_db opt_signoff_optimize_replicated_modules true
opt_signoff -hold
```

The same template can be used to perform inter-partition power recovery.

Template 3:

The *Two Sessions* flow performs DRV, Setup, Hold timing closure on a Top+BM database based on the user-defined Setup/Hold path_groups, and a list of DRV pins during STA. The boundary model generation flow will remain same as specified in *Template 1*.

Session 1:

The Top+BM session can be SMSC, MMMC, or DMMMC.

```
timing_enable_boundary_model_eco
<load_Top+BM_design>
<TCL script which creates mySetupPG and myHoldPG path_groups>
set_db opt_signoff_enable_high_capacity_eco true
set_db opt_signoff_include_setup_path_groups {mySetupPG}
set_db opt_signoff_include_hold_path_groups {myHoldPG}
set_db opt_signoff_select_drv_pin_file {myDRVPinList.txt}
set_db opt_signoff_write_eco_opt_db HC-ECO-DB
set_db power_dynamic_power_view setup_ss_1p170v
set_db opt_signoff_write_eco_opt_db HC-ECO-DB
write_eco_opt_db
```

Session 2:

The ECO DB directory must be provided in the script before the design is loaded.

```
set_multi_cpu_usage -local_cpu 32
set_db opt_signoff_read_eco_opt_db HC-ECO-DB
read_physical -lef {../libs/lef/FreePDK45_lib_v1.0.lef\  ../libs/MACRO/LEF/
pllclk.lef \  ../libs/MACRO/LEF/ram_256X16A.lef \     ../libs/MACRO/LEF/
rom_512x16A.lef }
read_mmmc viewDefinition.tcl
set_db add_fillers_<options>
read_hierarchical_def top.def blockA.def blockB.def
<load SPEFs>
set_db opt_signoff_include_setup_path_groups {mySetupPG}
set_db opt_signoff_include_hold_path_groups {myHoldPG}
set_db opt_signoff_select_drv_pin_file {myDRVPinList.txt}
set_db opt_signoff_partition_list_file partition.txt
set_db opt_signoff_optimize_replicated_modules true
opt_signoff -drv; opt_signoff -setup; opt_signoff -hold
```

Template 4:

The Paradime flow performs hold timing closure on a Top+BM database based on the timing violations from the Top+BM STA clients. You can choose to enable either GBA or PBA mode. The boundary model generation flow will remain the same as specified in *Template 1*.

The STA saved session can be SMSC or MMMC.

Top+BM STA Session:

```
timing_enable_boundary_model_eco
<load_Top+BM_design>
write_db view1.enc
```

Paradime Session:

```
set_distributed_hosts -local -time_out 10000
set_multi_cpu_usage -local_cpu 24 -remote_host 4 -cpu_per_remote_host 4
set_distributed_mmmc_mode -eco true
set_distributed_mmmc_mode -eco_merge_mmmc
set_distributed_mmmc_mode -enable_high_capacity_eco
```

```
set_db eco_setup_view_list {setup_1v setup_2v setup_3v setup_4v}
set_db eco_hold_view_list {hold_1v hold_2v hold_3v hold_4v}
set_db eco_dynamic_view {setup_3v} ; set_db eco_leakage_view {setup_3v}
set_db eco_lef_file_list "tech.lef std.lef ram_256X16A.lef"
set_db eco_def_file_list "top.def blockA.def blockB.def"
set_distributed_mmmc_mode -set_filler_mode {-corePrefix FILL}
set_db opt_signoff_optimize_replicated_modules true
distribute_read_db -restore_design all -restore_dir \
STA_211/ -out_dir reports_Paradime
set_db opt_signoff_partition_list_file partition.txt
opt_signoff -hold
```

Template 5:

The Paradime flow performs inter-partition Hold timing closure on a Top+BM database based on the user-defined partitions. The boundary model generation flow will remain the same as specified in *Template 1*.

The STA saved session can be SMSC or MMMC. Same template can be used to perform inter-partition Power recovery.

Top+BM STA Session:

```
timing_enable_boundary_model_eco
<load_Top+BM_design>
write_db view1.enc
```

Paradime Session:

```
set_distributed_hosts -local -time_out 10000
set_multi_cpu_usage -local_cpu 24 -remote_host 4 -cpu_per_remote_host 4
set_distributed_mmmc_mode -eco true
set_distributed_mmmc_mode -eco_merge_mmmc
set_distributed_mmmc_mode -enable_high_capacity_eco
set_db opt_signoff_inter_partition {blockA blockB}
set_db opt_signoff_intra_partition {}
set_db eco_setup_view_list {setup_1v setup_2v setup_3v setup_4v}
set_db eco_hold_view_list {hold_1v hold_2v hold_3v hold_4v}
set_db eco_dynamic_view {setup_3v}
set_db eco_leakage_view {setup_3v}
set_db eco_lef_file_list "tech.lef std.lef ram_256X16A.lef"
set_db eco_def_file_list "top.def blockA.def blockB.def "
set_distributed_mmmc_mode -set_filler_mode {-corePrefix FILL}
set_db opt_signoff_optimize_replicated_modules true
distribute_read_db -restore_design all -restore_dir \
STA_211/ -out_dir reports_Paradime
set_db opt_signoff_partition_list_file partition.txt
opt_signoff -hold
```

Template 6:

The Paradime flow performs DRV, Setup, Hold timing closure on a Top+BM database based on the user-defined Setup/Hold path_groups, and a list of DRV pins. The boundary model generation flow will remain same as specified in *Template 1*.

The STA saved session can be SMSC or MMMC.

Top+BM STA Session:

```
timing_enable_boundary_model_eco
<load_Top+BM_design>
write_db view1.enc
```

Paradime Session:

```
set_distributed_hosts -local -time_out 10000
set_multi_cpu_usage -local_cpu 24 -remote_host 4 -cpu_per_remote_host 4
set_distributed_mmmc_mode -eco true
set_distributed_mmmc_mode -eco_merge_mmmc
set_distributed_mmmc_mode -enable_high_capacity_eco
set_db opt_signoff_path_group_file_name {TCL script which creates mySetupPG and
myHoldPG path_groups}
set_db opt_signoff_include_setup_path_groups {mySetupPG}
set_db opt_signoff_include_hold_path_groups {myHoldPG}
set_db opt_signoff_select_drv_pin_file {myDRVPinList.txt}
set_db eco_setup_view_list {setup_1v setup_2v setup_3v setup_4v}
set_db eco_hold_view_list {hold_1v hold_2v hold_3v hold_4v}
set_db eco_dynamic_view {setup_3v}
set_db eco_leakage_view {setup_3v}
set_db eco_lef_file_list "~\tech.lef std.lef ram_256X16A.lef"
set_db eco_def_file_list "~\top.def blockA.def blockB.def "
set_distributed_mmmc_mode -set_filler_mode {-corePrefix FILL}
set_db opt_signoff_optimize_replicated_modules true
distribute_read_db -restore_design all -restore_dir  \
STA_211/ -out_dir reports_Paradime
set_db opt_signoff_partition_list_file partition.txt
opt_signoff -drv; opt_signoff -setup; opt_signoff -hold
```

Related Topics

- Signoff ECO Flow
- High Capacity ECO Flow for Timing and Power Closure
- Metal ECO Flow

9

Timing Model Generation

9.1 Extracted Timing Models on page 495

9.2 Boundary Models on page 539

9.3 Prototype Timing Modeling on page 552

9.4 Interface Logic Models on page 562

9.1 Extracted Timing Models

9.1.1 Overview on page 496

9.1.2 ETM Generation on page 497

9.1.3 ETM Generation for MMMC Designs on page 499

9.1.4 Slew Propagation Modes in Model Extraction on page 499

9.1.5 Basic Elements of Timing Model Extraction on page 500

9.1.6 Secondary Load Dependent Networks on page 513

9.1.7 Characterization Point Selection on page 514

9.1.8 Constraint Generation during Model Extraction on page 516

9.1.9 Parallel Arcs in ETM on page 520

9.1.10 Latency Arcs Modeling on page 521

9.1.11 Latch-Based Model Extraction on page 521

9.1.12 Model Extraction in AOCV Mode on page 522

9.1.13 Stage Weight Modeling in ETM on page 522

9.1.14 AOCV Derating Mode on page 523

9.1.15 PG Pin Modeling During Extraction on page 524

9.1.16 Extracted Timing Models with Noise (SI) Effect on page 525

9.1.17 Extracted Timing Models with SOCV on page 526

9.1.18 Merging Timing Models on page 526

9.1.19 Limitations of ETM on page 528

9.1.20 Validation of Generated ETM on page 532

9.1.21 Auto-Validation of ETM on page 535

9.1.22 ETM Extremity Validation on page 536

9.1.23 Limitations/Implications of EV-ETM on page 538

9.1.24 Ability to Check Timing Models on page 538

9.1.1 Overview

Timing model extraction is the process of extracting the interface timing of hierarchical blocks into a timing library. Typically, model extraction has the following advantages:

- Reduces the memory requirements by generation of extraction timing models (ETMs) for respective blocks, which may be huge in size.
- Reduces the static timing analysis (STA) run time.
- Hides the proprietary implementation details of the block from a third-party.

The `write_timing_model` command is used to build a timing model (.lib) of a digital block to be used with a timing analyzer. The extraction of the "actual" timing context for a lower-level module instance is required for achieving timing convergence with large hierarchical design databases. The timing analyzer has the following advantages while analyzing timing extracted models instead of a complete block design:

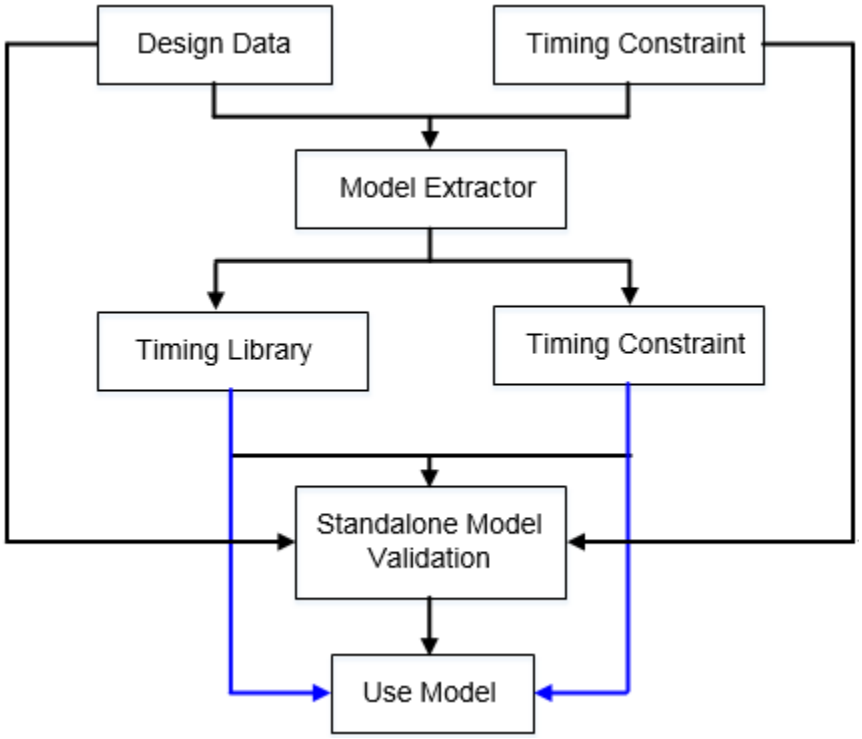
- Provides the timing engine capability to allow for quick derivation of a module's timing context.
- Extracts data correctly across multiple clock domains.
- Allows merging of various models in various modes for their respective blocks.

You can start with model extraction with the following data loaded in the software:

- Design netlist
- Timing libraries
- Block context (constraints, such as, clocks, path exceptions, operating conditions etc.)
- RC data of the nets

The following diagram illustrates this flow:

Figure 9-1 ETM Flow



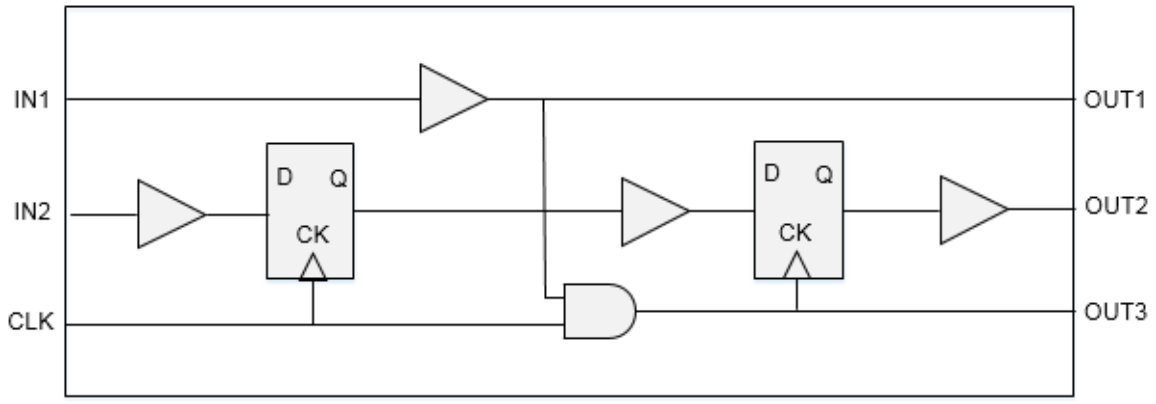
9.1.2 ETM Generation

ETM represents the timing for interface paths. Input to flop/gate, input to output and flop/gate to output paths of a design are preserved as timing arcs in the ETM. If there are multiple clocks capturing the data from an input port then an arc with respect to each input port is extracted.

The flop-to-flop type of paths are not written in the ETM, as these do not affect the interface paths timing.

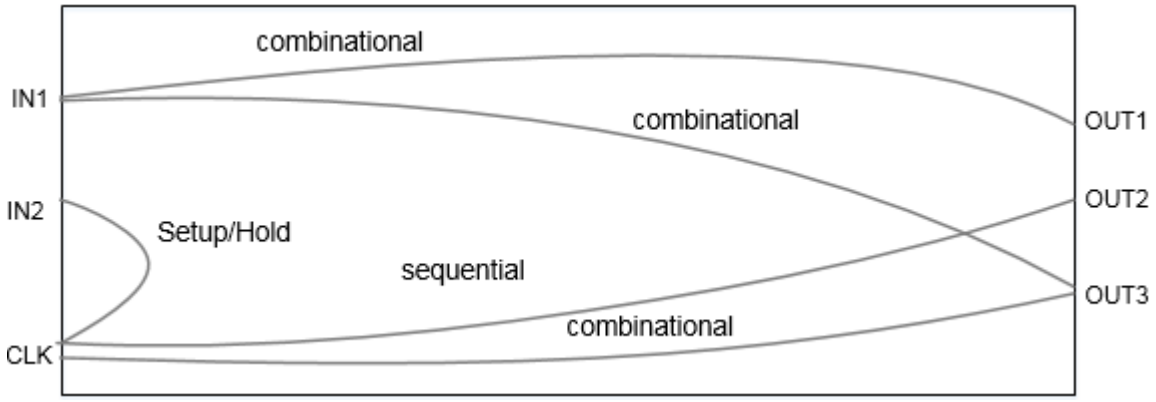
The following figure shows the equivalent ETM for a given netlist:

Figure 9-2 Equivalent ETM for a given netlist



The following figure shows the extracted model:

Figure 9-3 Extracted Model



The extracted timing model is context-independent because it gives the correct value of arcs/ delays, even with varying values of input transitions, output loads, input delays, or output delays. In such cases, it is not required to re-extract the model if some of the context is changed at a later stage of development.

However, the model depends upon the operating conditions, wire load models, annotated delays/loads, and RC data present on internal nets defined in the original design. So if these elements change at the development stage of design then we need to re-extract the model for correlation with the changed scenario.

9.1.3 ETM Generation for MMMC Designs

In MMMC configuration, for generating ETM, only one view should be active. If you need to generate ETM for a specified view, however, the view is not active in setup as well as hold mode, then the software will show an error. It is a prerequisite that the view should be active for both setup and hold analysis. You can use the `set_analysis_view -setup {$viewName} -hold {$viewName}` command before running the `write_timing_model` command to achieve this.

After model extraction for MMMC designs, the dumped assertion files will be added with `${viewName}` suffix, and the dumped derate related assertion files will be added with `${delayCornerName}` prefix. You need to choose the corresponding file while using the extracted models.

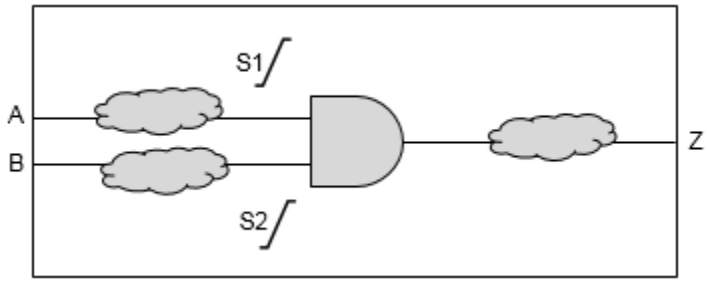
9.1.4 Slew Propagation Modes in Model Extraction

Model extraction supports worst slew and path-based slew propagation modes. These are described below.

- **Worst Slew Propagation**
  In worst slew propagation mode, the worst of all the slews coming at a convergent point is propagated for characterizing the delay of an arc.
- **Path-Based Slew Propagation**
  In path-based slew propagation mode, the slew in the actual path is propagated.

The slew propagation mode can be controlled by using the `timing_extract_model_slew_propagation_mode` attribute. The default value of this attribute is `worst_slew`.

An example of slew propagation is given below.



In worst slew propagation mode, for characterizing both A-Z and B-Z delay arcs, the worst slews S1 and S2 will be propagated to calculate the delay of logic downstream to the AND gate.

In path-based propagation mode, the slew S1 will be propagated to characterize the A-Z arc, whereas S2 will be propagated to characterize the B-Z arc.

9.1.5 Basic Elements of Timing Model Extraction

The following basic elements are relevant to model extraction:

- Nets – internal and boundary nets
- Timing Paths - check and delay paths – in2reg, in2out, and reg2out paths
- Minimum Pulse Width and Period Checks
- Path Exceptions – false, multi-cycle, min delay, max delay, disable timing
- Constants
- Unconstrained Paths
- Clock Gating Checks
- Annotate Delays, Slews, and Load
- Design Rules
- Clocks - created clocks and generated clocks

These are explained in the following sections.

Nets

Nets can mainly be classified into two types – boundary nets and internal nets. The nets which are directly connected to the input or output ports of the block are termed as the boundary nets. The nets which drive and are driven by some internal instance pin of the block are termed as the internal nets. Model extraction treats both the nets differently as boundary nets are always related and connected to the context.

- **Boundary Nets**

  The boundaries are connected to the context. So the RC data for them may change if the context of the block changes at a later stage. To be better context independent the model should not consider SPEF or DSPEF data for their delay calculation and a separate SPEF/DSPEF file for the boundary nets can be stitched to the top level data while instantiating the extracted model.

  Note: Currently, the software does not output the SPEF file for boundary nets which can be used at the top level. To solve this problem, you need to create the top level SPEF file

for boundary nets of this block or use the default behavior to consider the boundary nets RC data for model extraction.

- **Internal Nets**

  As the internal nets connect the pins for the internal instance of the block, they are in no way dependent on the context environment. So the RC data defined for the internal nets is used as it is for delay calculation and is added in the extracted paths. If no RC is defined for these nets, then the wire load models are used to calculate their delays.

  However, if a load or resistance is annotated using the `set_load`/`set_resistance` command, then it will override the annotated SPEF or the applied wire load model.

  If the nets are annotated with the delays using the `set annotated delay` command or SDF annotation, then the annotated delay is used instead of the calculated delays. In case of incremental delays the addition of calculated and incremental delays is used.

Timing Paths

Timing paths are broadly divided in four types:

- In2reg
- reg2out
- in2out
- reg2reg: The reg2reg paths are not relevant to the model extraction process.

These are described below.

- **In2Reg Paths**
  A In2reg path is a setup/hold check that starts from an input port and is captured at a flop or gating element by a clock. So the in2reg type of paths are captured in equivalent setup or hold checks. The setup/hold values to be written in ETM are calculated using the data path delay, the setup/hold value of the library cell and the clock path delay. The equations can be written as follows.

  Setup arc value = data path delay (input to flop) + setup value of flop – clock path delay (clock source to clk pin)
  Hold arc value = data path delay (input to flop) - hold value of flop – clock path delay (clock source to clk pin)

  The value for these arcs is the function of the transition on the input port and the transition at the clock source. If a flop is captured by multiple clocks then separate setup/hold arcs are extracted with respect to each clock source.

- **Reg2out Paths**
  The reg2out paths are paths starting from a register and ending up on an output port. These paths are a combination of trigger arcs (of starting register) and the combinational delay from sink of trigger arc to the output port. So for such paths an equivalent trigger/ sequential arc is modeled in the extracted model. The delay for the arc will be equal to:

  Sequential arc delay = delay (clock source to CK of register) + delay (register CK pin to out port).

  The delay for these arcs is a function of the slew at the clock source and the capacitance at the output port. As the extracted model can be used for max as well as min analysis, two arcs are preserved in the model to represent the longest and the shortest path. Different types (rising_edge/falling_edge) of arcs are extracted for the different valid clock edges.

- **In2Out Paths**
  The in2out paths are the path starting from an input port and ending up on an output port. These paths are pure combinational paths. So for such paths an equivalent combinational arc is modeled in the extracted model. The delay for the arc will be equal to:

  Combinational arc delay = delay (delay of all elements in the path)

  The delay for these arcs is a function of the slew at the input port and the capacitance at the output port. As the extracted model can be used for max as well as min analysis, two arcs are preserved in the model to represent the longest and the shortest combinational path. In case if no path exists for a particular transition (rise/fall), half unate (combinational_rise /combinational_fall) arcs will be extracted. The timing sense for the arc will depend on the function of worst (early/late) paths.

Minimum Pulse Width and Period Checks

The minimum pulse width and period constraints defined at the CK pin of the registers are transferred to the clock source pins during model extraction.

There may be several different type of registers in the fanout of a clock source. So while transferring the minimum pulse width to the clock source you can use the worst values present on the fanout registers. The pulse width is calculated as:

- pulse width = MAX(Arrival time of Launch clock Path - Arrival time of Capture Clock Path + pulse width at clock pins from library)

The minimum period for any clock pin is calculated in the same way as the minimum pulse width. The minimum period is calculated as:

- Min Period = MAX(Arrival time of Launch clock Path - Arrival time of Capture Clock Path + Min Period at clock pins from library)

These checks can be modeled in the following three ways:

- Library Attribute
- Scalar Tables
- Arc-Based Modeling

These are explained below.

Library Attribute

By default, these checks are written as library attributes in ETM.

```
pin (CLKIN2 ) {
   clock : true ;
   min_period : 2.0554;
   min_period : 2.6248;
   min_pulse_width_low : 0.2773;
   min_pulse_width_high : 0.7673;
}
```

If `min_period` checks corresponding to both rise and fall transitions are present in a design, then they wiil be modeled as duplicate entries in ETM when written as library attributes. To avoid duplicate entries we can model these checks as scalar tables or arcs, where rise and fall transition of `min_period` checks can be modeled as shown in following sections. These arcs will be honored by the timer depending on the context, hence will be more accurate.

Scalar Tables

If the `timing_extract_model_write_clock_checks_as_scalar_tables` attribute is set to `true`, then these checks will be written as scalar tables as shown below:

```
pin (CLKIN2 ) {
   clock : true ;
   timing() {
      timing_type : minimum_period ;
      rise_constraint (scalar){
         values( " 2.0554");
         }
      fall_constraint (scalar){
         values( " 2.6248");
         }
      related_pin :" CLKIN2 ";
  }
```

```
    timing() {
       timing_type : min_pulse_width ;
       rise_constraint (scalar){
          values( " 0.7673");
          }
       fall_constraint (scalar){
          values( " 0.2773");
          }
       related_pin :" CLKIN2 ";
   }
}
```

Arc-Based Modeling

These checks can also be written as arcs by setting the `timing extract model write clock checks as arc` attribute to `true`. In this case the resulting arc is the function of rise/fall slew of CLK port, hence delay and slew propagation of the clock network for rise and fall transitions is considered for writing these arcs. The arc-based modeling of clock-style checks is more accurate and is recommended for use.

```
pin (CLKIN2 ) {
    clock : true ;
    timing() {
        timing_type : minimum_period ;
        rise_constraint (lut_timing_2 ){
           values(" 2.000, 2.000, 2.000, 2.000, 2.000, 2.000, 2.000");
           }
        fall_constraint (lut_timing_2 ){
           values(" 2.000, 2.000, 2.000, 2.000, 2.000, 2.000, 2.000");
           }
        related_pin :" CLKIN2 ";
    }
    timing() {
        timing_type : min_pulse_width ;
        rise_constraint (lut_timing_2 ){
            values("0.767, 0.767, 0.767, 0.767, 0.742, 0.087, 0.100");
            }
        fall_constraint (lut_timing_2 ){
            values("0.201, 0.201, 0.201, 0.201, 0.238, 0.247, 0.2636");
            }
        related_pin :" CLKIN2 ";
    }
}
```

Path Exceptions

Timing extraction flow honors path exceptions defined in the given constraints' file. Typically, the following path exceptions are used:

- set_false_path
- set_multicycle_path
- set_min_delay
- set_max_delay

For more information on handling of these exceptions, refer to section "Constraint Generation during Model Extraction".

Constants

The case analysis and constants are propagated while extracting a model. When you specify the set_case_analysis command, you can use the following attribute to control how constants, propagated at o/p or bi-di ports, are modeled in ETM:

```
set timing_extract_model_case_analysis_in_library true (or false)
```

When set to `false`, the software models the propagated or applied constants to output ports in the generated constraints' file (which is generated using the `write_timing_model -assertions` command).

When set to `true`, the software models the propagated constants to output ports written as pin functions in the extracted model.

Unconstrained Paths

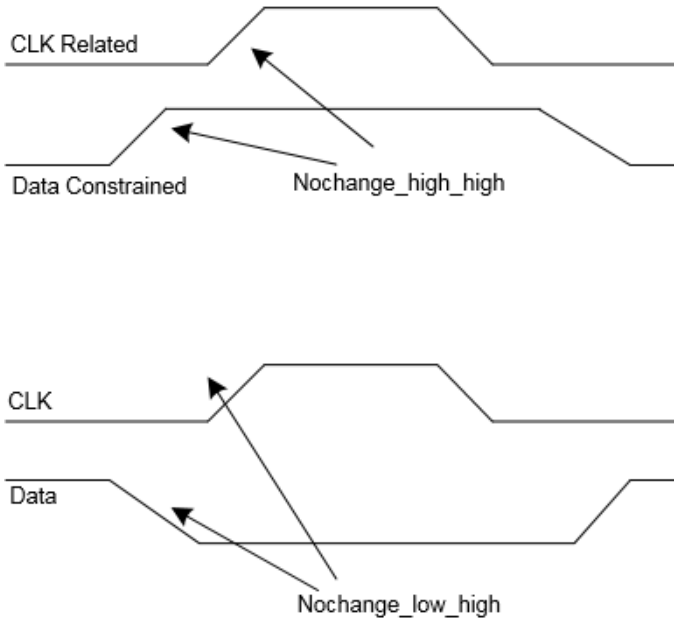
The unconstrained end points exist when proper launch/capture clock phase is not propagated at the desired endpoint due to any exceptions. The unconstrained input ports due to false path exceptions are ignored and are not modeled during ETM extraction. The unconstrained combinational IO paths due to false path exceptions are ignored and are not modeled during ETM extraction. The unconstrained trigger arcs are calculated during ETM extraction.

Clock Gating Checks

If integrated clock gating (ICG) cells are used for gating the clocks, then these arcs are preserved as setup/hold arcs. If combinational cells are used for gating, then these arcs will be preserved as no-change arcs between a clock pin and its enabling signal pin. If you wish to extract these checks as regular setup/hold checks, you can set the `timing_extract_model_gating_as_nochange_arc` attribute to `false`.

Depending on the type of clock gating situation, no change checks are inferred as follows:

Figure 9-4 no_change Checks Modeling



Simple Clock Gating with AND or Logic

The following diagram shows clock gating AND/OR logic:



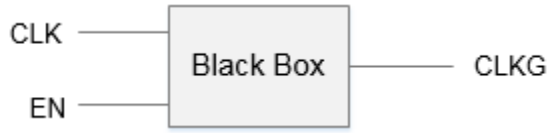
A simple AND gate check will be as follows:

```
timing () {
timing_type: nochange_high_high;
}
timing () {
timing_type: nochange_low_high;
}
```

Note: After extracting the clock gating check arc the signal downstream the gate output is not propagated to the clock pins of the registers.

Clock Gating with Blackbox or Unknown Logic

In case of clock gating with unknown logic, as shown below, no_change checks will be checked with respect to both the rising edge and the falling edge of the clock signal.



No Clock Gating Logic

There are some gates, such as multiplexers or XOR gates, where a clock signal cannot control the clock gate, as shown below.



In such cases, no checks are inferred so you need to explicitly set the gating check if you want these interface paths to be extracted in the ETM. To know where static timing analyzer will not infer the gating in such situations, you can use the following command:

```
check_timing -check_only clock_gating_controlling_edge_unknown -verbose
```

Annotate Delays, Slews, and Load

Model extraction uses the back annotated delay slews and the load information during the extraction process and reflect the effect in the extracted model. Here is a small description how these are used.

- **Annotated Delays**
  The annotated delays using SDF or the `set_annotated_delay` command override the calculated delay (from library or RC data) value for the arc; however, the output slew for the arcs is used as calculated. In case of incremental delays the delta delay is added to the calculated arc delay.
- **Annotated Slews**
  Annotated slews are ignored during timing model extraction. These constraints are dumped in assertions file generated with the `write_timing_model -assertions` parameter.

- **Annotated Load**
  Annotated load overrides the pin capacitance defined in the library, and is used for delay calculations.

Note: The annotated delays are generated with a particular context and remain true for that context (transition times) only. So annotating the delays/slews makes the model context dependent. So to create a context independent model it is recommended not to annotate the SDF.

Design Rules

Model extraction uses the design rules defined in the library. The worst (smaller for max design rules and vice-versa) among all the fanout pins (topologically first level) is used for the input ports and the worst of all the fanin pins (topologically first level) is used for the output ports.

If design rule limits are defined at the pin and library level, the design rule from the pin is chosen, rather than the worst of the pin and library level, because the pin level information overrides the cell level, which in turn overrides the library level information. If design rule violations (DRV) are coming from the constraints, then you can choose them based on the following setting:

`set timing extract model consider design level drv true (or false)`

When set to `false`, the user-asserted design level DRVs are not considered while modeling DRVs in the extracted timing model.

When set to `true`, the user-asserted design level DRVs are considered while modeling DRVs into the extracted timing model.

Clocks

- Created Clocks
- Generated Clocks
- Clocks with Sequential and Combinational Arcs

Created Clocks

If the `create_clock` assertion is applied on the pin of a design (not port), then the pin is modeled as an internal pin with the same name as that of the clock asserted on it. If a clock is created on a design port, then this port will be preserved as ETM I/O pin, with the same name as that of the port itself.

```
create_clock [get_ports {CLKIN}] -name clk -period 3 -waveform {0 1.5}
create_clock [get_pins buf5/Y] -name clk3 -period 3 -waveform {0 1.5}
```

```
    pin (CLKIN ) {
       clock : true ;
       direction : input ;
       capacitance : 0.0042;
   }
   pin (clk3 ) {
       clock : true ;
       direction : internal ;
       capacitance : 0.0110;
   }
```

Generated Clocks

The generated clocks that defined in a hierarchical block are supposed to be a part of the block itself and do not come from the top level. So ETM preserves generated clocks inside the model as internal pins using the Liberty generated_clock construct. The name of any such internal pin is the same as that of the generated clock.

Some of the examples are described below.

Example 1: Single generated clock on a single pin

```
create_generated_clock -name gclk -source [get_ports clk] -divide_by 2
[get_pins buf1/A]
```

During the extraction process the pin “buf1/A” will be preserved as an internal pin in the library with name gclk. The model will show as follows:

```
generated_clock (gclk) {
master_pin : clk;
divided_by : 2;
clock_pin : "gclk ";
}
pin (gclk ) {
clock: true ;
direction: internal ;
.
.
}
```

Example 2: Multiple clocks on the same pin

There are some design scenarios when multiple generated clocks are defined on a single pin. This asserts multiple clock definitions on the pins. In this case, during model extraction the pin is duplicated with the name of the clock. For example, in a design if there are two clock definitions, such as:

```
create_generated_clock -name gclk1 -source [get_ports clk] -add -master_clock CLK
-divide_by 2 [get_pins buf1/A]
create_generated_clock -name gclk2 -source [get_ports clk] -add -master_clock CLK
-divide_by 2 [get_pins buf1/A]
```

During the extraction process the pin buf1/A will be preserved as an internal pin with names gclk1 and gclk2. The model will be as follows:

```
generated_clock (gclk1) {
master_pin : clk;
divided_by : 2;
clock_pin : "gclk1 ";
}
generated_clock (gclk2) {
master_pin : clk;
divided_by : 2;
clock_pin : "gclk2 ";
}
pin (gclk1) {
clock: true;
direction: internal;
…..
}
pin (gclk2) {
clock: true;
direction: internal;
…..
}
```

Example 3: Generated clocks on multiple pins

When a generated clock is defined on multiple pins, the software asserts a single clock definition on multiple pins. To handle this scenario, all the pins asserted with this clock are preserved as internal pins in the model with the name `[clock_name_<count>]` and the generated_clock construct is written in the model with all the pin names in the clock_pin attributes with a space between them. For example, consider a netlist having clock definitions on multiple pins.

```
create_generated_clock -name gclk1 -source [get_ports clk] -add -master_clock CLK
-divide_by 2 [get_pins {buf1/A buf2/A}]
```

During the extraction process, the pin buf1/A will be preserved as an internal pin in the library with the name gclk1_1; and the pin buf2/A will be preserved as an internal pin with name gclk1. The model will be as follows:

```
generated_clock (gclk1) {
master_pin : clk;
divided_by : 2;
clock_pin : "gclk1_1 gclk1 ";
}
pin (gclk1_1) {
clock: true;
direction: internal;
…
}
pin (gclk1) {
clock: true;
direction: internal;
…
}
```

In this case when a ETM is read, two generated clocks gclk1_1 and gclk1 will be created on two internal pins - gclk1_1 and gclk1, respectively. Any constraint coming from the top level will match only with clock gclk1, and not with gclk1_1. To avoid such a situation, you can set `timing library generated clock use group name` attribute to `true`. When this attribute is enabled, the original clock (gclk1) is generated at two internal pins - gclk1_1 and gclk1.

The `report clocks` command shows the following output:

```
-------------------------------------------------------------------------
                        Clock Descriptions
-------------------------------------------------------------------------
Clock Name  Source                       Period  Lead    Trail   Gen  Prop
-------------------------------------------------------------------------
clk         clkin                        3.600   0.000   1.800   n    y
gclk1       BLOCK/gclk1_1 BLOCK/gclk1    7.200   0.000   3.600   y    y
-------------------------------------------------------------------------
On turning off this global, report_clocks will return:
-------------------------------------------------------------------------
                        Clock Descriptions
-------------------------------------------------------------
Clock Name   Source         Period   Lead    Trail   Gen Prop
-------------------------------------------------------------
clk          clkin          3.600    0.000   1.800   n   y
gclk1        BLOCK/gclk1    7.200    0.000   3.600   y   y
gclk1_1      BLOCK/gclk1_1  7.200    0.000   3.600   y   y
-------------------------------------------------------------
```

Example 4: Multiple clocks on master clock root pin

Liberty supports the `master_pin` attribute inside a `generated_clock` group definition, which refers to the pin where the corresponding master clock is defined.

Please note that you are not allowed to define a master clock name in the `generated_clock` group in Liberty. Due to this, the software may infer an incorrect master for the generated clock `genclk -` if multiple clocks are defined on the `CLKIN` pin.

```
generated_clock ( genclk ) {
divided_by : 2 ;
clock_pin : " genclk ";
master_pin : CLKIN ;
}
```

Hence, for the correct master clock association with the generated clocks, you need to provide the `create_generated_clock` definition with the corresponding master clock, while reading the ETM at the top level.

Example 5: Generated clock in case of multi-ETM instantiation at top

When the timing prefix module name with library generated clock attribute is set to `true`, the software appends the instance name to the clock pin name when creating a generated clock.

For example, assume that the cell for instance a/b in the timing library contains the following generated clock group:

```
generated_clock ( genclk1 ) {
      divided_by    :  2 ;
      clock_pin     :  " genclk1 ";
      master_pin    :  CLKIN ;
}
```

By default (`true`), the software creates a generated clock with the name - `genclk1`.

When set to `false`, the software uses only the clock pin name when creating a generated clock.

If ETM is instantiated at the top level as BLOCK1 and BLOCK2, then by default the report_clocks command output will show the following:

```
------------------------------------------------------------
                      Clock Descriptions
-------------------------------------------------------------------------------
Clock Name     Source          Period  Lead   Trail   Generated   Propagated
-------------------------------------------------------------------------------
BLOCK1/genclk1 BLOCK1/genclk1  7.200   0.000  3.600   y           y
BLOCK2/genclk1 BLOCK2/genclk1  7.200   0.000  3.600   y           y
-------------------------------------------------------------------------------
```

If you do not want to differentiate between `genclk1` generated in various ETM instantiations, you can set `timing_prefix_module_name_with_library_generated_clock` attribute to `false`. In this case, the `report_clocks` command will show the following output:

```
-------------------------------------------------------------------------
                     Clock Descriptions
-------------------------------------------------------------------------
Clock Name  Source          Period   Lead   Trail  Generated Propagated
-------------------------------------------------------------------------
genclk1     BLOCK2/genclk1  7.200    0.000  3.600  y         y
-------------------------------------------------------------------------
```

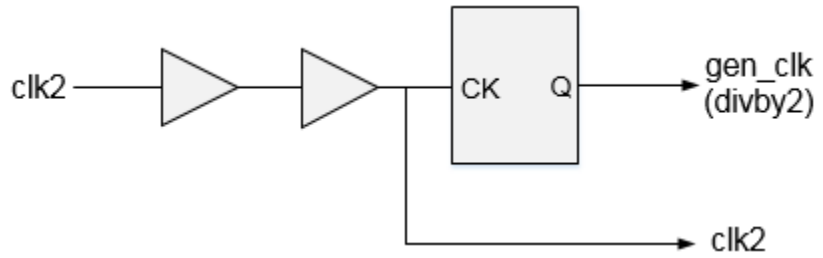
Note: You will need to ensure that the constraints are applied with respect to the generated clock naming. If required, you can remove the library generated clocks and then apply the specified generated clocks so that minimal changes are required in the constraints.

Clocks with Sequential and Combinational Arcs

Consider a case where both the sequential and combinational arcs exist between a clock (CLK) and an output (OUT) pin. If any of the early/late sequential/combination arcs are missing (due to a path exception) between the CLK and OUT pins, the CLK pin is duplicated

as two pins with _SEQ_pin and _COMB_pin suffixes. All the combinational arcs starting from this clock root to the duplicated pin are bound with the “_COMB_pin” suffix and all the sequential arcs are bound to be duplicated pin with the “_SEQ_pin” suffix.

Figure 9-5 Sequential and combinational arcs between a clock



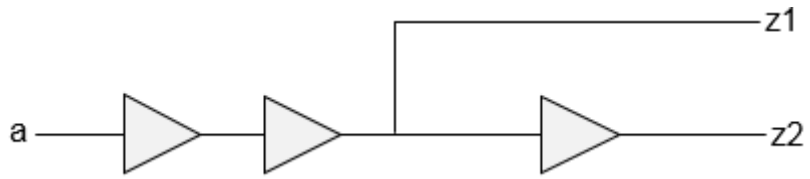
The following will be created in the ETM:

```
pin (CLKIN_SEQ_pin ) {
    clock : true ;
    direction : internal ;
    capacitance : 0.0110;
    }
pin (CLKIN_COMB_pin ) {
    clock : true ;
    direction : internal ;
    capacitance : 0.0110;
    }
pin (DATAOUT ) {
    direction : output ;
    timing() {
        timing_type : combinational ;
        fall_transition (lut_timing_3 ){ … }
        related_pin :" CLKIN_COMB_pin ";
    }
    timing() {
        timing_type : rising_edge ;
        fall_transition (lut_timing_3 ){ … }
        related_pin :" CLKIN_SEQ_pin ";
    }
```

9.1.6 Secondary Load Dependent Networks

If there is a timing path from one output pin to another pin, the model extractor considers both the primary output loading and the secondary output loading when output-to-output delays are computed. In the following figure there is an output-to-output path from z1 to z2:

Figure 9-6 Output-to-Output path from z1 to z2



The extracted timing model for this block consists of two delay arcs - one from a -> z1 and another from a -> z2. But the delay of arc a -> z2 also depends on the secondary loading, that is, at port z1.

This gives rise to a delay arc from a to z2 which is dependent on two loads and one input slew. So this arc will be dumped as a three-dimensional delay table because the delay depends on the primary output loading at z2, as well as a secondary output loading at z1.

The table template will be as follows:

```
lu_table_template (lut_timing_1) {
variable_1: input_net_transition;
index_1 ("0...0 1.0 2.0 3.0 4.0 5.0");
variable_2: total_output_net_capacitance;
index_2 ("0...0 1.0 2.0 3.0 4.0 5.0");
variable_3: related_out_total_output_net_capacitance;
index_3 ("0...0 1.0 2.0 3.0 4.0 5.0");
}
```

By default, 3D arc modeling is disabled - the software considers 3D arcs as 2D arcs in timing modeling.

Note: You should always buffer the port to avoid 3D dependencies. If there is more than a couple of load dependencies, then there will be accuracy loss since model extraction cannot accurately model beyond 3D arcs accurately.

9.1.7 Characterization Point Selection

The selection of characterization during point extracted models depends on the selection criteria described below.

If no input_slews/clock_slews/output_load is specified as an argument in the `write_timing_model` command, then the software determines the characterization points from the designs' interface elements - that are supplied with the external slew/load. This can be explained as follows.

All the check arcs (e.g., clk->in) will be characterized for the slew points as follows:

- Reference slew points will be taken from the slew index of the first element just after the "clk" port.

- Signal slew points will be taken from the slew index of the first element just after the “in” port.

All the sequential (e.g., clk->out) arcs will be characterized as follows:

- Input slew will be taken from the slew index of the first element just after the *clk* port.
- Output load will be taken from the load index of the last element just before the *out* port.

All the combinational arcs (e.g., in->out) will be characterized as follows:

- Input slew will be taken from the slew index of the first element just after the *in* port.
- Output load will be taken from the load index of the last element just before the *out* port.

Consider the following topology in a netlist:

in --------> buf1-- -----> ff1/d -------->buf2 --------> out

clk ------> clk_buf ---- ->ff1/clk-------->buf2 --------> out

in ------> buf3 ------> buf4 --------> buf5 --------> out

A snapshot of the original timing library for the interface elements (i.e., BUF and CLK_BUF) is as follows:

```
Cell (BUF)
{
timing ()
index_1 ("0.0500, 1.4000, 4.5000");
index_2 ("1.0500, 6.5000, 10.0000");
}
Cell (CLK_BUF)
{
timing ()
index_1 ("1.0500, 2.4000, 3.5000");
index_2 ("0.0500, 4.5000, 5.0000");
}
```

If the `write timing model` command is used, then the extracted model will be characterized as follows:

- The signal slew is taken from buf1 slew-index in the original libraries.

  ```
  Check Arc clk->in
  index_1 ("0 0.0500, 1.4000, 4.5000");
  ```

- The reference slew is taken from clk_buf slew-index in the original libraries.

  ```
  index_1 ("0 1.0500, 2.4000, 3.5000");
  ```

- The input slew is taken from clk_buf slew-index in the original libraries.

```
Seq. Arc clk->out
index_1 ("0 1.0500, 2.4000, 3.5000");
```

- The output load is taken from buf2 slew-index in the original libraries.

```
index_2 ("0 1.0500, 6.5000, 10.0000");
```

- The input slew is taken from buf3 slew-index in the original libraries.

```
Combinational arc in->out
index_1 ("0 1.0500, 1.4000, 4.5000");
```

- The output load is taken from buf5 slew-index in the original libraries.

```
index_2 ("0 1.0500, 6.5000, 10.0000");
```

Note: A value of 0 is always added to the characterization points.

9.1.8 Constraint Generation during Model Extraction

The extracted models need to be validated before they are transferred to other designers to be used for a top level analysis or optimization flow. For validation purposes, we need a subset of the original timing constraints and further a subset of these constraints is needed for fitting the model in a top level netlist.

The model extraction process will output a timing library and two constraint files containing the subset of the original constraint. One of these constraint files will be used for a standalone validation of the model compared to the original netlist. By default model.asrt and model.asrt.latchInferredMCP will be written in the CWD. If the `write_timing_model -assertions test.asrt` command is specified, then three assertion files are generated, named -test.asrt. test.asrt.latchInferredMCP, and top_model.asrt. The other constraint file (top_model.asrt) will be used when the extracted model will be stitched to a top level netlist and test.asrt will be used for standalone model validation.

Two separate constraint files are required for the following:

- Standalone validation will need all the context parameters of the design.
- STA or optimization flow when stitched to a top level environment, the context parameter will be automatically coming from the top level. So some of the constraints (in the above file) need to be filtered.

The following sections describe constraints in a model extraction flow:

- False Path Exceptions
- Multi-Cycle Path Exceptions

False Path Exceptions

If the set_false_path constraint is applied through some internal pins/ports, then those paths/arcs are removed during extraction.

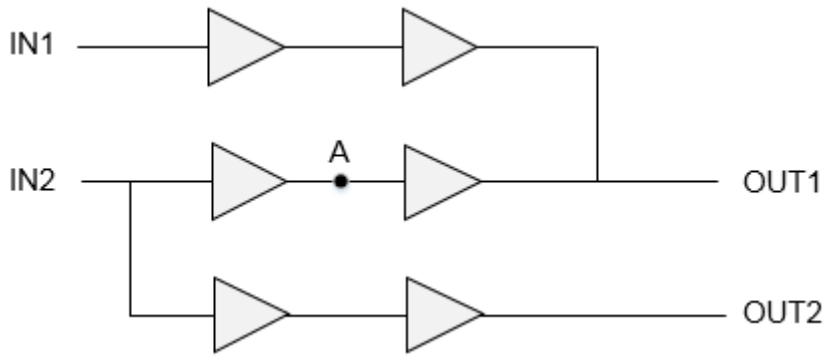
If the set_false_path constraint is applied between clocks or a clock and a port, then these paths are removed and these exceptions are saved in a top_model.asrt as well as test.asrt file.

Multi-Cycle Path Exceptions

If multi-cycle paths through pins/ports are applied, then during extraction the worst paths through them mapped to the IO ports are calculated and the exceptions through these IOs are dumped in the top_model.asrt file.

Consider the following design scenario with multi-cycle path exception applied at A.

Figure 9-7 Multi-cycle path exception



If there is no path from IN1/OUT1 and IN2/OUT2, the multi-cycle path will be pushed to IN2/OUT1. Since there are other paths going through these ports, multi-cycle paths cannot be pushed to IN2/OUT1 and cycle adjustment is done in delay values of the arcs. This will make the model context dependent.

When you set timing_extract_model_disable_cycle_adjustment attribute to true, the software will disable the cycle adjustment to make ETM context independent. You will have to manually apply the multi-cycle path at the top level while using this ETM.

Note: You should examine the constraint file (”top_model.asrt”), as these constraints are just indicative of what is needed to be modeled at the top level. In certain situations, it may not be possible to extract all possible exceptions for top level due to limitations of liberty to model them correctly for a given path in case of conflicts.

- set_disable_timing

  If the set_disable_timing constraint is applied on any arc, then this arc gets disabled and is not used for the extraction purpose. Thus, such arcs are handled internally by the tool and are not written in any of the constraint files. Also, the path broken by them will not be extracted during ETM generation.

- set_case_analysis

  Constants applied/propagated by the set_case_analysis command can be written as a function attribute of pins in the ETM by setting the timing_extract_model_case_analysis_in_library attribute to true. This can be written in the ETM-validation constraint file generated by setting this attribute to false.

- create_clock

  The pins/ports having create_clock definitions on them are preserved in the timing model. If a clock is created on some internal pin then the pin name needs to be modified to reflect the instance name of the extracted model. This is achieved using the same variable approach.

  ```
  [get_pins "$ETM_CORE/pin1 $ETM_CORE/pin2 $ETM_CORE/pin3"]
  ```

  When the extracted model is stitched to some top level netlist, then the clock definitions are supposed to come from the top level netlist itself. So that such clock definitions are not needed to be dumped in the top level constraint file, but need to exist in the assertion file for standalone validation.

- create_generated_clock

  The pins having the create_generated_clock defined on them are preserved in the extracted model as internal pins. As generated clocks are very much intended for the block itself and are not supposed to be coming from the top level, liberty provides syntax to define the generated clocks in the timing model. You can use the following use model:

  ```
  generated_clock (gclk) {
  clock_pin : gclk ;
  master_pin : clk ;
  multiplied by : 2 ;
  }
  ```

  While loading a model, the software names the generated clocks after their target pin names, so while dumping the generated clocks in the library the target pin name is modified to the clock name itself. This facilitates the same name of generated clock names in the original netlist and the extracted model. This name changing cuts down the need to modify other constraints (clock uncertainty/path exceptions) related to this clock. In this case it is not required to dump these generated clocks as a separate constraint.

- set_input_delay/set_output_delay

The constraints like `set_input_delay` and `set_output_delay` lie in this category. Though these constraints can be applied on ports as well as some internal pins. Such constraints applied on some internal pins are of no use for the extracted models, as in an extracted model only interface timing is considered.

The constraints applied on the ports need not any change as the port will be preserved with the same name. So these constraints are dumped out in the assertion file for validation and do not have any impact on the generated ETM. These need not to be dumped out for the top level constraint file as in a top level the data must be coming to the port from some other top level register or port. So the input delay value is supposed to be the delay of path from top level register/port to this port.

- `set_max_transition/set_max_capacitance`

  If these constraints exist in the SDC file for block and the `timing_extract_model_consider_design_level_drv` attribute is set to `false`, then these max_cap/max_trans constraints (from SDC) will not have any impact on the generated model.

  If the `timing_extract_model_consider_design_level_drv` attribute is set to `true`, then these constrains are considered while generating the timing model. The conservative value of max_transition defined at a library cell associated with a port or `set_max_transition` defined in the SDC is chosen for all the ports. Similarly, in case of max_capacitance, a conservative value defined at a library cell associated with a port or defined in the SDC is chosen for all the output ports.

  Note: The `set_max_transition`/`set_max_transition` constraint defined in the SDC is always dumped in the side constraints file that is read during validation.

- `set_load/set_resistance/set_annotated_transition`

  The `set_load`, `set_resistance`, or `set_annotated_transition` constraints can be applied on pins as well as ports. The constraints applied on internal pins are handled in the extraction flow during delay calculation and are included in the model. But the constraints applied on the ports are treated as out-of-context environment and need to be written out in the constraints file to be read for model validation.

- `set_annotated_delay/set_annotated_check`

  The `set_annotated_delay` or `set_annotated_check` constraint is applied on the internal arcs as incremental or absolute values. The annotated delays/checks are handled in the extraction flow during delay calculation, so these are included in the model. This constraint is not written out in any of the constraint files.

- `set_input_transition/set_driving_cell`

The `set_input_transition` or `set_driving_cell` constraints are applied only on ports. At the top level , the input transition at ETM pins depends upon the transition of the signal coming from the top-level circuitry. Similarly, the driving cell is needed only at the block level to replicate the actual driver of ETM at the top level. Hence, these are written only in the validation constraint file.

- `set_clock_uncertainty/set_clock_latency`

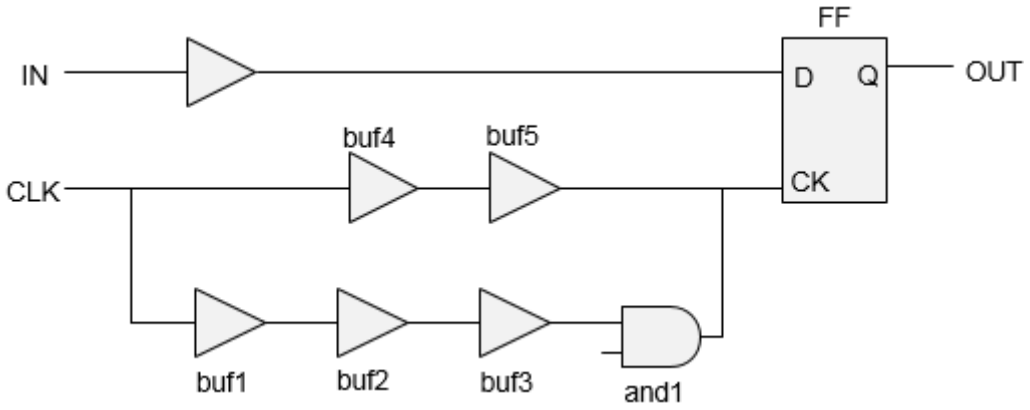
  By default, the clock uncertainty or clock latency constraints are written in the validation constraint file. But if the `timing_extract_model_include_latency_and_uncertainty` attribute is set to `true`, then these values will be taken care of during ETM characterization. And the delay tables in ETM are written with these values included.

9.1.9 Parallel Arcs in ETM

If a model is extracted in the BC-WC or OCV mode, then timing is handled using the early and late path analysis (as done with the `report timing` command).

Consider the following design:

Figure 9-8 Parallel arcs in ETM



In the above design there are two paths CLK->IN (check path) and CLK->OUT (sequential path).

Here, the check (setup) path will be captured by the early path of clock (CLK->buf4->buf5->FF/CK). But the sequential path will be using the late clock path (CLK->buf1->buf2->buf3->and1->FF/CK). The extraction of early/late attributes is differentiated through the min_delay_arc attribute. So, in the extracted model for combinational/trigger/latency arc, both the rise/fall maximum and minimum arcs exist.

9.1.10 Latency Arcs Modeling

Latency arcs are between a generated clock and the respective master clock. The actual paths are considered during extraction. For example, consider a generated clock with -divide_by 2. The edge relationship of master and generated clock will be “R ->R” and “R -> F”. The paths “F->F” and “R->F” arc will not be dumped, even if such paths are present in the design for data propagation.

In case of an ideal master clock, the arcs between master and generated clock source, is controlled through the following setting:

`set timing_extract_model_ideal_clock_latency_arc false (or true)`

When set to `true`, the zero delay arc from the master clock to generated clock is considered in the extracted model. When set to `false`, this arc is not modeled.

9.1.11 Latch-Based Model Extraction

Extraction model handles transparent latches during model extraction based on the arrival times defined on input ports and the specified clock periods - whether a path from an input port to a latch causes borrowing or not.

The following points should be noted:

- If the path causes borrowing then path traversal should continue through the latch until either a non-borrowing latch or edge-triggered flip-flop is encountered. If the path does not cause borrowing, then path tracing should be stopped and a setup arc should be extracted relative to the closing edge of the first (interface) latch.
- The borrowing behavior of latches does not impact the extracted hold arc. So hold arcs should always be extracted relative to the close edge of the first interface register (latch or flip-flop) that is encountered while tracing paths from input ports.
- Borrowing can result in paths being traced through latches from an input port to an output port. In this case, a delay (comb) arc should be extracted from the input port to output port. In scenarios where path becomes more than 1 cycle, inferred multi-cycle paths are dumped in a separate file named as “`model.asrt. latchInferredMCP`”.
- When tracing paths from interface registers to output ports, transparent latches will be handled by tracing through borrowing latches backwards and generating a sequential delay arc from a clock port to an output port.

Note: As all such latch traversal arcs are true for setup analysis, there will be a `set_false_path -hold` statement in the assertion file for any such arcs. Please also note that “model.asrt. latchInferredMCP” file is indicative of what assertions you may need to be

used at the top level. The assertions should be audited before using multi-cycle paths, as they cannot be inferred in all the cases.

9.1.12 Model Extraction in AOCV Mode

Tempus supports modeling of AOCV derated delays and stage weights in ETM. These stage weights and AOCV derates are used during the top level analysis with ETM by correctly computing the stage counts and derates of the paths related to ETM.

The AOCV analysis mode can be enabled by using the following command:

```
set_db timing_analysis_aocv true
```

9.1.13 Stage Weight Modeling in ETM

The stage weight modeling feature can be enabled by using the following command:

```
write_timing_model -include_aocv_weights
```

Using this command will enable the software to write stage weights on individual arcs.

The weights written in the ETM are not meant to derate the model. The model will be containing the derated delays. The stage weight extracted on arcs will be used to calculate the stage count of chip level paths going across the ETM.

The extraction of stage weight will be done by tracing the minimum stage count path on arc-by-arc basis. A path-based cell stage count will be printed on all sequential and combinational arcs. The extraction strategy for computing stage weight will be as follows:

The stage weights in the extracted model are dumped as user-defined attributes. This will require defining three attributes at the arc level.

```
define ("aocv_weight","timing","float");
define ("clock_aocv_weight","timing","float");
The first attribute will be used for combinational and trigger arc and will be
dumped at arc level
as below.
timing () {
timing_type : combinational;
timing_sense : positive_unate;
aocv_weight : 4;
.
.
}
```

The check arcs will be dumped with two separate stage weights for data and clock:

```
timing () {
timing_type : setup_rising;
aocv_weight : 4;
```

```
clock_aocv_weight : 3;
.
.
}
```

9.1.14 AOCV Derating Mode

The AOCV derating mode can be specified as path-based, graph-based, or none using the following setting:

`set timing extract model aocv mode {graph_based | path_based | none}`

The global values are explained below:

- `graph_based`: Delays in ETM are derated using the graph-based stage counts.
- `path_based`: Delay in ETM are derated using total path stage count of worst path between those pin pairs.
- `none`: Models derated delays that contain the effect of user-derate but does not contain effect of AOCV derates. The default is set to `none`.

Before setting this attribute to `graph_based` or `path_based`, it is mandatory that the AOCV libraries are read in and AOCV analysis is enabled (using the `set_db timing_analysis_aocv true` setting).

Merging Model with stage_weight Attribute

The merged timing model contains minimum stage_weight defined between various input library files, hence the merged timing model is pessimistic.

Points to be Considered for AOCV-ETM Flow

The following points need to be considered for a block-level AOCV run:

1. If analysis of a block is performed in the standalone mode, then you need to specify the stage weight seen at the port(s). The applied stage weight affects the internal weight count of the block for the IO paths. If these weights are not applied, the pessimistic analysis will be done for the interfacing paths of the block (when block is top). Hence the ETMs will also have pessimistic numbers for such paths.
2. The same ETM may not be used for multiple instantiation at the top level, due to the following reasons:
   - Different stage counts at the ports (as seen from the top) is possible.

- Different critical paths between the ports for different instances is possible.

  You can use ETM with most conservative stage count for all the instances, but this will lead to pessimistic analysis.

3. The ETMs are written taking AOCV derates into account. At the top level, you need to have two types of AOCV derate tables:
   - Cell level AOCV tables
   - Design level AOCV tables for net
4. While loading ETM for model validation the AOCV derates are already modeled in the ETM. In this case, you should turn off AOCV analysis mode.

If design level cell AOCV derates are used at the top level, then the block delays (as seen from the top) will have the AOCV effect twice - once during model extraction at the block level and second at the top level from design level cell AOCV tables. In case design level AOCV tables are being passed with ETM at the top level, then you should provide a cell level table for ETM with derating as 1 for all the possible stage counts and/or distance.

9.1.15 PG Pin Modeling During Extraction

The power-ground rails of a block are modeled as PG pins of a macro in the corresponding ETM. Since the software infers the information of power-ground rails from CPF/UPF, these files should be read to model PG-pins in the ETM.

To write PG pin information in a timing model, you can use the `write_timing_model -include_power_ground` parameter. The low power attributes can be written at the library level, cell level, and pin level in the ETM.

PG Modeling in ETM

Consider the following scenarios:

- **Modeling of voltage maps**
  Voltage maps definitions are typically modeled inside the Liberty dotlib format at library scope level. It specifies the voltage value associated with the power rail name (voltage names). At cell level pg_pin groups are associated with voltage values using the voltage names.

  ```
  voltage_map (VDD, 0.8);
  voltage_map (VSS, 0.0);
  ```

- **Modeling of power/ground pins**
  At the cell level, the pg_pin groups will be modeled in the ETM in the following way:

```
pg_pin (VDD1) {
voltage_name : VDD ;
pg_type : primary_power;
}
pg_pin (VSS1) {
voltage_name : VSS ;
pg_type : primary_ground ;
}
```

The supported pg_type in model extractions: primary_power, primary_ground, internal_power, internal_ground. If a power rail is the output of a power switch cell, it will be marked as internal_power/ground.

- **Associating power and ground pins to signal pin**
  This information will be written for all the logic input/output/in-out pins that are written in the ETM. For internal pins that are preserved inside the ETM dotlib (e.g., a pin on which the generated clock was asserted), no such information will be retained.

```
pin (A) {
…
related_power_pin: VDD1;
related_ground_pin: VSS1;
}
```

For ports, that are written as pins in ETM, the associated pin (driver or receiver of a net) will be checked and printed in the same way as related_power_pin and related_ground_pin information.

ETM Merging Requirements for Power/Ground-Aware ETMs

- Voltage maps from two libraries will be merged successfully as long as the voltage rail names are distinct.
- The pg_pins will be merged successfully as long as power rail names and the pg_type match.
- The related_power_pin and related_ground_pin will be merged as long as they are available in all the libraries and refer to the same power rail name.

9.1.16 Extracted Timing Models with Noise (SI) Effect

To generate a timing model with SI effects contained within a block, you can use the following flow for a block under consideration:

- Load the database and the relevant information for performing STA/SI analysis.
- Perform SI analysis, or if you have an incremental SDF from the previous SI run just load the incremental SDF.

- Run model extraction. During this phase, the incremental delays will be added to the computed base delays for characterization of the dotlib table. When the final dotlib is written, it contains the effect of the SI analysis in terms of characterized delays.

The block ETM can then be instantiated in the top level flow and the required file can be loaded for performing top level analysis with ETM.

9.1.17 Extracted Timing Models with SOCV

To write ETMs with variation data, set the `timing_extract_model_write_lvf` attribute to `true`. This will enable separate mean-sigma modeling of all the delay, constraint, and transition values.

When set to `false`, the product of sigma multiplier and the sigma value of delay/constraint/transition will be added (or subtracted) to (or from) the corresponding mean value and modeled in the ETM.

9.1.18 Merging Timing Models

The software supports merging library models through the `merge_model_timing` command. You can generate ETM in various modes and then merge them in a single library (in which all the modes are defined inside a merged group) that can be used for timing optimization during design flow. These mode_group/modes are defined in merged ETM as mode_definition/mode_value.

To merge two ETMs (corresponding to funct and scan views), use the following command:

```
merge_model_timing -library_file funct_etm.lib scan_etm.lib -modes funct scan -
mode_group funct_scan -outfile merged_funct_scan.lib
```

These modes are defined in a merged ETM as:

```
mode_definition (funct_scan){
mode_value (funct){
}
mode_value (scan){
}
```

In a merged ETM, the arcs corresponding to different modes are defined as given below:

```
timing() {
mode( funct_scan," funct " );
min_delay_arc : "true" ;
related_pin :" SI_ClkIn ";
}

timing() {
mode( funct_scan ," scan " );
```

```
related_pin :" EJ_TCK ";
}
```

To read arcs corresponding to funct (scan) mode, use set_mode funct (scan). If the set_mode command is not issued, the arcs corresponding to both the modes will be reported by the report_timing command.

The single mode can be specified using the set_mode command. An error is issued if more than one mode is passed as an argument to this command. The mode merging enables you to do the analysis in various modes on a single shell.

Some considerations while merging models are given below:

- All the input timing models should have the same cell name.
- If the timing arcs are not comparable i.e., the number of index values differ, then they are written as separate timing arcs with separate mode definitions.
- Boundary pins of single mode libraries should be equivalent in number and name while merging. An error message will be flagged if that is not the case.
- If you have generated clocks with the same name, as well as same definition in ETM libraries, then just this clock is preserved in the merged library, without issuing an error message.
- If you have generated a clock with the same name, but different divided_by/multiply_by/ master_pin/clock_pin definition, then an error message is displayed stating that “genclk is defined with different specification in two modes, merging is not possible”.
- If there is a mismatch in timing arcs among the input libraries, then the merged library will contain the union of all the timing arcs with the respective mode. For example, if the first library has only setup arcs and the second library has only hold arcs then the merged library will have both setup and hold arcs with respective modes appended.
- If there are mismatches in internal pins among input libraries, that is, if a particular internal pin is missing in one or more libraries then that internal pin and its timing arcs will be part of a merged library with respective mode.
- For both maximum and minimum DRV, you can use the most conservative value. The minimum value from maximum DRVs will be used and the maximum value from minimum DRVs will be used in the merging.
- When same modes are specified in the modes list for two libraries, for example, merge_model_timing -input_library_files "setup.lib hold.lib" - modes "M M" -mode_group “MG”, then the arcs from the two libraries will be assigned the same mode, that is, “M” while merging.

9.1.19 Limitations of ETM

The limitations of ETM are as follows:

- In path-slew propagation mode, the slew/delay dependency on side inputs will be lost.
- Consider the following two cases:
  - clk-in check ac will depend on the slew at the D and CK pins of the register. Hence these check arcs are modeled as 2D-check arcs, as expected.
  - In the second case, the slew at D pin will depend upon the loading at the out port. Hence check arc value will essentially depend upon the slews at the D and CK pins and the out load as well. But currently model extraction does not support 3D modeling of check arcs.

Figure 9-9 Check arc

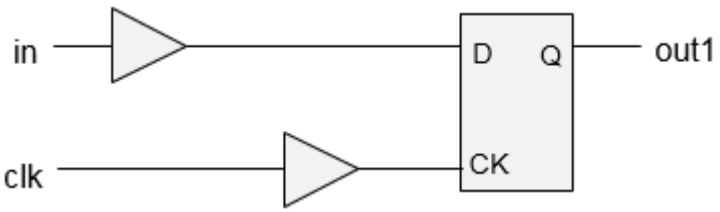
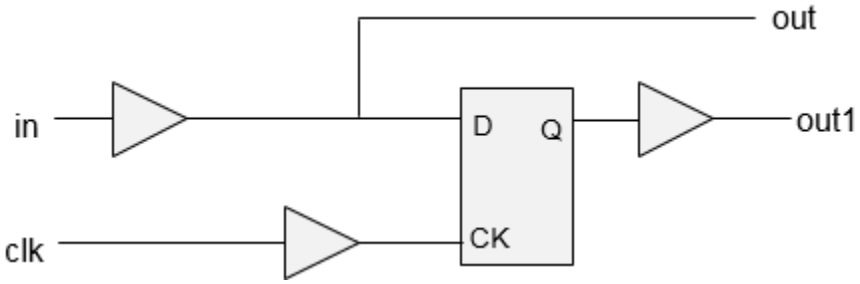


Figure 9-10 Load dependent check arc



The following timing report shows setup_rising present in the ETM:

```
Path 1: VIOLATED Hold Check with Pin cppr_block/CLKIN
Endpoint: cppr_block/DATAIN (^) checked with leading edge of 'clk'
Beginpoint: DATAIN (^) triggered by leading edge of 'clk2'
Path Groups: {clk}
Other End Arrival Time          0.000
+ Hold                          5.000
+ Phase Shift                   0.000
= Required Time                 5.000
Arrival Time                    0.500
Slack Time                     -4.500
```

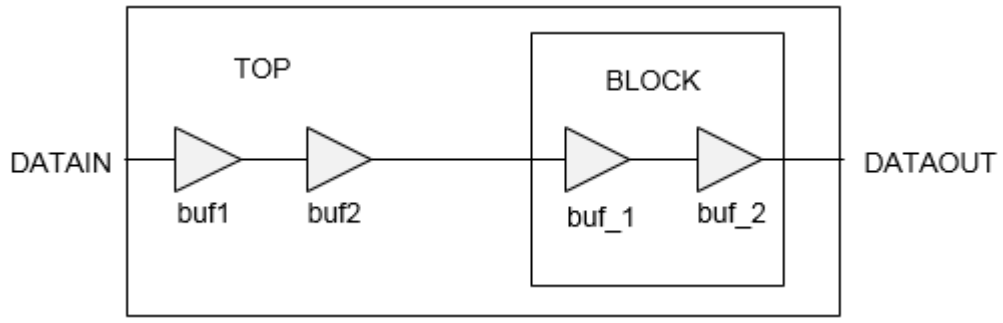
```
Clock Rise Edge                          0.000
+ Input Delay                            0.500
= Beginpoint Arrival Time                0.500
Timing Path:
-------------------------------------------------------------------
Pin                  Arc            Cell            Edge      Arrival
                                                               Time
-------------------------------------------------------------------
DATAIN ->            DATAIN ^                       ^          0.500
cppr_block/DATAIN                 cppr_block        ^          0.500
-------------------------------------------------------------------
```

The following timing report shows no setup_rising in the ETM:

```
Path 1: VIOLATED Hold Check with Pin cppr_block/CLKIN
Endpoint: cppr_block/DATAIN (^) checked with leading edge of 'clk'
Beginpoint: DATAIN (^) triggered by leading edge of 'clk2'
Path Groups: {clk}
Other End Arrival Time         0.000
+ Hold                         5.000
+ Phase Shift                  1.800
= Required Time                6.800
Arrival Time                   0.500
Slack Time                    -6.300
Clock Rise Edge                          0.000
+ Input Delay                            0.500
= Beginpoint Arrival Time                0.500
Timing Path:
-------------------------------------------------------
Pin                 Arc      Cell        Edge   Arrival
Time
-------------------------------------------------------
DATAIN ->           DATAIN ^                ^     0.500
cppr_block/DATAIN             cppr_block    ^     0.500
-------------------------------------------------------
```

- When ETM is read at the top, the information about the stage count inside the block is lost. Hence for paths coming in/out of the block, incorrect stage count is calculated while applying AOCV derate (on the top level instances which fall in this path).

Figure 9-11 ETM at the top level



When BLOCK netlist is read at the top level, the stage count for buf1 and buf2 is 4, but when an ETM of BLOCK is read, their stage count becomes 2.

The following example shows a timing report with BLOCK netlist:

```
Path 1: MET Late External Delay Assertion
Endpoint: DATAOUT2 (^) checked with leading edge of 'clk'
Beginpoint: DATAIN (^) triggered by leading edge of '@'
Path Groups: {clk}
Other End Arrival Time          0.000
- External Delay                0.010
+ Phase Shift                   3.600
= Required Time                 3.590
- Arrival Time                  0.354
= Slack Time                    3.236
Clock Rise Edge                          0.000
+ Input Delay                            0.000
= Beginpoint Arrival Time                0.000
-------------------------------------------------
Aocv       Aocv       Cell       Pin
Stage      Derate
Count
-------------------------------------------------
                                 DATAIN
4.000      1.169     BUF         buf2/Y
4.000      1.169     BUF         buf_2/Y
4.000      1.169     BUF         BLOCK/buf7/Y
4.000      1.169     BUF         BLOCK/buf8/Y
5.000      1.167                 DATAOUT2
-------------------------------------------------
```

The following example shows a timing report with BLOCK ETM:

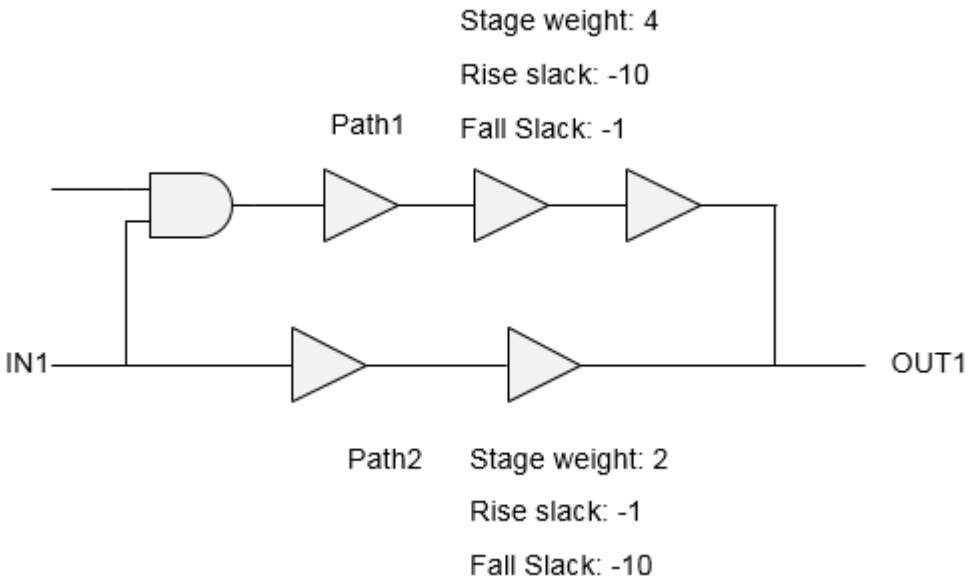
```
Path 1: MET Late External Delay Assertion
Endpoint: DATAOUT2 (^) checked with leading edge of 'clk'
Beginpoint: DATAIN (^) triggered by leading edge of '@'
Path Groups: {clk}
Other End Arrival Time         0.000
- External Delay               0.010
+ Phase Shift                  3.600
= Required Time                3.590
- Arrival Time                 0.325
= Slack Time                   3.265
Clock Rise Edge                         0.000
+ Input Delay                           0.000
= Beginpoint Arrival Time               0.000
-----------------------------------------------------------------------
Aocv      Aocv        Cell         Pin
Stage     Derate
Count
-----------------------------------------------------------------------
                                   DATAIN
2.000    1.173       BUF          buf2/Y
2.000    1.173       BUF          buf_2/Y
3.000    1.171       cppr_block   BLOCK/DATAOUT2
```

```
4.000    1.169                        DATAOUT2
-------------------------------------------------------------------
```

While modeling any arc, the rise and fall constraints are written separately. While modeling stage weights, the worst of rise/fall stage weight is written in ETM.

Consider the following example showing two paths with given stage weight and rise/fall slack values. The rise and fall arcs will be modeled corresponding to Path1 and Path2, respectively, between IN1/OUT1. The single value of stage weight (4 in this case) will be written in ETM. Ideally, the fall constraint should have the stage weight of 2 in this case, but due to the limitation, the stage weight becomes 4 adding to the pessimism in analysis.

Figure 9-12 Modeling stage weights




```
pin (DATAOUT ) {
direction : output ;
timing() {
    timing_type : combinational ;
    timing_sense : positive_unate ;
    cell_rise (lut_timing_2 ){
       values(\
       " -10, -20", \
       " -30, -40" \
       );
     }
cell_fall (lut_timing_2 ){
    values(\
    " -10, -20", \
    " -30, -40" \
```

```
    );
   }
aocv_weight : 4.0000;
related_pin :" CLKIN ";
}
```

9.1.20 Validation of Generated ETM

The extracted models should be validated before stitching to the top level for their accuracy and coverage. This block-level validation can be achieved by comparing the interface timing of the extracted timing models against the original gate-level netlist, by using the report_timing_model and check_timing_model commands.

A brief description of the ETM commands used in a validation flow is given below:

Commands Used in Validation Flow

- **write_timing_model**
  The ETM is generated by using the write_timing_model command. The use model is given below:

  ```
  write_timing_model top.lib -verilog_shell_file test.v -
  verilog_shell_module test-assertions test.asrt
  ```

  The following are the outputs:

  - verilog_shell_file (test.v): Verilog having an ETM instantiated in it. In case the block constraints file has the set_driving_cell command, then the cell used there will be instantiated in test.v.
  - verilog_shell_module(test): Module name of test.v.
  - top.lib: The resulting ETM.
  - test.asrt: The resulting exception file to be used in a validation flow.

- **report_timing_model**
  The report_timing_model command writes a report on the interface timing of a specified netlist. The use model of the command is as follows:

  ```
  report_timing_model –type slack ref.rpt
  ```

  The report ref.rpt contains the following timing view properties to capture the timing characteristics of the design being extracted as a timing model:

  - Slack or Arc Value: Reports the worst-case slack or arc value for each path from input port to clock, from clock to output port, and from input port to output port.

- Transition Time: Reports the actual transition time at each port for the four delay types: min_fall, min_rise, max_fall, and max_rise.
- Capacitance: Reports the maximum total (lumped) capacitance at each port, and if available, the effective capacitance.
- Design Rules: Reports all the design rules that apply to each port, including the maximum capacitance, minimum capacitance, maximum transition time, maximum fan-out (for input ports), and fan-out load (for output ports).

- **check_timing_model**
  The check_timing_model command compares two reports generated by the write_model_timing command. If the timing parameter values in the two files are the same or within the specified tolerance, then the result is pass, or otherwise fail. The use model is as follows:

```
check_timing_model –ref ref.rpt -compare comp.rpt -outFile final.rpt
percent_tolerance 3 -absolute_tolerance [expr 0.30/$timeUnit]
```

  The check_timing_model report shows the comparison results for individual paths, ports, and timing parameters. The resulting comparison report has the same sections as the interface timing report: slack, or arc value, transition time, capacitance, design rules. There are two tolerance settings as follows:

  - Absolute tolerance: Indicates the absolute acceptable difference between compared parameters in library time units.
  - Percentage tolerance: Shows the percentage of acceptable difference between compared parameters.

  A path is flagged as a failure by the check_timing_model command if it violates both the absolute and percentage tolerance values.

Validation Flow- SMSC

- **Step1**: Generating timing model (ETM) (using the write_timing_model command) and the report_timing_model command output from the original session, as described below.

```
read_db
write_timing_model test.lib -verilog_shell_file test.v -
verilog_shell_module test -assertions test.asrt
report_timing_model -type slack ref.rpt
exit
```

- **Step2**: Loading the generated ETM, generating the report_timing_model report for the ETM session, and then comparing reports with the reference session using the check_timing_model command as described below.

```
read_libs test.lib
read_netlist test.v -top test
init_design
read_sdc test.asrt
report_timing_model -type slack comp.rpt
check_timing_model -reference ref.rpt -compare comp.rpt -out_file
final.rpt -percentage_tolerance 3 -absolute_tolerance 0.003
```

Note: It is recommended to set timing_prefix_module_name_with_library_generated_clock attribute to false, and timing_library_generated_clock_use_group_name to true. For more information, refer to section on Generated Clocks.

At the end of a complete validation flow, the following files will be written:

- Files generated by write_timing_model (test.lib, test.v, test.asrt)
- Interface timing (report_timing_model) report from netlist session (ref.rpt)
- Interface timing (write_model_timing) report from ETM session (comp.rpt)
- Comparison (check_timing_model) report of the above two files (final.rpt)

Validation Flow - MMMC

- **Step1**: In MMMC configuration, only one view should be active before running the write_timing_model command, which is done by using the set_analysis_view -setup {$viewName} -hold {$viewName} command.

  Suppose if there are two views in setup, two views in hold, and two RC corners, as given below:

  ```
  set_analysis_view -setup {view1 view2 } -hold {view3 view4}
  ```

  To run all the four views in CMMMC mode and in order to model the libraries in ETM, use the following flow:

  ```
  set_analysis_view -setup {view1 view2 view3 view4} -hold { view1 view2
  view3 view4}
  read_spef -rc_corner <corner1>
  read_spef -rc_corner <corner2>
  .....
  set viewList [all_analysis_view -type active]
  foreach viewName $viewList {
  file mkdir $viewName
  write_timing_model -view $viewName $viewName/test.lib -assertions \
  $viewName/test.asrt -verilog_shell_file $viewName/test.v \
  -verilog_shell_module test
  ```

```
report_timing_model -view $viewName -type slack -verbose $viewName/
ref.rpt
}
```

- **Step2**: For validation of ETM generated from multiple views, the following syntax can be used to read the ETMs from their corresponding directory.

```
foreach viewName $viewList {
free_design
read_verilog $viewName/test.v
read_lib $viewName/test.lib
set_top_module test
read_sdc $viewName/test.asrt.$viewName
write_model_timing -type slack -verbose $viewName/comp.rpt
compare_model_timing -ref $viewName/ref.rpt -compare $viewName/comp.rpt
-ignore_view -outFile $viewName/final.rpt -percent_tolerance 2 -
absolute_tolerance 0.003
}
exit
```

  It is recommended to set `timing_prefix_module_name_with_library_generated_clock` attribute to `false`, and `timing_library_generated_clock_use_group_name` to `true`.

  For more information, refer to Generated Clocks.

  At the end of a complete validation flow, the following will be written out in each view-specific directory for the corresponding view:

  - Files generated by `write_timing_model` (`test.lib`, `test.v`, `test.asrt.$view`)
  - Interface timing (`report_timing_model`) report from netlist session (`ref.rpt`)
  - Interface timing (`report_timing_model`) report from ETM session (`comp.rpt`)
  - Comparison (`check_timing_model`) report of above two files (`final.rpt`)

9.1.21 Auto-Validation of ETM

Tempus provides an automated validation flow for ETM, in which all the validation steps mentioned in the previous steps can be run automatically. This can be done by using the `write_timing_model -validate` parameter.

Note: When the `-validate` parameter is used, the existing validation directory will be removed and a new directory will be created. It is recommended to specify different directories for the output ETM file and auto-validation reports.

This flow will write out the required report timing model and check timing model reports in $val_dir. At the end of the auto-validation, the shell with block netlist is retained. The comparison of both setup and hold checks is performed by the auto-validation.

The following summary is generated at the end of the auto-validation:

```
-------------------------------------------------------------------------
|Total_fail  | Slack_Fail | Cap_Fail | Trans_Fail | DRV_Fail | checkType
-------------------------------------------------------------------------
| 5          |          2 |        1 |          0 |        2 | setup
| 2          |          1 |        0 |          0 |        1 | hold
-------------------------------------------------------------------------
```

9.1.22 ETM Extremity Validation

The ETM models the timing arcs in the design for a range of discrete input-slews/output-loads. The slew/load asserted using the SDC at the block-level during ETM characterization represents the anticipated context of the ETM instance at the top-level. However, the current validation of the ETM is done only for the input-slews/output-loads that have been asserted on the design during the ETM extraction. Therefore, the current validation of ETM is incomplete in the sense that the validation is not being done for a complete range of slew/load points for which ETM has been characterized.

Additionally, in GBA mode the problem of ETM validation is computationally more difficult. In GBA mode, the assertion of a slew on a particular input port may have an impact on the timing of a path starting from some other input port. Therefore, different combinations of input slews at the input ports will result in different timing behavior of the block in GBA mode. Since there can be exponentially large number of combination of slews at the input ports, validation of ETM in GBA mode is computationally more difficult.

The “Extremity-Validation” of ETM (also referred as EV_ETM) validates ETM at the minimum/maximum value of the slew at the input ports and minimum/maximum value of the output load at the output ports, thereby validating the ETM to a greater extent of the possible scenarios at the top. The minimum/maximum slew at the input port is defined as the minimum/maximum slew for which that input port has been characterized in the ETM library. Similarly, the minimum/maximum load at the output port is defined as the minimum/maximum load for which that output port has been characterized in the ETM library.

The minimum/maximum value of the slew/load will yield four corners of the ETM context, as given below:

- Corner 1 (Fast): the minimum slew at the input ports and the minimum load at the output ports
- Corner 2: the minimum slew at the input ports and the maximum load at the output ports
- Corner 3: the maximum slew at the input ports and the minimum load at the output ports

- Corner 4 (Slow): the maximum slew at the input ports and the maximum load at the output ports

The validation flow in exhaustive mode is the same as that in non-exhaustive mode, that is,

- write_timing_model
- report_timing_model at the block level
- Read ETM at top level
- report_timing_model at the top level
- check_timing_model

This flow is controlled by the timing_extract_model_exhaustive_validation_mode attribute.

The additional files generated in the EV-ETM (but not generated in normal validation) will be stored in the current working directory by default. However, the location to save these files can be controlled by the timing_extract_model_exhaustive_validation_dir attribute. These files are written as hidden files.

Also note that the timing_extract_model_exhaustive_validation_dir attribute should point to the same directory at the block and top level. When this attribute is not used at the block level, then the EV-ETM files (in step 1 and 2 above) will be dumped in the working directory at the block level. If the working directory at the top level is different than that at the block level, then at the top level timing_extract_model_exhaustive_validation_dir attribute should point to the block level working directory.

The check_timing_model command will compare the report_timing_model reports written at the block level with the corresponding top-level report. The output of check_timing_model command, corresponding to the four corners is written to a file named: <output_file_name>_ev - where <output_file_name> is the name of the output-file specified using the check_timing_model command. The results of four different corners are written in four different columns as shown below:

```
#Slack(SS)/Status Slack(SF)/Status Slack(FS)/Status Slack(FF)/Status Transition
Arc-Type From To
--------------------------------------------------------------------------------
--------------------
7.511[7.511]/PASS 7.511[7.511]/PASS 7.466[7.466]/PASS 7.466[7.466]/PASS rise/rise
setup CLK1 CLOCK1
9.666[9.665]/PASS 9.666[9.665]/PASS 9.336[9.321]/PASS 9.336[9.321]/PASS fall/rise
setup CLK3 CLOCK2
```

In this mode, there is no change in the use models of the write_timing_model, report_timing_model, and check_timing_model commands. The behavioral

difference (between default validation and EV_ETM) is the creation of additional four files corresponding to the four corners.

If the slew propagation is set to worst, then the `write_timing_model` and `report_timing_model` commands will issue a warning.

Currently, the extremity validation feature is not supported with auto-validation. You will need to run the two-step validation flow in this mode.

9.1.23 Limitations/Implications of EV-ETM

The EV-ETM flow has the following limitations/implications:

- The runtime of validation will appreciably increase.
- The EV-ETM will be supported only in path-based mode and not in the worst-case mode.
- The EV-ETM, though validates the ETM at the corners, cannot be considered as fully rigorous. For example, the proposed methodology does not validate the timing obtained by interpolation of the tables characterized in ETM.

9.1.24 Ability to Check Timing Models

The `write_timing_model -post_check` parameter command checks the extracted timing model for any possible Liberty-related issue. When specified, the command displays detailed error and warning messages and their summaries.

9.2 Boundary Models

9.2.1 Overview on page 540

9.2.2 Boundary Model Flow on page 540

9.2.3 Hierarchical Flow with Boundary Model on page 541

9.2.4 Clock Mapping in Top Level Run with Boundary Models on page 545

9.2.5 Recommendations for Generating Accurate Boundary Models on page 546

9.2.6 Glitch-Specific Boundary Models on page 547

9.2.7 Boundary Models in MMMC Mode on page 550

9.2.8 Save-Restore of Hierarchical Analysis on page 551

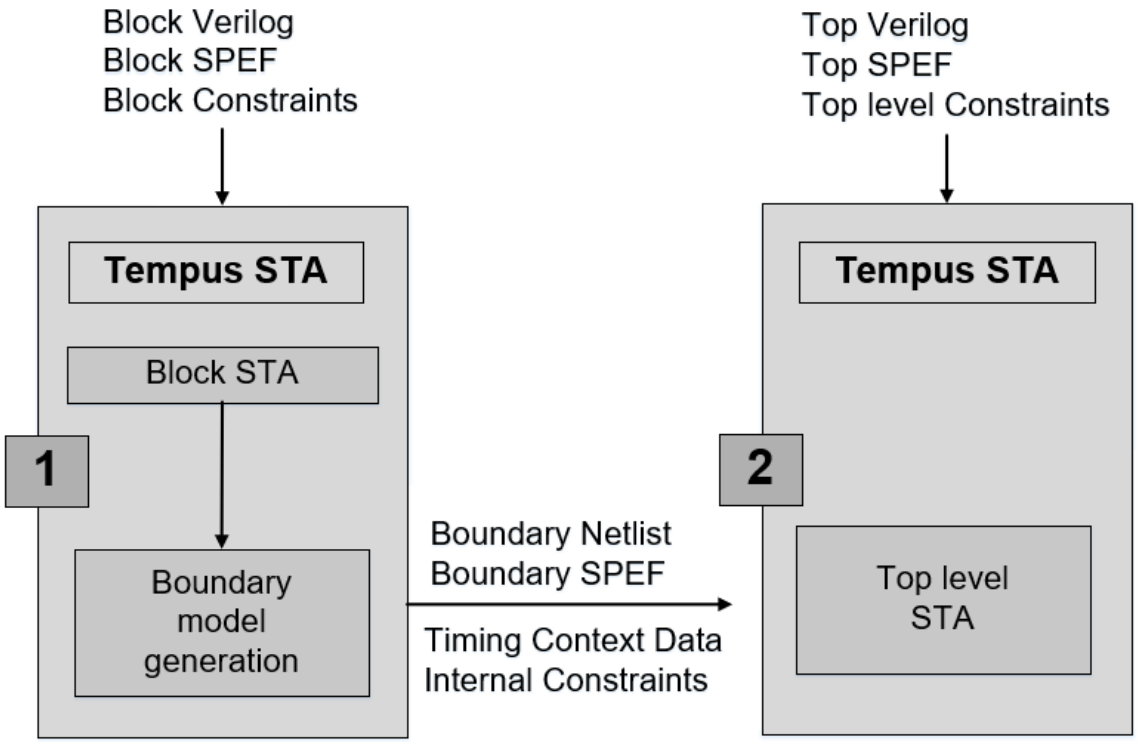
9.2.1 Overview

A boundary model for a module provides sufficient details of its interface paths in order to accurately analyze these paths for timing (including path-based analysis) when loading the model into timing analysis of a higher-level module. The model includes timing context information that improves the accuracy of the model, for example, the timing windows of attacker nets that are not on interface paths for SI analysis.

9.2.2 Boundary Model Flow

The following figure shows the flow for creating and using a boundary model.

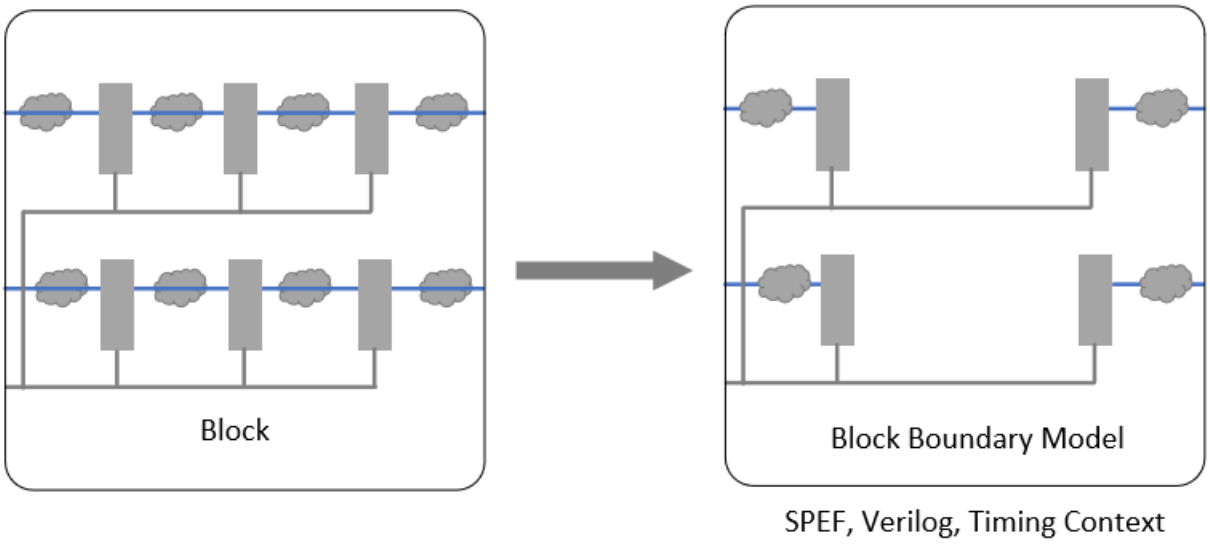
Figure 9-13 Boundary Model Flow



The first step is to perform a full timing analysis on the lower-level module. This analysis is done using block-level constraints for the module. After the analysis is complete, the boundary model can be generated.

The following figure shows a typical block and its boundary model.

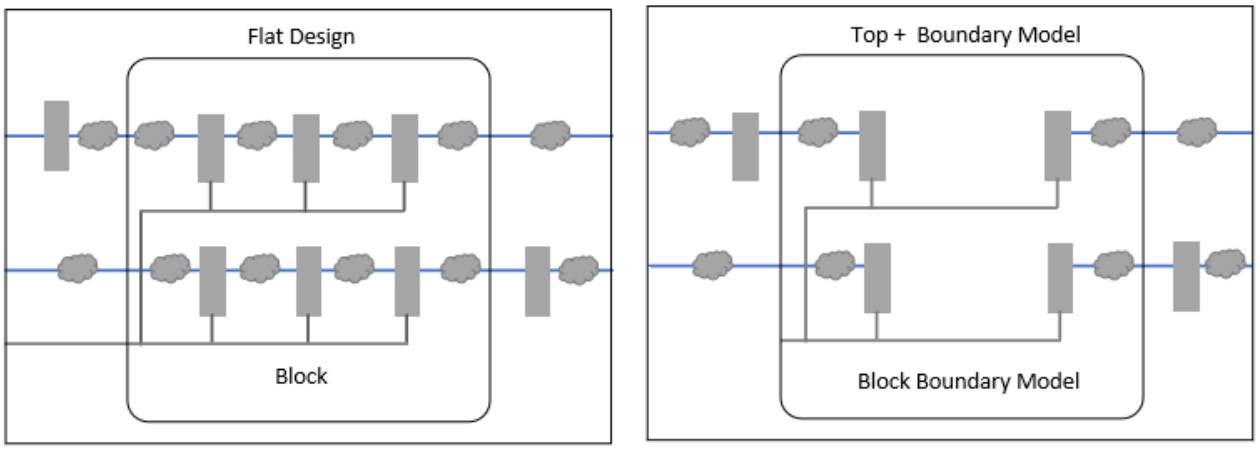
Figure 9-14 Block with the generated boundary model



This model can be subsequently loaded into a timing session for a higher-level module. The higher-level session is timed using flat constraints for those constraints that affect the interface paths of the instances of all the boundary model blocks.

The following figure shows the top level design with a block flat design vs block boundary model:

Figure 9-15 Top level design with flat design vs block boundary model



9.2.3 Hierarchical Flow with Boundary Model

The boundary model hierarchical flow is described below.

Creating Boundary Models

For creating a boundary model of a block, you can use the create_boundary_model command.

The block run script can be modified as follows:

```
read_libs <>
read_netlist block.v -top block
init_design
read_spef block.spef
read_sdc block.sdc
create_boundary_model -dir bm
```

Note: The create_boundary_model command runs the update_timing command by default. It is recommended not to invoke this command explicitly. If you manually include the update_timing command, this might result in redundant execution and increased runtime, especially for large designs.

Boundary Model Usage

In the top-level timing run, the boundary model can be loaded (instead of the full block Verilog and SPEF) by using the read boundary model command before the init design command is run. The boundary models can be loaded for multiple blocks.

The following example shows a sample top run script for flat timing analysis:

```
read_libs [lib1, lib2]
read_boundary_model -dir BM1
read_boundary_model -dir BM2
read_netlist 'block.v'
init_design
read_sdc tst.sdc
read_spef tst.spef
update_timing -full
```

This script can be modified for top analysis with boundary models, as shown below:

```
read_libs <>
read_boundary_model -dir bm
init_design
read_spef top.spef
read_sdc top.sdc
update_timing
```

It may be noted that none of the higher-level SPEF files should contain parasitics for the boundary model block, because this will lead to extra parasitics being present on the primary I/O nets of the block instances. Therefore, for performing timing analysis with boundary models you must use the hierarchical SPEF.

If both the boundary model and Verilog of a block are read at the top level, then the model netlist will override the block netlist. This run will have a higher run time and memory usage as compared to the run with only block boundary models - with the same accuracy. Hence, it is recommended to remove the block Verilog from the top run script.

Boundary Model Contents

A boundary model consists of the following:

- Netlist for a model in a binary form:
  - Interface paths

    The logic which constitutes the interface paths of a block are saved in the boundary model netlist. This includes the paths from the primary input ports to the sequential input pins, from the sequential output pins to the output ports, and from the input port(s) to the output ports. The interface paths are timed in the top level analysis.
  - Extended fanin

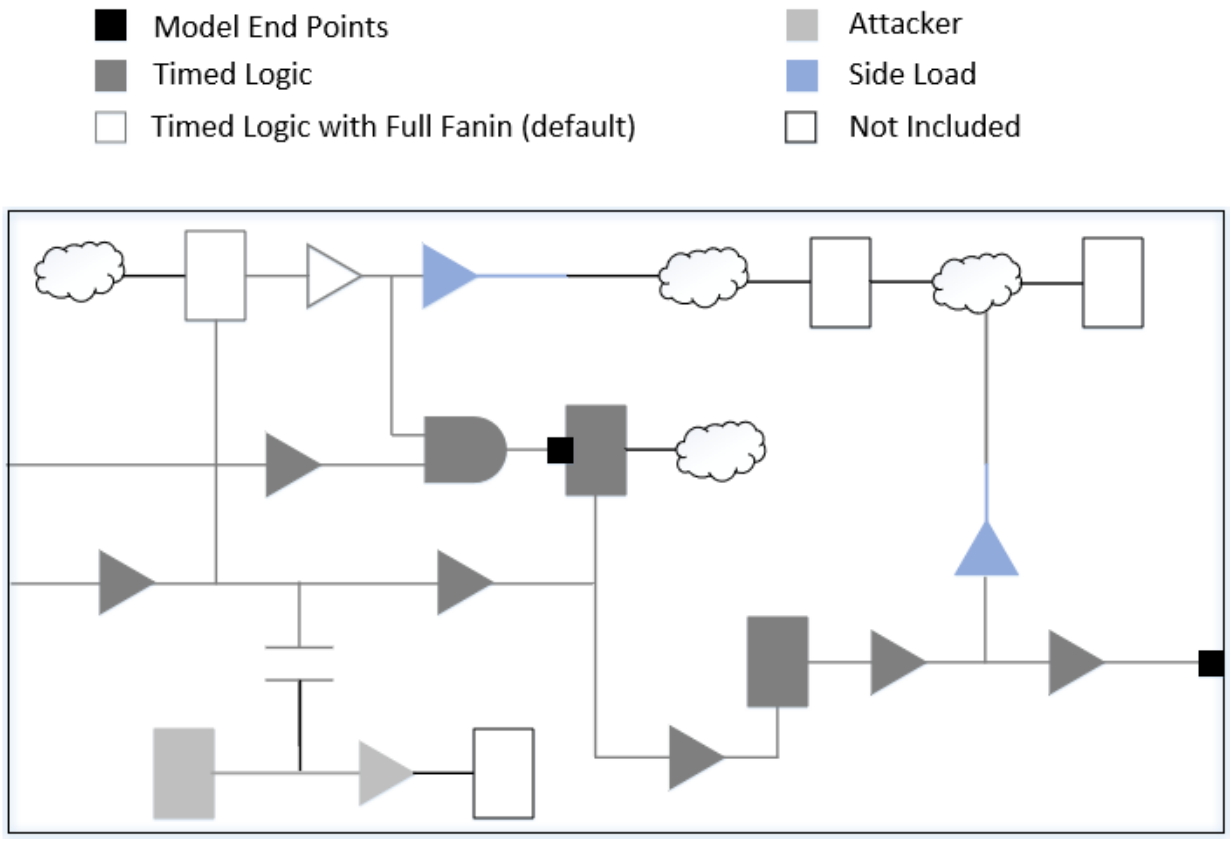
    The slew/arrival and the graph-based analysis (GBA) slack at an endpoint depends on its full fanin cone, therefore, the cone is also saved in the boundary model for an accurate top level timing analysis. The extended fanin logic is timed during the top level analysis.
  - Internal attackers

    Internal path nets, that are SI attackers to the interface paths nets, along with their slew and timing windows (TW) are preserved in the boundary model. All the leaf driver and receiver instances of such nets are also included. These attacker nets are not timed in the top level analysis but their SI impact, as per the stored slew and timing windows adjusted for clock latencies, is taken into account for better accuracy.
  - Side load receivers

    The receiver instances on the interface path nets that are not part of the interface logic are included.

The following figure shows an example of a netlist saved for a boundary model of a simple block.

Figure 9-16 Saved netlist for a boundary model



The netlist contains the following data:

- Parasitics for the nets in the model in the SPEF format.
- The SDC constraints for elements on block internal paths. These constraints are not used by the top-level timing but are used if a block scope is generated for the boundary modeled block in the top-level session.
- Timing context data. Several types of timing data are saved to improve the accuracy of the model. Some of these are:
  - Timing windows and slew data for non-interface attacker nets.
  - Clock definitions. The definitions of all the clocks used in the block-level timing is stored in the model, in order to automatically map block-level clocks to the top-level clocks. This mapping is required for timing window application.
  - Constant values on pins of sequential instances that are not part of a boundary path. These constants can affect timing results of sequential instances and are therefore

maintained in the boundary model. The extra constants on non-extended fanin pins are also maintained in the boundary model.

9.2.4 Clock Mapping in Top Level Run with Boundary Models

The context that gets saved in a boundary model is with respect to the clocks defined in the block run. The block level clocks, whose definition is also saved in the boundary model, are mapped to the top level clocks for the application of context data in the top run.

You can specify the mapping for some or all the clocks for each block instance by using a clock mapping file. You can specify a clock map file using the `read boundary model -clock_mapping_file` parameter.

The software attempts the auto clock mapping for clocks that do not have any user-specified mapping. The user-specified clock mapping settings have higher precedence over auto inferred clock mapping.

The auto clock mapping rules are given below:

- To map a clock ‘clkA’ defined at pin ‘pinA’ at a block:
  - The software will identify the source pin of clock clkA at block level, that is, pinA.
  - In the top level run with boundary model, the software will examine the clocks at pinA of the block, that is, at block/pinA.
    - Top level clock at block/pinA having waveform matching with clkA will be mapped to clkA.
    - If multiple clocks with matching waveforms are found, then the mapping of clkA is dropped due to ambiguity.
  - If no match is found, a check is made for the top-level clock with the same name (irrespective of clock source):
    - If a top level clock with name ‘clkA’ exists and has the same definition as block level ‘clkA’, then the mapping is successful.
    - If a top level clock with name ‘clkA’ exists and has a definition different from the block level ‘clkA’, then the mapping is ignored.
- Mapping virtual clocks:
  - Block virtual clocks are mapped only to the top virtual clocks.
  - Clocks with the same name/waveform will be mapped.

- Mapping generated clocks:
  - Block generated clocks will be mapped only with the top generated clocks, and cannot be mapped with top master clock.
  - Block master clock can be mapped with the top generated clocks.

The details of all the mapped and unmapped clocks are written in files named - blkname_clock_map.txt and blkName_unmapped_clocks.txt respectively, where blkName is the name of the block module. These files are written for every boundary modeled block. If a block is multiply instantiated at the top level, then the software writes mapped and unmapped clocks corresponding to each instance of a block.

If a block-level clock is not mapped to the top-level clock, then the timing windows (saved in the boundary model context), which reference the unmapped clock, are treated as unconstrained windows.

In addition to finding the corresponding clock, the timing window application applies a latency adjustment for a better correlation between the saved timing windows and the timing windows present in a full flat run. This latency adjustment is to account for the difference in clock arrival time between the block-level run and the top-level run. This is based on the difference in the arrival time between the matching clock phase on the block instance pin and the arrival time seen in the block-level run. This latency adjustment is made for each boundary model instance and clock, when there is a matching phase found. No latency adjustment is done where mapping is done by the clock name.

9.2.5 Recommendations for Generating Accurate Boundary Models

The timing correlation between a fully flat higher level run and a higher level run using one or more boundary models is expected to be close. However, the following factors can negatively affect the correlation:

- The timer settings between the lower-level timing run that generate the model and the higher-level run which loads the mode should be consistent.
- The SDC constraints applied in the lower-level timing run and the constraints that are applied to instances of the model in the higher level timing run should be consistent.
- Multiple SI passes: The timing windows in the boundary model context data are the timing windows used in the final SI delay calculation pass in the block level run. These timing windows will be used for every SI delay calculation pass in the top-level run. If there are multiple SI passes, the timing window convergence can differ between the full flat top-level run and the top-level run with the model. Hence, it is recommended to use two SI iterations in both the block and top runs for good correlation.

- Clock Mapping: A successful mapping of all the block clocks is necessary for performing timing correlation between the top flat vs top boundary model runs. If there are unmapped clocks, you can use the `set_db si_unconstrained_net_use_infinite_timing_window true` command setting for top runs for conservative correlation purpose. However, in some rare scenarios, an optimistic result might be observed while clocks are unmapped.~~`si_use_infinite_timing`~~
- Power Supply Voltage: If the power supply voltage setting is different in the block and the top CPF, then the cell delay and slew will change in two runs. This difference affects the slew and timing window calculation of the non-interface attacker nets. Due of this timing window difference, the top scope STA run can either get pessimistic or optimistic as compared to a full flat run.

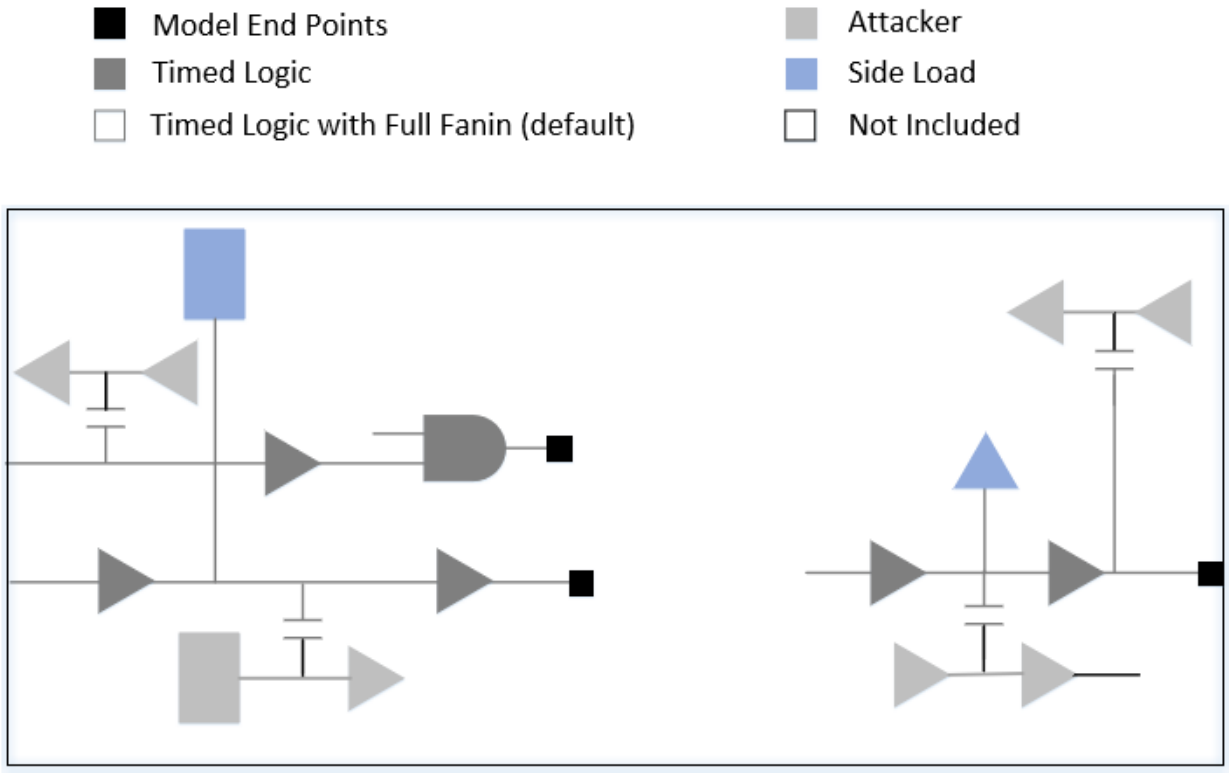
9.2.6 Glitch-Specific Boundary Models

To improve efficiency of glitch-only analysis, a glitch-specific model can be created using the `create_glitch_boundary_model` command.

For glitch-only analysis, the software will keep N-levels (default is 2 levels) of logic (controlled by the `create_glitch_boundary_model -max_stages` command parameter) from the boundary along with the RCs connected to any of those nets. If a multi-stage cell is encountered, then the noise will not propagate through the logic and multi-stage cells will be considered as path end or start points.

The following figure shows an example of a netlist saved for a boundary model of a simple block for a maximum of 2 levels of glitch stages.

Figure 9-17 Saved netlist for a boundary model with 2 levels of glitch stages



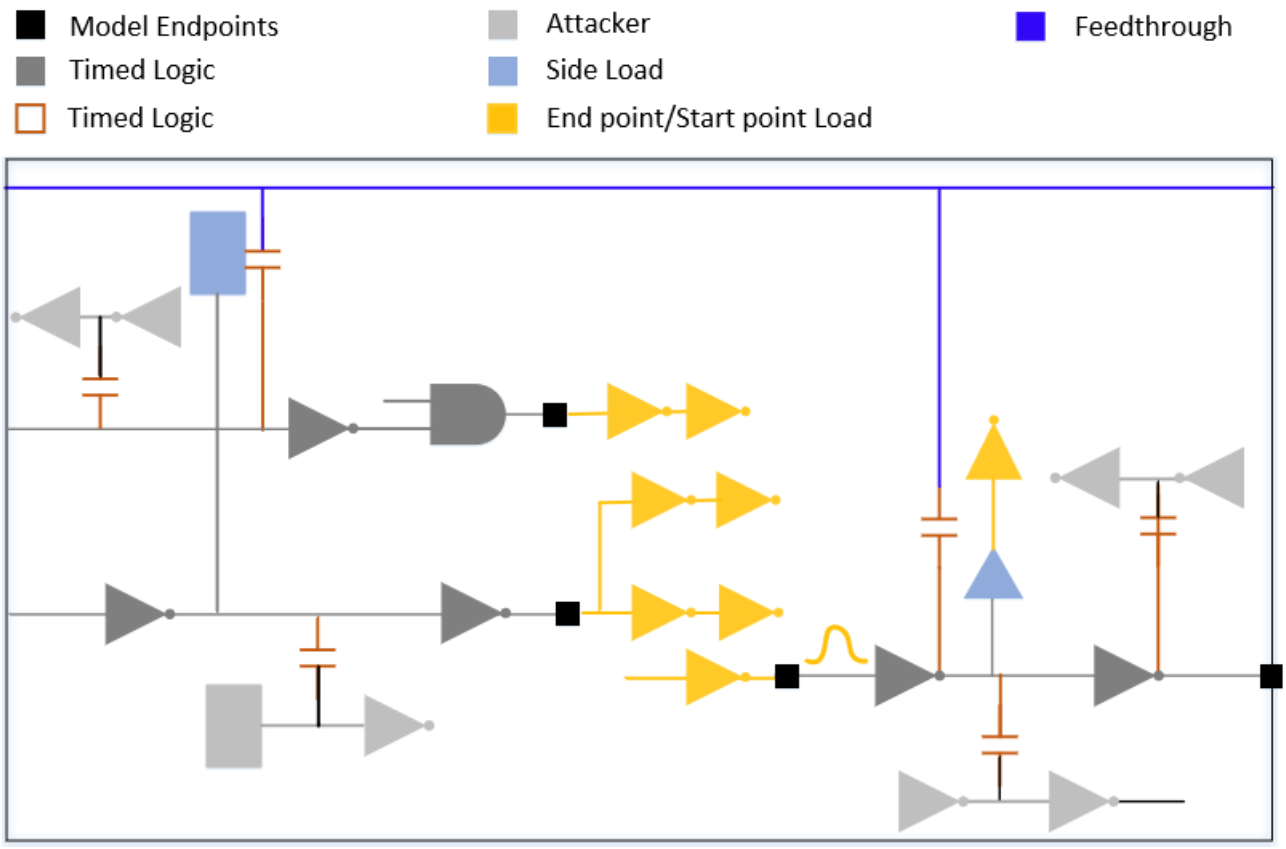
If a cell is encountered without any noise model, then it is treated as a single-stage cell for the purpose of logic level tracing. The next stage after the end point is also kept as load so that the slew/waveform can be put on the driver of the net connected to the start point, but is not analyzed when the block is used. The driver stage of the start point is kept as a driver and noise is injected if the cell is single-stage or a buffer. This logic is described below:

- The internal path nets that are SI attackers to the N-level path nets. All the leaf drivers and receiver instances of such nets are also included.
- The receiver instances on the N-level path nets that are not part of the interface logic are included. These side load receivers can occur in the clock network and on the paths from the register to the output port paths.
- The end points are saved to ensure proper loading, but they are not analyzed when the boundary model is used.
- The attacker driver and receiver are saved, but not analyzed when the boundary model is used.
- The analyzed noise on the drivers to the start points is saved and injected when the driver is a single-stage cell or buffer.

- The feedthrough is saved as part of the boundary model.

The following figure illustrates the logic described above:

Figure 9-18 Glitch-specific boundary model with feedthrough



The glitch correlation between a fully flat higher level run and a higher level run using one or more boundary models is expected to be close. However, the following are some of the factors that can negatively affect the correlation:

- Boundary slews at the block and the top level will not match perfectly, but the expectation from the boundary constraints is that they will be worst case. The differences in slews can cause differences in glitch results.
- User errors in the form of inconsistent SI settings between the block level and the top level.

Glitch Boundary Model Usage

The following command is used to generate a glitch boundary model:

- `create glitch boundary model`

Glitch Boundary Model Contents

The contents of a glitch boundary model are as follows:

- <block>.db: Contains the Verilog files that are equivalent to the block:
  - model_<#>.v.gz
- bound_libs.txt:
  - File containing the timing libraries used in the block.
- context: Context of the boundary model:
  - ct.gz
  - es.gz
  - et.gz
  - glitch.tcl.gz
  - timestamp
  - tw.gz
- libs
  - File containing all the timing libraries loaded from the Tcl to create the model.
- Partition: Contains the SPEF file and partition information:
  - SPEF file
  - endpoints.gz
  - endpoints2.gz
  - stoppoints2_wf.gz
  - Ignoredpins.gs

9.2.7 Boundary Models in MMMC Mode

The boundary model flow is MMMC-aware. The requirement is that the block run used to generate the boundary model should be CMMMC with the same (or larger) set of views as in the top CMMMC run.

The boundary model generated from the MMMC environment saves the SPEF data for all the active RC corners, the constrains for all active modes, and the context for all the active views.

9.2.8 Save-Restore of Hierarchical Analysis

A hierarchical analysis run with a boundary model can be saved and restored like a non-hierarchical run. In addition, boundary models can be generated from the restored sessions without performing a timing update.

9.3 Prototype Timing Modeling

9.3.1 Overview on page 553

9.3.2 Inputs and Outputs on page 553

9.3.3 Using the PTM Flow to Generate Dotlib Model on page 554

9.3.4 Handling of Bus Bits in PTM on page 558

9.3.1 Overview

The prototype timing model (PTM) feature enables you to define and generate a prototype timing model database, which contains preliminary information about the block, such as ports, delay arcs, timing check or constraints. This information is then used to generate a library (.lib), Verilog (.v), and What-If (.cmd) command files.

PTM models are useful for performing feasibility studies on a logical block. In addition, PTM can be used to generate preliminary timing information, which becomes the validation check point for early development, and evolves with design.

With PTM, you can easily create a block without providing many details, which would then provide preliminary timing information for it. This block can be replaced by a detailed timing model later in the design cycle to get more accurate timing information.

This chapter describes how to define and generate prototype timing models (PTM). It explains the inputs and outputs for PTM flow and the procedure to use PTM models.

9.3.2 Inputs and Outputs

- Inputs
- Outputs

Inputs

- Standard cell libraries or macros.
- A set of PTM commands to define the PTM model.

Outputs

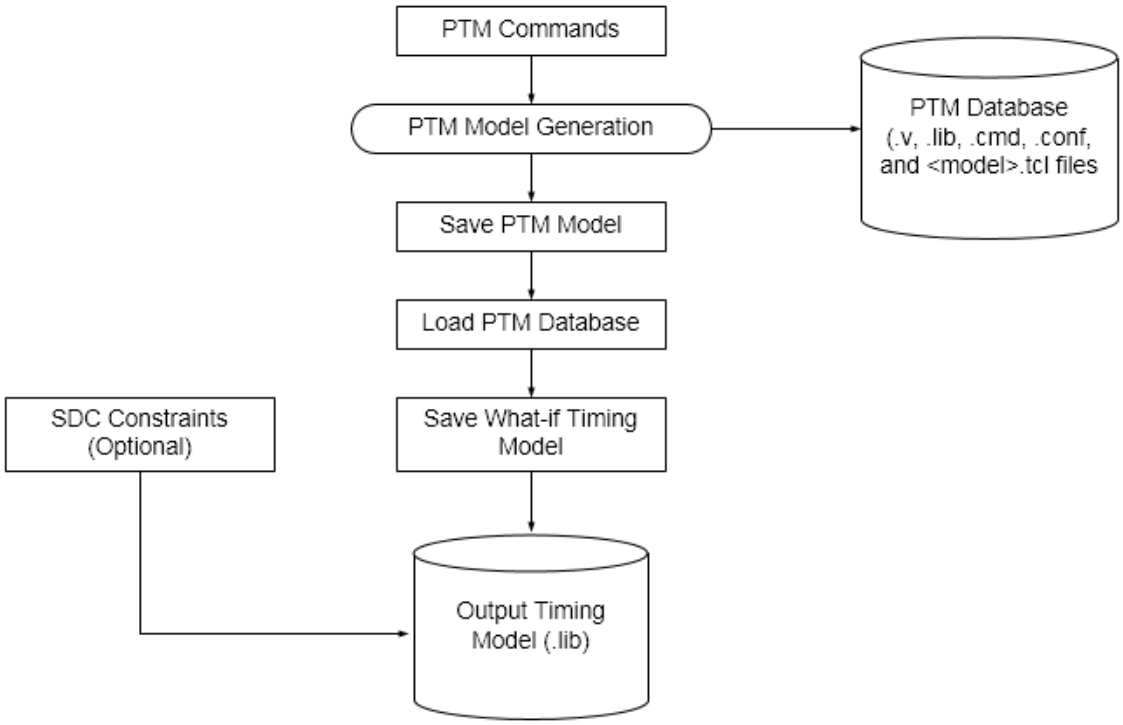
A PTM database that contains the following information:

- Verilog (.v) model: Contains the port definition with the instantiated PTM cell model.
- Library (.lib) model: Contains pin or bus definitions (direction and capacitance).
- What-If Command file (.cmd): Contains the mapped What-If commands.
- Configuration file (.conf): Contains information to load the generated library and Verilog models with the `read_db` command.
- Top-level Tcl script: Contains the commands to load the configuration file, source the What-If constraints, and write the timing model and assertions.

9.3.3 Using the PTM Flow to Generate Dotlib Model

The following diagram shows how the PTM commands are used to generate the PTM model database. This PTM model database is later used to save a What-If timing model, which can be used to generate a dotlib model that contains detailed timing information. You can optionally load the SDC constraints while generating the output dotlib model.

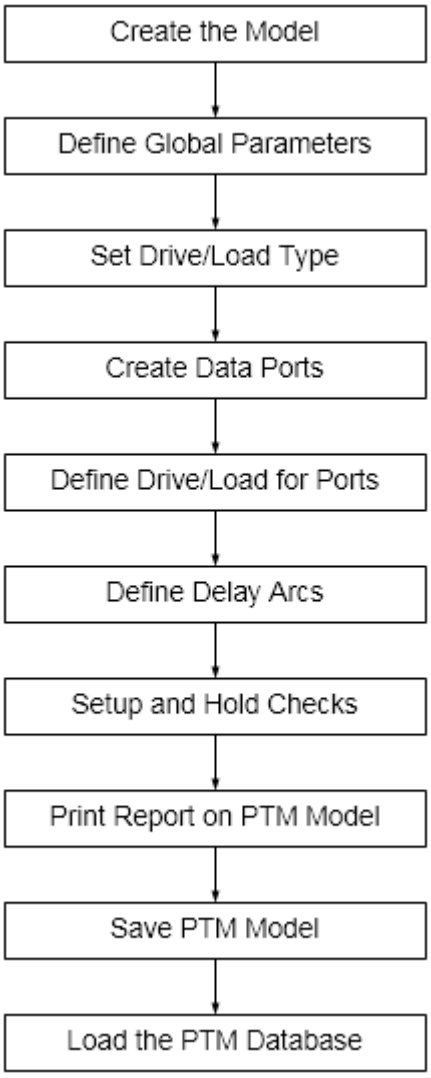
Figure 9-19 Flow Diagram



Command Flow

The following figure shows the PTM command flow for generating a dotlib model:

Figure 9-20 Sample PTM Model Flow



Generating a PTM Model

The following procedure shows how to define a PTM model and use it to generate a dotlib timing model, which can later be instantiated in a top-level design:

1. To create the PTM model, use the `create_ptm_model` command:

   ```
   create_ptm_model dma
   ```

   Note: All other PTM commands will work only after this command is executed.

2. To define global values for setup check arcs, hold check arcs, or sequential delay arcs across the PTM model, use the `set_ptm_global_parameter` command:

```
set_ptm_global_parameter -param setup -value 1.5
set_ptm_global_parameter -param hold -value 1.5
set_ptm_global_parameter -param clk_to_output -value 1.5
```

Note: The global value specified with this command is added to the arc specific delay value. As a result, the final arc's delay is calculated as a sum of the global parameter value and the arc delay specified using the create_ptm_constraint_arc or the create_ptm_delay_arc command.

3. To create drive types for specifying the drive strength equivalent to existing library cells, use the create_ptm_drive_type command:

   ```
   create_ptm_drive_type -lib_cell INVX1 stdDrive
   ```

4. To create a load type by assigning it a unique name, use the create_ptm_load_type command:

   ```
   create_ptm_load_type -lib_cell INVX4 stdLoad
   ```

   Typically, the load type represents a capacitance value from a library input or bidirectional pin. Use multiple `create_ptm_load_type` commands to create multiple load types for a PTM model.

5. To create PTM data ports for the current model, use the `create_ptm_port` command:

   ```
   create_ptm_port -type input data_in
   create_ptm_port -type output data_out
   create_ptm_port -type input data_test_1
   create_ptm_port -type inout data_test_2
   create_ptm_port -type input bcstop
   create_ptm_port -type input bdis1
   ```

6. To create a PTM clock port for the current model, use the `create_ptm_port` command:

   ```
   create_ptm_port -type clock clock_in
   ```

7. To set the drive for the output and bidirectional ports, use the set_ptm_port_drive command:

   ```
   set_ptm_port_drive -type stdDrive data_out
   ```

   The drive type specifies the type of library cell within the model as driver on the interface nets. You cannot specify two different drivers for two timing arcs ending at the same output port.

8. To define a load for the PTM model ports by specifying a capacitance value, use the set_ptm_port_load command:

   ```
   set_ptm_port_load -value 4.0 bdis1
   ```

9. To set the delay for combinational or sequential paths from input ports to the output ports, use the `create_ptm_delay_arc` command:

```
create_ptm_delay_arc -from clock_in -to data_out -edge rise -value 0.05
create_ptm_delay_arc -from data_test_1 -to data_test_2 -value 1.0
create_ptm_delay_arc -from data_test_2 -to data_out -value 1.5
```

Note: If you do not specify the drive type on the output port, a constant timing table is created, which signifies that the delay is constant for this arc. This means that the delay does not vary with respect to input slew and output capacitance.

10. To define the timing check at data input port with a reference clock port, use the create_ptm_constraint_arc command:

```
create_ptm_constraint_arc -setup -value 0.3 -from clock_in -to
data_in -edge rise
create_ptm_constraint_arc -setup -value 0.4 -from clock_in -to
data_in -edge fall
create_ptm_constraint_arc -hold -value 0.1 -from clock_in -to data_in -
edge rise
create_ptm_constraint_arc -hold -value 0.2 -from clock_in -to data_in -
edge fall
```

11. To generate a report on ports, arcs, and global delay parameter details for the current PTM model, use the report_ptm_model command:

```
report_ptm_model
```

12. To save the PTM model in dotlib library file format, use the save_ptm_model command:

```
save_ptm_model
```

13. To use the PTM database for generating a dotlib timing model, source the dma.tcl file:

```
source dma.tcl
```

14. To instantiate the model block in a top-level design, use the read_libs command:

```
read_lib dma.lib
```

This serves the purpose of validating the top-level timing information related to the instantiated block.

PTM Example

The following example provides a list of commands that can be used to generate a PTM model illustrated in the figure (*Sample PTM Block)* below:

```
# Start of model dma
create_ptm_model dma

# Define drive and load type
create_ptm_drive_type -lib_cell INVX1 stdDrive
create_ptm_load_type -lib_cell inv_hivt_4 stdLoad

# Create data ports
create_ptm_port -type input data_in
create_ptm_port -type output data_out
```

```
create_ptm_port -type input data_test_1
create_ptm_port -type inout data_test_2

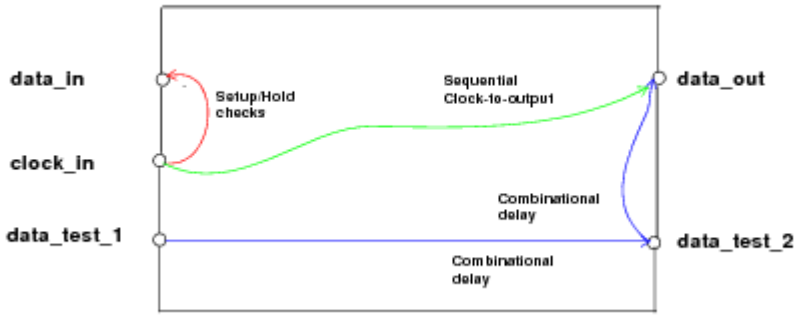
# Create clock ports
create_ptm_port -type clock clock_in

# Define drive for ports
set_ptm_port_drive -type stdDrive data_out
# Define load for ports
set_ptm_port_load -type stdload data_test_2
# Define delay arcs
# Data delays
create_ptm_delay_arc -from data_test_1 -to data_test_2 -value 1.0
create_ptm_delay_arc -from data_test_2 -to data_out -value 0.5
# Sequntial clock to output delay
create_ptm_delay_arc -from clock_in -to data_out -edge rise -value 0.05

# Setup and hold checks
create_ptm_constraint_arc -setup -value 0.3 -from clock_in -to data_in -edge rise
create_ptm_constraint_arc -setup -value 0.4 -from clock_in -to data_in -edge fall

create_ptm_constraint_arc -hold -value 0.1 -from clock_in -to data_in -edge rise
create_ptm_constraint_arc -hold -value 0.2 -from clock_in -to data_in -edge fall
```

Figure 9-21 Sample PTM Block



Handling of Bus Bits in PTM

The following examples provide a list of commands that can be used to handle bus bits in PTM:

Defining Bus Bits

To create a bus port along with bus members definition, use the `create_ptm_port` command. An integer specified with a bus name designates the number of bus bits, as shown below:

```
create_ptm_port -type input test_input[31:0]
```

Creating Arcs from/to Multiple Bits of a Bus

Delay ARC

Example1:

The example shows how to create delay arc on pin, `test_clk`, with respect to each bit of `test_input` bus:

```
create_ptm_delay_arc -from test_input[31:0] -to test_clk  -value 1.0
```

Example 2:

The example shows how to create delay arc on pin, `test_inout[7:0]`, with respect to each bus bit, `test_input[31:0]`:

```
create_ptm_delay_arc -from test_input[31:0] -to test_inout[7:0]  -value 5.0
```

This command creates 32 arcs on all 8 bus pins, `test_inout [7:0]`.

Example 3:

The example shows how to create delay arc on pin, `test_inout[7:0]`, with respect to selected bus bit:

```
create_ptm_delay_arc -from test_input[20:0] -to test_inout[7:0] -value 5.0
```

This command creates 21 arcs on all 8 bus pins, `test_inout [7:0]`.

Constraint ARC

Example 1:

The example shows how to create constraint arc on bus bits, `test_input[31:0]`, with respect to clock pin, `test_clk1`:

```
create_ptm_constraint_arc -setup -value 0.3 -from test_clk1 -to
test_input[31:0] -edge rise
```

Example 2:

The example shows how to create constraint arcs with different values for selected bits of bus, `test_input[31:0]`, with respect to clock pin, `test_clk1`:

```
create_ptm_constraint_arc -setup -value 0.3 -from test_clk1 -to
test_input[3:0] -edge rise
create_ptm_constraint_arc -setup -value 0.8 -from test_clk1 -to
test_input[7:4] -edge rise
```

Define DRV on Bus Bits

Set load on bus bits:

Example 1:

The example shows how to create a load type by assigning it a unique name using the `create_ptm_load_type` command:

```
create_ptm_load_type -lib_cell ABC -input_pin A load1
```

The example shows how to set the load type, `load1` on all the bus bits using the `set_ptm_port_load` command:

```
set_ptm_port_load -type load1 test_input[31:0]
```

Example 2:

The example shows how to set the different load type on specified bus bits using the `set_ptm_port_load` command:

```
create_ptm_load_type -lib_cell ABC -input_pin A load1
set_ptm_port_load -type load1 test_input[21:0]
create_ptm_load_type -lib_cell XYZ -input_pin A load2
set_ptm_port_load -type load2 test_input[31:22]
```

This will set `load1` on `0-21` bits and `load2` on `22-31` bits of the `test_input` bus.

Example 3:

The example shows how to define a load for ports by specifying a capacitance value using the `set_ptm_port_load` command:

```
set_ptm_port_load -value 4.0 test_input[31:0]
```

To set DRV on bus bits:

Example 1:

The example shows how to specify a value for the `max_transition`, `max_capacitance`, and `max_fanout` design constraints:

```
set_ptm_design_rule -max_transition 1.1 -port test_input[31:0]
set_ptm_design_rule  -max_capacitance 1.2 -port test_inout [7:0]
set_ptm_design_rule -max_fanout 4 -port test_inout[7:0]
```

Example 2:

The example shows how to specify a value for the `max_transition`, `max_capacitance`, and `max_fanout` using the `set_ptm_design_rule` command on specified bit:

```
set_ptm_design_rule -max_transition 1.5 -port test_input[21:0]
set_ptm_design_rule -max_fanout 5 -port test_inout[4:0]
set_ptm_design_rule -max_capacitance 1.8 -port test_inout[4:0]
```

9.4 Interface Logic Models

9.4.1 ILM Overview on page 563

9.4.2 Creating ILMs on page 564

9.4.3 Specifying ILM Directories at the Top Level on page 567

9.4.4 ILMs Supported in MMMC Analysis on page 568

9.4.5 ILM Validation Flow on page 568

9.4.6 Use of SDF, SDC, and XTWF in ILM Flow on page 570

9.4.7 Creating XILM on page 574

9.4.8 Using ILM/XILM for Hierarchical Analysis on page 574

9.4.1 ILM Overview

Models are compact and accurate representations of timing characteristics of a block.

Interface Logic Model (ILM) is a structural representation of a block, specifically a subset of the block's structure including instances along the I/O timing paths, clock-tree instances, and instances or net coupling affecting the signal integrity (SI) on I/O timing paths. It is a compact and accurate representation of timing characteristics of a block. Instead of using a blackbox at the top level, you create an ILM at the block level and use it as you would use a blackbox.

The ILM is not only a netlist; it comprises at least three files: verilog netlist, SPEF, and SDC constraints. The SI ILM logic model, or XILM additionally contains capacitances and other logic needed to model SI effects. An XILM would contain netlist, SPEF, and SDCs, and also a timing window file (.xtwf).

The advantages of using ILMs are as follows:

- More accurate analysis than a black box flow
  - More SI-aware than combined `.lib` or `.cdb` approach
  - Can model clock generator inside block
  - More accurate timing and SI reduces the number of design iterations to close timing and SI
- No need to characterize blocks
  - Works on a actual design data
- Can be used in the initial prototyping stage for very big designs when loading full design data is not feasible.
  - Allows you to modify only top-level data
  - Fully preserves implemented partitions
- Uses the original constraint file for top-level analysis
  - No abstraction for timing exceptions

Note: The ILM is not supported for Signoff in Tempus.

9.4.2 Creating ILMs

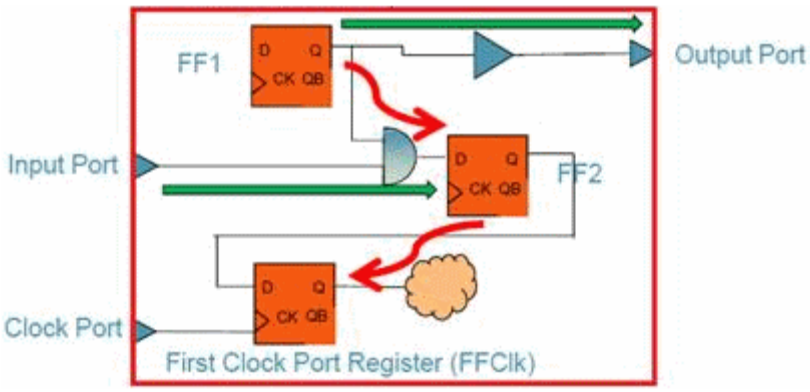
In the hierarchical design flow, you load a detailed block-level implementation of a block and its parasitic data, then specify the `write_ilm` command to create an ILM for the block. This command creates the specified directory containing ILM files.

Note: An ILM created for an incomplete block is not as accurate as an ILM created for a complete block. Always use ILMs for complete blocks to complete the top-level design.

The software generates ILM data for timing and signal integrity (SI):

- ILM data for timing
  The model contains the netlist of the circuitry leading from the I/O ports to interface sequential instances (that is, registers or latches), and from interface sequential instances to I/O ports. The clock tree leading to the interface registers is preserved.
- Internal register-to-register paths, if internal logic is not part of the interface path. However, in special cases some internal paths will be kept, as shown in the following example:

Figure 9-22 Internal Paths



- Internal paths
  Internal paths controlled by different clock, or clocks connected to the ILM module through different ports. If the logic between the I/O ports is pure combinational, it is preserved in an ILM.
- ILM data for SI
  The model includes all of the above, plus attacker drivers or nets which affect I/O paths. It also includes the timing window files in the ILM model directory.

Example ILM Creation

The following command generates the ILM ignoring certain ports and including certain objects in the `BlockB` directory. It also writes out the ILM SDF file under the `BlockB` directory.

```
write_ilm -writeSDF -ignore_ports $ignorePorts -include_objects $includeObjects -
dir BlockB
```

Sample Summary Report

The following is a sample summary report generated after running the `write_ilm` command:

```
-------------------------------------------------------------
                  createInterfaceLogic Summary
-------------------------------------------------------------
Model            Reduced Instances           Reduced Registers
-------------------------------------------------------------
ilm_data
+
setup                4/16 (25%)                 0/3 (0%)
hold                 4/16 (25%)                 0/3 (0%)
setupHold            4/16 (25%)                 0/3 (0%)
-------------------------------------------------------------
si_data

setup               3/16 (18%)                  0/3 (0%)
hold                3/16 (18%)                  0/3 (0%)
setupHold           3/16 (18%)                  0/3 (0%)
-------------------------------------------------------------
```

In this report, the reduction ratio in the `ilm_data` model is 25 percent which means that 4 out the total 16 instances for this block have been eliminated.

Note: You can run the `set_db -ilm_keep_high_fanout_ports false` command for improving the reduction ratio.

Preserving Selected Instances in ILMs

The `write_ilm` command can preserve specified instances and/or nets. The `-include_objects` parameter of the `write_ilm` command accepts a collection of instances and/or nets. All the specified instances and nets are preserved in the generated ILM.

Creating ILMs for Shared Modules

You can use the same sub-block module in different ILM blocks, enabling reuse of versatile modules. The `write_ilm` command considers constant propagate, so that only the enabled

parts of a module are considered when creating ILMs for the reused modules. Because the Tempus database cannot handle the same module name in different circuits, the software automatically modifies the module names with the following rule:

```
topModuleName+timestamp+$+moduleName
```

As an example, one ILM block (ModuleA) uses an ALU module (ALU) as an unsigned ALU, and a second block (ModuleB) uses the ALU as a signed ALU. You can change the input signal to use the ALU differently, setting one ALU as sign enabled and the other to off. When you run the write_ilm command, the software considers only the enabled parts of the ALU when creating ILMs for ModuleA and ModuleB. The software also ensures that the name of the ALU module in ModuleA and the name of the ALU module in ModuleB are different.

Creating ILMs Without Using Tempus Database

If you do not have Tempus database for an implemented block but have a Verilog netlist, constraints, and SPEF and TWF for that block, then use the `create_ilm_from_files` command to store data in the ILM/XILM format.

The following example shows the `create_ilm_from_files` command in case of a MMMC design:

```
create_ilm_from_files -si -out_dir block_A.ILM -cell block_A -mmmc -verilog_file
myfile.v
create_ilm_from_files -si -out_dir block_A.ILM -cell block_A -mmmc -incremental\
-spef max.spef.gz -rc_corner rcMax
create_ilm_from_files -si -out_dir block_A.ILM -cell block_A -mmmc -incremental\
-spef typ.spef.gz -rc_corner rcTyp
create_ilm_from_files -si -out_dir block_A.ILM -cell block_A -mmmc -incremental\
-spef min.spef.gz -rc_corner rcMin
create_ilm_from_files -si -out_dir block_A.ILM -cell block_A -mmmc -incremental\
-sdc_file funMaxMax.sdc -view_name funct-devSlow-rcMax
create_ilm_from_files -si -out_dir block_A.ILM -cell block_A -mmmc -incremental\
-sdc_file funMaxTyp.sdc -view_name funct-devSlow-rcTyp
create_ilm_from_files -si -out_dir block_A.ILM -cell block_A -mmmc -incremental\
-sdc_file tstMaxMax.sdc -view_name test-devSlow-rcMax
create_ilm_from_files -si -out_dir block_A.ILM -cell block_A -mmmc -incremental\
-sdc_file funMinMin.sdc -view_name funct-devFast-rcMin
create_ilm_from_files -si -out_dir block_A.ILM -cell block_A -mmmc -incremental\
-sdc_file tstMinMin.sdc -view_name test-devFast-rcMin
```

The following example shows the `create_ilm_from_files` command in case of a non-MMMC design:

```
create_ilm_from_files -out_dir test -verilog_file \ ILM/ilm_setup/ilm_data/\
BlockA_0/BlockA_0_postRoute.v.gz -cell BlockA_0 -spef_file \ ILM/ilm_setup/
ilm_data/BlockA_0\ BlockA_0_postRoute.spef.gz -\ sdc\ILM/ilm_setup/ilm_data/
BlockA_0\ BlockA_0_postRoute.core.sdc.gz -si \
```

```
-overwrite -si_timing_window_file ILM/ilm_setup/si_ilm_data/BlockA_0/
BlockA_0_SI.xtwf.gz
```

9.4.3 Specifying ILM Directories at the Top Level

You can use the set_ilm command to use the ILM data for a block at the top partition level rather than using the default dotlib model. You can run `set_ilm` multiple times in the same session. Each time you run this command, the software overwrites the previous setting for a block. If master/clones exist in a design, the cell name will have the name of the master partition.

You can use the reset_ilm command to revert to using the dotlib model for a block.

Note: You must use the `set_ilm` (or `reset_ilm`) command before specifying read_ilm. This is because the ILM settings cannot be changed after using `read_ilm`.

Example Top-Level Signoff Flow with ILMs

1. Before starting the Tempus tool, prepare the top-level Verilog file, if needed.

   You need the following in the top-level directory:

   - A Verilog netlist that includes dummy modules for the blocks (ILM or Liberty) in the design
   - A view definition file if you have a MMMC design. Create or load a view definition file that contains the following:

     ```
     set_analysis_view -setup {mode1_slowCorner} -hold {mode1_f astCorner}
     ```

2. Start a Tempus session from the top-level module directory within the directory where the partitions are saved.
3. Load the config file, including the top-level netlist, libraries, top-level constraint file, and ILM directory name.

   ```
   set_ilm -cell block_A -in_dir ../block_A/block_A.ILM
   set_ilm -cell block_B -in_dir ../block_B/block_B.ILM
   read_design filename
   ```

4. Run read_spef to load the top-level Standard Parasitic Extraction File (SPEF).
5. Run the `read_ilm` command.
6. Run the report_timing command to perform timing analysis.

9.4.4 ILMs Supported in MMMC Analysis

Cadence strongly recommends that you use ILMs in the MMMC mode. If you have a non-MMMC design, create and load a view definition file that contains the following:

```
set_analysis_view -setup {mode1_slowCorner} -hold {mode1_fastCorner}
```

The MMMC analysis for designs including ILMs is identical to MMMC analysis for black box designs except for the following considerations:

1. Views, modes, and corners at the top and partition levels must have same names.
2. When you use `create_constraint_mode` or `update_constraint_mode` to specify constraints for MMMC, you must specify the ILM constraints using the `-ilm_sdc_files` parameter (that is, timing in the presence of ILMs get constraints from the `-ilm_sdc_files` parameter, not the `-sdc_files` parameter). The SDC files specified with the `-ilm_sdc_files` parameter should be the constraint file of the full-chip flat netlist, where it allows referencing nets or pins internal to the ILM model.

Note: In the current ILM flow, the SDC constraints (originally specified against the complete flat netlist for the design) that reference parts of the design that were pruned cause warnings and errors during constraint loading. In this release, you can set the temporary `timing_suppress_ilm_constraint_mismatches` attribute to `true` to suppress all error and warning messages related to the unfound objects. Note that this command might also suppress error messages that might be of use (that is, where the top-level pins or nets or instances cannot be found).

9.4.5 ILM Validation Flow

The ILM validation flow can be used to validate a model at the block level. For example, an ILM netlist and the .sdc file can be read in a separate Tempus session and timing analysis can be run on all paths. Then, the results can be compared against timing for the same path during full-block implementation.

Use the following steps to run the automated ILM validation flow:

1. Load a block level design.
2. Load the SPEF file.
3. Run the following command:

   ```
   write_ilm -dir ilm_dir -validation_dir validation_dir
   ```

   Two directories are created - one for saving the ILM data and the second for saving the validation information. The validation directory contains following files:

Table 9-1 List of Files in the Validation Directory

File	Description
init.tcl	Used to initialize the ILM data.
nets.tcl	Used to get the list of net names in which only the real ILM nets are included and other nets like attacker nets are excluded.
full.tcl	Used to run timing analysis on the original netlist using the `report_timing_model` command.
xilm.tcl	Used to run timing analysis on the generated ILM data using the `report_timing_model` command.
rpt.tcl	Compares the output of original netlist with the output of ILM data using the `check_timing_model` command.
validation.src	Used to run the ILM validation flow by sourcing the Tcl files automatically.

4. Switch to the validation directory.
5. Source the validation.src file.
   After running the ILM validation flow, the validation.sum file is generated. This file provides the summary of the validation checks.

Sample ILM Validation Summary Report

```
#-------------------------------------------------------------------------
#                      Tolerance    FAIL/PASS   Ave          Max
#                      (abs,per)                AbsDiff      AbsDiff
#-------------------------------------------------------------------------

ILM   SETUP  Slack    (0.050 , 2%)    0/6       0.000         0.000  #
             PinCap   (0.050 , 2%)    0/5       0.000         0.00   #
             Tran     (0.050 , 2%)    0/5       0.000         0.000  #
             DRV      (     , 2%)     0/5        --           NA     #

ILM   HOLD   Slack    (0.050 , 2%)    0/0       NA            NA     #
             PinCap   (0.050 , 2%)    0/5       0.000         0.000  #
             Tran     (0.050 , 2%)    0/5       0.000         0.000  #
             DRV      (     , 2%)     0/5        --           NA     #

XILM   SETUP Slack    (0.100 , 3%)    0/6       0.000         0.000  #
             PinCap   (0.100 , 3%)    0/5       0.000         0.000  #
             Tran     (0.100 , 3%)    0/5       0.000         0.000  #
             DRV      (     , 3%)     0/5        --           NA     #

XILM   HOLD  Slack    (0.100 , 3%)    0/0       NA            NA     #
             PinCap   (0.100 , 3%)    0/5       0.000         0.000  #
             Tran     (0.100 , 3%)    0/5       0.000         0.000  #
```

```
             DRV      (     , 3%)     0/5       --             NA     #
#-------------------------------------------------------------------------
# ILM/XILM Validation PASS
```

Note:

- When write_ilm is specified, all the views are generated for multi-corner, multi-mode (MMMC) analysis.
- The automated ILM validation performs both ILM/XILM validation depending upon the data available.

Interactive Use of ILMs

The Global Timing Debugger (GTD) also requires the design to be in a flattened state. The GTD displays rows with instances or nets as grayed out, which are internal to ILM.

9.4.6 Use of SDF, SDC, and XTWF in ILM Flow

- **High-Level view of the ILM-based flow**
  The ILM flow falls into two essential flows that are placed in a bottom-up/top-down fashion. The two flows are done separately, and in separate sessions.
  - ILM generation flow, usually done bottom-up.
  - ILM analysis flow, usually done top-down.
    For example, a hierarchical design made up of BlockA and BlockB sub-blocks.
  - In the ILM generation flow, each sub-block has to be implemented as an ILM (write_ilm) after timing and SI analysis sign-off.
  - In the ILM analysis flow, the whole design is stitched together by reading the ILM models for BlockA and BlockB, top level netlist, top level SPEF, top level TWF and chip level SDC constraints to perform top level timing and SI analysis. This usually takes place in a separate session.

Use of XTWF Generated in ILM

The write_ilm command creates a directory structure containing the ILM data for timing and SI analysis as described below:

The information is saved in the following directories:

- For Timing:

```
$dir/ilm_data
*.v
*.sdc
*.spef
```

- For SI:

```
$dir/si_ilm_data
*.v
*.sdc
*.spef
*.xtwf
```

The XTWF is a timing file that contains timing window (the earliest and the latest possible arrival times that a signal may arrive on a net or a pin) for attacker nets. Tempus does use the XTWF for SI analysis in the ILM flow. The TWF file is usually used in Tempus if you are performing timing analysis in third-party tool and you want to bring the design into Tempus for SI analysis.

The ILM model is self-contained with all the needed information for both Timing and SI analysis which is suitable for the ILM bottom-up generation/top down analysis flow. See example below:

Example 1: Block based ILM generation for a sub-module called blockA

```
read_libs "$libDir/worst.lib"
read_libs -cdb "$libDir/worst.cdB"
read_netlist "$designDir/blockA.v" -top blockA
init_design
read_sdc "$designDir/blockA.sdc"
set_power_rail -global_voltage 1.65
read_spef "$designDir/blockA.spef"
set_db -delaycal_enable_si true
report_timing
write_sdf blockA.sdf
write_ilm -dir blockA
```

In a new session (when the hierarchical design is all stitched together), you read the ILMs of the sub-blocks to perform top-level STA and SI analysis sign-off.

Example 2: Top-Level ILM based flow of design made up of two sub-module blockA and blockB

For SI Analysis:

```
read_libs, read_libs  -cdb …
```

```
read_netlist top.v -top TOP

set_ilm -cell blockA -dir dir1
set_ilm -cell blockB -dir dir2

init_design
read_spef top.spef
read_sdc top.sdc
read_ilm
set_db -delaycal_enable_si true
report_timing
write_sdf
```

For Timing Analysis:

```
read_libs
read_netlist top.v -top TOP

set_ilm -cell blockA -dir dir1
set_ilm -cell blockB -dir dir2

init_design
read_spef top.spef
read_sdc top.sdc
read_ilm
report_timing
…
```

ILM Models and SDC-Generated Constraints

The SDC files generated within the ILM directories do not reflect a complete normal set of constraints. You will notice that only `set_annotated_transition` and `set_case_analysis` constraints are written in the SDC.

The SDC constraints found in the ILM directories are primarily used to save the input slew information for loop back path between interface logic and non-interface logic. This feature provides higher accuracy for both Timing/SI analysis using the ILM/XILM flow in Innovus/ Tempus. Here is a scenario describing how the slews of reduced nets are stored for better accuracy of timing and SI:

An ILM instance inst1 has following two input pins:

- **Input pin1**: connects with ILM path, its input slew will be calculated and propagated from an ILM path.
- **Input pin2**: connects with non-ILM path which is not kept in ILM netlist. This often happens in a loop-back path. The path is reduced in which as a result its pin becomes dangling but its input slew information is needed to calculate the delay of the inst1. In this case, Tempus uses the slew kept in ILM/XILM .sdc file instead of the default input slew which improves the ILM analysis accuracy.

To perform timing analysis using ILM data files, you will need to use the chip level SDC constraints and the generated SDC constrains for the ILM block. The idea here is that these constraints are giving high accuracy while not colliding with top-level SDC constraints during ILM top down analysis flow.

Example:

Generating ILM for BlockA:

```
read_libs tech.lib
read_netlist blockA.v -top blockA
init_design
read_sdc blockA.sdc
read_spef blockA.spef
write_ilm -dir blockA
report_timing > sta1.rpt
```

Now to use the ILM data for timing analysis and suppose the file names in the ILM directory are: ilm.v, ilm.spef, ilm.sdc, ilm.xtwf.

Simply read the original SDC and the SDC generated for the ILM.

Example:

```
read_libs tech.lib
read_netlist ilm.v -top blockA
init_design
read_sdc ilm.sdc
read_sdc blockA.sdc *
read_spef ilm.spef
report_timing > sta2.rpt
```

The timing report of the ILM interface paths should be consistent with the timing report of original full netlist (sta1.rpt should match sta2.rpt). This is usually done for validation purposes and not a real usage of the ILM flow.

If an ILM is read back to represent a sub-module in a hierarchical design, then to perform timing analysis, you will need to load the top level original constraints only. The timing report of all the top nets and interface paths should be consistent with original full flat netlist. This is the real usage of ILM as it reflects accurate timing and SI information derived from top level constraints.

9.4.7 Creating XILM

In Tempus, ILMs and XILMs are created using the `write_ilm` command.

To create an ILM or XILM for a block in Tempus, you must be at that block's hierarchy level; you cannot write a block ILM from the top-chip level of hierarchy. To provide the data needed for the ILM, you need the block's netlist, timing libraries, SPEF annotation, and SDC constraints.

In the following example, an XILM is created for a sub-block, named blockA:

```
read_libs "$libDir/worst.lib"
read_libs -cdb "$libDir/worst.cdB"
read_netlist "$designDir/blockA.v" -top blockA
init_design
read_sdc "$designDir/blockA.sdc"
set_power_rail -global_voltage 1.65
read_spef "$designDir/blockA.spef"
write_ilm -dir blockA
set_db -delaycal_enable_si true
report_timing
write_sdf blockA.sdf
```

When ILM is created using the `write_ilm` command, a directory named blockA will be created with the MMMC sub-directory. Within this directory you will see another level of sub-directories, named `ilm_data` and `si_ilm_data`, depending upon the `-modelType` parameter settings.

9.4.8 Using ILM/XILM for Hierarchical Analysis

The Tempus command for loading an ILM or XILM is `read ilm`. To use one or more ILMs, use the `read_ilm` command just once. If there are multiple ILM blocks, specify them in a list, as shown in the example below. The `-cell` and `-dir` parameters accept Tcl lists, but note that the order of the cells and directories must match, for example:

```
set_ilm -cell cell1 -dir dir1
set_ilm -cell cell2 -dir dir2
read_libs
```

```
read_netlist top.v -top TOP
set_db init_read_netlist_allow_undefined_cells false
read_spef top.spef
read_ilm
report_timing
```

You must run `read_ilm` after `read_spef`, and before `report_timing`. If you get an error such as:

```
**ERROR: (xxxILM-334): Specified ILM blockA has non-empty module definition; no ILM
created for it.
```

You will need to remove gates, assign statements, and so on from the `blockA` module in the top verilog netlist, because an ILM cannot overwrite the existing logic. The module, if it exists in the verilog netlist, must be empty.

```
create_constraint_mode -name mymode -sdc_files {file1_top_level.sdc
file2_top_level.sdc ...} \
-ilm_sdc_files {file1_chip_level.sdc file2_chip_level.sdc ...}
```

If ILM constraint files (at the chip level SDC) are not specified with the `create_constraint_mode` command (in the viewdefinition.tcl while reading the design), then these can be read using the `update_constraint_mode` command (after the `set_ilm` command is issued), as shown in the following example:

```
update_constraint_mode –name mymode \
-ilm_sdc_files {file1_chip_level.sdc file2_chip_level.sdc ...}
```

10

Low Power Flows

10.1 Low Power Overview on page 577

10.2 Low Power Flow on page 577

- Low Power Flow for CPF on page 577
- Low Power Flow for PGV Flow on page 580

10.3 PGV Flows on page 582

- PGV SMSC Flow on page 582
- PGV MMMC Flow on page 585

10.4 Importing the Design in Low Power Flow on page 588

10.5 Non-MMMC Flow Settings in Low Power Flow on page 588

10.1 Low Power Overview

Along with capturing the design and technology-related power constraints, the Common Power Format (CPF) defines timing analysis views based on a combination of different operating corners, power modes, and timing library sets. CPF does not contain RC corner information. To update the RC corners for each timing view, you must use the Tempus command to define the RC corners.

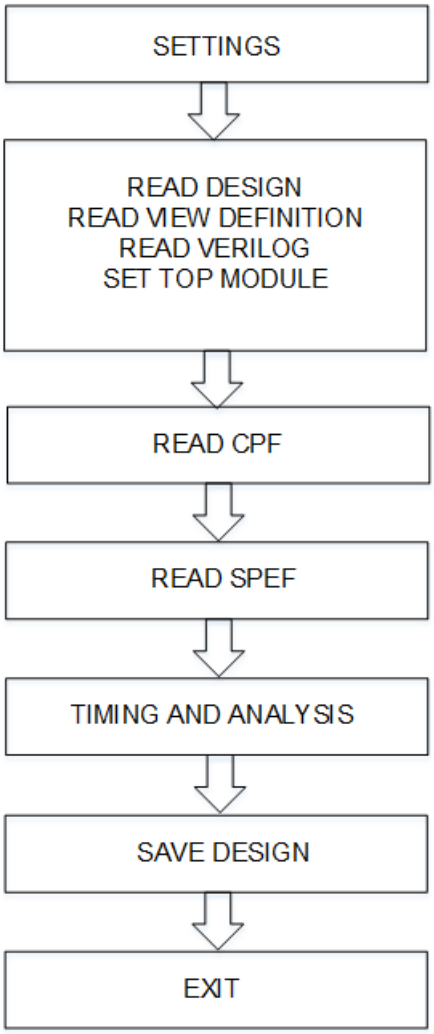
10.2 Low Power Flow

The Low Power flow is described below for CPF and PG Verilog (PGV):

Low Power Flow for CPF

The below flow shows the steps for setting-up Tempus using the CPF file:

Figure 10-1 Low Power Flow for CPF



Sample CPF File Script

You can use the following settings for Tempus using the CPF file:

1. Specify the number of CPUs to enable multi-threading:

   ```
   set_multi_cpu_usage -local_cpu n
   ```

2. Load the design data:

   ```
   read_mmmc <abc. tel>
   ```

3. Load the libraries:

   ```
   create_library_set
   ```

4. Create RC and Delay calculation corners:

   ```
   create_rc_corner
   create_delay_corner
   ```

5. Group SDC constraints files

   ```
   create_constraint_mode
   ```

6. Create an analysis view object that associates a delay calculation corner with a constraint mode.

   ```
   create_analysis_view
   ```

7. Set the analysis views used for setup and hold analysis:

   ```
   set_analysis_view
   ```

8. Schedule reading of structural gate-level Verilog netlist:

   ```
   read_verilog <abc.v>
   ```

9. Set the top module name, and checks for consistency between Verilog netlist and timing libraries:

   ```
   set_top_module <abc>
   ```

10. Specify the CPF file, which contains the power domain definitions:

    ```
    read_power_intent -cpf <xyz.cpf>
    ```

11. Load the cell parasitics:

    ```
    read_spef -rc_corner
    ```

12. Run a full timing update for the current design:

    ```
    update_timing -full
    ```

13. Generate a report of all defined views in the design:

    ```
    report_analysis_views
    ```

14. Specify the instance(s) for which the report is to be printed.

    ```
    report_instance_library <instance>
    ```

15. Report the voltage of the cell or net timing arc.

    ```
    report_delay_calculation -voltage
    ```

16. Generate the timing report:

    ```
    report_timing
    ```

17. Save the design:

    ```
    save_design -overwrite
    ```

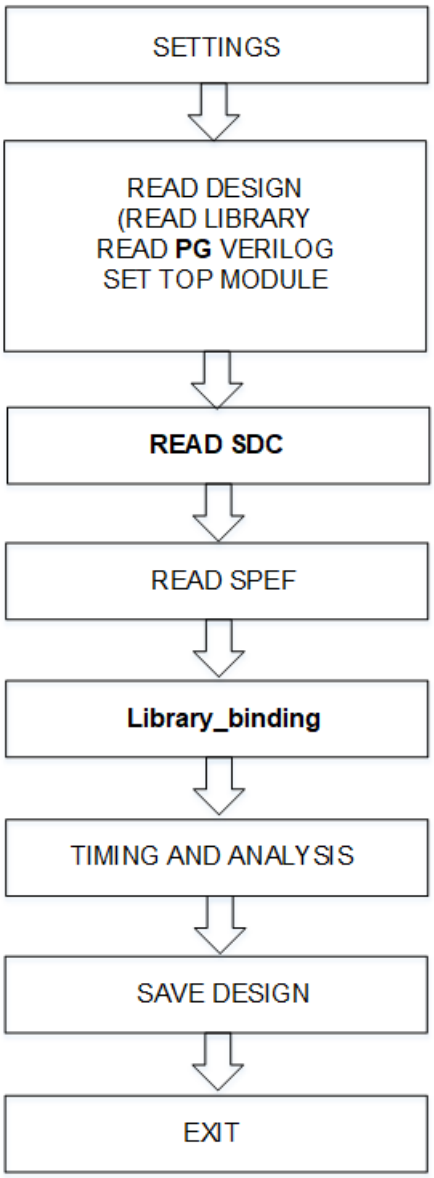
18. Exit the design:

```
exit
```

Low Power Flow for PGV Flow

The following figure shows the steps for setting-up Tempus using the PGV flow:

Figure 10-2 Low Power Flow for PGV



Sample PGV Flow Script

You can use the following settings for Tempus using the PGV flow:

1. Specify the number of CPUs to enable multi-threading:

   ```
   set_multi_cpu_usage -localCpu n
   ```

2. Read library file.

   ```
   read_libs < >
   ```

3. Schedule reading of structural gate-level Verilog netlist:

   ```
   read_verilog <PG verilog.v>
   ```

4. Set the top module name, and check for consistency between the Verilog netlist and timing libraries:

   ```
   set_top_module <abc>
   ```

5. Load the timing constraints file:

   ```
   read_sdc < >
   ```

6. Load the cell parasitics:

   ```
   read_spef -rc_corner
   ```

7. Initiate power intent inference and library binding:

   ```
   set_lib_binding_by_voltage -power_net_voltages
   ```

8. Run a full timing update for the current design:

   ```
   update_timing -full
   ```

9. Generate a report of all defined views in the design.

   ```
   report_analysis_views
   ```

10. Report library related information for an instance:

    ```
    report_instance_library <instance>
    ```

11. Report the voltage of the cell or net timing arc

    ```
    report_delay_calculation -voltage
    ```

12. Generate the timing report:

    ```
    report_timing
    ```

13. Overwrite the previously saved session with a new session:

    ```
    write_db -overwrite
    ```

14. Exit the design:

    ```
    exit
    ```

10.3 PGV Flows

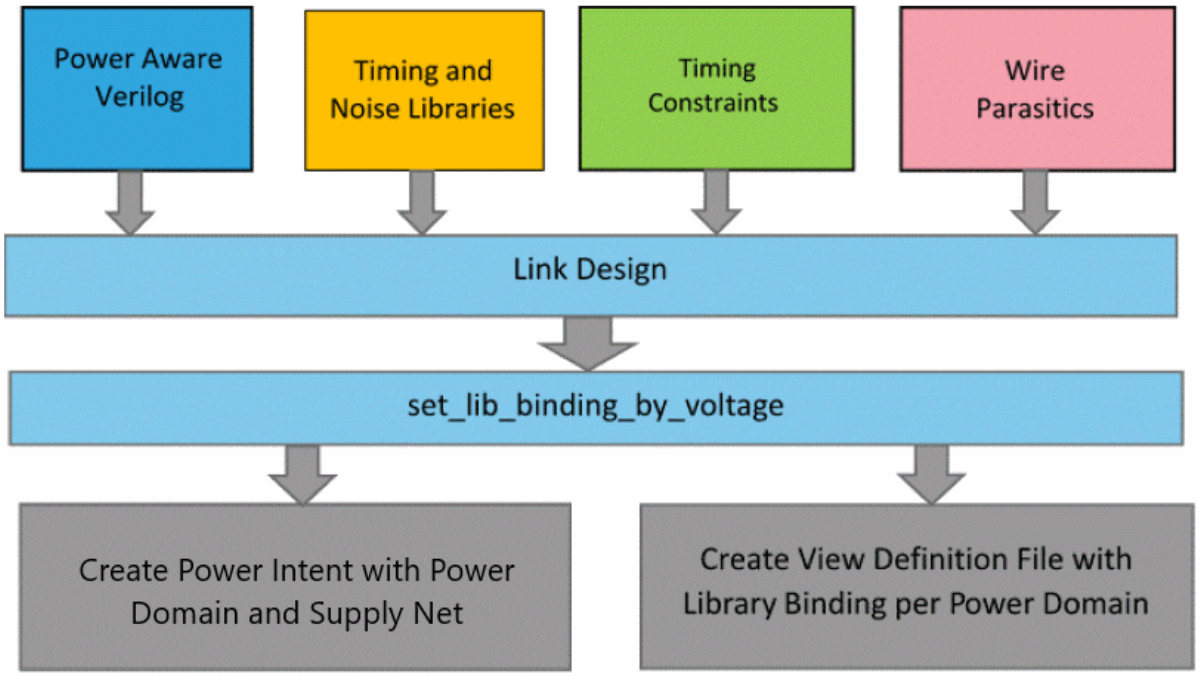
The PGV flows in Tempus can be categorized as below, based on the various timing analysis practices:

- PGV SMSC Flow (without MMMC)
- PGV MMMC Flow

PGV SMSC Flow

In the PGV SMSC flow, timing analysis can be performed in a separate session for each view (SMSC flow). The following figure represents the PGV SMSC flow:

Figure 10-3 PGV SMSC Flow



In the PGV SMSC flow, Tempus requires a PGV netlist as input along with other essential information required for static timing analysis. Run the `set_lib_binding_by_voltage` command after running the `init_design` command, which initiates auto inferencing of power intent and view definition. The `set_lib_binding_by_voltage` command provides voltages for each power net in the design.

In PGV, the instantiation of each library cell includes power and ground pins and their connectivity, as shown in the following example:

```
module block1 (clk, in1, in2, out1, rail1, rail2, gnd);
input clk, in1, in2;
output out1;
input rail1,rail2, gnd;
.BUFX1 i0 (.A(in1), .Y(n0), .VDD12(rail1), .VSS(gnd));
.NOR_HVT i1 (.N_CORE_O(n1), .CNTL_CORE_I(n0), .PORTS_ON_33_I(in2), .VDD12(rail1),
.VDDS33(rail2), .VSS(gnd));
.BUFX1 i2 (.A(n1), .Y(n3), .VDD12(rail1), .VSS(gnd));
.DFFHQX1 i3 (.CK(clk), .D(n3), .Q(out1), .VDD12(rail1), .VSS(gnd));
endmodule
```

SMSC PGV Flow Script

You can assert voltages for power rails using the `set_lib_binding_by_voltage` command, which initiates power intent inference and library binding. The following example presents the usage of the `set_lib_binding_by_voltage` command:

```
set_lib_binding_by_voltage
-power_net_voltages {<power_net_name> @<Voltage> <power_net_name>@<Voltage>}
[-process <num>]
[-temperature <num>]
```

You can specify the process and temperature of the corner using the `-process` and `-temperature` parameters of the `set_lib_binding_by_voltage` command along with the power rail voltages. You can provide all the libraries required for all the power domains upfront using the `read_lib` command. During the library binding, Tempus creates library sets for each power domain-based matching PVT conditions.

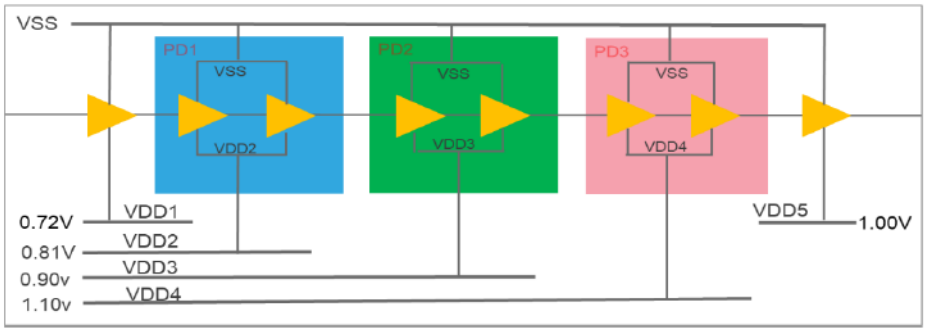
Following is the PGV SMSC flow script:

```
read_libs {list of Liberty files}
read_netlist <power_aware_verilog_file>
set_db init_read_netlist_allow_undefined_cells true; init_design <top_name>
read_sdc <sdc_file>
read_spef <spf_file>
set_lib_binding_by_voltage -power_net_voltages {net1@v1 net2@v2 .. } -process 1 -
temperature 125
report_timing ...
```

PGV SMSC Power Intent Inference

Following is an example of the MSV design:

Figure 10-4 PGV SMSC Power Intent Interface



In this example, the PGV flow creates an internal CPF with five power domains, one each for every power net, as shown below:

```
set_cpf_version 1.0e
set_hierarchy_separator "/"
set_db top
create_power_nets -nets VDD1
create_power_nets -nets VDD2
create_power_nets -nets VDD3
create_power_nets -nets VDD4
create_power_nets -nets VDD5
create_ground_nets -nets gnd
create_power_domain -name pd_VDD1 -default
update_power_domain -name pd_VDD1 -primary_power_net VDD1 -primary_ground_net gnd
create_power_domain -name pd_VDD2 -instances {PD1}
update_power_domain -name pd_VDD2 -primary_power_net VDD2 -primary_ground_net gnd
create_power_domain -name pd_VDD3 -instances {PD2}
update_power_domain -name pd_VDD3 -primary_power_net VDD3 -primary_ground_net gnd
create_power_domain -name pd_VDD4 -instances {PD3}
update_power_domain -name pd_VDD4 -primary_power_net VDD4 -primary_ground_net gnd
create_power_domain -name pd_VDD5
update_power_domain -name pd_VDD5 -primary_power_net VDD5 -primary_ground_net gnd
create_global_connection -net VDD1 -domain pd_VDD1 -pins VDD
create_global_connection -net gnd -domain pd_VDD1 -pins VSS
… Similarly for VDD2 thru VDD5
end_design
```

The `set_lib_binding_by_voltage` command also creates the view definition file for single corner analysis and assigns the operating voltage for each power domain accordingly. It also creates library sets for each power domain based on the assigned PVT and associates the corresponding library set for the power domains.

PG Net Voltages for Early and Late Analysis in PGV SMSC Flow

You can specify various early and late voltages for power rails using the `-early_power_net_voltages` and `-late_power_net_voltages` parameters of the `set_lib_binding_by_voltage` command. These parameters do not impact the inference of power domains.

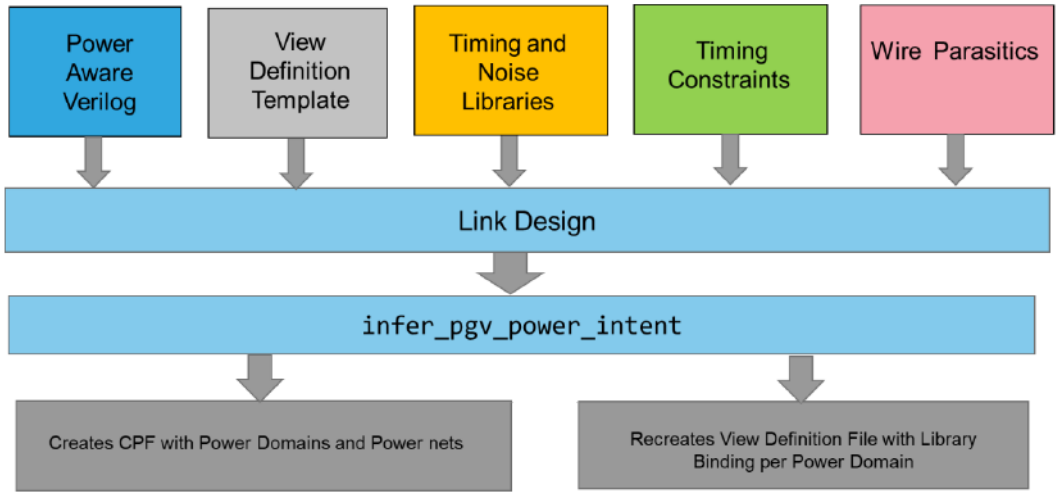
However, library binding will look for a library for both late and early voltages of each instance. As a result, the view definition file written out internally will consist of both early and late library sets and late and early `op_conds`.

Late analysis will use late operating condition and late library set. Similarly, early analysis is done based on early operating condition and early library set.

PGV MMMC Flow

In the PGV MMMC flow, timing analysis can be performed concurrently on multiple views. The following figure represents the PGV MMMC flow:

Figure 10-5 PGV MMMC Flow



In addition to a PGV netlist, the PGV MMMC flow requires a view definition template file. In this template file, you can define all the required analysis views, including the corresponding RC corner, constraint mode, and delay corner.

However, you do not need to define a delay corner definition to the power domain level. That means, you only need to specify the design-level operating conditions. You can also create a

single library set, which includes all the libraries to cover all the power domain voltages in a given timing analysis corner.

Power domains and power nets are independent of analysis corners or, rather, same for all corners. Only the voltages assigned to power nets are different for different analysis corners. You can define specific voltages for specific power nets in the view definition template file. For this requirement, use the `create_delay_corner -pg_net_voltages` parameter, as shown below:

```
create_delay_corner -pg_net_voltages \
    {<power_net_name> @<Voltage> <power_net_name>@<Voltage> … }
```

This parameter serves the same purpose as `set_lib_binding_by_voltage -power_net_voltages`. After the design is loaded with the template view definition file, you can use the `add_pg_netlist_power_intent` command to initiate power intent inferencing.

This command creates an individual power domain for each power net, similar to the PGV flow without MMMC (SMSC flow). This command also updates the view definition file to update `delay_corners` with the power domain information.

This command creates library sets for each power domain based on PVT. Power domain PVT is inferred based on the voltage from the associated power net of the domain, and the process and temperature of the design-level PVT for the corner.

View Definition Template

The view definition template is presented below:

```
create_library_set -name libset_all_dc1\
-timing [list stdLib_volt1.lib stdLib_volt2.lib stdLib_volt3.lib\
IO_volt1.lib IO_volt2.lib IO_volt3.lib ]
create_rc_corner -name default_rc_corner
create_timing_condition -name dc1 -library_set [list libset_all_dc1]\ -opcond slow -opcond_library stdLib_volt2.lib \
-pg_net_voltages {VDD1@0.535 VDD2@0.7 VDD3@1.05} -rc_corner default_rc_corner
create_library_set -name libset_all_dc2\
-timing [list stdLib_volt4.lib stdLib_volt5.lib stdLib_volt6.lib\
IO_volt4.lib IO_volt5.lib IO_volt6.lib ]
create_rc_corner -name default_rc_corner
create_timing_condition -name all -library_sets all_libs
create_rc_corner -name rcTyp
create_rc_corner -name rcMin
create_rc_corner -name rcMax
```

```
create_timing_condition -name dc2 -library_set [list libset_all_dc2] -opcond best
-opcond_library stdLib_volt4.lib \
-pg_net_voltages {VDD1@0.435 VDD2@0.65 VDD3@0.9} -rc_corner default_rc_corner
create_constraint_mode -name mode_func -sdc_files [list func.sdc]
create_analysis_view -name view1_func constraint_mode mode_func -delay_corner dc1
create_analysis_view -name view2_func -constraint_mode mode_func -delay_corner dc2
set_analysis_view -setup view1_func -hold view1_func
```

Note:

- The library set for each delay corner includes libraries for multiple voltages, which are assigned to different power domains during power intent inferencing and recreating the view definition file.
- The `opcond` specified for each delay corner provides the process and temperature values for the corner.
- The voltage for each power net is specified with the `create_delay_corner -pg_net_voltages` parameter.

MMMC PGV Flow Script

Following is the PGV MMMC flow script:

```
read_mmmcread_mmmc view_definition_template.tcl
read_netlist <power_aware_verilog_file>
set_db init_read_netlist_allow_undefined_cells true; init_design <top_name>
infer_pgv_power_intent
set_analysis_view -setup [….] -hold [….]
report_timing ...
```

Run the `set_analysis_view` command after running the `infer_pgv_power_intent` command to activate the MMMC views for PGV analysis. Tempus updates the MMMC view definition with power domains for each corner and corresponding library sets after the `set_analysis_view` command is run.

The view definition template file preferably has only one active view. It is recommended not to specify all active views in this file.

PGV MMMC Power Intent Inference

Power domain inferencing in the PGV MMMC flow is the same as in the PGV SMSC flow. The view definition template file creation is also very similar, except that the PGV MMMC flow has multiple corners/views defined.

PG Net Voltages for Early and Late Analysis in PGV MMMC Flow

Similar to the PGV SMSC flow, you can specify various early and late voltages for power rails in the PGV MMMC flow. You can specify the `-early_pg_net_voltages` and `-late_pg_net_voltages` parameters of the `create_delay_corner` command in the MMMC view definition template to assign various early/late voltages for power rails in a given corner.

During library binding, libraries for both late and early voltages of each instance will be bound to each cell as late/early libraries. As a result, the view definition template file written out internally will consist of both the early and late library sets and late and early `op_conds` for each corner.

Late analysis will use late operating condition and late library set. Similarly, early analysis is done based on early operating condition and early library set.

10.4 Importing the Design in Low Power Flow

Use the following Tempus commands to import the timing libraries, cdb library (for signal integrity analysis), and netlist in low power flow:

```
read_mmmcread_mmmc -cpf cpffile
read_netlist netlist
init_design
```

10.5 Non-MMMC Flow Settings in Low Power Flow

Use the below Non-MMMC flow settings in low power flow:

- Load the constraint and parasitic data files:

  ```
  read_sdc constraint_files
  read_spef parasitic_data_files
  ```

- Set the operating conditions for each power domain:

  ```
  set_opcond condition_name1 -library library_name1 -powerdomain domain_name1
  ```

- Load both the Min and Max timing libraries:

  ```
  set_timing_library -max Max -min Min
  set_opcond -max condition_name1 -maxlibrary maxLibrary_name1 -min condition_name2
  -minLibrary minLibrary_name1 -powerdomain domain_name1
  ```

After completing the above steps, you can run other Tempus commands to continue with the timing flow.

11

Appendix

This chapter covers the following topics:

- Appendix - Timing Debug GUI on page 590
- Appendix - Multi-CPU Usage on page 616
- Appendix - Base Delay, SI Delay, and SI Glitch Correlation with SPICE on page 622
- Appendix - Supported CPF Commands on page 632
- CPF 1.0e Script Example on page 633
- CPF 1.0 Script Example on page 640
- CPF 1.1 Script Example on page 650
- Supported CPF 1.0 Commands on page 657
- Supported CPF 1.0e Commands on page 666
- Supported CPF 1.1 Commands on page 677

Appendix - Timing Debug GUI

- Overview on page 590
- Timing Debug Flow on page 591
- Generating Timing Debug Report on page 592
- Displaying Violation Report on page 592
- Analyzing Timing Results on page 592
  - Viewing Power Domain Information on page 598
- Creating Path Categories on page 599
  - Creating Predefined Categories on page 599
  - Creating New Categories on page 601
  - Creating Sub-Categories on page 602
  - Hiding path categories on page 605
  - Reporting Path Categories on page 605
  - Using Categories to Analyze Timing Results on page 606
  - Analyzing MMMC Categories on page 607
  - Manual Slack Correction of Categories on page 609
- Editing Table Columns on page 610
  - Cell Coloring on page 612
- Viewing Schematics on page 613
- Running Timing Debug with Interface Logic Models on page 614

Overview

Tempus provides the Global Timing Debug feature for debugging the timing results. The various Timing Debug forms provide easy visual access to the timing reports and debugging tools.

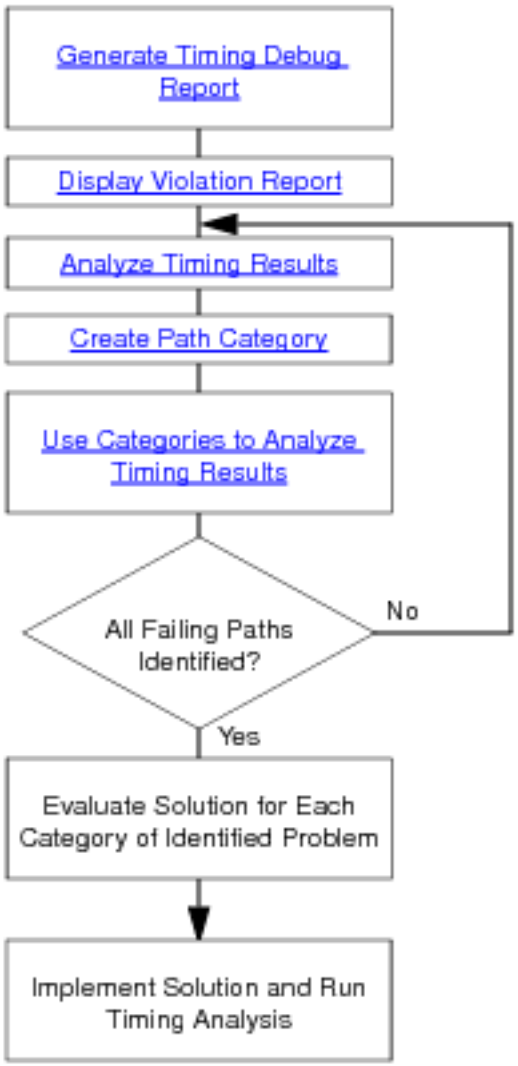
You can group all paths that are failing for the same reason and apply solutions for faster timing closure. You can cross-probe between the timing paths in the timing report and display area under Layout Tab in Tempus.

Note: If you have a previously saved timing debug report, you can use the timing debug feature even when the design is not loaded in the Tempus session.

Timing Debug Flow

You can generate a detailed violation report to list the details of all violating paths. You can then use the timing debug capability to visually identify problems with critical paths in this report. After identifying the problems, you group all paths with the same problem under a single category. You can define several categories to capture all problems related to the violating paths before fixing the problems and running timing analysis again.

Following is the flow for debugging timing results.



Note: The file `gtd.pref.tcl` is loaded automatically at launch.

Generating Timing Debug Report

Tempus uses a machine readable timing report to display timing debug information. The report is generated in the ASCII format and contains details of all violating paths. By default, the report has `.mtarpt` extension.

To generate a violation report, use one of the following options:

- Use `report_timing -machine_readable` command

You can also generate text-format report from a machine readable report.

To generate the text report, use the following:

- Use the `write_text_timing_report` command.
- Use the *Write Textual Timing Report* form.

Displaying Violation Report

To analyze the timing results, you need to load the machine readable timing report in Tempus.

To display the violation report, use one of the following options:

- Specify the file name in the *Display/Generate Timing Report* form.

  Note: To select an existing file, deselect the *Generate* option before clicking on the directory icon to the right of the *Timing Report File* field. By default, the global timing debug engine uses the following command to generate a machine-readable timing analysis report for the GUI display:

  ```
  report_timing -machine_readable -max_points 10000 -max_slack 0.75
  ```

- Use the `read_timing_debug_report` command.

  Note: Use the Append to Current Report option in the *Display/Generate Timing Report* form to load multiple reports in a single session.

Analyzing Timing Results

Tempus provides Timing Debug feature to visually analyze timing problems.

You analyze the following data in the *Analyze* tab in the main Tempus window:

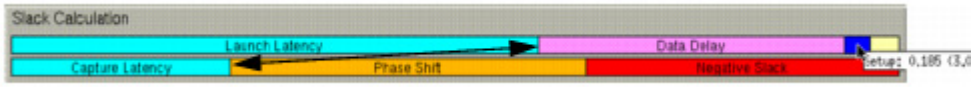
- Visual display of passing and failing paths as a histogram. Failing paths are represented in red and passing paths are represented in green color. The goal of timing debug process is to identify paths that fall in red category.
- Details of the critical paths in the Path list. You identify a critical path in this list for further analyses using the Timing Path Analyzer form.
- Visual display of paths reported in different timing reports. When you load multiple debug reports in a single timing debug session, the paths are displayed in different colors corresponding to the report file they are coming from. You can move the cursor over a path to display the name of the report file.

You analyze the following data in the *Timing Path Analyzer* form:

- Slack calculation bars for arrival and required times. You can identify clock skew issues, latency balancing or large clock uncertainty issues using these bars.

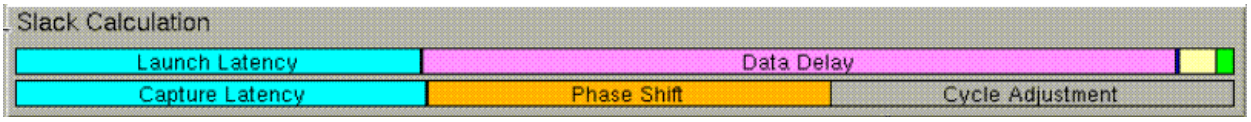
The following examples illustrate the problems that you can identify using the slack calculation bars in the *Timing Path Analyzer* form.

Example 9-1



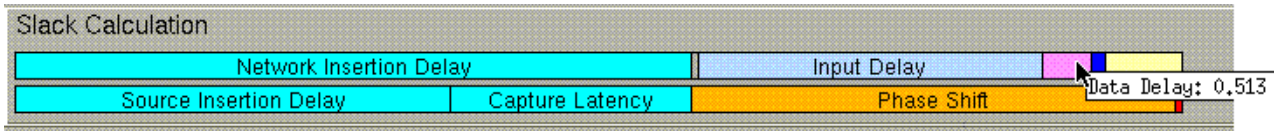
Launch and capture latency components are not aligned. Therefore there can be large clock-latency mismatch in this path.

Example 9-2



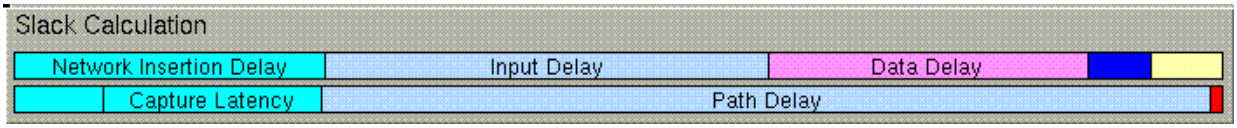
The cycle adjustment bar in the required time indicates presence of multicycle path.

Example 9-3



Large input delay in an I/O path is represented by the blue bar in the arrival time.

Example 9-4



Path Delay bar in the required time indicates a `set_max_delay` constraint.

- Path details including launch and capture. This information is provided as tabs in the Timing Path Analyzer form. You can click on a single path to display it in the design display area. You can select each element consecutively to trace the entire path in the design display area. This form has a Status column that indicates the status of the path as follows:

Flag	Description
a	Assign net
b	Blackbox instance
c	Clock net
cr	Cover cell
f	Preplaced instance
i	Ignore net
s	Skip route net
t	"don't touch" net
t	instance marked as "don't touch"
T	instance not marked as "don't touch" when the module it belongs to is marked as "don't touch"
u	Unplaced cell
x	External net

- SDC related to the path. The Path SDC tab displays the SDC constraints related to the selected path. The list contains the name of the SDC file, the line number that indicates the position of the constraint in the SDC file, and the constraint definition.

The commands that can be displayed in the Path SDC tab are:

- `create_clock`

- create_generated_clock
- group_path
- set_multicycle_path
- set_false_path
- set_clock_transition
- set_max_delay
- set_min_delay
- set_max_fanout
- set_fanout_load
- set_min_capacitance
- set_max_capacitance
- set_min_transition
- set_max_transition
- set_input_transition
- set_capacitance
- set_drive
- set_driving_cell
- set_logic_one
- set_logic_zero
- set_dont_use
- set_dont_touch
- set_case_analysis
- set_input_delay
- set_output_delay
- set_annotated_check
- set_clock_uncertainty
- set_clock_latency
- set_propagated_clock
- set_load

- `set_disable_clock_gating_check`
- `set_clock_gating_check`
- `set_max_time_borrow`
- `set_clock_groups`

When you create a constraint on the command line, the Path SDC tab interactively displays the result of the additional constraint.

Note: MMMC views are not displayed interactively.

Note: You can create a path category directly from SDC constraints in the Path SDC form. When you right-click a constraint and view the *Create Path Category* form, to see the line number (from the SDC file) and the name of the constraint.

In Tempus, you can also create a path category based on SDC constraint using the `-sdc` parameter of the `create_path_category` command:

```
create_path_category -name category_name -sdc {file_name line_number}
```

where *file_name* is the name of the constraint file and *line_number* is the line number of the SDC constraint.

- Schematic display of the path. The Schematics tab displays the gate-level schematic view of the critical path. For more information, see *Viewing Schematics.*
- Timing interpretation for the path. This feature provides rule-based path analysis to help you discover sources of potential timing problems in a path. By default, the software performs the following checks on the following rules:

Path structure

- Transparent Latch in Path
- Clock Gating
- Hard Macros
- HVT Cells
- Buffering List
- Net Fanout
- Level Shifters
- Isolation Cells

- Net too long
- Couple capacitance ratio

Timing and constraints

- Large Skew
- Divider in Clock Path
- Total SI Delay
- SI Delay
- External Delay

Floorplan

- Fixed Cells
- Distance from start to end
- Distance of repeater chain
- Detour
- Multiple power domains

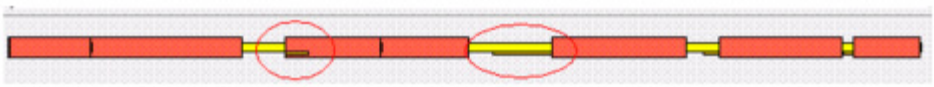
DRVs

- Max transitions
- Max capacitance
- Max fanout

You can customize the type of timing information reported. The *Edit Timing Interpretation* GUI Tempus lets you add, modify, or delete rules you want the tool to check and report.

- Timing bar to analyze delays associated with instances and nets in a path. Use this information to identify issues related to large instance or net delays, repeater chains, paths with large number of buffers, and large macro delays. The small bars

superimposed on net delays or within element delays show incremental (longer or shorter) delays due to noise effects:

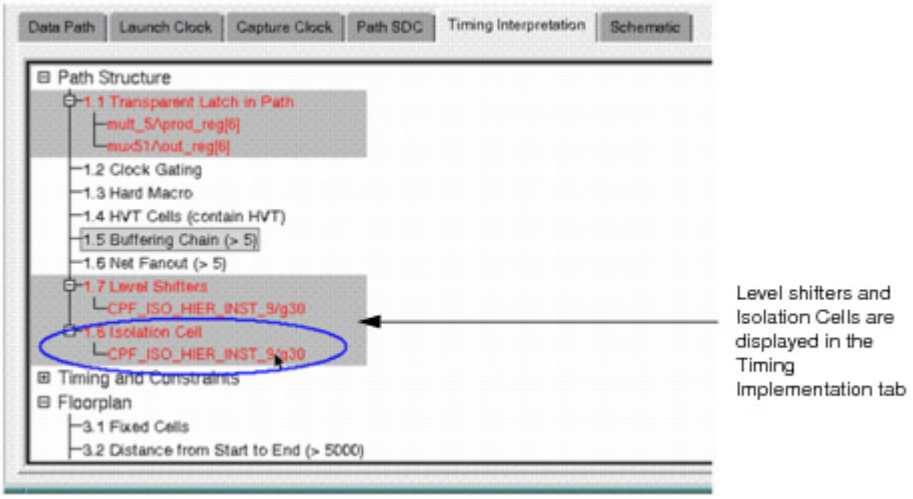


- Hierarchical representation of the path in the Hierarchy View field. This representation of the path-delay shows the traversal of a path through the design hierarchy drawn on the time axis. A longer arrow means that there are more instances on its path. Use this information to see the module where the path is spending more time or to identify inter-partition timing problems.

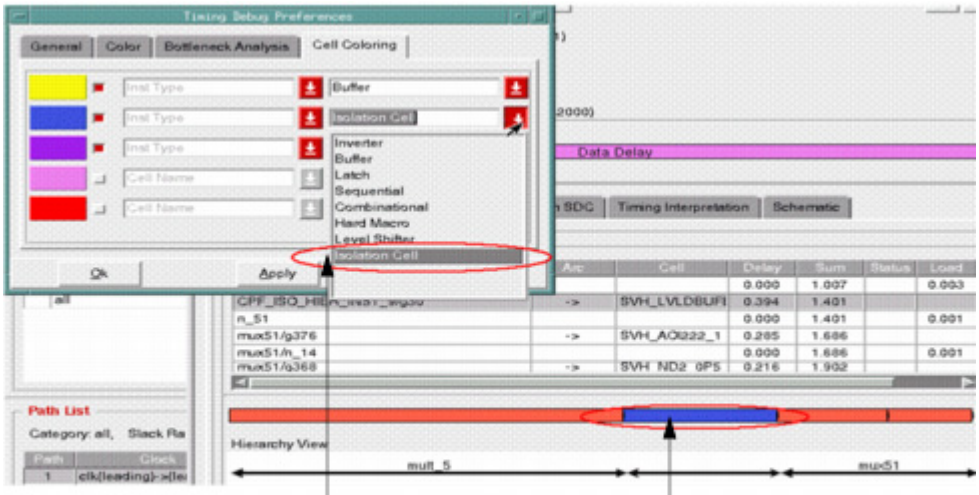
Viewing Power Domain Information

While debugging critical paths on MSV designs, it is useful to be able to identify power domains and low power cells. The Timing Debug feature displays this information in the following ways:

- The level shifters and isolation cells are listed in the Timing Interpretation tab of the Timing Path Analyzer.



- The delay bar of the Timing Path Analyzer can display the level shifters and isolation cells. You can also use the Preferences form to specify the colors in which the level shifters and isolation cells are displayed.



Creating Path Categories

After analyzing the paths in the timing report, you identify problems in various paths. Then you create a group of paths such that all paths in that group have the same timing problem and can be fixed at the same time. In timing debug such a group of paths is called a category. In Tempus, you can either define your own category or use predefined categories to group your paths. The categories that you define are then displayed in the *Path Category* field in the Analysis tab. The form also displays the paths associated with each category.

Creating Predefined Categories

There are following predefined categories:

- Basic Path Group

  Creates standard path categories according to basic path groups.

The basic path groups are:

- Register to register
- Input to register
- Register to output

- Input to Output

Registers can be macros, latches, or sequential-cell types. To create categories by basic path groups, choose the *Analysis* tab - *Analysis* drop-down menu - *Path Group Analysis* option.

- Clock Paths

  Creates categories according to launch clock - capture clock combinations.

Following categories are created:

- Paths with clock fully contained in a single domain, clk1.
- Paths with clocks starting one clock domain and ending at another, clk1->clk2.

To create categories for clock paths, choose the *Analysis* tab - *Analysis* drop-down menu - *Clock Analysis* option.

- Hierarchical Floorplan

  Creates categories according to the hierarchical characteristics of a path.

- View

  Creates categories according to the view for which the path was generated.

To create view path categories, choose the *Analysis* tab - *Analysis* drop-down menu - *View Analysis* option.

- False Paths

  Creates a category with paths defined as False paths.

To create view path categories, choose the *Analysis* tab - *Analysis* drop-down menu - *Critical False Path* option.

- Bottleneck

  Creates categories based on instances that occur often in critical paths.

To create view path categories, choose the *Analysis* tab - *Analysis* drop-down menu - *Bottleneck Analysis* option.

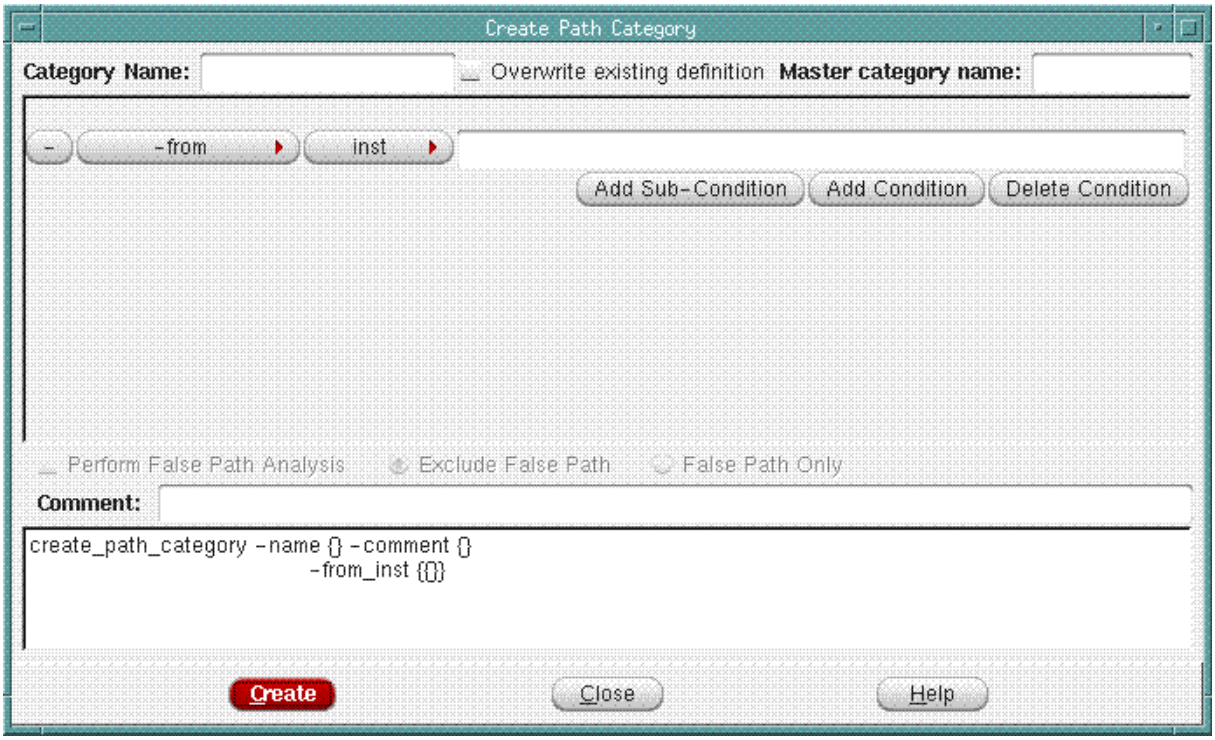
- DRV Analysis

  Generates or loads a DRV report containing capacitance, transition, or fanout violations. Paths that are affected by the selected DRV types are grouped in a category.

To create or load this report, choose the *Analysis* tab - *Analysis* drop-down menu - *DRV Analysis* option.

Creating New Categories

To define a new category, use the *Create Path Category* form. The Create Path Category form contains drop-down menus with conditions that you use to define a path category. The conditions are characteristics that a path must have to be added to the named category. You can define multiple conditions that a path must meet to be added to the category.



The category that you create is added in the *Path Category* field in the *Analysis* tab. All paths that meet the conditions set for this category are grouped under the category name. Paths are separated automatically according to MMMC views into different categories, for example:

```
CLOCK1<View_test_mode>
CLOCK2<View_mission_mode>
```

- Double-click on the category name in the *Path Category* field in the *Analysis* tab to display the list of paths in the *Path List* field.

Note: You can add a comment in the *Comment* field to record any notes that you would like to include with the category. The comment appears in the category report file.

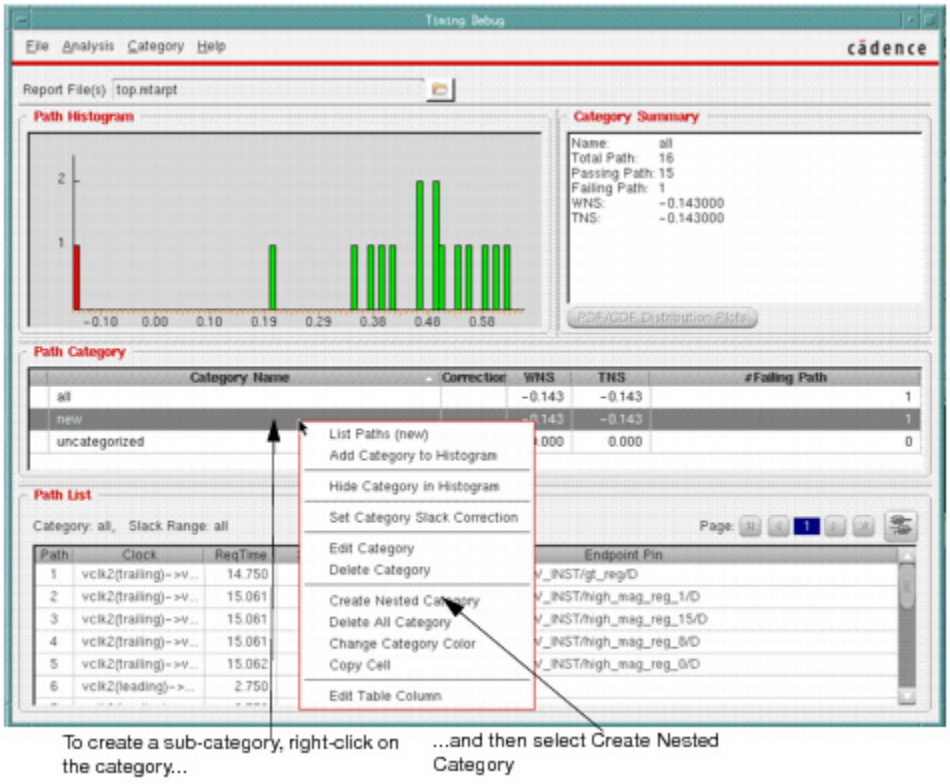
Creating Sub-Categories

You can create sub-categories based on existing categories. While analyzing a sub-category, global timing debug will traverse the paths in the master category instead of all the paths in the current report.

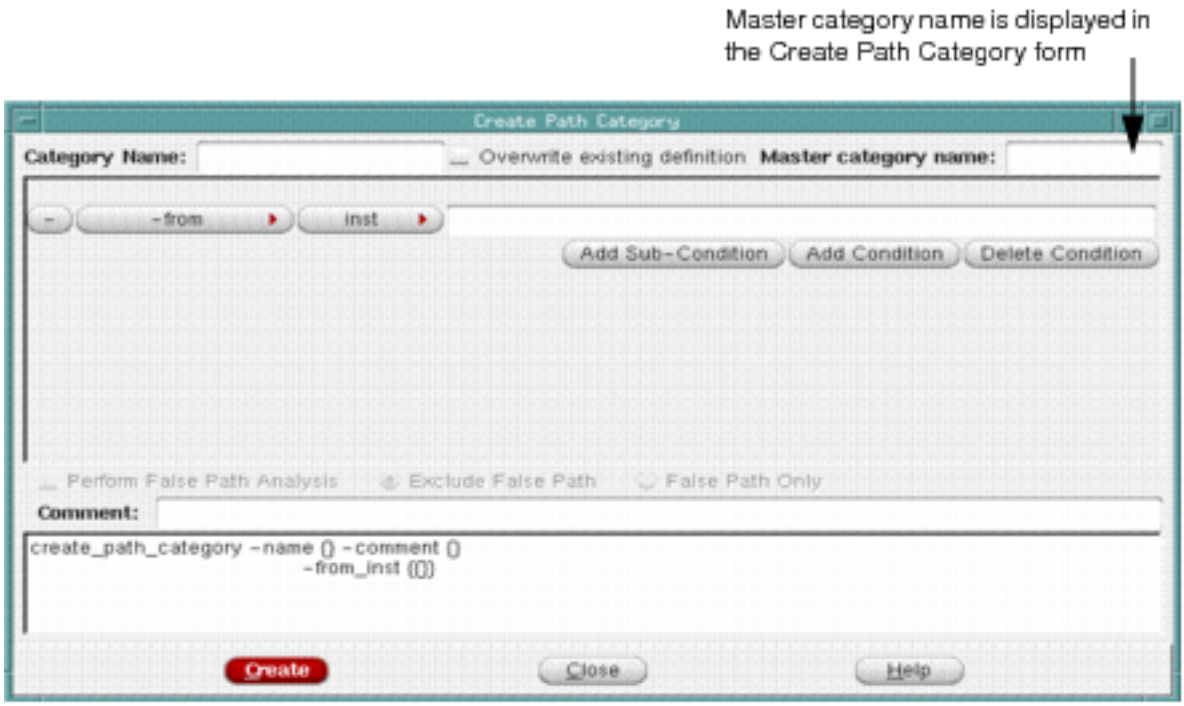
- Creating Sub-Categories through the GUI on page 602
- Creating Sub-Categories through Command Line on page 603

Creating Sub-Categories through the GUI

- In the GUI, you can create a sub category as follows:
- Right-click on a path category, and select *Nested Category*.



The *Create Path Category* form is displayed.



In the Master category name field, the name of the category you selected previously is displayed. Create one or more subcategories as explained in *Creating Path Categories.*

Note: You can create nested sub-categories, that is, you can further create sub-categories for a sub-category.

You can also use the Category - Create menu command to bring up the Create Path Category form. In the *Master category name* field, type the name of the category for which you want to create the sub-category, and then create one or more subcategories as explained in *Creating Path Categories*.

Creating Sub-Categories through Command Line

Use the `-master` parameter of the `create_path_category` command to create sub-categories. The category created will be a sub-category of the category name specified with the `-master` parameter.

The following other commands also support sub-categories; to run these commands only on the sub-categories of a particular master category, specify the master category name with the `-master` parameter.

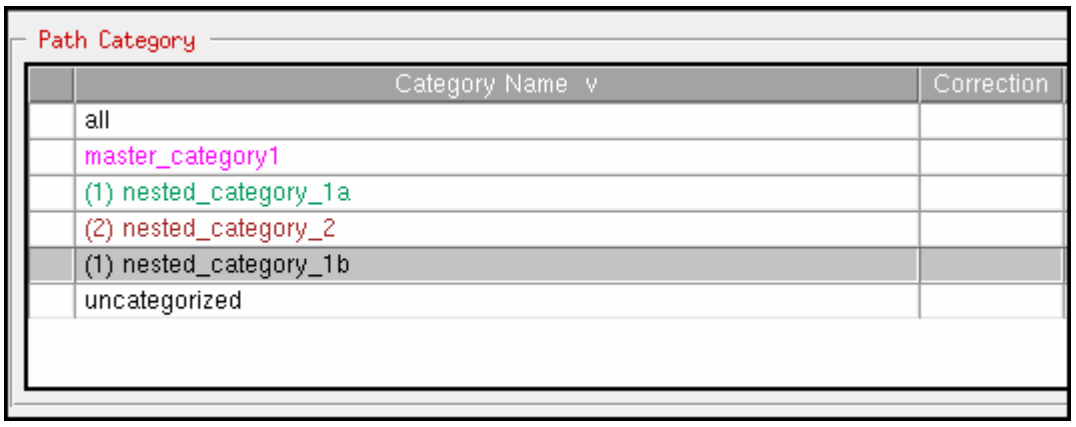
- `create_path_category`

- analyze_paths_by_default_path_group
- analyze_paths_by_bottleneck
- analyze_paths_by_clock_domain
- analyze_paths_by_critical_false_path
- analyze_paths_by_drv
- analyze_paths_by_hierarchy
- analyze_paths_by_view

Note: If the parent category of a sub-category is deleted, the sub-category cannot be edited or changed anymore. However, the sub-category is still displayed in case you want to refer to it.

Viewing Sub-Categories

The subcategories for a master category are displayed in a hierarchically numbered list below the master category. As an illustration, consider the example shown here:



In this example:

- `master_category1` is the master category
- `nested_category_1a` and `nested_category_1b` are the sub-categories of `master_category1`.
  The prefix (1) is displayed with `nested_category_1a` and `nested_category_1b`.
- `nested_category_2` is the sub-category of `nested_category_1a`.
  The prefix (2) is shown with `nested_category_2`.

Hiding path categories

To remove a path category from the histogram display, right-click on a path and select *Hide Category*. The category name in the category list is not hidden, but is marked with an "H" as hidden.

Reporting Path Categories

To generate a report containing information about path categories, use the following options:

- Use the `write_category_summary` command
- Use the *Write Category Report File GUI*

The text file contains the following information:

- Category name
- Total number of paths
- Number of passing paths
- Number of failing paths
- Worst negative slack
- Total negative slack
- TNS

Sample report:

Category name	Total path	Passing Path	Failing path	WNS	TNS
test_clock<view_test_	3869	768	3101	-5.699	-4802.857
100MHz_1.00V>	Clock Domain Analysis				
@->test_clock<view_test	484	55	429	-2.292	-378.432
_100MHz_1.00V>	Clock Domain Analysis				
my_clk-><view_mission	1	2	11	1.931	-1.931
_166MHz_1.08V>	Clock Domain Analysis				
my_clk_2x<view_mission	156	154	2	-.013	-0.21
_166MHz_1.08V>	Clock Domain Analysis				
cat1875	77	13	64	-3.524	-100.893
	Category of paths that cross i_1875 and start with test_clock - need to change the uncertainty value -advised --by Don				

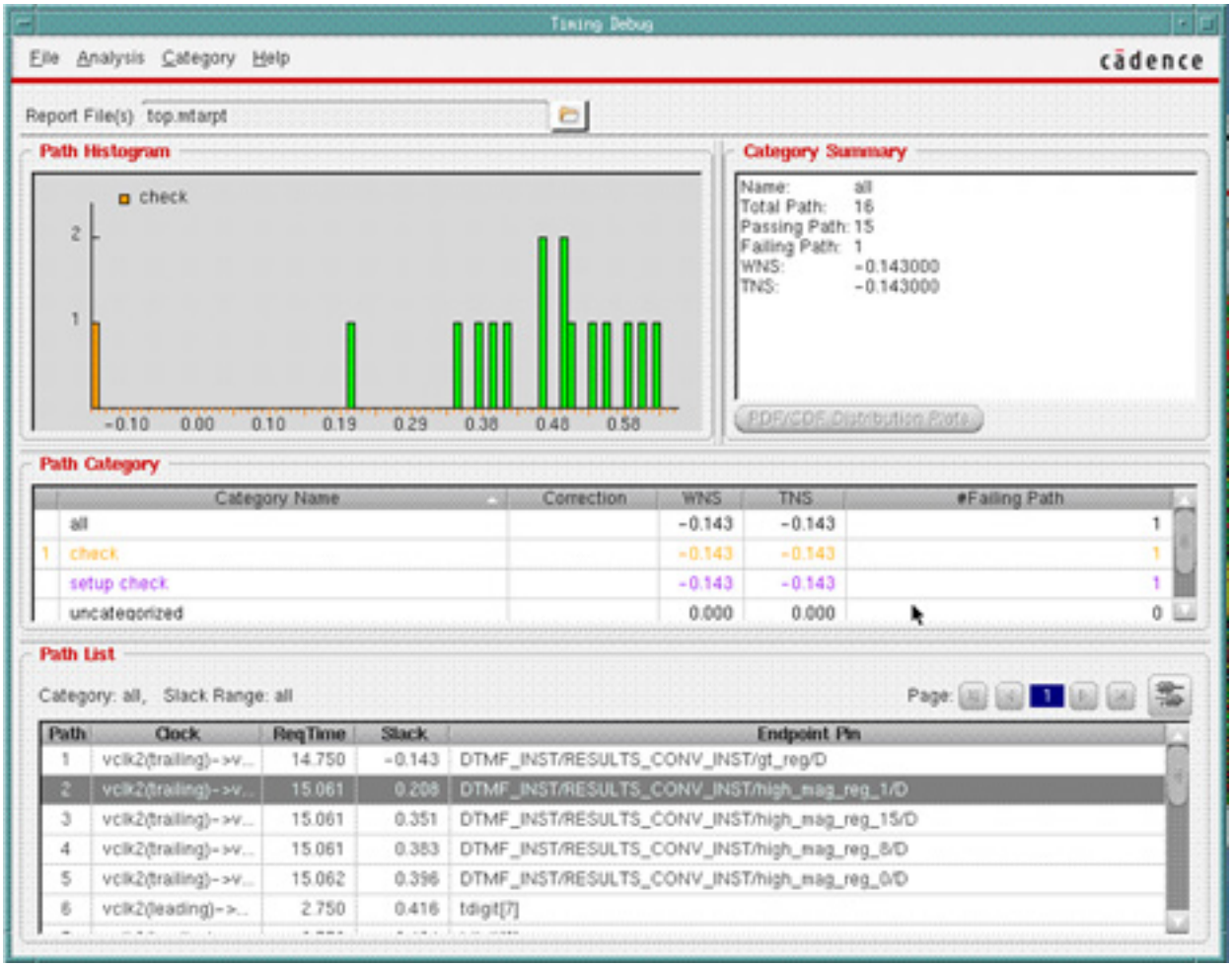
Using Categories to Analyze Timing Results

You can use the categories that you create to group the timing paths, and display them as histogram in the *Analysis* tab. The *Analysis* tab displays the category details in the *Path Category* field. You can perform the following tasks in the *Analysis* tab to analyze the timing results:

Double-click on any category to display the details of the paths grouped in that category in the *Path List* field.

Right-click on the category name and select *Add to Histogram* option. The paths related to the selected category are highlighted in a different color in the histogram. This gives you a visual representation of the number of paths that meet the conditions in that category and can possibly have the same timing problem.

For example, in the following figure the *SetupCheck* category was added to the histogram.



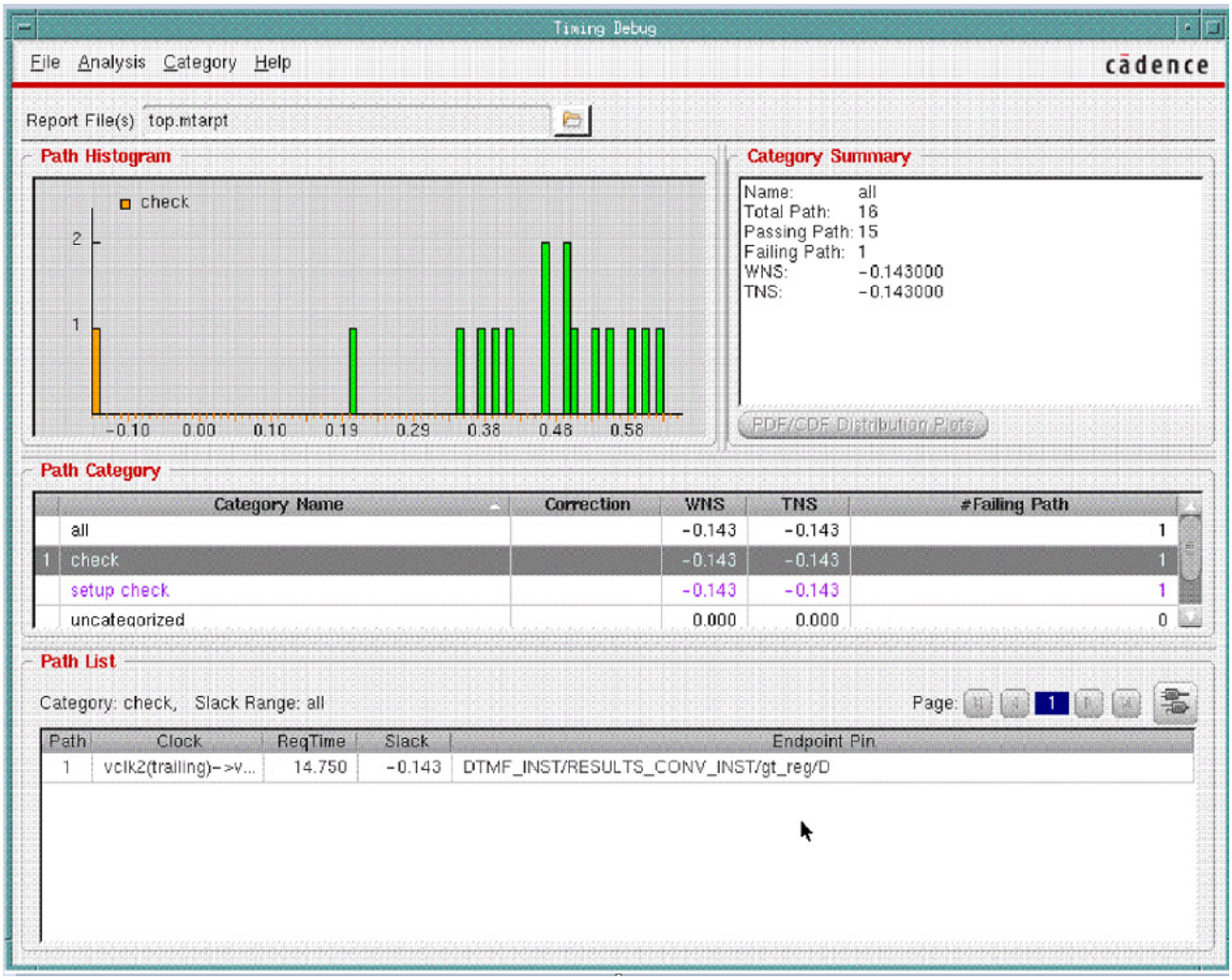
Analyzing the resulting the *Analysis* tab gives you information for fixing problems related to larger sets of timing paths. After identifying the problems, you can make the required changes such as modify floorplan, script or SDC files and run timing analysis again for further analysis.

Analyzing MMMC Categories

Paths are separated automatically according to MMMC views into different categories, for example, the following figure shows two categories based on MMMC views:

1. `view_mission_140MHz_1.0V`

2. `view_mission_166MHz_1.8V`

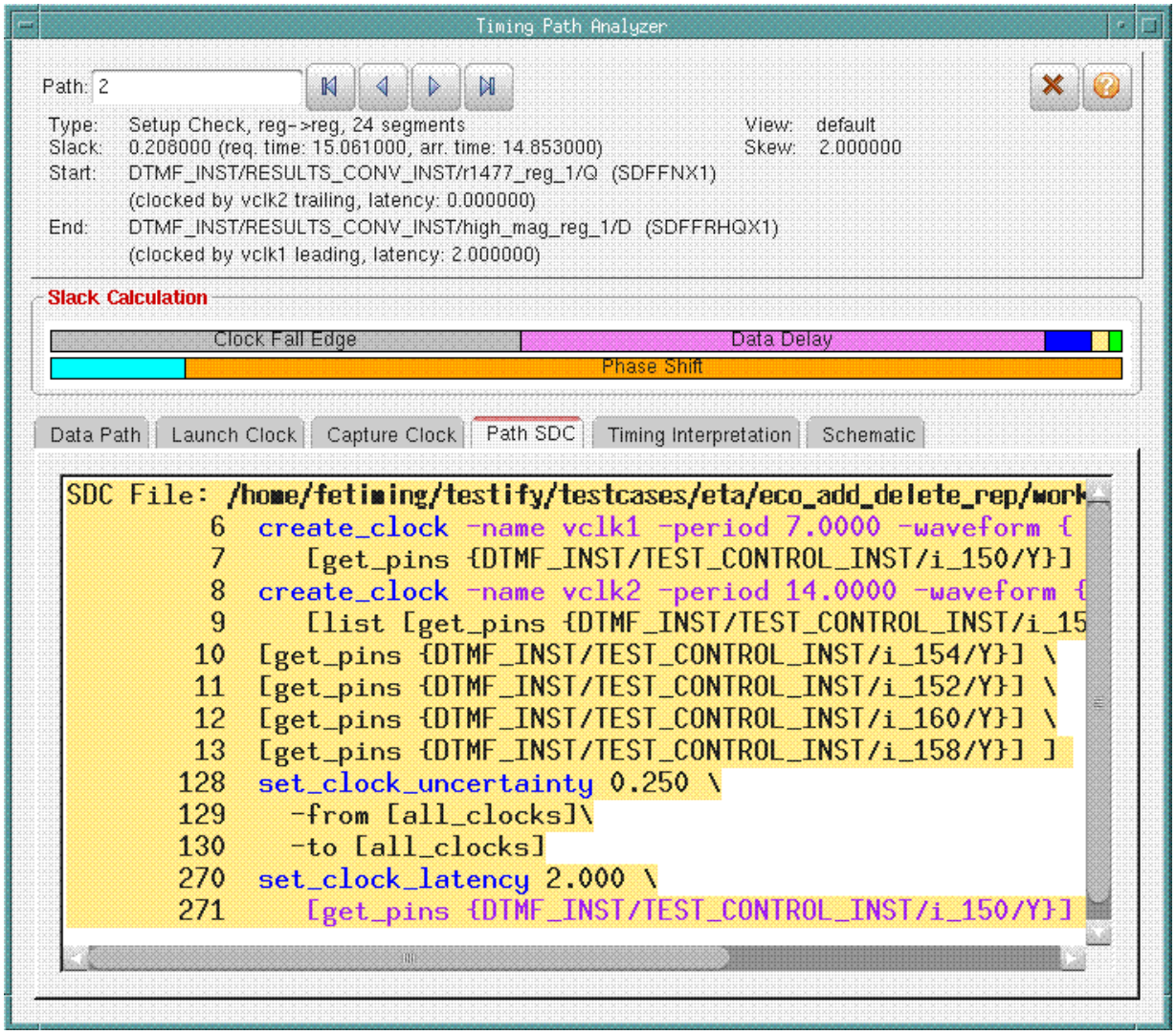


Do the following:

- Right-click on one of `view_mission_140MHz_1.0V` and choose *List Paths.*
- Right-click on one the paths and choose *Show Timing Path Analyzer*.

The Timing Path Analyzer is displayed.

- Click on the *Path SDC* tab to display the SDCs:



Note that the SDCs relative to mode `mission_140MHz` that produced the path are highlighted.

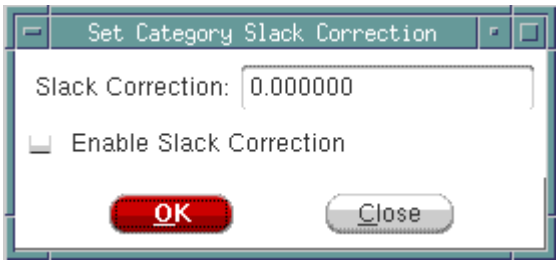
Manual Slack Correction of Categories

Use the Set Category Slack Correction form to specify the estimated slack correction for the selected category of paths. A slack correction that you apply to a category modifies all the paths in that category. If a path belongs to several categories, all the correction from the categories are added. The worst negative slack and total negative slack values of a category can be affected by the correction applied to another category.

Once you enable the slack correction, the histogram is updated to reflect the slack correction. An asterisk (*) is added next to the slack value of paths that belong to this category in the *Path List* field in the *Analysis* tab. Paths are reordered based on new specified slack. This allows you to filter out the paths that can be fixed and work on the remaining paths.

To access the Set Category Slack Correction form complete the following steps:

- Click on the *Analysis* Tab in the main Tempus window.
- Right click on the category name in the *Path Category* field.
- Choose the *Set Category Slack Correction* option.



To disable the set slack correction value:

- Right click on the category name in the *Path Category* field.
- Choose the *Deactive Category Slack correction* option.

Editing Table Columns

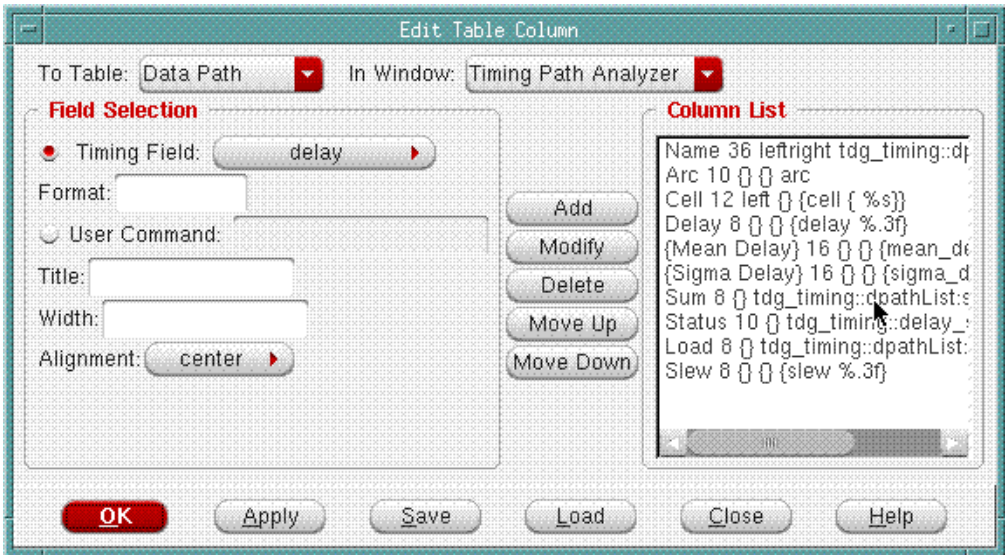
You can customize the dimensions and contents of table columns to suit your needs.

- To begin customizing a table, click on the *Analysis* tab, then right-click on a path.

A drop-down menu is displayed.

- Select *Edit Table* Column.

The *Edit Table Column* form is displayed.



- Choose the timing window that contains the table to want to customize.
- Choose the table whose columns you want to customize. The selections change according to the timing window you choose.
- Choose a column item or specify a command.

For commands, specify the procedure you want to use to determine the information you want to include in the column. Source the file containing the procedure before you specify the procedure here.

For example:

```
# Combine fedge (from edge) and tedge (to edge) #information into a single field
proc my_get_edge {id var} {
     upvar #0 $var p

    ($ added before p)
     if {p(type) == "inst"} {
        return "$p(fedge) -> $p(tedge)"
     } elseif {$p(type) == "port" } {
        return $p(fedge)
     } else {
        return ""
     }
}
```

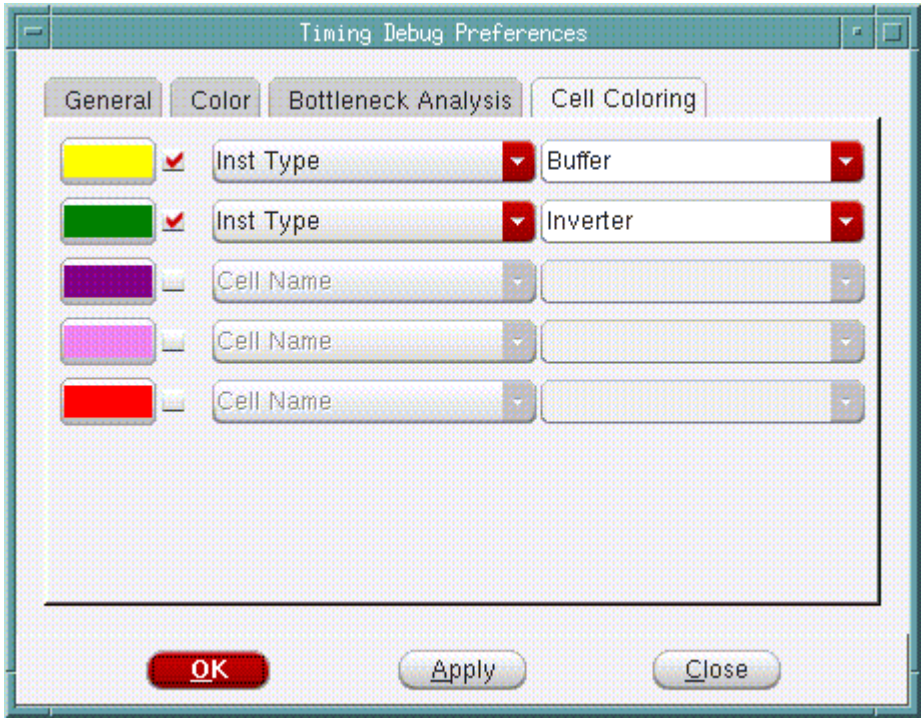
- Build the column list.
  - *Add* adds a column to the column list.

- *Modify* let you modify characteristics. Click on a column in the column list. Edit the information, then click *Modify*.
- *Delete* removes a column from the column list.
- *Move Up* moves a column up in the column list. This effectively moves a column to the left in the table.
- *Move Down* moves a column down in the column list. This effectively moves a column to the right table.

■ (Optional) Click *Load*. The opens the GTD (Global Timing Debug) Preferences form. Specify a file name.

Cell Coloring

Use the *Cell Coloring* page of the Timing Debug Preferences form to choose colors for specific cells in the delay bar.



When you assign colors, this same colors will be restored when you start a new session.

In the *Cell Name Selection Elements* field for each color, you can choose whether you are providing one of the following:

- Cell name
- Instance
- Procedure that you have defined

The procedure is invoked with the full instance name as the argument. You must source the file containing the procedure before you use this feature.

For example:

```
# Colors when the instance name contains "core/block1"
proc belongs_to_block1 {inst_name} {
     if [regexp {core/block1} $inst_name] {
        return 1
     } else {
        return 0
     }
}
```

Viewing Schematics

The *Critical Path Schematic Viewer* displays the gate-level schematic view of the critical path. To display the Schematic Viewer, click on the schematics icon in the *Path List* field of the *Analysis* tab. You can display additional paths in the Schematic Viewer by using the middle mouse button to drag the path from path list to Schematic Viewer.

You can also display the Schematics by selecting the Schematics tab in the *Timing Path Analyzer* form. The form is displayed when you double-click on a critical path in the Path List in the *Analysis* tab.

On displaying the Schematic Viewer, you can see the power instance colored and the power domain information displayed in a popup message box as well as in terminal.



You can perform the following tasks in the Module Schematic Viewer:

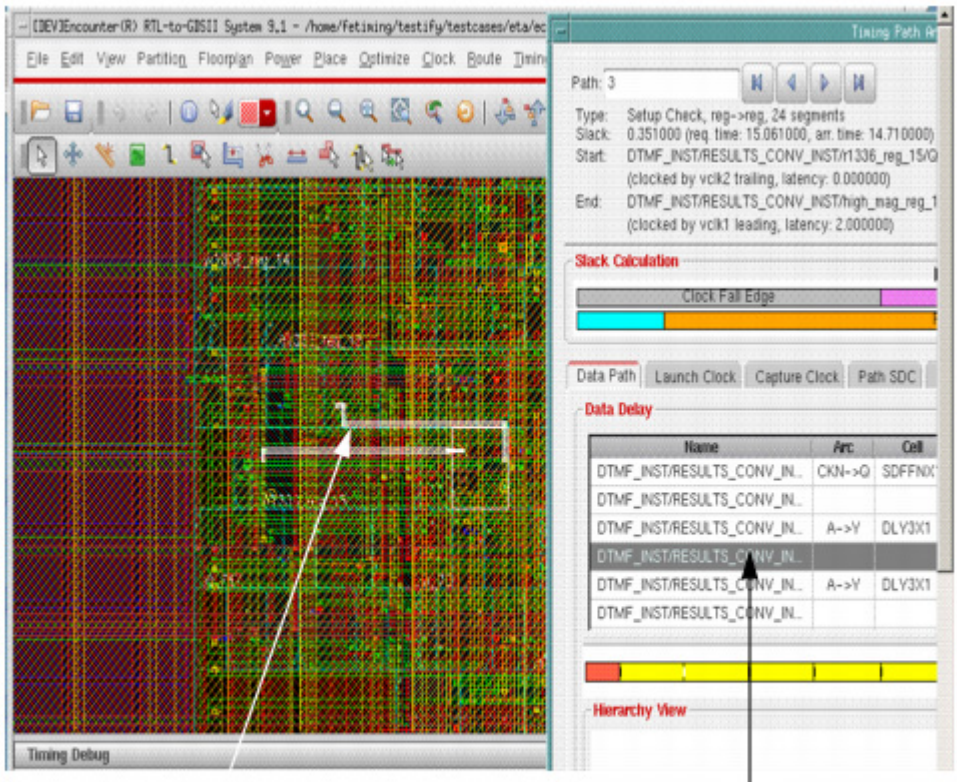
- View the Gate-level design elements.
- Select an element in the schematic.
  - Click on an object in the schematic to select and highlight it. When you move the cursor to an object, the object type and name of the object appear in the information box.
- Scroll over an object to display the object type and name of the object in the *Object* field.
- Cross-probe between the Schematics window and *Path List* field.
  - Select a path and left-click on the Schematics button above the Path List Table. (This is the button at the far-right side, just above the table).
  - To show multiple paths, select another path, and drag and drop it to the Schematics window.
- Use the menu options provided in the Schematic Viewer. To access the menu options, you can either click on the menu bar or right-click on an object in the schematic. You can use the menu options to perform the following tasks:
  - Manipulate schematic views of fan-in and fan-out cones.
  - Trace connectivity between drivers, objects, and loads.
  - Move between different levels of instance views.

Running Timing Debug with Interface Logic Models

You can use the timing debug feature with designs containing Interface Logic Models (ILMs).

- The Timing Path Analyzer - Path SDC form displays ILM SDCs rather than the original SDCs.

- The software highlights the entire ILM module instead of the instances and nets inside the ILM. The instances and the nets inside the ILMs are greyed out in the Timing Path Analyzer - Path SDC form.



The entire ILM module is highlighted

Instances and nets inside the ILM module are greyed out in the Timing Path Analyzer – Path SDC form.

Appendix - Multi-CPU Usage

- Overview on page 616
- Running Distributed Processing on page 617
- Running Multi-Threading on page 617
- Memory and Run Time Control on page 618
- Checking the Distributed Computing Environment on page 620
- Setting and Changing the License Check-Out Order on page 620
- Limiting the Multi-CPU License Search to Specific Products on page 620
- Releasing Licenses Before the Session Ends on page 620
- Controlling the Level of Usage Information in the Log File on page 621

Overview

You can accelerate portions of the design flow by using multiple-CPU processing. The Tempus Timing Solution software has the following multiple-CPU modes:

- Multi-threading

  In this mode, a job is divided into several threads, and multiple processors in a single machine process them concurrently.

- Distributed processing

  In this mode, a job is processed by two or more networked computers running concurrently.

You can configure multiple-CPU processing by using the commands described in the *Multiple-CPU Processing Commands* chapter of the *Tempus Text Command Reference*.

The following Tempus features support multiple-CPU processing:

- Multi-mode multi-corner timing analysis

  For information, see Distributed Multi-Mode Multi-Corner Timing Analysis.

Running Distributed Processing

To run the software in distributed processing mode, the following two commands are required:

- set_distributed_hosts

  Use this command to specify a configuration file for distributed processing or create the configuration for the remote shell, secure shell, or load-sharing facility queue to use for distributed processing. If you request more machines than are available, most applications wait until all requested machines are available.

  To display the current setting for `set_distribute_host`, use the `get_distribute_host` command.

- set_multi_cpu_usage

  Use this command to specify the maximum number of computers to use for processing.

  To display the current setting for `set_multi_cpu_usage`, use the get_multi_cpu_usage command.

Running Multi-Threading

To run the software in multi-threading mode, the following command is required:

- `set_multi_cpu_usage`

Use this command to specify the number of threads to use. Upon completion, the log file generated by each thread is appended to the main log file.

Note: The `-local_cpu` parameter limits the number of threads running concurrently. Although the software can create additional threaded jobs during run time, depending on the application in use, only the number of threads specified with this parameter are run at a given time.

If you ask for more threads than are available, the software issues a warning and runs with the maximum number of available threads.

For example, to run signal integrity analysis with four threads, specify the following commands:

```
set_multi_cpu_usage -local_cpu 4
set_db delaycal_enable_si true
report_timing
```

Memory and Run Time Control

Use the `report_resource` command to report memory/run time in multiple-CPU processing. This command allows you to determine how much memory is being used at any time and of what form (physical vs. virtual), and to determine real time and CPU time. You can use the `-verbose` parameter of the `report_resource` command to get detailed memory usage information.

When you run `report_resource -verbose`, the following detailed memory information is displayed:

Current (total cpu=0:00:12.9, real=0:05:48, peak res=275.8M, current mem=383.9M)				
Cpu(s) 2, load average: 4.63				
Mem: 16443800k total, 16378412k used, 65388k free, 105704k buffers				
Swap: 16777208k total, 17460k used, 16759748k free, 12528212k cached				
Memory Detailed Usage:				
	Data Resident Set(DRS)	Private Dirty(DRT)	Virtual Size(VIRT)	Resident Size(RES)
Total current:	383.9M	275.8M	854.1M	358.9M
peak:	383.9M	275.8M	854.1M	358.9M

- `Cpu(s)` is the number of available processors in the machine.
- `Load` average is the system load averages for the past 1 minute.
- `Mem` and `Swap` are the current memory information of the machine.
  The value of `MEM` in the LSF report corresponds to the value of `RES` in the `report_resource` report, and the value of `SWAP` in the LSF report corresponds to the value of `VIRT` in the `report_resource` report.
- `Data Resident Set (DRS)` is the amount of physical memory devoted to other than executable code. "`current mem`" shows this value (Total current DRS).
- `Private Dirty (DRT)` is the memory which must be written to disk before the corresponding physical memory location can be used for some other virtual page.
- "`peak res`" shows the value (Total peak DRT). This is the minimum number that you must reserve to run the program.
- `Virtual Size (VIRT)` is the total amount of virtual memory used by the task. It includes the swapped and non-swapped memory.

- `Resident Size (RES)` is the non-swapped physical memory a task has used. The number of "Total Peak RES" is the recommended physical memory to reserve.

The `-verbose` parameter also works in conjunction with the `-peak` and `-start/-end` parameters of the `report_resource` command. When you run the local distributed host (`set_distributed_hosts -local`) command, the memory information will include the memory consumed by master and clients. Otherwise, the master and client details are not displayed.

The following command script specifies to display detailed memory information during the `report_timing` command:

```
report_resource -start report_timing
set_distributed_hosts -local
set_multi_cpu_usage -local_cpu 8
report_timing -machine_readable -max_paths 100
report_resource -end report_timing -verbose
```

Note: For `-start/-end` parameters, use `-verbose` with the `-end` parameter only.

The following message is displayed:

Ending "report_timing" (total cpu=0:57:18, real=0:33:24, peak res=6493.1M, current mem=5305.0M)				
Memory Detailed Usage:				
	Data Resident Set(DRS)	Private Dirty(DRT)	Virtual Size(VIRT)	Resident Size(RES)
Total current:	5305.0M	4012.1M	5919.2M	4255.4M
peak:	10712.8M	6493.1M	15090.3M	7312.5M
Master current:	5305.0M	4012.1M	5919.2M	4255.4M
peak:	5565.5M	4055.1M	6064.5M	4298.4M
Task peak:	748.8M	368.7M	1219.4M	456.4M

`Task peak` reports peak value of each item from all clients, therefore, it is possible that eight values come from eight different clients.

Checking the Distributed Computing Environment

To check if distributed processing can work in the software environment, use the `check_multi_cpu_usage` command. This command checks if the specified CPUs can be accessed.

Setting and Changing the License Check-Out Order

To change the license check-out order, use the following command:

```
set_multi_cpu_usage -license_list {tempusl cndc tempusxl edsl edsxl}
```

For information on the default check-out order, see Product and Licensing Information.

Limiting the Multi-CPU License Search to Specific Products

Each base license allows a set of specific licenses to be used for Multi-CPU processing. This list can be obtained from the `get_multi_cpu_usage` command after invoking the software.

```
tempus 16> get_multi_cpu_usage
Total CPU(s) Enabled: 16
Current free CPU(s): 16
Current License(s): 1 Tempus_Timing_Signoff_XL
keepLicense: true
licenseList: tpsmp tpsl tpsxl
true
```

This license list can be customized from among the available choices by using the `set_multi_cpu_usage -license_list` command.

Releasing Licenses Before the Session Ends

By default, the software holds multi-CPU licenses for the duration of the current session. To release the multi-CPU licenses before the Tempus session ends, complete one of the following steps:

- Before running any multi-CPU applications, specify the following command to keep the acquired multiple CPU-licenses until the current session ends:

  ```
  set_multi_cpu_usage -keep_license true
  ```

  To display the current setting, use the `get_multi_cpu_usage -keep_license` command.

- To release multi-CPU licenses (for example, when global placement finishes), specify the following command:

  ```
  set_multi_cpu_usage -release_license
  ```

Controlling the Level of Usage Information in the Log File

Use the following command to set the level of usage information in the log file:

```
set_multi_cpu_usage -thread_info {0 | 1 | 2}
```

By default, the software does not write starting and ending information for threads or timing details to the log file, but you can change this behavior by specifying `1` or `2` for the `-thread_info` parameter.

- Specify `1` to write the final message to the log file.
- Specify `2` to write additional starting/ending information for each thread.

For information on Multi-CPU licensing, see Product and Licensing Information.

Appendix - Base Delay, SI Delay, and SI Glitch Correlation with SPICE

- Spice Deck Overview on page 623
- Spice Deck Generation on page 623
- Running Path Simulation and Results on page 628
- Results of the Simulation on page 628
- Debugging Correlation Issues on page 630

Spice Deck Overview

To maintain signoff accuracy of the design, designers often use SPICE to correlate critical paths in timing analysis with path simulation. Tempus offers a built-in critical path simulation for base delay correlation with SPICE.

This chapter describes how to perform SPICE correlation with base delay timing in Tempus. The methodology is explained by providing an overview of the `write_spice_deck` command used for SPICE deck generation and then, by explaining the process to run path simulation.

Spice Deck Generation

The `write_spice_deck` command is used to generate the SPICE deck, which means that the SPICE trace for a path is generated.

The output SPICE deck includes the following details:

- All nets in the path and their instance connections
- Standard cell gate information for the instances and their port connections
- Initial conditions and voltage sources
- Measure statements for slew and delay measurements
- RC parasitic network information

Using write_spice_deck Command Parameters for SPICE Deck Generation

The following parameters of the `write_spice_deck` command are frequently used for SPICE deck generation in base timing analysis correlation.

-report_timing_cmd {report_timing_options}

Description:

Specifies the path(s) for which SPICE deck needs to be generated. SPICE deck will contain the SPICE trace of the path exactly as reported by the `report_timing` options.

Options Used:

- `-path_type full_clock`: Is used when launch or capture clock path is also needed in the SPICE deck.
- `-late/-early`: Is used to generate a SPICE deck for the setup or hold mode.

Note: Without using the `report_timing` options, SPICE deck will be generated for worst path identified by the software.

-run_simulation

Description:

Enables the tool to perform path simulation using the Spectre™ simulator in Tempus.

Note: If this option is used, it is recommended to use the `report_timing -retime path_slew_propagation` option of `write spice deck` for more accurate correlation with SPICE because SPICE deck is path specific.

-spice_subckt_files string

Description:

Specifies the name of the SPICE sub-circuit file for the library cells. This option is used to get the port directions.

Note: When `write spice deck` is specified with `-run_simulation` and this option is not specified, Tempus uses the SPICE sub-circuits from the cdB files loaded in the design.

-spice_model_files string

Description:

Specifies the name of the SPICE models file. This option must be used when you specify a SPICE sub-circuit file for simulation.

Note: When `write spice deck` is specified with `-run_simulation` and this option is not specified, Tempus uses the SPICE models from the cdB files loaded in the design.

-spectre_exe_path

Description:

Specifies the location of the Spectre™ version, which is to be used for path simulation.

-spice_include_file

Description:

Specifies the user-defined SPICE statements that are written to the SPICE deck. For example, the measure statement or any SPICE option represents a user-defined SPICE statement.

-out_dir

Description:

Specifies the name of the output directory that contains the generated SPICE decks. The default directory name is `tempus_pathsim`.

Note:

- If the `-run_simulation` parameter is not specified, the SPICE deck is saved in the directory as `path_<index_of_path>_setup.sp`.
- If a single path is specified, the SPICE deck filename is saved as `path_1_setup.sp`.
- If more than one path is specified (using the `max_path/max_point/nworst` options of `report_timing`), SPICE decks are saved as `path_1_setup.sp`, `path_2_setup.sp`, and so on.
- If the `-run_simulation` option is specified in addition to the `path_1_setup.sp` file, some result files are also saved as mentioned in the *Running Path Simulation and Results* section.

-side_path_level number

Description:

Specifies the number of levels for the side path stages to be included in the SPICE deck. By default, the side path stages are not included in the SPICE deck. You can assign the maximum value of 4 for a side path stage. As you increase `-side_path_level`, the size of the SPICE deck and simulation run time will increase.

The default value of `-side_path_level` is 0, which means that only lumped cap model of side loads are provided.

Note: It is recommended to use value of 1 to include real gate load. It provides good accuracy with reasonable SPICE deck size and simulation run time.

-power_nets {list_of_power_nets} | - ground {list_of_ground_nets}

Specifies the list of global names for power/ground nets.

If the `-power_nets` and `-ground` parameters are not specified, Tempus retrieves this information from the power/ground supplies set via other ways in the design, such as the

`set_dc_sources` command used from libraries/cdb and so on. If the information is not available, Tempus uses the default list of power node names (VDD and VCC) and ground node names (GND and VSS).

For more details on the parameters and options, see `write_spice_deck` in the *Tempus Text Command Reference Guide*.

-xtalk {delay | vl_glitch | vh_glitch}

The `-xtalk` parameter generates Spice deck for a single net (not a path).

For example, the following commands create a VL and VH Spice deck for the receiver pin `x1/A` and will place them in the `spice_rip/outlier` directory:

```
write_spice_deck -xtalk vl_glitch -glitch_receiver_pin x1/A -out_dir spice_rip/
outlier -spice_model_files models.sp -spice_subckt_files base.sp
write_spice_deck -xtalk vh_glitch -glitch_receiver_pin x1/A -out_dir spice_rip/
outlier -spice_model_files models.sp -spice_subckt_files base.sp
```

If you do not specify the `report_timing -point_to_point` parameter, the Spice deck can be generated (with victim sweep). However, path level Spice deck generation is not supported.

Examples for SPICE Deck Generation Command

The following section shows examples of a generated SPICE deck.

Example 1: The following section in SPICE deck specifies global supplies and transient analysis window time.

```
*--------------------
*Command and Options
*--------------------
.GLOBAL vdd vbbnw
V0Xvdd  vdd 0 1.0
V0Xvbbnw vbbnw 0 1.0
.GLOBAL vss vbbpw
V0Xvss vss 0 0
V0Xvbbpw vbbpw 0 0
.TEMP -40.0000
.TRAN 1p 10256p
*---------------------------------
```

Example 2: The following section shows an example of a net and its instance connections.

```
*------------------------
*Net Instances
*------------------------
```

```
*Net module1/inst_1/net
*------------------------
Xmodule1/inst_1/net module1/inst_1/q
+ module1/inst_2/a wc:module1/inst_1/q
*------------------------
```

Example 3: The following section shows an example of a gate instance and its port connections.

```
*---------------------------------------
*Gate Instances
*---------------------------------------
*Gate module1/inst_2
*---------------------------------------
*Cell ports in order : a y vss vdd vbbpw vbbnw
*---------------------------------------
Xmodule1/inst_2/a module1/inst_2/y
+ 0 module1/inst_2/vdd 0 module1/inst_2/vbbnw
+ inv1
*---------------------------------------
```

Example 4: The following section shows the measurement statements for delay and slew.

```
*--------------------
*Measurement statements
*--------------------
.MEASURE TRAN Stage_0001_module1/inst_1/clk_DELAY_
module1/inst_1/q
+ TRIG V(module1/inst_1/clk) VAL=0.5 TD=167.9p RISE=1
+ TARG V(module1/inst_1/q) VAL=0.5 TD=167.9p RISE=1
.MEASURE TRAN Stage_0001_ module1/inst_1/q_SLEW
+ TRIG V(module1/inst_1/q) VAL=0.30 TD=167.9p RISE=1
+ TARG V(module1/inst_1//q) VAL=0.70 TD=167.9p RISE=1
.MEASURE TRAN Stage_0002_module1/inst_1/q_DELAY_
module1/inst_2/a
+ TRIG V(module1/inst_1/q) VAL=0.5 TD=167.9p RISE=1
+ TARG V(module1/inst_2/a) VAL=0.5 TD=167.9p RISE=1
.MEASURE TRAN Stage_0002_ module1/inst_2/a_SLEW
+ TRIG V(module1/inst_2/a) VAL=0.30 TD=167.9p RISE=1
+ TARG V(module1/inst_2/a) VAL=0.70 TD=167.9p RISE=1
---------------------
```

clk to q delay of module1/inst_1

Slew at module1/inst_1/q pin

Delay and Input slews

Note: The DELAY and SLEW keywords in the MEASURE statements indicate that these are the measurement statements for delay and slew, respectively.

Also, stage numbers (Stage_0001 and Stage_0002) in the MEASURE statement indicate the respective stages; first and second stage, in the timing path, which makes it easier for correlation with the `report_timing` report.

Running Path Simulation and Results

You can perform path simulation in the following two ways:

- Using the Spectre™ path simulator available in the Tempus installation, or `write_spice_deck -run_simulation`.
- Using the standalone (outside the Tempus environment) path simulation that can be run using Spectre™ or any simulator that understands the SPICE syntax.

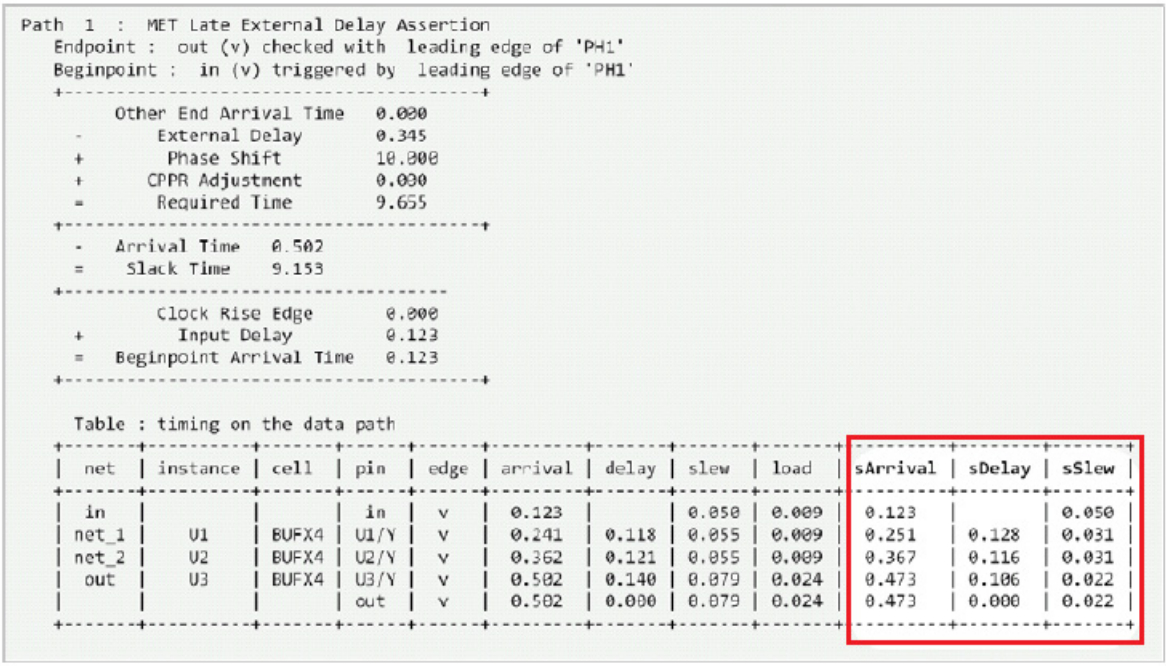
Using SPECTRE Path Simulator

You can use the `write_spice_deck -run_path_simulation` parameter to run the path simulation. In addition, you can specify the Spectre™ version using the `-spectre_exe_path` parameter of the `write_spice_deck` command. If the path is not specified with the `-spectre_exe_path` parameter, the software will check if you have the Spectre™ location defined in your path. If it is not defined in your path, it will use the Spectre™ version that is available under the Tempus installation. The generated log file will have the Spectre™ version used for path simulation.

Results of the Simulation

A table of timing (see figure below) with the slew, delay, and arrival columns from `report_timing` and path simulation, which are used for correlation comparison, is displayed.

If `report_timing -path_type full_clock` is used, two separate tables for launch and capture paths are also shown.



```
Path  1  :  MET Late External Delay Assertion
   Endpoint :  out (v) checked with  leading edge of 'PH1'
   Beginpoint :  in (v) triggered by  leading edge of 'PH1'
   +--------------------------------------+
         Other End Arrival Time   0.000
     -       External Delay       0.345
     +        Phase Shift        10.000
     +      CPPR Adjustment       0.000
     =       Required Time        9.655
   +--------------------------------------+
     -   Arrival Time   0.502
     =    Slack Time    9.153
   +-----------------------------------
           Clock Rise Edge       0.000
     +       Input Delay         0.123
     =   Beginpoint Arrival Time   0.123
   +--------------------------------------+

     Table : timing on the data path
+-------+----------+-------+------+------+---------+-------+-------+-------+---------+--------+-------+
|  net  | instance | cell  | pin  | edge | arrival | delay | slew  | load  | sArrival | sDelay | sSlew |
+-------+----------+-------+------+------+---------+-------+-------+-------+---------+--------+-------+
|  in   |          |       |  in  |  v   |  0.123  |       | 0.050 | 0.009 |  0.123  |        | 0.050 |
| net_1 |    U1    | BUFX4 | U1/Y |  v   |  0.241  | 0.118 | 0.055 | 0.009 |  0.251  | 0.128  | 0.031 |
| net_2 |    U2    | BUFX4 | U2/Y |  v   |  0.362  | 0.121 | 0.055 | 0.009 |  0.367  | 0.116  | 0.031 |
|  out  |    U3    | BUFX4 | U3/Y |  v   |  0.502  | 0.140 | 0.079 | 0.024 |  0.473  | 0.106  | 0.022 |
|       |          |       | out  |  v   |  0.502  | 0.000 | 0.079 | 0.024 |  0.473  | 0.000  | 0.022 |
+-------+----------+-------+------+------+---------+-------+-------+-------+---------+--------+-------+
```

Using Standalone Path Simulation

If the `write_spice_deck` command is run without the `-run_path_simulation` option, it will save the SPICE deck of the path (`path_1_setup.sp`) in the specified directory (which is specified using the `-out_dir` option). By default, it saves the SPICE deck in `tempus_pathsim` directory.

Using the standalone simulation generates the SPICE deck in the user-specified directory. The SPICE deck of the path (`path_1_setup.sp`) is saved at the specified path within the specified directory. Specify the directory through the `-out_dir` parameter of the `write_spice_deck` command. By default, the generated SPICE deck is saved in the `tempus_pathsim` directory.

You can also run standalone path simulation using Spectre™ or any other simulator, which understands the SPICE syntax on the SPICE deck (`path_1_setup.sp`) saved by Tempus.

Results of the Simulation

The `path_1_setup.measure` file is generated, which can be used to extract path simulation results.

The figure below is an example of the `path_1_setup.measure` file, which shows the slew and delay measurements of the two stages. This file can be easily post-processed to extract simulation results. For example, sum of all “delay” stages will give the path delay of the total path.

Sample path_1_setup.measure File

```
Exported variables from results directory:  ./new.raw

date               :  3:53:38 AM, Wed Aug 8, 2012
design             :  *
simulator          :  spectre

Measurement Name   :  transient1
Analysis Type      :  tran
stage_0001_u_cpu/ff1/clk_delay_u_cpu/ff1/q =  1.08834e-10
stage_0001_u_cpu/ff1/q_slew =  4.41217e-11
stage_0002_u_cpu/inst1/b_slew =  4.43192e-11
stage_0002_u_cpu/ff1/q_delay_u_cpu/inst1/b =  4.01084e-13
~
```

Debugging Correlation Issues

If the results between the SPICE path simulation and STA results do not correlate, you need to check the following:

- Ensure that the `.lib/.cdB` files are consistent with the SPICE models/subckt files being used for path simulation. Characterize libraries with the same SPICE models/subckt files which are used for running path simulation.
- Designers compare simulation results with the `report_timing` output. In such cases, since SPICE deck is always path-based, the comparison must always be done with the path-based analysis (PBA) report in Tempus. The PBA report can be generated using the `report_timing -retime path_slew_propagation` command. For slew/delay comparison with SPICE, make sure to use the `retime_slew` and `retime_delay` columns of the `report_timing` command instead of slew and delay, respectively.
- If there is any miscorrelation in any particular stage in delay or slew, then ensure that the slew/delay reported in Tempus analysis is not using extrapolation of library data. You can search for the IMPTS-403 message (Delay calculation was forced to extrapolate…) in `update_timing` as well as for the report of extrapolated arcs in a file with the nomenclature below written out by Tempus:

`<top_cell>_dc_outbound_<view>_*.rpt`

- SPICE deck generation and built-in path simulation do not account for any timing derates that may have been specified. Derate values can come from various sources, such as user-defined derates, AOCV derates, and so on.

  For correlation comparison, ensure that you compare underated STA numbers from Tempus. To do this, you need to load the design in Tempus without any derates.

- Another source of miscorrelation could be the waveform used for simulation. Check that the library file has normalized driver waveforms (NDW) specified.

- Another potential source of differences between SPICE path simulation and STA results is the difference in analysis mode in both runs. This can happen when STA is done in the On-Chip Variation (OCV) mode (which is the default in Tempus) because the SPICE simulation is done only in one corner, either MAX or MIN. This means in SPICE both the launch and capture paths are using the same corner SPICE data for path simulation results.

- If path being correlated has latches, Tempus timing will have time borrowing during timing analysis while SPICE run will not have any time borrowing. In that case, turn off time borrowing in Tempus using set `timing_use_latch_time_borrow false` setting for correlation with SPICE.

Appendix - Supported CPF Commands

- CPF 1.0e Script Example on page 633
- CPF 1.0 Script Example on page 640
- CPF 1.1 Script Example on page 650
- Supported CPF 1.0 Commands on page 657
- Supported CPF 1.0e Commands on page 666
- Supported CPF 1.1 Commands on page 677

CPF 1.0e Script Example

The following section contains an example of the CPF 1.0e file using a sample design and library.

For list of supported CPF commands and options within Encounter product family, see "Supported CPF 1.0e Commands".

```
#-------------------------------------------------------------------------------
------------
# setting
#-------------------------------------------------------------------------------
set_cpf_version 1.0e
set_hierarchy_separator /

#-------------------------------------------------------------------------------
# define library_set/cells
#-------------------------------------------------------------------------------
define_library_set -name wc_0v81 -libraries { \
 ../LIBS/timing/library_wc_0v81.lib }
define_library_set -name bc_0v81 -libraries { \
 ../LIBS/timing/library_bc_0v81.lib }
define_library_set -name wc_0v72 -libraries { \
 ../LIBS/timing/library_wc_0v72.lib }
define_library_set -name bc_0v72 -libraries { \
 ../LIBS/timing/library_bc_0v72.lib }

define_always_on_cell -cells {PTLVLHLD* AOBUFF*} -power_switchable \
 VDD -power TVDD -ground VSS

define_isolation_cell -cells { LVLLH* } -power VDD -ground VSS -enable \
 NSLEEP -valid_location to
define_isolation_cell -cells { ISOHID* ISOLOD* } -power VDD -ground VSS \
 -enable ISO -valid_location to

define_level_shifter_cell -cells { LVLHLD* } -input_voltage_range \
 0.72:0.81:0.09 -output_voltage_range 0.72:0.81:0.09 -direction down \
 -output_power_pin VDD -ground VSS -valid_location to
define_level_shifter_cell -cells { PTLVLHLD* } -input_voltage_range \
 0.72:0.81:0.09 -output_voltage_range 0.72:0.81:0.09 -direction down \
 -output_power_pin TVDD -ground VSS -valid_location to
define_level_shifter_cell -cells { LVLLHCD* } -input_voltage_range \
```

```
 0.72:0.81:0.09 -output_voltage_range 0.72:0.81:0.09 \
 -output_voltage_input_pin NSLEEP -direction up -input_power_pin VDDL \
 -output_power_pin VDD -ground VSS -valid_location to
define_level_shifter_cell -cells { LVLLHD* } -input_voltage_range \
 0.72:0.81:0.09 -output_voltage_range 0.72:0.81:0.09 -direction up \
 -input_power_pin VDDL -output_power_pin VDD -ground VSS -valid_location to

define_power_switch_cell -cells { HEADERHVT1 HEADERHVT2 } \
 -stage_1_enable NSLEEPIN1 -stage_1_output NSLEEPOUT1 -stage_2_enable \
 NSLEEPIN2 -stage_2_output NSLEEPOUT2 -type header -power_switchable VDD \
 -power TVDD

define_state_retention_cell -cells { MSSRPG* } -cell_type \
 master_slave -clock_pin CP -restore_check !CP -save_function !CP \
 -always_on_components { DFF_inst } -power_switchable VDD -power TVDD \
 -ground VSS
define_state_retention_cell -cells { BLSRPG* } -cell_type ballon_latch \
 -clock_pin CP -restore_function !NRESTORE -save_function SAVE \
 -always_on_components { save_data } -power_switchable VDD -power TVDD \
 -ground VSS

#-------------------------------------------------------------------------------
# macro models
#-------------------------------------------------------------------------------

#-------------------------------------------------------------------------------
# top design
#-------------------------------------------------------------------------------
set_design top

create_operating_corner -name PMdvfs2_bc -voltage 0.88 -process 1 -temperature \
 0 -library_set bc_0v72
create_operating_corner -name PMdvfs1_bc -voltage 0.99 -process 1 -temperature \
 0 -library_set bc_0v81
create_operating_corner -name PMdvfs1_wc -voltage 0.81 -process 1 -temperature \
 125 -library_set wc_0v81
create_operating_corner -name PMdvfs2_wc -voltage 0.72 -process 1 -temperature \
 125 -library_set wc_0v72

create_power_nets -nets VDD -voltage { 0.72:0.81:0.09 } -peak_ir_drop_limit 0 \
 -average_ir_drop_limit 0
```

```
create_power_nets -nets VDD_sw -voltage { 0.72:0.81:0.09 } -internal \
 -peak_ir_drop_limit 0 -average_ir_drop_limit 0
create_power_nets -nets VDDL -voltage 0.72 -peak_ir_drop_limit 0 \
 -average_ir_drop_limit 0
create_power_nets -nets VDDL_sw -voltage 0.72 -internal -peak_ir_drop_limit 0 \
 -average_ir_drop_limit 0
create_power_nets -nets Avdd -voltage 0.81 -peak_ir_drop_limit 0 \
 -average_ir_drop_limit 0
create_power_nets -nets VDD_IO -voltage { 0.72:0.81:0.09 } \
 -external_shutoff_condition { io_shutoff_ack } -peak_ir_drop_limit 0 \
 -average_ir_drop_limit 0

create_ground_nets -nets Avss -voltage 0.00 -peak_ir_drop_limit 0 \
 -average_ir_drop_limit 0
create_ground_nets -nets VSS -voltage 0.00 -peak_ir_drop_limit 0 \
 -average_ir_drop_limit 0

create_nominal_condition -name nom_0v81 -voltage 0.81
create_nominal_condition -name nom_0v72 -voltage 0.72

#--------------------------------------------------------------------------------
# create power domains
#--------------------------------------------------------------------------------
create_power_domain -name PDdefault -default
create_power_domain -name PDshutoff_io -instances { IOPADS_INST/Pspifsip \
 IOPADS_INST/Pspidip } -boundary_ports { spi_fs spi_data } \
 -external_controlled_shutoff -shutoff_condition io_shutoff_ack
create_power_domain -name PDpll -instances { INST/PLLCLK_INST \
 IOPADS_INST/Pibiasip IOPADS_INST/Ppllrstip IOPADS_INST/Prefclkip \
 IOPADS_INST/Presetip IOPADS_INST/Pvcomop IOPADS_INST/Pvcopop } -boundary_ports {
ibias reset \
 refclk vcom vcop pllrst }
create_power_domain -name PDram_virtual
create_power_domain -name PDram -instances { INST/RAM_128x16_TEST_INST } \
 -shutoff_condition !INST/PM_INST/power_switch_enable \
 -secondary_domains { PDram_virtual }
create_power_domain -name PDtdsp -instances { INST/RAM_128x16_TEST_INST1 \
 INST/DSP_CORE_INST0 INST/DSP_CORE_INST1 } -shutoff_condition \
 !INST/PM_INST/power_switch_enable -secondary_domains { PDdefault }

#--------------------------------------------------------------------------------
```

```
# set instances
#-------------------------------------------------------------------------------
set_instance INST/RAM_128x16_TEST_INST1/RAM_128x16_INST -domain_mapping \
 { {RAM_DEFAULT PDtdsp} }

set_macro_model ram_256x16A

create_power_domain -name RAM_DEFAULT -boundary_ports { A* D* CLK CEN WEN Q* } \
 -default -external_controlled_shutoff

create_state_retention_rule -name RAM_ret -instances { mem* } -save_edge !CLK

update_power_domain -name RAM_DEFAULT -primary_power_net VDD \
 -primary_ground_net VSS

end_macro_model

#-------------------------------------------------------------------------------
# create power modes
#-------------------------------------------------------------------------------
create_power_mode -name PMdvfs1 -default -domain_conditions { PDpll@nom_0v81 \
 PDdefault@nom_0v81 PDtdsp@nom_0v81 PDram@nom_0v72 PDshutoff_io@nom_0v81 \
 PDram_virtual@nom_0v72 }
create_power_mode -name PMdvfs1_off -domain_conditions { PDpll@nom_0v81 \
 PDdefault@nom_0v81 PDshutoff_io@nom_0v81 PDram_virtual@nom_0v72 }
create_power_mode -name PMdvfs1_shutoffio_off -domain_conditions { \
 PDpll@nom_0v81 PDdefault@nom_0v81 PDram_virtual@nom_0v72 }
create_power_mode -name PMdvfs2 -domain_conditions { PDpll@nom_0v81 \
 PDdefault@nom_0v72 PDtdsp@nom_0v72 PDram@nom_0v72 PDshutoff_io@nom_0v72 \
 PDram_virtual@nom_0v72 }
create_power_mode -name PMdvfs2_off -domain_conditions { PDpll@nom_0v81 \
 PDdefault@nom_0v72 PDshutoff_io@nom_0v72 PDram_virtual@nom_0v72 }
create_power_mode -name PMdvfs2_shutoffio_off -domain_conditions { \
 PDpll@nom_0v81 PDdefault@nom_0v72 PDram_virtual@nom_0v72 }
create_power_mode -name PMscan -domain_conditions { PDpll@nom_0v81 \
 PDdefault@nom_0v81 PDtdsp@nom_0v81 PDram@nom_0v72 PDshutoff_io@nom_0v81 \
 PDram_virtual@nom_0v72 }

create_analysis_view -name AV_dvfs1_BC -mode PMdvfs1 -domain_corners { \
 PDpll@PMdvfs1_bc PDdefault@PMdvfs1_bc PDtdsp@PMdvfs1_bc PDram@PMdvfs2_bc \
 PDshutoff_io@PMdvfs1_bc }
```

```
create_analysis_view -name AV_dvfs1_WC -mode PMdvfs1 -domain_corners { \
 PDpll@PMdvfs1_wc PDdefault@PMdvfs1_wc PDtdsp@PMdvfs1_wc PDram@PMdvfs2_wc \
 PDshutoff_io@PMdvfs1_wc }
create_analysis_view -name AV_dvfs1_off_BC -mode PMdvfs1_off -domain_corners { \
 PDpll@PMdvfs1_bc PDdefault@PMdvfs1_bc PDshutoff_io@PMdvfs1_bc }
create_analysis_view -name AV_dvfs1_off_WC -mode PMdvfs1_off -domain_corners { \
 PDpll@PMdvfs1_wc PDdefault@PMdvfs1_wc PDshutoff_io@PMdvfs1_wc }
create_analysis_view -name AV_dvfs1_shutoffio_off_BC -mode \
 PMdvfs1_shutoffio_off -domain_corners { PDpll@PMdvfs1_bc \
 PDdefault@PMdvfs1_bc }
create_analysis_view -name AV_dvfs1_shutoffio_off_WC -mode \
 PMdvfs1_shutoffio_off -domain_corners { PDpll@PMdvfs1_wc \
 PDdefault@PMdvfs1_wc }
create_analysis_view -name AV_dvfs2_BC -mode PMdvfs2 -domain_corners { \
 PDpll@PMdvfs1_bc PDdefault@PMdvfs2_bc PDtdsp@PMdvfs2_bc PDram@PMdvfs2_bc \
 PDshutoff_io@PMdvfs2_bc }
create_analysis_view -name AV_dvfs2_WC -mode PMdvfs2 -domain_corners { \
 PDpll@PMdvfs1_wc PDdefault@PMdvfs2_wc PDtdsp@PMdvfs2_wc PDram@PMdvfs2_wc \
 PDshutoff_io@PMdvfs2_wc }
create_analysis_view -name AV_PMdvfs2_off_BC -mode PMdvfs2_off -domain_corners \
 { PDpll@PMdvfs1_bc PDdefault@PMdvfs2_bc PDshutoff_io@PMdvfs2_bc }
create_analysis_view -name AV_PMdvfs2_off_WC -mode PMdvfs2_off -domain_corners \
 { PDpll@PMdvfs1_wc PDdefault@PMdvfs2_wc PDshutoff_io@PMdvfs2_wc }
create_analysis_view -name AV_dvfs2_shutoffio_off_BC -mode \
 PMdvfs2_shutoffio_off -domain_corners { PDpll@PMdvfs1_bc \
 PDdefault@PMdvfs2_bc }
create_analysis_view -name AV_dvfs2_shutoffio_off_WC -mode \
 PMdvfs2_shutoffio_off -domain_corners { PDpll@PMdvfs1_wc \
 PDdefault@PMdvfs2_wc }
create_analysis_view -name AV_scan_BC -mode PMscan -domain_corners { \
 PDpll@PMdvfs1_bc PDdefault@PMdvfs1_bc PDtdsp@PMdvfs1_bc PDram@PMdvfs2_bc \
 PDshutoff_io@PMdvfs1_bc }
create_analysis_view -name AV_scan_WC -mode PMscan -domain_corners { \
 PDpll@PMdvfs1_wc PDdefault@PMdvfs1_wc PDtdsp@PMdvfs1_wc PDram@PMdvfs2_wc \
 PDshutoff_io@PMdvfs1_wc }

#-------------------------------------------------------------------------
# create rules
#-------------------------------------------------------------------------
create_power_switch_rule -name PDram_SW -domain PDram -external_power_net VDDL
create_power_switch_rule -name PDtdsp_SW -domain PDtdsp -external_power_net \
```

```
 VDD

create_isolation_rule -name ISORULE1 -isolation_condition { \
 !INST/PM_INST/isolation_enable } -from { PDtdsp } -to { PDdefault } \
 -isolation_target from -isolation_output high
create_isolation_rule -name ISORULE3 -isolation_condition { \
 !INST/PM_INST/isolation_enable } -from { PDram } -to { PDdefault } \
 -isolation_target from -isolation_output high
create_isolation_rule -name ISORULE4 -isolation_condition { \
 !INST/PM_INST/spi_ip_isolate } -from { PDshutoff_io } \
 -isolation_target from -isolation_output low

create_level_shifter_rule -name LSRULE_H2L3 -from { PDdefault } -to { PDram } \
 -exclude { INST/PM_INST/power_switch_enable }
create_level_shifter_rule -name LSRULE_H2L_PLL -from { PDpll }
create_level_shifter_rule -name LSRULE_L2H2 -from { PDram } -to { PDdefault }

create_state_retention_rule -name \
 INST/DSP_CORE_INST0/PDtdsp_retention_rule -instances { \
 INST/DSP_CORE_INST0 } -save_edge !INST/DSP_CORE_INST0/clk
create_state_retention_rule -name \
 INST/DSP_CORE_INST1/PDtdsp_retention_rule -instances { \
 INST/DSP_CORE_INST1 } -restore_edge \
 !INST/PM_INST/state_retention_restore -save_edge \
 INST/PM_INST/state_retention_save

#-------------------------------------------------------------------------------
# update domains/modes
#-------------------------------------------------------------------------------
update_nominal_condition -name nom_0v81 -library_set wc_0v81
update_nominal_condition -name nom_0v72 -library_set wc_0v72

update_power_domain -name PDdefault -primary_power_net VDD -primary_ground_net \
 VSS
update_power_domain -name PDshutoff_io -primary_power_net VDD_IO \
 -primary_ground_net VSS
update_power_domain -name PDpll -primary_power_net Avdd -primary_ground_net \
 Avss
update_power_domain -name PDram_virtual -primary_power_net VDDL \
 -primary_ground_net VSS
update_power_domain -name PDram -primary_power_net VDDL_sw -primary_ground_net \
```

```
 VSS
update_power_domain -name PDtdsp -primary_power_net VDD_sw -primary_ground_net \
 VSS

update_power_mode -name PMdvfs1 -sdc_files ../RELEASE/mmmc/dvfs1.sdc
update_power_mode -name PMdvfs1_off -sdc_files ../RELEASE/mmmc/dvfs1.sdc
update_power_mode -name PMdvfs1_shutoffio_off -sdc_files \
 ../RELEASE/mmmc/dvfs1.sdc
update_power_mode -name PMdvfs2 -sdc_files ../RELEASE/mmmc/dvfs2.sdc
update_power_mode -name PMdvfs2_off -sdc_files ../RELEASE/mmmc/dvfs2.sdc
update_power_mode -name PMdvfs2_shutoffio_off -sdc_files \
 ../RELEASE/mmmc/dvfs2.sdc
update_power_mode -name PMscan -sdc_files ../RELEASE/mmmc/scan.sdc

#-------------------------------------------------------------------------------
# update rules
#-------------------------------------------------------------------------------
update_power_switch_rule -name PDram_SW -cells { HEADERHVT1 } -prefix \
 CDN_SW_RAM -peak_ir_drop_limit 0 -average_ir_drop_limit 0
update_power_switch_rule -name PDtdsp_SW -cells { HEADERHVT2 } -prefix \
 CDN_SW_TDSP -peak_ir_drop_limit 0 -average_ir_drop_limit 0

update_isolation_rules -names ISORULE1 -location to -prefix CPF_ISO_
update_isolation_rules -names ISORULE3 -location to -prefix CPF_ISO_
update_isolation_rules -names ISORULE4 -location to -prefix CPF_ISO_

update_level_shifter_rules -names LSRULE_H2L3 -location to -cells { LVLHLD* \
 } -prefix CPF_LS_
update_level_shifter_rules -names LSRULE_H2L_PLL -location to -prefix CPF_LS_
update_level_shifter_rules -names LSRULE_L2H2 -location to -prefix CPF_LS_

update_state_retention_rules -names \
 INST/DSP_CORE_INST0/PDtdsp_retention_rule -cell_type master_slave
update_state_retention_rules -names \
 INST/DSP_CORE_INST1/PDtdsp_retention_rule -cell_type ballon_latch

#-------------------------------------------------------------------------------
# end
#-------------------------------------------------------------------------------
end_design
```

CPF 1.0 Script Example

The following section contains an example of the CPF 1.0 file using a sample design and library.

For list of supported CPF commands and options within, see "Supported CPF 1.0 Commands".

```
set_cpf_version 1.0

#####################################################
# #
# Technology portion of the CPF: #
# Defining the special cells for low-power designs #
# #
#####################################################

################################
#### High-to-Low level shifters
################################

define_level_shifter_cell -cells LVLH2L* \
 -input_voltage_range 0.8:1.0:0.1 \
 -output_voltage_range 0.8:1.0:0.1 \
 -direction down \
 -output_power_pin VDD \
 -ground VSS \
 -valid_location to

########################################
#### Always-on High-to-low level shifters
########################################

define_level_shifter_cell -cells AOLVLH2L* \
 -input_voltage_range 0.8:1.0:0.1 \
 -output_voltage_range 0.8:1.0:0.1 \
 -direction down \
 -output_power_pin TVDD \
 -ground VSS \
 -valid_location to
```

```
################################
#### Low-to-High Level Shifters
################################
define_level_shifter_cell -cells LVLL2H* \
 -input_voltage_range 0.8:1.0:0.1 \
 -output_voltage_range 0.8:1.0:0.1 \
 -input_power_pin VDDI \
 -output_power_pin VDD \
 -direction up \
 -ground VSS \
 -valid_location to

#############################################################
#### Low-to-High level shifting plus isolation combo cells
#############################################################
define_level_shifter_cell -cells LVLCIL2H* \
 -input_voltage_range 0.8:1.0:0.1 \
 -output_voltage_range 0.8:1.0:0.1 \
 -output_voltage_input_pin ISO \
 -input_power_pin VDDI \
 -output_power_pin VDD \
 -direction up \
 -ground VSS \
 -valid_location to

####################
#### Isolation cells
####################

define_isolation_cell -cells LVLCIL2H* \
 -power VDD \
 -ground VSS \
 -enable ISO \
 -valid_location to

################################
#### Power switch cells: headers
################################

define_power_switch_cell -cells {HEADERHVT HEADERAOPHVT} \
```

```
 -power_switchable VDD -power TVDD \
 -stage_1_enable !ISOIN1 \
 -stage_1_output ISOOUT1 \
 -stage_2_enable !ISOIN2 \
 -stage_2_output ISOOUT2 \
 -type header

#################
#### SRPG cells
#################

define_state_retention_cell -cells { SRPG2Y } \
 -clock_pin CLK \
 -power TVDD \
 -power_switchable VDD \
 -ground VSS \
 -save_function "SAVE" \
 -restore_function "!NRESTORE"

################################################
#### Always-on cells: buffers and level shifters
################################################

define_always_on_cell -cells {AOBUFF2Y AOLVLH2L*} \
 -power_switchable VDD -power TVDD -ground VSS
################################################
# #
# Design part of the CPF #
# #
################################################

set_design top

set_hierarchy_separator "/"

set constraintDir ../CONSTRAINTS

set libdir ../LIBS

################################################
#### create the power and ground nets in this design
```

```
#####################################################

### VDD will connect the power follow-pin of the instances in the always-on
#power domain
### VDD_core_SW will connect the power follow-pin of the instances in the
#switchable power domain and is the power net that can be shut-off
### VDD_core_AO is the always-on power net for the switchable power domain

create_power_nets -nets VDD -voltage {0.8:1.0:0.1}
create_power_nets -nets VDD_core_AO -voltage 0.8
create_power_nets -nets VDD_core_SW -internal -voltage 0.8
create_power_nets -nets AVDD -voltage 1.0

create_ground_nets -nets VSS
create_ground_nets -nets AVSS

#########################################################################
#### Creating three power domains:
#### AO is the default always-on power domain
#### CORE is the switchable power domain
#### PLL is another always-on power domain
### Also specifying the power net-pin connection in each power domain
#########################################################################

#########################
### For power domain "AO"
#########################

create_power_domain -name AO -default
update_power_domain -name AO -internal_power_net VDD

create_global_connection -domain AO -net VDD -pins VDD
create_global_connection -domain AO -net VSS -pins VSS
create_global_connection -domain AO -net VDD_core_SW -pins VDDI

############################
### For power domain "CORE"
############################

create_power_domain -name core -instances CORE_INST \
 -shutoff_condition {PWR_CONTROL/power_switch_enable}
```

```
update_power_domain -name core -internal_power_net VDD_core_SW

create_global_connection -domain CORE -net VSS -pins VSS
create_global_connection -domain CORE -net VDD_core_AO -pins TVDD
create_global_connection -domain CORE -net VDD_core_SW -pins VDD

############################
### For power domain "PLL"
############################

### PLL conatins a single PLL macro and five top-level boundary ports which
#connect to the PLL macro directly

create_power_domain -name PLL -instances PLLCLK_INST -boundary_ports \
{refclk vcom vcop ibias pllrst}
update_power_domain -name PLL -internal_power_net AVDD

create_global_connection -domain PLL -net AVDD -pins avdd!
create_global_connection -domain PLL -net AVSS -pins agnd!
create_global_connection -domain PLL -net VDD -pins VDDI
create_global_connection -domain PLL -net AVDD -pins VDD
create_global_connection -domain PLL -net AVSS -pins VSS

########################################
########################################

set lib_1p1_wc "$libdir/technology45_std_1p1.lib"
set lib_1p3_bc "$libdir/technology45_std_1p3.lib"
set lib_1p0_bc "$libdir/technology45_std_1p0.lib"
set lib_0p8_wc "$libdir/technology45_std_0p8.lib"



set lib_ao_wc_extra "\
 $libdir/technology45_lvll2h_1p1.lib \
"
set lib_ao_bc_extra "\
 $libdir/technology45_lvll2h_1p3.lib \
"
set lib_core_wc_extra "\
 $libdir/technology45_lvlh2l_0p8.lib \
 $libdir/technology45_headers_0p8.lib \
```

```
 $libdir/technology45_sprg_ao_0p8.lib \
"
set lib_core_bc_extra "\
 $libdir/technology45_lvlh2l_1p0.lib \
 $libdir/technology45_headers_1p0.lib \
 $libdir/technology45_srpg_ao_1p0.lib \
"

set lib_pll_wc "\
 $libdir/pll_slow.lib \
 $libdir/ram_256x16_slow.lib \
 $libdir/rom_512x16_slow.lib \
"
set lib_pll_bc "\
 $libdir/pll_fast.lib \
 $libdir/ram_256x16_fast.lib \
 $libdir/rom_512x16_fast.lib \
"
########################
#### Define library sets
########################
define_library_set -name ao_wc_0p8 -libraries "$lib_0p8_wc $lib_ao_wc_extra"
define_library_set -name ao_bc_1p0 -libraries "$lib_1p0_bc $lib_ao_bc_extra"

define_library_set -name ao_wc_1p1 -libraries "$lib_1p1_wc_base $lib_ao_wc_extra"
define_library_set -name ao_bc_1p3 -libraries "$lib_1p3_bc_base $lib_ao_bc_extra"

define_library_set -name core_wc_0p8 -libraries "$lib_0p8_wc $lib_core_wc_extra"
define_library_set -name core_bc_1p0 -libraries "$lib_1p0_bc $lib_core_bc_extra"

define_library_set -name pll_wc_1p1 -libraries "$lib_pll_wc"
define_library_set -name pll_bc_1p3 -libraries "$lib_pll_bc"

############################
#### Create operating corners
############################

create_operating_corner -name BC_PVT_AO_L \
 -process 1 -temperature 0 -voltage 1.0 \
 -library_set ao_bc_1p0
```

```
create_operating_corner -name WC_PVT_AO_L \
 -process 1 -temperature 125 -voltage 0.8 \
 -library_set ao_wc_0p8

create_operating_corner -name BC_PVT_AO_H \
 -process 1 -temperature 0 -voltage 1.3 \
 -library_set ao_bc_1p3

create_operating_corner -name WC_PVT_AO_H \
 -process 1 -temperature 125 -voltage 1.1 \
 -library_set ao_wc_1p1

create_operating_corner -name BC_PVT_CORE \
 -process 1 -temperature 0 -voltage 1.0 \
 -library_set core_bc_1p0

create_operating_corner -name WC_PVT_CORE \
 -process 1 -temperature 125 -voltage 0.8 \
 -library_set tdsp_wc_0p8


create_operating_corner -name BC_PVT_PLL \
 -process 1 -temperature 0 -voltage 1.3 \
 -library_set core_bc_1p3

create_operating_corner -name WC_PVT_PLL \
 -process 1 -temperature 125 -voltage 1.1 \
 -library_set tdsp_wc_1p1


########################################
#### Create and update nominal conditions
########################################

create_nominal_condition -name high_ao -voltage 1.1
update_nominal_condition -name high_ao -library_set ao_wc_1p1

create_nominal_condition -name low_ao -voltage 0.8
update_nominal_condition -name low_ao -library_set ao_wc_0p8

create_nominal_condition -name low_core -voltage 0.8
```

```
update_nominal_condition -name low_core -library_set core_wc_0p8

create_nominal_condition -name high_pll -voltage 1.1
update_nominal_condition -name high_pll -library_set pll_wc_1p1

create_nominal_condition -name off -voltage 0

#####################################
#### Create and upDate four power modes
#####################################

create_power_mode -name PM_HL_FUNC \
 -domain_conditions {AO@high_ao CORE@low_core PLL@high_pll} \
 -default
update_power_mode -name PM_HL_FUNC -sdc_files ${constraintDir}/top_func.sdc

create_power_mode -name PM_HL_TEST \
 -domain_conditions {AO@high_ao CORE@low_core PLL@high_pll}
update_power_mode -name PM_HL_TEST -sdc_files ${constraintDir}/top_test.sdc

create_power_mode -name PM_HO_FUNC \
 -domain_conditions {AO@high_ao CORE@off PLL@high_pll}
update_power_mode -name PM_HO_FUNC -sdc_files ${constraintDir}/top_func.sdc

create_power_mode -name PM_LO_FUNC \
 -domain_conditions {AO@low_ao CORE@off PLL@high_pll}
update_power_mode -name PM_LO_FUNC -sdc_files ${constraintDir}/top_slow.sdc

#################################
##### Creating ten analysis views
#################################

create_analysis_view -name AV_HL_FUNC_MIN_RC1 -mode PM_HL_FUNC \
 -domain_corners {AO@BC_PVT_AO_H CORE@BC_PVT_CORE PLL@BC_PVT_PLL}
create_analysis_view -name AV_HL_FUNC_MIN_RC2 -mode PM_HL_FUNC \
 -domain_corners {AO@BC_PVT_AO_H CORE@BC_PVT_CORE PLL@BC_PVT_PLL}

create_analysis_view -name AV_HL_FUNC_MAX_RC1 -mode PM_HL_FUNC \
 -domain_corners {AO@WC_PVT_AO_H CORE@WC_PVT_CORE PLL@WC_PVT_PLL}
create_analysis_view -name AV_HL_FUNC_MAX_RC2 -mode PM_HL_FUNC \
 -domain_corners {AO@WC_PVT_AO_H CORE@WC_PVT_CORE PLL@WC_PVT_PLL}
```

```
create_analysis_view -name AV_HL_SCAN_MIN_RC1 -mode PM_HL_TEST \
 -domain_corners {AO@BC_PVT_AO_H CORE@BC_PVT_CORE PLL@BC_PVT_PLL}
create_analysis_view -name AV_HL_SCAN_MAX_RC1 -mode PM_HL_TEST \
 -domain_corners {AO@WC_PVT_AO_H CORE@WC_PVT_CORE PLL@WC_PVT_PLL}

create_analysis_view -name AV_HO_FUNC_MIN_RC1 -mode PM_HO_FUNC \
 -domain_corners {AO@BC_PVT_AO_H CORE@BC_PVT_CORE PLL@BC_PVT_PLL}
create_analysis_view -name AV_HO_FUNC_MAX_RC1 -mode PM_HO_FUNC \
 -domain_corners {AO@WC_PVT_AO_H CORE@WC_PVT_CORE PLL@WC_PVT_PLL}

create_analysis_view -name AV_LO_FUNC_MIN_RC1 -mode PM_LO_FUNC \
 -domain_corners {AO@BC_PVT_AO_L CORE@BC_PVT_CORE PLL@BC_PVT_PLL}
create_analysis_view -name AV_LO_FUNC_MAX_RC1 -mode PM_LO_FUNC \
 -domain_corners {AO@WC_PVT_AO_L CORE@WC_PVT_CORE PLL@WC_PVT_PLL}

############################################################
#### Creating and updating the rules for the insertion
#### of power switch, level shifter, isolation cell
############################################################

############################
##### One power switch rule
############################

create_power_switch_rule -name PWRSW_CORE -domain CORE \
-external_power_net VDD_core_AO

update_power_switch_rule -name PWRSW_CORE \
 -cells HEADERHVT \
 -prefix CDN_SW_ \
 -acknowledge_receiver SIWTCH_ENOUT
#########################################################################
##### One isolation rule using level-shifting and isolation combo cells
#########################################################################

create_isolation_rule -name ISORULE -from CORE \
-isolation_condition "!PWR_CONTROL/isolation_enable" \
-isolation_output high

update_isolation_rules -names ISORULE -location to -cells LVLCIL2H2Y
```

```
################################
##### Three level shifting rules
################################

#### For signals from AO to CORE

create_level_shifter_rule -name LSRULE_H2L -from AO -to CORE \
 -exclude {PWR_CONTROL/power_switch_enable PWR_CONTROL \
        /state_retention_enable PWR_CONTROL/state_retention_restore}
update_level_shifter_rules -names LSRULE_H2L -cells LVLH2L2Y -location to

#### Only for the control signals from AO to CORE

create_level_shifter_rule -name LSRULE_H2L_AO -from AO -to CORE \
 -pins {PWR_CONTROL/power_switch_enable PWR_CONTROL/state_retention_enable\
PWR_CONTROL/state_retention_restore}
update_level_shifter_rules -names LSRULE_H2L_AO -cells AOLVLH2L2Y -location to

#### For signals from PLL to AO

create_level_shifter_rule -name LSRULE_H2L_PLL -from PLL -to AO
update_level_shifter_rules -names LSRULE_H2L_PLL -cells LVLH2L2Y -location to

####################
##### One SRPG rule
####################

create_state_retention_rule -name SRPG_CORE \
 -domain CORE \
 -restore_edge {!PWR_CONTROL/state_retention_restore} \
 -save_edge {PWR_CONTROL/state_retention_enable}

update_state_retention_rules -names SRPG_CORE \
 -cell SRPG2Y \
 -library_set tdsp_wc_0v792

end_design
```

CPF 1.1 Script Example

The following section contains an example of the CPF 1.1 file using a sample design and library.

For list of supported CPF commands and options within Encounter product family, see Supported CPF 1.1 Commands.

```
#-------------------------------------------------------------------------------
------------
# setting
#-------------------------------------------------------------------------------
set_cpf_version 1.1
set_hierarchy_separator /

#-------------------------------------------------------------------------------
# define library_set/cells
#-------------------------------------------------------------------------------
define_library_set -name wc_0v81 -libraries { \
 ../LIBS/timing/library_wc_0v81.lib }
define_library_set -name bc_0v81 -libraries { \
 ../LIBS/timing/library_bc_0v81.lib }
define_library_set -name wc_0v72 -libraries { \
 ../LIBS/timing/library_wc_0v72.lib }
define_library_set -name bc_0v72 -libraries { \
 ../LIBS/timing/library_bc_0v72.lib }

define_always_on_cell -cells {PTLVLHLD* AOBUFF*} -power_switchable \
 VDD -power TVDD -ground VSS

define_isolation_cell -cells { LVLLH* } -power VDD -ground VSS -enable \
 NSLEEP -valid_location to
define_isolation_cell -cells { ISOHID* ISOLOD* } -power VDD -ground VSS \
 -enable ISO -valid_location to

define_level_shifter_cell -cells { LVLHLD* } -input_voltage_range \
 0.72:0.81:0.09 -output_voltage_range 0.72:0.81:0.09 -direction down \
 -output_power_pin VDD -ground VSS -valid_location to
define_level_shifter_cell -cells { PTLVLHLD* } -input_voltage_range \
 0.72:0.81:0.09 -output_voltage_range 0.72:0.81:0.09 -direction down \
 -output_power_pin TVDD -ground VSS -valid_location to
define_level_shifter_cell -cells { LVLLHCD* } -input_voltage_range \
```

```
 0.72:0.81:0.09 -output_voltage_range 0.72:0.81:0.09 \
 -output_voltage_input_pin NSLEEP -direction up -input_power_pin VDDL \
 -output_power_pin VDD -ground VSS -valid_location to
define_level_shifter_cell -cells { LVLLHD* } -input_voltage_range \
 0.72:0.81:0.09 -output_voltage_range 0.72:0.81:0.09 -direction up \
 -input_power_pin VDDL -output_power_pin VDD -ground VSS -valid_location to

define_power_switch_cell -cells { HEADERHVT1 HEADERHVT2 } \
 -stage_1_enable NSLEEPIN1 -stage_1_output NSLEEPOUT1 -stage_2_enable \
 NSLEEPIN2 -stage_2_output NSLEEPOUT2 -type header -power_switchable VDD \
 -power TVDD -stage_1_on_resistance 10 - stage_2_on_resistance 10

define_state_retention_cell -cells { MSSRPG* } -cell_type \
 master_slave -clock_pin CP -restore_check !CP -save_function !CP \
 -always_on_components { DFF_inst } -power_switchable VDD -power TVDD \
 -ground VSS
define_state_retention_cell -cells { BLSRPG* } -cell_type ballon_latch \
 -clock_pin CP -restore_function !NRESTORE -save_function SAVE \
 -always_on_components { save_data } -power_switchable VDD -power TVDD \
 -ground VSS

#-------------------------------------------------------------------------------
# macro models
#-------------------------------------------------------------------------------

#-------------------------------------------------------------------------------
# top design
#-------------------------------------------------------------------------------
set_design top

create_operating_corner -name PMdvfs2_bc -voltage 0.88 -process 1 -temperature \
 0 -library_set bc_0v72
create_operating_corner -name PMdvfs1_bc -voltage 0.99 -process 1 -temperature \
 0 -library_set bc_0v81
create_operating_corner -name PMdvfs1_wc -voltage 0.81 -process 1 -temperature \
 125 -library_set wc_0v81
create_operating_corner -name PMdvfs2_wc -voltage 0.72 -process 1 -temperature \
 125 -library_set wc_0v72

create_power_nets -nets VDD -voltage { 0.72:0.81:0.09 } -peak_ir_drop_limit 0 \
 -average_ir_drop_limit 0
```

```
create_power_nets -nets VDD_EQ -voltage { 0.72:0.81:0.09 } -peak_ir_drop_limit 0\
 -average_ir_drop_limit 0
create_power_nets -nets VDD_sw -voltage { 0.72:0.81:0.09 } -internal \
 -peak_ir_drop_limit 0 -average_ir_drop_limit 0
create_power_nets -nets VDDL -voltage 0.72 -peak_ir_drop_limit 0 \
 -average_ir_drop_limit 0
create_power_nets -nets VDDL_sw -voltage 0.72 -internal -peak_ir_drop_limit 0 \
 -average_ir_drop_limit 0
create_power_nets -nets Avdd -voltage 0.81 -peak_ir_drop_limit 0 \
 -average_ir_drop_limit 0
create_power_nets -nets VDD_IO -voltage { 0.72:0.81:0.09 } \
 -external_shutoff_condition { io_shutoff_ack } -peak_ir_drop_limit 0 \
 -average_ir_drop_limit 0

create_ground_nets -nets Avss -voltage 0.00 -peak_ir_drop_limit 0 \
 -average_ir_drop_limit 0
create_ground_nets -nets VSS -voltage 0.00 -peak_ir_drop_limit 0 \
 -average_ir_drop_limit 0

create_nominal_condition -name nom_0v81 -voltage 0.81
create_nominal_condition -name nom_0v72 -voltage 0.72

#-------------------------------------------------------------------------------
# create power domains
#-------------------------------------------------------------------------------
create_power_domain -name PDdefault -default
create_power_domain -name PDshutoff_io -instances { IOPADS_INST/Pspifsip \
 IOPADS_INST/Pspidip } -boundary_ports { spi_fs spi_data } \
 -external_controlled_shutoff -shutoff_condition io_shutoff_ack
create_power_domain -name PDpll -instances { INST/PLLCLK_INST \
 IOPADS_INST/Pibiasip IOPADS_INST/Ppllrstip IOPADS_INST/Prefclkip \
 IOPADS_INST/Presetip IOPADS_INST/Pvcomop IOPADS_INST/Pvcopop } -boundary_ports {
ibias reset \
 refclk vcom vcop pllrst }
create_power_domain -name PDram_virtual
create_power_domain -name PDram -instances { INST/RAM_128x16_TEST_INST } \
 -shutoff_condition !INST/PM_INST/power_switch_enable \
 -base_domains { PDram_virtual }
create_power_domain -name PDtdsp -instances { INST/RAM_128x16_TEST_INST1 \
 INST/DSP_CORE_INST0 INST/DSP_CORE_INST1 } -shutoff_condition \
 !INST/PM_INST/power_switch_enable -base_domains { PDdefault }
```

```
#-------------------------------------------------------------------------------
# set instances
#-------------------------------------------------------------------------------
set_instance INST/RAM_128x16_TEST_INST1/RAM_128x16_INST -domain_mapping \
 { {RAM_DEFAULT PDtdsp} }

set_macro_model ram_256x16A

create_power_domain -name RAM_DEFAULT -boundary_ports { A* D* CLK CEN WEN Q* } \
 -default -external_controlled_shutoff

create_state_retention_rule -name RAM_ret -instances { mem* } -save_edge !CLK

update_power_domain -name RAM_DEFAULT -primary_power_net VDD \
 -primary_ground_net VSS

end_macro_model ram 256x16A

#-------------------------------------------------------------------------------
# create power modes
#-------------------------------------------------------------------------------
create_power_mode -name PMdvfs1 -default -domain_conditions { PDpll@nom_0v81 \
 PDdefault@nom_0v81 PDtdsp@nom_0v81 PDram@nom_0v72 PDshutoff_io@nom_0v81 \
 PDram_virtual@nom_0v72 }
create_power_mode -name PMdvfs1_off -domain_conditions { PDpll@nom_0v81 \
 PDdefault@nom_0v81 PDshutoff_io@nom_0v81 PDram_virtual@nom_0v72 }
create_power_mode -name PMdvfs1_shutoffio_off -domain_conditions { \
 PDpll@nom_0v81 PDdefault@nom_0v81 PDram_virtual@nom_0v72 }
create_power_mode -name PMdvfs2 -domain_conditions { PDpll@nom_0v81 \
 PDdefault@nom_0v72 PDtdsp@nom_0v72 PDram@nom_0v72 PDshutoff_io@nom_0v72 \
 PDram_virtual@nom_0v72 }
create_power_mode -name PMdvfs2_off -domain_conditions { PDpll@nom_0v81 \
 PDdefault@nom_0v72 PDshutoff_io@nom_0v72 PDram_virtual@nom_0v72 }
create_power_mode -name PMdvfs2_shutoffio_off -domain_conditions { \
 PDpll@nom_0v81 PDdefault@nom_0v72 PDram_virtual@nom_0v72 }
create_power_mode -name PMscan -domain_conditions { PDpll@nom_0v81 \
 PDdefault@nom_0v81 PDtdsp@nom_0v81 PDram@nom_0v72 PDshutoff_io@nom_0v81 \
 PDram_virtual@nom_0v72 }

create_analysis_view -name AV_dvfs1_BC -mode PMdvfs1 -domain_corners { \
```

```
 PDpll@PMdvfs1_bc PDdefault@PMdvfs1_bc PDtdsp@PMdvfs1_bc PDram@PMdvfs2_bc \
 PDshutoff_io@PMdvfs1_bc }
create_analysis_view -name AV_dvfs1_WC -mode PMdvfs1 -domain_corners { \
 PDpll@PMdvfs1_wc PDdefault@PMdvfs1_wc PDtdsp@PMdvfs1_wc PDram@PMdvfs2_wc \
 PDshutoff_io@PMdvfs1_wc }
create_analysis_view -name AV_dvfs1_off_BC -mode PMdvfs1_off -domain_corners { \
 PDpll@PMdvfs1_bc PDdefault@PMdvfs1_bc PDshutoff_io@PMdvfs1_bc }
create_analysis_view -name AV_dvfs1_off_WC -mode PMdvfs1_off -domain_corners { \
 PDpll@PMdvfs1_wc PDdefault@PMdvfs1_wc PDshutoff_io@PMdvfs1_wc }
create_analysis_view -name AV_dvfs1_shutoffio_off_BC -mode \
 PMdvfs1_shutoffio_off -domain_corners { PDpll@PMdvfs1_bc \
 PDdefault@PMdvfs1_bc }
create_analysis_view -name AV_dvfs1_shutoffio_off_WC -mode \
 PMdvfs1_shutoffio_off -domain_corners { PDpll@PMdvfs1_wc \
 PDdefault@PMdvfs1_wc }
create_analysis_view -name AV_dvfs2_BC -mode PMdvfs2 -domain_corners { \
 PDpll@PMdvfs1_bc PDdefault@PMdvfs2_bc PDtdsp@PMdvfs2_bc PDram@PMdvfs2_bc \
 PDshutoff_io@PMdvfs2_bc }
create_analysis_view -name AV_dvfs2_WC -mode PMdvfs2 -domain_corners { \
 PDpll@PMdvfs1_wc PDdefault@PMdvfs2_wc PDtdsp@PMdvfs2_wc PDram@PMdvfs2_wc \
 PDshutoff_io@PMdvfs2_wc }
create_analysis_view -name AV_PMdvfs2_off_BC -mode PMdvfs2_off -domain_corners \
 { PDpll@PMdvfs1_bc PDdefault@PMdvfs2_bc PDshutoff_io@PMdvfs2_bc }
create_analysis_view -name AV_PMdvfs2_off_WC -mode PMdvfs2_off -domain_corners \
 { PDpll@PMdvfs1_wc PDdefault@PMdvfs2_wc PDshutoff_io@PMdvfs2_wc }
create_analysis_view -name AV_dvfs2_shutoffio_off_BC -mode \
 PMdvfs2_shutoffio_off -domain_corners { PDpll@PMdvfs1_bc \
 PDdefault@PMdvfs2_bc }
create_analysis_view -name AV_dvfs2_shutoffio_off_WC -mode \
 PMdvfs2_shutoffio_off -domain_corners { PDpll@PMdvfs1_wc \
 PDdefault@PMdvfs2_wc }
create_analysis_view -name AV_scan_BC -mode PMscan -domain_corners { \
 PDpll@PMdvfs1_bc PDdefault@PMdvfs1_bc PDtdsp@PMdvfs1_bc PDram@PMdvfs2_bc \
 PDshutoff_io@PMdvfs1_bc }
create_analysis_view -name AV_scan_WC -mode PMscan -domain_corners { \
 PDpll@PMdvfs1_wc PDdefault@PMdvfs1_wc PDtdsp@PMdvfs1_wc PDram@PMdvfs2_wc \
 PDshutoff_io@PMdvfs1_wc }

#-------------------------------------------------------------------------------
# create rules
#-------------------------------------------------------------------------------
```

```
create_power_switch_rule -name PDram_SW -domain PDram -external_power_net VDDL
create_power_switch_rule -name PDtdsp_SW -domain PDtdsp -external_power_net \
 VDD

create_isolation_rule -name ISORULE1 -isolation_condition { \
 !INST/PM_INST/isolation_enable } -from { PDtdsp } -to { PDdefault } \
 -isolation_target from -isolation_output high
create_isolation_rule -name ISORULE3 -isolation_condition { \
 !INST/PM_INST/isolation_enable } -from { PDram } -to { PDdefault } \
 -isolation_target from -isolation_output high
create_isolation_rule -name ISORULE4 -isolation_condition { \
 !INST/PM_INST/spi_ip_isolate } -from { PDshutoff_io } \
 -isolation_target from -isolation_output low

create_level_shifter_rule -name LSRULE_H2L3 -from { PDdefault } -to { PDram } \
 -exclude { INST/PM_INST/power_switch_enable }
create_level_shifter_rule -name LSRULE_H2L_PLL -from { PDpll }
create_level_shifter_rule -name LSRULE_L2H2 -from { PDram } -to { PDdefault }

create_state_retention_rule -name \
 INST/DSP_CORE_INST0/PDtdsp_retention_rule -instances { \
 INST/DSP_CORE_INST0 } -save_edge !INST/DSP_CORE_INST0/clk
create_state_retention_rule -name \
 INST/DSP_CORE_INST1/PDtdsp_retention_rule -instances { \
 INST/DSP_CORE_INST1 } -restore_edge \
 !INST/PM_INST/state_retention_restore -save_edge \
 INST/PM_INST/state_retention_save

#-------------------------------------------------------------------------------
# update domains/modes
#-------------------------------------------------------------------------------
update_nominal_condition -name nom_0v81 -library_set wc_0v81
update_nominal_condition -name nom_0v72 -library_set wc_0v72

update_power_domain -name PDdefault -primary_power_net VDD -primary_ground_net \
 VSS -equivalent_power_nets VDD_EQ
update_power_domain -name PDshutoff_io -primary_power_net VDD_IO \
 -primary_ground_net VSS
update_power_domain -name PDpll -primary_power_net Avdd -primary_ground_net \
 Avss
update_power_domain -name PDram_virtual -primary_power_net VDDL \
```

```
 -primary_ground_net VSS
update_power_domain -name PDram -primary_power_net VDDL_sw -primary_ground_net \
 VSS
update_power_domain -name PDtdsp -primary_power_net VDD_sw -primary_ground_net \
 VSS

update_power_mode -name PMdvfs1 -sdc_files ../RELEASE/mmmc/dvfs1.sdc
update_power_mode -name PMdvfs1_off -sdc_files ../RELEASE/mmmc/dvfs1.sdc
update_power_mode -name PMdvfs1_shutoffio_off -sdc_files \
 ../RELEASE/mmmc/dvfs1.sdc
update_power_mode -name PMdvfs2 -sdc_files ../RELEASE/mmmc/dvfs2.sdc
update_power_mode -name PMdvfs2_off -sdc_files ../RELEASE/mmmc/dvfs2.sdc
update_power_mode -name PMdvfs2_shutoffio_off -sdc_files \
 ../RELEASE/mmmc/dvfs2.sdc
update_power_mode -name PMscan -sdc_files ../RELEASE/mmmc/scan.sdc

#-------------------------------------------------------------------------------
# update rules
#-------------------------------------------------------------------------------
update_power_switch_rule -name PDram_SW -cells { HEADERHVT1 } -prefix \
 CDN_SW_RAM -peak_ir_drop_limit 0 -average_ir_drop_limit 0
update_power_switch_rule -name PDtdsp_SW -cells { HEADERHVT2 } -prefix \
 CDN_SW_TDSP -peak_ir_drop_limit 0 -average_ir_drop_limit 0

update_isolation_rules -names ISORULE1 -location to -prefix CPF_ISO_
update_isolation_rules -names ISORULE3 -location to -prefix CPF_ISO_
update_isolation_rules -names ISORULE4 -location to -prefix CPF_ISO_

update_level_shifter_rules -names LSRULE_H2L3 -location to -cells { LVLHLD* \
 } -prefix CPF_LS_
update_level_shifter_rules -names LSRULE_H2L_PLL -location to -prefix CPF_LS_
update_level_shifter_rules -names LSRULE_L2H2 -location to -prefix CPF_LS_

update_state_retention_rules -names \
 INST/DSP_CORE_INST0/PDtdsp_retention_rule -cell_type master_slave
update_state_retention_rules -names \
 INST/DSP_CORE_INST1/PDtdsp_retention_rule -cell_type ballon_latch
#-------------------------------------------------------------------------------
# end
#-------------------------------------------------------------------------------
end_design
```

Supported CPF 1.0 Commands

Note: The following commands are supported unless otherwise noted.

Command Name	Option	Notes
		N/A = not available in this release
create_analysis_view		
	-name	
	-mode	
	-domain_corners	
create_bias_net		
	-net	
	-driver	N/A
	-user_attributes	Accessible by getCPFUserAttr
	-peak_ir_drop_limit	N/A
	-average_ir_drop_limit	N/A
create_global_connection		
	-net	
	-pins	
	-domain	
	-instances	
create_ground_nets		
	-nets	
	-voltage	N/A
	-internal	N/A
	-user_attributes	Accessible by getCPFUserAttr
	-peak_ir_drop_limit	N/A

	-average_ir_drop_limit	N/A
create_isolation_rule		
	-name	
	-isolation_condition	
	-pins	
	-from	
	-to	
	-isolation_target	N/A
	-isolation_output	
	-exclude	
create_level_shifter_rule		
	-name	
	-pins	
	-from	
	-to	
	-exclude	
create_mode_transition		N/A
	-start_condition	
create_nominal_condition		
	-name	
	-voltage	
	-pmos_bias_voltage	N/A
	-nmos_bias_voltage	N/A
create_operating_corner		
	-name	
	-voltage	
	-process	
	-temperature	
	-library_set	

create_power_domain		
	-name	
	-default	
	-instances	
	-boundary_ports	
	-shutoff_condition	
	-default_restore_edge	
	-default_save_edge	
	-power_up_states	N/A
create_power_mode		
	-name	
	-domain_conditions	
	-default	
create_power_nets		
	-nets	
	-voltage	
	-external_shutoff_condition	
	-internal	
	-user_attributes	Accessible by getCPFUserAttr
	-peak_ir_drop_limit	N/A
	-average_ir_drop_limit	N/A
create_power_switch_rule		
	-name	
	-domain	
	-external_power_net	
	-external_ground_net	
create_state_retention_rule		
	-name	

	-domain	
	-instances	
	-restore_edge	
	-save_edge	
define_always_on_cell	-cells -power_switchable -power -ground_switchable -ground	
define_isolation_cell		
	-cells	
	-library_set	
	-always_on_pin	
	-power_switchable	
	-ground_switchable	
	-power	
	-ground	
	-valid_location	
	-non_dedicated	N/A
	-enable	
define_level_shifter_cell		
	-cells	
	-library_set	
	-always_on_pin	N/A
	-input_voltage_range	
	-output_volage_range	
	-direction	
	-output_voltage_input_pin	N/A

	-input_power_pin	
	-output_power_pin	
	-ground	
	-valid_location	
define_open_source_input_pin		
	-cells	
	-pin	
	-library_set	
define_power_clamp_cell		N/A
	-cells	
	-data	
	-ground	
	library_set	
	-power	
define_power_switch_cell		
	-cells	
	-library_set	
	-stage_1_enable	
	-stage_1_output	
	-stage_2_enable	
	-stage_2_output	
	-type	
	-power_switchable	
	-power	
	-ground	
	-ground_switchable	
	-on_resistance	Accessible by ::CPF::getCpfPsoCell

	-stage_1_saturation_current	Accessible by ::CPF::getCpfPsoCell
	-stage_2_saturation_current	Accessible by ::CPF::getCpfPsoCell
	-leakage_current	Accessible by ::CPF::getCpfPsoCell
define_state_retention_cell		
	-cells	
	-library_set	
	-always_on_pin	N/A
	-clock_pin	N/A
	-restore_function	
	-restore_check	N/A
	-save_function	
	-save_check	N/A
	-power_switchable	
	-ground_switchable	
	-power	
	-ground	
define_library_set		
	-name	
	-libraries	
end_design		
identify_always_on_driver		N/A
identify_power_logic		
	-type	
	-instances	
set_array_naming_style		
set_cpf_version		
set_design		

	-ports	
	module	
set_hierarchy_separator		
set_instance		
	-port_mapping	
	-merge_default_domains	
	hier_instance	
set_power_target		N/A
set_power_unit		N/A
set_register_naming_style		
set_switching_activity		
	-all	
	-pins	
	-instances	
	-hierarchical	
	-probability	
	-toggle_rate	
	-clock_pins	N/A
	-toggle_percentage	N/A
	-mode	N/A
set_time_unit		
update_isolation_rules		
	-names	`
	-location	
	-cells	
	-library_set	
	-prefix	
	-combine_level_shifting	N/A
	-open_source_pins_only	

-update_level_shifter_rules		
	-names	
	-location	
	-cells	
	-library_set	
	-prefix	
update_nominal_condition		
	-name	
	-library_set	
update_power_domain		
	-name	
	-internal_power_net	
	-internal_ground_net	
	-min_power_up_time	N/A
	-max_power_up_time	N/A
	-pmos_bias_net	N/A
	-nmos_bias_net	N/A
	-user_attributes	Accessible by ::CPF::getCpfUserAttr
	-rail_mapping	N/A
	-library_set	
update_power_mode		
	-name	
	-activity_file	N/A
	-activity_file_weight	N/A
	-sdc_files	
	-peak_ir_drop_limit	N/A
	-average_ir_dropt_limit	N/A
	-leakage_power_limit	N/A

	-dynamic_power_limit	N/A
update_power_switch_rule		
	-name	
	-enable_condition_1	
	-enable_condition_2	
	-acknowledge_receiver	
	-cells	
	-library_set	
	-prefix	
	-peak_ir_drop_limit	Accessible by ::CPF::getCpfUserAttr
	-average_ir_drop_limit	Accessible by ::CPF::getCpfUserAttr
update_state_retention_rules		
	-name	
	-cell_type	
	-cell	
	-library_set	

Supported CPF 1.0e Commands

Note: The following commands and options are supported unless otherwise noted.

Command Name	Option	Notes
create_analysis_view		
	-name	
	-mode	
	-domain_corners	
	-group_views	
	-user_attributes	
create_assertion_control		
	-name	Unsupported
	-assertions	Unsupported
	-domains	Unsupported
	-shutoff_condition	Unsupported
	-type	Unsupported
create_bias_net		
	-net	
	-driver	
	-user_attributes	Supported: query getCPFUserAttributes
	-peak_ir_drop_limit	
	-average_ir_drop_limit	
create_global_connection		
	-net	
	-pins	
	-domain	
	-instances	
create_power_domain		

	-name	
	-instances	
	-boundary_ports	
	-default	
	-shutoff_condition	
	-external_controlled_shutoff	
	-default_isolation_condition	
	-default_restore_edge	
	-default_save_edge	
	-default_restore_level	Supported
	-default_save_level	Supported
	-power_up_states	Unsupported
	-active_state_condition	Unsupported
	-secondary_domains	
create_ground_nets		
	-nets	
	-voltage	
	-external_shutoff_condition	
	-user_attributes	Supported: query getCPFUserAttributes
	-peak_ir_drop_limit	
	-average_ir_drop_limit	
create_isolation_rule		
	-name	
	-isolation_condition	
	-no_condition	Unsupported
	-pins	
	-from	
	-to	

	-exclude	
	-isolation_target	
	-isolation_output	
	-secondary_domain	
create_level_shifter_rule		
	-name	
	-pins	
	-from	
	-to	
	-exclude	
create_mode_transition		
	-name	
	-from	
	-to	
	-start_condition	
	-end_condition	
	-cycles	
	-clock_pin	
	-latency	
create_nominal_condition		
	-name	
	-voltage	
	-ground_voltage	
	-state	Unsupported
	-pmos_bias_voltage	Unsupported
	-nmos_bias_voltage	Unsupported
create_operating_corner		
	-name	
	-voltage	

	-ground_voltage	Unsupported
	-pmos_bias_voltage	Unsupported
	-nmos_bias_voltage	Unsupported
	-process	
	-temperature	
	-library_set	
create_power_mode		
	-name	
	-default	
	-group_modes	
	-domain_conditions	
create_power_nets		
	-nets	
	-voltage	
	-external_shutoff_condition	
	-user_attributes	Supported: query getCPFUserAttributes
	-peak_ir_drop_limit	
	-average_ir_drop_limit	
create_power_switch_rule		
	-name	
	-domain	
	-external_power_net	
	-external_ground_net	
create_state_retention_rule		
	-name	
	-domain	
	-instances	
	-exclude	

	-restore_edge	
	-save_edge	
	-restore_precondition	
	-save_precondition	
	-target_type	
	-secondary_domain	
define_always_on_cell		
	-cells	
	-library_set	
	-power_switchable	
	-ground_switchable	
	-power	
	-ground	
define_isolation_cell		
	-cells	
	-library_set	
	-always_on_pins	
	-power_switchable	
	-ground_switchable	
	-power	
	-ground	
	-valid_location	
	-enable	
	-no_enable	Unsupported
	-non_dedicated	
define_level_shifter_cell		
	-cells	
	-library_set	
	-always_on_pins	

	-input_voltage_range	
	-output_voltage_range	
	-ground_output_voltage_range	Unsupported
	-groung_output_voltage_range	
	-direction	
	-input_power_pin	
	-output_power_pin	
	-input_ground_pin	Unsupported
	-output_ground_pin	Unsupported
	-ground	
	-power	Unsupported
	-enable	
	-valid_location	
define_library_set		
	-name	
	-libraries	
	-user_attributes	cdb: specify cdb libraries for the library set aocv: specify aocv libraries for the library set
define_power_clamp_cell		
	-location	Unsupported
	-within_hierarchy	Unsupported
	-cells	Unsupported
	-prefix	Unsupported
define_power_switch_cell		
	-cells	

	-library_set	
	-stage_1_enable	
	-stage_1_output	
	-stage_2_enable	
	-stage_2_output	
	-type	
	-enable_pin_bias	Unsupported
	-gate_bias_pin	Unsupported
	-power_switchable	
	-power	
	-ground_switchable	
	-ground	
	-on_resistance	Supported (for use with addPowerSwitch)
	-stage_1_saturation_current	Supported (for use with addPowerSwitch)
	-stage_2_saturation_current	Supported (for use with addPowerSwitch)
	-leakage_current	Supported (for use with addPowerSwitch)
define_state_retention_cell		
	-cells	
	-library_set	
	-cell_type	
	-always_on_pins	
	-clock_pin	
	-restore_function	
	-save_function	
	-restore_check	
	-save_check	

	-always_on_components	Unsupported
	-power_switchable	
	-ground_switchable	
	-power	
	-ground	
end_macro_model		
end_power_mode_control_group		
get_parameter		
include		
identify_power_logic		
	-type	Only "isolation" is supported for the -type
	-instances	Supported
	-module	Supported
set_array_naming_style		
set_cpf_version		
set_hierarchy_separator		
set_design		
	-ports	
	-honor_boundary_port_domain	
	-parameters	
set_equivalent_control_pins		
	-master	Unsupported
	-pins	Unsupported
	-domain	Unsupported
	-rules	Unsupported
set_floating_ports		Unsupported

set_input_voltage_tolerance		
	-ports	Unsupported
	-bias	Unsupported
set_instance		
	-design	
	-model	
	-port_mapping	
	-domain_mapping	
	-parameter_mapping	
set_macro_model		
set_power_mode_control_group		
	-name	
	-domains	
	-groups	Unsupported
set_power_target		
	-leakage	Unsupported
	-dynamic	Unsupported
set_power_unit		
set_register_naming_style		
set_switching_activity		
	-all	Supported
	-pins	Supported
	-instances	Supported
	-hierarchical	Supported
	-probability	Supported
	-toggle_rate	Supported
	-clock_pins	Unsupported
	-toggle_percentage	Unsupported

	-mode	Supported
set_time_unit		
set_wire_feedthrough_ports		
update_isolation_rules		
	-names	
	-location	
	-within_hierarchy	
	-cells	
	-prefix	
	-open_source_pins_only	Supported
update_level_shifter_rules		
	-names	
	-location	
	-within_hierarchy	
	-cells	
	-prefix	
update_nominal_condition		
	-name	
	-library_set	
update_power_domain		
	-name	
	-primary_power_net	
	-primary_ground_net	
	-pmos_bias_net	Unsupported
	-nmos_bias_net	Unsupported
	-user_attributes	Supported: query getCPFUserAttributes
	-transition_slope	Unsupported
	-transition_latency	Unsupported

	-transition_cycles	Unsupported
update_power_mode		
	-name	
	-activity_file	Unsupported
	-activity_file_weight	Unsupported
	-sdc_files	
	-peak_ir_drop_limit	
	-average_ir_drop_limit	
	-leakage_power_limit	Unsupported
	-dynamic_power_limit	Unsupported
update_power_switch_rule		
	-name	
	-enable_condition_1	
	-enable_condition_2	
	-acknowledge_receiver	
	-cells	
	-gate_bias_net	Unsupported
	-prefix	
	-peak_ir_drop_limit	
	-average_ir_drop_limit	
update_state_retention_rules		
	-names	
	-cell_type	
	-cells	
	-set_rest_control	Unsupported

Supported CPF 1.1 Commands

Note: The following commands and options are supported unless otherwise noted.

Command Name	Option	Notes
asset_illegal_domain_configurations		
	-domain_conditions	
	-group_modes	
	-name	
create_analysis_view		
	-domain_corners	
	-group_views	
	-mode	
	-name	
	-user_attributes	
create_assertion_control		
	-assertions	Unsupported
	-domains	Unsupported
	-exclude	
	-name	Unsupported
	-shutoff_condition	Unsupported
	-type	Unsupported
create_bias_net		
	-average_ir_drop_limit	
	-driver	
	-net	
	-peak_ir_drop_limit	
	-user_attributes	Supported: query getCPFUserAttributes

create_global_connection		
	-domain	
	-instances	
	-net	
	-pins	
create_ground_nets		
	-average_ir_drop_limit	
	-external_shutoff_condition	
	-nets	
	-peak_ir_drop_limit	
	-user_attributes	Supported: query getCPFUserAttributes
	-voltage	
create_isolation_rule		
	-exclude	
	-from	
	-isolation_condition	
	-isolation_output	
	-isolation_target	
	-name	
	-no_condition	
	-pins	
	-secondary_domain	
	-to	
	-force	
create_level_shifter_rule		
	-exclude	
	-from	
	-name	

	-pins	
	-to	
	-force	
create_mode_transition		
	-clock_pin	
	-cycles	
	-end_condition	
	-from	
	-latency	
	-name	
	-to	
	-start_condition	
create_nominal_condition		
	-ground_voltage	
	-name	
	-nmos_bias_voltage	Unsupported
	-pmos_bias_voltage	Unsupported
	-state	Unsupported
	-voltage	
create_operating_corner		
	-ground_voltage	Unsupported
	-library_set	
	-name	
	-nmos_bias_voltage	Unsupported
	-pmos_bias_voltage	Unsupported
	-process	
	-temperature	
	-voltage	
create_power_domain		

	-active_state_condition	Unsupported
	-base_domains	
	-boundary_ports	
	-default	
	-default_isolation_condition	
	-default_restore_edge	
	-default_restore_level	Supported
	-default_save_edge	
	-default_save_level	Supported
	-external_controlled_shutoff	
	-instances	
	-name	
	-power_up_states	Unsupported
	-shutoff_condition	
create_power_mode		
	-default	
	-domain_conditions	
	-group_modes	
	-name	
create_power_nets		
	-average_ir_drop_limit	
	-external_shutoff_condition	
	-nets	
	-peak_ir_drop_limit	
	-user_attributes	Supported: query getCPFUserAttributes
	-voltage	
create_power_switch_rule		
	-domain	

	-external_ground_net	
	-external_power_net	
	-name	
create_state_retention_rule		
	-domain	
	-exclude	
	-instances	
	-name	
	-restore_edge	
	-restore_precondition	
	-save_edge	
	-save_precondition	
	-secondary_domain	
	-target_type	
define_always_on_cell		
	-cells	
	-ground	
	-ground_switchable	
	-library_set	
	-power	
	-power_switchable	
define_isolation_cell		
	-always_on_pins	
	-cells	
	-enable	
	-ground	
	-ground_switchable	
	-library_set	
	-no_enable	

	-non_dedicated	
	-power	
	-power_switchable	
	-valid_location	
define_level_shifter_cell		
	-always_on_pins	
	-cells	
	-direction	
	-enable	
	-ground	
	-ground_input_voltage_range	
	-ground_output_voltage_range	
	-input_ground_pin	
	-input_power_pin	
	-input_voltage_range	
	-library_set	
	-output_ground_pin	
	-output_power_pin	
	-output_voltage_range	
	-power	
	-valid_location	
define_library_set		
	-libraries	
	-name	

	-user_attributes	cdb: specify cdb libraries for the library set aocv: specify aocv libraries for the library set
define_power_clamp_cell		
	-cells	Unsupported
	-data	Unsupported
	-power pin	Unsupported
	-ground	Unsupported
	-library_set	
define_power_switch_cell		
	-cells	
	-enable_pin_bias	Unsupported
	-gate_bias_pin	Unsupported
	-ground	
	-ground_switchable	
	-leakage_current	Supported (for use with addPowerSwitch)
	-library_set	
	-power	
	-power_switchable	
	-stage_1_on_resistance	
	-stage_2_on_resistance	
	-stage_1_enable	
	-stage_1_output	
	-stage_1_saturation_current	Supported (for use with addPowerSwitch)
	-stage_2_enable	
	-stage_2_output	

	-stage_2_saturation_current	Supported (for use with addPowerSwitch)
	-type	
define_state_retention_cell		
	-always_on_components	Unsupported
	-always_on_pins	
	-cell_type	
	-cells	
	-clock_pin	
	-ground	
	-ground_switchable	
	-library_set	
	-power	
	-power_switchable	
	-restore_check	
	-restore_function	
	-save_check	
	-save_function	
end_design		
end_macro_model		
end_power_mode_control_group		
get_parameter		
include		
identify_power_logic		
	-instances	Supported
	-module	Supported
	-type	Only "isolation" is supported for the -type
set_array_naming_style		

set_cpf_version		
set_hierarchy_separator		
set_design		
	module	
	-ports	
	-honor_boundary_port_domain	
	-parameters	
set_equivalent_control_pins		
	-domain	Unsupported
	-master	Unsupported
	-pins	Unsupported
	-rules	
set_floating_ports		Unsupported
set_input_voltage_tolerance		
	-bias	Unsupported
	-ports	Unsupported
set_instance		
	-design	
	-domain_mapping	
	-model	
	-parameter_mapping	
	-port_mapping	
set_macro_model		
set_power_mode_control_group		
	-domains	
	-groups	Unsupported
	-name	

set_power_target		
	-dynamic	Unsupported
	-leakage	Unsupported
set_power_unit		
set_register_naming_style		
set_switching_activity		
	-all	Supported
	-clock_pins	Unsupported
	-hierarchical	Supported
	-instances	Supported
	-mode	Supported
	-pins	Supported
	-probability	Supported
	-toggle_percentage	Unsupported
	-toggle_rate	Supported
set_time_unit		
set_wire_feedthrough_ports		
update_isolation_rules		
	-cells	
	-location	
	-names	
	-open_source_pins_only	Supported
	-prefix	
	-within_hierarchy	
update_level_shifter_rules		
	-cells	
	-location	
	-names	
	-prefix	

	-within_hierarchy	
update_power_domain		
	-equivalent_ground_nets	
	-equivalent_power_nets	
	-name	
	-nmos_bias_net	Unsupported
	-pmos_bias_net	Unsupported
	-primary_ground_net	
	-primary_power_net	
	-transition_cycles	Unsupported
	-transition_latency	Unsupported
	-transition_slope	Unsupported
	-user_attributes {{disable_secondary_domains {list of domains}}}	Supported: query getCPFUserAttributes
update_power_mode		
	-activity_file	Unsupported
	-activity_file_weight	Unsupported
	-average_ir_drop_limit	
	-dynamic_power_limit	Unsupported
	-leakage_power_limit	Unsupported
	-name	
	-peak_ir_drop_limit	
	-sdc_files	
update_power_switch_rule		
	-acknowledge_reciever_1	
	-acknowledge_reciever_2	
	-average_ir_drop_limit	
	-cells	

	-enable_condition_1	
	-enable_condition_2	
	-gate_bias_net	Unsupported
	-name	
	-peak_ir_drop_limit	
	-prefix	
update_state_retention_rules		
	-cell_type	
	-cells	
	-names	
	-set_rest_control	Unsupported

12

Glossary

The below glossary defines the terms and concepts that you should understand to use Tempus effectively.

A

arc

Is the timing relation between two points. This could either be a cell arc or net arc.

arrival time

Is the time elapsed from the source of launch clock till the arrival of the data at the end point.

asynchronous clocks

Two clocks that do not have a fixed phase relationship between their time period.

attribute

A value related to an object.

attacker

A net that causes undesirable cross-coupling effects on a victim net.

B

bindkey

A keyboard key or mouse button linked (bound) to a Cadence command or function name.

bounding box

A rectangular area, identified by a lower-left point and an upper-right point. Typically used to identify the area of an object (a path or an instance) or a collection of objects (the selected set).

branch

A path between two nodes. Each branch has two associated quantities, a potential and a flow, with a reference direction for each.

C

capacitance

The function of a design to store energy in the form of an electrostatic field.

cell

A component of a design; a collection of different aspects (representations) of component implementations, such as its schematic, layout, or symbol representations. A design object consisting of a set of views that can be stored and referenced independently. A cell can include other cells, forming a hierarchical design. A cell is an individual building block of a chip or system. In the database, a cell contains all the cellviews of that cell.

An inverter and a buffer are examples of a small cell. A decoder register, arithmetic logic unit (ALU), memories, complete chips, and printed circuit boards are examples of large cells.

cell delay

Represents the time difference of a signal when transmitted from an input to an output.

cellview

A specific representation (view) of a cell. A particular representation of a particular component, such as the physical layout of a flip-flop or the schematic symbol of a NAND gate. A database object containing all the information unique to a particular representation of a particular component. Cellviews are classified by their view type. Each cellview has a view name and can have one or more versions.

clock

Timing signals generated periodically by a device in the digital domain.

clock gating

One of the basic power-saving techniques where the clock is gated in a logic block only when a next state logic is required to be captured.

clock latency

Is the amount of time taken from the clock origin point to the clock pin of the flop (sink pin).

clock path

Is the timing path from the clock source to the sink(clock) pin of the flop.

clock skew

Is the difference between the clock arrival time at two different nodes.

clone

A layout structure that has been replicated from a specified source structure already implemented in the layout view. Each clone inherits its physical characteristics from the layout structure on which it is based, and the connectivity information from the associated schematic structure.

constraint

The restrictions set on the objects in a layout or schematic to meet the routing or placement requirements in a design.

corner

A corner is defined as a set of libraries (characterized for process, voltage, and temperature variations) and net delay, which together account for delay variations. Corners are not dependent on functional settings; they are meant to capture variations in the manufacturing process, along with expected variations in the voltage and temperature of the environment in which the chip will operate.

coupled capacitance

Is the parasitic capacitance between two wires.

coupling

Is the transfer of electrical energy from one circuit segment to another. In the context of Signal integrity, coupling refers to the capacitive coupling that is often unintended because of the capacitance between two wires that are adjacent to each other. Usually, one signal capacitively couple with another signal and causes noise.

critical path

Is defined as the path that has the worst amount of slack relative to the desired arrival time.

crosstalk

Unwanted interaction between signals on different electrical nets in proximity because of coupling capacitance between them.

cross coupling

Is the effect introduced on neighboring nets because of the coupling capacitance between them.

current design

Is the top-level design in a design hierarchy.

D

data path

Is the timing path between output of the launch flop to the input of capture flop.

default value

The value used by the software unless you specify otherwise. The default is frequently the initial state.

delay

Is the time that it takes a signal to propagate from a given point to another given point.

delay calculation

Is the term used in an integrated circuit design for the sum of the delay of logic gates and delay of wires attached to it; basically the delay of any path from a start point to end point. This also refers to the process by which cell delay or net delay is calculated.

design

The collection of all data in a cellview, including all nested data, starting at the top and expanding the hierarchy.

design library

A library that contains data for the current design. It is usually in the designer's own directory or in the design group's directory.

design verification

Process of verifying the functional and performance requirements of a design.

device

A design element that has both a symbol view and a layout view, with corresponding pins.

distributed processing

Runs a program on a distributed computing system of multiple independent machines that communicate through a network.

distortion

Is the changing of any of the attributes of a signal, such as amplitude, pulse width, rise time, and so on because of the medium, such as a connector or a cable through which the signal flows.

DMMMC

Abbreviation for Distributed Multi-mode multi-corner timing analysis. It is a method for processing timing analysis views where the software automatically distributes the runs to multiple machines for analysis.

driver

A primitive device or behavioral construct that affects the digital value of a signal.

DSTA

Abbreviation for Distributed Static Timing Analysis. It is the Master/Client architecture that fully leverages multiple threads on both the Master/Client processes allowing maximum scalability, even with the growing size and complexity of the designs.

dynamic analysis

Dynamic analysis is a method of analyzing a circuit to obtain operating currents and voltages. It is a time-based analysis, where the circuit netlist is analyzed over a specified period, such as a clock cycle.

E

endpoint

Point in a design where the timing path ends. This is either a primary output port or the launching pin of a sequential cell (CK pin of a flop or EN pin of a latch).

environment

The hardware and software setup and conditions within which the system operates.

extrapolation

Using two or more points on a curve to determine an intermediate curve point that provides a more accurate value than the provided available points. For example, calculating delay for cases where the inputs are lying outside of look up table.

F

false path

Is a timing path not required to meet its timing constraints for the design to function properly.

fanin

Is the number of pins or ports in the cone of logic connected to the input of a logic gate.

fanout

Is the number of gate inputs that can connect to the gate output. For example, with a fanout of 4, one TTL gate can drive 4 others.

floorplanning

Placing ports, cells, memories, macros, and so on to either create a rough plan to estimate whether a design meets the timing and routability criteria, or create a starting point for design implementation.

G

GBA

Abbreviation for Graph Based Analysis. It calculates the worst-case delays of all cells considering the worst-case slew for all the inputs of a gate when the worst slew propagation is ON.

GUI

Abbreviation for Graphical User Interface.

glitch

Unintended noise pulse induced on the neighboring nets due to capacitive (or inductive) coupling.

H

hierarchical design

A hierarchical design is a way of structuring the logic at more than one level of abstraction; that is, a symbol at one level of abstraction represents the internal contents of a cell at a higher level of hierarchy.

hierarchy

Nested design levels, such as instances within a cell. By default, you open the top level in the hierarchy when you open a cellview.

hierarchy representation

Specification of how the physical hierarchy is generated from your logical design, including which logical components are to be generated or ignored in the physical implementation and which physical views are used to implement logical components.

I

IPD

Inter Power Domain

IPD Logic

Design logic having PD transitions in fanin/fanout cone and hence identified for IPD analysis is termed as IPD logic.

IR drop

IR drop is a reduction in the voltage that occurs on the VDD network of an integrated circuit because of the current flowing through the resistance of the VDD network itself (V = I x R).

impedance

The opposition of an electronic component to the flow of current.

interconnect

Refers to the physical wires and vias between the gates and cells in an integrated circuit.

instance

A database object that creates a level of hierarchy. An instance creates an occurrence of the referenced cellview.

L

latch

Latch is a level-sensitive register with three ports - input, output, and enable. When enable is active, input drives output, which means that the latch is open or transparent. When enable is inactive, the output is kept at the existing value.

latency

Is the time taken by a clock signal to be transmitted from the input source to output.

layer

A layer refers to a mask material. Various database shape objects are created referencing a layer and a purpose.

library

A library is a logical collection of design data implemented as a physical collection of the directories and files that can reside anywhere in the file system. A library can be shared by all users or controlled by a single person.

M

man page

A description of a command, variable, or message code displayed using the man command in Tempus. For example, entering man report_timing at the Tempus prompt displays the man page showing the description of the command and its related parameters.

miller capacitance

Is the coupling capacitance between nets associated with the coupling between the gate and the source/drain of a transistor.

mode

A mode is defined by a set of clocks, supply voltages, timing constraints, and libraries. It can also have annotation data, such as SDF or parasitics files. Many chips have multiple modes such as functional modes, test mode, sleep mode, and so on.

modeling

Refers to the creation and simulation of electronic circuits on a machine.

MSV

Abbreviation for Multi Supply Voltage. MSV is used to indicate instances/cells that should be identified as IPD logic.

multithreading

Ability of a processing unit to run multiple processes or threads concurrently.

MMMC

Abbreviation for Multi Mode Multi Corner. MMMC is the ability to configure the software to support multiple combinations of modes and corners, and to evaluate them concurrently.

MSV Designs

Abbreviation for Multi Supply Voltage Design where the design is using more than one voltage sources.

N

net

A logical signal connection between a set of pins on different instances. After routing, a net consists of routed wires on the routing layers.

netlist

Is a collection of related lists of the terminals (pins) in a circuit along with their interconnections.

Non IPD Logic

Design logic having no PD transitions in its fanin/fanout cone and hence not left out from IPD analysis is termed as non-IPD logic.

O

optimization

Ability to achieve the most efficient design of a product. There are various types of optimizations, which are performed by different tools in the design flows for chips, boards, and so on.

P

path

A path object is a shape-based object represented by a point array with a specified width, style, and layer-purpose pair.

path search

The list of directories the software searches for files, libraries, and commands.

parasitic capacitance

Parasitic capacitance is any capacitance causing wanted or unwanted effects between proximity conductors or in devices.

PBA

Abbreviation for Path Based Analysis. Here, a timing path that is found by graph-based analysis (GBA), is re-timed for increased accuracy and for removing pessimism due to slew merging effects, AOCV stage counts, and so on.

peak current

Peak current is the maximum value of a current waveform.

phase timing

Is the timing relationship between the edges of a clock and other clock signals or data signals.

placement and routing

Process of placing and routing the connections of circuit.

pin

A physical implementation of a terminal. You can place pins on any layer. Pins have a figure defining the physical implementation and a terminal defining the type, such as the input, output, and so on. Pins also have names, access directions and placement status.

ports

Endpoints that let the model and the application to interact during simulation.

propagation delay

Is the speed with which a signal travels on a conductor path.

properties

Properties are a name-value pair defining an attribute or characteristic. Properties can be created on any database object. A property can be edited and deleted. Certain properties are mandatory for certain applications. Properties are defined and managed by the application.

pulse

Pulse is a standard representation of a waveform that is typically used as a voltage source in circuit simulation. The waveform displays a recurring pattern.

R

receiver

A primitive device or behavioral construct that samples the digital value of a signal.

required time

Is the latest time at which a signal can arrive without making the clock cycle longer than desired.

routing

Physically connecting objects in a design according to the design rules set in the reference library or technology file.

RCDB

Is Binary RC Database. It is a random-access format that provides parasitics information for timing analysis and reduces memory footprint. The RC database has better runtime and memory performance.

RSH

Abbreviation for Remote Shell. It is the method of launching jobs on remote machines.

S

scaling

Scaling is the process of globally reducing or enlarging a design. Scaling is typically used to match a design to a specific process.

session window

The current working window. A session window can contain multiple tabs, each potentially running a different application. You can have more than one session window open at a time, each with a workspace of its own.

skew

Is the difference in the timing that arises when a clock transition occurs.

slack

Is associated with each connection and is the difference between the required time and the arrival time. A positive slack s at a node implies that the arrival time at that node may be increased by s without affecting the overall delay of the circuit. Whereas, a negative slack

implies that a path is too slow, and the path must be sped up (or the reference signal delayed) if the whole circuit is to work at the desired speed.

slew

Is the time taken by a signal to change from rise to fall or from fall to rise . This is also known as the transition time.

signal integrity

Refers to a signal's freedom from noise and the unpredictable delay caused by increasing the coupling capacitance between neighboring lines.

standalone

A mode in which a machine can run only locally installed and licensed applications.

startpoint

Is a point in a design where the timing path begins. This is either a primary input port or the capturing pin of a sequential cell (D pin of a flop, a latch or synchronous reset/set pins of a reset/set flop).

static analysis

Method of analyzing a circuit without using time-based simulation.

superthreading

Is a type of multithreading that enables different threads to be executed by a single processor without truly executing them at the same time.

SDF file

Abbreviation for Standard Database Format files. These are used to store values and the data of fixed lengths in a defined way and can easily be transferred across databases and programs.

SMSC

Abbreviation for Single Mode Single Corner. This is the ability to configure the software to support single mode and single corner at one time.

STA

Abbreviation for Static Timing Analysis. It is a simulation method of computing the expected timing of a digital circuit without requiring a simulation of the full circuit.

SSH

Abbreviation for Secure Shell. It is the method of securely launching jobs on remote machines.

SPEF

SPEF is an acronym for Standard Parasitic Exchange Format, which is a form of SPF that allows true coupling capacitance to be carried between nodes and is passed to downstream analysis tools.

synchronous

When specific transitions occur at the same time between two clocks or signals, thereby maintaining a timing relationship with each other.

T

task

Work objective. For example, a hierarchical task comprising a complex objective, such as optimizing device sizing, or a simple task such as adding a new pin to a schematic.

time borrowing

Is a technique that allows logic to automatically borrow time from the next cycle, thereby reducing the time available for the data to arrive for the following cycle or permitting the logic to use the slack from the previous cycle in the current cycle.

timing arc

Is the path set when a signal travels from one input to another output.

tool

Is a shorthand term for an EDA product in the electronic design automation industry.

top-down design

An approach to hierarchical design that uses estimates and floorplanning to start at the top level of a design.

transistor

Is a semiconductor device, which is used to amplify a signal, or to open and close a circuit.

transient analysis

Transient analysis is a time-based analysis of a system.

tapeout

Process of transferring the final chip layout design files to a mask-making vendor.

V

via

A via is a connection point between two adjacent metal routing layers defined by a pad in each layer.

victim

A victim net receives undesirable cross-coupling effects from any neighboring attacker net.

VDD

VDD is the common name of the power node, which is the voltage supply. It is the same as VCC.

VSS

VSS is the common name of the ground node, which is a conducting body used as the electrical current return. It is the same as GND.

W

wire

Wire refers to the routed physical metal conducting an electrical signal in contrast to a net, which refers to the logical electrical signal that is being transferred.